Guide Statistics · Evaluation · Training

Frontier Model Field Guide

How large language models are measured and how they are made, explained with pictures you can poke at. Every topic opens in plain words with an everyday example, then switches to what a leader needs to decide, then goes under the hood to the math and code.

31	3	55+	⌘K
chapters in 4 parts	depths per topic	interactive widgets	search anything

Contents

Start here

1 How to use this guide

Part I: Statistical foundations

2 Distributions: the basics
3 Sampling, error bars, p values
4 Reading a shape
5 The distribution zoo
6 Long tails and outliers
7 From shape to method

Part II: Evaluating models

8 Statistics for eval sets
9 Intervals, bootstrap, clustering
10 Classification metrics, precision to AUC
11 LLM as judge
12 The offline to online gap
13 Contamination and benchmark integrity
14 Building the eval harness
15 Diagnosing a regression
16 Which metric or test, when

Part III: Training models

17 The training pipeline
18 Pretraining
19 Optimisation and infrastructure
20 Scaling laws
21 Supervised fine-tuning
22 LoRA and parameter-efficient tuning
23 Distillation, synthetic data, and merging
24 Preferences and reward models
25 RLHF and PPO
26 DPO and offline preference methods
27 GRPO and RL with verifiable rewards
28 Inference, decoding, and test-time compute
29 Safety and alignment training

Part IV: Leading the work

30 Connecting training and evaluation
31 Glossary

Introduction

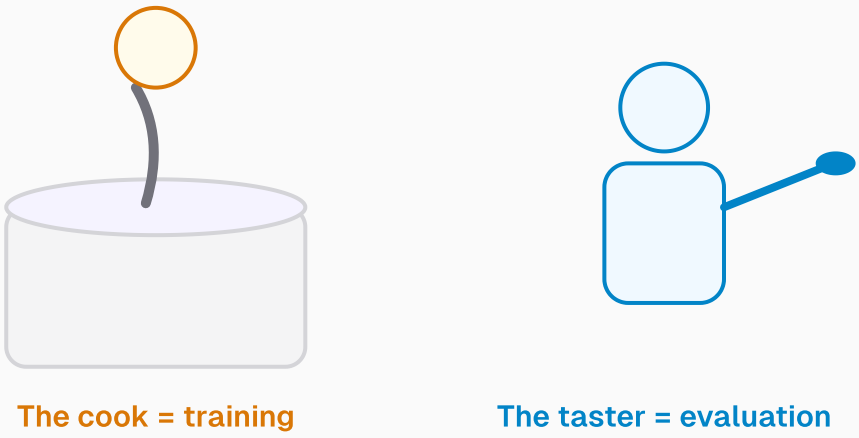
How to use this guide

This guide follows one idea from end to end: you cannot improve what you cannot measure, and you cannot trust a measurement you do not understand. Part I gives you the statistics. Part II turns that into model evaluation. Part III explains how models are trained, always connecting back to how you would measure whether the training worked. Part IV is about leading that work.

Diagram

The whole guide in one kitchen

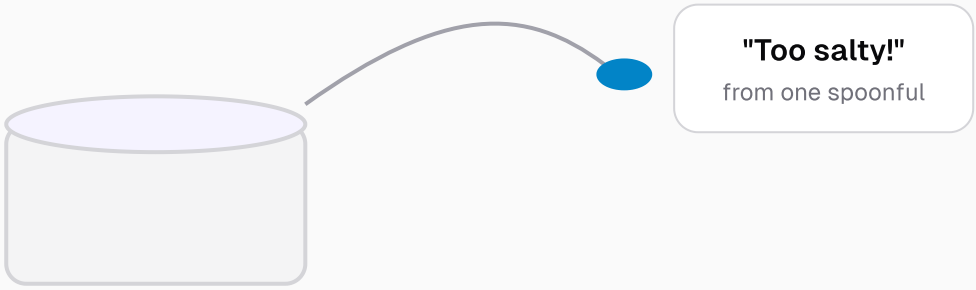
Five diagrams. Use Next or the step numbers.



The cook = training

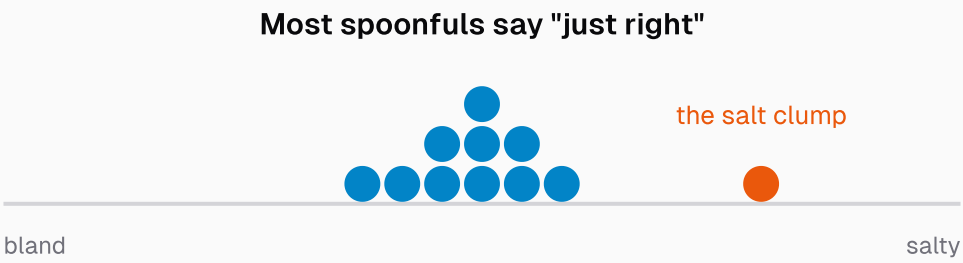
The taster = evaluation

Making a model is cooking: you choose ingredients (data), a recipe (the training method), and how long to cook (compute). Checking the model is tasting. Part III is the cooking. Part II is the tasting.

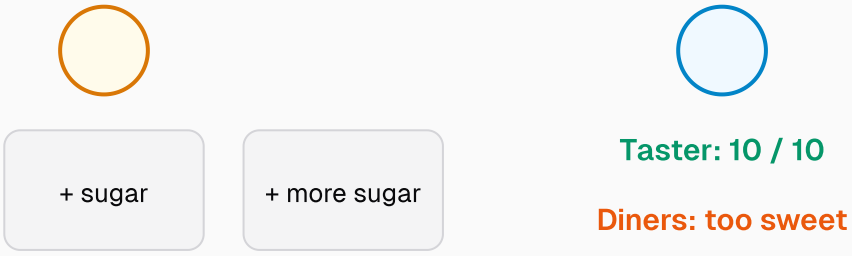


One spoonful might have landed on a salt clump.

One taste is one sample. It might be unlucky. Part I is about how much a handful of samples can and cannot tell you.

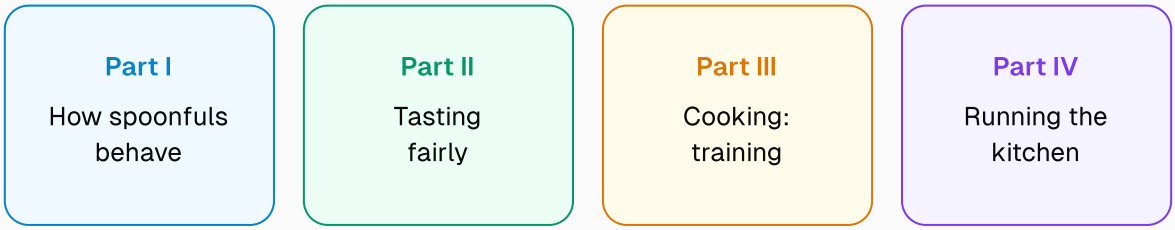


Taste twenty spoonfuls and a shape appears. The shape is a distribution. With it you can say "just right, give or take a little", which is what a confidence interval says about a model's score.



When a score becomes the target, it stops measuring what you wanted.

If the cook learns that one taster loves sugar, every dish gets sweeter and the taster's score climbs while real diners suffer. In model training this is reward hacking, and it is why evaluation and training have to be designed together (Part IV).



Each chapter picks its own everyday example, like this one.

That is the map. Every chapter opens with a picture like this, using whatever everyday example fits best: mail sorting, exams, sports, shopping, maps.

Three layers in every chapter

Plain words

An analogy and the core idea, with no jargon and no formulas. Read this first, even if you think you know the topic.

For leaders

What decision the idea informs, what it costs, how it fails, and the questions a senior reviewer asks. This is the layer for staff, principal, and partner conversations.

Under the hood

The math, the derivations, and short runnable code. Enough to check someone's work, not just follow it.

Each topic shows its three depths as tabs. The **Plain / Leader / Deep** switch in the top bar flips every topic in the guide at once; **All** stacks them. A good first pass is Plain for everything, then Leader, then Deep for the chapters that matter to your work. Press `⌘K` (or `/`) to search chapters, topics, widgets, and glossary terms.

Plain words

What a layer looks like

Imagine tasting soup to decide if it needs salt. One spoonful tells you something, but not everything. Evaluation is tasting the model. Training is cooking. This guide teaches both, and how to tell when one spoonful is lying to you.

For leaders

Why both halves matter at senior levels

At senior levels in a frontier lab, decisions rarely turn on a single technique. They turn on whether a result is real, what it cost, and what it will break. Training choices (data, objective, RL method) determine what can go wrong. Evaluation choices determine whether you notice. Leaders who understand both can push back on a launch, scope a research bet, or kill a project with evidence instead of instinct.

Under the hood

How the math and code are shown

Formulas are typeset, for example the cross-entropy loss that almost every chapter in Part III builds on:

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

The average negative log probability the model assigns to each correct next token x_t, given everything before it.

python

```python
import math

def cross_entropy(probs_of_correct_token):
    # average surprise, in nats, at the token that actually came next
    return -sum(math.log(p) for p in probs_of_correct_token) / len(probs_of_correct_token)

print(round(cross_entropy([0.9, 0.5, 0.25]), 3))  # 0.728
print(round(math.exp(0.728), 2))                  # perplexity 2.07
```

rust

```rust
fn cross_entropy(probs_of_correct_token: &[f64]) -> f64 {
    // average surprise, in nats, at the token that actually came next
    -probs_of_correct_token.iter().map(|p| p.ln()).sum::<f64>() / probs_of_correct_token.len() as f64
}

fn main() {
    println!("{:.3}", cross_entropy(&[0.9, 0.5, 0.25])); // 0.728
    println!("{:.2}", 0.728_f64.exp());                  // perplexity 2.07
}
```

Map of the guide

Part	What you get	Start here if
I. Statistical foundations	Distributions, sampling, standard error, intervals, p values, reading a shape, heavy tails	It has been years since you did statistics
II. Evaluating models	Pass-rate intervals, paired comparisons, power, multiple comparisons, classification metrics through AUC, LLM judges, the offline to online gap, contamination, harness engineering, regression triage	You need to defend or challenge an eval result
III. Training models	The pipeline, pretraining, optimisation, scaling laws, SFT, LoRA, reward models, RLHF and PPO, DPO and its relatives, GRPO and RL with verifiable rewards, distillation, inference-time compute, safety training	You need to talk credibly about how models are built
IV. Leading the work	How training and evaluation connect, review checklists, the questions principals ask, and a glossary	You are preparing for a senior conversation tomorrow

Note

Interactive panels run in the web version. The PDF shows each one at its default setting.

Part I

Statistical foundations

Distributions, sampling, intervals, and how to read the shape of your data. Everything later leans on this.

Part I · Chapter 2

Distributions: centre, spread, and shape

Every number an eval produces, whether a pass rate, a latency, a token count, or a judge score, is a summary of a spread of measurements. This chapter gives you the vocabulary for describing that spread and the reflexes for spotting when a summary is lying.

Diagram

A hundred stopwatches on one bus route

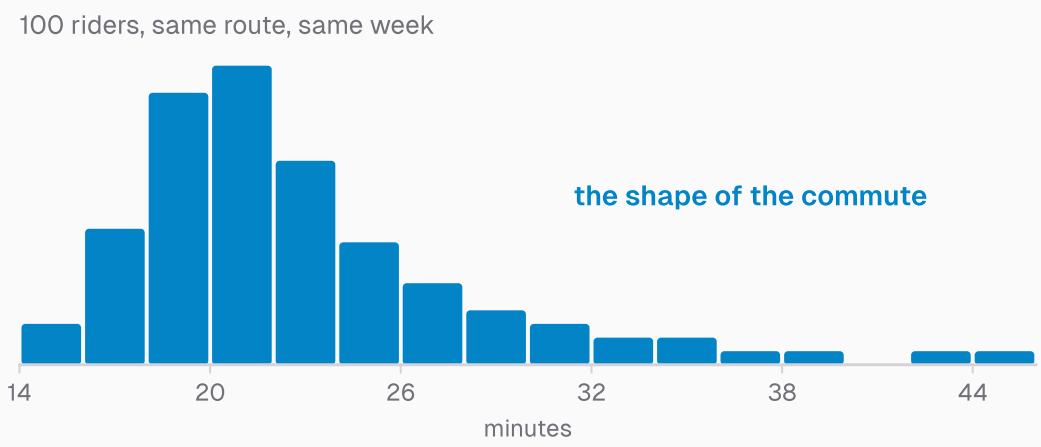
What a distribution is, and the three words that describe it: centre, spread, shape.

Route 12, Tuesday 8:05

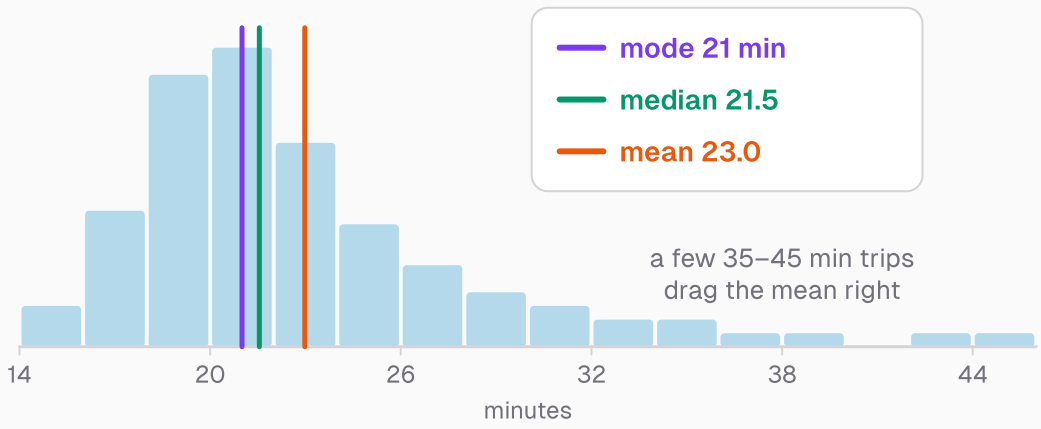
20 minutes

One trip gives one number. Is 20 minutes typical?

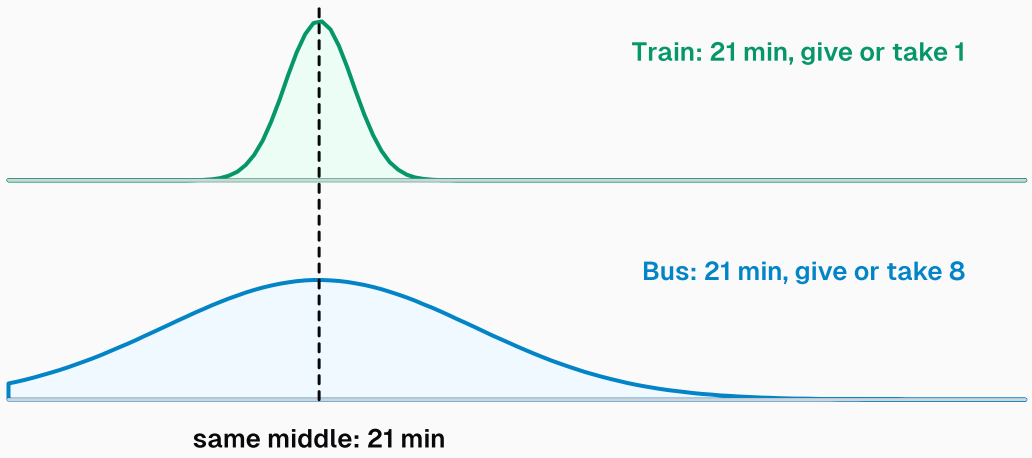
You time your bus ride once: 20 minutes. One measurement is a single dot. It cannot tell you whether 20 is normal, lucky, or slow.



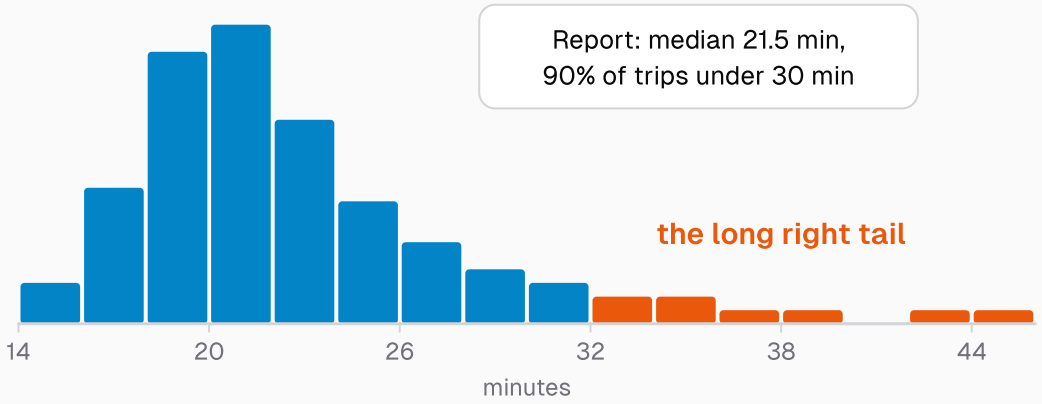
Ask 100 riders on the same route and stack their times into bars. Most rides take 18 to 24 minutes, a few take much longer. That outline is the **distribution**.



"Typical" has three answers. The **mode** (most common) is about 21, the **median** (half above, half below) is 21.5, and the **mean** is 23.0. The handful of 35 to 45 minute trips pull the mean to the right.



Two routes can share a middle and feel completely different. The train is 21 minutes almost every day; the bus is 21 on average but swings by 8. That swing is the **spread**, and it is what makes you late.



The **shape** leans right: a long tail of bad days. When a distribution leans, a single average misleads. Say what riders feel: the median plus a high percentile. Eval latency, token counts, and loop lengths all lean like this.

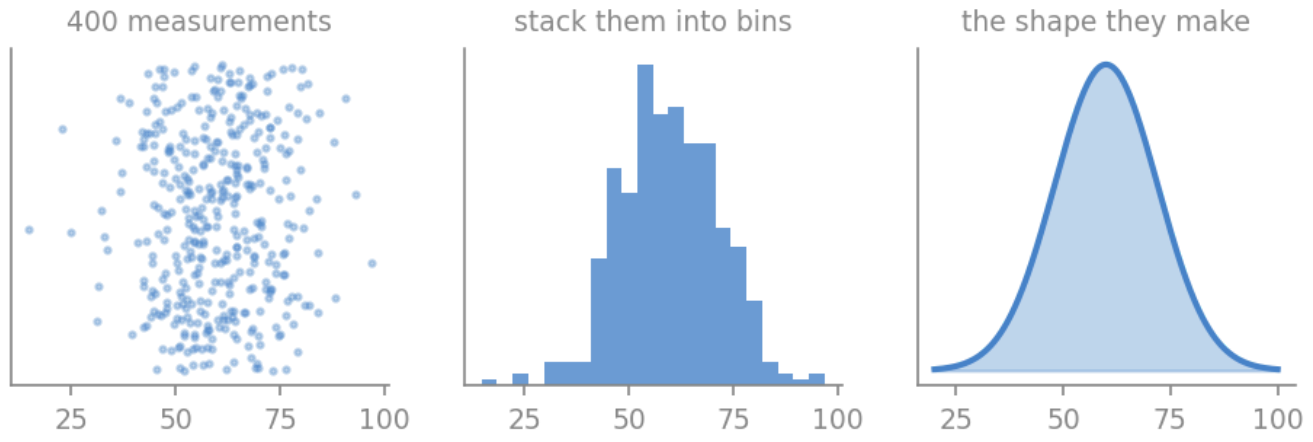
Plain words

A distribution is the shape a spread of answers makes

Measure the same thing many times and you do not get one answer, you get a spread of answers. A **distribution** is the shape that spread makes. That is the whole idea. Everything else in this chapter is vocabulary for describing shapes: where the middle is, how wide it is, and which way it leans.

Think of a hundred people timing the same bus route. Most say about twenty minutes, a few say fifteen, and one person who hit every red light says forty-five. If you stack their answers into a bar chart, the shape you see is the distribution of the commute.

What a distribution actually is



Left: 400 individual measurements. Middle: stack them into bins. Right: the shape they make. That shape is the distribution.

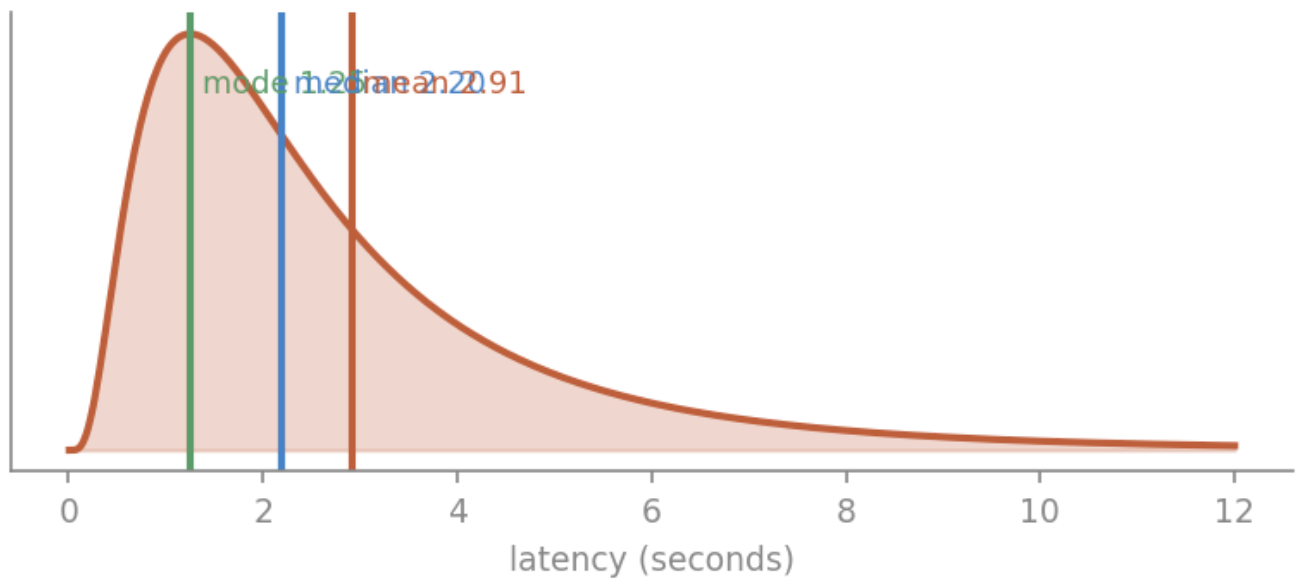
Three words get used loosely and mean different things:

- **Population** is every possible measurement you could ever take. Usually imaginary, always unavailable.
- **Sample** is the measurements you actually have. 200 tasks, 5000 sessions.
- **Distribution** is the shape. A population has one, and your sample gives you a blurry picture of it.

Eval lens

Your eval suite is a sample. Production is closer to the population. The offline to online gap is, in one sentence, the sample not looking like the population.

Centre: three different answers to "what is typical"



The same data has three different centres. On a skewed distribution they are far apart, and picking the wrong one is the most common reporting error there is.

Measure	What it is	Use when	Breaks when
Mean	Add everything, divide by n	Symmetric data. Also anything where the total matters, like compute cost	The tail is long. One 400 second request drags it and no user experiences it
Median	The middle value when sorted	Skewed data. Almost all latency and token reporting	You need totals, or the data is bimodal (the middle can fall in the empty valley)
Mode	The most common value	Categorical data, spotting bimodality	Continuous data, where no value repeats exactly

Key idea

On a right skewed distribution the order is always mode < median < mean. If someone reports a mean latency well above the median, they are reporting a number no user experienced.

Under the hood

Mean and median, defined

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{median} = x_{((n+1)/2)} \text{ of the sorted values}$$

For even n, the median is the average of the two middle sorted values.

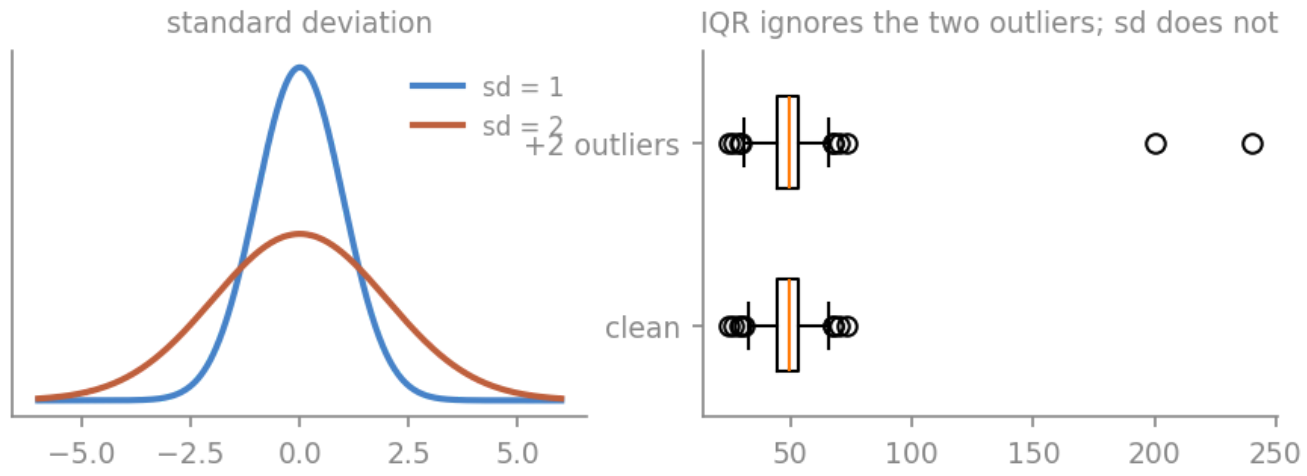
python

```python
import numpy as np

lat = [1.9, 2.1, 2.0, 2.4, 2.2, 1.8, 2.3, 2.0, 41.0]   # one slow request

# THE MEAN: add everything, divide by n. One outlier drags it a long way.
print(sum(lat) / len(lat))              # 6.41 - no request took anywhere near this

# THE MEDIAN: sort, take the middle. The outlier moves it by nothing.
s = sorted(lat)
n = len(s)
median = s[n // 2] if n % 2 else (s[n // 2 - 1] + s[n // 2]) / 2
print(median)                           # 2.10 - describes a typical request

# THE RATIO: the fastest diagnostic in this whole part of the guide.
print((sum(lat) / len(lat)) / median)   # 3.05 -> heavily right skewed
```

rust

```rust
fn main() {
    let lat = [1.9, 2.1, 2.0, 2.4, 2.2, 1.8, 2.3, 2.0, 41.0]; // one slow request

    // THE MEAN: add everything, divide by n. One outlier drags it a long way.
    let mean = lat.iter().sum::<f64>() / lat.len() as f64;
    println!("{}", mean);           // 6.41 - no request took anywhere near this

    // THE MEDIAN: sort, take the middle. The outlier moves it by nothing.
    let mut s = lat.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));
    let n = s.len();
    let median = if n % 2 == 1 { s[n / 2] } else { (s[n / 2 - 1] + s[n / 2]) / 2.0 };
    println!("{}", median);         // 2.10 - describes a typical request

    // THE RATIO: the fastest diagnostic in this whole part of the guide.
    println!("{}", mean / median); // 3.05 -> heavily right skewed
}
```

Spread: how much things vary



Left: standard deviation controls the width. Right: two outliers barely move the box (which is built from quartiles) but would move the standard deviation a lot.

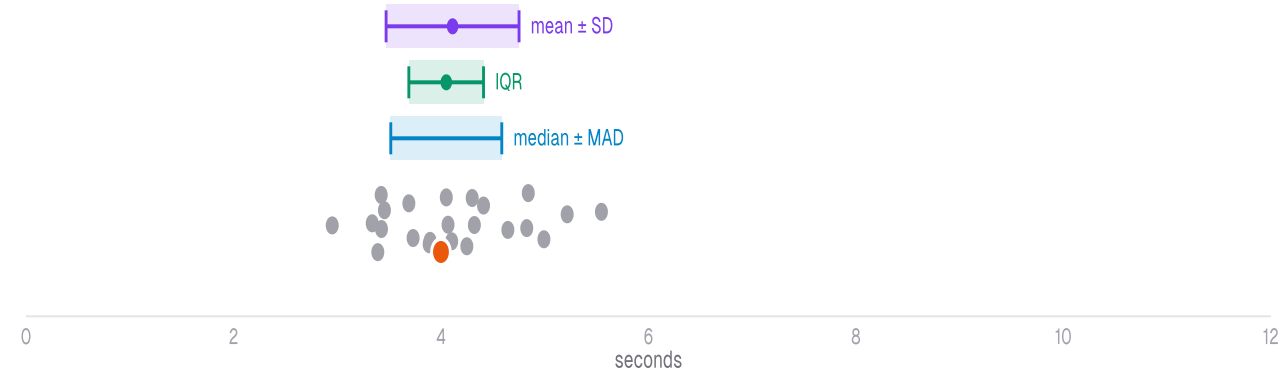
Measure	Definition in words	Strength	Weakness
Variance	Average squared distance from the mean	Adds up nicely across independent sources, which is why the maths uses it	Wrong units. Variance of seconds is in seconds squared
Standard deviation	Square root of variance, back in the original units	Interpretable. Roughly 68% of normal data sits within 1 sd	One extreme value can dominate it
IQR	The range covering the middle 50%, p75 minus p25	Outliers cannot touch it	Ignores the tails entirely, which may be the thing you care about
MAD	Median of the absolute distances from the median	Very robust	Less familiar, and needs a scaling constant to compare with sd

The centre tells you what is typical. Spread tells you how far from typical things wander. There are four measures you will meet everywhere, and they disagree in useful ways: two are built from squared distances

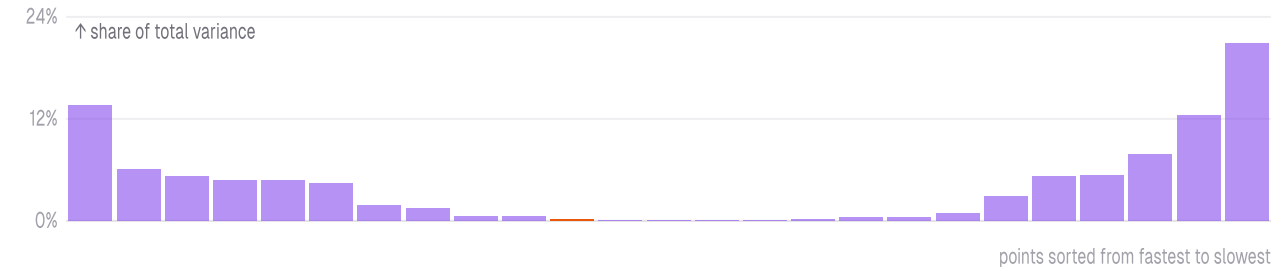
and react strongly to extremes, two are built from ranks and barely notice them. Try it below, then read each one in depth.

Interactive **Spread explorer: one dataset, four rulers** (shown at its default setting)

Twenty-four response times (seconds) plus one request you control. Drag it out to the right and watch which measures react. The lower chart shows how much of the total variance each point owns.



mean ± 1 SD · median and IQR (middle 50%) · median ± scaled MAD · the request you control



Variance (s^2)	Standard deviation	IQR	MAD (scaled ×1.4826)
0.41	**0.64 s**	**0.72 s**	**0.54 s**

Your request alone owns **0%** of the variance. Compared with a typical 4 s request, SD is **1.0×** larger, IQR **1.00×**, MAD **1.00×**. SD and scaled MAD agree: no single point is dominating.

Plain words **Four ways to say "how much does my commute vary?"**

Picture a month of commutes. Most days take about 25 minutes, give or take a few. One day a crash makes it 95.

- **Variance** asks: on average, how big is the surprise, squared? Squaring makes the crash day count enormously, and the answer comes out in "minutes squared", which nobody can picture.
- **Standard deviation** takes the square root of that, back to minutes: "a typical day is about this far from average". It is still pulled up by the crash.
- **IQR** ignores the best and worst quarter of days and reports how wide the middle half is: "half my commutes fall in this 6-minute window". The crash does not change it.
- **MAD** asks: on a typical day, how far am I from my usual commute? It looks at the middle of the distances, so one disaster cannot move it.

None is "right". Variance and SD describe the whole month including the disaster. IQR and MAD describe a normal day. Which one you want depends on whether the disaster is the point.

For leaders

Which spread to ask for, and when

- **Error bars and significance run on variance.** Every standard error, confidence interval, power calculation, and sample-size estimate in Part II is built from variance. If someone shows an interval, the variance behind it is the thing to question.
- **For skewed operational metrics, ask for IQR or percentiles, not SD.** Latency, cost per request, and tokens per response are long tailed. A mean plus or minus SD on them describes no real user and can imply negative latencies.
- **For monitoring and anomaly alerts, MAD is the safe default.** A single incident inflates the SD, which then hides the next incident. MAD-based thresholds stay put.
- **When SD and scaled MAD disagree by more than about 1.5 times, the data has heavy tails or outliers.** That gap is a finding. Ask what is in the tail before choosing which number to report.

Principal asks

Is that standard deviation of the users, or standard error of the average? What does the spread look like with the top 1% removed? Are we alerting on SD-based thresholds, and what happens to them the day after an incident?

Under the hood

Four measures of spread

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \qquad s = \sqrt{s^2}$$

Sample variance and standard deviation. Dividing by $n - 1$ rather than n corrects for having used the same data to estimate the mean.

$$\text{IQR} = Q_3 - Q_1 = p_{75} - p_{25} \qquad \text{MAD} = \text{median}\left(|x_i - \text{median}(x)|\right)$$

The two robust measures. Multiply MAD by 1.4826 to compare it with the sd on normal data.

python

```python
import numpy as np

def spread(x):
    x = np.asarray(x, dtype=float)
    n = len(x)
    m = x.mean()

    # VARIANCE: mean squared distance from the mean.
    # n-1, not n. Dividing by n underestimates, because you used the same
    # data to compute the mean you are measuring distance from.
    var = ((x - m) ** 2).sum() / (n - 1)

    q1, q3 = np.quantile(x, [0.25, 0.75])          # IQR: middle 50%
    med = np.median(x)
    mad = np.median(np.abs(x - med))               # robust alternative to sd

    return {"sd": var ** 0.5, "var": var, "iqr": q3 - q1,
            "mad": mad, "mad_scaled": 1.4826 * mad}   # scaled to match sd on normal data

clean = [1.9, 2.1, 2.0, 2.4, 2.2, 1.8, 2.3, 2.0]
print(spread(clean))                    # sd 0.203, iqr 0.250, mad 0.150
print(spread(clean + [41.0]))           # sd 13.0 (!), iqr 0.300, mad 0.200
# One outlier multiplied the sd by 64x and barely touched the IQR or MAD.
```

rust

```rust
#[derive(Debug)]
struct Spread {
    sd: f64,
    var: f64,
    iqr: f64,
    mad: f64,
    mad_scaled: f64, // scaled to match sd on normal data
}

/// Sum the way numpy does: eight running totals combined in pairs.
/// More accurate than one running total, and it reproduces numpy's last digit.
```

```rust
fn pairwise_sum(x: &[f64]) -> f64 {
    if x.len() < 8 {
        return x.iter().sum();
    }
    if x.len() > 128 {
        let half = x.len() / 2 / 8 * 8;
        return pairwise_sum(&x[..half]) + pairwise_sum(&x[half..]);
    }
    let mut r = [0.0; 8];
    let full = x.len() / 8 * 8;
    for chunk in x[..full].chunks(8) {
        for j in 0..8 {
            r[j] += chunk[j];
        }
    }
    let mut total = ((r[0] + r[1]) + (r[2] + r[3])) + ((r[4] + r[5]) + (r[6] + r[7]));
    for v in &x[full..] {
        total += v;
    }
    total
}

fn mean(x: &[f64]) -> f64 {
    pairwise_sum(x) / x.len() as f64
}

/// Linear-interpolation quantile of SORTED data (numpy's default method).
fn quantile(sorted: &[f64], q: f64) -> f64 {
    let h = (sorted.len() - 1) as f64 * q;
    let lo = h.floor() as usize;
    let hi = (lo + 1).min(sorted.len() - 1);
    let (a, b, t) = (sorted[lo], sorted[hi], h - lo as f64);
    // interpolate from the nearer end, as numpy does
    if t >= 0.5 { b - (b - a) * (1.0 - t) } else { a + (b - a) * t }
}

fn median(x: &[f64]) -> f64 {
    let mut s = x.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));
    let n = s.len();
    if n % 2 == 1 { s[n / 2] } else { (s[n / 2 - 1] + s[n / 2]) / 2.0 }
}

fn spread(x: &[f64]) -> Spread {
    let n = x.len() as f64;
    let m = mean(x);

    // VARIANCE: mean squared distance from the mean.
    // n-1, not n. Dividing by n underestimates, because you used the same
    // data to compute the mean you are measuring distance from.
    let sq: Vec<f64> = x.iter().map(|v| (v - m).powi(2)).collect();
    let var = pairwise_sum(&sq) / (n - 1.0);

    let mut s = x.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));
    let (q1, q3) = (quantile(&s, 0.25), quantile(&s, 0.75)); // IQR: middle 50%
    let med = median(x);
    let dev: Vec<f64> = x.iter().map(|v| (v - med).abs()).collect();
    let mad = median(&dev); // robust alternative to sd
```

```
    Spread { sd: var.sqrt(), var, iqr: q3 - q1, mad, mad_scaled: 1.4826 * mad }
}

fn main() {
    let clean = vec![1.9, 2.1, 2.0, 2.4, 2.2, 1.8, 2.3, 2.0];
    println!("{:?}", spread(&clean)); // sd 0.203, iqr 0.250, mad 0.150
    let mut dirty = clean.clone();
    dirty.push(41.0);
    println!("{:?}", spread(&dirty)); // sd 13.0 (!), iqr 0.300, mad 0.200
    // One outlier multiplied the sd by 64x and barely touched the IQR or MAD.
}
```

Variance in depth

$$\sigma^2 = \mathbb{E}\big[(X-\mu)^2\big] \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i-\bar{x})^2$$

Left: the population variance, a property of the process that generates the data. Right: the sample variance, your estimate of it from n observations.

By hand

Data 2, 4, 4, 4, 5, 5, 7, 9. Mean 5. Squared distances 9, 1, 1, 1, 0, 0, 4, 16, sum 32. Divide by $n-1=7$: $s^2 = 4.57$. Divide by $n=8$ instead and you get 4.00, which underestimates.

Why squares

Squares make variance **add up**: for independent sources, $\operatorname{Var}(X+Y) = \operatorname{Var}X + \operatorname{Var}Y$. Scaling multiplies it: $\operatorname{Var}(aX) = a^2 \operatorname{Var}X$. These two rules are why all of inference is built on variance.

Use it for

The arithmetic underneath standard errors, power, bootstrap, ANOVA, the design effect, and in training: Adam's second moment, advantage normalisation, LayerNorm.

Watch out

Units are squared and one point can own most of it. Add a 60 to the data above and that single point contributes 88% of the total variance.

The $n - 1$ (Bessel's correction) exists because the deviations are measured from $\bar{x}$, which was fitted to the same data and sits closer to the points than the true μ does. Dividing by $n - 1$ makes s^2 unbiased for σ^2. For large n the difference vanishes; for a 5-seed ablation it is 25%.

The general addition rule includes a covariance term: $\operatorname{Var}(X + Y) = \operatorname{Var} X + \operatorname{Var} Y + 2\operatorname{Cov}(X, Y)$. That term is why paired comparisons are so powerful: for a difference $X - Y$ the sign flips, and positively correlated scores on shared tasks *subtract* variance (see paired comparisons). It is also why clustered data needs care: correlated items add variance a naive formula ignores (see clustering).

Standard deviation in depth

$$\sigma = \sqrt{\sigma^2} \qquad s = \sqrt{s^2} \qquad z = \frac{x - \bar{x}}{s}$$

SD is variance brought back to the original units. The z-score expresses any value as "how many SDs from the mean".

By hand

Same data: $s = \sqrt{4.57} = 2.14$. A value of 9 has $z = (9 - 5)/2.14 = 1.87$: almost two typical distances above the mean.

Normal benchmarks

For normal data, about 68%, 95%, and 99.7% of values fall within 1, 2, and 3 SDs. Only for normal data: for skewed data these percentages are wrong.

Use it for

Describing variability in readable units, z-scores, effect sizes such as Cohen's $d = (\bar{x}_1 - \bar{x}_2)/s$, seed-to-seed noise in training runs, and as the numerator of the standard error.

Watch out

SD does not add. $\sigma_{X+Y} = \sqrt{\sigma_X^2 + \sigma_Y^2}$, not $\sigma_X + \sigma_Y$. And SD is as fragile as variance: with the 60 added, SD jumps from 2.14 to 18.4.

Interquartile range in depth

$$\mathrm{IQR} = Q_3 - Q_1 = F^{-1}(0.75) - F^{-1}(0.25) \qquad \text{fences: } \left[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}\right]$$

The width of the middle half of the data, from the quantile function F^{-1}. The Tukey fences on the right are the usual box-plot rule for flagging outliers.

By hand

Sorted 2, 4, 4, 4, 5, 5, 7, 9. With linear interpolation (numpy's default) $Q_1 = 4$, $Q_3 = 5.5$, IQR $= 1.5$. Add the 60 and IQR becomes 3.0. Quantile conventions differ slightly between tools, so state which one you use.

Normal benchmark

On normal data $\mathrm{IQR} = 1.349\,\sigma$, so $\mathrm{IQR}/1.349$ is a robust stand-in for SD.

Use it for

Box plots, outlier fences, describing latency and token counts, and robust scaling of features. It pairs naturally with the median, the same way SD pairs with the mean.

Watch out

It ignores the outer half of the data by design. If the tail is the product (p99 latency, rare catastrophic answers), IQR hides exactly what matters. Report a tail percentile alongside it.

Median absolute deviation in depth

$$\mathrm{MAD} = \mathrm{median}_i\left(|x_i - \mathrm{median}(x)|\right) \qquad \hat{\sigma}_{\mathrm{MAD}} = 1.4826 \cdot \mathrm{MAD} \qquad M_i = \frac{0.6745\,(x_i - \tilde{x})}{\mathrm{MAD}}$$

The middle of the distances from the middle. The constant $1.4826 = 1/\Phi^{-1}(0.75)$ rescales MAD so it estimates σ on normal data. M_i is the robust (modified) z-score; $|M_i| > 3.5$ is a common outlier flag.

By hand

Median of the data is 4.5. Distances 2.5, 0.5, 0.5, 0.5, 0.5, 0.5, 2.5, 4.5; their median is 0.5. So MAD $= 0.5$, scaled $= 0.74$. Five of the eight values sit within half a unit of the median, and MAD reports exactly that.

Breakdown point

Up to half the data can be replaced by garbage before MAD can be pushed anywhere. Add the 60: MAD moves from 0.5 to 1.0. SD moved from 2.14 to 18.4.

Use it for

Anomaly detection and alert thresholds, cleaning before fitting, flagging flaky eval tasks, and any dashboard where one bad day should not redefine "normal".

Watch out

It is less precise than SD when data really is normal (about 37% efficiency), and it can be zero when more than half the values are identical, as with pass/fail scores. Use it on continuous metrics.

The four side by side

Property	Variance	Standard deviation	IQR	MAD
Units	Squared	Original	Original	Original
Built from	Squared distances from the mean	Same, square-rooted	Ranks (quartiles)	Ranks (median of distances)
Breakdown point (share of bad data it survives)	0%	0%	25%	50%
On normal data equals	σ^2	σ	$1.349\,\sigma$	$0.6745\,\sigma$
Efficiency on normal data	100%	100%	≈37%	≈37%
Adds across independent parts	Yes	No (add variances, then root)	No	No
Natural partner	Mean	Mean	Median	Median

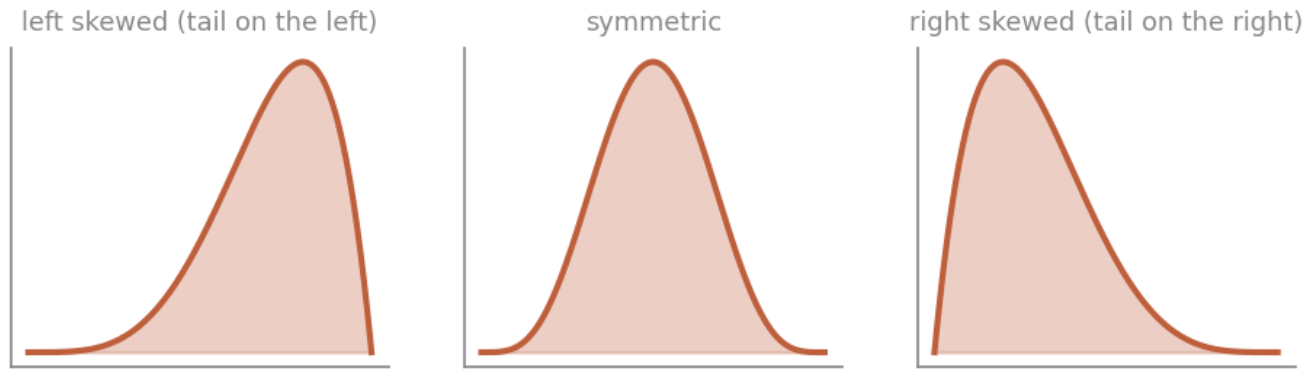
Where each one shows up in the rest of the guide

Measure	Where you will meet it
Variance	Standard errors and confidence intervals (chapter 3), paired tests and power (eval statistics), the design effect for clustered agent traces (intervals), Adam's per-parameter running variance of gradients (optimisation), advantage normalisation in PPO and GRPO (RLHF, GRPO), the bias–variance view of overfitting (SFT)
Standard deviation	Seed-to-seed noise and how many seeds you need (leading), z-scores and effect sizes, GRPO's division by the group SD and why Dr. GRPO removes it (GRPO), flaky-task quarantine thresholds (harness)
IQR	Box plots and outlier fences (long tails), latency and token-count reporting (shape to method), robust comparisons when data is skewed
MAD	Robust outlier detection (long tails), regression alerts that survive incidents (triage), cleaning noisy offline metrics before backtesting (offline to online)

Key idea

Spread of the data is not precision of an estimate. Everything above describes how individual values vary. The **standard error** describes how much an *estimate*, such as the mean of those values, would vary if you repeated the whole measurement. Formally, the standard error of an estimator $\hat{\theta}$ is the standard deviation of its sampling distribution, $\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$. For the mean of n independent values it is $\sigma/\sqrt{n}$. Variance stays put as you collect more data; standard error shrinks. The full definition, the derivation, and an interactive comparison are in chapter 3.

Shape: skew and modality



Skew is named for the side the tail is on, not the side the bulk is on. This trips up almost everyone at first.

- **Right skewed** (positive skew). Long tail to the right. Latency, token counts, tool calls, repo sizes. By far the most common shape in eval work.
- **Left skewed**. Long tail to the left. Rarer. Shows up in scores near a ceiling, like a pass rate close to 100%.
- **Symmetric**. Both tails alike. Test statistics, averages of many things.
- **Bimodal**. Two humps. Almost always two populations mixed together, not one distribution being odd.

Key idea

The quickest skew check you can do in your head: **divide the mean by the median**. Above about 1.2 means right skewed enough that you should stop reporting the mean. Below about 0.85 means left skewed. This one ratio catches most bad reporting.

Under the hood

Skewness, formally and quickly

$$g_1 = \frac{\frac{1}{n}\sum(x_i - \bar{x})^3}{s^3}$$

Formal skewness. Positive means a right tail. In practice the mean/median ratio below is quicker and just as useful.

$$\frac{\bar{x}}{\text{median}(x)} > 1.2 \Rightarrow \textbf{right skewed}$$

Density, cumulative, and quantiles

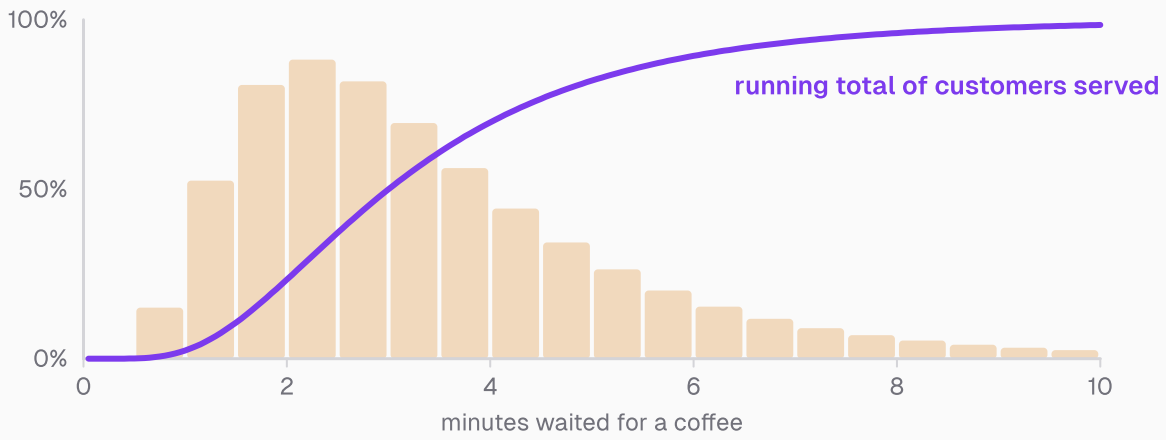
Three ways of describing the same shape. You need all three because different questions want different ones.

Diagram

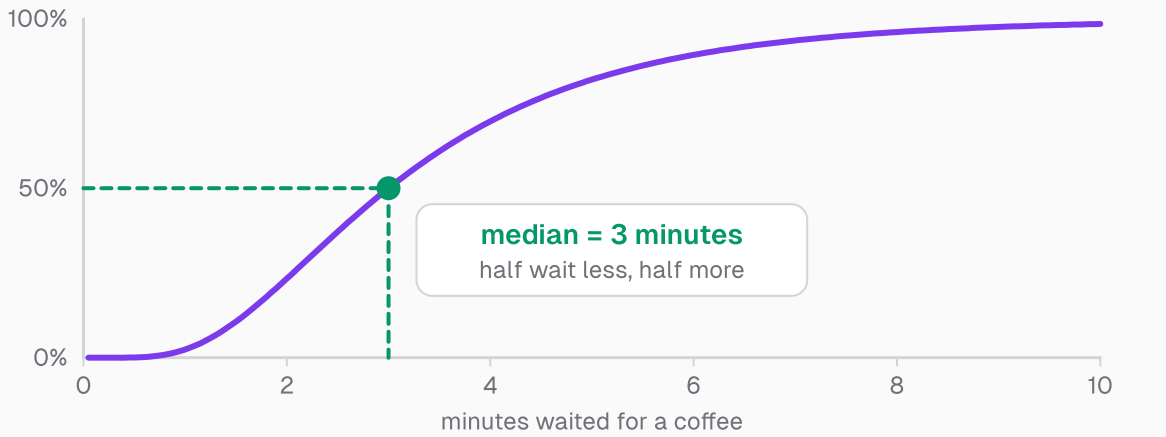
Reading a café queue as a cumulative curve

How a CDF turns a pile of waiting times into answers like "half of customers wait under 3 minutes".

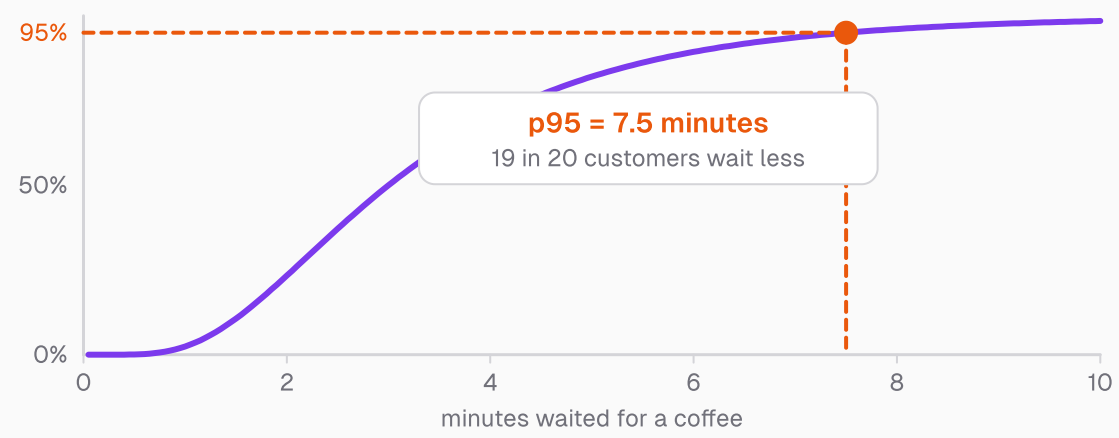
Two hundred coffee orders, timed. The histogram shows where waits pile up, but it is hard to read off a question like "how long do most people wait?"



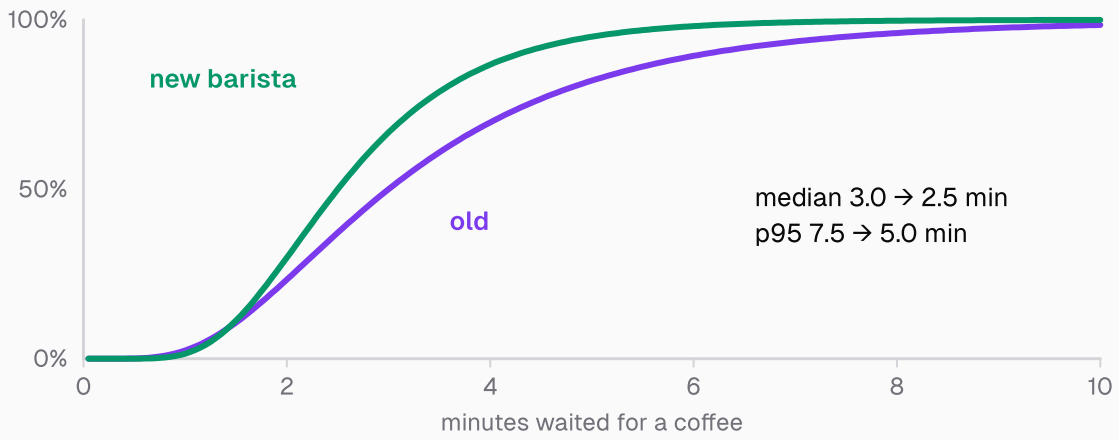
Walk left to right and keep a running total of the share of customers already served. That rising curve is the **cumulative distribution** (CDF). It always climbs from 0% to 100%.



To read the **median**, go across from 50% until you hit the curve, then drop down: 3 minutes. Half the customers waited less.



The same move at 95% gives the **p95**: 7.5 minutes. Nineteen customers in twenty waited less. Latency targets in model serving are written exactly like this.



Compare two curves at a glance: a curve that sits to the left is faster everywhere, from the median to the tail. Overlaying CDFs is the quickest honest way to compare two latency or token-count distributions.

- **Density (PDF)** answers "how common is a value around here". This is the humped curve you picture when someone says distribution.
- **Cumulative (CDF)** answers "what share of values are at or below here". Always rises from 0 to 1.
- **Quantiles** are the CDF read backwards: "what value has 95% of the data below it". p95 is a quantile. So is the median, which is p50.

> **Note**
>
> Percentile and quantile are the same thing at different scales. p95, the 95th percentile, and the 0.95 quantile are three names for one number.

Under the hood CDF and quantile function

$$F(x) = P(X \le x) \qquad Q(p) = F^{-1}(p)$$

The CDF gives the share at or below a value. The quantile function is its inverse: give it 0.95 and it hands back p95.

python

```python
import numpy as np

lat = np.array([1.2, 1.5, 1.9, 2.1, 2.4, 3.0, 4.5, 8.0, 15.0, 40.0])

# QUANTILE: what value has p of the data below it?
print(np.quantile(lat, 0.50))     # 2.70  -> the median
print(np.quantile(lat, 0.95))     # 28.7  -> p95

# CDF: run it the other way. What share is at or below 4.5?
print((lat <= 4.5).mean())        # 0.70

# These are inverses of each other. Same information, two directions.
```

rust

```rust
/// Linear-interpolation quantile of SORTED data (numpy's default method).
fn quantile(sorted: &[f64], q: f64) -> f64 {
    let h = (sorted.len() - 1) as f64 * q;
    let lo = h.floor() as usize;
    let hi = (lo + 1).min(sorted.len() - 1);
    let (a, b, t) = (sorted[lo], sorted[hi], h - lo as f64);
    // interpolate from the nearer end, as numpy does
    if t >= 0.5 { b - (b - a) * (1.0 - t) } else { a + (b - a) * t }
}

fn main() {
    let lat = [1.2, 1.5, 1.9, 2.1, 2.4, 3.0, 4.5, 8.0, 15.0, 40.0]; // already sorted

    // QUANTILE: what value has p of the data below it?
    println!("{}", quantile(&lat, 0.50)); // 2.70  -> the median
    println!("{}", quantile(&lat, 0.95)); // 28.7  -> p95

    // CDF: run it the other way. What share is at or below 4.5?
    let share = lat.iter().filter(|&&v| v <= 4.5).count() as f64 / lat.len() as f64;
    println!("{}", share);              // 0.70
```

```
    // These are inverses of each other. Same information, two directions.
}
```

For leaders

Reading a dashboard number without being fooled

Most bad decisions built on eval data are not caused by exotic statistics. They come from a single summary number standing in for a shape nobody looked at. The questions below take thirty seconds and catch most of it.

- **"Is that a mean or a median, and what is the other one?"** For latency, cost per task, or tokens per response, a mean far above the median tells you a tail is driving the number. A mean is right for capacity and budget (totals); a median or p95 is right for user experience.
- **"What is the spread, and in what units?"** A 3 point win means little if per-task scores swing by 20 points. Ask for an IQR or a percentile band, not just an average.
- **"Has anyone plotted it?"** A bimodal distribution, such as cache hits versus cold starts or easy versus hard tasks, means there are two stories. Averaging them produces a number that describes neither and hides which one moved.
- **"Which population is this a sample of?"** An eval suite is a sample chosen by people. If it over-represents easy or old tasks, every summary inherits that.

Principal asks

"The mean latency improved by 12%. What happened to p50 and p95?" If the mean improved because a handful of pathological requests got fixed, that is good engineering but says nothing about the typical user. If p95 got worse while the mean improved, the launch may hurt the users who already suffer most.

Part I · Chapter 3

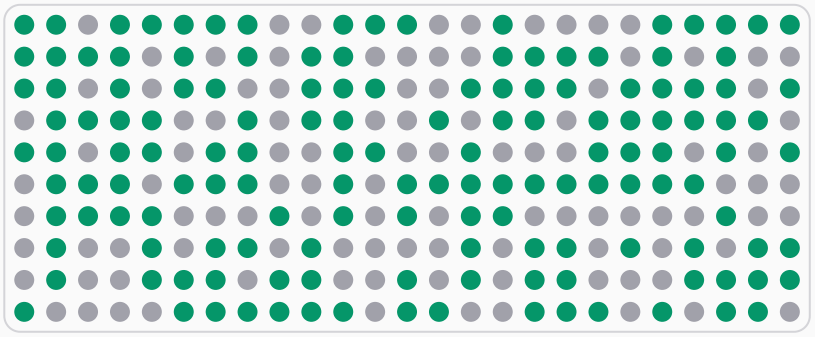
Two big ideas: the sampling distribution and the standard error

If you take only two things from Part I, take these. Nearly every statistical mistake in eval work is a misunderstanding of one of them, and every confidence interval and p value later in the guide is built on them.

Diagram

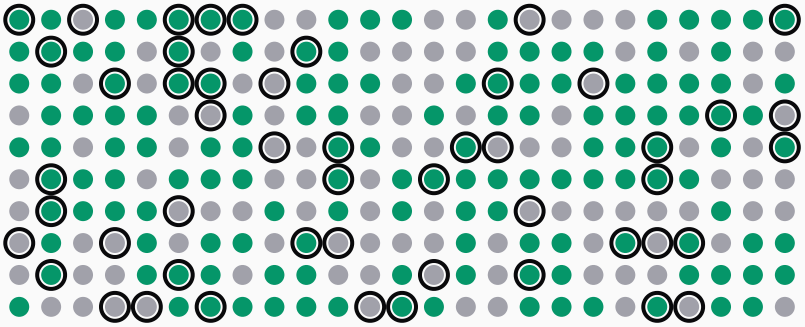
Polling a neighbourhood about a new park

The sampling distribution and the standard error, with dots you can count.



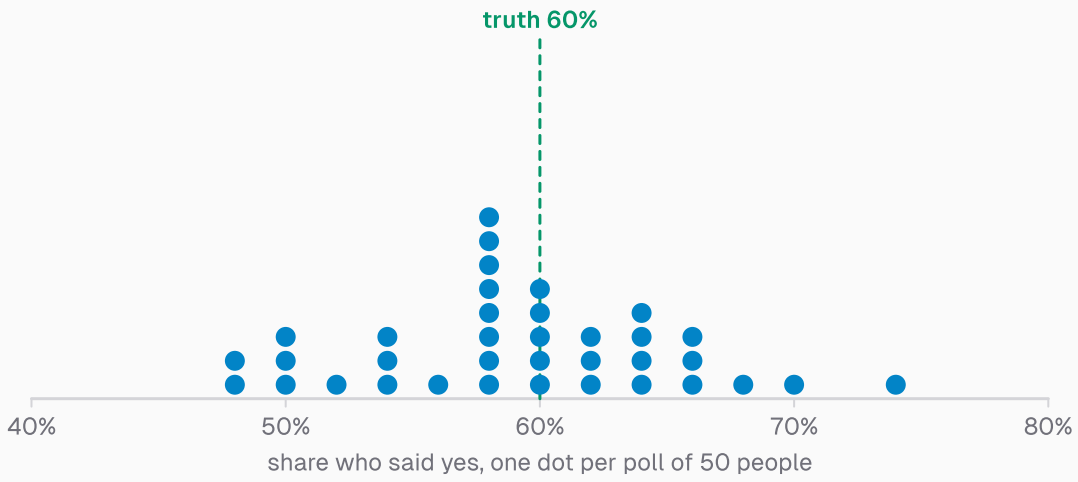
likes the new park does not **true share: 60%**

A town of 10,000 people (each dot stands for 40). Exactly 60% like the plan for a new park. In real life you never see this picture; you only get to ask a few people.

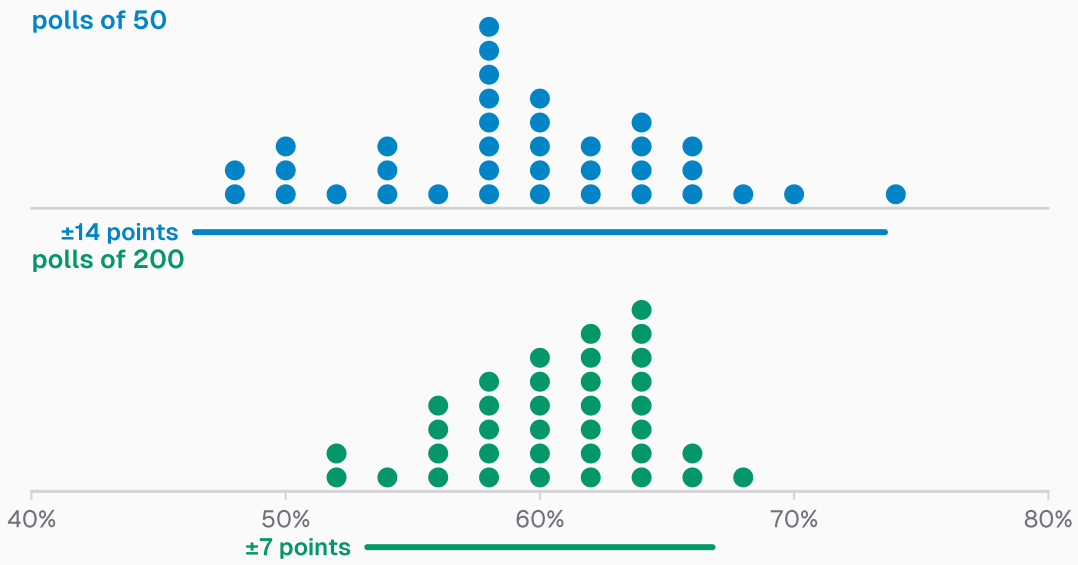


You ask 50 people
31 say yes: 62%

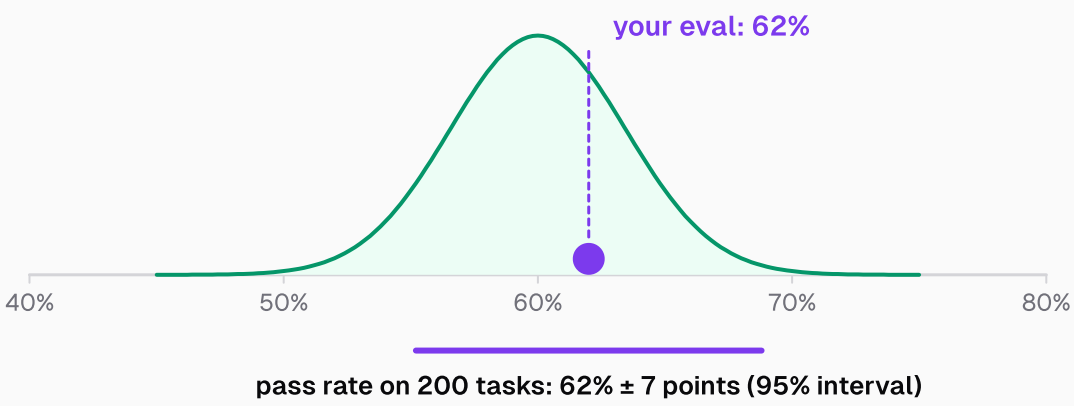
You stop 50 people at random. 31 say yes, so your poll says 62%. Close to the truth, but not equal to it, and you have no way to know how close.



Imagine 36 different pollsters, each asking 50 different people. Their answers pile up around 60%, some as low as 48%, some as high as 72%. This pile is the **sampling distribution**.



Ask 200 people per poll and the pile tightens to about half the width. Four times the people buys half the uncertainty. The width of the pile is the **standard error**.



pass rate on 200 tasks: 62% ± 7 points (95% interval)

An eval score is one poll of tasks. A 62% pass rate on 200 tasks is one dot from a pile about ±7 points wide. That is why "62% vs 60%" on 200 tasks is not a result yet.

Plain words

Your score is one draw from a lottery of possible scores

You ran 200 tasks and got 62%. If you had run a *different* 200 tasks you would have got something else, maybe 59%, maybe 65%. The spread of all those hypothetical alternative answers is the **sampling distribution**. Every confidence interval and every p value is a statement about it, not about your data.

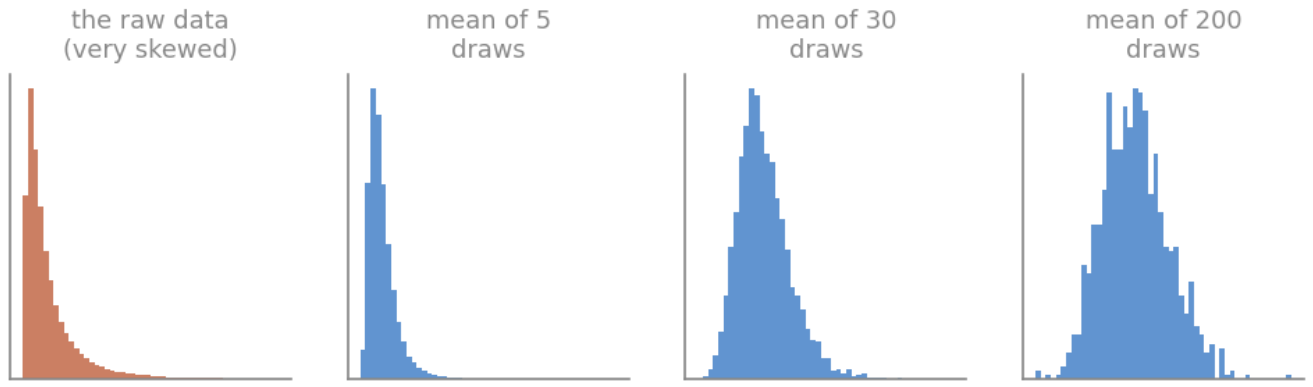
The **standard error** is how wide that lottery is. It shrinks as you test more tasks, but slowly: to cut the uncertainty in half you need four times as many tasks.

The sampling distribution, and why the bell curve is everywhere

Here is the part that surprises people. The raw data can be any shape at all. But the *average* of many draws is always roughly a bell, and it gets more bell-like as you average more. That is the central limit theorem.

$$\bar{X}_n \longrightarrow \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right) \quad \text{as } n \text{ grows}$$

The average of n draws tends toward a normal shape centred on the true mean, with variance shrinking as one over n.



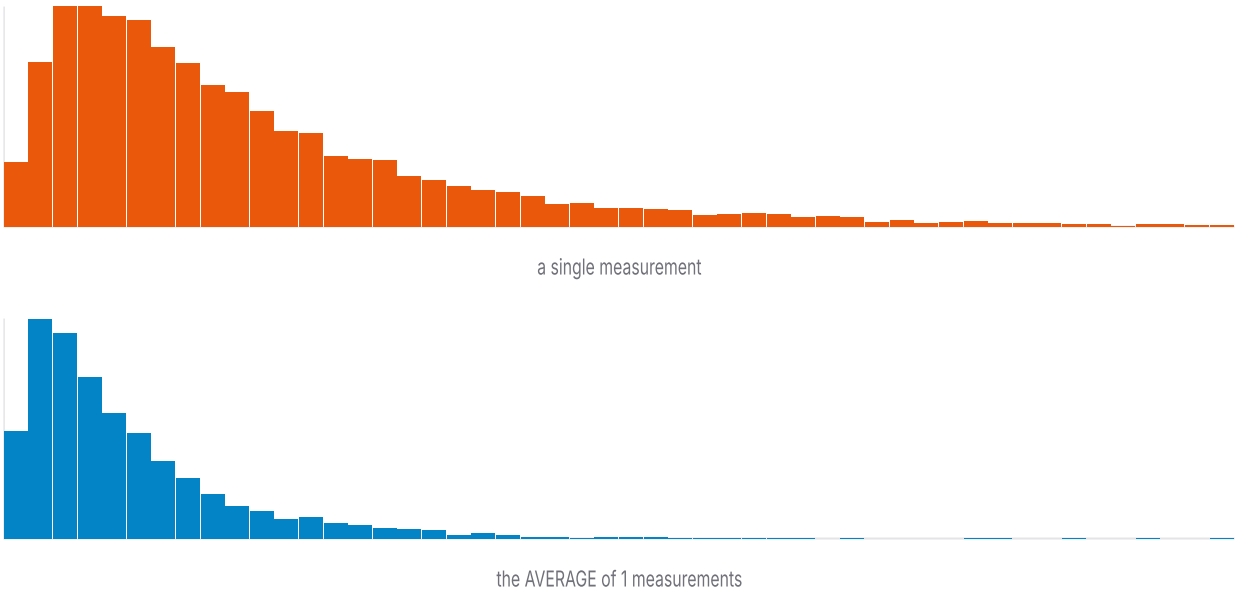
The raw data on the left is wildly skewed. Average 5 of them and it is still skewed. Average 30 and it is nearly a bell. Average 200 and it is a bell.

Key idea

This is why so much of statistics assumes normality even though almost no real data is normal. The assumption is not about your measurements, it is about their average. Your latency data is lognormal and always will be; the mean of 500 latencies is close to normal.

Interactive **Watch the central limit theorem happen** (shown at its default setting)

Pick any starting shape, however ugly. The top panel is a single measurement. The bottom is the average of n measurements. Push n up and watch the bottom become a bell regardless of what the top looks like.



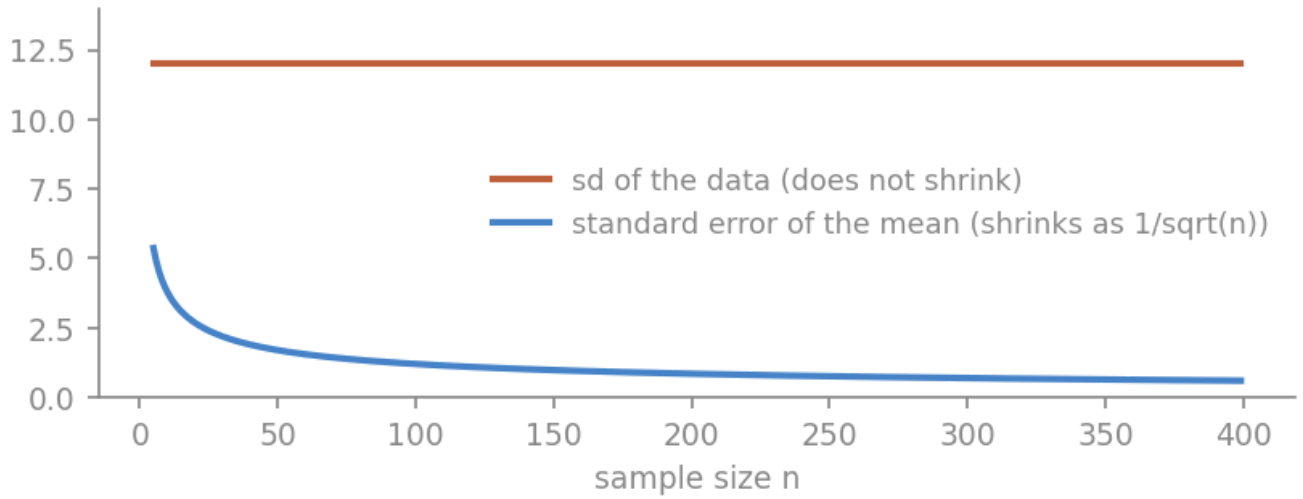
Raw data: mean 4.11, median 3.03, skew ratio 1.35
Averages of 1: mean 4.00, median 2.94, skew ratio 1.36, sd 3.68
The top shape can be anything at all. The bottom one becomes a bell as n grows, and its spread shrinks like 1/sqrt(n). That is the central limit theorem, and it is why so much of statistics assumes normality even when the raw data is nothing like normal.

Try it

Set the starting shape to Pareto and push n to 200. It becomes a bell far more slowly than the others. That is what a genuinely heavy tail does: the central limit theorem still applies, but it needs much more data before you can lean on it.

Standard deviation is not standard error

This is the single most common confusion in applied statistics, and the two get swapped in dashboards constantly.



The standard deviation of your data does not shrink as you collect more. The standard error of the mean does, as one over the square root of n.

	Standard deviation	Standard error
Describes	How spread out the individual measurements are	How precisely you know the average
Formula	s	$s/\sqrt{n}$
As n grows	Stays put. More data does not make users more alike	Shrinks, but slowly
Use for	Describing variability, setting SLOs	Error bars on a mean, confidence intervals

Key idea

Because standard error shrinks as one over the square root of n, **quadrupling your sample halves your error bar**. Not doubling, quadrupling. That single fact explains why every sample size answer feels so unsatisfying and why chasing small effects gets expensive so fast.

Plain words

Scattered arrows, wobbling averages

An archer shoots a volley of 20 arrows. The arrows land all over the target: that scatter is the **standard deviation**, and practising more volleys will not make a single arrow land closer to the others.

Now mark the average landing spot of each volley. Shoot another volley and the average lands somewhere slightly different. How much that average wobbles from volley to volley is the **standard error**. Shoot 80 arrows per volley instead of 20 and the average wobbles half as much, even though each arrow is just as scattered as before.

Variance is the standard deviation squared. It describes the arrows. The standard error describes your *estimate* built from the arrows.

For leaders

Reading error bars correctly

- **Ask which one the bars are.** SD bars show how varied the users or tasks are and never shrink. SE bars (or a confidence interval, about ±2 SE) show how sure we are of the average and shrink with more data. Decks swap them constantly.
- **Precision is bought with the square root.** Halving an error bar costs four times the data. That is why small effects get expensive and why reducing variance (paired designs, cleaner tasks) is often cheaper than collecting more.
- **Standard errors assume independent items.** Multi-turn conversations, several calls per agent session, or many tasks from one repository are correlated. The honest SE is larger, sometimes several times larger.

Principal asks

Are those SD or SE bars? What is n, and are the items independent? If we doubled the eval set, how much would the interval shrink, and is that cheaper than a paired design?

Under the hood

Standard error, formally, and the square-root law in code

Definition. An estimator $\hat{\theta}$ (a mean, a pass rate, a median, a difference between two models) is computed from a random sample, so it is itself a random variable with its own distribution across hypothetical repeats of the study: its *sampling distribution*. The **standard error** is the standard deviation of that sampling distribution:

$$\mathrm{SE}(\hat{\theta}) \;=\; \sqrt{\mathrm{Var}(\hat{\theta})} \;=\; \sqrt{\mathbb{E}\Big[\big(\hat{\theta} - \mathbb{E}[\hat{\theta}]\big)^2\Big]}$$

Same formula as a standard deviation, applied to the estimate instead of to individual observations. In practice it is unknown and estimated from the data, written $\widehat{\mathrm{SE}}$.

Derivation for the mean. Let $X_1, \ldots, X_n$ be independent with variance σ^2. Two variance rules from chapter 2 do all the work: variances of independent terms add, and scaling by a multiplies variance by a^2.

$$\mathrm{Var}(\bar{X}) = \mathrm{Var}\left(\tfrac{1}{n}\sum_{i=1}^{n} X_i\right) = \tfrac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(X_i) = \tfrac{1}{n^2}\cdot n\sigma^2 = \frac{\sigma^2}{n} \qquad \Longrightarrow \qquad \mathrm{SE}(\bar{X}) =$$

The $1/n^2$ from averaging beats the n from adding, leaving $1/n$ for the variance and $1/\sqrt{n}$ for the standard error. Independence is used in the second step; drop it and covariance terms appear.

Four quantities that get confused

Quantity	Symbol	Describes	Units	As n grows
Population variance	σ^2	How much individual values vary in the process	Squared	Fixed: a property of the world
Sample variance	s^2	Your estimate of σ^2	Squared	Settles toward σ^2
Variance of the mean	σ^2/n	How much the average would vary across repeats	Squared	Shrinks like $1/n$
Standard error of the mean	$\sigma/\sqrt{n}$	The same, in original units	Original	Shrinks like $1/\sqrt{n}$

So the difference between variance and standard error is not just a square root. Variance (and SD) describe the *data*; standard error describes an *estimate*, and it depends on three things the data's variance does not: the sample size, the estimator, and the dependence between observations.

Standard errors you will actually use

Estimate	Standard error	Note
Mean	$s/\sqrt{n}$	The workhorse
Pass rate $\hat{p}$	$\sqrt{\hat{p}(1-\hat{p})/n}$	124 of 200 passing: 0.0343, so about ±6.7 points at 95%. Use Wilson near 0 or 1 (eval statistics)
Difference, independent samples	$\sqrt{s_1^2/n_1 + s_2^2/n_2}$	Variances add, so the difference is noisier than either part
Difference, paired on the same items	$s_d/\sqrt{n}$, with s_d the SD of per-item differences	Usually far smaller, because shared item difficulty cancels
Median (normal data, large n)	$\approx 1.2533\,\sigma/\sqrt{n}$	The price of robustness: 25% wider than the mean's
Mean of clustered data	$\frac{s}{\sqrt{n}}\sqrt{1+(m-1)\rho}$	m items per cluster, intra-cluster correlation ρ (clustering)
Anything else	SD of bootstrap replicates	When no formula exists (bootstrap)

python

```python
import math

def se_mean(sd, n):
    return sd / math.sqrt(n)

def se_proportion(p, n):
    return math.sqrt(p * (1 - p) / n)

def se_diff_independent(s1, n1, s2, n2):
    # variances add for independent estimates; standard errors do not
    return math.sqrt(s1 ** 2 / n1 + s2 ** 2 / n2)

def se_clustered(se_naive, m, icc):
    # design effect: m items per cluster, intra-cluster correlation icc
    return se_naive * math.sqrt(1 + (m - 1) * icc)

print(round(se_mean(12.0, 400), 3))                        # 0.6 seconds
print(round(se_proportion(124 / 200, 200), 4))             # 0.0343 -> about +/- 6.7 points at 95%
print(round(se_diff_independent(0.49, 200, 0.48, 200), 4)) # 0.0485 two models on different task sets
print(round(se_clustered(0.0343, 12, 0.6), 4))             # 0.0946 same pass rate, 12 calls per session
```

rust

```rust
fn se_mean(sd: f64, n: f64) -> f64 {
    sd / n.sqrt()
}

fn se_proportion(p: f64, n: f64) -> f64 {
    (p * (1.0 - p) / n).sqrt()
}

fn se_diff_independent(s1: f64, n1: f64, s2: f64, n2: f64) -> f64 {
    // variances add for independent estimates; standard errors do not
    (s1.powi(2) / n1 + s2.powi(2) / n2).sqrt()
}

fn se_clustered(se_naive: f64, m: f64, icc: f64) -> f64 {
    // design effect: m items per cluster, intra-cluster correlation icc
    se_naive * (1.0 + (m - 1.0) * icc).sqrt()
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

fn main() {
    println!("{}", round(se_mean(12.0, 400.0), 3));                      // 0.6 seconds
    println!("{}", round(se_proportion(124.0 / 200.0, 200.0), 4));       // 0.0343 -> about +/- 6.7 points at
95%
    println!("{}", round(se_diff_independent(0.49, 200.0, 0.48, 200.0), 4)); // 0.0485 two models on different
task sets
    println!("{}", round(se_clustered(0.0343, 12.0, 0.6), 4));           // 0.0946 same pass rate, 12 calls per
session
}
```

The square-root law, simulated

$$\mathrm{SE} = \frac{s}{\sqrt{n}} \qquad \mathrm{SE} \propto n^{-1/2} \;\Rightarrow\; 4n \text{ gives half the error bar}$$

python

```python
import numpy as np

rng = np.random.default_rng(0)
population_sd = 12.0

for n in (25, 100, 400, 1600):
    sample = rng.normal(60, population_sd, n)

    sd = sample.std(ddof=1)          # spread of the MEASUREMENTS
    se = sd / np.sqrt(n)             # spread of the ESTIMATE of their mean
```

```
    print(n, "sd:", round(sd, 2), " se:", round(se, 3),
          " 95% CI half width:", round(1.96 * se, 3))

# n=25   sd 10.47  se 2.095   CI half width 4.106
# n=100  sd 11.68  se 1.168   CI half width 2.29
# n=400  sd 12.35  se 0.617   CI half width 1.21
# n=1600 sd 11.97  se 0.299   CI half width 0.587
#
# The sd column does not move. More data does not make users more alike.
# The se column halves every time n quadruples. That is the whole story.
```

rust

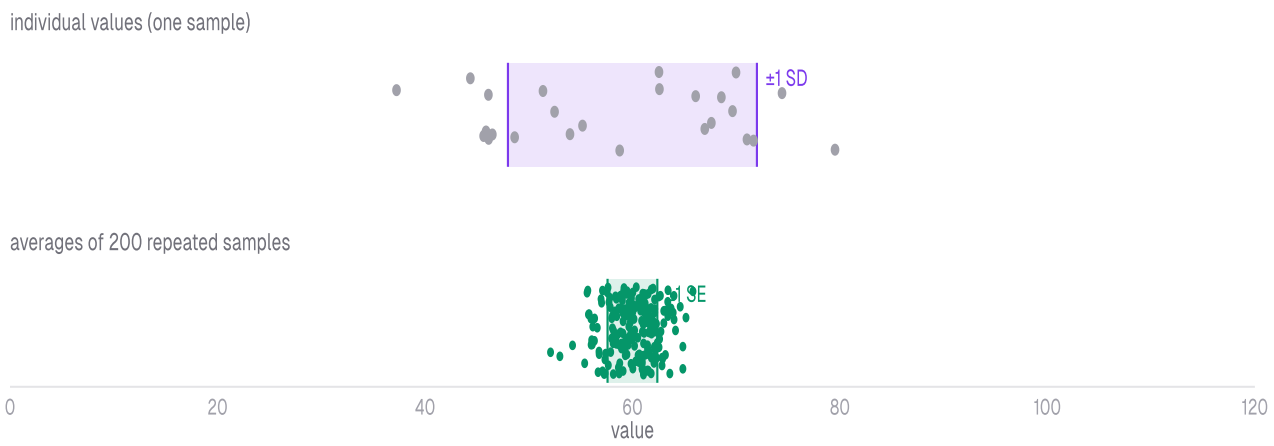
```
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

/// Sample standard deviation (ddof = 1).
fn sample_sd(x: &[f64]) -> f64 {
    let n = x.len() as f64;
    let m = x.iter().sum::<f64>() / n;
    (x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / (n - 1.0)).sqrt()
}

fn main() {
    let mut rng = Rng::new(0);
    let population_sd = 12.0;

    for n in [25, 100, 400, 1600] {
        let sample: Vec<f64> = (0..n).map(|_| rng.normal(60.0, population_sd)).collect();

        let sd = sample_sd(&sample);    // spread of the MEASUREMENTS
```

```
        let se = sd / (n as f64).sqrt(); // spread of the ESTIMATE of their mean

        println!("{} sd: {}  se: {}  95% CI half width: {}",
                 n, round(sd, 2), round(se, 3), round(1.96 * se, 3));
    }

    // SplitMix64 draws, so the numbers differ from numpy's; the pattern is the same:
    // n=25   sd 11.56  se 2.313  CI half width 4.533
    // n=100  sd 11.12  se 1.112  CI half width 2.179
    // n=400  sd 11.85  se 0.592  CI half width 1.161
    // n=1600 sd 12.02  se 0.3    CI half width 0.589
    //
    // The sd column does not move. More data does not make users more alike.
    // The se column halves every time n quadruples. That is the whole story.
}
```

Interactive

Standard deviation vs standard error (shown at its default setting)

Top: one sample of n values. Their spread is the SD. Bottom: the averages of 200 repeated samples of the same size. Their spread is the SE. Raise n and watch only the bottom row tighten.



SD of the sample	SE by formula σ/√n	SD of the 200 averages	Variance of the mean σ²/n
11.67	**2.400**	**2.329**	**5.760**

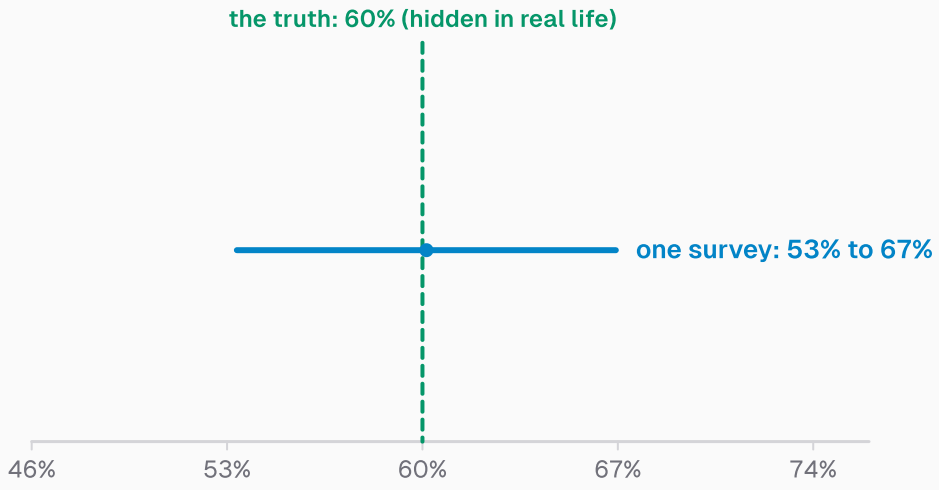
With n = 25, the values still spread about **11.7** around their mean, close to σ = 12. Their average is pinned to within about **±4.70** (95%). Quadruple n to 100 and that window halves to ±2.35; the SD does not move.

What a confidence interval actually means

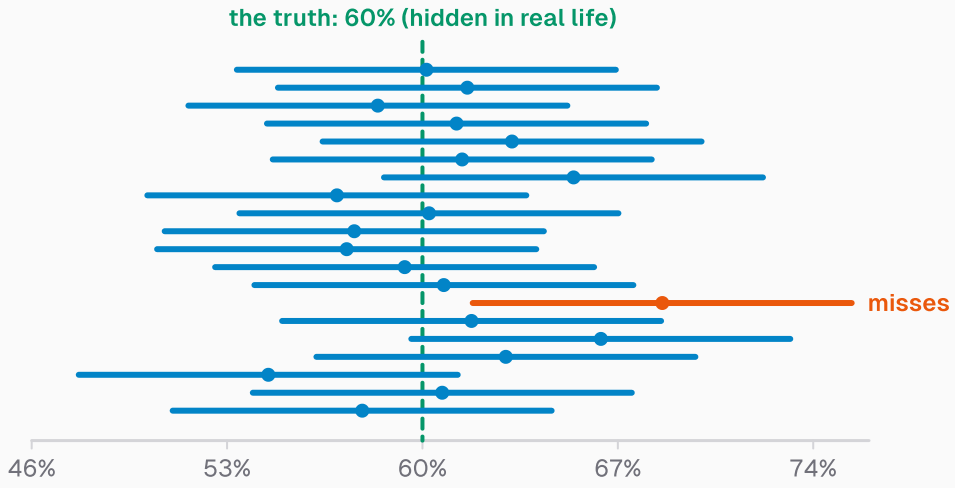
Diagram

What "95% confident" really means

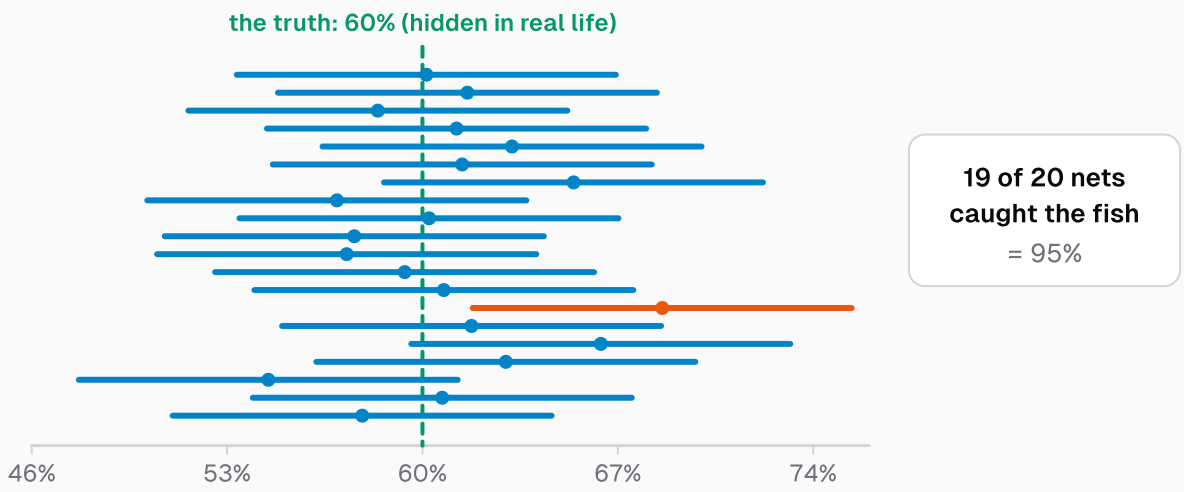
Twenty surveys, twenty nets thrown at a fish you cannot see.



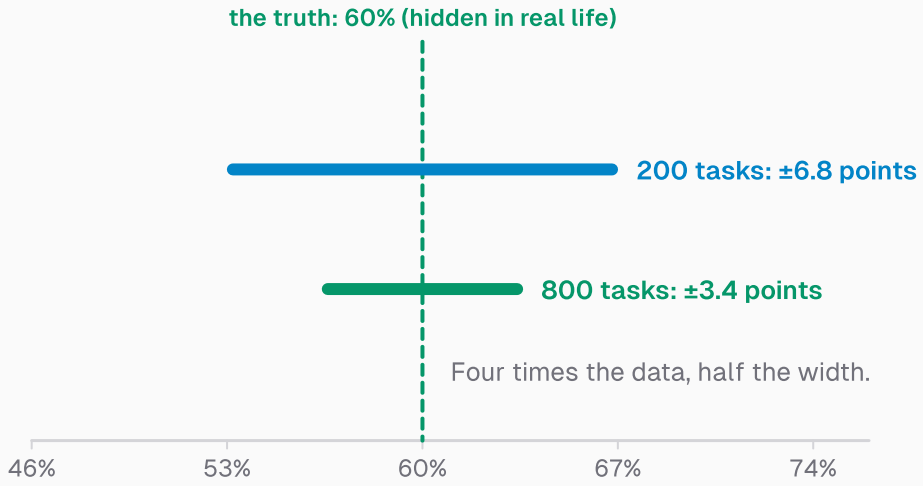
The true pass rate is 60%, but you never get to see it. You run one survey of 200 tasks and build an interval around your result. Think of the interval as a net thrown at a hidden fish.



Now repeat the whole survey 20 times with fresh samples. Each one throws its own net. Most land across the truth. One, in red, misses.



That is all "95%" means: the **method** catches the truth in about 19 of 20 throws. It does not mean a 95% chance the truth is inside *your* particular net; that net either caught it or did not.



To make the net smaller without missing more often, you need more data. Quadrupling the tasks halves the width, the same square-root law as the polls.

☆ **In plain words**

A 95% confidence interval is produced by a procedure that captures the true value 95 times out of 100. It is **not** a 95% chance the truth is inside the one interval in front of you. The truth is a fixed number; the interval is the thing that moves from experiment to experiment.

Under the hood

The textbook interval and the Wilson interval

$$\bar{x} \pm 1.96 \cdot \frac{s}{\sqrt{n}} \qquad \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

Left: the textbook interval for a mean. Right: Wilson, for a proportion $\hat{p} = k/n$. Use Wilson for pass rates: the textbook version breaks near 0 and 1, and can hand you an upper bound above 100 percent.

python

```python
import math

def wilson(k: int, n: int, z: float = 1.96):
    """Interval for a pass rate. k passed out of n."""
    if n == 0:
        return (0.0, 1.0)
    p = k / n
    den = 1 + z * z / n                       # shrink factor, -> 1 as n grows

    # The centre is NOT p. It is pulled toward 0.5 by z^2/(2n), and that pull
    # is what keeps the interval inside [0, 1] at extreme rates.
    centre = (p + z * z / (2 * n)) / den

    half = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / den
    return (max(0.0, centre - half), min(1.0, centre + half))


def naive(k, n, z=1.96):
    p = k / n
    h = z * math.sqrt(p * (1 - p) / n)
    return (p - h, p + h)                     # note: no clamping

print(wilson(49, 50))    # (0.8950, 0.9965)  stays inside [0, 1]
print(naive(49, 50))     # (0.9412, 1.0188)  claims a rate above 100%
```

rust

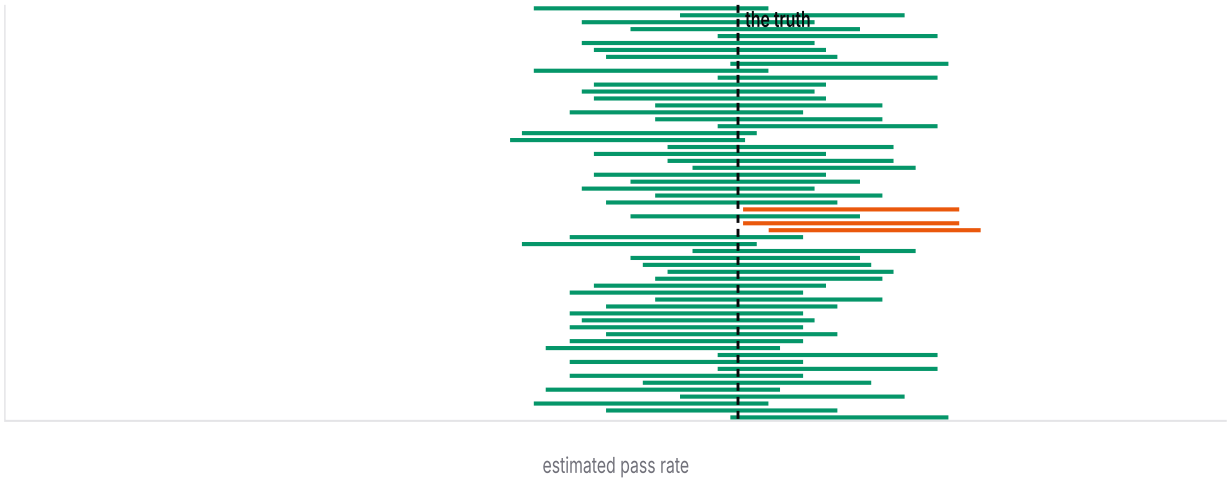
```rust
/// Interval for a pass rate. k passed out of n.
fn wilson(k: u32, n: u32, z: f64) -> (f64, f64) {
    if n == 0 {
        return (0.0, 1.0);
    }
    let (k, n) = (k as f64, n as f64);
    let p = k / n;
    let den = 1.0 + z * z / n; // shrink factor, -> 1 as n grows

    // The centre is NOT p. It is pulled toward 0.5 by z^2/(2n), and that pull
    // is what keeps the interval inside [0, 1] at extreme rates.
    let centre = (p + z * z / (2.0 * n)) / den;
```

```rust
    let half = z * (p * (1.0 - p) / n + z * z / (4.0 * n * n)).sqrt() / den;
    ((centre - half).max(0.0), (centre + half).min(1.0))
}

fn naive(k: u32, n: u32, z: f64) -> (f64, f64) {
    let p = k as f64 / n as f64;
    let h = z * (p * (1.0 - p) / n as f64).sqrt();
    (p - h, p + h) // note: no clamping
}

fn main() {
    println!("{:?}", wilson(49, 50, 1.96)); // (0.8950, 0.9965)  stays inside [0, 1]
    println!("{:?}", naive(49, 50, 1.96));  // (0.9412, 1.0188)  claims a rate above 100%
}
```

Interactive

Sixty experiments, sixty intervals (shown at its default setting)

Each bar is one experiment on the same underlying truth. Green bars caught it, orange missed. The truth never moves; the bars do. Shrink n and watch the bars get wider but the hit rate stay near 95%.



57 of 60 intervals (95%) contain the true rate. Each orange bar is an experiment that got unlucky.

This is what "95% confident" actually means: the PROCEDURE captures the truth 95 times in 100. It is not a 95% chance that the truth is inside the one interval you happen to be looking at. The truth is fixed; the interval is what moves.

What a p value is, and the four things it is not

A p value answers exactly one question: **if there were genuinely no difference, how often would I see a result at least this extreme by chance alone?** Small p means your result would be surprising in a world with no real effect.

$$p = P\left(T \geq t_{\text{obs}} \mid H_0 \text{ true}\right)$$

The probability of a result at least this extreme, assuming there is no real effect. Note which side of the bar the assumption sits on.

- It is **not** the probability that the null hypothesis is true.
- It is **not** the probability your result was a fluke.
- It is **not** a measure of effect size. A tiny, useless difference gets a tiny p value with enough data.
- p = 0.06 is **not** "no effect". It is slightly weaker evidence than p = 0.04. The 0.05 line is a convention, not a law of nature.

Key idea

Always report the effect size next to the p value. "Model B is better, p = 0.003" is nearly useless on its own. "Model B passes 4.2 points more, 95% CI [1.1, 7.3], p = 0.003" tells someone whether to ship.

Under the hood

A useless effect becomes "significant" with enough data

python

```python
import numpy as np
from scipy import stats

rng = np.random.default_rng(0)

# A tiny, useless difference. 0.5 percentage points.
def experiment(n):
    a = rng.random(n) < 0.605
    b = rng.random(n) < 0.600
    return stats.fisher_exact([[a.sum(), n - a.sum()],
                               [b.sum(), n - b.sum()]]).pvalue

for n in (200, 2_000, 20_000, 200_000):
    print(n, round(experiment(n), 4))

# The effect never changes. Only n does. With enough data a difference
```

```
# nobody could notice becomes "statistically significant".
# THIS is why a p value alone tells you nothing about whether to ship.
```

rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}

/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

fn ln_choose(n: u64, k: u64) -> f64 {
    ln_gamma(n as f64 + 1.0) - ln_gamma(k as f64 + 1.0) - ln_gamma((n - k) as f64 + 1.0)
}

/// Two-sided Fisher exact test on [[a, b], [c, d]]: add up every table with the
/// same margins that is no more likely than the one observed.
fn fisher_exact(t: [[u64; 2]; 2]) -> f64 {
    let [[a, b], [c, d]] = t;
    let (row1, col1, n) = (a + b, a + c, a + b + c + d);
    let ln_pmf = |x: u64| ln_choose(row1, x) + ln_choose(n - row1, col1 - x) - ln_choose(n, col1);
    let cutoff = ln_pmf(a) + 1e-7_f64.ln_1p(); // small tolerance for rounding
    let lo = col1.saturating_sub(n - row1);
    let hi = row1.min(col1);
    let p: f64 = (lo..=hi).map(ln_pmf).filter(|&l| l <= cutoff).map(f64::exp).sum();
    p.min(1.0)
}
```

```rust
fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

// A tiny, useless difference. 0.5 percentage points.
fn experiment(rng: &mut Rng, n: u64) -> f64 {
    let a = (0..n).filter(|_| rng.random() < 0.605).count() as u64;
    let b = (0..n).filter(|_| rng.random() < 0.600).count() as u64;
    fisher_exact([[a, n - a], [b, n - b]])
}

fn main() {
    let mut rng = Rng::new(0);
    for n in [200, 2_000, 20_000, 200_000] {
        println!("{} {}", n, round(experiment(&mut rng, n), 4));
    }
    // 200 0.2518 | 2000 0.3487 | 20000 0.0521 | 200000 0.0029  (SplitMix64 draws;
    // numpy's give 0.6869, 0.1832, 0.2365, 0.0001). Small samples wander at random;
    // at 200,000 per arm a 0.5 point gap is about 3 standard errors, so p < 0.05
    // is the typical outcome.

    // The effect never changes. Only n does. With enough data a difference
    // nobody could notice becomes "statistically significant".
    // THIS is why a p value alone tells you nothing about whether to ship.
}
```

For leaders

What these two ideas let you demand

The sampling distribution is the reason an eval number is never just a number. It comes with a margin, and that margin decides whether a result can carry a decision.

- **No interval, no decision.** A pass rate without a confidence interval cannot support a ship call. On 200 tasks at around 60%, the 95% interval is roughly ±7 points. Two models 3 points apart on that suite are usually indistinguishable unless the comparison is paired (Part II covers how pairing shrinks the margin).
- **Budget with the square-root law.** Halving the error bar costs four times the eval compute and four times the labelling. Detecting a 1 point change typically needs thousands of tasks. Put that in planning documents before someone promises "we'll know by Friday".
- **Separate "real" from "matters".** A p value says whether a difference is likely real. It says nothing about whether it is big enough to care about. Ask for the effect size and its interval, and agree in advance on the smallest difference worth acting on.
- **Watch for SD and SE swaps.** Error bars drawn with the standard deviation look alarmingly wide; error bars drawn with the standard error look reassuringly tight. Both are legitimate for different questions, and dashboards frequently mix them up.

Principal asks

"If we reran this eval on a fresh draw of tasks, how much would the headline move?" A team that can answer with a number, and knows where it came from, understands its own result. A team that cannot has a point estimate and a hope.

Part I · Chapter 4

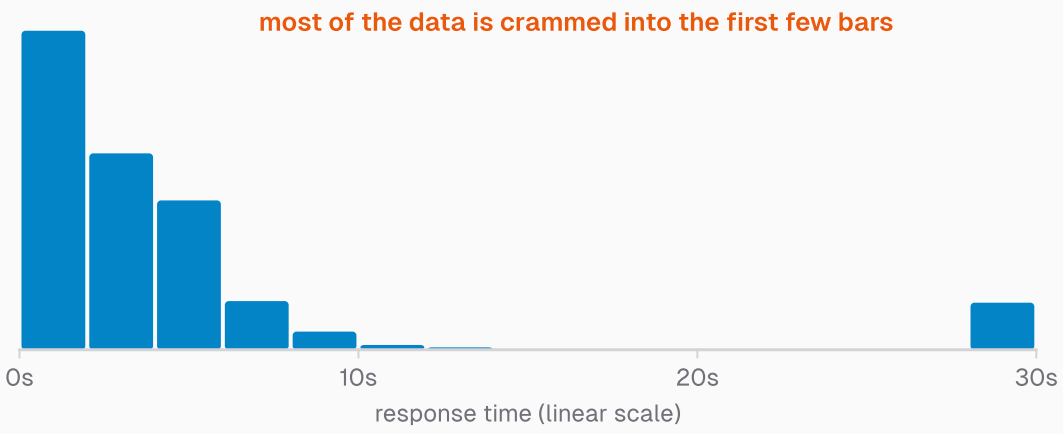
Reading a shape: work out what you have before choosing a method

This is the practical skill: look at data and work out what you have. The checklist takes about two minutes and catches nearly everything that would otherwise invalidate the analysis that follows.

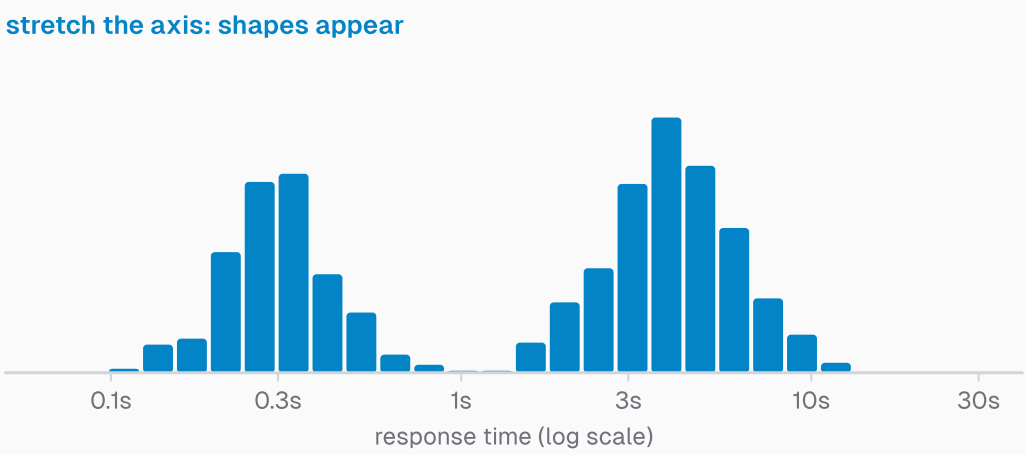
Diagram

A two-minute checkup for a chatbot's response times

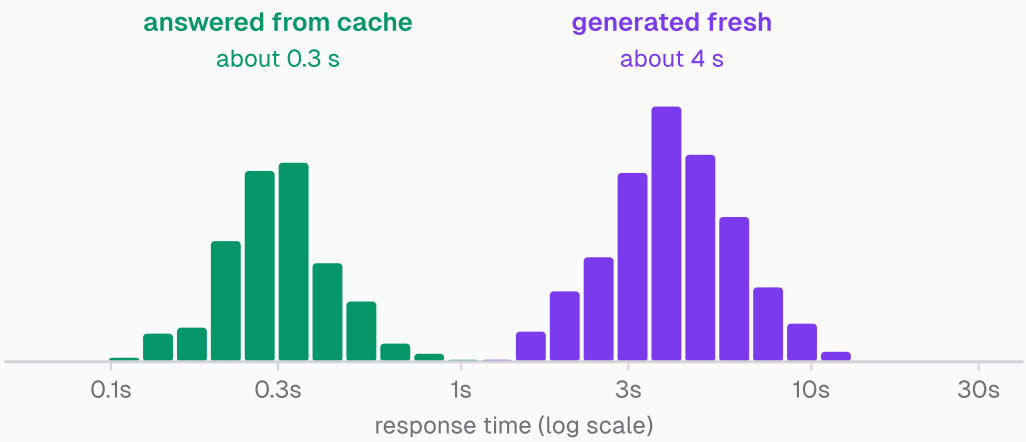
Four looks that tell you what kind of data you have before you summarise it.



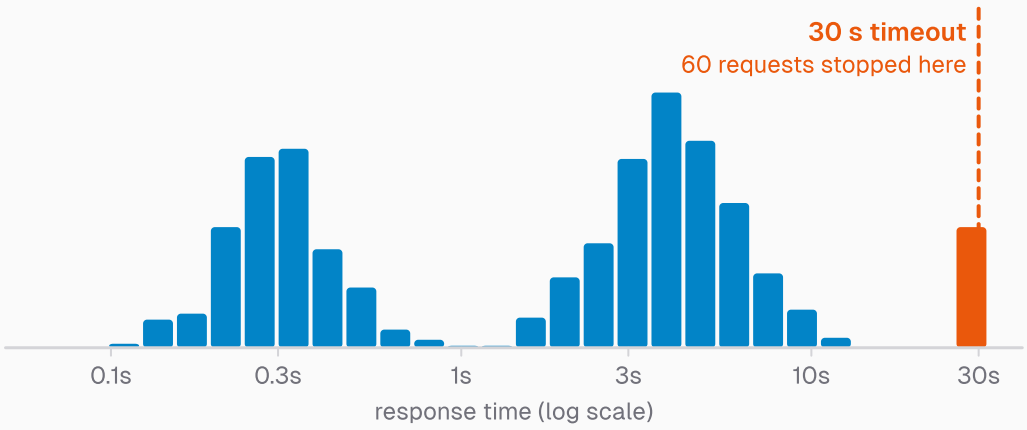
A thousand response times on a normal axis. A few tall bars on the left and a long empty stretch. You cannot see anything useful, which is itself a clue: the data is heavily skewed.



Put time on a log scale, where each tick is about three times the last. Skewed timing data usually turns into tidy bumps. If it looks like a bell on a log axis, it is roughly lognormal.



Here there are two bumps: answers served from a cache in about 0.3 seconds, and answers generated fresh in about 4 seconds. Two humps mean two populations. Summarise each separately, or the average describes nobody.



Last look: a spike at a wall. The 60 requests that hit the 30 second timeout were recorded as exactly 30. The real tail is longer than the data shows. That is **censoring**.

Plot it
see the shape first

Log it
skew becomes a bell

Count humps
two groups? split

Find walls
caps hide the tail

Two minutes of looking, before any statistics.

The checkup in four moves. Each one changes which summary and which test are honest, which is what the rest of this chapter and the lookup table are for.

Plain words

Look before you measure

A doctor looks at a patient before prescribing. A summary statistic is a prescription. Before you choose one, look at the data: draw a picture, check whether it leans, check whether there are two groups hiding in it, and check whether something chopped off the top. Most mistakes come from skipping the look.

The two-minute checklist

1. **Plot a histogram.** Always. Before any number. Summary statistics hide bimodality, censoring and outliers with total reliability.
2. **Divide mean by median.** Above 1.2 is right skewed, below 0.85 is left skewed.
3. **Divide variance by mean** for counts. Near 1 means Poisson is plausible. Well above 1 means overdispersed.
4. **Check whether values are whole numbers**, and whether they are bounded (at 0, at 1, at n).
5. **Try a log scale.** If the shape turns symmetric, you have something lognormal.
6. **Look for a spike at a round number.** That is censoring, not data.
7. **Look for two humps.** That is two populations.

$$\frac{s^2}{\bar{x}} \approx 1 \Rightarrow \text{Poisson;} \qquad \frac{s^2}{\bar{x}} \gg 1 \Rightarrow \text{overdispersed}$$

The dispersion check for count data: variance divided by mean.

Under the hood

The checklist as a function

python

```python
import numpy as np
from scipy import stats

def diagnose(x):
    """Run this before choosing any method. Takes two seconds, catches most errors."""
    x = np.asarray(x, dtype=float)
    n = len(x)
    m, med = x.mean(), np.median(x)
    var = x.var(ddof=1)

    out = {
        "n": n,
        "mean": m,
        "median": med,
        "skew_ratio": m / med if med else float("inf"),   # >1.2 = right skewed
        "var_over_mean": var / m if m else float("inf"),  # ~1 = Poisson-like
        "whole_numbers": bool(np.all(np.abs(x - np.round(x)) < 1e-9)),
        "min": x.min(), "max": x.max(),
        "zeros": int((x == 0).sum()),                     # blocks a log transform
    }

    # Does a log transform fix the skew? Only meaningful for positive data.
    if x.min() > 0:
        out["skew_raw"] = float(stats.skew(x))
        out["skew_logged"] = float(stats.skew(np.log(x)))
```

```
        out["log_helps"] = abs(out["skew_logged"]) < abs(out["skew_raw"]) / 2

    # CENSORING CHECK: is there a pile-up at a single value near the max?
    top = x[x >= np.quantile(x, 0.98)]
    if len(top):
        common = stats.mode(np.round(top, 6), keepdims=False)
        out["censoring_suspect"] = bool(common.count > 0.01 * n)
        out["suspect_value"] = float(common.mode)

    return out


lat = stats.lognorm.rvs(0.9, scale=3, size=4000, random_state=0)
for k, v in diagnose(lat).items():
    print(k, v)
# skew_ratio 1.49 -> right skewed, do not report the mean
# log_helps True  -> it is lognormal, analyse on the log scale
```

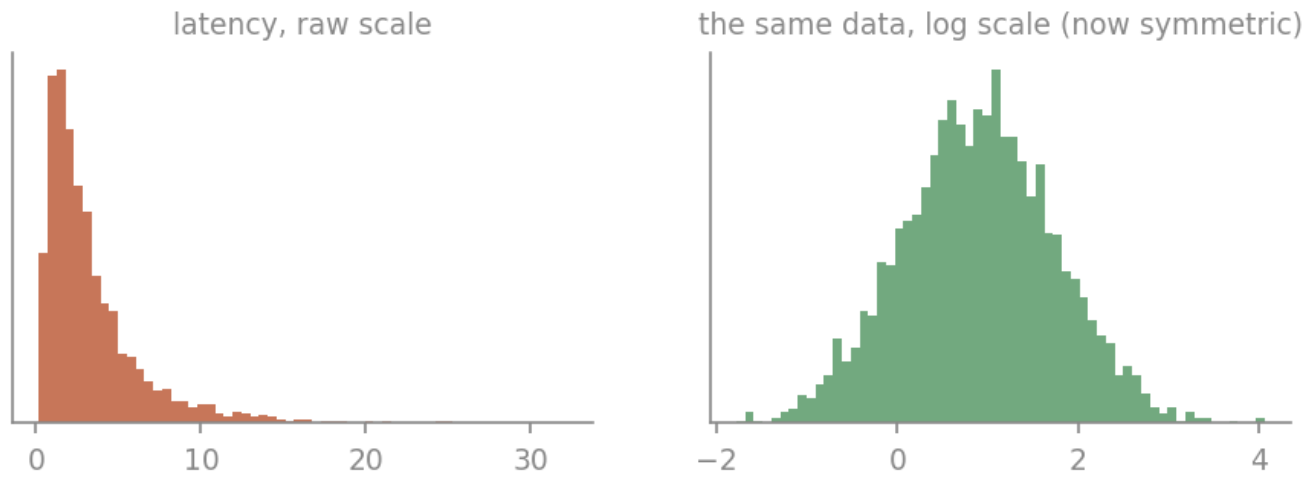
rust

```
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

fn sorted(x: &[f64]) -> Vec<f64> {
    let mut s = x.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));
    s
}

/// Linear-interpolation quantile (numpy's default method).
fn quantile(x: &[f64], q: f64) -> f64 {
    let s = sorted(x);
```

```rust
    let h = (s.len() - 1) as f64 * q;
    let lo = h.floor() as usize;
    let hi = (lo + 1).min(s.len() - 1);
    let (a, b, t) = (s[lo], s[hi], h - lo as f64);
    if t >= 0.5 { b - (b - a) * (1.0 - t) } else { a + (b - a) * t }
}

/// Sample skewness, as scipy.stats.skew computes it: m3 / m2^1.5.
fn skew(x: &[f64]) -> f64 {
    let m = mean(x);
    let m2 = x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / x.len() as f64;
    let m3 = x.iter().map(|v| (v - m).powi(3)).sum::<f64>() / x.len() as f64;
    m3 / m2.powf(1.5)
}

/// Run this before choosing any method. Takes two seconds, catches most errors.
fn diagnose(x: &[f64]) -> Vec<(&'static str, String)> {
    let n = x.len();
    let (m, med) = (mean(x), quantile(x, 0.5));
    let var = x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / (n as f64 - 1.0);
    let min = x.iter().cloned().fold(f64::INFINITY, f64::min);
    let max = x.iter().cloned().fold(f64::NEG_INFINITY, f64::max);

    let mut out: Vec<(&'static str, String)> = vec![
        ("n", n.to_string()),
        ("mean", m.to_string()),
        ("median", med.to_string()),
        ("skew_ratio", (if med != 0.0 { m / med } else { f64::INFINITY }).to_string()), // >1.2 = right skewed
        ("var_over_mean", (if m != 0.0 { var / m } else { f64::INFINITY }).to_string()), // ~1 = Poisson-like
        ("whole_numbers", x.iter().all(|v| (v - v.round()).abs() < 1e-9).to_string()),
        ("min", min.to_string()),
        ("max", max.to_string()),
        ("zeros", x.iter().filter(|&&v| v == 0.0).count().to_string()), // blocks a log transform
    ];

    // Does a log transform fix the skew? Only meaningful for positive data.
    if min > 0.0 {
        let logs: Vec<f64> = x.iter().map(|v| v.ln()).collect();
        let (skew_raw, skew_logged) = (skew(x), skew(&logs));
        out.push(("skew_raw", skew_raw.to_string()));
        out.push(("skew_logged", skew_logged.to_string()));
        out.push(("log_helps", (skew_logged.abs() < skew_raw.abs() / 2.0).to_string()));
    }

    // CENSORING CHECK: is there a pile-up at a single value near the max?
    let q98 = quantile(x, 0.98);
    let top: Vec<f64> = x.iter().filter(|&&v| v >= q98).map(|v| (v * 1e6).round() / 1e6).collect();
    if !top.is_empty() {
        // the most common value; ties go to the smallest, as scipy.stats.mode does
        let top = sorted(&top);
        let (mut mode, mut count) = (top[0], 0);
        for run in top.chunk_by(|a, b| a == b) {
            if run.len() > count {
                (mode, count) = (run[0], run.len());
            }
        }
        out.push(("censoring_suspect", (count as f64 > 0.01 * n as f64).to_string()));
        out.push(("suspect_value", mode.to_string()));
    }
```

```
    out
}

fn main() {
    // lognormal(sigma 0.9, median 3): exp of a normal
    let mut rng = Rng::new(0);
    let lat: Vec<f64> = (0..4000).map(|_| 3.0 * rng.normal(0.0, 0.9).exp()).collect();
    for (k, v) in diagnose(&lat) {
        println!("{} {}", k, v);
    }
    // skew_ratio 1.47 -> right skewed, do not report the mean
    // log_helps true -> it is lognormal, analyse on the log scale
}
```

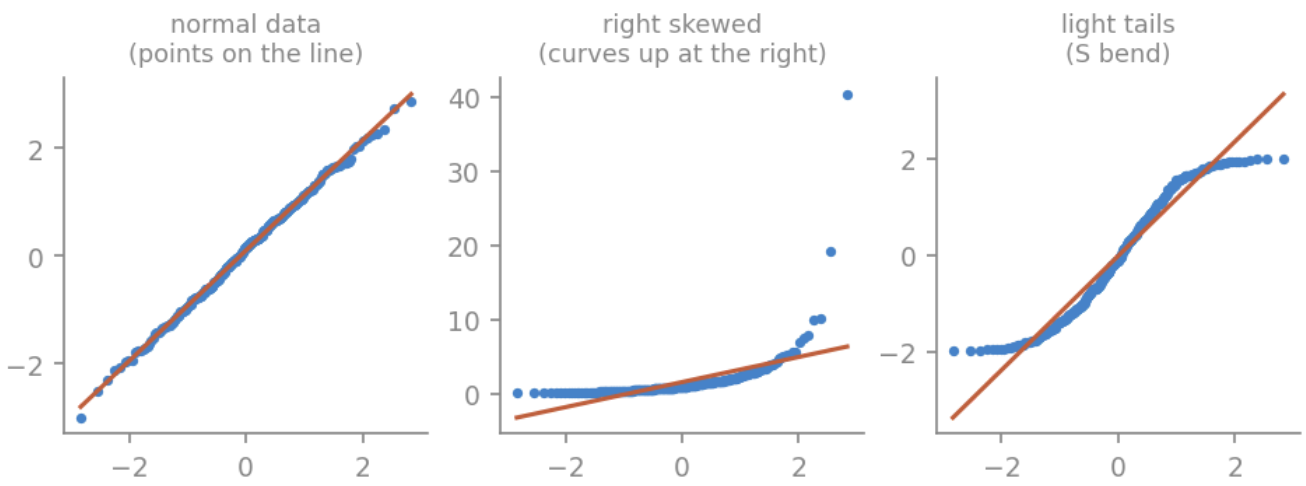
The log scale check



Same data, two scales. If the log version looks symmetric, the raw version is lognormal, and the log scale is where the analysis should happen.

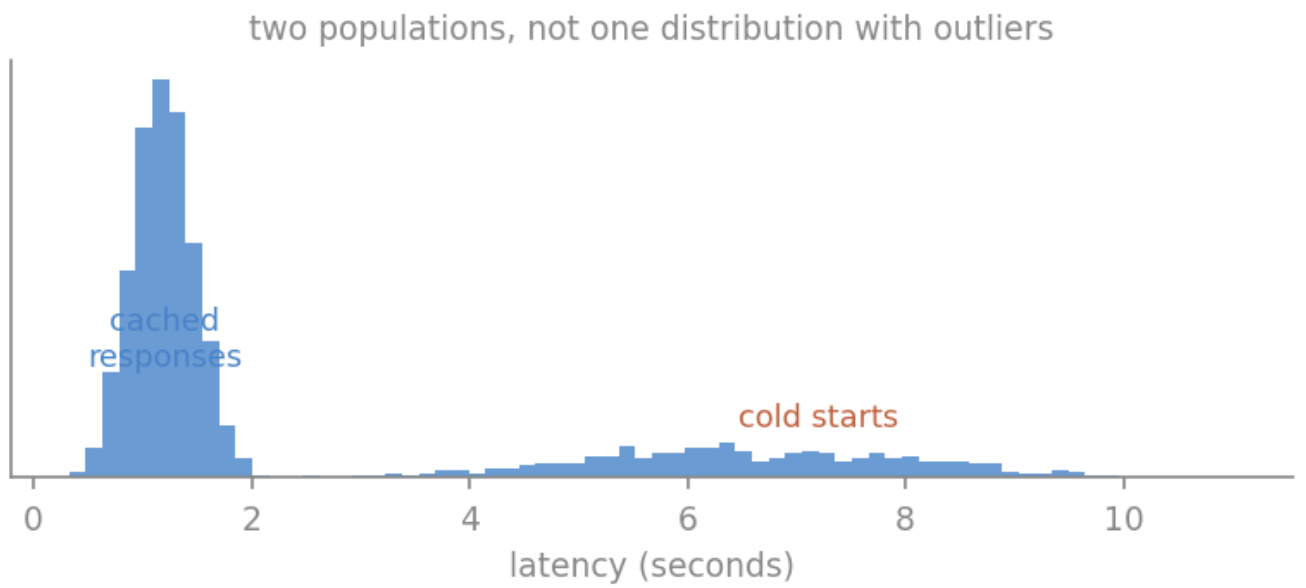
QQ plots, once you want more than eyeballing

A QQ plot compares your data's quantiles against a reference distribution's. A straight line means a good match. The way it bends tells you how it differs.



Left: normal data sits on the line. Middle: right skew curves up at the right end. Right: light tails make an S bend.

Two humps means two populations



This is not one distribution with outliers. It is cached responses and cold starts sharing an axis.

Eval lens

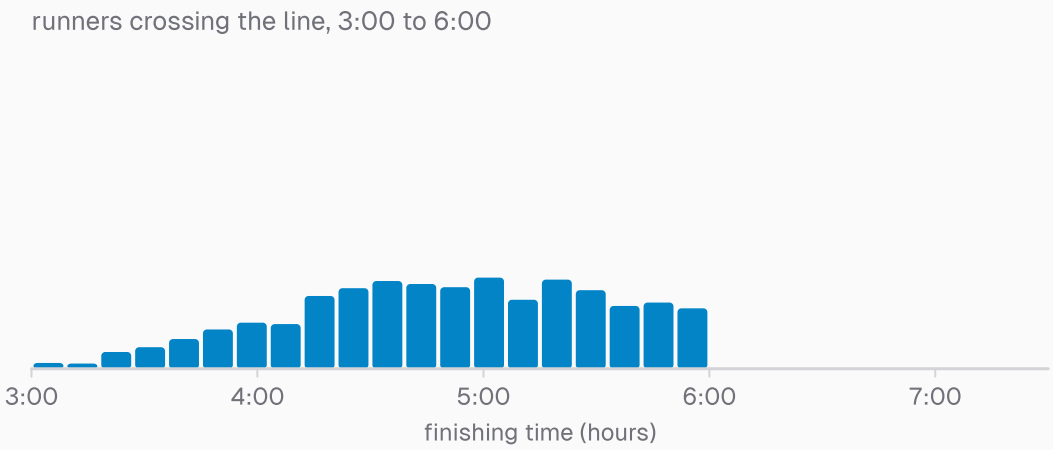
When you see this, **stop reaching for statistics and go find the segmenting variable**. Split by cache hit, retry status, repo size, task category. Two clean distributions beat one messy one every time, and the split is usually the actual finding.

Censoring: the one that hides

Diagram

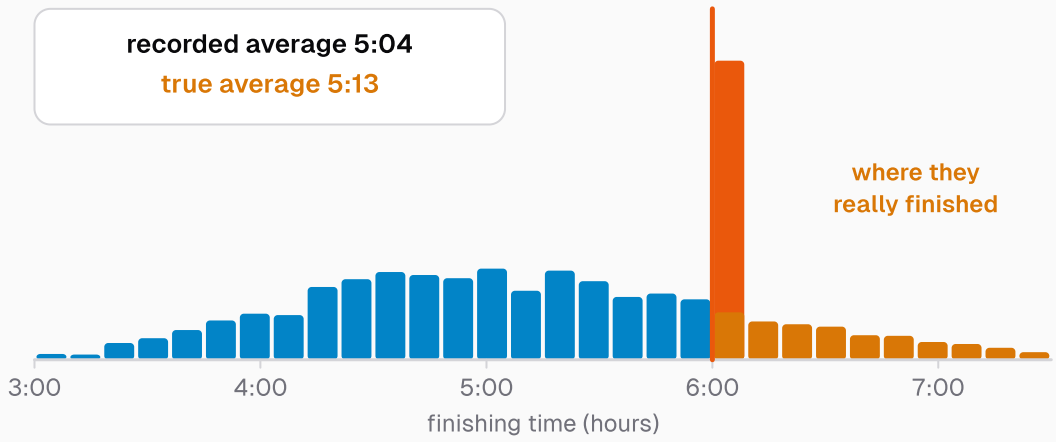
The marathon that closed at six hours

Censoring: when the data stops before the story does.

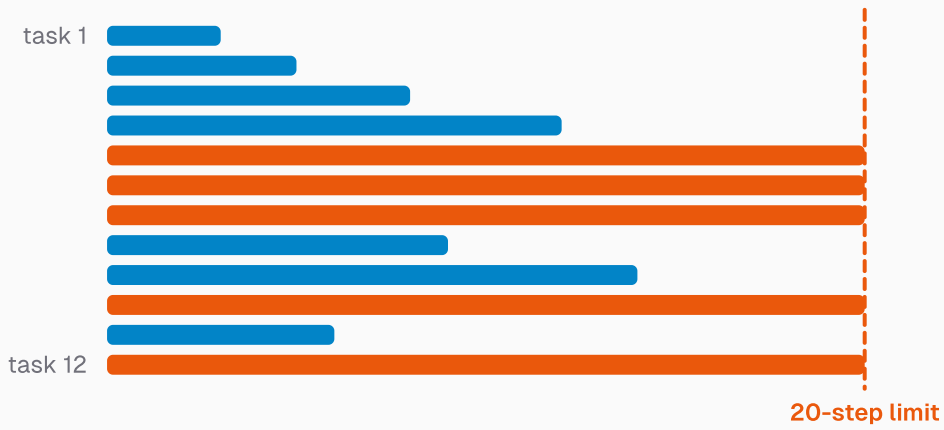


Finishing times for 3,000 marathon runners. Most finish around five hours. So far, an ordinary bell-shaped pile.

But the course closes at 6:00. The 21% of runners still out there get written down as "6:00". All of them pile into one spike at the wall.

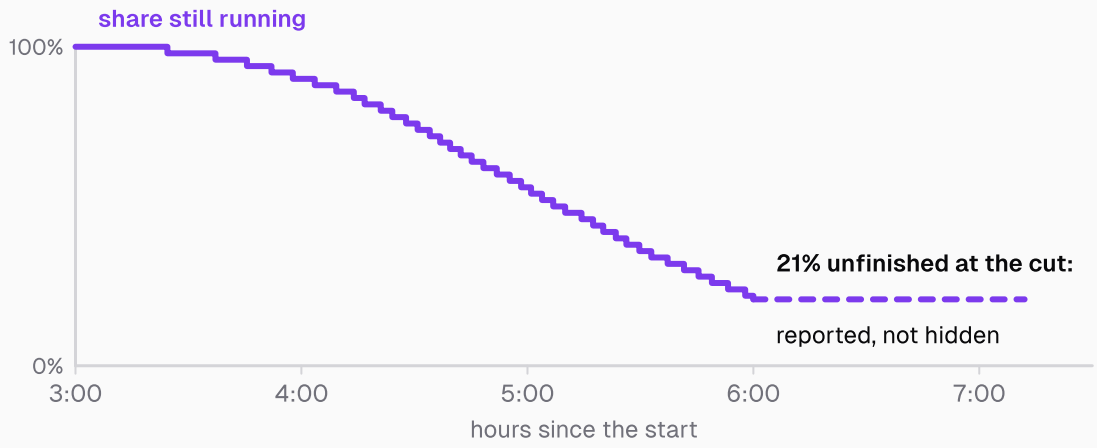


Their real times were later, so the recorded average (5:04) is too optimistic; the true average is 5:13. Dropping them would be worse: you would be averaging only the faster runners.

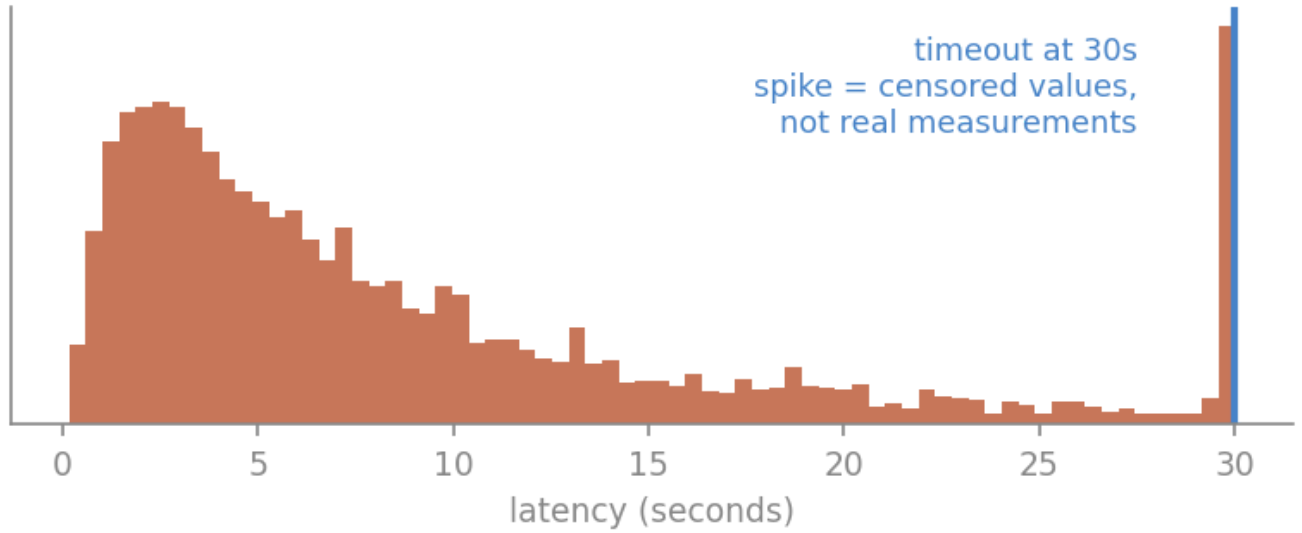


steps an agent took per task: 5 of 12 hit the limit and were cut off

Agent evals have the same wall. A task stopped at a step limit or timeout is recorded at the cap. Average "steps to solve" then looks better than reality, and the hardest tasks vanish into the spike.



The fix: say how many hit the wall, and summarise with a survival curve (Kaplan-Meier) that treats "still running at the cut" as information, not as a finish time.



A spike sitting exactly at your timeout is not a measurement. Those requests were 30 seconds or more and you will never know which.

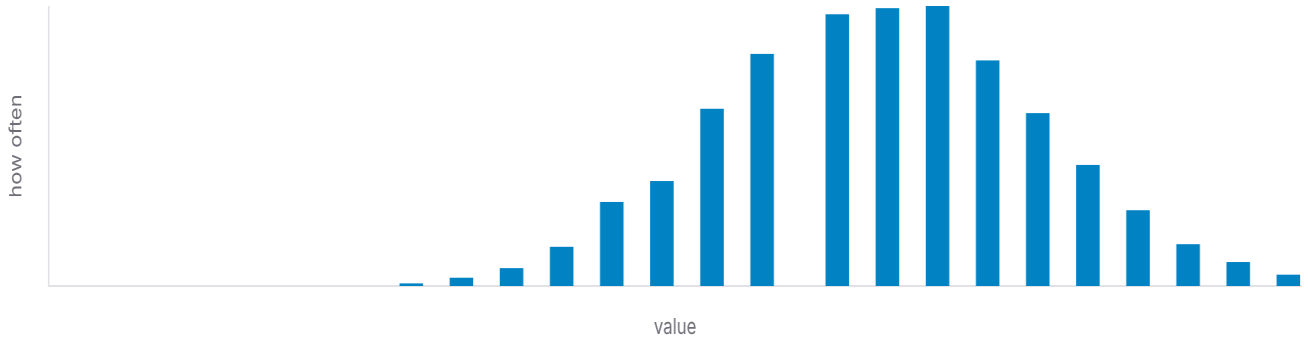
Watch out

Timeouts do not create outliers, they create **censored data**. Every value at the cap is really "at least this much". Your mean, p95 and p99 are all biased downward by an amount you cannot see from the data. Report the censoring rate next to any percentile, or use survival methods. Check for this before reaching for any outlier technique.

Interactive

Name that distribution (shown at its default setting)

A random sample with its diagnostics. Work through the checklist above, make a guess, and get the reasoning either way. Do ten of these and the skill sticks.



- Lognormal
- Normal
- Poisson
- Negative binomial
- Exponential
- Binomial
- Mixture (two populations)
- Uniform
- Pareto (power law)
- Geometric

Diagnostics: mean 16.07 median 16.00 mean/median 1.00 variance/mean 0.58 min 7.00 max 27.00 whole numbers: yes

For leaders

Make "show me the histogram" a habit

The cheapest review step in evaluation is asking to see the raw distribution before anyone argues about the summary. It surfaces three failure modes that no amount of downstream rigour can fix.

Hidden segments

Two humps mean two populations. A change that helps one and hurts the other can net to zero on average. The useful question becomes "which segment moved?"

Silent censoring

A spike at a timeout or token cap means the worst cases were truncated. A model that times out more often can look faster. Insist the censoring rate is reported beside every percentile.

Wrong scale

Latency, cost and token counts are multiplicative. Comparing them on a linear scale overweights the tail. Ratios and log-scale comparisons are usually the honest framing.

Principal asks

"What fraction of runs hit the timeout or the max-token limit, before and after?" If that fraction changed, every latency and length metric in the report is comparing different things, and the headline needs to be redone.

Part I · Chapter 5

The distribution zoo: seventeen shapes and the stories that make them

Seventeen distributions. For each one: the picture, the story that generates it, what it looks like in eval work, and when not to use it. The generating story is the most useful part. If you can match the story, you have found your distribution.

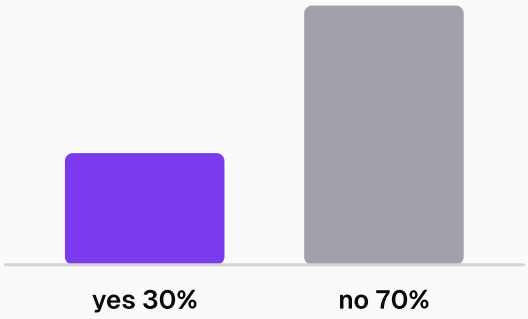
Diagram

One café, six distributions

Every shape in the zoo comes from a story about how the numbers were made. One morning at a café tells six of them.

Bernoulli

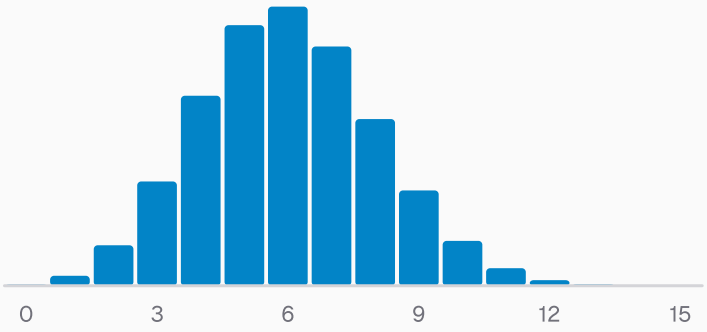
One customer:
pastry, yes or no?



One customer either buys a pastry or does not. A single yes/no with a fixed chance (here 30%) is a **Bernoulli**. In evals: one task, pass or fail.

Binomial

Out of 20 customers,
how many buy one?

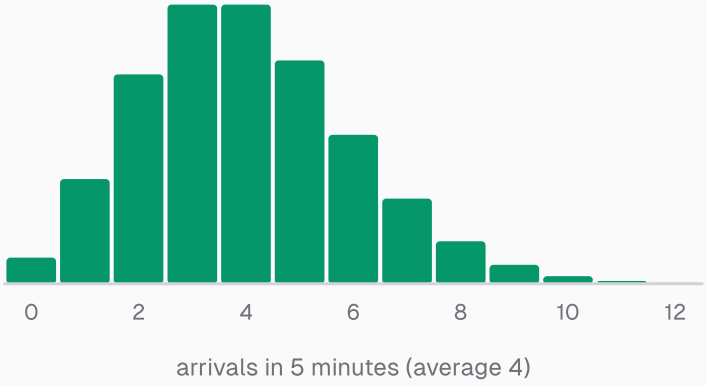


pastries sold per 20 customers (peak at 6)

Count the yeses among 20 customers and you get a **binomial**: usually around 6, sometimes 2, sometimes 11. In evals: passes out of n tasks.

Poisson

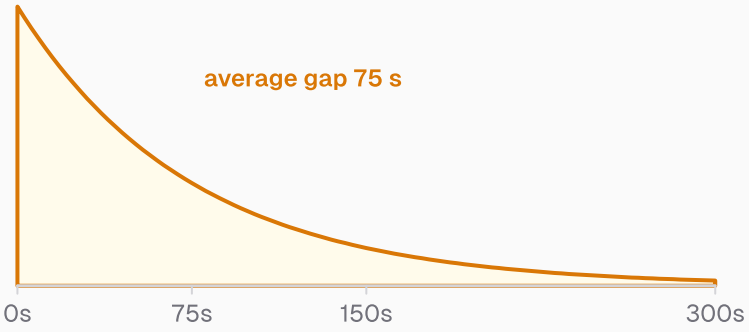
How many customers walk in per 5 minutes?



Count how many people walk in during any 5 minutes, when arrivals are independent and the average is 4: that count is **Poisson**. In evals: errors per thousand requests.

Exponential

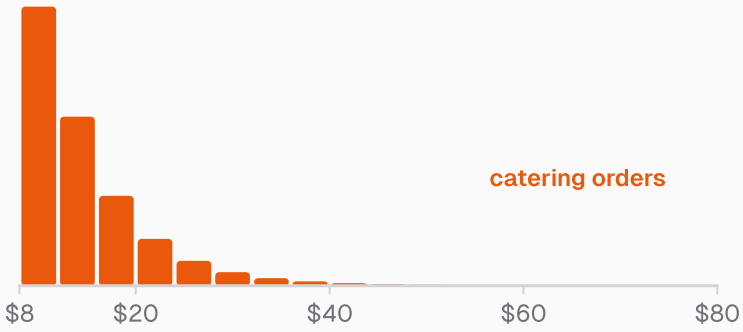
Seconds between one arrival and the next



Measure the gap between one arrival and the next: lots of short gaps, a few long ones, falling away smoothly. That is **exponential**, with an average of 75 seconds here. In evals: time between failures.

Lognormal

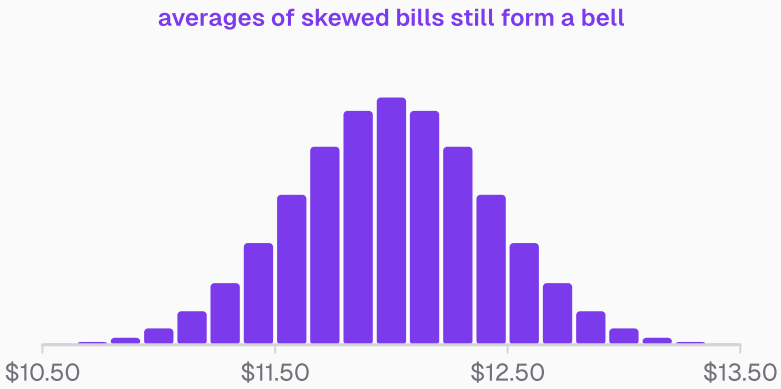
How big is each bill?
mostly $8–15, a few $70



Bill sizes multiply out of several factors (people, items, extras), which makes a right-skewed **lognormal**: mostly $8 to $15, occasionally a $70 catering order. In evals: latency and token counts.

Normal

Average bill per day,
over 100 days



Average each day's bills and plot 100 days of averages: a symmetric bell, the **normal**, even though single bills were skewed. That is the central limit theorem from the previous chapter, and why intervals on averages work.

Plain words

Match the story, find the shape

Each distribution is the fingerprint of a process. Flip one coin and you get a Bernoulli. Count heads in 200 flips and you get a binomial. Add up many small nudges and you get a bell curve. Multiply many small nudges, like a slow network times a big payload times a cold cache, and you get a lopsided

lognormal. You do not need to memorise formulas. You need to ask "what process made these numbers?" and match it to a story below.

For leaders

Why the choice of shape changes decisions

Picking a distribution sounds academic until it changes a ship decision. The common failure is assuming a simple shape when the real process is messier, which makes uncertainty look smaller than it is.

- **Assuming independence that is not there.** Treating tool calls inside one session as independent coin flips (binomial) understates uncertainty, often by a factor of two or more. Results look significant that are not.
- **Assuming Poisson for agent counts.** Tool calls, turns and retries vary with task difficulty, so variance runs several times the mean. A Poisson model will flag a routine long session as a one-in-a-million event and send engineers chasing noise.
- **Reporting a mean for a heavy tail.** For lognormal data the mean describes nobody; for a genuine power law it may not even settle as data grows. Percentiles are the defensible summary.
- **Fitting one shape to a mixture.** Two populations on one axis need to be split, not modelled.

Principal asks

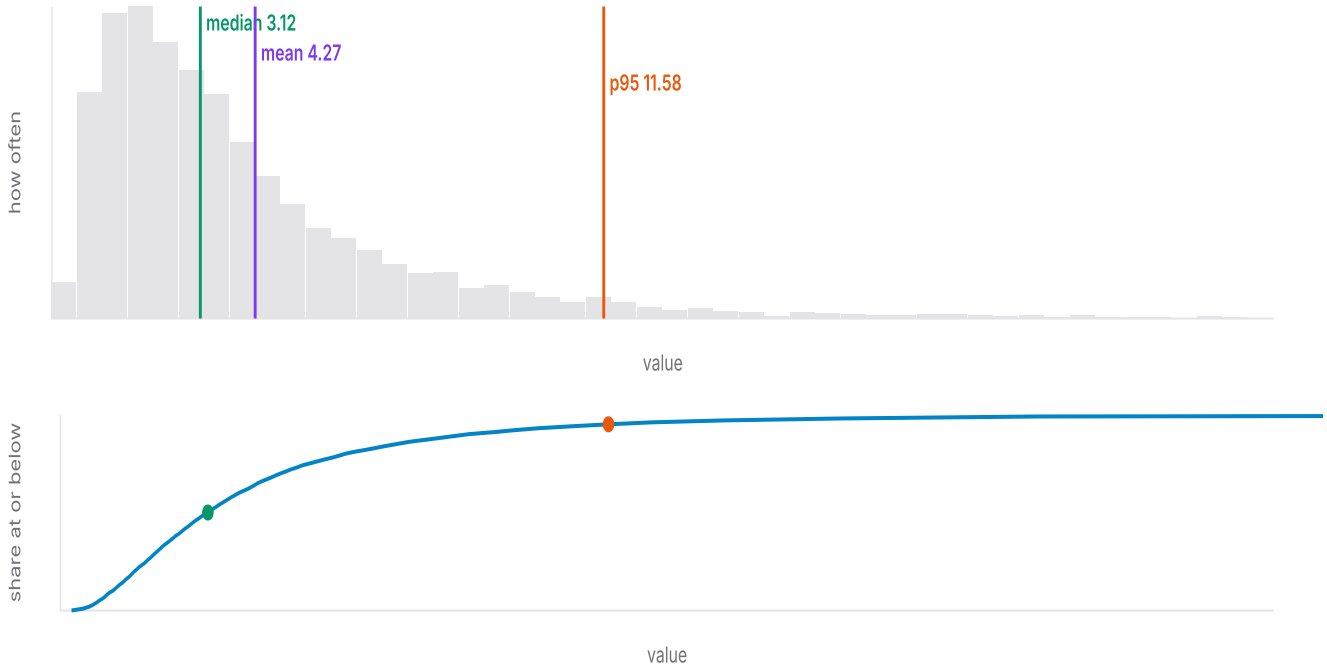
"What is the variance-to-mean ratio of that count metric, and what model did the interval assume?" If the answer is Poisson and the ratio is 4, the error bars on the dashboard are about half as wide as they should be.

Quick reference

Distribution	The story	Typical eval quantity
Bernoulli	One yes/no trial	One task result
Binomial	Successes in n independent trials	Tasks passed out of 200
Uniform	Every value equally likely	p values under a true null
Normal	Sum of many small independent effects	Averages, test statistics
Student's t	Normal mean with estimated spread	Small-sample means
Lognormal	Product of many small effects	Latency, token counts
Exponential	Waiting time, constant rate	Time between failures
Gamma	Sum of several waits	Multi-stage pipeline latency
Poisson	Events in a window, constant rate	Requests per minute
Negative binomial	Counts when the rate varies	Tool calls per session
Geometric	Attempts until first success	Retry counts
Beta	Belief about a rate	Uncertainty about a pass rate
Chi square	Sum of squared normal deviations	Test statistics
F	Ratio of two variances	ANOVA
Pareto / power law	Scale free, no typical size	Repo and file sizes
Categorical / multinomial	One draw picks one class	Intent labels, failure codes
Mixture	Several populations on one axis	Cache hits and cold starts

Interactive **Distribution explorer** (shown at its default setting)

Every distribution below, live. The top panel is the density, the bottom is the cumulative view with the median and p95 marked. Move the parameters and watch the mean detach from the median as skew grows.

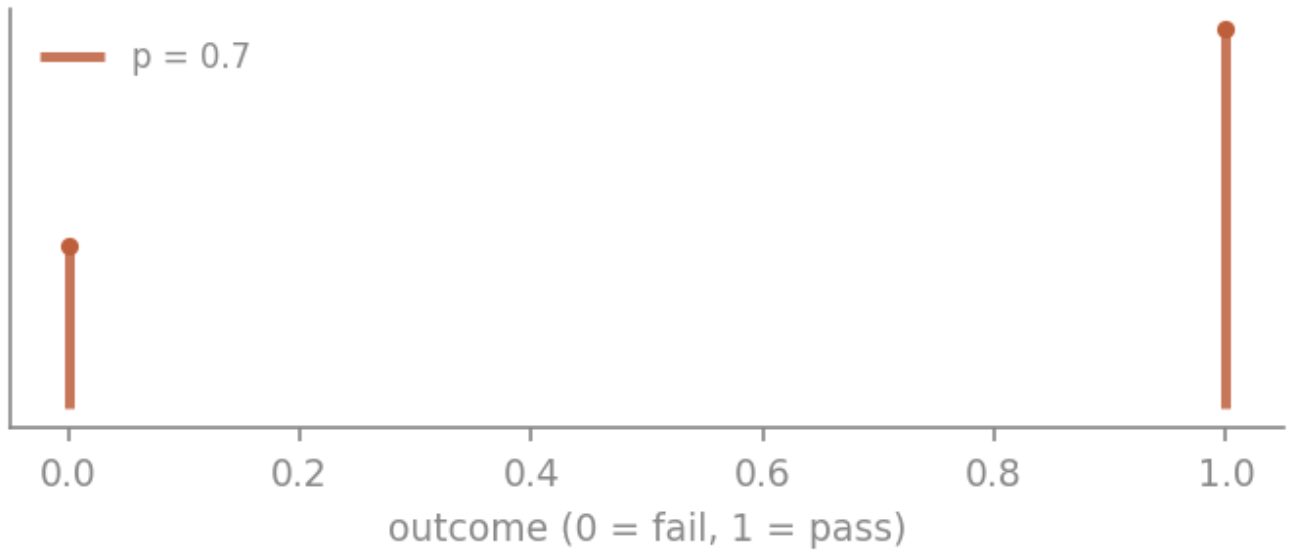


mean 4.27 **median** 3.12 **sd** 4.02 **p95** 11.58 **max** 49.22

Right skewed. mean/median = 1.37. Report the median and percentiles, not the mean.

Bernoulli

One yes/no trial.



Bernoulli: one yes/no trial.

The story	A single task either passes or fails. That is it. The building block everything else in pass/fail land is made from.
In eval work	One task result. One tool call succeeding. One thumbs up.
Use when	Use for a single binary outcome. It has one parameter, p. Its variance is p(1-p), which is largest at p = 0.5 and shrinks to nothing at the extremes, which is why rates near 0 or 1 are easier to pin down.
Avoid when	There is nothing to misuse. The mistakes come one level up, in how you aggregate them.

Under the hood

Bernoulli: formulas and code

$$P(X = k) = p^k (1-p)^{1-k}, \quad k \in \{0, 1\}$$

$$E[X] = p \qquad \mathrm{Var}(X) = p(1-p)$$

python

```python
import random

# GENERATE: one task result. This is the atom everything else is built from.
def one_task(p_pass: float, rng) -> int:
    return 1 if rng.random() < p_pass else 0        # 1 = passed, 0 = failed

rng = random.Random(0)
results = [one_task(0.7, rng) for _ in range(200)]  # 200 independent tasks

# SUMMARISE
n = len(results)
k = sum(results)                                    # how many passed
print(k / n)                                        # the pass rate, e.g. 0.695

# NOTE the variance: p(1-p) is largest at p = 0.5 and shrinks toward the edges.
# That is WHY a model at 50% needs more tasks to pin down than one at 95%.
for p in (0.5, 0.7, 0.95):
    print(p, "variance per task:", round(p * (1 - p), 4))
```

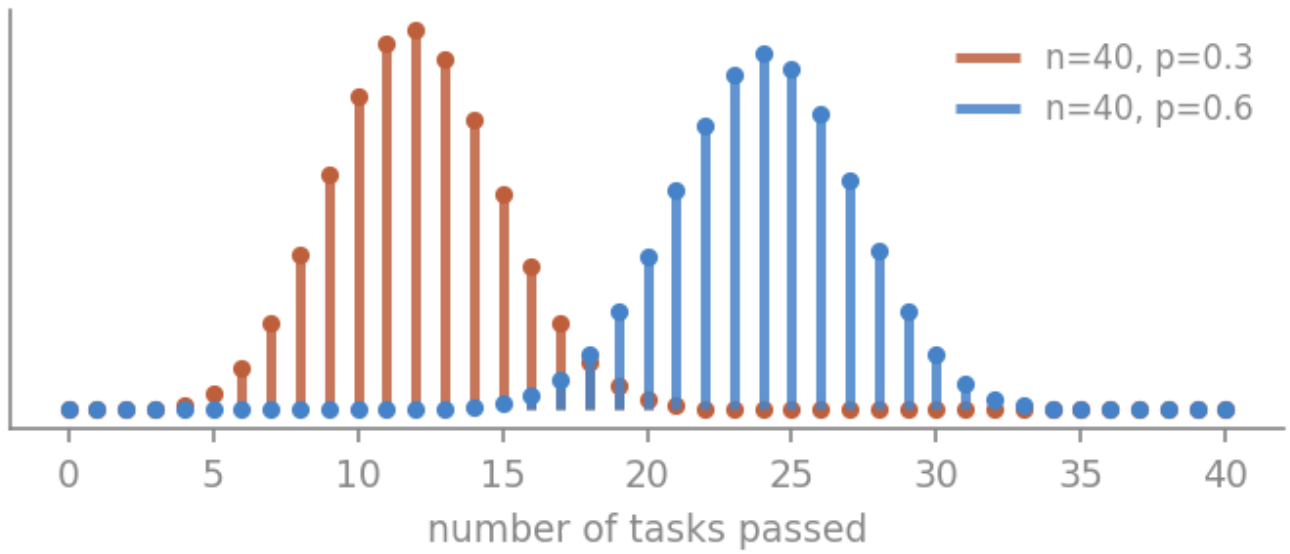
rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

// GENERATE: one task result. This is the atom everything else is built from.
fn one_task(p_pass: f64, rng: &mut Rng) -> u32 {
    if rng.random() < p_pass { 1 } else { 0 } // 1 = passed, 0 = failed
}

fn main() {
    let mut rng = Rng::new(0);
    let results: Vec<u32> = (0..200).map(|_| one_task(0.7, &mut rng)).collect(); // 200 independent tasks

    // SUMMARISE
    let n = results.len();
```

```
    let k: u32 = results.iter().sum(); // how many passed
    println!("{}", k as f64 / n as f64); // the pass rate, e.g. 0.68 (Python's draws give 0.695)

    // NOTE the variance: p(1-p) is largest at p = 0.5 and shrinks toward the edges.
    // That is WHY a model at 50% needs more tasks to pin down than one at 95%.
    for p in [0.5, 0.7, 0.95] {
        println!("{} variance per task: {}", p, round(p * (1.0 - p), 4));
    }
}
```

Binomial

Count of successes in n independent trials.



Binomial: count of successes in n independent trials.

The story	Run n tasks, each passing with probability p, count the passes. The distribution of that count is binomial.
In eval work	Number of tasks passed out of 200. Number of sessions with a successful tool call, IF sessions are independent.
Use when	Use when trials are independent and share the same p. Wilson intervals are built on this.
Avoid when	**Do not use when trials are not independent**, which is the big one. Multiple tool calls inside one session are not independent, and treating them as binomial understates your uncertainty, sometimes by a factor of two or more.

Under the hood

Binomial: formulas and code

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$E[X] = np \qquad \mathrm{Var}(X) = np(1-p)$$

python

```python
from scipy import stats
import math

# The count of passes out of n tasks IS binomial, if the tasks are independent.
n, p = 200, 0.62
print("expected passes:", n * p)                        # 124.0
print("sd of that count:", math.sqrt(n * p * (1 - p))) # 6.86

# How likely is exactly 130 passes? And 130 or more?
print(stats.binom.pmf(130, n, p))     # exactly     -> 0.0401
print(stats.binom.sf(129, n, p))      # 130 or more -> 0.2121

# The interval you should report for an observed rate.
def wilson(k, n, z=1.96):
    p_hat = k / n
    den = 1 + z * z / n
    centre = (p_hat + z * z / (2 * n)) / den
    half = z * math.sqrt(p_hat * (1 - p_hat) / n + z * z / (4 * n * n)) / den
    return max(0.0, centre - half), min(1.0, centre + half)
```

```
print(wilson(124, 200))   # (0.5511, 0.6844)

# THE TRAP: this is only valid if tasks are independent.
# 200 tool calls from 25 sessions are NOT 200 independent trials.
```

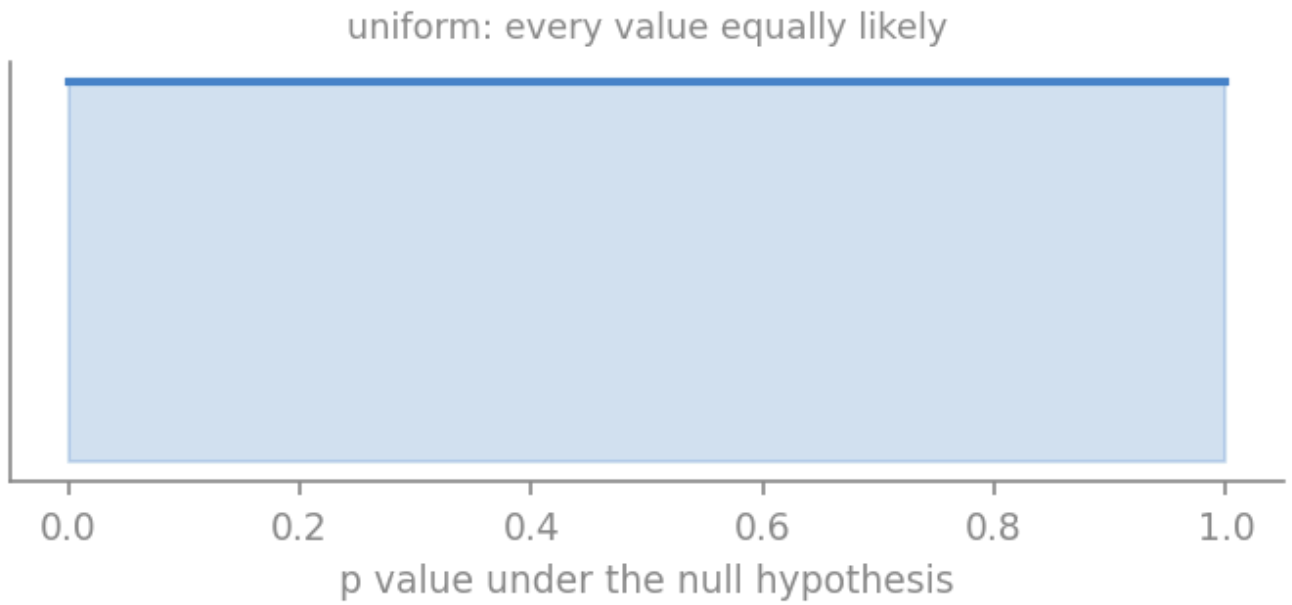
rust

```
/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

fn ln_choose(n: u64, k: u64) -> f64 {
    ln_gamma(n as f64 + 1.0) - ln_gamma(k as f64 + 1.0) - ln_gamma((n - k) as f64 + 1.0)
}

fn binom_pmf(k: u64, n: u64, p: f64) -> f64 {
    (ln_choose(n, k) + k as f64 * p.ln() + (n - k) as f64 * (1.0 - p).ln()).exp()
}

/// Survival function P(X > k): add up the upper tail.
fn binom_sf(k: u64, n: u64, p: f64) -> f64 {
    (k + 1..=n).map(|j| binom_pmf(j, n, p)).sum()
}

// The interval you should report for an observed rate.
fn wilson(k: u64, n: u64, z: f64) -> (f64, f64) {
    let (k, n) = (k as f64, n as f64);
    let p_hat = k / n;
    let den = 1.0 + z * z / n;
    let centre = (p_hat + z * z / (2.0 * n)) / den;
    let half = z * (p_hat * (1.0 - p_hat) / n + z * z / (4.0 * n * n)).sqrt() / den;
    ((centre - half).max(0.0), (centre + half).min(1.0))
}

fn main() {
    // The count of passes out of n tasks IS binomial, if the tasks are independent.
    let (n, p) = (200u64, 0.62);
    println!("expected passes: {:?}", n as f64 * p);                          // 124.0
    println!("sd of that count: {}", (n as f64 * p * (1.0 - p)).sqrt());      // 6.86

    // How likely is exactly 130 passes? And 130 or more?
    println!("{}", binom_pmf(130, n, p)); // exactly     -> 0.0401
    println!("{}", binom_sf(129, n, p));  // 130 or more -> 0.2121

    println!("{:?}", wilson(124, 200, 1.96)); // (0.5511, 0.6844)

    // THE TRAP: this is only valid if tasks are independent.
```

```
    // 200 tool calls from 25 sessions are NOT 200 independent trials.
}
```

Uniform

Every value in a range equally likely.



Uniform: every value in a range equally likely.

The story	Nothing favours any part of the range. Flat.
In eval work	P values under a true null hypothesis are uniform. That fact is what makes the multiple comparisons problem work the way it does: run 40 benchmarks with no real effect and you get 40 uniform draws, so about 2 land below 0.05.
Use when	Use as a null model, for random baselines, and for reasoning about p values.
Avoid when	Almost no real measurement is uniform. If your data looks flat, suspect a rounding or bucketing artefact.

Under the hood

Uniform: formulas and code

$$f(x) = \frac{1}{b-a}, \quad a \le x \le b$$

$$E[X] = \frac{a+b}{2} \qquad \mathrm{Var}(X) = \frac{(b-a)^2}{12}$$

python

```python
import random

# The single most useful fact about the uniform distribution in eval work:
# p values from a TRUE NULL are uniform. That is the whole multiple
# comparisons problem in one line.
rng = random.Random(1)
suite = 40                                  # benchmarks in your standing suite
p_values = [rng.random() for _ in range(suite)]   # all nulls, no real effect

false_alarms = sum(1 for p in p_values if p < 0.05)
print(false_alarms)      # 6 with this seed; 2 on average, purely by luck

# Expected number of spurious "significant" results:
print(suite * 0.05)      # 2.0
```

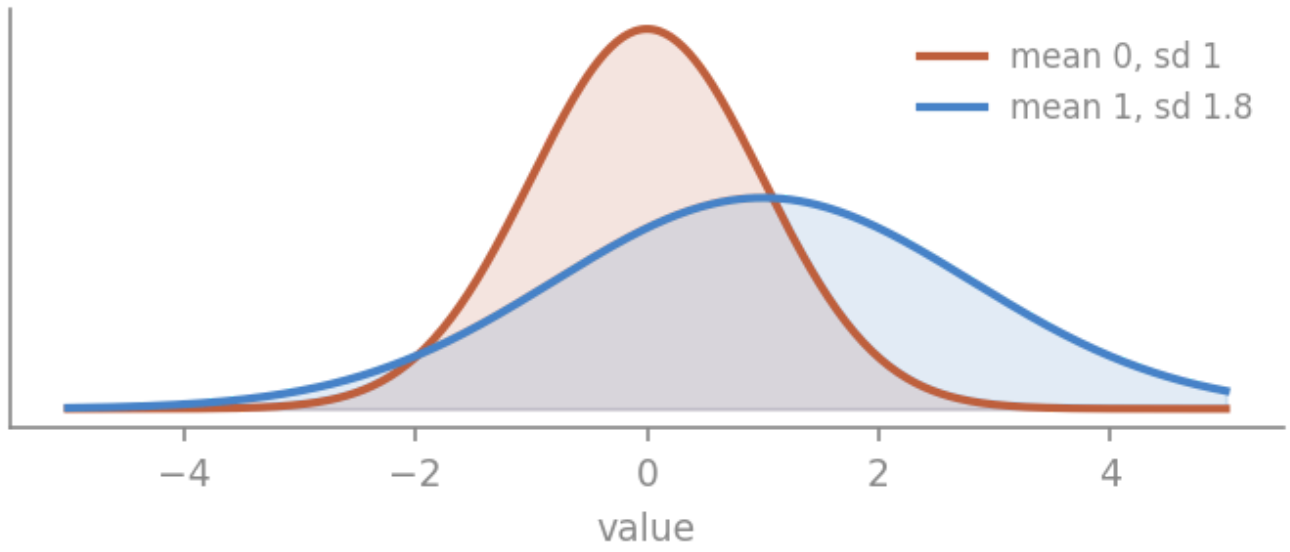
rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}

fn main() {
    // The single most useful fact about the uniform distribution in eval work:
```

```rust
    // p values from a TRUE NULL are uniform. That is the whole multiple
    // comparisons problem in one line.
    let mut rng = Rng::new(1);
    let suite = 40; // benchmarks in your standing suite
    let p_values: Vec<f64> = (0..suite).map(|_| rng.random()).collect(); // all nulls, no real effect

    let false_alarms = p_values.iter().filter(|&&p| p < 0.05).count();
    println!("{}", false_alarms);            // 2 with these draws (Python's: 6); 2 on average, purely by luck

    // Expected number of spurious "significant" results:
    println!("{:?}", suite as f64 * 0.05);   // 2.0
}
```

Normal (Gaussian)

The sum of many small independent effects.



Normal (Gaussian): the sum of many small independent effects.

The story	When lots of small independent things add up, the total is normal. That is the only story it tells, and it is why normality shows up in averages and test statistics rather than in raw measurements.
In eval work	Averages of large samples. Test statistics. Almost never your raw latency or token counts.
Use when	Use for averages, for differences of averages, and as the reference for QQ plots and z scores.
Avoid when	**Do not assume your raw data is normal.** Latency is not. Token counts are not. Counts are not. Assuming it and then using a t test on skewed data is the classic error.

Under the hood

Normal (Gaussian): formulas and code

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$E[X] = \mu \qquad \mathrm{Var}(X) = \sigma^2$$

python

```python
from scipy import stats
import numpy as np

# Raw eval data is almost never normal. AVERAGES of it get close.
latency = stats.lognorm.rvs(0.9, scale=3, size=5000, random_state=0)  # skewed
print(stats.skew(latency))                    # ~4.4, strongly right skewed

means = latency[:5000].reshape(-1, 50).mean(axis=1)   # averages of 50
print(stats.skew(means))                      # ~0.27 with this seed (theory ~0.67), far closer to symmetric

# The 68 / 95 / 99.7 rule, for genuinely normal data:
for k in (1, 2, 3):
    print(k, "sd covers", round(100 * (stats.norm.cdf(k) - stats.norm.cdf(-k)), 1), "%")

# Test normality rather than assuming it. Small p = not normal.
print(stats.shapiro(latency[:500]).pvalue)    # ~0 -> definitely not normal
print(stats.shapiro(means[:80]).pvalue)       # 0.35 with this seed, but across seeds this rejects
                                              # about half the time: closer to normal, not exactly normal
```

rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

/// Regularized incomplete gamma functions (P, Q) = (lower, upper), P + Q = 1.
fn gamma_pq(a: f64, x: f64) -> (f64, f64) {
    if x <= 0.0 {
        return (0.0, 1.0);
    }
    let front = (-x + a * x.ln() - ln_gamma(a)).exp();
    if x < a + 1.0 {
        // power series converges fast here
        let (mut ap, mut term) = (a, 1.0 / a);
        let mut sum = term;
        while term.abs() > sum.abs() * 1e-16 {
            ap += 1.0;
            term *= x / ap;
            sum += term;
```

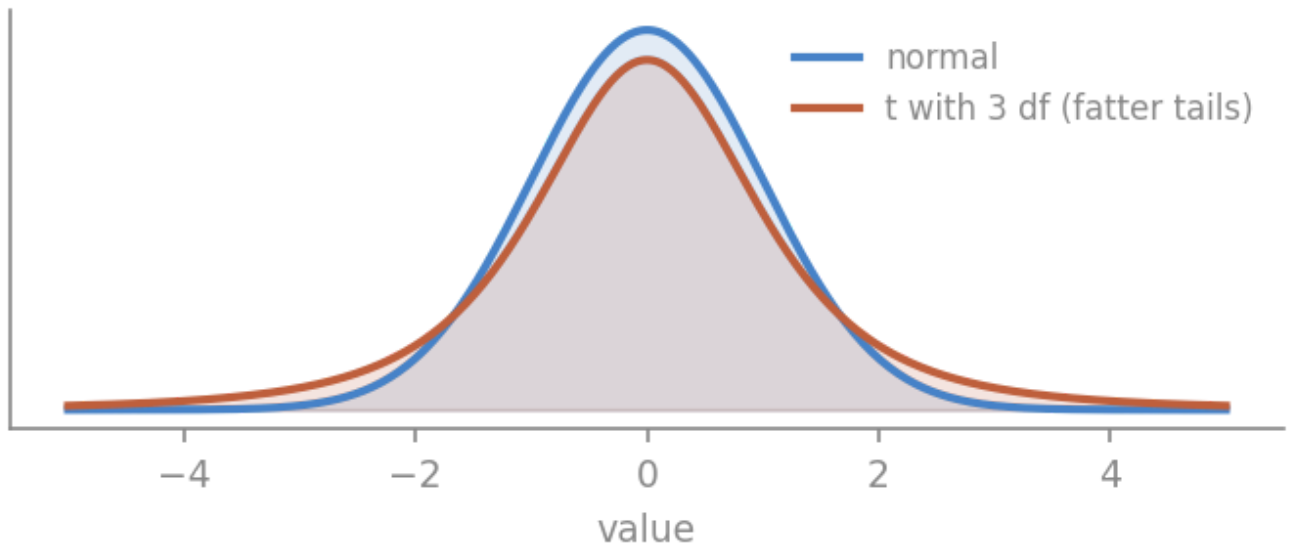
```rust
        }
        let p = sum * front;
        (p, 1.0 - p)
    } else {
        // continued fraction (modified Lentz) converges fast here
        let tiny = 1e-300;
        let (mut b, mut c) = (x + 1.0 - a, 1.0 / tiny);
        let mut d = 1.0 / b;
        let mut h = d;
        for i in 1..1000 {
            let an = -(i as f64) * (i as f64 - a);
            b += 2.0;
            d = an * d + b;
            if d.abs() < tiny { d = tiny; }
            c = b + an / c;
            if c.abs() < tiny { c = tiny; }
            d = 1.0 / d;
            h *= d * c;
            if (d * c - 1.0).abs() < 1e-16 { break; }
        }
        let q = front * h;
        (1.0 - q, q)
    }
}

/// Standard normal CDF, via the incomplete gamma function (erf(x) = P(1/2, x^2)).
fn norm_cdf(z: f64) -> f64 {
    let tail = 0.5 * gamma_pq(0.5, z * z / 2.0).1; // P(Z > |z|), accurate far out
    if z >= 0.0 { 1.0 - tail } else { tail }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// Sample skewness, as scipy.stats.skew computes it: m3 / m2^1.5.
fn skew(x: &[f64]) -> f64 {
    let m = mean(x);
    let m2 = x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / x.len() as f64;
    let m3 = x.iter().map(|v| (v - m).powi(3)).sum::<f64>() / x.len() as f64;
    m3 / m2.powf(1.5)
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

/// Inverse normal CDF: Acklam's rational approximation, polished by one Newton step.
fn norm_ppf(p: f64) -> f64 {
    let a = [-3.969683028665376e1, 2.209460984245205e2, -2.759285104469687e2,
             1.383577518672690e2, -3.066479806614716e1, 2.506628277459239];
    let b = [-5.447609879822406e1, 1.615858368580409e2, -1.556989798598866e2,
             6.680131188771972e1, -1.328068155288572e1];
    let c = [-7.784894002430293e-3, -3.223964580411365e-1, -2.400758277161838,
             -2.549732539343734, 4.374664141464968, 2.938163982698783];
    let d = [7.784695709041462e-3, 3.224671290700398e-1, 2.445134137142996, 3.754408661907416];
    let tail = |q: f64| (((((c[0] * q + c[1]) * q + c[2]) * q + c[3]) * q + c[4]) * q + c[5])
        / ((((d[0] * q + d[1]) * q + d[2]) * q + d[3]) * q + 1.0);
    let x = if p < 0.02425 {
```

```
        tail((-2.0 * p.ln()).sqrt())
    } else if p > 1.0 - 0.02425 {
        -tail((-2.0 * (1.0 - p).ln()).sqrt())
    } else {
        let q = p - 0.5;
        let r = q * q;
        (((((a[0] * r + a[1]) * r + a[2]) * r + a[3]) * r + a[4]) * r + a[5]) * q
            / (((((b[0] * r + b[1]) * r + b[2]) * r + b[3]) * r + b[4]) * r + 1.0)
    };
    let pdf = (-x * x / 2.0).exp() / (2.0 * std::f64::consts::PI).sqrt();
    x - (norm_cdf(x) - p) / pdf
}

fn poly(c: &[f64], x: f64) -> f64 {
    c.iter().rev().fold(0.0, |acc, &ci| acc * x + ci)
}

/// Shapiro-Wilk test (Royston's 1995 algorithm, as scipy uses), for n >= 12.
/// Returns the p value: small means "not normal".
fn shapiro_pvalue(x: &[f64]) -> f64 {
    let n = x.len();
    let nf = n as f64;
    let mut s = x.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));

    // expected normal order statistics, and the weights built from them
    let m: Vec<f64> = (1..=n).map(|i| norm_ppf((i as f64 - 0.375) / (nf + 0.25))).collect();
    let mm: f64 = m.iter().map(|v| v * v).sum();
    let u = 1.0 / nf.sqrt();
    let an = m[n - 1] / mm.sqrt() + poly(&[0.0, 0.221157, -0.147981, -2.071190, 4.434685, -2.706056], u);
    let an1 = m[n - 2] / mm.sqrt() + poly(&[0.0, 0.042981, -0.293762, -1.752461, 5.682633, -3.582633], u);
    let phi = (mm - 2.0 * m[n - 1].powi(2) - 2.0 * m[n - 2].powi(2))
        / (1.0 - 2.0 * an * an - 2.0 * an1 * an1);
    let mut a: Vec<f64> = m.iter().map(|v| v / phi.sqrt()).collect();
    (a[0], a[1], a[n - 2], a[n - 1]) = (-an, -an1, an1, an);

    // W = squared correlation between the sorted data and the weights
    let xbar = mean(&s);
    let sax: f64 = a.iter().zip(&s).map(|(ai, xi)| ai * (xi - xbar)).sum();
    let ssa: f64 = a.iter().map(|v| v * v).sum();
    let ssx: f64 = s.iter().map(|v| (v - xbar).powi(2)).sum();
    let w = sax * sax / (ssa * ssx);

    // ln(1 - W) is close to normal; its mean and sd depend on ln n
    let ln_n = nf.ln();
    let mu = poly(&[-1.5861, -0.31082, -0.083751, 0.0038915], ln_n);
    let sigma = poly(&[-0.4803, -0.082676, 0.0030302], ln_n).exp();
    norm_cdf(-((1.0 - w).ln() - mu) / sigma) // upper tail
}

fn main() {
    // Raw eval data is almost never normal. AVERAGES of it are.
    let mut rng = Rng::new(0);
    let latency: Vec<f64> = (0..5000).map(|_| 3.0 * rng.normal(0.0, 0.9).exp()).collect(); // skewed (lognormal)
    println!("{}", skew(&latency)); // 4.51, strongly right skewed

    let means: Vec<f64> = latency.chunks(50).map(mean).collect(); // averages of 50
    println!("{}", skew(&means));   // 0.46, much closer to symmetric

    // The 68 / 95 / 99.7 rule, for genuinely normal data:
```

```
    for k in [1.0, 2.0, 3.0] {
        println!("{} sd covers {} %", k, round(100.0 * (norm_cdf(k) - norm_cdf(-k)), 1));
    }

    // Test normality rather than assuming it. Small p = not normal.
    println!("{:e}", shapiro_pvalue(&latency[..500])); // 8.3e-35 -> definitely not normal
    println!("{}", shapiro_pvalue(&means[..80]));      // 0.025 with these draws: closer to normal, not there yet
    // (theory: a mean of 50 still has skew 4.75 / sqrt(50) = 0.67)
}
```

Student's t

A normal mean when you had to estimate the spread too.



Student's t: a normal mean when you had to estimate the spread too.

The story	Same bell shape but with fatter tails, because estimating the standard deviation from a small sample adds uncertainty. As n grows it converges on the normal.
In eval work	Small sample comparisons of means, under about 30 observations.
Use when	Use for means of small samples where the data is roughly symmetric.
Avoid when	The fatter tails do not rescue you from skew. On lognormal latency, t is still the wrong tool. Bootstrap or work on the log scale instead.

Under the hood

Student's t: formulas and code

$$f(x) \propto \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

$$E[X] = 0 \ (\nu > 1) \qquad \mathrm{Var}(X) = \frac{\nu}{\nu - 2} \ (\nu > 2)$$

python

```python
from scipy import stats

# t is what you use when you estimated the spread from the same small sample.
# Compare the critical value at 95% as the sample grows:
for df in (5, 10, 30, 100, 1000):
    print(df, round(stats.t.ppf(0.975, df), 3))
# 5 -> 2.571,  30 -> 2.042,  1000 -> 1.962.  It converges on the normal 1.96.

# A paired t test on per task score differences:
import numpy as np
rng = np.random.default_rng(0)
model_a = rng.normal(0.62, 0.20, 40)
model_b = rng.normal(0.58, 0.20, 40)
print(stats.ttest_rel(model_a, model_b))

# BUT: check the differences are roughly symmetric first. On skewed data
# this is the wrong tool no matter how fat the t tails are.
```

rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
```

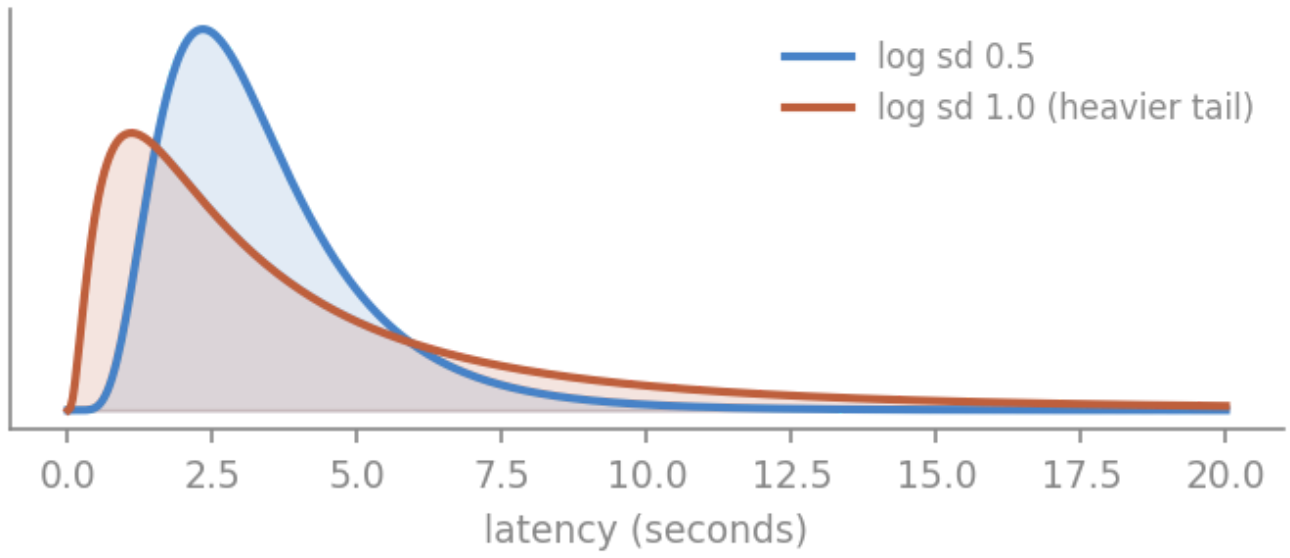
```rust
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

/// Regularized incomplete beta I_x(a, b), by continued fraction (modified Lentz).
fn beta_inc(a: f64, b: f64, x: f64) -> f64 {
    if x <= 0.0 {
        return 0.0;
    }
    if x >= 1.0 {
        return 1.0;
    }
    let front = (ln_gamma(a + b) - ln_gamma(a) - ln_gamma(b) + a * x.ln() + b * (1.0 - x).ln()).exp();
    // the fraction converges fast on one side of the mean; use symmetry for the other
    if x < (a + 1.0) / (a + b + 2.0) {
        front * beta_cf(a, b, x) / a
    } else {
        1.0 - front * beta_cf(b, a, 1.0 - x) / b
    }
}

fn beta_cf(a: f64, b: f64, x: f64) -> f64 {
```

```
    let tiny = 1e-300;
    let clamp = |v: f64| if v.abs() < tiny { tiny } else { v };
    let mut c = 1.0;
    let mut d = 1.0 / clamp(1.0 - (a + b) * x / (a + 1.0));
    let mut h = d;
    for m in 1..1000 {
        let m = m as f64;
        let even = m * (b - m) * x / ((a + 2.0 * m - 1.0) * (a + 2.0 * m));
        d = 1.0 / clamp(1.0 + even * d);
        c = clamp(1.0 + even / c);
        h *= d * c;
        let odd = -(a + m) * (a + b + m) * x / ((a + 2.0 * m) * (a + 2.0 * m + 1.0));
        d = 1.0 / clamp(1.0 + odd * d);
        c = clamp(1.0 + odd / c);
        h *= d * c;
        if (d * c - 1.0).abs() < 1e-16 {
            break;
        }
    }
    h
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

/// Student t CDF with df degrees of freedom, via the incomplete beta function.
fn t_cdf(t: f64, df: f64) -> f64 {
    let tail = 0.5 * beta_inc(df / 2.0, 0.5, df / (df + t * t)); // P(T > |t|)
    if t >= 0.0 { 1.0 - tail } else { tail }
}

/// Inverse t CDF by bisection: the CDF only goes up, so halve the bracket.
fn t_ppf(p: f64, df: f64) -> f64 {
    let (mut lo, mut hi) = (-100.0, 100.0);
    for _ in 0..200 {
        let mid = 0.5 * (lo + hi);
        if t_cdf(mid, df) < p { lo = mid } else { hi = mid }
    }
    0.5 * (lo + hi)
}

#[derive(Debug)]
struct TtestResult {
    statistic: f64,
    pvalue: f64,
    df: usize,
}

/// Paired t test: a one-sample t test on the per-task differences.
fn ttest_rel(a: &[f64], b: &[f64]) -> TtestResult {
    let d: Vec<f64> = a.iter().zip(b).map(|(x, y)| x - y).collect();
    let n = d.len() as f64;
    let m = d.iter().sum::<f64>() / n;
    let sd = (d.iter().map(|v| (v - m).powi(2)).sum::<f64>() / (n - 1.0)).sqrt();
    let t = m / (sd / n.sqrt());
    let pvalue = 2.0 * (1.0 - t_cdf(t.abs(), n - 1.0)); // two sided
    TtestResult { statistic: t, pvalue, df: d.len() - 1 }
}
```

```rust
fn main() {
    // t is what you use when you estimated the spread from the same small sample.
    // Compare the critical value at 95% as the sample grows:
    for df in [5, 10, 30, 100, 1000] {
        println!("{} {}", df, round(t_ppf(0.975, df as f64), 3));
    }
    // 5 -> 2.571,  30 -> 2.042,  1000 -> 1.962.  It converges on the normal 1.96.

    // A paired t test on per task score differences:
    let mut rng = Rng::new(0);
    let model_a: Vec<f64> = (0..40).map(|_| rng.normal(0.62, 0.20)).collect();
    let model_b: Vec<f64> = (0..40).map(|_| rng.normal(0.58, 0.20)).collect();
    println!("{:?}", ttest_rel(&model_a, &model_b));
    // with these draws: statistic 1.398, pvalue 0.170, df 39

    // BUT: check the differences are roughly symmetric first. On skewed data
    // this is the wrong tool no matter how fat the t tails are.
}
```

Lognormal

The PRODUCT of many small independent effects.



Lognormal: the PRODUCT of many small independent effects.

The story	Where normal comes from adding, lognormal comes from multiplying. Latency is multiplicative: a slow network multiplies with a big payload multiplies with a cold cache. So the log of latency is normal, and latency itself is lognormal.
In eval work	**Latency. Token counts. Session durations. Repo sizes.** This is the workhorse shape of eval engineering.
Use when	Use for anything strictly positive with a long right tail. Analyse on the log scale, report percentiles in the original units. The geometric mean is its natural centre.
Avoid when	Do not report the arithmetic mean unless you need a total. Do not run a t test on it raw. And distinguish it from a genuine power law: lognormal has a finite, well behaved mean, a power law may not.

Under the hood

Lognormal: formulas and code

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right)$$

$$E[X] = e^{\mu+\sigma^2/2} \qquad \text{median} = e^{\mu} \qquad \text{Var}(X) = \left(e^{\sigma^2} - 1\right) e^{2\mu+\sigma^2}$$

python

```python
import numpy as np
from scipy import stats

lat = stats.lognorm.rvs(0.85, scale=2.5, size=5000, random_state=3)

# WRONG: the arithmetic mean sits between the median and p95 and
# describes nobody's actual experience.
print("mean  ", lat.mean())                    # 3.48
print("median", np.median(lat))                # 2.42
print("p95   ", np.quantile(lat, 0.95))        # 9.90

# RIGHT: work on the log scale, where it is symmetric.
logs = np.log(lat)
print("log data is symmetric:", round(stats.skew(logs), 3))   # 0.037

# The geometric mean is the natural centre, and it recovers the median.
```

```
geo = np.exp(logs.mean())
print("geometric mean:", geo)                  # 2.42, matches the median closely

# Fit and read off the parameters:
shape, loc, scale = stats.lognorm.fit(lat, floc=0)
print("sigma:", shape, " median:", scale)      # 0.846, 2.42 - recovered
```

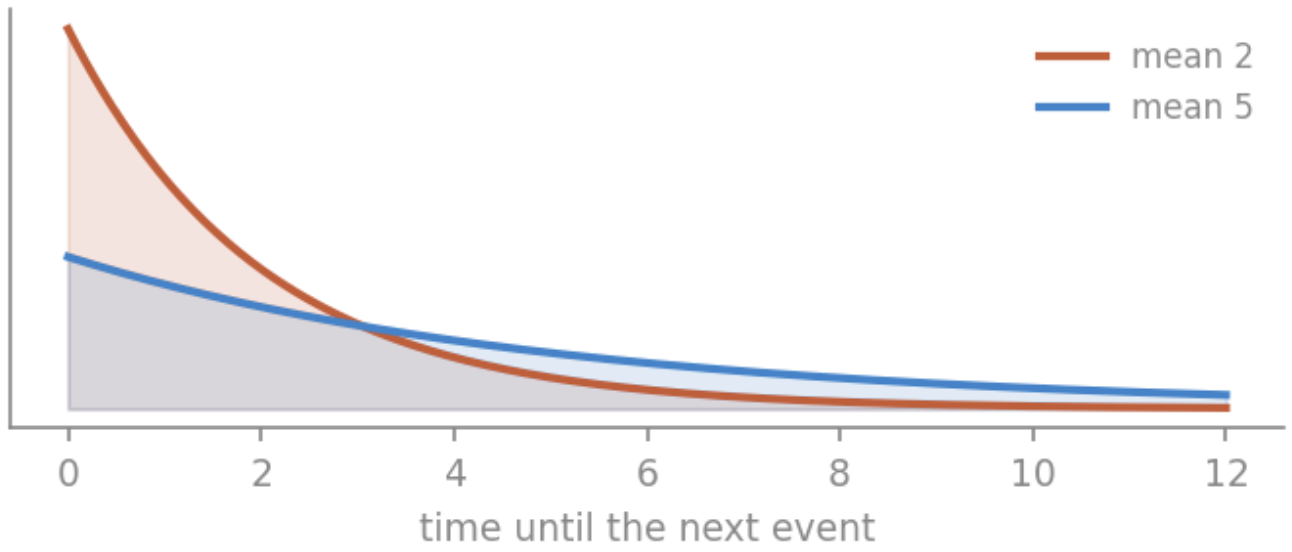
rust

```
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// Linear-interpolation quantile of SORTED data (numpy's default method).
fn quantile(sorted: &[f64], q: f64) -> f64 {
    let h = (sorted.len() - 1) as f64 * q;
    let lo = h.floor() as usize;
    let hi = (lo + 1).min(sorted.len() - 1);
    let (a, b, t) = (sorted[lo], sorted[hi], h - lo as f64);
    if t >= 0.5 { b - (b - a) * (1.0 - t) } else { a + (b - a) * t }
}

/// Sample skewness, as scipy.stats.skew computes it: m3 / m2^1.5.
fn skew(x: &[f64]) -> f64 {
    let m = mean(x);
    let m2 = x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / x.len() as f64;
    let m3 = x.iter().map(|v| (v - m).powi(3)).sum::<f64>() / x.len() as f64;
    m3 / m2.powf(1.5)
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
```

```rust
}

fn main() {
    // lognormal(sigma 0.85, median 2.5): exp of a normal
    let mut rng = Rng::new(3);
    let lat: Vec<f64> = (0..5000).map(|_| 2.5 * rng.normal(0.0, 0.85).exp()).collect();
    let mut sorted = lat.clone();
    sorted.sort_by(|a, b| a.total_cmp(b));

    // WRONG: the arithmetic mean sits between the median and p95 and
    // describes nobody's actual experience.
    println!("mean   {}", mean(&lat));                  // 3.52
    println!("median {}", quantile(&sorted, 0.5));      // 2.53
    println!("p95    {}", quantile(&sorted, 0.95));     // 9.75

    // RIGHT: work on the log scale, where it is symmetric.
    let logs: Vec<f64> = lat.iter().map(|v| v.ln()).collect();
    println!("log data is symmetric: {}", round(skew(&logs), 3)); // -0.021

    // The geometric mean is the natural centre, and it recovers the median.
    let geo = mean(&logs).exp();
    println!("geometric mean: {}", geo);                // 2.47, close to the median

    // Fit and read off the parameters. With loc fixed at 0 the maximum likelihood
    // fit is closed form: sigma = sd of the logs (dividing by n), median = geo.
    let mu = mean(&logs);
    let shape = (logs.iter().map(|v| (v - mu).powi(2)).sum::<f64>() / logs.len() as f64).sqrt();
    let scale = mu.exp();
    println!("sigma: {}  median: {}", shape, scale);    // 0.845, 2.47 - recovered
}
```

Exponential

Waiting time when the event rate is constant.



Exponential: waiting time when the event rate is constant.

The story	How long until the next event, when the chance of it happening in the next instant never changes. It is memoryless: having waited 10 seconds tells you nothing about how much longer you will wait.
In eval work	Time between independent failures. Idle time between requests, roughly.
Use when	Use for waiting times with a constant hazard rate.
Avoid when	Real systems are rarely memoryless. If a request has already run 60 seconds it is probably stuck, and its remaining time is not the same as a fresh request's. Use gamma or lognormal when the hazard changes.

Under the hood

Exponential: formulas and code

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

$$E[X] = \frac{1}{\lambda} \qquad \mathrm{Var}(X) = \frac{1}{\lambda^2}$$

python

```python
from scipy import stats
import numpy as np

# Waiting time with a CONSTANT hazard rate.
gaps = stats.expon.rvs(scale=4.0, size=4000, random_state=1)  # mean 4 seconds

# Memorylessness, demonstrated. Among requests that already waited 5 seconds,
# the REMAINING wait still averages 4 seconds. Waiting tells you nothing.
already = gaps[gaps > 5] - 5
print(gaps.mean(), already.mean())     # 3.98  vs  3.91

# THE TEST: is your data actually memoryless? Compare the mean and sd.
# For an exponential they are equal.
print(gaps.mean(), gaps.std())         # 3.98, 3.90 -> consistent

lat = stats.lognorm.rvs(0.9, scale=3, size=4000, random_state=1)
print(lat.mean(), lat.std())           # 4.56, 5.28 -> not exponential
```

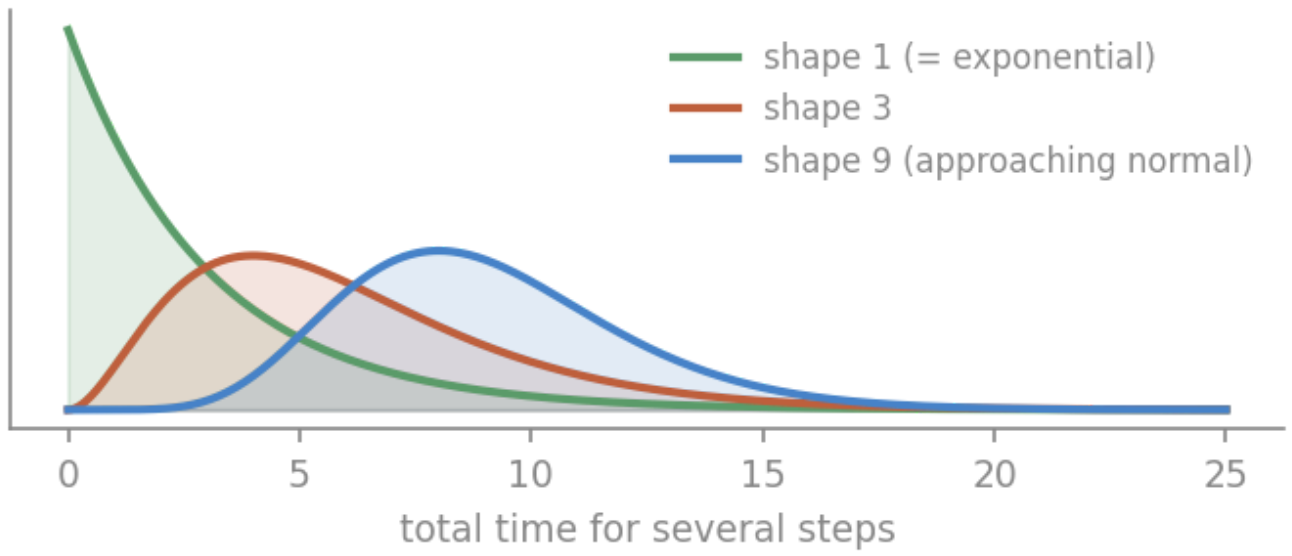
rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// Population standard deviation (ddof = 0, numpy's default).
fn std(x: &[f64]) -> f64 {
    let m = mean(x);
    (x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / x.len() as f64).sqrt()
}

fn main() {
```

```rust
    // Waiting time with a CONSTANT hazard rate.
    let mut rng = Rng::new(1);
    let gaps: Vec<f64> = (0..4000).map(|_| -4.0 * (1.0 - rng.random()).ln()).collect(); // mean 4 seconds

    // Memorylessness, demonstrated. Among requests that already waited 5 seconds,
    // the REMAINING wait still averages 4 seconds. Waiting tells you nothing.
    let already: Vec<f64> = gaps.iter().filter(|&&g| g > 5.0).map(|g| g - 5.0).collect();
    println!("{} {}", mean(&gaps), mean(&already)); // 3.91  vs  4.13

    // THE TEST: is your data actually memoryless? Compare the mean and sd.
    // For an exponential they are equal.
    println!("{} {}", mean(&gaps), std(&gaps));     // 3.91, 4.03 -> consistent

    let lat: Vec<f64> = (0..4000).map(|_| 3.0 * rng.normal(0.0, 0.9).exp()).collect(); // lognormal
    println!("{} {}", mean(&lat), std(&lat));       // 4.44, 5.04 -> not exponential
}
```

Gamma

Total time for several sequential steps.



Gamma: total time for several sequential steps.

The story	Add up several exponential waits and you get a gamma. Shape parameter = how many steps.
In eval work	End to end latency through a multi stage pipeline. Total time for an agent loop with several tool calls.
Use when	Use for sums of waiting times. With shape 1 it IS exponential; with large shape it approaches normal.
Avoid when	Needs the steps to be independent and similarly scaled. A pipeline with one dominant slow stage is not gamma, it is basically that stage's own distribution.

Under the hood

Gamma: formulas and code

$$f(x) = \frac{\lambda^k x^{k-1} e^{-\lambda x}}{\Gamma(k)}$$

$$E[X] = \frac{k}{\lambda} \qquad \mathrm{Var}(X) = \frac{k}{\lambda^2}$$

python

```python
from scipy import stats
import numpy as np

# Total time through a pipeline of k independent stages.
# Sum of k exponentials with the same rate IS a gamma with shape k.
rng = np.random.default_rng(0)
stages = rng.exponential(2.0, size=(5000, 3))     # 3 stages, mean 2s each
total = stages.sum(axis=1)

print(total.mean(), 3 * 2.0)                      # 6.0 - matches k/lambda
print(stats.gamma.fit(total, floc=0)[0])          # recovers shape ~3

# Shape 1 IS exponential; large shape approaches normal.
for k in (1, 3, 20):
    print(k, round(stats.skew(stats.gamma.rvs(k, size=20000, random_state=1)), 2))
# 1 -> 1.93 (very skewed),  3 -> 1.18,  20 -> 0.45 (nearly symmetric)
# theory, 2 / sqrt(k): 2.0, 1.15, 0.45
```

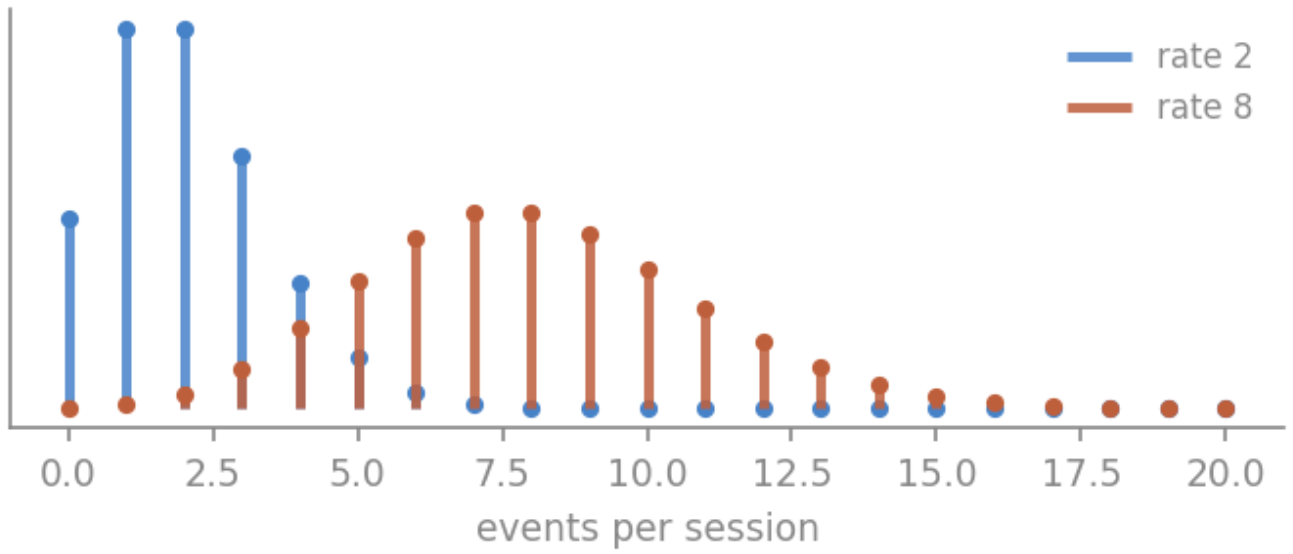
rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

impl Rng {
    /// Gamma(shape, scale) draw, Marsaglia-Tsang method (shape >= 1).
    fn gamma(&mut self, shape: f64, scale: f64) -> f64 {
        let d = shape - 1.0 / 3.0;
        let c = 1.0 / (9.0 * d).sqrt();
        loop {
            let z = self.normal(0.0, 1.0);
            let v = (1.0 + c * z).powi(3);
            if v > 0.0 && self.random().ln() < 0.5 * z * z + d - d * v + d * v.ln() {
                return d * v * scale;
            }
        }
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// Sample skewness, as scipy.stats.skew computes it: m3 / m2^1.5.
fn skew(x: &[f64]) -> f64 {
    let m = mean(x);
    let m2 = x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / x.len() as f64;
    let m3 = x.iter().map(|v| (v - m).powi(3)).sum::<f64>() / x.len() as f64;
    m3 / m2.powf(1.5)
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}
```

```rust
fn digamma(mut x: f64) -> f64 {
    let mut r = 0.0;
    while x < 6.0 {
        r -= 1.0 / x; // psi(x) = psi(x + 1) - 1/x
        x += 1.0;
    }
    let f = 1.0 / (x * x); // asymptotic series for large x
    r + x.ln() - 0.5 / x - f * (1.0 / 12.0 - f * (1.0 / 120.0 - f * (1.0 / 252.0 - f / 240.0)))
}

/// Maximum likelihood gamma shape with loc fixed at 0:
/// solve ln(k) - digamma(k) = ln(mean) - mean(ln x). The left side falls as k grows.
fn gamma_fit_shape(x: &[f64]) -> f64 {
    let s = mean(x).ln() - x.iter().map(|v| v.ln()).sum::<f64>() / x.len() as f64;
    let (mut lo, mut hi) = (1e-3_f64, 1e3_f64);
    for _ in 0..200 {
        let mid = (lo * hi).sqrt();
        if mid.ln() - digamma(mid) > s { lo = mid } else { hi = mid }
    }
    (lo * hi).sqrt()
}

fn main() {
    // Total time through a pipeline of k independent stages.
    // Sum of k exponentials with the same rate IS a gamma with shape k.
    let mut rng = Rng::new(0);
    let total: Vec<f64> = (0..5000)
        .map(|_| (0..3).map(|_| -2.0 * (1.0 - rng.random()).ln()).sum()) // 3 stages, mean 2s each
        .collect();

    println!("{} {:?}", mean(&total), 3.0 * 2.0); // 5.97 - matches k/lambda
    println!("{}", gamma_fit_shape(&total));      // recovers shape ~3 (3.04)

    // Shape 1 IS exponential; large shape approaches normal.
    let mut rng = Rng::new(1);
    for k in [1, 3, 20] {
        let draws: Vec<f64> = (0..20000).map(|_| rng.gamma(k as f64, 1.0)).collect();
        println!("{} {}", k, round(skew(&draws), 2));
    }
    // 1 -> 1.93 (very skewed),  3 -> 1.13,  20 -> 0.48 (nearly symmetric)
    // theory, 2 / sqrt(k): 2.0, 1.15, 0.45
}
```

Poisson

Count of events in a fixed window, at a constant rate.



Poisson: count of events in a fixed window, at a constant rate.

The story	Events happen independently at a steady average rate; you count how many landed in one window. Its defining property: **variance equals the mean**.
In eval work	Requests per minute during steady traffic. Errors per hour when nothing unusual is happening.
Use when	Use when your variance/mean ratio comes out near 1.
Avoid when	**Check that ratio first, because it usually is not near 1.** Tool calls per session run 3 to 6, not 1. Assuming Poisson there makes your intervals roughly sqrt(ratio) times too narrow, and you will call noise a change.

● Under the hood

Poisson: formulas and code

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

$$E[X] = \lambda \qquad \mathrm{Var}(X) = \lambda \qquad \frac{\mathrm{Var}}{E} = 1$$

python

```python
from scipy import stats
import numpy as np

# THE defining property: variance equals the mean. Test this FIRST,
# before you use any Poisson based interval.
counts = stats.poisson.rvs(6, size=3000, random_state=0)
print(counts.mean(), counts.var(ddof=1))          # 5.90, 6.01 -> ratio 1.02

def dispersion_test(counts):
    m = np.mean(counts)
    v = np.var(counts, ddof=1)
    ratio = v / m if m else float("inf")
    return {"mean": m, "var": v, "ratio": ratio,
            "verdict": "Poisson ok" if ratio < 1.5 else "OVERDISPERSED"}

print(dispersion_test(counts))                    # Poisson ok

# Real agent data almost never passes this.
real = np.concatenate([stats.poisson.rvs(2, size=1000, random_state=1),
                       stats.poisson.rvs(15, size=1000, random_state=2)])
print(dispersion_test(real))                      # ratio ~5.9, OVERDISPERSED
```

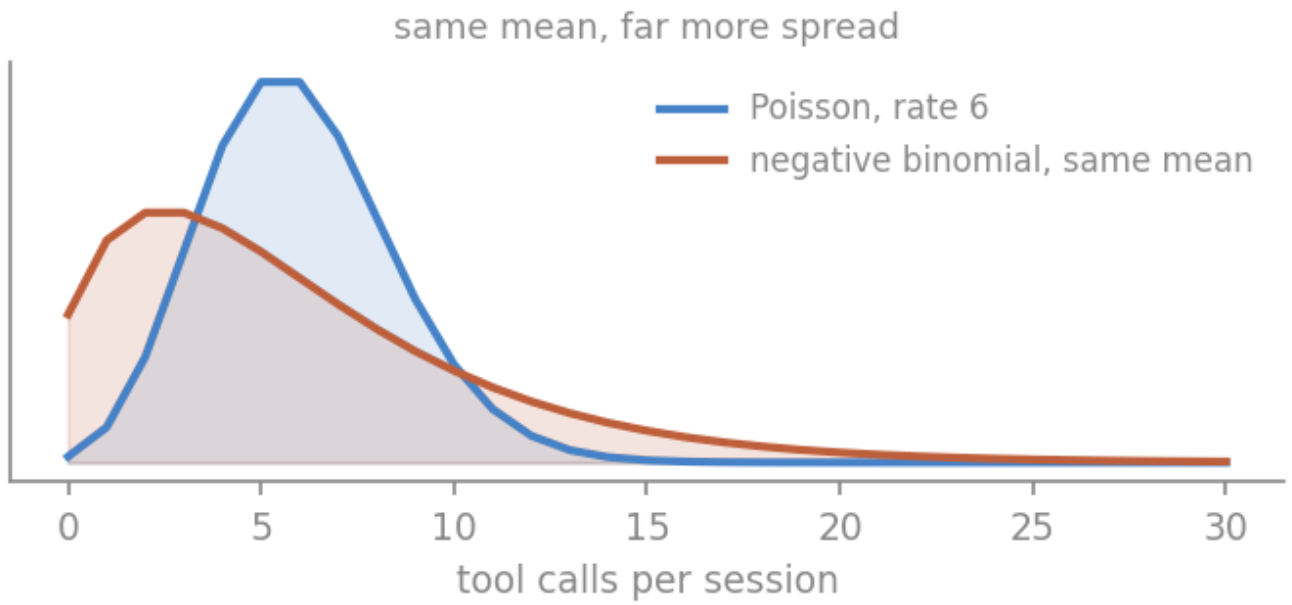
rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}
```

```rust
impl Rng {
    /// Poisson draw: multiply uniforms until the product drops below e^-lambda.
    fn poisson(&mut self, lambda: f64) -> u64 {
        let limit = (-lambda).exp();
        let (mut k, mut prod) = (0, self.random());
        while prod > limit {
            k += 1;
            prod *= self.random();
        }
        k
    }
}

#[derive(Debug)]
struct Dispersion {
    mean: f64,
    var: f64,
    ratio: f64,
    verdict: &'static str,
}

fn dispersion_test(counts: &[u64]) -> Dispersion {
    let n = counts.len() as f64;
    let m = counts.iter().sum::<u64>() as f64 / n;
    let v = counts.iter().map(|&c| (c as f64 - m).powi(2)).sum::<f64>() / (n - 1.0);
    let ratio = if m != 0.0 { v / m } else { f64::INFINITY };
    Dispersion { mean: m, var: v, ratio, verdict: if ratio < 1.5 { "Poisson ok" } else { "OVERDISPERSED" } }
}

fn main() {
    // THE defining property: variance equals the mean. Test this FIRST,
    // before you use any Poisson based interval.
    let mut rng = Rng::new(0);
    let counts: Vec<u64> = (0..3000).map(|_| rng.poisson(6.0)).collect();
    let d = dispersion_test(&counts);
    println!("{} {}", d.mean, d.var);         // 5.97, 5.73 -> ratio 0.96

    println!("{:?}", d);                      // Poisson ok

    // Real agent data almost never passes this.
    let mut real: Vec<u64> = (0..1000).map(|_| rng.poisson(2.0)).collect();
    real.extend((0..1000).map(|_| rng.poisson(15.0)));
    println!("{:?}", dispersion_test(&real)); // ratio ~6.0, OVERDISPERSED
}
```

Negative binomial

Counts when the rate itself varies.



Negative binomial: counts when the rate itself varies.

The story	A mixture of Poissons with different rates is a negative binomial. Easy tasks take two tool calls, hard ones take twenty. The rate is not constant across tasks, so the counts spread far wider than Poisson allows.
In eval work	**Tool calls per session. Turns per conversation. Retries per task.** Nearly every count in agent evaluation.
Use when	Use whenever variance/mean is well above 1, which for agent counts is essentially always.
Avoid when	It has one more parameter than Poisson, so it needs a bit more data. If you only need an interval rather than a model, bootstrapping quantiles is simpler and just as defensible.

Under the hood

Negative binomial: formulas and code

$$P(X = k) = \binom{k + r - 1}{k} (1 - p)^k p^r$$

$$E[X] = \mu \qquad \mathrm{Var}(X) = \mu + \frac{\mu^2}{r} \; > \; \mu$$

python

```python
from scipy import stats
import numpy as np

# A mixture of Poissons with DIFFERENT rates is a negative binomial.
# That is exactly what varied task difficulty produces.
rng = np.random.default_rng(0)
rates = rng.gamma(shape=2.0, scale=3.0, size=4000)   # difficulty varies
calls = rng.poisson(rates)                           # tool calls per session

m, v = calls.mean(), calls.var(ddof=1)
print(m, v, v / m)                    # 6.15, 25.6, ratio 4.16 -> not Poisson

# Fit: parameterise by mean and dispersion r. Var = mu + mu^2 / r.
mu = m
r = mu ** 2 / (v - mu)                # method of moments estimate
print("dispersion r:", r)             # 1.94, recovering the gamma shape of 2.0

# Compare what each model says about seeing 20+ calls in a session:
print("Poisson says: ", stats.poisson.sf(19, mu))            # 7.3e-06 (absurd)
print("NegBinom says:", stats.nbinom.sf(19, r, r / (r + mu)))  # 0.022 (real)
# The Poisson model would flag a routine session as a once-in-a-million event.
```

rust

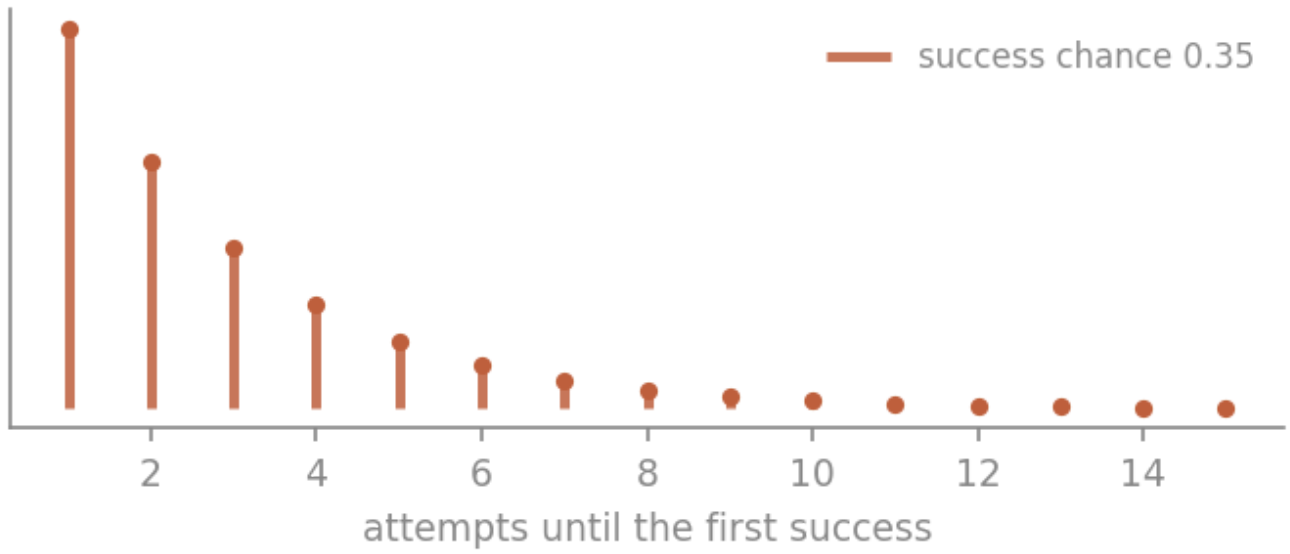
```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
```

```
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

impl Rng {
    /// Gamma(shape, scale) draw, Marsaglia-Tsang method (shape >= 1).
    fn gamma(&mut self, shape: f64, scale: f64) -> f64 {
        let d = shape - 1.0 / 3.0;
        let c = 1.0 / (9.0 * d).sqrt();
        loop {
            let z = self.normal(0.0, 1.0);
            let v = (1.0 + c * z).powi(3);
            if v > 0.0 && self.random().ln() < 0.5 * z * z + d - d * v + d * v.ln() {
                return d * v * scale;
            }
        }
    }
}

impl Rng {
    /// Poisson draw: multiply uniforms until the product drops below e^-lambda.
    fn poisson(&mut self, lambda: f64) -> u64 {
        let limit = (-lambda).exp();
        let (mut k, mut prod) = (0, self.random());
        while prod > limit {
            k += 1;
            prod *= self.random();
        }
        k
    }
}

/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

fn poisson_pmf(k: u64, mu: f64) -> f64 {
    (k as f64 * mu.ln() - mu - ln_gamma(k as f64 + 1.0)).exp()
}

/// Negative binomial pmf: failures before the r-th success, success chance p.
fn nbinom_pmf(k: u64, r: f64, p: f64) -> f64 {
    let k = k as f64;
    (ln_gamma(k + r) - ln_gamma(r) - ln_gamma(k + 1.0) + r * p.ln() + k * (1.0 - p).ln()).exp()
```

```rust
}

/// Survival function P(X > k): add up the upper tail (far enough that the rest is negligible).
fn sf(pmf: impl Fn(u64) -> f64, k: u64) -> f64 {
    (k + 1..k + 2000).map(pmf).sum()
}

fn main() {
    // A mixture of Poissons with DIFFERENT rates is a negative binomial.
    // That is exactly what varied task difficulty produces.
    let mut rng = Rng::new(0);
    let rates: Vec<f64> = (0..4000).map(|_| rng.gamma(2.0, 3.0)).collect();  // difficulty varies
    let calls: Vec<f64> = rates.iter().map(|&r| rng.poisson(r) as f64).collect(); // tool calls per session

    let n = calls.len() as f64;
    let m = calls.iter().sum::<f64>() / n;
    let v = calls.iter().map(|c| (c - m).powi(2)).sum::<f64>() / (n - 1.0);
    println!("{} {} {}", m, v, v / m); // 5.94, 23.5, ratio 3.95 -> not Poisson

    // Fit: parameterise by mean and dispersion r. Var = mu + mu^2 / r.
    let mu = m;
    let r = mu * mu / (v - mu);        // method of moments estimate
    println!("dispersion r: {}", r);   // 2.01, recovering the gamma shape of 2.0

    // Compare what each model says about seeing 20+ calls in a session:
    println!("Poisson says:  {:e}", sf(|k| poisson_pmf(k, mu), 19));           // 4.4e-6 (absurd)
    println!("NegBinom says: {}", sf(|k| nbinom_pmf(k, r, r / (r + mu)), 19)); // 0.018 (real)
    // The Poisson model would flag a routine session as a once-in-a-million event.
}
```

Geometric

How many attempts until the first success.



Geometric: how many attempts until the first success.

The story	Keep retrying until something works; count the attempts. The discrete cousin of the exponential, and also memoryless.
In eval work	**Retry counts.** Attempts before a flaky tool call succeeds.
Use when	Use for attempts-until-success with a constant success chance.
Avoid when	Assumes the success chance does not change between attempts. If a retry is more likely to succeed because a cache warmed up, or less likely because the task is genuinely broken, this is the wrong model.

Under the hood

Geometric: formulas and code

$$P(X = k) = (1-p)^{k-1}p, \quad k = 1, 2, \ldots$$

$$E[X] = \frac{1}{p} \qquad \mathrm{Var}(X) = \frac{1-p}{p^2}$$

python

```python
from scipy import stats
import numpy as np

# Attempts until the first success. Directly models harness retries.
p_success = 0.7
attempts = stats.geom.rvs(p_success, size=5000, random_state=0)

print("mean attempts:", attempts.mean(), " theory:", 1 / p_success)   # 1.43

# What fraction of tasks need more than one attempt?
print((attempts > 1).mean())                    # 0.30, i.e. 1 - p

# Retry budget: what share of tasks are still unfinished after k attempts?
for k in (1, 2, 3, 4):
    print(k, "attempts leaves", round((1 - p_success) ** k, 4), "unfinished")
# 4 attempts leaves 0.8% - that residue is your silently dropped tasks.

# ASSUMPTION CHECK: this needs a constant success chance per attempt.
# If a retry succeeds because a cache warmed up, the model does not hold.
```

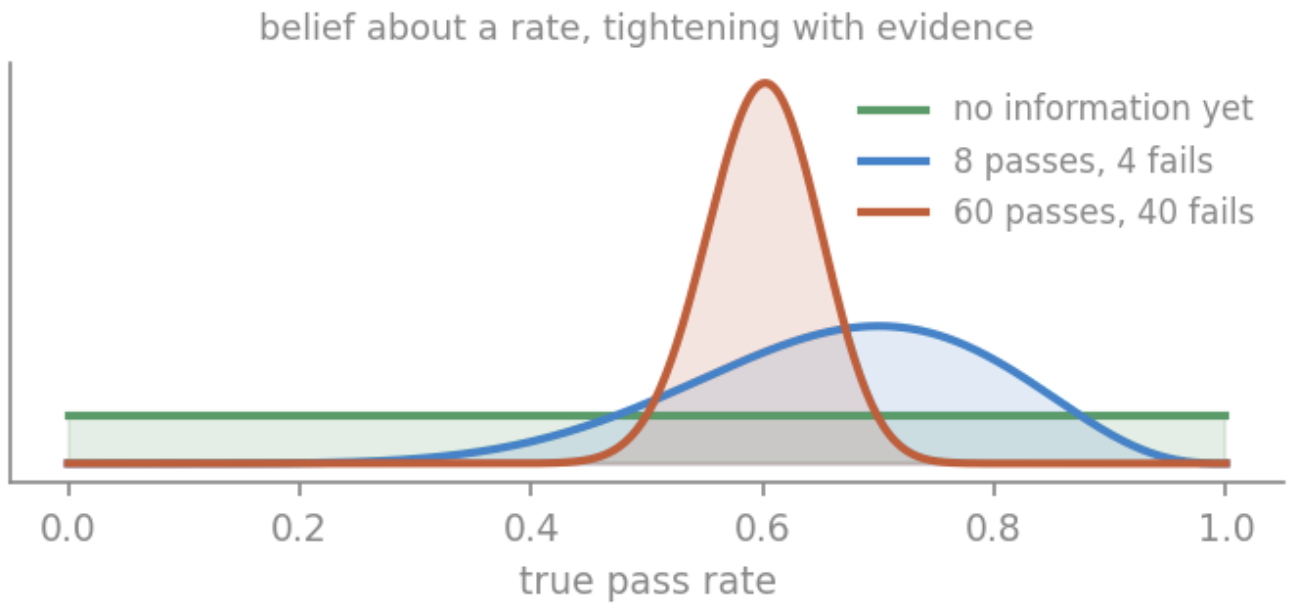
rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

fn main() {
    // Attempts until the first success. Directly models harness retries.
    let p_success = 0.7;
    let mut rng = Rng::new(0);
    let attempts: Vec<u32> = (0..5000)
        .map(|_| {
            let mut k = 1;
            while rng.random() >= p_success {
                k += 1;
            }
```

```
            k
        })
        .collect();

    let n = attempts.len() as f64;
    let mean = attempts.iter().sum::<u32>() as f64 / n;
    println!("mean attempts: {}  theory: {}", mean, 1.0 / p_success); // 1.428 vs 1.43

    // What fraction of tasks need more than one attempt?
    println!("{}", attempts.iter().filter(|&&a| a > 1).count() as f64 / n); // 0.30, i.e. 1 - p

    // Retry budget: what share of tasks are still unfinished after k attempts?
    for k in 1..=4 {
        println!("{} attempts leaves {} unfinished", k, round((1.0 - p_success).powi(k), 4));
    }
    // 4 attempts leaves 0.8% - that residue is your silently dropped tasks.

    // ASSUMPTION CHECK: this needs a constant success chance per attempt.
    // If a retry succeeds because a cache warmed up, the model does not hold.
}
```

Beta

A distribution over a RATE you are uncertain about.



Beta: a distribution over a RATE you are uncertain about.

The story	Everything else here describes data. Beta describes your belief about a probability. After seeing 8 passes and 4 fails, Beta(8,4) is the shape of your uncertainty about the true pass rate. More evidence, tighter shape.
In eval work	Uncertainty about a pass rate. Priors when you have very few tasks. Smoothing a rate estimated from a handful of observations.
Use when	Use for reasoning about rates, especially with small samples. It is the natural partner to the binomial.
Avoid when	It is a belief, not a measurement. Do not report a Beta as if it described your data, and be explicit about any prior you chose, because with small n the prior is doing real work.

Under the hood

Beta: formulas and code

$$f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}, \quad 0 \le x \le 1$$

$$E[X] = \frac{\alpha}{\alpha+\beta} \qquad \mathrm{Var}(X) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

python

```python
from scipy import stats

# Beta describes your BELIEF about a rate, not the data itself.
# Start with no information, then add evidence.
passes, fails = 8, 4
post = stats.beta(1 + passes, 1 + fails)          # uniform prior + evidence

print("best guess:", post.mean())                 # 0.643
print("95% credible interval:", post.interval(0.95))   # (0.386, 0.861)

# With only 12 tasks the interval is enormous. Watch it tighten:
for n in (12, 50, 200, 1000):
    a = 1 + round(0.65 * n)
    b = 1 + n - round(0.65 * n)
    lo, hi = stats.beta(a, b).interval(0.95)
    print(n, "tasks ->", round(hi - lo, 3), "wide")
# 12 -> 0.476,  200 -> 0.131,  1000 -> 0.059
```

```
# Probability model A truly beats a 0.60 baseline, given the data:
print(1 - stats.beta(1 + 8, 1 + 4).cdf(0.60))     # 0.647 - genuinely unsure
```

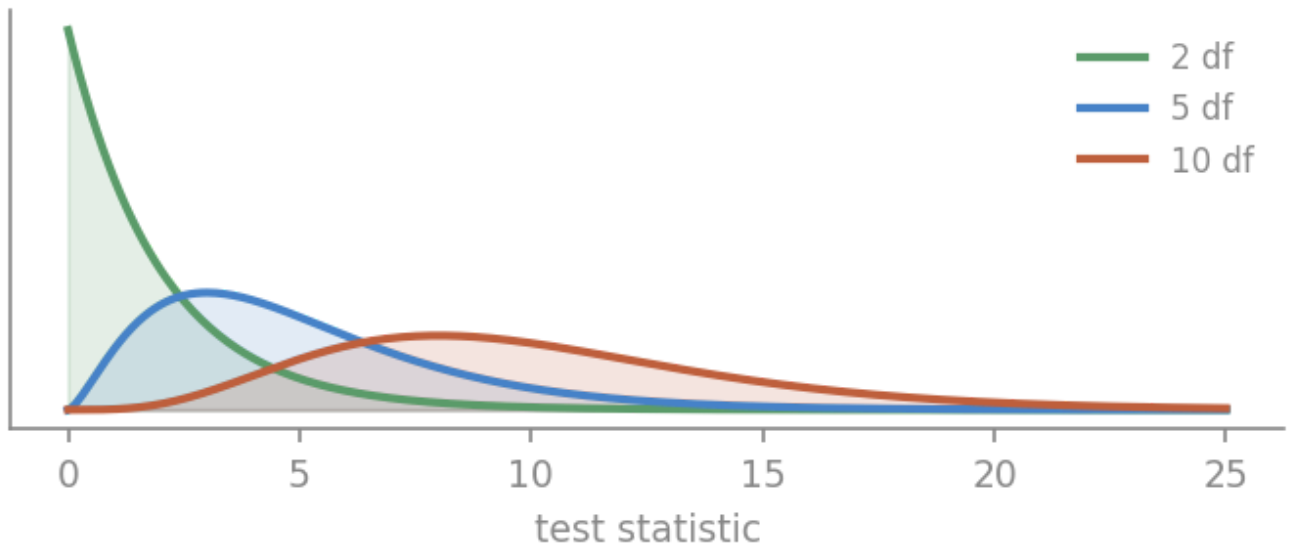
rust

```
/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

/// Regularized incomplete beta I_x(a, b), by continued fraction (modified Lentz).
fn beta_inc(a: f64, b: f64, x: f64) -> f64 {
    if x <= 0.0 {
        return 0.0;
    }
    if x >= 1.0 {
        return 1.0;
    }
    let front = (ln_gamma(a + b) - ln_gamma(a) - ln_gamma(b) + a * x.ln() + b * (1.0 - x).ln()).exp();
    // the fraction converges fast on one side of the mean; use symmetry for the other
    if x < (a + 1.0) / (a + b + 2.0) {
        front * beta_cf(a, b, x) / a
    } else {
        1.0 - front * beta_cf(b, a, 1.0 - x) / b
    }
}

fn beta_cf(a: f64, b: f64, x: f64) -> f64 {
    let tiny = 1e-300;
    let clamp = |v: f64| if v.abs() < tiny { tiny } else { v };
    let mut c = 1.0;
    let mut d = 1.0 / clamp(1.0 - (a + b) * x / (a + 1.0));
    let mut h = d;
    for m in 1..1000 {
        let m = m as f64;
        let even = m * (b - m) * x / ((a + 2.0 * m - 1.0) * (a + 2.0 * m));
        d = 1.0 / clamp(1.0 + even * d);
        c = clamp(1.0 + even / c);
        h *= d * c;
        let odd = -(a + m) * (a + b + m) * x / ((a + 2.0 * m) * (a + 2.0 * m + 1.0));
        d = 1.0 / clamp(1.0 + odd * d);
        c = clamp(1.0 + odd / c);
        h *= d * c;
        if (d * c - 1.0).abs() < 1e-16 {
            break;
        }
    }
```

```rust
    h
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

struct Beta {
    a: f64,
    b: f64,
}

impl Beta {
    fn mean(&self) -> f64 {
        self.a / (self.a + self.b)
    }
    fn cdf(&self, x: f64) -> f64 {
        beta_inc(self.a, self.b, x)
    }
    /// Inverse CDF by bisection: the CDF only goes up, so halve the bracket.
    fn ppf(&self, p: f64) -> f64 {
        let (mut lo, mut hi) = (0.0, 1.0);
        for _ in 0..100 {
            let mid = 0.5 * (lo + hi);
            if self.cdf(mid) < p { lo = mid } else { hi = mid }
        }
        0.5 * (lo + hi)
    }
    /// Central interval holding `level` of the probability.
    fn interval(&self, level: f64) -> (f64, f64) {
        (self.ppf((1.0 - level) / 2.0), self.ppf((1.0 + level) / 2.0))
    }
}

fn main() {
    // Beta describes your BELIEF about a rate, not the data itself.
    // Start with no information, then add evidence.
    let (passes, fails) = (8.0, 4.0);
    let post = Beta { a: 1.0 + passes, b: 1.0 + fails }; // uniform prior + evidence

    println!("best guess: {}", post.mean());                        // 0.643
    println!("95% credible interval: {:?}", post.interval(0.95));  // (0.386, 0.861)

    // With only 12 tasks the interval is enormous. Watch it tighten:
    for n in [12, 50, 200, 1000] {
        let k = (0.65 * n as f64).round_ties_even(); // Python's round() sends halves to even
        let (lo, hi) = Beta { a: 1.0 + k, b: 1.0 + n as f64 - k }.interval(0.95);
        println!("{} tasks -> {} wide", n, round(hi - lo, 3));
    }
    // 12 -> 0.476,  200 -> 0.131,  1000 -> 0.059

    // Probability model A truly beats a 0.60 baseline, given the data:
    println!("{}", 1.0 - Beta { a: 1.0 + 8.0, b: 1.0 + 4.0 }.cdf(0.60)); // 0.647 - genuinely unsure
}
```

Chi square

Sum of squared normal deviations.



Chi square: sum of squared normal deviations.

The story	Add up squared standardised differences. You rarely measure something chi square distributed; you compute a test statistic that follows it.
In eval work	The distribution behind chi square tests of independence, Cochran's Q, and variance tests.
Use when	Use as the reference distribution for those tests.
Avoid when	Its approximations get unreliable when expected cell counts fall below about 5. On small eval sets prefer exact tests or a permutation test, which is why the exact McNemar and permutation approaches are worth having.

Under the hood

Chi square: formulas and code

$$X = \sum_{i=1}^{k} Z_i^2, \quad Z_i \sim \mathcal{N}(0,1)$$

$$E[X] = k \qquad \mathrm{Var}(X) = 2k$$

python

```python
from scipy import stats
import numpy as np

# You rarely MEASURE something chi square. You compute a statistic that follows it.
# Example: are failure reasons distributed the same across two models?
observed = np.array([[42, 31, 27],       # model A: timeout / wrong / crash
                     [18, 55, 29]])      # model B
chi2, p, dof, expected = stats.chi2_contingency(observed)
print(chi2, p, dof)                      # 16.4, 2.8e-04, 2

# THE ASSUMPTION nobody checks: every expected count should exceed ~5.
print(expected.min())                    # 27.7 -> fine here
# If it is below 5, the chi square approximation is unreliable.
# Use an exact test or a permutation test instead:
print(stats.fisher_exact(observed[:, :2]))
```

rust

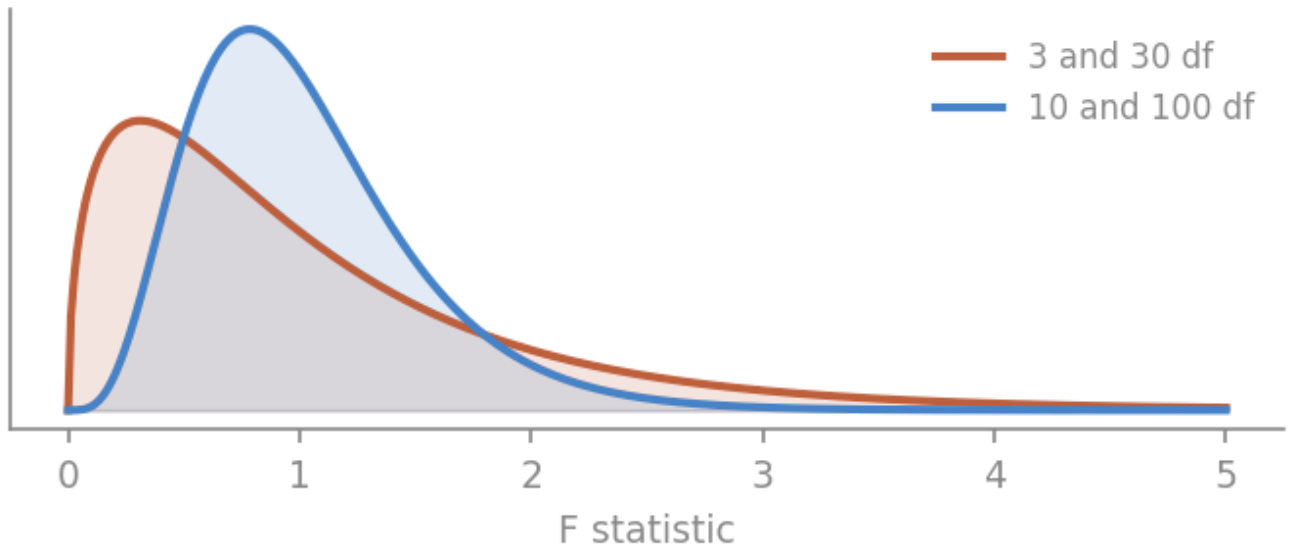
```rust
/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

/// Regularized incomplete gamma functions (P, Q) = (lower, upper), P + Q = 1.
fn gamma_pq(a: f64, x: f64) -> (f64, f64) {
    if x <= 0.0 {
        return (0.0, 1.0);
    }
    let front = (-x + a * x.ln() - ln_gamma(a)).exp();
    if x < a + 1.0 {
        // power series converges fast here
        let (mut ap, mut term) = (a, 1.0 / a);
```

```
        let mut sum = term;
        while term.abs() > sum.abs() * 1e-16 {
            ap += 1.0;
            term *= x / ap;
            sum += term;
        }
        let p = sum * front;
        (p, 1.0 - p)
    } else {
        // continued fraction (modified Lentz) converges fast here
        let tiny = 1e-300;
        let (mut b, mut c) = (x + 1.0 - a, 1.0 / tiny);
        let mut d = 1.0 / b;
        let mut h = d;
        for i in 1..1000 {
            let an = -(i as f64) * (i as f64 - a);
            b += 2.0;
            d = an * d + b;
            if d.abs() < tiny { d = tiny; }
            c = b + an / c;
            if c.abs() < tiny { c = tiny; }
            d = 1.0 / d;
            h *= d * c;
            if (d * c - 1.0).abs() < 1e-16 { break; }
        }
        let q = front * h;
        (1.0 - q, q)
    }
}

fn ln_choose(n: u64, k: u64) -> f64 {
    ln_gamma(n as f64 + 1.0) - ln_gamma(k as f64 + 1.0) - ln_gamma((n - k) as f64 + 1.0)
}

/// Chi square test of independence on an r x c table of counts.
/// Returns (statistic, p value, degrees of freedom, expected counts).
fn chi2_contingency(observed: &[Vec<f64>]) -> (f64, f64, usize, Vec<Vec<f64>>) {
    let row: Vec<f64> = observed.iter().map(|r| r.iter().sum()).collect();
    let col: Vec<f64> = (0..observed[0].len()).map(|j| observed.iter().map(|r| r[j]).sum()).collect();
    let total: f64 = row.iter().sum();
    let expected: Vec<Vec<f64>> = row.iter().map(|ri| col.iter().map(|cj| ri * cj / total).collect()).collect();
    let chi2: f64 = observed.iter().flatten().zip(expected.iter().flatten())
        .map(|(o, e)| (o - e).powi(2) / e)
        .sum();
    let dof = (row.len() - 1) * (col.len() - 1);
    let p = gamma_pq(dof as f64 / 2.0, chi2 / 2.0).1; // chi square upper tail
    (chi2, p, dof, expected)
}

#[derive(Debug)]
struct SignificanceResult {
    statistic: f64, // the sample odds ratio
    pvalue: f64,
}

/// Two-sided Fisher exact test on [[a, b], [c, d]]: add up every table with the
/// same margins that is no more likely than the one observed.
fn fisher_exact(t: [[u64; 2]; 2]) -> SignificanceResult {
    let [[a, b], [c, d]] = t;
    let (row1, col1, n) = (a + b, a + c, a + b + c + d);
```

```
    let ln_pmf = |x: u64| ln_choose(row1, x) + ln_choose(n - row1, col1 - x) - ln_choose(n, col1);
    let cutoff = ln_pmf(a) + 1e-7_f64.ln_1p(); // small tolerance for rounding
    let lo = col1.saturating_sub(n - row1);
    let hi = row1.min(col1);
    let p: f64 = (lo..=hi).map(ln_pmf).filter(|&l| l <= cutoff).map(f64::exp).sum();
    SignificanceResult { statistic: (a * d) as f64 / (b * c) as f64, pvalue: p.min(1.0) }
}

fn main() {
    // You rarely MEASURE something chi square. You compute a statistic that follows it.
    // Example: are failure reasons distributed the same across two models?
    let observed = vec![vec![42.0, 31.0, 27.0],  // model A: timeout / wrong / crash
                        vec![18.0, 55.0, 29.0]]; // model B
    let (chi2, p, dof, expected) = chi2_contingency(&observed);
    println!("{} {} {}", chi2, p, dof); // 16.4, 2.8e-04, 2

    // THE ASSUMPTION nobody checks: every expected count should exceed ~5.
    let min_expected = expected.iter().flatten().cloned().fold(f64::INFINITY, f64::min);
    println!("{}", min_expected);       // 27.7 -> fine here
    // If it is below 5, the chi square approximation is unreliable.
    // Use an exact test or a permutation test instead:
    println!("{:?}", fisher_exact([[42, 31], [18, 55]]));
}
```

F

Ratio of two variances.



F: ratio of two variances.

The story	Signal variance over noise variance. Large F means the differences between groups are big relative to the scatter inside them.
In eval work	The distribution behind ANOVA.
Use when	Use as the reference for ANOVA.
Avoid when	Sensitive to non-normality and unequal variances. On skewed eval data, prefer Friedman or a permutation test.

Under the hood

F: formulas and code

$$F = \frac{\chi^2_{d_1}/d_1}{\chi^2_{d_2}/d_2} = \frac{\text{MS}_{\text{between}}}{\text{MS}_{\text{within}}}$$

$$E[F] = \frac{d_2}{d_2 - 2} \quad (d_2 > 2)$$

python

```python
from scipy import stats
import numpy as np

# F is a ratio of variances: between-group signal over within-group noise.
rng = np.random.default_rng(0)
config_a = rng.normal(2.0, 0.4, 60)      # latency under three serving configs
config_b = rng.normal(2.1, 0.4, 60)
config_c = rng.normal(2.6, 0.4, 60)

F, p = stats.f_oneway(config_a, config_b, config_c)
print(F, p)                              # 32.8, 7.5e-13

# ALWAYS report the effect size next to F. Eta squared = share of variance explained.
groups = [config_a, config_b, config_c]
allv = np.concatenate(groups)
ss_between = sum(len(g) * (g.mean() - allv.mean()) ** 2 for g in groups)
ss_total = ((allv - allv.mean()) ** 2).sum()
print("eta squared:", ss_between / ss_total)     # 0.27

# On skewed data (latency usually is), prefer the rank based version:
print(stats.kruskal(config_a, config_b, config_c))
```

rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
}

/// Regularized incomplete gamma functions (P, Q) = (lower, upper), P + Q = 1.
fn gamma_pq(a: f64, x: f64) -> (f64, f64) {
    if x <= 0.0 {
        return (0.0, 1.0);
    }
    let front = (-x + a * x.ln() - ln_gamma(a)).exp();
    if x < a + 1.0 {
        // power series converges fast here
        let (mut ap, mut term) = (a, 1.0 / a);
        let mut sum = term;
        while term.abs() > sum.abs() * 1e-16 {
            ap += 1.0;
            term *= x / ap;
            sum += term;
```

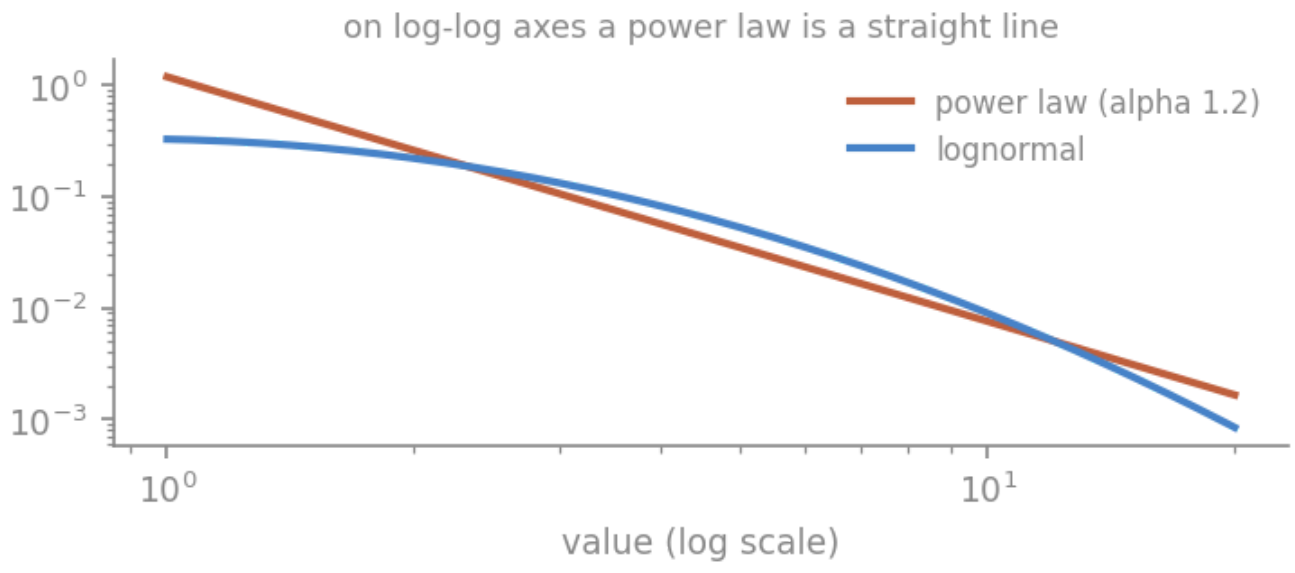
```
        }
        let p = sum * front;
        (p, 1.0 - p)
    } else {
        // continued fraction (modified Lentz) converges fast here
        let tiny = 1e-300;
        let (mut b, mut c) = (x + 1.0 - a, 1.0 / tiny);
        let mut d = 1.0 / b;
        let mut h = d;
        for i in 1..1000 {
            let an = -(i as f64) * (i as f64 - a);
            b += 2.0;
            d = an * d + b;
            if d.abs() < tiny { d = tiny; }
            c = b + an / c;
            if c.abs() < tiny { c = tiny; }
            d = 1.0 / d;
            h *= d * c;
            if (d * c - 1.0).abs() < 1e-16 { break; }
        }
        let q = front * h;
        (1.0 - q, q)
    }
}

/// Regularized incomplete beta I_x(a, b), by continued fraction (modified Lentz).
fn beta_inc(a: f64, b: f64, x: f64) -> f64 {
    if x <= 0.0 {
        return 0.0;
    }
    if x >= 1.0 {
        return 1.0;
    }
    let front = (ln_gamma(a + b) - ln_gamma(a) - ln_gamma(b) + a * x.ln() + b * (1.0 - x).ln()).exp();
    // the fraction converges fast on one side of the mean; use symmetry for the other
    if x < (a + 1.0) / (a + b + 2.0) {
        front * beta_cf(a, b, x) / a
    } else {
        1.0 - front * beta_cf(b, a, 1.0 - x) / b
    }
}

fn beta_cf(a: f64, b: f64, x: f64) -> f64 {
    let tiny = 1e-300;
    let clamp = |v: f64| if v.abs() < tiny { tiny } else { v };
    let mut c = 1.0;
    let mut d = 1.0 / clamp(1.0 - (a + b) * x / (a + 1.0));
    let mut h = d;
    for m in 1..1000 {
        let m = m as f64;
        let even = m * (b - m) * x / ((a + 2.0 * m - 1.0) * (a + 2.0 * m));
        d = 1.0 / clamp(1.0 + even * d);
        c = clamp(1.0 + even / c);
        h *= d * c;
        let odd = -(a + m) * (a + b + m) * x / ((a + 2.0 * m) * (a + 2.0 * m + 1.0));
        d = 1.0 / clamp(1.0 + odd * d);
        c = clamp(1.0 + odd / c);
        h *= d * c;
        if (d * c - 1.0).abs() < 1e-16 {
            break;
```

```
        }
    }
    h
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// One-way ANOVA. Returns (F, p value).
fn f_oneway(groups: &[&[f64]]) -> (f64, f64) {
    let all: Vec<f64> = groups.concat();
    let grand = mean(&all);
    let ss_between: f64 = groups.iter().map(|g| g.len() as f64 * (mean(g) - grand).powi(2)).sum();
    let ss_within: f64 = groups.iter()
        .map(|g| { let m = mean(g); g.iter().map(|v| (v - m).powi(2)).sum::<f64>() })
        .sum();
    let (df_b, df_w) = ((groups.len() - 1) as f64, (all.len() - groups.len()) as f64);
    let f = (ss_between / df_b) / (ss_within / df_w);
    let p = beta_inc(df_w / 2.0, df_b / 2.0, df_w / (df_w + df_b * f)); // F upper tail
    (f, p)
}

#[derive(Debug)]
struct KruskalResult {
    statistic: f64,
    pvalue: f64,
}

/// Kruskal-Wallis H test: ANOVA on ranks, so skew and outliers do not matter.
fn kruskal(groups: &[&[f64]]) -> KruskalResult {
    // pool, sort, and give tied values their average rank
    let mut pooled: Vec<(f64, usize)> = groups.iter().enumerate()
        .flat_map(|(gi, g)| g.iter().map(move |&v| (v, gi)))
        .collect();
    pooled.sort_by(|a, b| a.0.total_cmp(&b.0));
    let n = pooled.len() as f64;
    let mut rank_sum = vec![0.0; groups.len()];
    let mut ties = 0.0; // for the tie correction
    let mut i = 0;
    while i < pooled.len() {
        let mut j = i;
        while j + 1 < pooled.len() && pooled[j + 1].0 == pooled[i].0 {
            j += 1;
        }
        let avg_rank = (i + j) as f64 / 2.0 + 1.0;
        for item in &pooled[i..=j] {
            rank_sum[item.1] += avg_rank;
        }
        let t = (j - i + 1) as f64;
        ties += t * t * t - t;
        i = j + 1;
    }
    let h: f64 = 12.0 / (n * (n + 1.0))
        * groups.iter().zip(&rank_sum).map(|(g, r)| r * r / g.len() as f64).sum::<f64>()
        - 3.0 * (n + 1.0);
    let h = h / (1.0 - ties / (n * n * n - n));
    let df = (groups.len() - 1) as f64;
    KruskalResult { statistic: h, pvalue: gamma_pq(df / 2.0, h / 2.0).1 } // chi square upper tail
}
```

```rust
fn main() {
    // F is a ratio of variances: between-group signal over within-group noise.
    let mut rng = Rng::new(0);
    let config_a: Vec<f64> = (0..60).map(|_| rng.normal(2.0, 0.4)).collect(); // latency under three serving
configs
    let config_b: Vec<f64> = (0..60).map(|_| rng.normal(2.1, 0.4)).collect();
    let config_c: Vec<f64> = (0..60).map(|_| rng.normal(2.6, 0.4)).collect();
    let groups: [&[f64]; 3] = [&config_a, &config_b, &config_c];

    let (f, p) = f_oneway(&groups);
    println!("{} {:e}", f, p); // 38.9, 1e-14 (Python's draws: 32.8, 7.5e-13)

    // ALWAYS report the effect size next to F. Eta squared = share of variance explained.
    let allv: Vec<f64> = groups.concat();
    let grand = mean(&allv);
    let ss_between: f64 = groups.iter().map(|g| g.len() as f64 * (mean(g) - grand).powi(2)).sum();
    let ss_total: f64 = allv.iter().map(|v| (v - grand).powi(2)).sum();
    println!("eta squared: {}", ss_between / ss_total); // 0.31

    // On skewed data (latency usually is), prefer the rank based version:
    println!("{:?}", kruskal(&groups));
    // with these draws: statistic 53.9, pvalue 2.0e-12
}
```

Pareto / power law

Scale free: no typical size.



Pareto / power law: scale free: no typical size.

The story	The heaviest tail you will meet in practice. Doubling the value divides its frequency by a constant, which produces a straight line on log-log axes. Depending on the tail index, the mean and even the variance may not converge.
In eval work	Repo sizes. File sizes. Occasionally session lengths in a long tailed product.
Use when	Use when a log-log plot is straight and the tail refuses to settle down.
Avoid when	**Never report a mean for a genuine power law.** It may not exist, and your sample mean will keep drifting as you collect more data. Percentiles and log scales only. Distinguish carefully from lognormal, which looks similar but is much better behaved.

Under the hood

Pareto / power law: formulas and code

$$P(X > x) = \left(\frac{x_m}{x}\right)^{\alpha}$$

$$E[X] = \frac{\alpha x_m}{\alpha - 1} \quad (\alpha > 1), \quad \text{else undefined}$$

python

```python
import numpy as np
from scipy import stats

# The test that separates a power law from a lognormal: does the MEAN settle?
heavy = stats.pareto.rvs(1.2, size=200000, random_state=0)     # power law
mild = stats.lognorm.rvs(1.0, scale=3, size=200000, random_state=0)

for n in (1000, 10000, 100000, 200000):
    print(n, "power law mean:", round(heavy[:n].mean(), 1),
             " lognormal mean:", round(mild[:n].mean(), 2))
# lognormal: 4.67 -> 4.81 -> 4.94 -> 4.95, settling down.
# power law: 5.7 -> 6.7 -> 5.4 -> 5.9, still wandering at 200,000 samples.

# Estimate the tail index. alpha <= 1 means the mean does not exist at all.
def hill_estimator(x, k=500):
    s = np.sort(x)[::-1][:k + 1]
    return 1.0 / np.mean(np.log(s[:k] / s[k]))
```

```
print("alpha:", hill_estimator(heavy))     # 1.135, close to the true 1.2
# alpha < 2 -> infinite variance. Never report a mean or an sd.
```

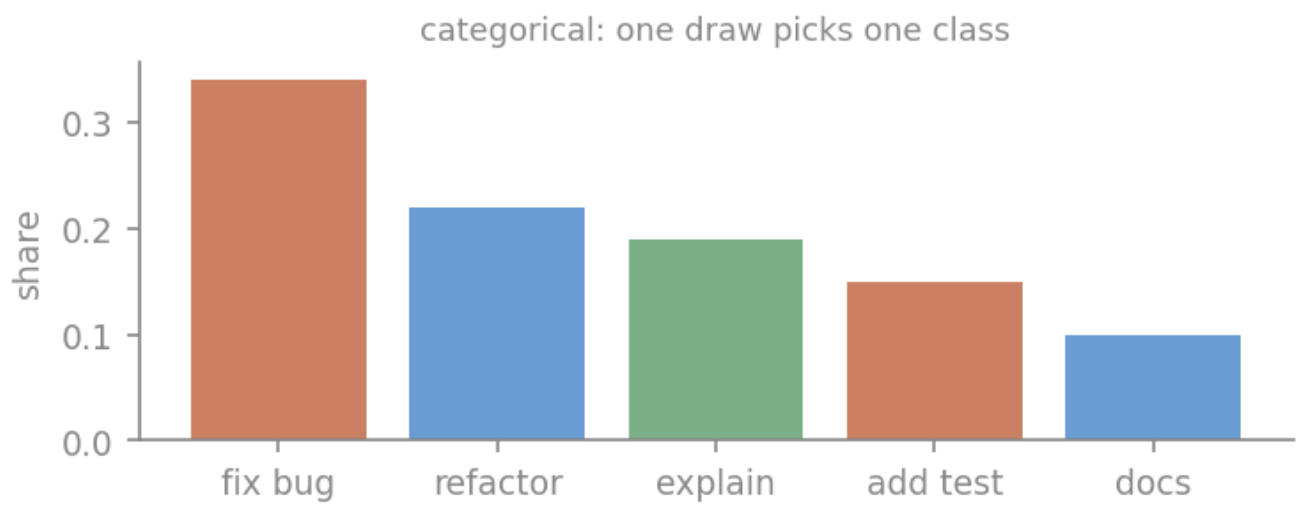
rust

```
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

// Estimate the tail index. alpha <= 1 means the mean does not exist at all.
fn hill_estimator(x: &[f64], k: usize) -> f64 {
    let mut s = x.to_vec();
    s.sort_by(|a, b| b.total_cmp(a)); // largest first
    1.0 / (s[..k].iter().map(|v| (v / s[k]).ln()).sum::<f64>() / k as f64)
}

fn main() {
    // The test that separates a power law from a lognormal: does the MEAN settle?
    let mut rng = Rng::new(0);
    let heavy: Vec<f64> = (0..200_000).map(|_| (1.0 - rng.random()).powf(-1.0 / 1.2)).collect(); // power law
(Pareto, alpha 1.2)
    let mild: Vec<f64> = (0..200_000).map(|_| 3.0 * rng.normal(0.0, 1.0).exp()).collect();      // lognormal

    for n in [1000, 10000, 100000, 200000] {
        println!("{} power law mean: {:?}  lognormal mean: {:?}",
                 n, round(mean(&heavy[..n]), 1), round(mean(&mild[..n]), 2));
    }
    // lognormal: 4.82 -> 5.08 -> 4.96 -> 4.95, settling down.
```

```
    // power law: 4.0 -> 5.2 -> 5.2 -> 5.3, still drifting (the true mean is 6).

    println!("alpha: {}", hill_estimator(&heavy, 500)); // 1.23, close to the true 1.2
    // alpha < 2 -> infinite variance. Never report a mean or an sd.
}
```

Categorical / multinomial

One draw picks one class.



Categorical / multinomial: one draw picks one class.

The story	Each observation lands in exactly one of k buckets, with a probability per bucket. Multinomial is the count version across many draws.
In eval work	**Intent classification.** Task categories. Failure reason codes.
Use when	Use for any labelled class assignment. The counts across classes are multinomial.
Avoid when	Class shares are not independent: they must sum to 1, so pushing one up pulls the others down. Treating each class as its own separate binomial ignores that constraint.

Under the hood

Categorical / multinomial: formulas and code

$$P(X = i) = p_i, \qquad \sum_{i=1}^{k} p_i = 1$$

$$E[\text{count}_i] = np_i \qquad \text{Cov}(n_i, n_j) = -np_ip_j$$

python

```python
from collections import Counter
from scipy import stats
import numpy as np

# Intent classification: each session gets exactly one label.
labels = ["fix_bug", "refactor", "explain", "add_test", "docs"]
counts = np.array([340, 220, 190, 150, 100])
n = counts.sum()

for lab, c in zip(labels, counts):
    print(lab, round(c / n, 3))

# THE CONSTRAINT people forget: shares must sum to 1, so they are NOT
# independent. Pushing one class up pulls the others down, and the
# covariance between two class counts is negative.
p = counts / n
print("cov(fix_bug, refactor):", -n * p[0] * p[1])    # negative, by construction

# Did the intent mix shift between two releases?
before = np.array([340, 220, 190, 150, 100])
after = np.array([290, 260, 185, 165, 100])
print(stats.chisquare(after, f_exp=before * after.sum() / before.sum()))
```

rust

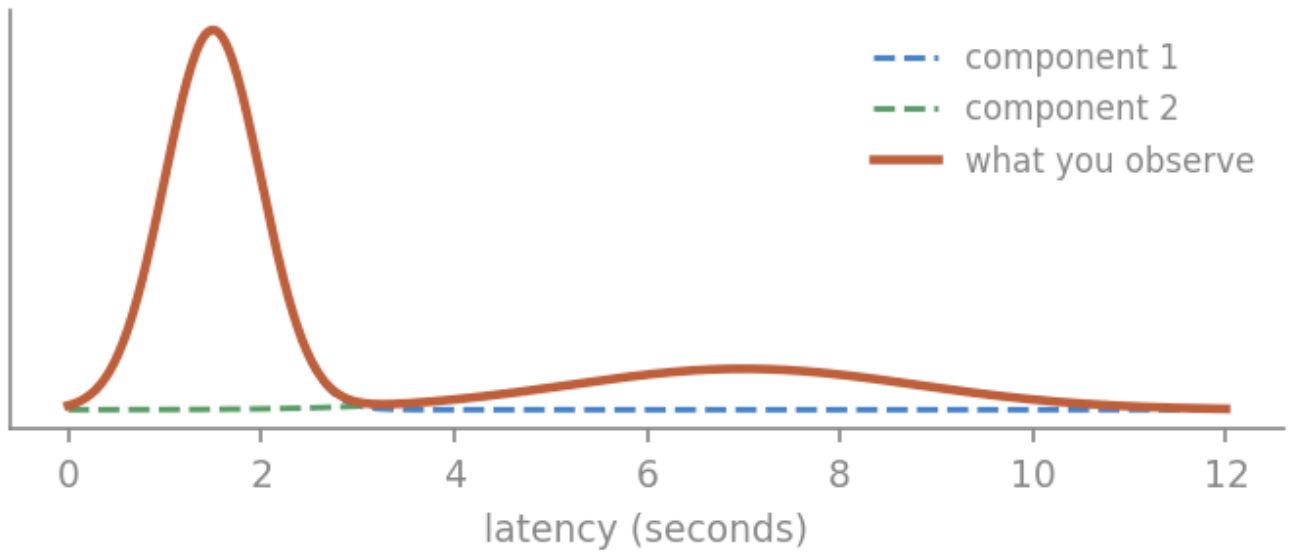
```rust
/// ln Gamma(x) by the Lanczos approximation (g = 7, 9 terms).
fn ln_gamma(x: f64) -> f64 {
    const G: [f64; 9] = [
        0.999_999_999_999_809_93, 676.520_368_121_885_1, -1259.139_216_722_402_8,
        771.323_428_777_653_13, -176.615_029_162_140_59, 12.507_343_278_686_905,
        -0.138_571_095_265_720_12, 9.984_369_578_019_571_6e-6, 1.505_632_735_149_311_6e-7,
    ];
    let pi = std::f64::consts::PI;
    if x < 0.5 {
        return (pi / (pi * x).sin()).ln() - ln_gamma(1.0 - x); // reflection
    }
    let x = x - 1.0;
    let t = x + 7.5;
    let a = G[0] + (1..9).map(|i| G[i] / (x + i as f64)).sum::<f64>();
    0.5 * (2.0 * pi).ln() + (x + 0.5) * t.ln() - t + a.ln()
```

```rust
}

/// Regularized incomplete gamma functions (P, Q) = (lower, upper), P + Q = 1.
fn gamma_pq(a: f64, x: f64) -> (f64, f64) {
    if x <= 0.0 {
        return (0.0, 1.0);
    }
    let front = (-x + a * x.ln() - ln_gamma(a)).exp();
    if x < a + 1.0 {
        // power series converges fast here
        let (mut ap, mut term) = (a, 1.0 / a);
        let mut sum = term;
        while term.abs() > sum.abs() * 1e-16 {
            ap += 1.0;
            term *= x / ap;
            sum += term;
        }
        let p = sum * front;
        (p, 1.0 - p)
    } else {
        // continued fraction (modified Lentz) converges fast here
        let tiny = 1e-300;
        let (mut b, mut c) = (x + 1.0 - a, 1.0 / tiny);
        let mut d = 1.0 / b;
        let mut h = d;
        for i in 1..1000 {
            let an = -(i as f64) * (i as f64 - a);
            b += 2.0;
            d = an * d + b;
            if d.abs() < tiny { d = tiny; }
            c = b + an / c;
            if c.abs() < tiny { c = tiny; }
            d = 1.0 / d;
            h *= d * c;
            if (d * c - 1.0).abs() < 1e-16 { break; }
        }
        let q = front * h;
        (1.0 - q, q)
    }
}

fn round(x: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (x * f).round() / f
}

#[derive(Debug)]
struct PowerDivergenceResult {
    statistic: f64,
    pvalue: f64,
}

/// Chi square goodness of fit: observed counts against expected counts.
fn chisquare(observed: &[f64], expected: &[f64]) -> PowerDivergenceResult {
    let statistic: f64 = observed.iter().zip(expected).map(|(o, e)| (o - e).powi(2) / e).sum();
    let dof = (observed.len() - 1) as f64;
    PowerDivergenceResult { statistic, pvalue: gamma_pq(dof / 2.0, statistic / 2.0).1 }
}

fn main() {
```

```
    // Intent classification: each session gets exactly one label.
    let labels = ["fix_bug", "refactor", "explain", "add_test", "docs"];
    let counts = [340.0, 220.0, 190.0, 150.0, 100.0];
    let n: f64 = counts.iter().sum();

    for (lab, c) in labels.iter().zip(&counts) {
        println!("{} {}", lab, round(c / n, 3));
    }

    // THE CONSTRAINT people forget: shares must sum to 1, so they are NOT
    // independent. Pushing one class up pulls the others down, and the
    // covariance between two class counts is negative.
    let p: Vec<f64> = counts.iter().map(|c| c / n).collect();
    println!("cov(fix_bug, refactor): {}", -n * p[0] * p[1]); // negative, by construction

    // Did the intent mix shift between two releases?
    let before = [340.0, 220.0, 190.0, 150.0, 100.0];
    let after = [290.0, 260.0, 185.0, 165.0, 100.0];
    let scale = after.iter().sum::<f64>() / before.iter().sum::<f64>();
    let f_exp: Vec<f64> = before.iter().map(|b| b * scale).collect();
    println!("{:?}", chisquare(&after, &f_exp));
}
```

Mixture

Two or more populations sharing an axis.



Mixture: two or more populations sharing an axis.

The story	Not a distribution family so much as a warning. Any two distributions can be blended, and the result often looks like one distribution with strange outliers.
In eval work	**Cache hits and cold starts. Retried and first-try requests. Easy and hard task categories.** Very common and very often missed.
Use when	Use it as a diagnosis, not a model. When you see two humps, go find the variable that separates them.
Avoid when	Do not fit a single distribution to a mixture and then report its mean or median. The median can land in the empty valley between the humps, describing a case that never occurs.

Under the hood

Mixture: formulas and code

$$f(x) = \sum_j w_j f_j(x), \qquad \sum_j w_j = 1$$

$$E[X] = \sum_j w_j \mu_j \quad \text{(but the mean may describe nobody)}$$

python

```python
import numpy as np
from sklearn.mixture import GaussianMixture   # or fit by hand

lat = np.concatenate([
    np.random.default_rng(0).normal(1.2, 0.3, 2200),   # cached
    np.random.default_rng(1).normal(6.5, 1.5, 800),    # cold start
])

# DIAGNOSIS, not a model. Two components fit far better than one?
logs = np.log(lat).reshape(-1, 1)
one = GaussianMixture(1, random_state=0).fit(logs)
two = GaussianMixture(2, random_state=0).fit(logs)
print("BIC one component:", round(one.bic(logs)))     # higher = worse
print("BIC two components:", round(two.bic(logs)))    # much lower -> mixture

# Why the median is dangerous here:
print("median:", np.median(lat))          # 1.33 - describes the fast group only
print("mean:  ", lat.mean())              # 2.58 - describes nobody
```

```
# THE RIGHT MOVE: stop modelling and go find the variable that splits them.
# Then report each segment separately, with its own share of traffic.
```

rust

```rust
// A tiny inline PRNG (SplitMix64) so the snippet needs no crates.
// Its draws differ from numpy's, so the random numbers below differ from the Python's.
struct Rng(u64);

impl Rng {
    fn new(seed: u64) -> Self {
        Rng(seed)
    }
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    /// Uniform on [0, 1).
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    /// Normal draw by Box-Muller.
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let u1 = 1.0 - self.random(); // in (0, 1], safe for ln
        let u2 = self.random();
        mean + sd * (-2.0 * u1.ln()).sqrt() * (std::f64::consts::TAU * u2).cos()
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

fn median(x: &[f64]) -> f64 {
    let mut s = x.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));
    let n = s.len();
    if n % 2 == 1 { s[n / 2] } else { (s[n / 2 - 1] + s[n / 2]) / 2.0 }
}

fn normal_pdf(x: f64, mu: f64, var: f64) -> f64 {
    (-(x - mu).powi(2) / (2.0 * var)).exp() / (2.0 * std::f64::consts::PI * var).sqrt()
}

/// Fit a 1-D Gaussian mixture with k components by EM (fitting by hand).
/// Returns the BIC: -2 log likelihood + parameters * ln n. Lower is better.
fn gmm_bic(x: &[f64], k: usize) -> f64 {
    let n = x.len();
    let mut s = x.to_vec();
    s.sort_by(|a, b| a.total_cmp(b));
    // start the means spread across the data, each component with the overall variance
    let mut mu: Vec<f64> = (0..k).map(|j| s[(2 * j + 1) * n / (2 * k)]).collect();
    let m = mean(x);
    let mut var = vec![x.iter().map(|v| (v - m).powi(2)).sum::<f64>() / n as f64; k];
    let mut w = vec![1.0 / k as f64; k];
    let mut resp = vec![vec![0.0; k]; n];
    let mut loglik = f64::NEG_INFINITY;
```

```rust
    for _ in 0..500 {
        // E step: how much each component "owns" each point
        let mut ll = 0.0;
        for (i, &xi) in x.iter().enumerate() {
            let dens: Vec<f64> = (0..k).map(|j| w[j] * normal_pdf(xi, mu[j], var[j])).collect();
            let total: f64 = dens.iter().sum();
            ll += total.ln();
            for j in 0..k {
                resp[i][j] = dens[j] / total;
            }
        }
        // M step: refit each component to the points it owns
        for j in 0..k {
            let nj: f64 = resp.iter().map(|r| r[j]).sum();
            w[j] = nj / n as f64;
            mu[j] = x.iter().zip(&resp).map(|(xi, r)| r[j] * xi).sum::<f64>() / nj;
            var[j] = x.iter().zip(&resp).map(|(xi, r)| r[j] * (xi - mu[j]).powi(2)).sum::<f64>() / nj + 1e-6;
        }
        let done = (ll - loglik).abs() / (n as f64) < 1e-6; // average log likelihood has settled
        loglik = ll;
        if done {
            break;
        }
    }
    let params = (3 * k - 1) as f64; // k means, k variances, k - 1 free weights
    -2.0 * loglik + params * (n as f64).ln()
}

fn main() {
    let mut rng0 = Rng::new(0);
    let mut rng1 = Rng::new(1);
    let mut lat: Vec<f64> = (0..2200).map(|_| rng0.normal(1.2, 0.3)).collect(); // cached
    lat.extend((0..800).map(|_| rng1.normal(6.5, 1.5)));                        // cold start

    // DIAGNOSIS, not a model. Two components fit far better than one?
    let logs: Vec<f64> = lat.iter().map(|v| v.ln()).collect();
    println!("BIC one component: {}", gmm_bic(&logs, 1).round());  // higher = worse
    println!("BIC two components: {}", gmm_bic(&logs, 2).round()); // much lower -> mixture

    // Why the median is dangerous here:
    println!("median: {}", median(&lat)); // 1.34 - describes the fast group only
    println!("mean:   {}", mean(&lat));   // 2.62 - describes nobody

    // THE RIGHT MOVE: stop modelling and go find the variable that splits them.
    // Then report each segment separately, with its own share of traffic.
}
```

Part I · Chapter 6

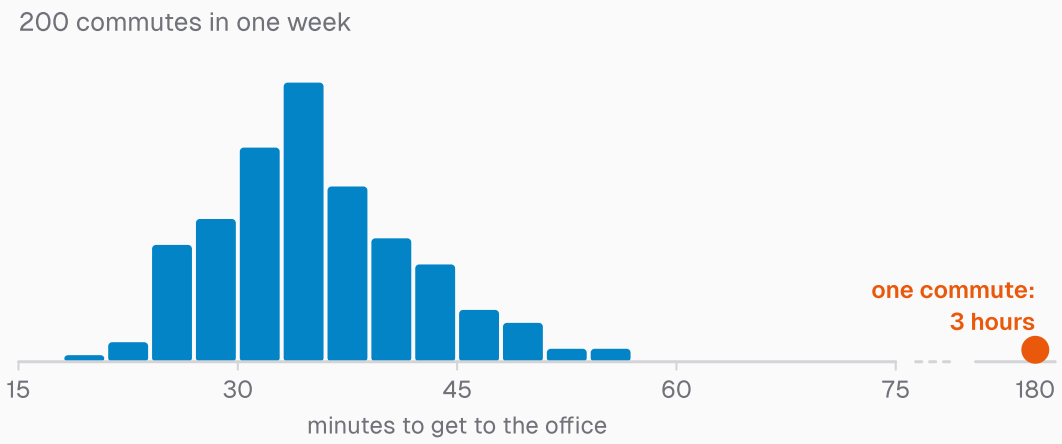
Long tails and outliers: error or product?

Latency, token counts, and agent loop lengths all have long tails, and every team eventually argues about what to do with the extreme values. The first question is never which technique. It is whether the outlier is an **error** or the **product**.

Diagram

The three-hour commute

An outlier is either a broken measurement or a real person. Treat them differently.



A week of office commutes: mostly 25 to 45 minutes. Then one trip of three hours, far off the chart. The instinct is to delete it so the chart looks normal.

An error

The badge reader logged a commute of −4 minutes.

Fix it or drop it.

The product

Her train broke down. She really waited three hours.

Keep it. It happened.

The first question is never "which technique?"

Ask first: is it an error or the product? A negative commute is a broken sensor; fix or drop it. A three-hour wait after a breakdown really happened to someone. In eval data, a 40-second response is a user who waited 40 seconds.

average with her

36.0 min

→

average after deleting

35.2 min

The number got better. Nobody got to work sooner.

Deleting the real one moves the average from 36.0 to 35.2 minutes. The dashboard improves; the commute does not. Teams do this to latency charts all the time.

median	p95	over 90 min
34 min	**48 min**	**0.5%**
what a typical person feels	1 in 20 trips is worse	1 trip in 200, kept

Change the summary, not the data.

Keep the data and change the summary: the median for the typical trip, a high percentile for the bad days, and a count of extreme cases. All three are honest and none hides the person who waited.

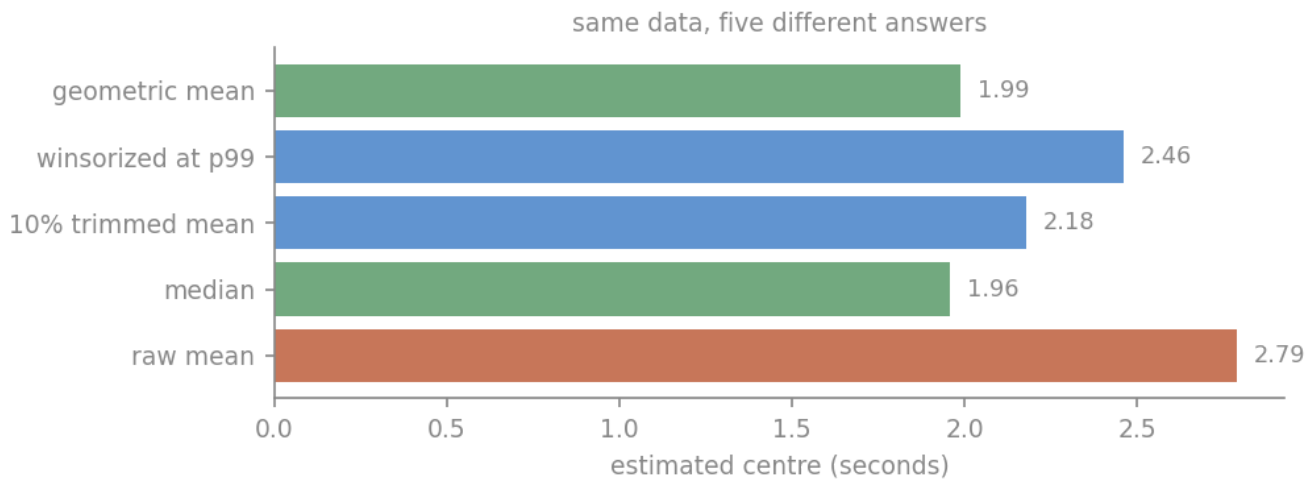
Plain words

The slow request is a real customer

If a restaurant serves most meals in ten minutes but one table waits an hour, that hour is not a typo. It is a customer who left angry. Deleting it from the spreadsheet makes the average look better and the restaurant no better. Only remove a value if you can show the measurement itself was broken, such as a clock error or a logging bug. Otherwise, change how you summarise, not what you measured.

Key idea

In eval data most long tails are signal. A 40 second response is something a real user actually waited through. Winsorizing it makes your chart prettier and your product no better. The honest default is to change the summary statistic, not the data.



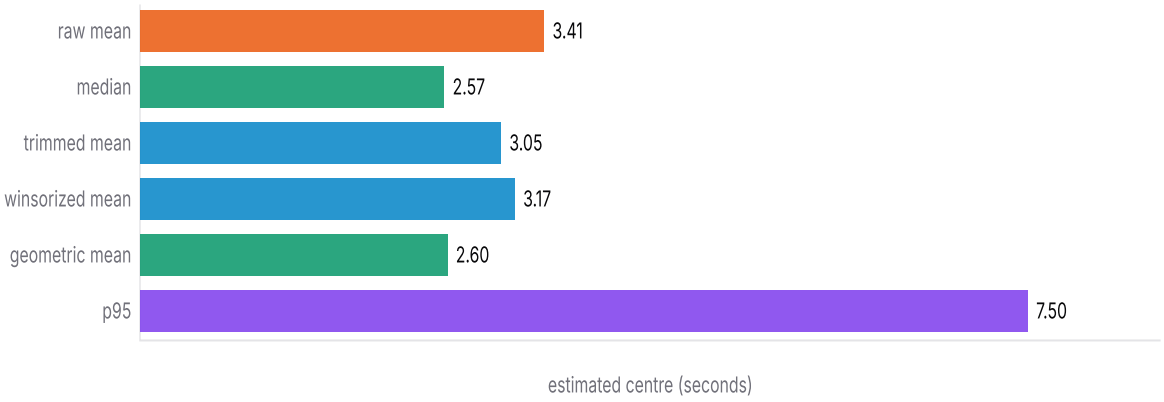
The same 1000 measurements, five different answers for "the centre". The spread between them is entirely a choice someone made.

Interactive

Outlier handling, side by side

(shown at its default setting)

Add outliers and watch which estimators move. The median barely notices. The raw mean tracks the tail directly. Move the winsorization percentile and see how much the answer depends on a threshold somebody picked arbitrarily.



8 outliers out of 1000 spread the "centre" estimates by 1.33x.

Notice the median barely moves no matter what you do to the tail, while the raw mean tracks it directly. Before you pick a method, decide whether those 8 points are broken instrumentation (remove them and say so) or slow requests real users actually waited through (keep them and report percentiles).

The options, in the order to try them

Option	What it does	Cons	Use when
1. Change the statistic	Report p50, p95, p99 with bootstrap intervals instead of a mean	No single headline number; p99 unstable at small n	The tail is real user experience. Almost always, for latency
2. Segment	Find the variable that separates the tail into its own population	Needs a plausible mechanism; you can fish until something separates	You can name a candidate first. **Highest value option and the most skipped**
3. Log transform	Analyse on the log scale, report back in original units	Geometric mean is not the arithmetic mean, so it cannot answer capacity or cost questions. Breaks on zeros	Data is roughly lognormal and you want to model it
4. Winsorize	Cap values above a percentile at that percentile	Biases the mean by an unknown amount; the cap point is arbitrary and two teams will pick differently	You have **confirmed instrumentation errors**, not slow requests
5. Trimmed mean	Drop a fixed share from each end, average the rest	Symmetric trimming on an asymmetric tail removes real data from one side and nothing from the other	Rarely right for a one sided tail
6. Robust estimators	Huber, Hodges-Lehmann: downweight rather than delete	Harder to explain; tuning constants are another arbitrary choice	You need a mean-like number that resists contamination
7. Rank based tests	Mann-Whitney instead of a t test	No longer a statement about means; effect sizes less intuitive	Comparing two groups where outliers would flip a t test
8. Model the tail	Fit a generalized Pareto above a threshold	Real machinery, real assumptions	The tail itself is the deliverable

> ⚠ **Watch out**
>
> **Check for censoring before any of this.** A spike of values sitting exactly at your timeout means those are not measurements, they are lower bounds. No outlier technique fixes that, and every one of them will quietly make the bias worse.

Per metric

- **Latency:** quantiles, never a mean. Segment by cold start and retry status first.
- **Token counts:** log scale for analysis, quantiles for reporting. Watch for a bimodal split between short answers and agentic loops, which is segmentation, not outliers.
- **Tool calls per session:** the tail is task difficulty, so it is signal. Negative binomial or quantiles. Never trim.
- **Pass rates:** bounded at 0 and 1, so no tail. Different problem entirely.

Under the hood

Winsorized, trimmed, and geometric means

$$\tilde{x}_i = \min\big(x_i,\ Q(0.99)\big) \qquad \bar{x}_\alpha = \frac{1}{n - 2\lfloor \alpha n \rfloor} \sum_{i=\lfloor \alpha n \rfloor + 1}^{n - \lfloor \alpha n \rfloor} x_{(i)}$$

Left: winsorizing caps each value at the 99th percentile. Right: the α-trimmed mean averages the sorted values $x_{(i)}$ after dropping a share α from each end.

$$\mathrm{GM} = \exp\left(\frac{1}{n}\sum \ln x_i\right) = \left(\prod x_i\right)^{1/n}$$

The geometric mean: average on the log scale, then come back. On lognormal data it lands on the median.

python

```python
import numpy as np
from scipy import stats

lat = np.concatenate([stats.lognorm.rvs(0.7, scale=2, size=995, random_state=7),
                      [45, 52, 61, 80, 120]])          # five real slow requests
```

```python
def centres(x):
    x = np.asarray(x, dtype=float)
    cap = np.quantile(x, 0.99)
    return {
        "raw mean":        x.mean(),                  # moves with every outlier
        "median":          np.median(x),              # ignores them entirely
        "10% trimmed":     stats.trim_mean(x, 0.10),  # drops both tails
        "winsorized p99":  np.minimum(x, cap).mean(), # caps rather than drops
        "geometric mean":  np.exp(np.log(x).mean()),  # centre on the log scale
        "p95":             np.quantile(x, 0.95),      # when the tail IS the point
    }

for k, v in centres(lat).items():
    print(k.ljust(16), round(v, 3))

# CENSORING CHECK FIRST. A pile-up at one value near the top means those
# are lower bounds, not measurements, and every method above is then biased.
top = lat[lat >= np.quantile(lat, 0.98)]
print("distinct values in the top 2%:", len(np.unique(np.round(top, 3))))
# If that comes back as 1 or 2, you have a timeout, not a tail.
```

rust

```rust
// numpy and scipy pieces written out by hand. The 995 lognormal draws come
// from a small SplitMix64 + Box-Muller generator, so they differ from scipy's
// random_state=7 draws and the numbers below are this program's output.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn uniform(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    fn normal(&mut self) -> f64 {
        // Box-Muller: two uniforms in, one standard normal out
        let u1 = 1.0 - self.uniform(); // in (0, 1], so ln is finite
        let u2 = self.uniform();
        (-2.0 * u1.ln()).sqrt() * (2.0 * std::f64::consts::PI * u2).cos()
    }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

// np.quantile's default: linear interpolation between order statistics
fn quantile(sorted: &[f64], q: f64) -> f64 {
    let pos = q * (sorted.len() - 1) as f64;
    let lo = pos.floor() as usize;
    let hi = (lo + 1).min(sorted.len() - 1);
    sorted[lo] + (pos - lo as f64) * (sorted[hi] - sorted[lo])
}

// stats.trim_mean: cut int(prop * n) points from EACH end, average the rest
```

```rust
fn trim_mean(sorted: &[f64], prop: f64) -> f64 {
    let cut = (prop * sorted.len() as f64) as usize;
    mean(&sorted[cut..sorted.len() - cut])
}

fn centres(x: &[f64]) -> Vec<(&'static str, f64)> {
    let mut s = x.to_vec();
    s.sort_by(f64::total_cmp);
    let cap = quantile(&s, 0.99);
    let capped: Vec<f64> = x.iter().map(|&v| v.min(cap)).collect();
    let logs: Vec<f64> = x.iter().map(|v| v.ln()).collect();
    vec![
        ("raw mean", mean(x)),                    // moves with every outlier
        ("median", quantile(&s, 0.5)),            // ignores them entirely
        ("10% trimmed", trim_mean(&s, 0.10)),     // drops both tails
        ("winsorized p99", mean(&capped)),        // caps rather than drops
        ("geometric mean", mean(&logs).exp()),    // centre on the log scale
        ("p95", quantile(&s, 0.95)),              // when the tail IS the point
    ]
}

fn round_to(v: f64, digits: i32) -> f64 {
    let f = 10f64.powi(digits);
    (v * f).round() / f
}

fn main() {
    let mut rng = Rng(7);
    // lognorm(0.7, scale=2) is 2 * exp(0.7 * Z)
    let mut lat: Vec<f64> = (0..995).map(|_| 2.0 * (0.7 * rng.normal()).exp()).collect();
    lat.extend([45.0, 52.0, 61.0, 80.0, 120.0]);       // five real slow requests

    for (k, v) in centres(&lat) {
        println!("{:<16} {:?}", k, round_to(v, 3));
    }
    // This program's draws (they differ from scipy's):
    // raw mean         2.941
    // median           1.998
    // 10% trimmed      2.25
    // winsorized p99   2.611
    // geometric mean   2.068
    // p95              6.574

    // CENSORING CHECK FIRST. A pile-up at one value near the top means those
    // are lower bounds, not measurements, and every method above is then biased.
    let mut s = lat.clone();
    s.sort_by(f64::total_cmp);
    let cut = quantile(&s, 0.98);
    let mut top: Vec<i64> = lat.iter().filter(|&&v| v >= cut).map(|&v| (v * 1000.0).round() as i64).collect();
    top.sort();
    top.dedup();
    println!("distinct values in the top 2%: {}", top.len()); // 20
    // If that comes back as 1 or 2, you have a timeout, not a tail.
}
```

For leaders

Outlier policy is a product decision

How a team handles extreme values decides which users the metric speaks for. That choice should be explicit, written down, and fixed before results come in.

- **Pre-register the rule.** If the winsorization threshold or exclusion criterion is chosen after seeing the data, it can be tuned (often unconsciously) to produce the desired answer. Two teams picking different cut-offs on the same data can report opposite conclusions.
- **Treat "we removed outliers" as a red flag until explained.** Ask what was removed, how many, and what evidence shows each was a measurement error rather than a slow or failed run. In agent evals, the tail is often exactly the failures you most need to see.
- **Ask for the segment, not the trim.** The highest-value response to a long tail is finding the variable that explains it: a task category, a tool, a context length bucket. That turns a statistics argument into a fixable engineering problem.
- **Separate cost questions from experience questions.** Budgets need means (totals). Users experience percentiles. A report that uses one summary for both is answering one of the two questions wrongly.

Principal asks

"If we had not excluded anything, would the conclusion change?" Ask for both versions side by side. If the conclusion flips, the exclusion rule is carrying the result, and it needs much stronger justification than a footnote.

Part I · Chapter 7

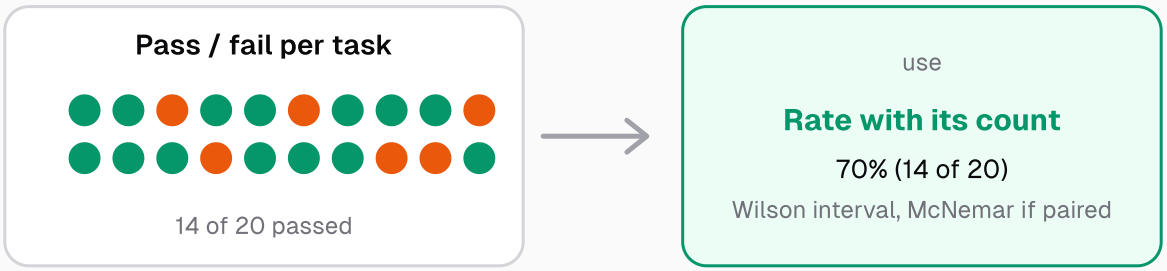
From shape to method: a lookup table

Given what you have, use this. Every row assumes you have already plotted a histogram and checked for censoring and bimodality, as in the reading-a-shape checklist.

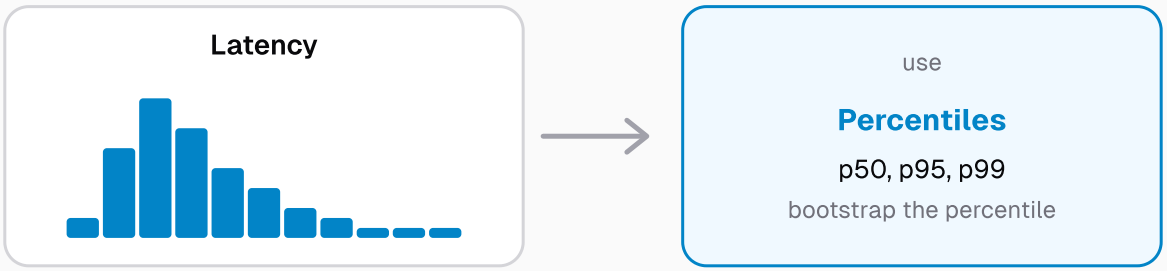
Diagram

Match the data to the tool

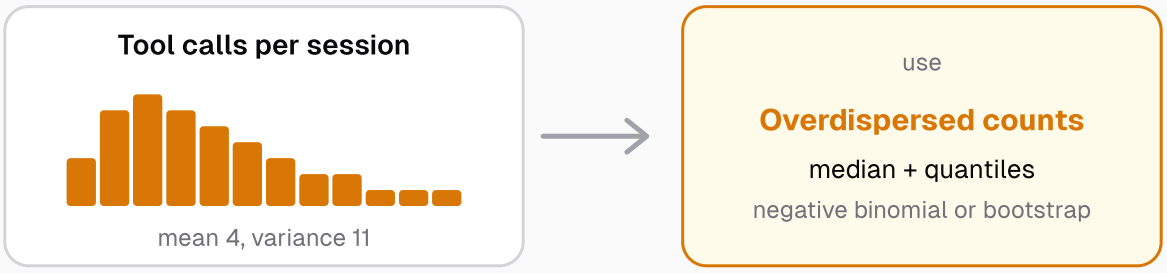
You would not weigh flour with a thermometer. Five kinds of eval data, five toolboxes.



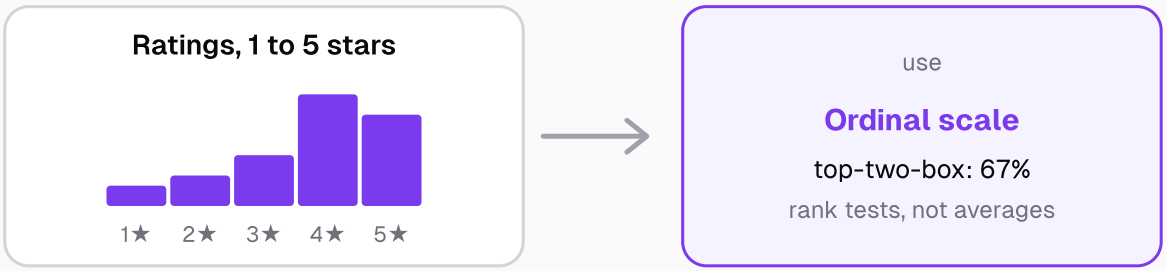
Pass or fail on each task is a pile of coin flips. Report the rate *with* its count, give it a Wilson interval, and compare paired runs with McNemar.



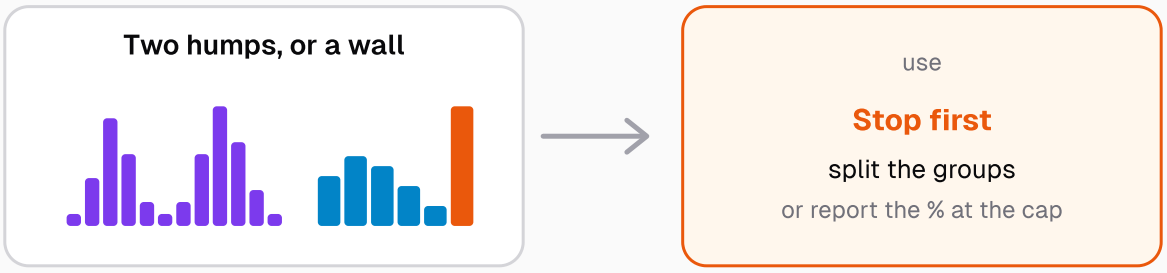
Latency leans right with a long tail. Averages hide the bad days, so report percentiles and get their uncertainty from the bootstrap.



Counts like tool calls per session usually vary far more than their average would suggest (variance 11 against a mean of 4). Poisson tools will claim false precision; use quantiles or a negative binomial.



Star ratings are ordered labels, not measurements: the gap from 2 to 3 stars is not the gap from 4 to 5. Report the share of 4s and 5s and compare with rank tests.



Two humps or a spike at a cap means stop before you summarise. Split the populations, or report how many hit the wall. The table below lists every row.

You have	Shape	Summarise with	Compare with	Interval from
Pass / fail per task	Bernoulli / binomial	Rate + count	Exact McNemar (paired), Cochran's Q (3+)	Wilson
Pass / fail nested in sessions	Clustered binomial	Rate + effective n	Cluster-level comparison	Cluster bootstrap
Latency	Lognormal, right skewed	p50, p95, p99	Mann-Whitney, or t test on logs	Bootstrap on the percentile
Token counts	Right skewed, overdispersed	Median + quantiles	Mann-Whitney	Bootstrap
Tool calls per session	Negative binomial	Median + quantiles, report var/mean	Negative binomial model, or bootstrap	Bootstrap
Retry counts	Geometric	Median, share needing >1 attempt	Proportion test on "needed a retry"	Wilson on that proportion
Likert 1-5	Ordinal	Top-two-box share	Mann-Whitney + effect size	Wilson on top-two-box
Judge score 0-10	Ordinal, often compressed	Median + distribution	Friedman (3+), Mann-Whitney (2)	Bootstrap
A rate you barely have data for	Beta	Posterior mean + interval	Overlap of posteriors	Beta credible interval
Anything with two humps	Mixture	**Split it first**	Compare within segments	Per segment
Anything with a spike at a cap	Censored	**Report the censoring rate**	Survival methods	Kaplan-Meier

> **Note**
>
> If a row makes you reach for the bootstrap, remember its limits: it fails at very small n, on extreme quantiles, on maxima, and on clustered data unless you resample the clusters. Part II covers each of these in detail.

Plain words

Pick the tool that fits the data

You would not measure flour with a thermometer. Each kind of eval data, yes/no results, waiting times, counts, star ratings, has tools that fit it and tools that quietly give wrong answers. This table matches each kind of data to a way of summarising it, a way of comparing two models, and a way of drawing an error bar.

For leaders

Use the table as a review checklist

You do not need to run these methods yourself to use this table well. When a result lands in front of you, find the row that matches the metric and check the analysis against it.

1. **Was the comparison paired?** If both models ran the same tasks, the comparison should use the pairing (McNemar, paired bootstrap). An unpaired test throws away most of the statistical power and can miss real wins.
2. **Does the interval method match the data?** Wilson for rates, bootstrap for percentiles, cluster bootstrap for anything nested in sessions or conversations. A normal-approximation interval on a 98% pass rate or a session-level metric is a warning sign.
3. **Was the data split or flagged first?** Mixtures need segmenting and censored data needs its censoring rate reported. If neither check is mentioned, ask.
4. **Is the summary the one users feel?** Percentiles for latency, top-two-box for satisfaction, rates with counts for pass/fail.

> **Principal asks**
>
> "Which row of the table is this metric, and does the method match?" A team that can answer instantly has thought about its measurement. A pause usually means the default tool was used without checking whether it fits.

Under the hood

The decision, compressed

The table encodes four questions asked in order:

1. **Is it binary, count, continuous, or ordinal?** This fixes the family: binomial and beta for binary, Poisson or negative binomial for counts, lognormal or quantile methods for positive continuous data, rank methods for ordinal scores.
2. **Are observations independent?** If they are nested (calls in sessions, turns in conversations), the effective sample size is closer to the number of clusters, and intervals must resample clusters.
3. **Are the comparisons paired?** Shared tasks call for paired tests, which use within-task differences and remove between-task difficulty from the noise.
4. **Is the statistic smooth?** Means and medians bootstrap well at moderate n. Extreme quantiles and maxima do not, and need more data, a parametric tail model, or an honest "insufficient data".

Part II

Evaluating models

Turning model outputs into numbers you can defend: eval statistics, classification metrics, judges, the offline to online gap, and the harness that produces it all.

Part II · Chapter 8

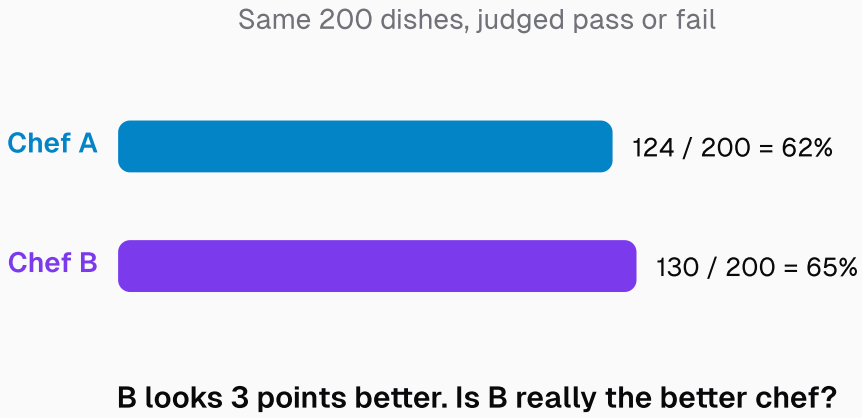
Statistics for eval sets: is the difference real?

Every eval number is an estimate from a finite sample of tasks. This chapter covers the handful of methods that decide whether a result can be trusted: an interval on one pass rate, a paired comparison of two models, power and sample size, many benchmarks at once, pass@k, and comparisons across more than two models. It assumes the ideas from sampling distributions and standard error; here they are applied to the numbers an eval team actually reports.

Diagram

Two chefs, the same 200 dishes

Why comparing two models on shared tasks only counts the disagreements.



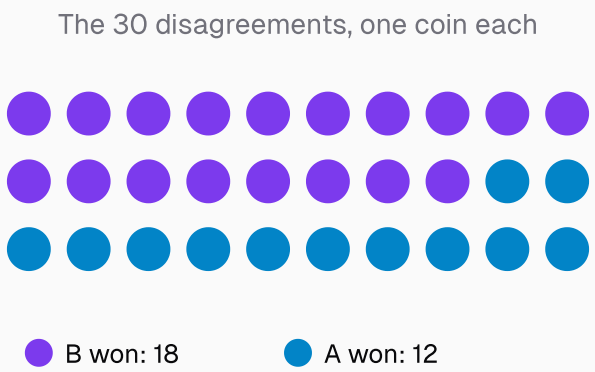
Two chefs cook the same 200 dishes and a judge marks each pass or fail. Chef B scores 130, chef A 124. Product wants to promote B. Comparing the two totals is the obvious move, and the wrong one.

Line the dishes up side by side instead of counting totals.

Line the dishes up one by one. On 112 dishes both chefs passed, and on 58 both failed. Only 30 dishes split them: B alone passed 18, A alone passed 12.

170 dishes where the chefs tied say nothing about who is better.

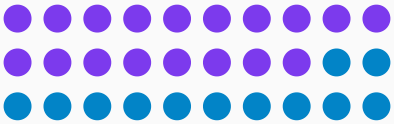
The 170 dishes where both chefs got the same result cannot tell you who is better, so set them aside. The whole comparison rests on the 30 dishes where they differed. This is a paired comparison.



A fair coin splits 18 to 12 or worse 36% of the time (p = 0.36).

Treat each disagreement as a coin toss. If the chefs were equally good, a split of 18 to 12 or more lopsided would happen about 36% of the time. That is not evidence. The exact McNemar test gives p = 0.36.

Kitchen 1



30 disagreements, 18 to 12

p = 0.36: could be luck

Kitchen 2

6 disagreements, 6 to 0

p = 0.03: convincing

Both kitchens: 124 vs 130, the same 3-point headline.

Same 124 versus 130 headline, different story. If the chefs had disagreed on only 6 dishes and B won all 6, the result would be convincing (p = 0.03). The totals cannot tell these kitchens apart. The disagreements can.

Key idea

Most eval mistakes are not exotic. They are a missing interval, an ignored pairing, an underpowered suite, or forty benchmarks read as forty independent findings. Get those four right and you are ahead of most results decks.

Missing interval

A score with no range. Fix: Wilson interval on every pass rate (section 1).

Ignored pairing

Comparing totals on shared tasks. Fix: count only the disagreements (section 2).

Underpowered suite

Too few tasks to see the effect you care about. Fix: size the suite first (sections 3 and 4).

Forty "findings"

Many benchmarks read one at a time. Fix: correct for multiple comparisons (section 5).

1. The uncertainty on a single pass rate

Plain words

One score is one spoonful

If a model passes 49 of 50 tasks, its "true" pass rate is probably high, but you only tasted 50 spoonfuls. A different 50 tasks would give a slightly different score. A **confidence interval** is the range of true pass rates that could reasonably have produced what you saw. Fewer tasks means a wider range.

The most common shortcut for building that range breaks when scores are near 0% or 100%, which is exactly where good models live. For 49 of 50 it claims the true rate might be 102%, which is impossible. A better recipe, the **Wilson interval**, never does that.

For leaders

What to require in every report

- **No pass rate without n and an interval.** "62%" is not a result. "62% (124/200, 95% CI 55 to 68)" is.
- **Near the ceiling, intervals get lopsided.** On saturated benchmarks the honest interval is asymmetric, and small suites cannot separate 96% from 99%. That is a sign the benchmark has stopped being informative, not that the models are equal.

- **Red flag:** symmetric "±" error bars on a score above 90% or below 10%. They come from the normal approximation and are wrong in a predictable direction.

Under the hood

The Wilson score interval

With k passes out of n, $\hat{p} = k/n$, and $z = 1.96$ for 95%:

$$\frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}$$

The centre is pulled toward 0.5 by $z^2/2n$, and the width gets a small-sample correction $z^2/4n^2$. Wilson comes from inverting the score test, so the interval is asymmetric in the right direction near a boundary.

python

```python
import math

def wilson(k: int, n: int, z: float = 1.96) -> tuple[float, float]:
    """Wilson score interval for a binomial proportion."""
    # With no data every proportion is possible; avoids dividing by zero below.
    if n == 0:
        return (0.0, 1.0)

    p = k / n                          # observed pass rate, the point estimate
    denom = 1.0 + z * z / n            # shrink factor; tends to 1 as n grows

    # The CENTRE is not p. Wilson pulls it toward 0.5 by z^2/(2n), which is what
    # stops the interval running past 0 or 1 at extreme rates.
    center = (p + z * z / (2 * n)) / denom

    # Half width: binomial variance p(1-p)/n plus the small-sample term z^2/(4n^2).
    margin = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / denom

    # Clamp against floating point nudging a bound past a boundary.
    return (max(0.0, center - margin), min(1.0, center + margin))


def normal_approx(k: int, n: int, z: float = 1.96) -> tuple[float, float]:
    """The default most people reach for. Shown so you can see why it fails."""
    p = k / n
    margin = z * math.sqrt(p * (1 - p) / n)    # symmetric, no correction
    return (p - margin, p + margin)            # no clamping: can exceed 1


print(wilson(49, 50))         # (0.8950, 0.9965)  stays inside [0, 1]
print(normal_approx(49, 50))  # (0.9412, 1.0188)  claims a rate above 100%
```

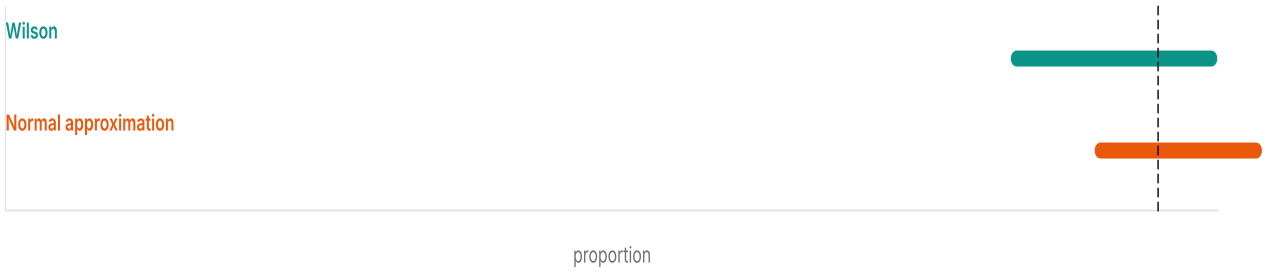
rust

```rust
/// Wilson score interval for a binomial proportion.
fn wilson(k: u64, n: u64, z: f64) -> (f64, f64) {
    // With no data every proportion is possible; avoids dividing by zero below.
    if n == 0 {
        return (0.0, 1.0);
    }
    let (kf, nf) = (k as f64, n as f64);

    let p = kf / nf;                    // observed pass rate, the point estimate
    let denom = 1.0 + z * z / nf;       // shrink factor; tends to 1 as n grows

    // The CENTRE is not p. Wilson pulls it toward 0.5 by z^2/(2n), which is what
    // stops the interval running past 0 or 1 at extreme rates.
    let center = (p + z * z / (2.0 * nf)) / denom;

    // Half width: binomial variance p(1-p)/n plus the small-sample term z^2/(4n^2).
    let margin = z * (p * (1.0 - p) / nf + z * z / (4.0 * nf * nf)).sqrt() / denom;

    // Clamp against floating point nudging a bound past a boundary.
    ((center - margin).max(0.0), (center + margin).min(1.0))
}

/// The default most people reach for. Shown so you can see why it fails.
fn normal_approx(k: u64, n: u64, z: f64) -> (f64, f64) {
    let p = k as f64 / n as f64;
    let margin = z * (p * (1.0 - p) / n as f64).sqrt();  // symmetric, no correction
    (p - margin, p + margin)                             // no clamping: can exceed 1
}

fn main() {
    println!("{:?}", wilson(49, 50, 1.96));         // (0.8950, 0.9965)  stays inside [0, 1]
    println!("{:?}", normal_approx(49, 50, 1.96));  // (0.9412, 1.0188)  claims a rate above 100%
}
```

The normal approximation fails because it assumes the sampling distribution of $\hat{p}$ is symmetric around $\hat{p}$. Near a boundary it is not. Exact coverage at a true rate of 0.95 with $n = 30$: Wilson 0.94, normal approximation 0.78. At $n = 50$: 0.96 versus 0.92.

Interactive **Wilson vs the normal approximation** (shown at its default setting)

Both claim to be 95 percent intervals. The readout runs 3000 simulated experiments at your settings and reports what fraction actually contained the true rate. Drag the rate toward 0.95 and shrink n to watch the normal approximation quietly stop being a 95 percent interval.



Actual coverage over 3000 simulated runs

Wilson 0.932 Normal 0.781

Nominal is 0.950. The normal approximation is not delivering a 95% interval here.

Normal upper bound exceeds 1.0, which is impossible.

⚠ **Watch out**

At a true rate of 0.95 with n = 30, the normal approximation covers about 78 percent of the time and its upper bound can exceed 1.0. Extreme rates on small sets are where eval suites live.

2. Comparing two models on the same tasks

Plain words **Only the disagreements count**

Two students take the same 200-question exam. On most questions they both get it right, or both get it wrong. Those questions tell you nothing about who is better. Only the questions where one was right and the other wrong carry information. If there are 30 such questions and one student wins 18 of them, that could easily be luck. If they win 22 of 30, it probably is not.

Evals work the same way. When two models run on the same tasks, you compare them task by task. This is called a **paired** comparison, and it is far more sensitive than comparing two separate averages.

For leaders

What to say when product wants to ship

- **Overlapping intervals is not a test.** Two intervals can overlap heavily while the paired difference is clearly significant, and the reverse. Ask for the paired result.
- **The headline delta is not enough.** The same 3-point gap can be noise or real depending on how the disagreements split. Ask: "How many tasks did they disagree on, and how did those split?"
- **Frame the answer as a decision.** "The 3-point gain rests on 30 disagreements split 18 to 12. A coin flip does that about a third of the time. I can't call this an improvement yet. Here is what it would take to know."

Under the hood

Paired bootstrap and exact McNemar

Let b be the tasks A passed and B failed, and c the reverse. Under the null hypothesis that the models are equally good, each discordant task is a fair coin flip, so $b \sim \mathrm{Binomial}(b + c, \tfrac{1}{2})$. The exact two-sided p value is

$$p = \min\left(1,\ 2 \sum_{i=0}^{\min(b,c)} \binom{n_d}{i} 2^{-n_d}\right), \qquad n_d = b + c$$

Tasks both models passed or both failed never enter the formula. That is why the test has power on small suites. Use the exact version rather than the chi-squared approximation when n_d is small.

python

```python
from math import comb

def mcnemar_exact(a: list[int], b: list[int]) -> dict:
    """Exact McNemar test on paired binary outcomes."""
    b_count = sum(1 for x, y in zip(a, b) if x == 1 and y == 0)   # A won, B lost
    c_count = sum(1 for x, y in zip(a, b) if x == 0 and y == 1)   # B won, A lost
    n = b_count + c_count            # discordant pairs: the effective sample
    if n == 0:
        return {"b": 0, "c": 0, "p": 1.0}
    k = min(b_count, c_count)
    tail = sum(comb(n, i) for i in range(0, k + 1)) / (2 ** n)
    return {"b": b_count, "c": c_count, "n_discordant": n, "p": min(1.0, 2 * tail)}
```

```
# 200 tasks, the models agree on 188. A wins 9 of the 12 disagreements.
print(mcnemar_exact([1]*9 + [0]*3 + [1]*94 + [0]*94,
                    [0]*9 + [1]*3 + [1]*94 + [0]*94))
# {'b': 9, 'c': 3, 'n_discordant': 12, 'p': 0.146}   nine to three is not enough
```

rust

```
struct McNemar {
    b: usize,
    c: usize,
    n_discordant: usize,
    p: f64,
}

/// P(X <= k) for X ~ Binomial(n, 1/2): sum of C(n, i) over 2^n.
fn binom_half_cdf(n: usize, k: usize) -> f64 {
    if n <= 120 {
        // Exact integers, like Python's math.comb; u128 holds C(120, 60) * 61.
        let (mut c, mut total) = (1u128, 1u128);
        for i in 1..=k {
            c = c * (n - i + 1) as u128 / i as u128;
            total += c;
        }
        total as f64 / 2f64.powi(n as i32)
    } else {
        // Large n: add the terms in log space so nothing overflows.
        let mut log_term = -(n as f64) * std::f64::consts::LN_2;
        let mut total = log_term.exp();
        for i in 1..=k {
            log_term += ((n - i + 1) as f64 / i as f64).ln();
            total += log_term.exp();
        }
        total
    }
}

/// Exact McNemar test on paired binary outcomes.
fn mcnemar_exact(a: &[u8], b: &[u8]) -> McNemar {
    let b_count = a.iter().zip(b).filter(|&(&x, &y)| x == 1 && y == 0).count(); // A won, B lost
    let c_count = a.iter().zip(b).filter(|&(&x, &y)| x == 0 && y == 1).count(); // B won, A lost
    let n = b_count + c_count;           // discordant pairs: the effective sample
    if n == 0 {
        return McNemar { b: 0, c: 0, n_discordant: 0, p: 1.0 };
    }
    let k = b_count.min(c_count);
    let tail = binom_half_cdf(n, k);
    McNemar { b: b_count, c: c_count, n_discordant: n, p: (2.0 * tail).min(1.0) }
}

fn main() {
    // 200 tasks, the models agree on 188. A wins 9 of the 12 disagreements.
    let a: Vec<u8> = [vec![1; 9], vec![0; 3], vec![1; 94], vec![0; 94]].concat();
    let b: Vec<u8> = [vec![0; 9], vec![1; 3], vec![1; 94], vec![0; 94]].concat();
    let r = mcnemar_exact(&a, &b);
    println!("{{'b': {}, 'c': {}, 'n_discordant': {}, 'p': {:?}}}", r.b, r.c, r.n_discordant, r.p);
    // {'b': 9, 'c': 3, 'n_discordant': 12, 'p': 0.146}   nine to three is not enough
}
```

The paired bootstrap gives an interval on the difference itself. The one line that matters is resampling *task indices* and reading both models at each index.

python

```python
import random

def paired_bootstrap(a: list[int], b: list[int], iters: int = 10_000, seed: int = 0) -> dict:
    """CI for the difference in pass rate between two models on the SAME tasks."""
    assert len(a) == len(b)          # misaligned lists silently produce garbage
    n = len(a)
    rng = random.Random(seed)
    observed = sum(a) / n - sum(b) / n

    diffs = []
    for _ in range(iters):
        # Resample TASK INDICES, then read BOTH models at each index.
        # Sampling a[] and b[] independently would destroy the pairing.
        sample = [rng.randrange(n) for _ in range(n)]
        diffs.append(sum(a[i] for i in sample) / n - sum(b[i] for i in sample) / n)
    diffs.sort()
    lo, hi = diffs[int(0.025 * iters)], diffs[int(0.975 * iters)]
    p_val = 2 * min(sum(d <= 0 for d in diffs), sum(d >= 0 for d in diffs)) / iters
    return {"observed": observed, "ci95": (lo, hi), "p": min(1.0, p_val)}
```

rust

```rust
// Small inline PRNG (SplitMix64); its draws differ from Python's random module.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn randrange(&mut self, n: usize) -> usize {
        ((self.next_u64() >> 11) as f64 / (1u64 << 53) as f64 * n as f64) as usize
    }
}

struct PairedBootstrap {
    observed: f64,
    ci95: (f64, f64),
    p: f64,
}

/// CI for the difference in pass rate between two models on the SAME tasks.
fn paired_bootstrap(a: &[u8], b: &[u8], iters: usize, seed: u64) -> PairedBootstrap {
    assert_eq!(a.len(), b.len());        // misaligned lists silently produce garbage
    let n = a.len();
    let nf = n as f64;
    let mut rng = Rng(seed);
    let rate = |x: &[u8]| x.iter().map(|&v| v as f64).sum::<f64>() / nf;
    let observed = rate(a) - rate(b);

    let mut diffs = Vec::with_capacity(iters);
```

```
    for _ in 0..iters {
        // Resample TASK INDICES, then read BOTH models at each index.
        // Sampling a[] and b[] independently would destroy the pairing.
        let sample: Vec<usize> = (0..n).map(|_| rng.randrange(n)).collect();
        let sa: f64 = sample.iter().map(|&i| a[i] as f64).sum();
        let sb: f64 = sample.iter().map(|&i| b[i] as f64).sum();
        diffs.push(sa / nf - sb / nf);
    }
    diffs.sort_by(f64::total_cmp);
    let (lo, hi) = (diffs[(0.025 * iters as f64) as usize], diffs[(0.975 * iters as f64) as usize]);
    let below = diffs.iter().filter(|&&d| d <= 0.0).count();
    let above = diffs.iter().filter(|&&d| d >= 0.0).count();
    let p_val = 2.0 * below.min(above) as f64 / iters as f64;
    PairedBootstrap { observed, ci95: (lo, hi), p: p_val.min(1.0) }
}
```

Worked problem

Two models, 200 shared tasks. Ship or not? Model A passed 124 of 200. Model B passed 130 of 200. Product wants to ship B.

Wilson intervals: A is 0.551 to 0.684, B is 0.582 to 0.713. They overlap heavily, and the instinct is "no significant difference". That instinct uses the wrong method. The tasks were shared, so treating the models as independent throws away most of the power.

The right analysis looks at the discordant pairs. Suppose the models agreed on 170 tasks and split the other 30.

python

```python
from math import comb

def mcnemar_exact(b, c):
    """b = tasks B won and A lost. c = the reverse."""
    n = b + c
    if n == 0:
        return 1.0
    k = min(b, c)
    tail = sum(comb(n, i) for i in range(k + 1)) / (2 ** n)
    return min(1.0, 2 * tail)

print(mcnemar_exact(18, 12))   # 0.3616  not significant
print(mcnemar_exact(6, 0))     # 0.0312  significant
```

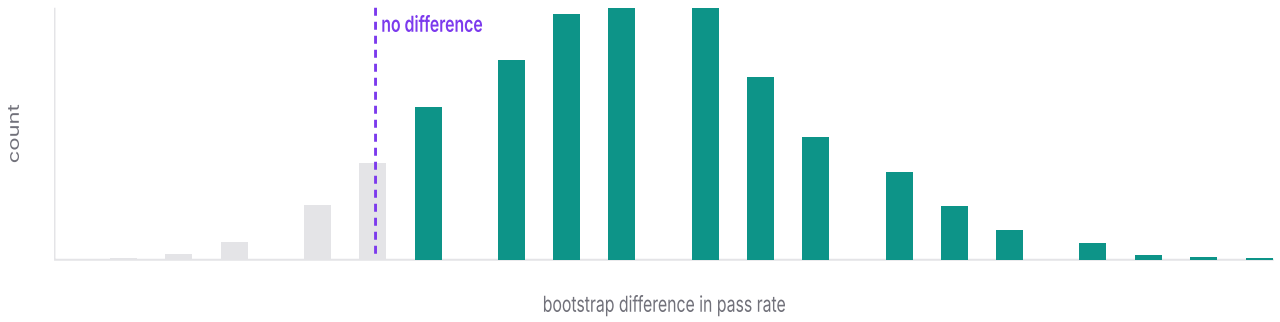
rust

```rust
/// b = tasks B won and A lost. c = the reverse.
fn mcnemar_exact(b: u32, c: u32) -> f64 {
    let n = b + c;
    if n == 0 {
        return 1.0;
    }
    let k = b.min(c);
    // sum of C(n, i) for i in 0..=k, in exact integers (fine up to n = 120)
    let (mut comb, mut sum) = (1u128, 1u128);
    for i in 1..=k {
        comb = comb * (n - i + 1) as u128 / i as u128;
        sum += comb;
    }
    let tail = sum as f64 / 2f64.powi(n as i32);
    (2.0 * tail).min(1.0)
}

fn main() {
    println!("{:?}", mcnemar_exact(18, 12));   // 0.3616  not significant
    println!("{:?}", mcnemar_exact(6, 0));     // 0.0312  significant
}
```

Now suppose instead the models disagreed on only 6 tasks and B won all 6. B still beats A by 6 tasks, so the headline is the same 124 versus 130, but now p = 0.03. Same 3-point headline, opposite conclusions: 18 to 12 is noise, 6 to 0 is real. The pass rates alone cannot tell you which world you are in.

Interactive **Paired comparison, live** (shown at its default setting)

Two models on the same tasks. Only the disagreements carry information. Set agreement to 0.94 and the effect to 0.03 and watch a convincing looking win count fail to reach significance.



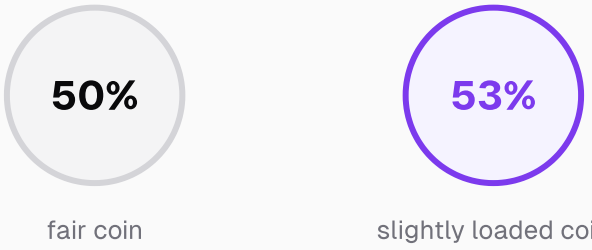
A wins (b)	B wins (c)
6	2
both pass	both fail
106	86

Discordant pairs: 8 of 200 tasks. Everything else carries no information.
Exact McNemar p = 0.2891 **bootstrap 95% CI** [-0.0050, 0.0500]
Observed difference 0.0200. **Not significant.** This difference is consistent with noise, however convincing the raw win count looks.

3. Power: the smallest difference you can see

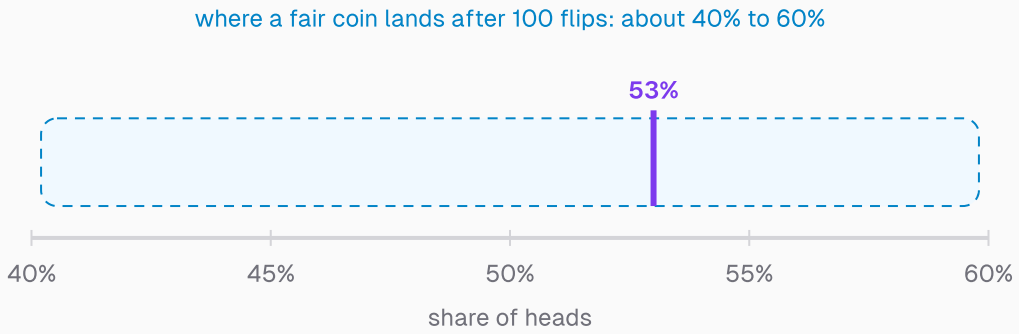
Diagram **How many flips to catch a loaded coin**

Power and sample size, with a coin that is only slightly unfair.

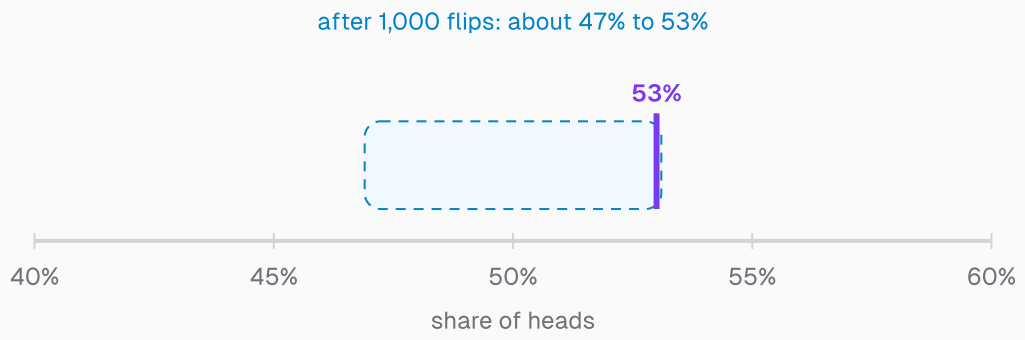


They look identical. How many flips before you can tell?

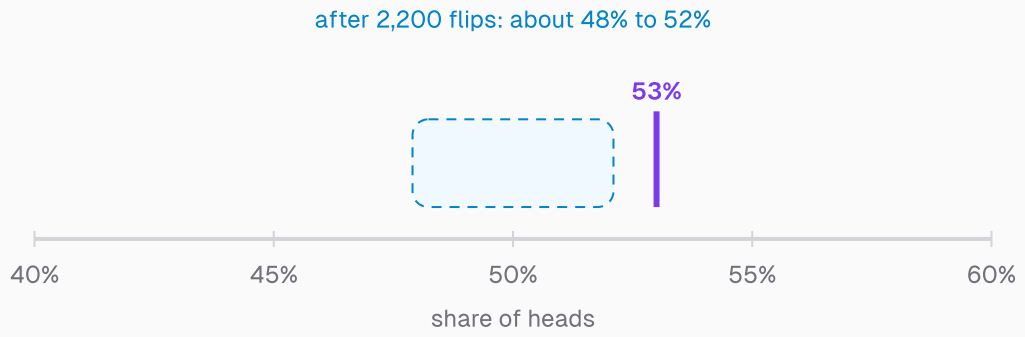
A fair coin lands heads 50% of the time. A slightly loaded one lands heads 53%. That 3-point gap is the size of a typical model improvement. The question is how many flips it takes to see it.



After 100 flips, a fair coin can easily land anywhere from about 40% to 60% heads. The loaded coin's 53% sits deep inside that band, so 100 flips cannot separate them. Neither can 100 eval tasks.

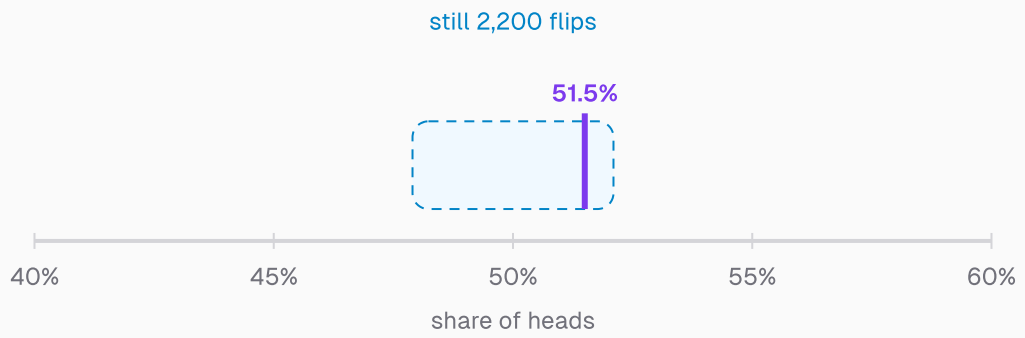


After 1,000 flips the band shrinks to about 47% to 53%. The loaded coin sits right at the edge. Sometimes you would catch it, often you would not.



Caught about 80% of the time: that is "80% power".

At about 2,200 flips the band is roughly 48% to 52%, and a 53% coin lands outside it about 80% of the time. That is what "80% power" means: the chance your test notices a difference that is really there.



Halve the gap and you need four times the flips: about 8,700.

Make the coin only 51.5% loaded and it hides inside the band again. Halving the gap you want to detect quadruples the flips needed, to about 8,700. Small improvements are expensive to prove. Pairing (the chefs story above) is the main way evals cut that cost.

Plain words

A blurry camera

A small eval set is a blurry camera. Big differences show up; small ones disappear into the blur. **Power** is the chance your eval detects a real improvement of a given size. The **minimum detectable effect** is the smallest improvement you could reliably see. If your camera can only see 5-point changes, it will report "no difference" for a real 2-point gain.

For leaders

Decide the effect size before you run

- **Ask what size of change matters, then size the suite.** If the team cares about 2 points, a 200-task suite cannot answer the question. Rule of thumb: a 2-point difference near a 50% base rate needs on the order of 1,000 to 4,000 paired tasks, depending on how often the models disagree.
- **Underpowered evals fail in two ways.** They miss real gains, and the gains they do "find" are inflated, because only lucky draws clear the bar. That second effect is the winner's curse, and it is why a surprising win on a small suite often shrinks on re-run.
- **"No significant difference" from an underpowered test is not evidence of no difference.** Ask for the minimum detectable effect alongside every null result.

Under the hood

Power by simulation

Simulation is the most defensible way to answer "how many tasks?", because the only assumptions are the ones you write into the simulator: base rate, effect, and how correlated the two models are per task.

python

```python
import random

def mde_by_simulation(base_rate: float, n_tasks: int, correlation: float = 0.8,
                      alpha: float = 0.05, power_target: float = 0.8,
                      trials: int = 2000, seed: int = 0) -> float:
    """Smallest true effect detectable at 80% power on n_tasks paired examples.

    correlation approximates how much the two models agree per task. Higher
    correlation gives a tighter paired comparison and a smaller detectable effect.
    """
    rng = random.Random(seed)

    def power_at(effect: float) -> float:
        hits = 0
        for _ in range(trials):
            a, b = [], []
            for _ in range(n_tasks):
                pa = base_rate + effect
                if rng.random() < correlation:          # shared difficulty draw
                    u = rng.random()
                    a.append(1 if u < pa else 0)
                    b.append(1 if u < base_rate else 0)
                else:                                   # independent draws
                    a.append(1 if rng.random() < pa else 0)
                    b.append(1 if rng.random() < base_rate else 0)
            if mcnemar_exact(a, b)["p"] < alpha:
                hits += 1
        return hits / trials

    lo, hi = 0.0, 0.30
    for _ in range(12):                         # binary search on the effect size
        mid = (lo + hi) / 2
        if power_at(mid) >= power_target:
            hi = mid
        else:
            lo = mid
    return hi
```

rust

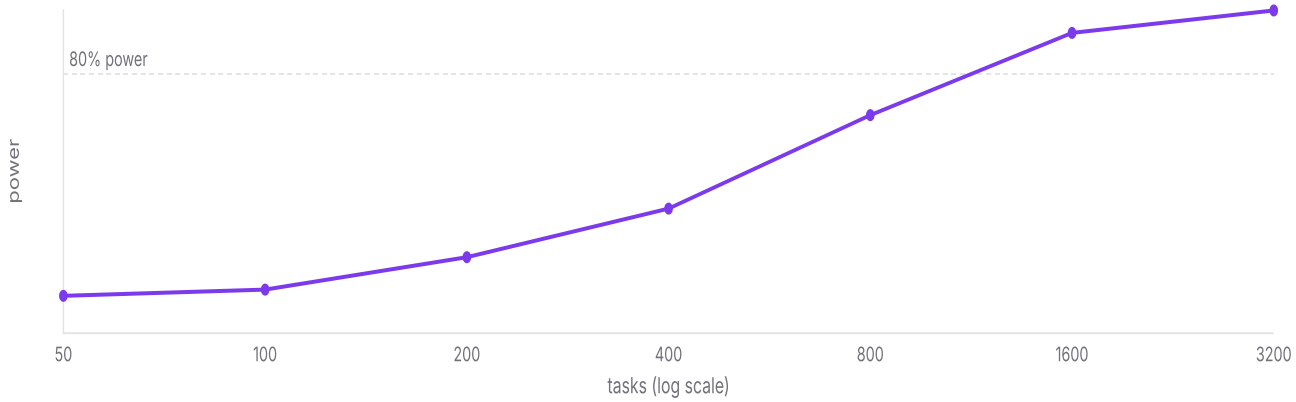
```rust
// Small inline PRNG (SplitMix64); its draws differ from Python's random module,
// so the effect this finds can differ from the Python version in the third digit.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
```

```
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}

// The exact McNemar p value from the paired-test snippet above, repeated so
// this one compiles on its own (summed in log space, so any n works).
fn mcnemar_exact_p(a: &[u8], b: &[u8]) -> f64 {
    let b_count = a.iter().zip(b).filter(|&(&x, &y)| x == 1 && y == 0).count();
    let c_count = a.iter().zip(b).filter(|&(&x, &y)| x == 0 && y == 1).count();
    let n = b_count + c_count;
    if n == 0 {
        return 1.0;
    }
    // tail = P(X <= min(b, c)) for X ~ Binomial(n, 1/2), summed in log space
    let mut log_term = -(n as f64) * std::f64::consts::LN_2;
    let mut tail = log_term.exp();
    for i in 1..=b_count.min(c_count) {
        log_term += ((n - i + 1) as f64 / i as f64).ln();
        tail += log_term.exp();
    }
    (2.0 * tail).min(1.0)
}

/// Smallest true effect detectable at 80% power on n_tasks paired examples.
///
/// correlation approximates how much the two models agree per task. Higher
/// correlation gives a tighter paired comparison and a smaller detectable effect.
fn mde_by_simulation(base_rate: f64, n_tasks: usize, correlation: f64,
                     alpha: f64, power_target: f64, trials: usize, seed: u64) -> f64 {
    let mut rng = Rng(seed);

    let mut power_at = |effect: f64| -> f64 {
        let mut hits = 0;
        for _ in 0..trials {
            let (mut a, mut b) = (Vec::with_capacity(n_tasks), Vec::with_capacity(n_tasks));
            for _ in 0..n_tasks {
                let pa = base_rate + effect;
                if rng.random() < correlation {          // shared difficulty draw
                    let u = rng.random();
                    a.push((u < pa) as u8);
                    b.push((u < base_rate) as u8);
                } else {                                 // independent draws
                    a.push((rng.random() < pa) as u8);
                    b.push((rng.random() < base_rate) as u8);
                }
            }
            if mcnemar_exact_p(&a, &b) < alpha {
                hits += 1;
            }
        }
        hits as f64 / trials as f64
    };

    let (mut lo, mut hi) = (0.0, 0.30);
    for _ in 0..12 {                         // binary search on the effect size
        let mid = (lo + hi) / 2.0;
        if power_at(mid) >= power_target {
            hi = mid;
        } else {
```

```
            lo = mid;
        }
    }
    hi
}
```

Interactive **Power and sample size** (shown at its default setting)

How many tasks do you actually need. Model agreement is the slider people forget: the more two models agree, the fewer discordant pairs carry information, and the more tasks you need to separate them. Drag it up and watch the curve collapse.



Tasks needed for 80% power: ≈ 1600

Raise model agreement and the pairing helps you. Shrink the effect and cost explodes: power scales with the square of the effect.

Principal asks

Set the effect to 0.02 and agreement to 0.9. The curve does not reach 80 percent power even at 3,200 tasks. That is the quantitative case for generating tasks where models actually differ instead of growing a static suite.

4. Sample size, stated plainly

For leaders **Why converging models get expensive to compare**

A paired test only learns from the tasks where models disagree. As frontier models converge on a fixed suite, they disagree on fewer tasks, and the suite has to grow just to keep the same resolution. At a fixed win share among disagreements, halving the disagreement rate doubles the tasks you need. This is the economic argument for dynamic benchmark generation: build tasks where current models differ, instead of adding more tasks they all pass.

Use the closed form below for a quick answer in a meeting, and the simulation in the previous section when the number has to be defended.

Under the hood

Closed-form sample size for a paired binary comparison

Let p_d be the fraction of tasks where the models disagree and π the share of those disagreements the better model wins. Testing $\pi = \frac{1}{2}$ on the discordant pairs needs

$$n_d = \frac{\left(z_{1-\alpha/2} \cdot \frac{1}{2} + z_{1-\beta}\sqrt{\pi(1-\pi)}\right)^2}{\left(\pi - \frac{1}{2}\right)^2}, \qquad n = \frac{n_d}{p_d}$$

The difference in pass rates is $p_d(2\pi - 1)$. For 95% confidence and 80% power, $z_{1-\alpha/2} = 1.96$ and $z_{1-\beta} = 0.84$.

python

```python
import math
from statistics import NormalDist

def paired_sample_size(p_discordant: float, win_share: float = 0.7,
                       alpha: float = 0.05, power: float = 0.8) -> int:
    """Tasks needed for a paired binary comparison.

    p_discordant: fraction of tasks where the two models disagree.
    win_share:    of those disagreements, the fraction the better model wins.
    """
    z_a = NormalDist().inv_cdf(1 - alpha / 2)
    z_b = NormalDist().inv_cdf(power)
    d = win_share - 0.5
    if d <= 0:
        return 10 ** 9
    n_discordant = (0.5 * z_a + z_b * math.sqrt(win_share * (1 - win_share))) ** 2 / d ** 2
    return math.ceil(n_discordant / p_discordant)


print(paired_sample_size(0.12, 0.70))   # 389   models disagree on 12% of tasks
print(paired_sample_size(0.04, 0.70))   # 1166  same models after they converge
```

rust

```rust
// Horner evaluation, highest coefficient first.
fn poly(coeffs: &[f64], r: f64) -> f64 {
    coeffs.iter().fold(0.0, |acc, &c| acc * r + c)
}

// Inverse standard normal CDF: Wichura's AS241, the algorithm behind
// Python's NormalDist().inv_cdf, so the results agree to the last bit.
fn inv_cdf(p: f64) -> f64 {
    let q = p - 0.5;
    if q.abs() <= 0.425 {
        let r = 0.180625 - q * q;
        let num = poly(&[2.5090809287301226727e+3, 3.3430575583588128105e+4, 6.7265770927008700853e+4,
                         4.5921953931549871457e+4, 1.3731693765509461125e+4, 1.9715909503065514427e+3,
                         1.3314166789178437745e+2, 3.3871328727963666080e+0], r) * q;
        let den = poly(&[5.2264952788528545610e+3, 2.8729085735721942674e+4, 3.9307895800092710610e+4,
                         2.1213794301586595867e+4, 5.3941960214247511077e+3, 6.8718700749205790830e+2,
                         4.2313330701600911252e+1, 1.0], r);
        return num / den;
    }
    let r = if q <= 0.0 { p } else { 1.0 - p };
    let r = (-r.ln()).sqrt();
    let (num, den) = if r <= 5.0 {
        let r = r - 1.6;
        (poly(&[7.7454501427834140764e-4, 2.2723844989269184583e-2, 2.4178072517745061177e-1,
                1.2704582524523683826e+0, 3.6478483247632046050e+0, 5.7694972214606914055e+0,
                4.6303378461565452959e+0, 1.4234371107496835773e+0], r),
         poly(&[1.0507500716444168432e-9, 5.4759380849953449460e-4, 1.5198666563616457197e-2,
                1.4810397642748007459e-1, 6.8976733498510000455e-1, 1.6763848301838038494e+0,
                2.0531916266377588219e+0, 1.0], r))
    } else {
        let r = r - 5.0;
        (poly(&[2.0103343992922881327e-7, 2.7115555687434875782e-5, 1.2426609473880784386e-3,
                2.6532189526576123093e-2, 2.9656057182850489123e-1, 1.7848265399172913358e+0,
                5.4637849111641143699e+0, 6.6579046435011037772e+0], r),
         poly(&[2.0442631033899397856e-15, 1.4215117583164458887e-7, 1.8463183175100546818e-5,
                7.8686913114561325910e-4, 1.4875361290850614853e-2, 1.3692988092273580531e-1,
                5.9983220655588793769e-1, 1.0], r))
    };
    if q < 0.0 { -num / den } else { num / den }
}

/// Tasks needed for a paired binary comparison.
///
/// p_discordant: fraction of tasks where the two models disagree.
/// win_share:    of those disagreements, the fraction the better model wins.
fn paired_sample_size(p_discordant: f64, win_share: f64, alpha: f64, power: f64) -> u64 {
    let z_a = inv_cdf(1.0 - alpha / 2.0);
    let z_b = inv_cdf(power);
    let d = win_share - 0.5;
    if d <= 0.0 {
        return 1_000_000_000;
    }
    let n_discordant = (0.5 * z_a + z_b * (win_share * (1.0 - win_share)).sqrt()).powi(2) / d.powi(2);
    (n_discordant / p_discordant).ceil() as u64
}

fn main() {
    println!("{}", paired_sample_size(0.12, 0.70, 0.05, 0.8));   // 389   models disagree on 12% of tasks
```

```
    println!("{}", paired_sample_size(0.04, 0.70, 0.05, 0.8));   // 1166  same models after they converge
}
```

Worked problem

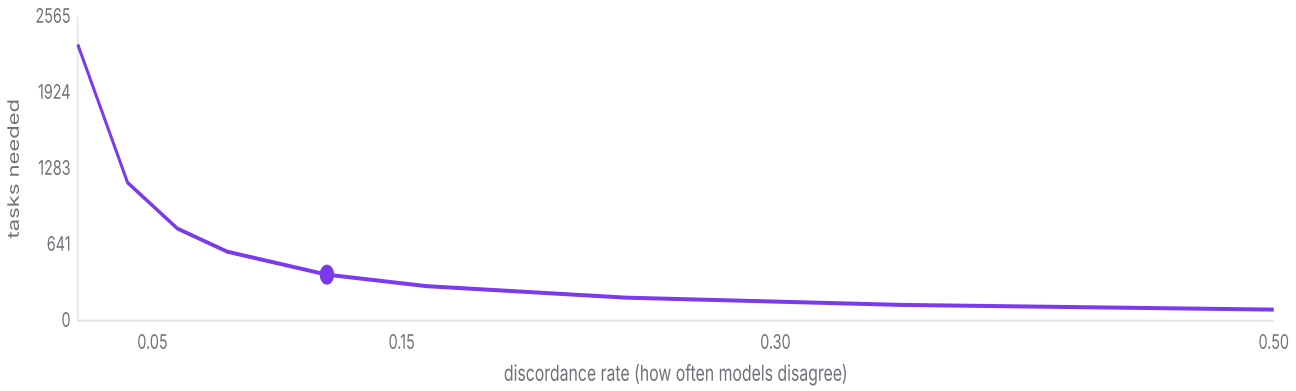
How many tasks do I need? A product team wants to detect an improvement of about 2 to 3 points.

Disagreement rate	Win share	Pass-rate gap	Tasks needed
0.12	0.62	2.9 pts	**1,116**
0.12	0.58	1.9 pts	**2,536**
0.06 (models have converged)	0.62	1.4 pts	**2,232**

A 2-point move needs over a thousand tasks, not two hundred. That is the honest answer to most sample size questions. And at a fixed win share, halving the disagreement rate doubles the requirement.

Interactive **Sample size calculator** (shown at its default setting)

The reciprocal curve that makes converging models so expensive to compare. Drag the discordance rate down toward 0.04 and watch the required task count run away.



Tasks needed: 389

The curve is a reciprocal: halving the discordance rate doubles the tasks required. As frontier models converge on a fixed task set, discordance falls and this number runs away from you. Generating tasks where models actually differ is the only thing that pushes back.

5. Many benchmarks at once

Diagram **Forty scratch cards**

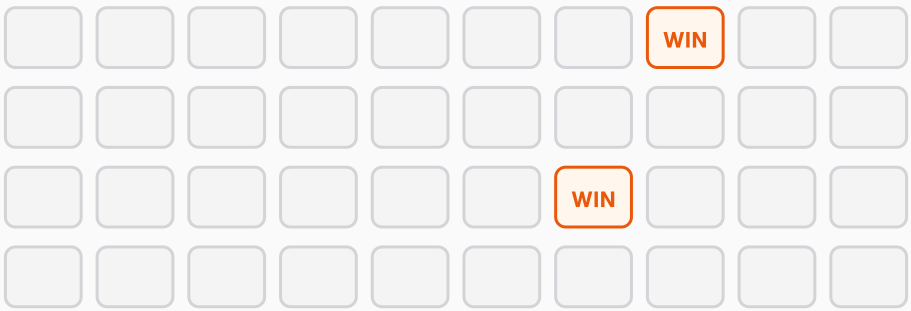
Why a big benchmark suite always shows a few "significant" wins.

40 benchmarks, and the new model changed none of them

Each card: did this benchmark "improve significantly"?

A new model runs on a standing suite of 40 benchmarks. Suppose, for now, that it is no better or worse on any of them. Each benchmark is a scratch card that might say "significant win".

With a 0.05 threshold, each card has a 1 in 20 chance of a fake win



40 × 0.05 = 2 fake wins on average, from pure luck.

Testing at p < 0.05 means each card has a 1 in 20 chance of a fake win even when nothing changed. Across 40 cards, expect about 2 fake wins every time you run the suite.

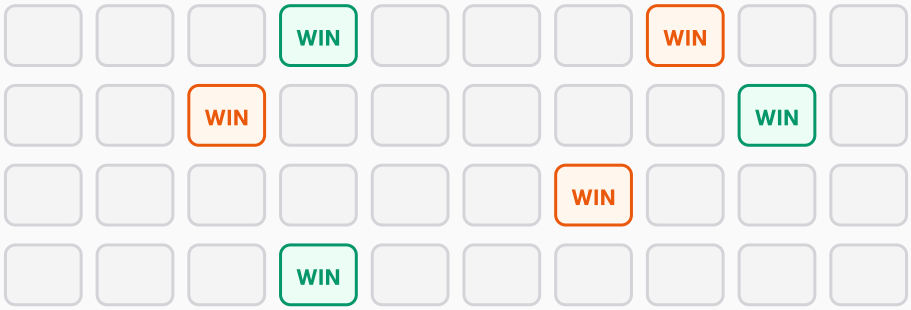
87%

chance that at least one of 40 null benchmarks lights up

$1 - 0.95^{40} \approx 0.87$

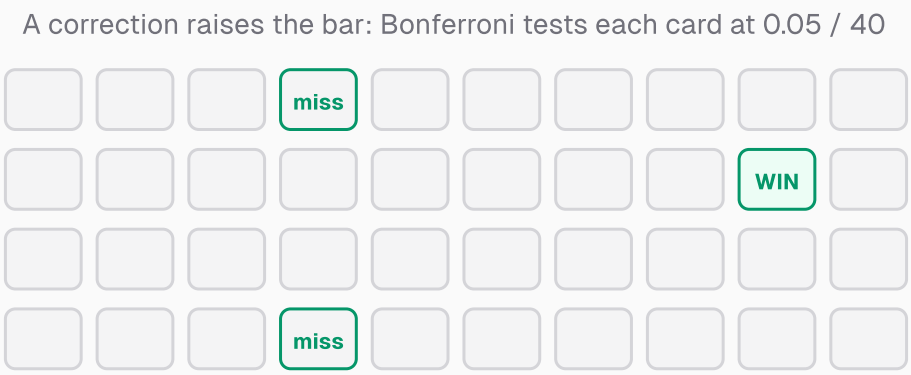
The chance that at least one card lights up is about 87%. A big standing suite will almost always show something "significant", which is why a list of wins with no correction is a red flag.

Now 3 benchmarks really improved (green) among 37 that did not



Six wins. From the outside, the real ones and the lucky ones look the same.

Now suppose 3 benchmarks genuinely improved. Without correction you see 6 wins: the 3 real ones and 3 lucky ones. This matches the worked problem below. Nothing on the card tells you which kind it is.



No fake wins left, but two real wins were missed too.

A correction raises the bar for every card. The lucky wins disappear, and so do two of the real ones. There is no free option, only a choice about which mistake costs less: for regression monitoring, a missed regression usually costs more than a false alarm.

Plain words

Forty coin flips

Flip 40 fair coins five times each and a couple will land heads every time, by luck. Run 40 benchmarks on a new model and a couple will "improve significantly" by luck too. The more things you test, the more false wins you get. A **multiple comparisons correction** raises the bar to account for how many things you looked at.

For leaders

Pick which error you would rather make

- At a 0.05 threshold, 30 benchmarks with no real change produce about 1.5 false "significant" results on average. A big standing suite will always have something lighting up.
- Corrections are not free. They cut false alarms and also miss real effects. There is no option without a cost, only a choice about which error is cheaper.
- **The asymmetry to say out loud:** for regression monitoring, a missed regression ships a worse model while a false alarm costs an engineer a day. So monitoring deliberately accepts more false alarms than a publication claim would. For a launch announcement, the reverse.
- **Red flag:** a deck that lists "6 of 40 benchmarks improved" with no correction and no note of how many got worse.

Under the hood

Bonferroni and Benjamini–Hochberg

Bonferroni controls the chance of any false positive (the family-wise error rate) by testing each of m hypotheses at α/m. It is simple and very conservative on large suites.

Benjamini–Hochberg controls the false discovery rate, the expected share of discoveries that are false. Sort the p values $p_{(1)} \le \cdots \le p_{(m)}$ and find

$$k^* = \max\left\{k : p_{(k)} \le \frac{k}{m}\, q\right\}$$

Reject the hypotheses with the k^* smallest p values, including any below k^* that individually failed the check. The threshold loosens as rank grows, which keeps power on a large suite.

python

```python
def benjamini_hochberg(p_values: list[float], fdr: float = 0.05) -> list[bool]:
    """Control the false discovery rate across a suite of benchmarks."""
    m = len(p_values)
    order = sorted(range(m), key=lambda i: p_values[i])   # indices, ascending p

    # PASS ONE: the largest rank whose p value clears rank/m * fdr.
    # Scan ALL ranks; a later rank can pass after an earlier one fails.
    max_rank = -1
    for rank, i in enumerate(order, start=1):
        if p_values[i] <= fdr * rank / m:
            max_rank = rank

    # PASS TWO: everything up to that rank is a discovery (the step-up rule,
    # and the part people most often implement wrong).
    keep = [False] * m
    for rank, i in enumerate(order, start=1):
        if rank <= max_rank:
            keep[i] = True
    return keep
```

rust

```rust
/// Control the false discovery rate across a suite of benchmarks.
fn benjamini_hochberg(p_values: &[f64], fdr: f64) -> Vec<bool> {
    let m = p_values.len();
    let mut order: Vec<usize> = (0..m).collect();          // indices, ascending p
    order.sort_by(|&i, &j| p_values[i].total_cmp(&p_values[j]));

    // PASS ONE: the largest rank whose p value clears rank/m * fdr.
    // Scan ALL ranks; a later rank can pass after an earlier one fails.
    let mut max_rank = 0;
    for (rank, &i) in (1..).zip(&order) {
        if p_values[i] <= fdr * rank as f64 / m as f64 {
            max_rank = rank;
        }
    }

    // PASS TWO: everything up to that rank is a discovery (the step-up rule,
    // and the part people most often implement wrong).
```

```
    let mut keep = vec![false; m];
    for (rank, &i) in (1..).zip(&order) {
        if rank <= max_rank {
            keep[i] = true;
        }
    }
    keep
}
```

Worked problem

Forty benchmarks, six moved. Which do you believe? A new model lands, the standing suite of 40 benchmarks runs, and six come back with p below 0.05. One illustrative simulation where exactly three benchmarks genuinely moved:

Method	Real found	False alarms	Real missed
No correction	3	**3**	0
Benjamini–Hochberg	1	0	**2**
Bonferroni	1	0	**2**

Half the uncorrected "discoveries" were noise. Both corrections removed them and also missed two real effects. The right choice depends on the loss function, which is a product decision as much as a statistical one.

Interactive **Multiple comparisons, live** (shown at its default setting)

Run a whole suite against a new model and see how many benchmarks light up by luck. Compare no correction, Benjamini Hochberg and Bonferroni on the same draw.

Run 40 benchmarks where only 3 truly moved.

No correction: 3 real, **3 false alarms**, 0 missed.

Benjamini Hochberg: 0 real, 0 false, 3 missed.

Bonferroni: 0 real, 0 false, **3 missed**.

Bonferroni controls the chance of ANY false alarm and pays for it in missed regressions. For a standing suite that is the wrong trade: a missed regression ships, a false alarm costs an engineer a day.

6. pass@k and sampling variance

Diagram **Best of three free throws**

pass@1, pass@k, and pass^k with one basketball player.

A player who makes 40% of free throws

One shot each time: success 40% of the time

pass@1 = 0.40

A player makes 40% of free throws. Give them one shot and they succeed 40% of the time. For a model, that is pass@1: how often the first answer is right.

Three shots, and you only need one to go in

All three miss: 0.6 × 0.6 × 0.6 = 0.216

pass@3 = 1 – 0.216 ≈ 0.78

Now give them three shots and count it a success if any go in. They only fail if all three miss, which happens 21.6% of the time, so pass@3 is about 78%. This is pass@k, and it only matters if something can pick out the good attempt, like unit tests for code.

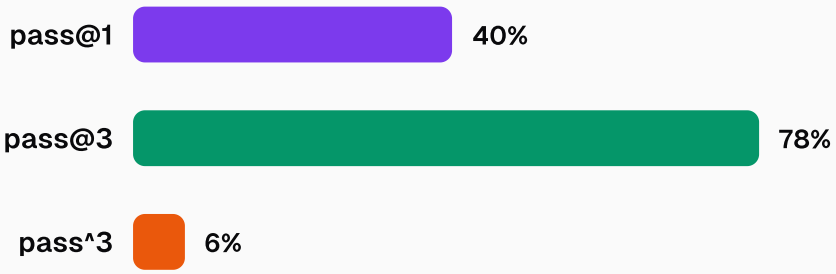
Three shots, and all three must go in

0.4 × 0.4 × 0.4 = 0.064

pass^3 ≈ 0.06

Flip the rule: all three must go in. That happens only 6.4% of the time. This is pass^k, a reliability measure. An agent that succeeds once and fails the next time is a problem pass@1 hides.

Same player, three different numbers



Ask which number matches how the product is used.

One player, one skill level, three honest numbers: 40%, 78% and 6%. A pass@k quoted next to a competitor's pass@1 is not a comparison. Always ask how many attempts, at what temperature, and whether the product can actually pick the best one.

Plain words

How many tries does the model get?

A model with some randomness can answer the same question differently each time. **pass@1** asks how often its first try is right. **pass@k** asks how often at least one of k tries is right, which matters

when something can check the answers (like running unit tests on code). The opposite question, how often *all* k tries are right, measures reliability.

For leaders

Match the metric to the product

- **pass@1** is what a user sees when they get one answer. **pass@k** is relevant only if the product samples several times and has a way to pick the right one, such as tests or a verifier.
- **pass^k** (all k attempts succeed, popularised by the τ-bench agent benchmark, Yao et al. 2024) measures consistency. Agents that pass once and fail the next time are a reliability problem pass@1 hides.
- Ask what temperature and how many samples produced a number. A pass@k quoted next to a competitor's pass@1 is not a comparison.

Under the hood

The unbiased pass@k estimator

Sample $n \geq k$ completions per task and count the c that pass. The unbiased estimator from the Codex paper (Chen et al. 2021) is the probability that a random subset of k contains at least one pass, averaged over tasks:

$$\widehat{\text{pass@}k} = \mathbb{E}_{\text{tasks}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

Sampling exactly k and checking is unbiased but high variance; sampling n and plugging c/n into $1 - (1 - c/n)^k$ is biased. The combinatorial form uses all n samples without bias.

python

```python
import numpy as np

def pass_at_k(n: int, c: int, k: int) -> float:
    """Unbiased estimate of pass@k given n samples of which c passed."""
    # Fewer than k failures: any k samples include a pass.
    if n - c < k:
        return 1.0
    # 1 - C(n-c, k)/C(n, k), computed as a product so large n does not overflow.
    return 1.0 - float(np.prod(1.0 - k / np.arange(n - c + 1, n + 1)))


def suite_pass_at_k(results: dict[str, tuple[int, int]], k: int) -> float:
    """results maps task_id -> (n_samples, n_passed). Average PER TASK,
    so one heavily sampled task cannot dominate the number."""
    return sum(pass_at_k(n, c, k) for n, c in results.values()) / len(results)
```

rust

```rust
use std::collections::HashMap;

/// Unbiased estimate of pass@k given n samples of which c passed.
fn pass_at_k(n: u64, c: u64, k: u64) -> f64 {
    // Fewer than k failures: any k samples include a pass.
    if n - c < k {
        return 1.0;
    }
    // 1 - C(n-c, k)/C(n, k), computed as a product so large n does not overflow.
    1.0 - (n - c + 1..=n).map(|i| 1.0 - k as f64 / i as f64).product::<f64>()
}

/// results maps task_id -> (n_samples, n_passed). Average PER TASK,
/// so one heavily sampled task cannot dominate the number.
fn suite_pass_at_k(results: &HashMap<String, (u64, u64)>, k: u64) -> f64 {
    results.values().map(|&(n, c)| pass_at_k(n, c, k)).sum::<f64>() / results.len() as f64
}
```

7. More than two models, and where the noise comes from

Plain words

A tournament, then a referee check

With three or more models, first ask whether they differ at all, then which pairs differ. Separately, it helps to know where the wobble in your scores comes from: are the tasks themselves very different in difficulty, or does the same task give different results each time you run it? That answer tells you whether to buy more tasks or more runs.

For leaders

More tasks or more samples?

This is a budgeting question that teams often guess at. Variance components answer it with a number. In one illustrative measurement on 60 tasks with 5 runs each, doubling the tasks reduced the standard error from 0.041 to 0.029, while doubling the runs per task only got it to 0.037, for the same compute. Task variance is divided only by the number of tasks; run-to-run noise is divided by tasks times runs. So once you have a few runs per task, new tasks are almost always the better buy.

Plain one-way ANOVA is usually the wrong tool for evals, because outcomes are binary and the design is paired. Ask for the paired version from the table below.

Under the hood

Cochran's Q, ANOVA, and variance components

Situation	Test	Why
2 models, binary, same tasks	McNemar exact	Paired, exact on small discordant counts
3+ models, binary, same tasks	Cochran's Q	The paired multi-group extension of McNemar
3+ models, continuous or ordinal scores	Friedman	Ranks within task, no distribution assumption
3+ models, continuous, independent groups	One-way ANOVA	Only when tasks are not shared, which is rare in evals
Score by model and task category	Two-way ANOVA	Separates model effect from task difficulty and their interaction
How much noise comes from where	Variance components	Tells you whether to add tasks or add samples

Cochran's Q

With k models, G_j the passes for model j, and L_i the passes on task i:

$$Q = (k-1)\,\frac{k\sum_j G_j^2 - \left(\sum_j G_j\right)^2}{k\sum_i L_i - \sum_i L_i^2} \;\sim\; \chi^2_{k-1} \text{ under } H_0$$

Tasks every model passed ($L_i = k$) or failed ($L_i = 0$) contribute nothing to the denominator: the same "only disagreement counts" logic as McNemar.

python

```python
import random

def cochran_q(X: list[list[int]]) -> float:
    """X[i][j] = 1 if model j passed task i. Paired across k models."""
    n, k = len(X), len(X[0])
    G = [sum(X[i][j] for i in range(n)) for j in range(k)]   # per model totals
    L = [sum(row) for row in X]                             # per task totals
    num = (k - 1) * (k * sum(g * g for g in G) - sum(G) ** 2)
    den = k * sum(L) - sum(l * l for l in L)
    return num / den if den else 0.0    # den == 0 when every task is unanimous
```

```python
def permutation_p(X, stat_fn, iters: int = 5000, seed: int = 0) -> tuple[float, float]:
    """p value with no distribution assumptions."""
    rng = random.Random(seed)
    observed = stat_fn(X)
    hits = 0
    for _ in range(iters):
        shuffled = []
        for row in X:
            r = list(row)
            rng.shuffle(r)      # shuffle WITHIN a task: models are exchangeable
            shuffled.append(r)  # under H0, but tasks differ in difficulty
        if stat_fn(shuffled) >= observed:
            hits += 1
    return observed, (hits + 1) / (iters + 1)   # +1 avoids ever reporting p = 0
```

rust

```rust
// Small inline PRNG (SplitMix64); its draws differ from Python's random module.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn randrange(&mut self, n: usize) -> usize {
        ((self.next_u64() >> 11) as f64 / (1u64 << 53) as f64 * n as f64) as usize
    }
    // Fisher-Yates, walking down from the end like random.shuffle
    fn shuffle<T>(&mut self, v: &mut [T]) {
        for i in (1..v.len()).rev() {
            v.swap(i, self.randrange(i + 1));
        }
    }
}

/// X[i][j] = 1 if model j passed task i. Paired across k models.
fn cochran_q(x: &[Vec<u8>]) -> f64 {
    let (n, k) = (x.len(), x[0].len() as i64);
    let g: Vec<i64> = (0..x[0].len()).map(|j| (0..n).map(|i| x[i][j] as i64).sum()).collect(); // per model
totals
    let l: Vec<i64> = x.iter().map(|row| row.iter().map(|&v| v as i64).sum()).collect();       // per task totals
    let g_sum: i64 = g.iter().sum();
    let num = (k - 1) * (k * g.iter().map(|v| v * v).sum::<i64>() - g_sum * g_sum);
    let den = k * l.iter().sum::<i64>() - l.iter().map(|v| v * v).sum::<i64>();
    if den != 0 { num as f64 / den as f64 } else { 0.0 }   // den == 0 when every task is unanimous
}

/// p value with no distribution assumptions.
fn permutation_p(x: &[Vec<u8>], stat_fn: impl Fn(&[Vec<u8>]) -> f64,
                 iters: usize, seed: u64) -> (f64, f64) {
    let mut rng = Rng(seed);
    let observed = stat_fn(x);
    let mut hits = 0;
    for _ in 0..iters {
        let mut shuffled = Vec::with_capacity(x.len());
```

```
        for row in x {
            let mut r = row.clone();
            rng.shuffle(&mut r);   // shuffle WITHIN a task: models are exchangeable
            shuffled.push(r);      // under H0, but tasks differ in difficulty
        }
        if stat_fn(&shuffled) >= observed {
            hits += 1;
        }
    }
    (observed, (hits + 1) as f64 / (iters + 1) as f64)   // +1 avoids ever reporting p = 0
}
```

> ⚠ **Watch out**
>
> Cochran's Q answers "are these models different at all". It does not say which. Running it and then reporting the winner is the multiple comparisons mistake in a different form. Follow a significant Q with pairwise McNemar tests and Benjamini–Hochberg across the pairs.

One-way ANOVA, for continuous scores

$$F = \frac{SS_{\text{between}}/(g-1)}{SS_{\text{within}}/(N-g)}, \qquad \eta^2 = \frac{SS_{\text{between}}}{SS_{\text{between}} + SS_{\text{within}}}$$

F is signal over noise. η^2 is the effect size: the share of variance explained by group membership. Always report η^2, because F grows with sample size and can be large for a difference nobody would notice.

python

```
def one_way_anova(groups: list[list[float]]) -> dict:
    g = len(groups)
    N = sum(len(x) for x in groups)
    grand = sum(sum(x) for x in groups) / N
    ss_between = sum(len(x) * (sum(x) / len(x) - grand) ** 2 for x in groups)   # signal
    ss_within = sum((v - sum(x) / len(x)) ** 2 for x in groups for v in x)       # noise
    df_b, df_w = g - 1, N - g
    ms_b, ms_w = ss_between / df_b, ss_within / df_w
    return {"F": ms_b / ms_w if ms_w else float("inf"), "df": (df_b, df_w),
            "eta_squared": ss_between / (ss_between + ss_within)}
```

rust

```
struct Anova {
    f: f64,
    df: (usize, usize),
    eta_squared: f64,
```

```rust
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

fn one_way_anova(groups: &[Vec<f64>]) -> Anova {
    let g = groups.len();
    let big_n: usize = groups.iter().map(|x| x.len()).sum();
    let grand = groups.iter().map(|x| x.iter().sum::<f64>()).sum::<f64>() / big_n as f64;
    let ss_between: f64 = groups.iter().map(|x| x.len() as f64 * (mean(x) - grand).powi(2)).sum();
// signal
    let ss_within: f64 = groups.iter().map(|x| { let m = mean(x); x.iter().map(|v| (v - m).powi(2)).sum::<f64>()
}).sum(); // noise
    let (df_b, df_w) = (g - 1, big_n - g);
    let (ms_b, ms_w) = (ss_between / df_b as f64, ss_within / df_w as f64);
    Anova {
        f: if ms_w != 0.0 { ms_b / ms_w } else { f64::INFINITY },
        df: (df_b, df_w),
        eta_squared: ss_between / (ss_between + ss_within),
    }
}
```

Variance components: more tasks or more samples

Run each of n tasks r times. Split the variance into a task part σ^2_{task} and a run-to-run part σ^2_{samp}. The standard error of the suite mean is

$$\text{SE} = \sqrt{\frac{\sigma^2_{\text{task}}}{n} + \frac{\sigma^2_{\text{samp}}}{n\,r}}$$

Adding runs only shrinks the second term. Adding tasks shrinks both. The ratio $\sigma^2_{\text{task}} / (\sigma^2_{\text{task}} + \sigma^2_{\text{samp}})$ is the intraclass correlation, which reappears as the design effect in the next chapter.

python

```python
from statistics import mean
import math

def variance_components(scores: list[list[float]]) -> dict:
    """scores[i] = r repeated runs of task i at identical settings."""
    n, r = len(scores), len(scores[0])
    task_means = [mean(s) for s in scores]
    grand = mean(task_means)
    # Between-task mean square, on the same scale as the within term.
    ms_between = r * sum((m - grand) ** 2 for m in task_means) / (n - 1)
    # Within-task spread: pure run-to-run noise, difficulty is held fixed.
    ms_within = sum((x - task_means[i]) ** 2 for i in range(n) for x in scores[i]) / (n * (r - 1))
    var_task = max(0.0, (ms_between - ms_within) / r)   # noise can push it negative
```

```
    var_sampling = ms_within

    def se(n_tasks, n_reps):
        return math.sqrt(var_task / n_tasks + var_sampling / (n_tasks * n_reps))

    return {"var_task": var_task, "var_sampling": var_sampling,
            "icc": var_task / (var_task + var_sampling) if var_task + var_sampling else 0.0,
            "se_current": se(n, r), "se_double_tasks": se(2 * n, r), "se_double_samples": se(n, 2 * r)}

# Illustrative, 60 tasks x 5 runs: var_task 0.063, var_sampling 0.188
#   SE now 0.0409 | double the tasks 0.0290 | double the samples 0.0369
```

rust

```
struct VarianceComponents {
    var_task: f64,
    var_sampling: f64,
    icc: f64,
    se_current: f64,
    se_double_tasks: f64,
    se_double_samples: f64,
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// scores[i] = r repeated runs of task i at identical settings.
fn variance_components(scores: &[Vec<f64>]) -> VarianceComponents {
    let (n, r) = (scores.len(), scores[0].len());
    let (nf, rf) = (n as f64, r as f64);
    let task_means: Vec<f64> = scores.iter().map(|s| mean(s)).collect();
    let grand = mean(&task_means);
    // Between-task mean square, on the same scale as the within term.
    let ms_between = rf * task_means.iter().map(|m| (m - grand).powi(2)).sum::<f64>() / (nf - 1.0);
    // Within-task spread: pure run-to-run noise, difficulty is held fixed.
    let ms_within = scores.iter().zip(&task_means)
        .map(|(s, m)| s.iter().map(|x| (x - m).powi(2)).sum::<f64>())
        .sum::<f64>() / (nf * (rf - 1.0));
    let var_task = ((ms_between - ms_within) / rf).max(0.0);   // noise can push it negative
    let var_sampling = ms_within;

    let se = |n_tasks: f64, n_reps: f64| (var_task / n_tasks + var_sampling / (n_tasks * n_reps)).sqrt();

    VarianceComponents {
        var_task,
        var_sampling,
        icc: if var_task + var_sampling != 0.0 { var_task / (var_task + var_sampling) } else { 0.0 },
        se_current: se(nf, rf),
        se_double_tasks: se(2.0 * nf, rf),
        se_double_samples: se(nf, 2.0 * rf),
    }
}

// Illustrative, 60 tasks x 5 runs: var_task 0.063, var_sampling 0.188
//   SE now 0.0409 | double the tasks 0.0290 | double the samples 0.0369
```

Principal asks

"The benchmark moved 4 points and nothing about the model changed. Why?" Usual suspects: temperature or top-p changed; the harness retried failures and quietly gave some tasks extra attempts; a timeout threshold moved; the judge prompt changed; the task set was silently filtered. See regression triage for the order to check them in.

Red flags in a results deck: a pass rate with no n; symmetric error bars near 100%; overlapping intervals used as a test; an unpaired comparison on shared tasks; "k of m benchmarks improved" with no correction and no count of regressions; a null result with no minimum detectable effect; pass@k compared with pass@1.

Part II · Chapter 9

Intervals, the bootstrap, and clustering in eval data

The previous chapter gave the tests. This chapter is the layer underneath, applied to eval metrics: what shape each metric has, what an interval promises, when the bootstrap saves you and when it quietly lies, and the clustering mistake that shrinks error bars on almost every agent eval. The general theory is in Part I, chapter 2 and the shapes themselves are in the distribution zoo.

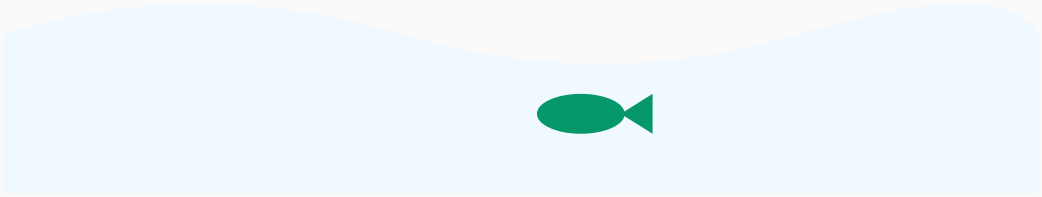
Diagram

Casting nets for a hidden fish

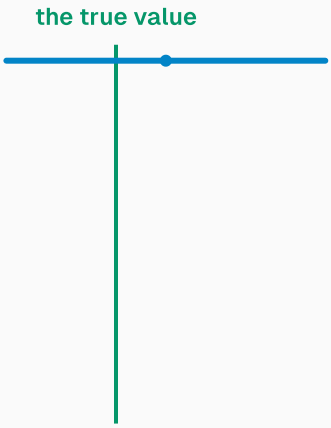
What a 95% confidence interval does and does not promise.

There is one true pass rate. You never see it directly.

Like a fish in murky water: you know it is there, not exactly where.

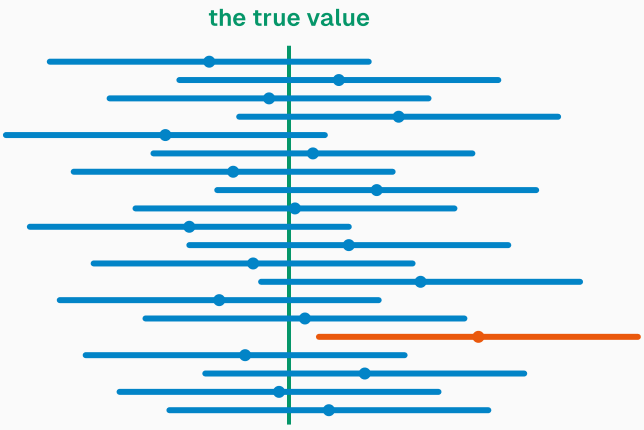


Every model has a true pass rate on the kind of tasks you care about. You never see it directly, only what one finite eval happened to show. Think of a fish in murky water.



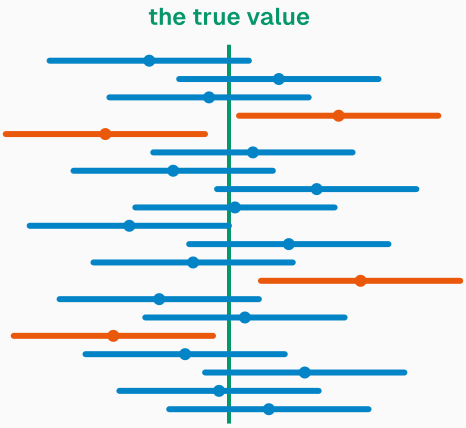
One eval run = one cast of the net

Each eval run casts a net: an interval around what you observed. This net happens to contain the fish. You cannot tell from one cast whether you caught it.



19 of 20 nets catch the fish: that is what "95%" means

Cast twenty times and about nineteen nets catch the fish; one misses. The 95% describes the casting method over many casts. It does not mean any one net has a 95% chance of holding the fish.



A narrower, overconfident net: about 16 of 20 catch

Some recipes make nets that are too small for their promise. The normal approximation at a 95% pass rate with 30 tasks catches the truth only about 78% of the time while still calling itself 95%. Checking coverage, as the Wilson widget in the previous chapter does, is how you catch this.

1. What shape is your metric?

Plain words **Different metrics, different math**

Measure the same thing many times and you get a spread of answers. The **shape** of that spread decides which math is allowed. Pass or fail behaves like coin flips. Latency is a pile on the left with a long tail to the right. Using the coin-flip tool on latency data is one of the most common mistakes in eval reporting.

For leaders

Which summary to trust

- For skewed metrics like latency and token counts, ask for the median and p95, not the mean. A single 40-second outlier moves the mean, and no user experiences the mean.
- A cheap automated guard: if mean divided by median is above about 1.2, the report should switch to quantiles. Put that check in the reporting code rather than in review comments.
- Count metrics such as tool calls per session vary far more than simple models assume. Intervals built on those assumptions are too narrow, so small changes look real.

Under the hood

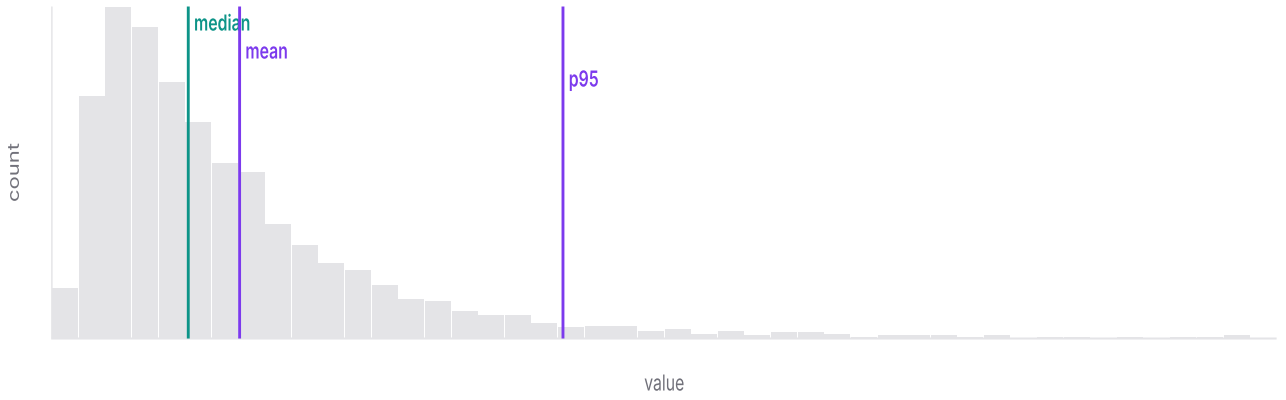
Sampling distributions converge at different speeds

The distribution that matters is the *sampling distribution* of your statistic: the spread of results you would get from different draws of tasks. The central limit theorem says sample means become normal regardless of the underlying shape. The catch is how fast. For symmetric data, quickly. For pass rates near 0 or 1, slowly. For heavy-tailed latency, slowly and from the wrong side, so the interval is wrong in a predictable direction. For statistics that are not means at all, such as p95 or the median, the theorem does not apply directly. That is where the bootstrap earns its keep.

A practical test: how far from a mean is your statistic, and how heavy is the tail? A pass rate is a mean of 0s and 1s and behaves well once n is decent and p is not extreme. p95 latency is an extreme order statistic on a heavy-tailed variable, the worst case for every closed form.

Interactive **Distribution explorer** (shown at its default setting)

Change the shape and watch which summary statistic stops being honest. Push a lognormal's skew up and see the mean detach from the median. Switch to negative binomial and watch variance over mean climb past the point where Poisson intervals are valid.



mean 2.74 **median** 1.99 **p95** 7.47 **var/mean** 2.47
skew ratio (mean/median) 1.38 — **do not report the mean.** Use median and percentiles.

Note

Set it to lognormal and drag the log standard deviation up. The skew ratio crosses 1.2 and the readout says to stop reporting the mean. That is the check to put in a reporting layer.

2. What a confidence interval promises

Plain words **A net that usually catches the fish**

A 95% confidence interval comes from a recipe that catches the true value 95 times out of 100 in the long run. It does **not** mean there is a 95% chance the truth is inside this particular range. The recipe is the thing with the 95% guarantee, and some recipes do not keep their promise.

For leaders **Nominal is not actual**

- Interval width depends on three things: how noisy the measurement is, how many samples you have, and how much confidence you demand. Only the middle one is under your control, and quadrupling the sample only halves the width.
- A "95%" interval can actually cover far less. Asking "did you check the coverage of your interval method at the rates we operate at?" is a senior question. It turns "we use Wilson" into "we verified Wilson holds up where our evals sit".

Under the hood

Measure coverage empirically

Different constructions have different real coverage. The normal approximation on a proportion falls well below 95% near a boundary; Wilson holds up. Percentile bootstrap intervals are asymptotically correct but under-cover on small samples and skewed statistics, which BCa corrects. You can check any method in a few lines:

python

```python
import random

def coverage_check(true_p: float, n: int, method, trials: int = 5000, seed: int = 0) -> float:
    """Empirically measure whether a '95%' interval covers 95% of the time.

    true p = 0.95, n = 30:   Wilson 0.94   normal approximation 0.79
    true p = 0.95, n = 50:   Wilson 0.96   normal approximation 0.93
    """
    rng = random.Random(seed)
    hits = 0
    for _ in range(trials):
        k = sum(1 for _ in range(n) if rng.random() < true_p)
        lo, hi = method(k, n)
        if lo <= true_p <= hi:
            hits += 1
    return hits / trials
```

rust

```rust
// Small inline PRNG (SplitMix64); its draws differ from Python's random module.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    fn randrange(&mut self, n: usize) -> usize {
        (self.random() * n as f64) as usize
    }
}
```

```rust
/// Empirically measure whether a '95%' interval covers 95% of the time.
///
/// true p = 0.95, n = 30:   Wilson 0.94   normal approximation 0.79
/// true p = 0.95, n = 50:   Wilson 0.96   normal approximation 0.92
/// (this PRNG's draws with trials = 5000, seed = 0)
fn coverage_check(true_p: f64, n: u64, method: impl Fn(u64, u64) -> (f64, f64),
                  trials: usize, seed: u64) -> f64 {
    let mut rng = Rng(seed);
    let mut hits = 0;
    for _ in 0..trials {
        let k = (0..n).filter(|_| rng.random() < true_p).count() as u64;
        let (lo, hi) = method(k, n);
        if lo <= true_p && true_p <= hi {
            hits += 1;
        }
    }
    hits as f64 / trials as f64
}
```

3. The bootstrap

Diagram

Marbles out of one bag

The bootstrap: measuring wobble without collecting more data.

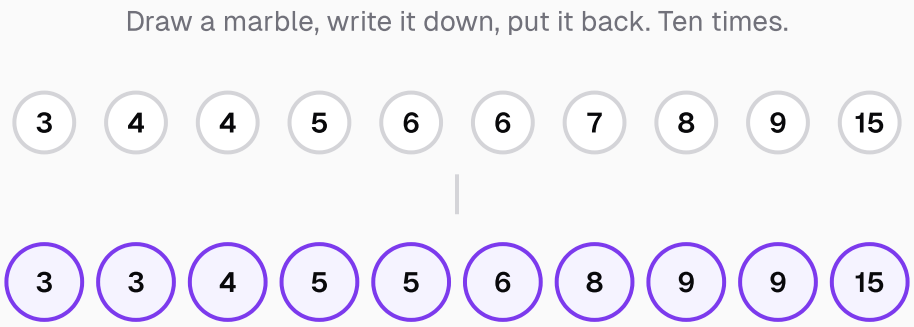
Your one sample: 10 response times, in seconds

3 4 4 5 6 6 7 8 9 15

Median = 6 s

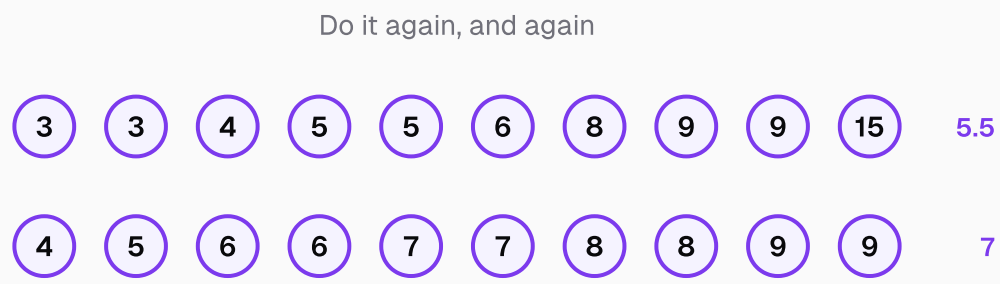
How much would that 6 wobble with a different sample?

You measured 10 response times. The median is 6 seconds. You want to know how much that would change with a different 10, but you cannot go back and collect more.



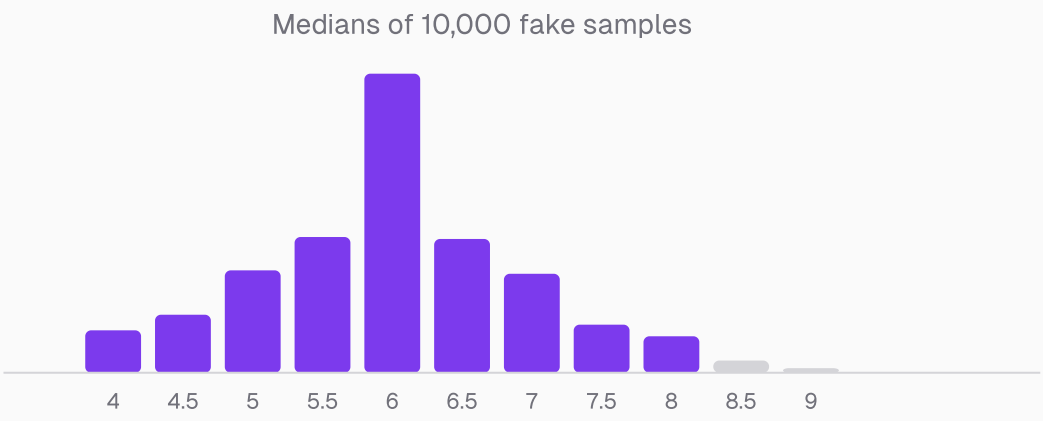
Fake sample 1: some marbles twice, some not at all. Median 5.5 s

The trick: draw from your own sample with replacement. Pick a marble, note it, put it back, ten times. Some marbles show up twice, some not at all. This fake sample has a median of 5.5 s.



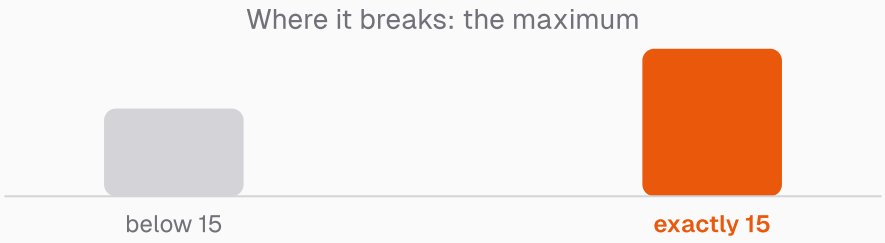
Each fake sample gives its own median.

Another fake sample gives a median of 7. Each draw gives a slightly different answer, and that variation is exactly the wobble you wanted to measure.



Middle 95% of the pile: about 4 s to 8 s. That is the bootstrap interval.

Repeat 10,000 times and the medians pile up. The middle 95% of the pile runs from about 4 to 8 seconds: that is a bootstrap confidence interval for the median, with no formula needed.



About 65% of fake samples have a maximum of exactly 15 s.

Fakes can never exceed your largest marble, so the bootstrap is falsely sure.

It fails for extremes. A fake sample can never contain anything bigger than your largest marble, and about 65% of them include it, so the bootstrap reports a tiny range for the maximum. The real worst case could be far worse. This is the failure the widget below demonstrates.

Plain words

Faking more samples from the one you have

You have one sample and want to know how much your answer would wobble if you collected more. The bootstrap fakes it: build a new dataset by drawing **with replacement** from the one you have,

recompute the number, do that ten thousand times, and look at the spread. The spread of the fakes approximates the spread of the real thing.

For leaders

The default for anything that is not an average

- The bootstrap works for almost any statistic you can compute: median, p95, AUC, a ratio of two rates, a calibration slope. That makes it the right default for eval work, where the interesting statistics are rarely simple means.
- Use the plain percentile version for a quick read. Ask for BCa when the number goes into a document someone will act on.
- It fails in specific, predictable situations, covered next. A team that can name those has understood the method rather than memorised it.

Under the hood

Percentile and BCa intervals

It works because your sample is your best estimate of the population, so resampling from it mimics resampling from the population. The percentile interval takes the $\alpha/2$ and $1-\alpha/2$ quantiles of the bootstrap replicates $\hat{\theta}^*_b$. BCa shifts those quantiles using a bias term $\hat{z}_0$ and an acceleration term $\hat{a}$ from the jackknife:

$$\hat{z}_0 = \Phi^{-1}\left(\frac{\#\{\hat{\theta}^*_b < \hat{\theta}\}}{B}\right), \qquad \hat{a} = \frac{\sum_i (\bar{\theta}_{(\cdot)} - \hat{\theta}_{(i)})^3}{6\left[\sum_i (\bar{\theta}_{(\cdot)} - \hat{\theta}_{(i)})^2\right]^{3/2}}$$

$$\alpha_{\text{lo,hi}} = \Phi\left(\hat{z}_0 + \frac{\hat{z}_0 + z_q}{1 - \hat{a}(\hat{z}_0 + z_q)}\right), \quad q \in \{\alpha/2,\ 1-\alpha/2\}$$

$\hat{\theta}_{(i)}$ is the statistic with point i left out, and $\bar{\theta}_{(\cdot)}$ their mean. When $\hat{z}_0 = \hat{a} = 0$, BCa reduces to the percentile interval.

python

```python
import random
from statistics import NormalDist, mean

def bootstrap_ci(data: list, stat_fn, iters: int = 10_000,
                 alpha: float = 0.05, seed: int = 0) -> tuple[float, float, float]:
    """Percentile bootstrap. Works for ANY statistic: median, p95, a ratio, a slope."""
    rng = random.Random(seed)
    n = len(data)
    # Draw n items WITH replacement each time. Without replacement every rep
```

```
    # would be the same dataset reshuffled.
    reps = sorted(stat_fn([data[rng.randrange(n)] for _ in range(n)]) for _ in range(iters))
    return stat_fn(data), reps[int((alpha / 2) * iters)], reps[int((1 - alpha / 2) * iters) - 1]


def bca_ci(data: list, stat_fn, iters: int = 10_000,
           alpha: float = 0.05, seed: int = 0) -> tuple[float, float, float]:
    """Bias corrected and accelerated bootstrap."""
    rng = random.Random(seed)
    n = len(data)
    theta = stat_fn(data)
    reps = sorted(stat_fn([data[rng.randrange(n)] for _ in range(n)]) for _ in range(iters))

    # Bias: if unbiased, half the reps land below theta.
    prop = sum(1 for r in reps if r < theta) / iters
    prop = min(max(prop, 1 / iters), 1 - 1 / iters)    # keep inv_cdf defined
    N = NormalDist()
    z0 = N.inv_cdf(prop)

    # Acceleration: skewness of the jackknife values.
    jack = [stat_fn(data[:i] + data[i + 1:]) for i in range(n)]
    jm = mean(jack)
    num = sum((jm - x) ** 3 for x in jack)
    den = 6 * (sum((jm - x) ** 2 for x in jack) ** 1.5)
    a = num / den if den else 0.0

    bounds = []
    for q in (alpha / 2, 1 - alpha / 2):
        z = N.inv_cdf(q)
        bounds.append(min(max(N.cdf(z0 + (z0 + z) / (1 - a * (z0 + z))), 0.0), 1.0))
    return theta, reps[min(iters - 1, int(bounds[0] * iters))], reps[min(iters - 1, int(bounds[1] * iters))]
```

rust

```
// Small inline PRNG (SplitMix64); its draws differ from Python's random module.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    fn randrange(&mut self, n: usize) -> usize {
        (self.random() * n as f64) as usize
    }
}

// Horner evaluation, highest coefficient first.
fn poly(coeffs: &[f64], r: f64) -> f64 {
    coeffs.iter().fold(0.0, |acc, &c| acc * r + c)
}

// Inverse standard normal CDF: Wichura's AS241, the algorithm behind
// Python's NormalDist().inv_cdf, so the results agree to the last bit.
```

```rust
fn inv_cdf(p: f64) -> f64 {
    let q = p - 0.5;
    if q.abs() <= 0.425 {
        let r = 0.180625 - q * q;
        let num = poly(&[2.5090809287301226727e+3, 3.3430575583588128105e+4, 6.7265770927008700853e+4,
                         4.5921953931549871457e+4, 1.3731693765509461125e+4, 1.9715909503065514427e+3,
                         1.3314166789178437745e+2, 3.3871328727963666080e+0], r) * q;
        let den = poly(&[5.2264952788528545610e+3, 2.8729085735721942674e+4, 3.9307895800092710610e+4,
                         2.1213794301586595867e+4, 5.3941960214247511077e+3, 6.8718700749205790830e+2,
                         4.2313330701600911252e+1, 1.0], r);
        return num / den;
    }
    let r = if q <= 0.0 { p } else { 1.0 - p };
    let r = (-r.ln()).sqrt();
    let (num, den) = if r <= 5.0 {
        let r = r - 1.6;
        (poly(&[7.7454501427834140764e-4, 2.2723844989269184583e-2, 2.4178072517745061177e-1,
                1.2704582524523683826e+0, 3.6478483247632046050e+0, 5.7694972214606914055e+0,
                4.6303378461565452959e+0, 1.4234371107496835773e+0], r),
         poly(&[1.0507500716444168432e-9, 5.4759380849953449460e-4, 1.5198666563616457197e-2,
                1.4810397642748007459e-1, 6.8976733498510000455e-1, 1.6763848301838038494e+0,
                2.0531916266377588219e+0, 1.0], r))
    } else {
        let r = r - 5.0;
        (poly(&[2.0103343992922881327e-7, 2.7115555687434875782e-5, 1.2426609473880784386e-3,
                2.6532189526576123093e-2, 2.9656057182850489123e-1, 1.7848265399172913358e+0,
                5.4637849111641143699e+0, 6.6579046435011037772e+0], r),
         poly(&[2.0442631033899397856e-15, 1.4215117583164458887e-7, 1.8463183175100546818e-5,
                7.8686913114561325910e-4, 1.4875361290850614853e-2, 1.3692988092273580531e-1,
                5.9983220655588793769e-1, 1.0], r))
    };
    if q < 0.0 { -num / den } else { num / den }
}

// Standard normal CDF via erfc (Numerical Recipes' Chebyshev fit, relative
// error below 1.2e-7: ample for picking a bootstrap percentile).
fn cdf(x: f64) -> f64 {
    let z = x.abs() / std::f64::consts::SQRT_2;
    let t = 1.0 / (1.0 + 0.5 * z);
    let erfc = t * (-z * z - 1.26551223 + t * (1.00002368 + t * (0.37409196 + t * (0.09678418
        + t * (-0.18628806 + t * (0.27886807 + t * (-1.13520398 + t * (1.48851587
        + t * (-0.82215223 + t * 0.17087277)))))))).exp();
    if x >= 0.0 { 1.0 - 0.5 * erfc } else { 0.5 * erfc }
}

fn mean(x: &[f64]) -> f64 {
    x.iter().sum::<f64>() / x.len() as f64
}

/// Percentile bootstrap. Works for ANY statistic: median, p95, a ratio, a slope.
fn bootstrap_ci(data: &[f64], stat_fn: impl Fn(&[f64]) -> f64, iters: usize,
                alpha: f64, seed: u64) -> (f64, f64, f64) {
    let mut rng = Rng(seed);
    let n = data.len();
    // Draw n items WITH replacement each time. Without replacement every rep
    // would be the same dataset reshuffled.
    let mut reps: Vec<f64> = (0..iters)
        .map(|_| stat_fn(&(0..n).map(|_| data[rng.randrange(n)]).collect::<Vec<_>>()))
        .collect();
    reps.sort_by(f64::total_cmp);
```

```
    let it = iters as f64;
    (stat_fn(data), reps[((alpha / 2.0) * it) as usize], reps[((1.0 - alpha / 2.0) * it) as usize - 1])
}

/// Bias corrected and accelerated bootstrap.
fn bca_ci(data: &[f64], stat_fn: impl Fn(&[f64]) -> f64, iters: usize,
          alpha: f64, seed: u64) -> (f64, f64, f64) {
    let mut rng = Rng(seed);
    let n = data.len();
    let it = iters as f64;
    let theta = stat_fn(data);
    let mut reps: Vec<f64> = (0..iters)
        .map(|_| stat_fn(&(0..n).map(|_| data[rng.randrange(n)]).collect::<Vec<_>>()))
        .collect();
    reps.sort_by(f64::total_cmp);

    // Bias: if unbiased, half the reps land below theta.
    let prop = reps.iter().filter(|&&r| r < theta).count() as f64 / it;
    let prop = prop.clamp(1.0 / it, 1.0 - 1.0 / it);   // keep inv_cdf defined
    let z0 = inv_cdf(prop);

    // Acceleration: skewness of the jackknife values.
    let jack: Vec<f64> = (0..n)
        .map(|i| stat_fn(&[&data[..i], &data[i + 1..]].concat()))
        .collect();
    let jm = mean(&jack);
    let num: f64 = jack.iter().map(|x| (jm - x).powi(3)).sum();
    let den = 6.0 * jack.iter().map(|x| (jm - x).powi(2)).sum::<f64>().powf(1.5);
    let a = if den != 0.0 { num / den } else { 0.0 };

    let bound = |q: f64| {
        let z = inv_cdf(q);
        cdf(z0 + (z0 + z) / (1.0 - a * (z0 + z))).clamp(0.0, 1.0)
    };
    let pick = |b: f64| reps[(iters - 1).min((b * it) as usize)];
    (theta, pick(bound(alpha / 2.0)), pick(bound(1.0 - alpha / 2.0)))
}
```

Interactive **Bootstrap, and where it breaks** (shown at its default setting)

Pick a statistic and watch the bootstrap distribution form. Switch to p95 at small n, or to max at any n, and the widget will tell you why the interval you are looking at is not trustworthy.

median = 2.070 **95% CI** [1.678, 2.352] width 0.674 (32.6% of the estimate)

⚠ **Watch out**

Select max. The bootstrap is inconsistent for maxima at every sample size, not just small ones. It is the cleanest example of a method failing because an assumption is violated rather than because there was too little data.

4. When not to bootstrap

The bootstrap estimates sampling variability under one assumption: your rows are exchangeable draws from the population. Every failure below is a violation of that assumption. If you can name the assumption, you can predict the failure.

Situation	What goes wrong	What to do instead
Very small n, under about 20	Resamples only contain values you already saw, so the interval cannot represent tails the sample missed	Exact methods: Clopper–Pearson for proportions, exact McNemar for pairs
Extreme quantiles, p99 on 200 points	p99 rests on about 2 observations. The bootstrap max is your observed max, so the upper bound is structurally censored	Report p95, collect more data, or fit a tail model and say you did
Dependent data, several calls per session	Resampling rows destroys the dependence and shrinks the interval, often by 2x or more	Cluster bootstrap: resample sessions, not rows (section 6)
Time-ordered data with drift	Resampling scrambles the ordering that carries the signal	Block bootstrap: resample contiguous chunks
Statistics at a boundary, near 0% or 100%	The bootstrap distribution piles up on the boundary and the interval degenerates	Wilson or Clopper–Pearson
The statistic is a max or min	The bootstrap is inconsistent for extremes. A known failure, not a small-sample issue	Extreme value theory, or change the metric

5. Each eval metric and what it needs

For leaders

The lookup table for common product metrics

Pass rates, latency, token counts, tool use, and user satisfaction are five different statistical objects, and three of them break the default method. When a dashboard shows one of these, check the method against this table.

Metric	Distribution	Right method	Common mistake
Pass / fail rate	Bernoulli; binomial in aggregate	Wilson CI. McNemar or paired bootstrap for comparisons	Normal approximation near 0 or 1; ignoring the pairing
Latency	Right-skewed, heavy tail, roughly lognormal	Report p50, p95, p99 with bootstrap CIs	Reporting the mean
Token counts	Right-skewed, overdispersed counts	Median and quantiles with bootstrap CI, or model on the log scale	t-test on raw counts
Tool calls per session	Overdispersed counts, often zero-inflated	Negative binomial, or quantiles plus bootstrap. Check variance over mean first	Assuming Poisson. Real variance/mean ratios often run 3 to 6
Tool call success rate	Binomial but **clustered** within session	Cluster bootstrap at the session level	Treating each call as independent
Thumbs-up rate	Binomial with severe selection bias	Wilson on responders, plus the response rate stated explicitly	Calling it satisfaction. Only a biased slice of users clicks
Likert 1 to 5	Ordinal, not interval	Top-two-box proportion, or Mann–Whitney U on ranks	Averaging. The gap from 4 to 5 is not the gap from 1 to 2
Acceptance rate	Binomial, clustered by user and session	Cluster bootstrap on the user	Pooling all events into a tight, wrong interval

Under the hood

Latency, counts, and ratings, worked

Latency

python

```python
def latency_report(samples: list[float]) -> dict:
    """What a latency report should contain.

    Illustrative lognormal sample of 400 requests:
      mean 2.81, median 2.15, p95 8.06, max 17.17
    The mean sits between the median and p95 and describes nobody's experience.
    """
    s = sorted(samples)
    n = len(s)
    q = lambda p: s[min(n - 1, int(p * n))]
    p50_ci = bootstrap_ci(samples, lambda d: sorted(d)[len(d) // 2])
    p95_ci = bootstrap_ci(samples, lambda d: sorted(d)[min(len(d) - 1, int(0.95 * len(d)))])
    return {
        "n": n,
        "p50": q(0.50), "p50_ci": (p50_ci[1], p50_ci[2]),
        "p95": q(0.95), "p95_ci": (p95_ci[1], p95_ci[2]),
        "p99": q(0.99), "p99_warning": "p99 on n<1000 rests on a handful of points.",
        "skew_ratio": (sum(samples) / n) / q(0.50),   # above 1.2: do not use the mean
    }
```

rust

```rust
// Small inline PRNG (SplitMix64); its draws differ from Python's random module.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    fn randrange(&mut self, n: usize) -> usize {
        (self.random() * n as f64) as usize
    }
}

// bootstrap_ci from the snippet above, repeated so this one compiles on its own.
fn bootstrap_ci(data: &[f64], stat_fn: impl Fn(&[f64]) -> f64, iters: usize,
                alpha: f64, seed: u64) -> (f64, f64, f64) {
    let mut rng = Rng(seed);
    let n = data.len();
    let mut reps: Vec<f64> = (0..iters)
        .map(|_| stat_fn(&(0..n).map(|_| data[rng.randrange(n)]).collect::<Vec<_>>()))
        .collect();
    reps.sort_by(f64::total_cmp);
    let it = iters as f64;
    (stat_fn(data), reps[((alpha / 2.0) * it) as usize], reps[(((1.0 - alpha / 2.0) * it) as usize - 1])
}

struct LatencyReport {
```

```
    n: usize,
    p50: f64,
    p50_ci: (f64, f64),
    p95: f64,
    p95_ci: (f64, f64),
    p99: f64,
    p99_warning: &'static str,
    skew_ratio: f64,
}

fn sorted(d: &[f64]) -> Vec<f64> {
    let mut s = d.to_vec();
    s.sort_by(f64::total_cmp);
    s
}

/// What a latency report should contain.
///
/// Illustrative lognormal sample of 400 requests:
///   mean 2.81, median 2.15, p95 8.06, max 17.17
/// The mean sits between the median and p95 and describes nobody's experience.
fn latency_report(samples: &[f64]) -> LatencyReport {
    let s = sorted(samples);
    let n = s.len();
    let q = |p: f64| s[(n - 1).min((p * n as f64) as usize)];
    let p50_ci = bootstrap_ci(samples, |d| sorted(d)[d.len() / 2], 10_000, 0.05, 0);
    let p95_ci = bootstrap_ci(samples, |d| sorted(d)[(d.len() - 1).min((0.95 * d.len() as f64) as usize)],
10_000, 0.05, 0);
    LatencyReport {
        n,
        p50: q(0.50), p50_ci: (p50_ci.1, p50_ci.2),
        p95: q(0.95), p95_ci: (p95_ci.1, p95_ci.2),
        p99: q(0.99), p99_warning: "p99 on n<1000 rests on a handful of points.",
        skew_ratio: (samples.iter().sum::<f64>() / n as f64) / q(0.50),   // above 1.2: do not use the mean
    }
}
```

Counts, and why Poisson usually fails

Poisson assumes variance equals mean. In agent evals the rate is not constant: an easy task takes two tool calls and a hard one takes twenty. A mixture of Poissons with gamma-distributed rates is a negative binomial, which is why real counts are overdispersed. With variance/mean ratio R, Poisson intervals are roughly $\sqrt{R}$ times too narrow.

python

```
from statistics import mean

def overdispersion_check(counts: list[int]) -> dict:
    """Illustrative session mix: mean 5.15, variance 25.53, ratio 4.95.
    A ratio of 5 makes Poisson intervals about sqrt(5) = 2.2x too narrow."""
    m = mean(counts)
    n = len(counts)
    v = sum((x - m) ** 2 for x in counts) / (n - 1)
    ratio = v / m if m else float("inf")
    return {"mean": m, "variance": v, "ratio": ratio,
```

```
            "verdict": "poisson ok" if ratio < 1.5 else "overdispersed: negative binomial or bootstrap",
            "zero_fraction": sum(1 for x in counts if x == 0) / n}
```

rust

```
struct Overdispersion {
    mean: f64,
    variance: f64,
    ratio: f64,
    verdict: &'static str,
    zero_fraction: f64,
}

/// Illustrative session mix: mean 5.15, variance 25.53, ratio 4.95.
/// A ratio of 5 makes Poisson intervals about sqrt(5) = 2.2x too narrow.
fn overdispersion_check(counts: &[u64]) -> Overdispersion {
    let n = counts.len() as f64;
    let m = counts.iter().sum::<u64>() as f64 / n;
    let v = counts.iter().map(|&x| (x as f64 - m).powi(2)).sum::<f64>() / (n - 1.0);
    let ratio = if m != 0.0 { v / m } else { f64::INFINITY };
    Overdispersion {
        mean: m,
        variance: v,
        ratio,
        verdict: if ratio < 1.5 { "poisson ok" } else { "overdispersed: negative binomial or bootstrap" },
        zero_fraction: counts.iter().filter(|&&x| x == 0).count() as f64 / n,
    }
}
```

Satisfaction: ordinal, not interval

python

```
def mann_whitney_u(x: list[float], y: list[float]) -> dict:
    """Compare two groups on ordinal data without pretending it is interval.

    Illustrative Likert shift: means 3.21 -> 3.35, which looks like nothing.
    Common-language effect 0.54: a random new rating beats a random old one 54%
    of the time. Top-two-box moved 0.470 -> 0.525. Those two are actionable.
    """
    combined = sorted([(v, 0) for v in x] + [(v, 1) for v in y], key=lambda t: t[0])
    ranks = [0.0] * len(combined)
    i = 0
    while i < len(combined):                # average ranks across ties
        j = i
        while j + 1 < len(combined) and combined[j + 1][0] == combined[i][0]:
            j += 1
        for k in range(i, j + 1):
            ranks[k] = (i + j) / 2 + 1
        i = j + 1
    r1 = sum(r for r, (_, g) in zip(ranks, combined) if g == 0)
    n1, n2 = len(x), len(y)
    u1 = r1 - n1 * (n1 + 1) / 2
    return {"U": u1, "common_language_effect": u1 / (n1 * n2)}   # P(random x > random y)


def top_two_box(ratings: list[int], scale_max: int = 5) -> float:
```

```
    """Fraction rating 4 or 5: the number product teams can act on."""
    return sum(1 for r in ratings if r >= scale_max - 1) / len(ratings)
```

rust

```rust
struct MannWhitney {
    u: f64,
    common_language_effect: f64,   // P(random x > random y)
}

/// Compare two groups on ordinal data without pretending it is interval.
///
/// Illustrative Likert shift: means 3.21 -> 3.35, which looks like nothing.
/// Common-language effect 0.54: a random new rating beats a random old one 54%
/// of the time. Top-two-box moved 0.470 -> 0.525. Those two are actionable.
fn mann_whitney_u(x: &[f64], y: &[f64]) -> MannWhitney {
    let mut combined: Vec<(f64, u8)> = x.iter().map(|&v| (v, 0)).chain(y.iter().map(|&v| (v, 1))).collect();
    combined.sort_by(|a, b| a.0.total_cmp(&b.0));
    let mut ranks = vec![0.0; combined.len()];
    let mut i = 0;
    while i < combined.len() {               // average ranks across ties
        let mut j = i;
        while j + 1 < combined.len() && combined[j + 1].0 == combined[i].0 {
            j += 1;
        }
        for k in i..=j {
            ranks[k] = (i + j) as f64 / 2.0 + 1.0;
        }
        i = j + 1;
    }
    let r1: f64 = ranks.iter().zip(&combined).filter(|(_, &(_, g))| g == 0).map(|(r, _)| r).sum();
    let (n1, n2) = (x.len() as f64, y.len() as f64);
    let u1 = r1 - n1 * (n1 + 1.0) / 2.0;
    MannWhitney { u: u1, common_language_effect: u1 / (n1 * n2) }
}

/// Fraction rating 4 or 5: the number product teams can act on.
fn top_two_box(ratings: &[u32], scale_max: u32) -> f64 {
    ratings.iter().filter(|&&r| r >= scale_max - 1).count() as f64 / ratings.len() as f64
}
```

Worked problem

p95 latency moved. Is it real? After a change, p95 latency went from 9.81 s to 11.79 s, on 800 requests each side. A 2-second regression. Do you block the release?

python

```python
import numpy as np

def p95(x):
    return float(np.quantile(x, 0.95))

def bootstrap_diff(before, after, iters=4000, seed=1):
    """CI on the DIFFERENCE in p95. No closed form exists, which is what the
    bootstrap is for. Resample each side separately: these are different requests."""
    rng = np.random.default_rng(seed)
    reps = []
    for _ in range(iters):
        a = rng.choice(before, len(before))
        b = rng.choice(after, len(after))
        reps.append(p95(b) - p95(a))
    reps = np.sort(reps)
    return float(np.quantile(reps, 0.025)), float(np.quantile(reps, 0.975))

# illustrative run: observed +1.98 s, bootstrap 95% CI [-0.45, +3.91]
```

rust

```rust
// Small inline PRNG (SplitMix64) in place of np.random.default_rng; its draws
// differ from numpy's, so a run gives a slightly different interval.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    fn randrange(&mut self, n: usize) -> usize {
        (self.random() * n as f64) as usize
    }
}

// np.quantile's default: linear interpolation between order statistics
fn quantile(x: &[f64], q: f64) -> f64 {
    let mut s = x.to_vec();
    s.sort_by(f64::total_cmp);
    let pos = q * (s.len() - 1) as f64;
    let lo = pos.floor() as usize;
    let hi = (lo + 1).min(s.len() - 1);
    s[lo] + (pos - lo as f64) * (s[hi] - s[lo])
}
```

```rust
fn p95(x: &[f64]) -> f64 {
    quantile(x, 0.95)
}

/// CI on the DIFFERENCE in p95. No closed form exists, which is what the
/// bootstrap is for. Resample each side separately: these are different requests.
fn bootstrap_diff(before: &[f64], after: &[f64], iters: usize, seed: u64) -> (f64, f64) {
    let mut rng = Rng(seed);
    let mut choice = |x: &[f64]| -> Vec<f64> { (0..x.len()).map(|_| x[rng.randrange(x.len())]).collect() };
    let mut reps = Vec::with_capacity(iters);
    for _ in 0..iters {
        let a = choice(before);
        let b = choice(after);
        reps.push(p95(&b) - p95(&a));
    }
    (quantile(&reps, 0.025), quantile(&reps, 0.975))
}

// illustrative run: observed +1.98 s, bootstrap 95% CI [-0.45, +3.91]
```

The interval crosses zero. p95 on 800 points rests on about 40 observations, so a 2-second move that looks alarming on a dashboard is not distinguishable from noise at this sample size. Do not block on this alone; say what sample size would settle it. In this data the mean/median ratio was 1.42, so anyone reporting mean latency was reporting a number no user experienced.

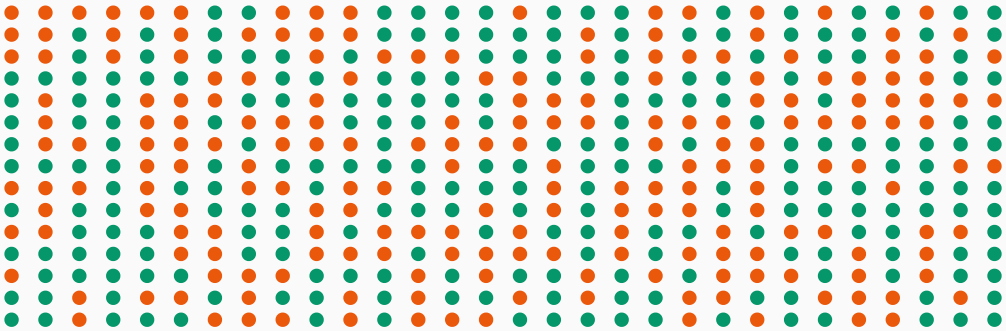
6. Clustering: the mistake that matters most for agent evals

Diagram

450 opinions from 40 kitchen tables

Clustering and the design effect, with a pizza survey.

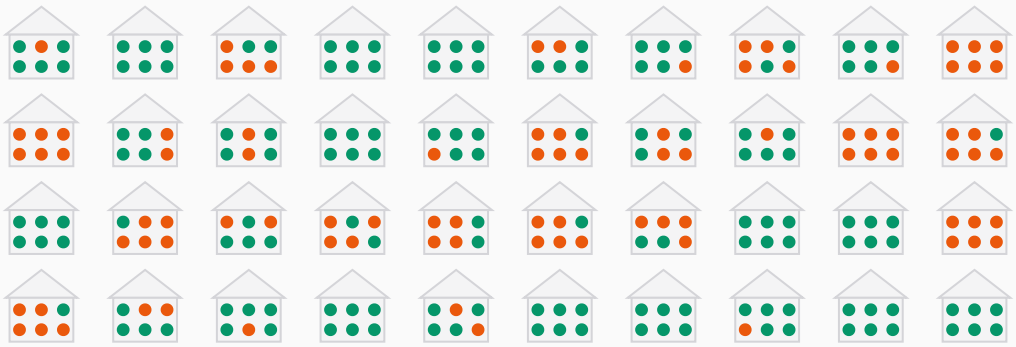
450 answers to "Do you like the new pizza?"



Looks like 450 independent opinions.

A pizza chain surveys 450 people: do you like the new recipe? 450 answers sounds like a solid sample, and the naive error bar on the "yes" share is narrow.

But they came from 40 households



People who share dinner mostly share opinions.

Then you learn the 450 answers came from 40 households, about 11 people each. Families ate the same pizza at the same table and mostly agree with each other. Inside a house, answers move together.

Effective sample size

design effect = 1 + (11 – 1) × 0.6 ≈ 7

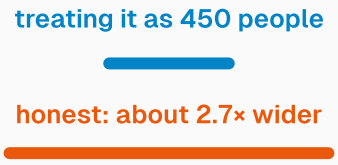
450 answers ÷ 7 ≈ 63 independent opinions

(about 11 people per house, within-house correlation 0.6)

A second answer from the same house adds little new information.

How much do they agree? With about 11 people per house and a within-house correlation of 0.6, the design effect is about 7. Your 450 answers carry roughly the information of 63 independent ones, much closer to 40 than to 450.

Error bar on the "yes" share



Fix: resample whole households, not individual people.

In agent evals: tool calls are the people, sessions are the households.

The honest error bar is about 2.7 times wider than the naive one (the square root of 7). Agent evals have the same shape: tool calls nested inside sessions. Resample whole sessions (a cluster bootstrap), or the interval invents confidence you do not have.

Plain words

450 measurements that are really 40

Suppose you measure 450 tool calls, but they came from only 40 sessions. Calls inside one session tend to succeed or fail together: a confused agent stays confused. So you do not have 450

independent measurements. Your real sample size is much closer to 40, and treating it as 450 makes your error bars far too small.

For leaders

Ask for effective sample size, not row count

- Agent data is nested: tool calls inside sessions, sessions inside users. Any interval computed as if rows were independent is too narrow, often by a factor of two or more.
- Ask "what is the unit of independence, and how many of those do we have?" A dashboard bragging about 50,000 events from 300 users has a sample size much closer to 300.
- A good team's sentence: "we report effective sample size rather than row count, because calls are nested in sessions; the naive interval was about 2.4 times too narrow when we checked."

Under the hood

Design effect and the cluster bootstrap

With average cluster size $\bar{m}$ and intraclass correlation ρ (the share of variance that lives between clusters), Kish's design effect and the effective sample size are

$$\mathrm{deff} = 1 + (\bar{m} - 1)\,\rho, \qquad n_{\mathrm{eff}} = \frac{N}{\mathrm{deff}}, \qquad \mathrm{SE}_{\mathrm{true}} \approx \sqrt{\mathrm{deff}}\,\mathrm{SE}_{\mathrm{naive}}$$

Bigger clusters amplify the penalty: $\rho = 0.3$ with 20 calls per session gives deff 6.7; the same ρ with 3 calls gives 1.6.

python

```python
import random
from statistics import mean

def cluster_bootstrap_ci(clusters: list[list[float]], iters: int = 10_000,
                         seed: int = 0) -> tuple[float, float, float]:
    """Resample SESSIONS, not individual calls."""
    rng = random.Random(seed)
    m = len(clusters)                # number of independent units: the real n
    # The point estimate is unaffected by clustering; only its uncertainty is.
    observed = sum(sum(c) for c in clusters) / sum(len(c) for c in clusters)
    reps = []
    for _ in range(iters):
        picked = [clusters[rng.randrange(m)] for _ in range(m)]   # whole sessions
        total, count = sum(sum(c) for c in picked), sum(len(c) for c in picked)
        reps.append(total / count if count else 0.0)             # recount: sizes vary
    reps.sort()
    return observed, reps[int(0.025 * iters)], reps[int(0.975 * iters) - 1]


def design_effect(clusters: list[list[float]]) -> float:
```

```
    sizes = [len(c) for c in clusters]
    grand = sum(sum(c) for c in clusters) / sum(sizes)
    between = sum(len(c) * (sum(c) / len(c) - grand) ** 2 for c in clusters) / (len(clusters) - 1)
    within = sum((x - sum(c) / len(c)) ** 2 for c in clusters for x in c) / max(1, sum(sizes) - len(clusters))
    icc = between / (between + within) if (between + within) else 0.0
    return 1 + (mean(sizes) - 1) * icc

# Illustrative, 40 sessions with 450 tool calls:
#   naive row bootstrap: [0.7289, 0.8067]  width 0.078
#   cluster bootstrap:   [0.6711, 0.8581]  width 0.187   (2.4x wider)
```

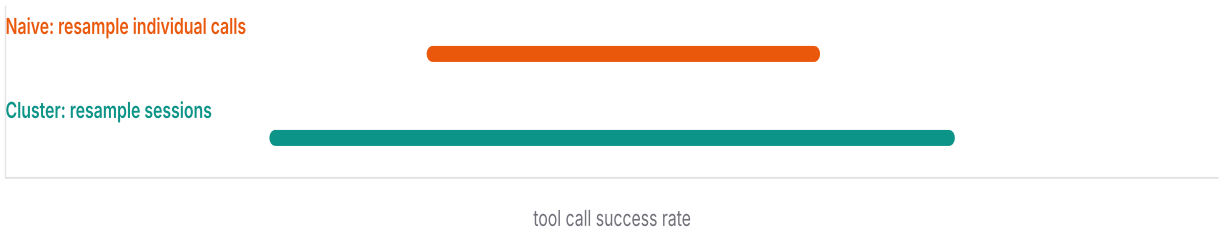
rust

```
// Small inline PRNG (SplitMix64); its draws differ from Python's random module.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
    fn randrange(&mut self, n: usize) -> usize {
        (self.random() * n as f64) as usize
    }
}

/// Resample SESSIONS, not individual calls.
fn cluster_bootstrap_ci(clusters: &[Vec<f64>], iters: usize, seed: u64) -> (f64, f64, f64) {
    let mut rng = Rng(seed);
    let m = clusters.len();              // number of independent units: the real n
    let total_of = |cs: &[&Vec<f64>]| cs.iter().map(|c| c.iter().sum::<f64>()).sum::<f64>();
    let count_of = |cs: &[&Vec<f64>]| cs.iter().map(|c| c.len()).sum::<usize>();
    // The point estimate is unaffected by clustering; only its uncertainty is.
    let all: Vec<&Vec<f64>> = clusters.iter().collect();
    let observed = total_of(&all) / count_of(&all) as f64;
    let mut reps = Vec::with_capacity(iters);
    for _ in 0..iters {
        let picked: Vec<&Vec<f64>> = (0..m).map(|_| &clusters[rng.randrange(m)]).collect();  // whole sessions
        let (total, count) = (total_of(&picked), count_of(&picked));
        reps.push(if count > 0 { total / count as f64 } else { 0.0 });                       // recount: sizes vary
    }
    reps.sort_by(f64::total_cmp);
    let it = iters as f64;
    (observed, reps[(0.025 * it) as usize], reps[(0.975 * it) as usize - 1])
}

fn design_effect(clusters: &[Vec<f64>]) -> f64 {
    let sizes: Vec<usize> = clusters.iter().map(|c| c.len()).collect();
    let n_total: usize = sizes.iter().sum();
    let mean_of = |c: &Vec<f64>| c.iter().sum::<f64>() / c.len() as f64;
    let grand = clusters.iter().map(|c| c.iter().sum::<f64>()).sum::<f64>() / n_total as f64;
    let between = clusters.iter().map(|c| c.len() as f64 * (mean_of(c) - grand).powi(2)).sum::<f64>()
        / (clusters.len() - 1) as f64;
    let within = clusters.iter().map(|c| { let m = mean_of(c); c.iter().map(|x| (x - m).powi(2)).sum::<f64>()
```

```
}).sum::<f64>()
        / 1.max(n_total - clusters.len()) as f64;
    let icc = if between + within != 0.0 { between / (between + within) } else { 0.0 };
    let mean_size = n_total as f64 / sizes.len() as f64;
    1.0 + (mean_size - 1.0) * icc
}

// Illustrative, 40 sessions with 450 tool calls:
//   naive row bootstrap: [0.7289, 0.8067]  width 0.078
//   cluster bootstrap:   [0.6711, 0.8581]  width 0.187   (2.4x wider)
```

Interactive **Clustering and the design effect** (shown at its default setting)

Tool calls nested in sessions. The naive bootstrap resamples individual calls, the cluster bootstrap resamples whole sessions. Raise the within session correlation and watch how much confidence the naive interval invents.



Observed rate 0.7854

Naive width 0.0708 Cluster width 0.1250 **understatement 1.76x**

480 calls, but only 40 independent units. Effective n is closer to 40 than to 480.

Principal asks

Push correlation and cluster size up together. That is a realistic agent trace, and the naive interval ends up several times too narrow. This is the most likely statistical mistake in agent evaluation, and the easiest one to show you avoid.

7. Choosing a method, in one flow

1. **What is the unit of independence?** Task, session, or user. Not row. Everything else follows from this.
2. **Is the outcome binary, count, continuous, or ordinal?** That picks the family.

3. **Is the statistic a mean, a proportion, or a quantile?** Means and proportions have closed forms. Quantiles need the bootstrap.

4. **Is the comparison paired?** If both models saw the same tasks, use the pairing or you throw away most of your power.

5. **Is n small or the statistic extreme?** If yes, use exact methods and stop bootstrapping.

6. **How skewed is it?** Mean over median above 1.2 means report quantiles, not averages.

> **Key idea**
>
> If you can walk that flow out loud in ninety seconds while someone describes a metric, you can handle most eval statistics questions. The reasoning is the same every time; the tables above are the lookup for each branch. For the general version, see from shape to method.

Part II · Chapter 10

Classification metrics, from precision to AUC

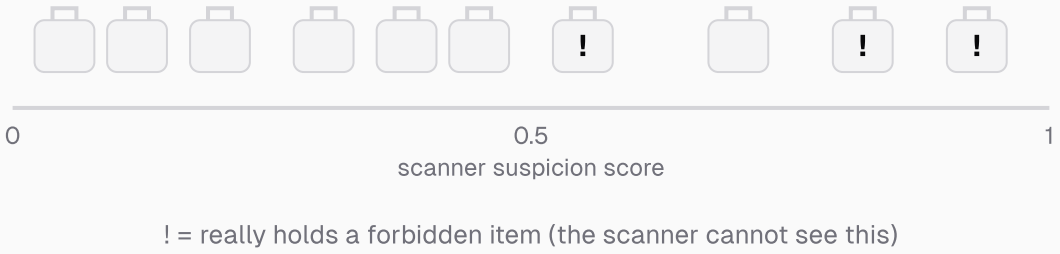
Every LLM judge is a classifier. So is a safety filter, a regression detector, an intent labeller, and an auto-approve gate. This chapter starts with someone sorting mail and ends with the metrics you would put in a launch review. One spam example runs through every chart, so each idea builds on the last.

The four outcomes

Diagram **Airport security: the dial that trades precision for recall**

Ten bags, one scanner, one dial. The same four outcomes as the mail sorter, in pictures.

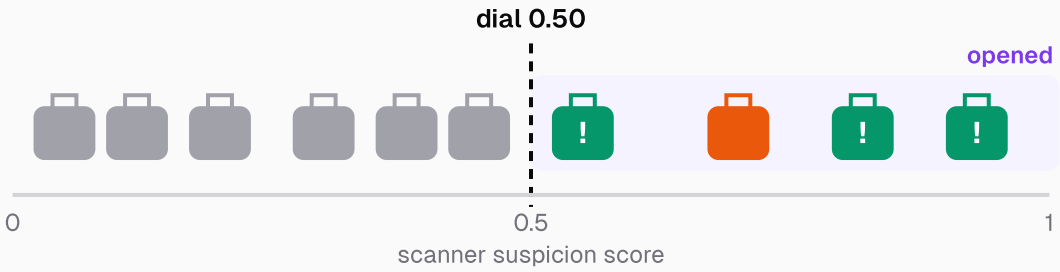
Ten bags. Three hide something forbidden.



! = really holds a forbidden item (the scanner cannot see this)

A scanner gives every bag a suspicion score from 0 to 1. Three bags (marked !) really hold something forbidden. The guard never sees inside; only the score. That score is exactly what a spam model or an LLM judge gives each item.

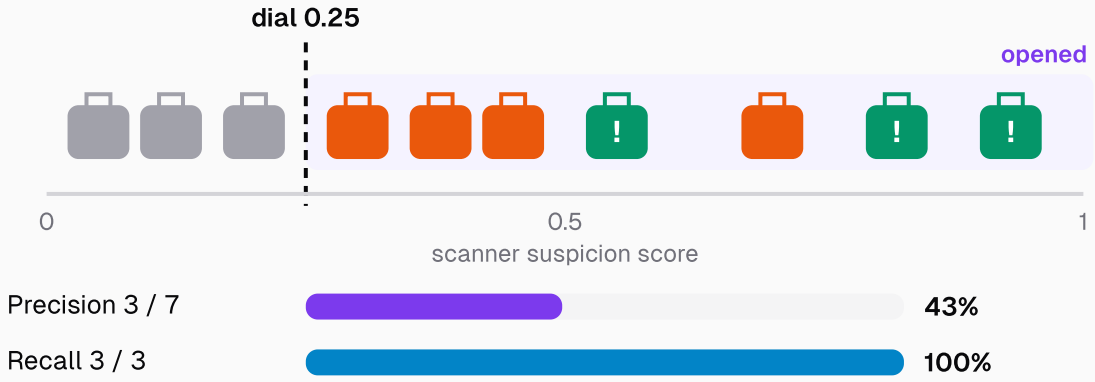
Dial at 0.50: open every bag to the right



3 caught	1 opened, harmless	0 missed	6 left alone
true positive	false positive	false negative	true negative

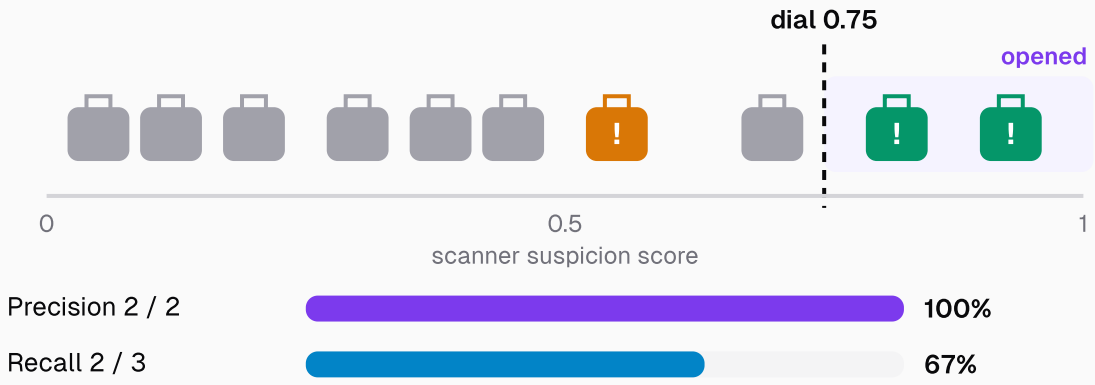
Opening everything above 0.50 produces four outcomes: 3 forbidden bags caught (true positives), 1 harmless bag opened (a false positive, one annoyed traveller), 0 missed (false negatives), and 6 harmless bags left alone (true negatives).

Strict dial at 0.25: catch everything, open lots of bags

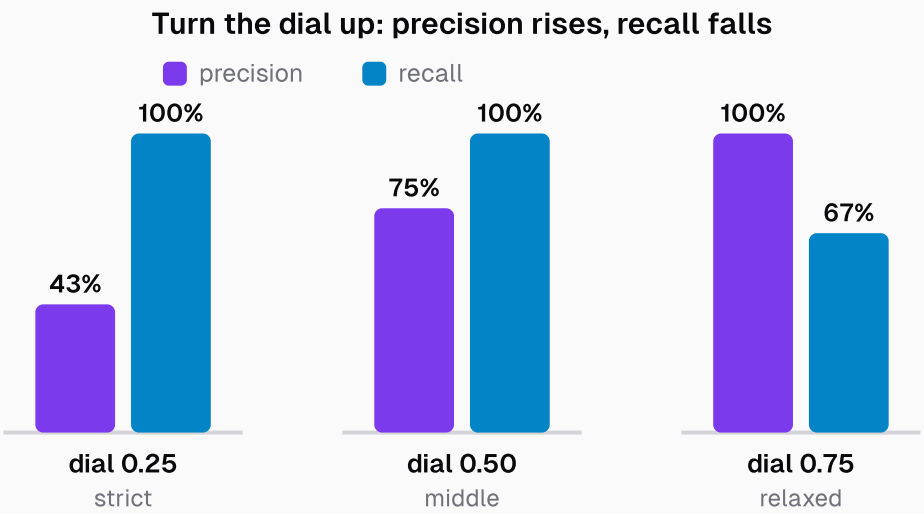


Lower the dial and the guard opens 7 bags. All 3 forbidden bags are caught, so **recall** is 3 of 3. But only 3 of the 7 opened bags were guilty, so **precision** falls to 43%. Four annoyed travellers.

Relaxed dial at 0.75: no false alarms, one slips through



Raise the dial and only 2 bags get opened, both guilty, so precision is 100%. But the forbidden bag that scored 0.55 walks straight through: recall drops to 2 of 3.



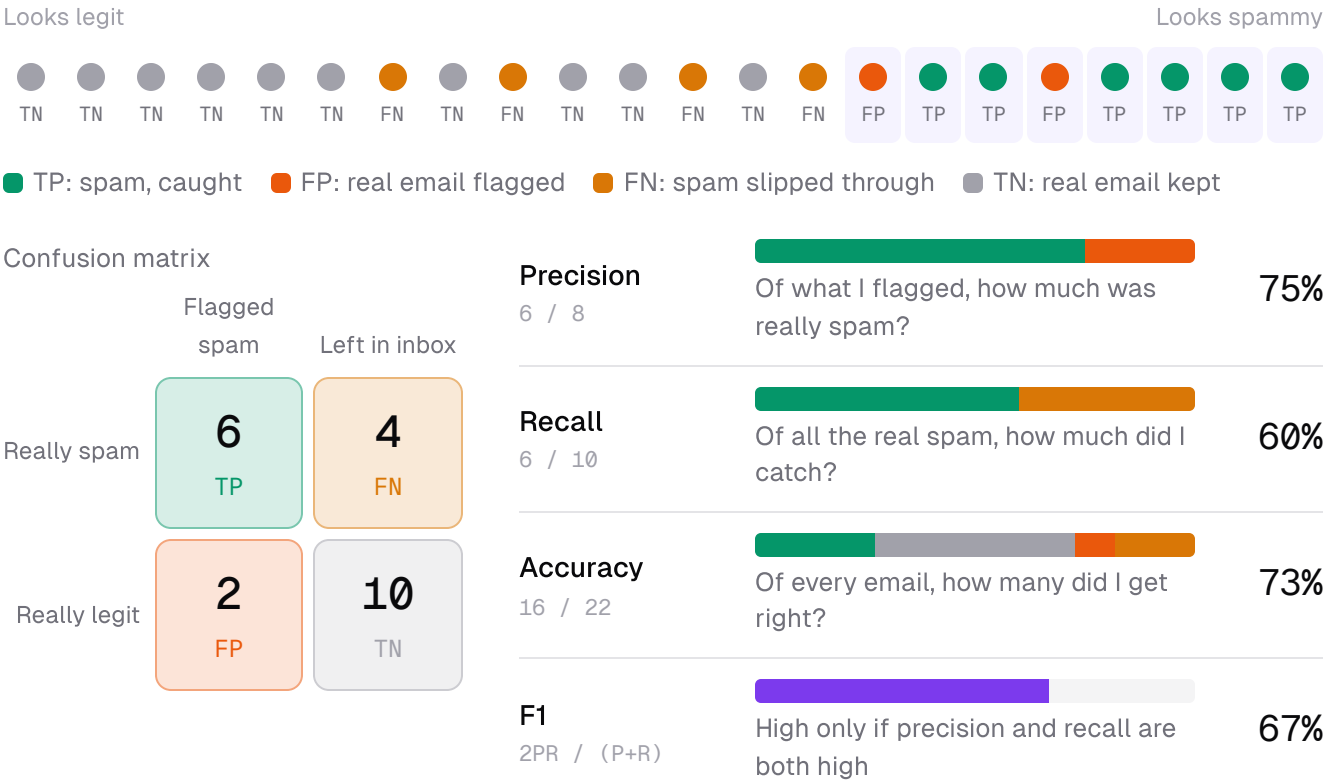
Precision: of the bags we opened, how many were guilty? **Recall**: of the guilty bags, how many did we open? The dial only trades one for the other. The only way to raise both is a better scanner, one that gives guilty bags higher scores.

A spam model does not really say "spam" or "not spam." It gives each email a score from 0 to 1 for how spammy it looks. You choose a cutoff, and anything at or above it goes to the spam folder. That cutoff is the **threshold**. Think of it as the passing grade on a test: the scores are fixed, but you decide where the line goes.

Interactive

Twenty-two emails and a threshold (shown at its default setting)

Emails are sorted by score. Drag the threshold; shaded cells go to the spam folder. Then press the two buttons and watch precision and recall pull against each other.



The 2×2 grid is the **confusion matrix**. Rows are the truth, columns are what the model decided. Everything else in this chapter is built from those four numbers.

Plain words

A helper sorts your mail into two piles

Your helper looks at every letter and decides: junk pile or keep pile. Each time, they can be right in two ways and wrong in two ways.

True positive

It was junk, and they put it in the junk pile. Good catch.

False positive

It was a real letter, and they threw it in the junk pile. A false alarm.

False negative

It was junk, and they left it in the keep pile. It slipped through.

True negative

It was a real letter, and they kept it. Also good.

Every metric in this chapter is a different way of counting those four piles. Each one asks a simple question:

Precision

When the helper says "junk," how often are they right?

Recall

Of all the junk that arrived, how much did they find?

Accuracy

Out of every letter, how many did they sort correctly?

F1

One score that is only high when precision and recall are both high.

"Positive" means the thing you are hunting for. Here that is spam. It does not mean good.

For leaders

A metric without its confusion matrix is unauditable

When someone shows you "precision 0.92," three things are hidden: the threshold, the prevalence of the positive class in the test set, and how many examples sat behind the number. Each one can move the headline more than the model change being reported. Make it a standing rule that any classifier metric in a review comes with the confusion matrix, the threshold, and n.

The second rule: decide which mistake costs more *before* you look at results. A false positive in a safety filter blocks a legitimate user. A false negative lets a harmful output through. A false positive in a regression detector costs an engineer an afternoon; a false negative ships a regression. If the team cannot say which error is worse, no metric choice will be defensible.

Principal asks

- At what threshold, and who chose it? Was it tuned on the same data you are reporting?
- What is the positive rate in this test set, and what is it in production?
- Which error are we optimising against, and what does one of each cost?

Under the hood

The confusion matrix is a function of the threshold

Given scores s_i, labels $y_i \in \{0, 1\}$, and a threshold τ, the prediction is $\hat{y}_i = \mathbb{1}[s_i \geq \tau]$. Each cell counts a combination:

$$\mathrm{TP}(\tau) = \sum_i \mathbb{1}[\hat{y}_i = 1, y_i = 1], \quad \mathrm{FP}(\tau) = \sum_i \mathbb{1}[\hat{y}_i = 1, y_i = 0], \quad \mathrm{FN}(\tau) = \sum_i \mathbb{1}[\hat{y}_i = ($$

TN is everything else. Raising τ can only move items from the predicted-positive column to the predicted-negative column, so TP and FP never increase as τ rises.

Report a single precision or recall and you have silently reported a threshold choice as if it were a property of the model. Two teams can report very different F1 for the same model because they picked different operating points. Threshold-free metrics (section on AUC below) sidestep this and bring their own problems.

python

```python
def confusion(y_true, y_pred):
    """The four numbers everything else is built from.
    Always compute this first and show it next to any derived metric."""
    tp = sum(1 for a, b in zip(y_true, y_pred) if a == 1 and b == 1)
    fp = sum(1 for a, b in zip(y_true, y_pred) if a == 0 and b == 1)
    fn = sum(1 for a, b in zip(y_true, y_pred) if a == 1 and b == 0)
    tn = sum(1 for a, b in zip(y_true, y_pred) if a == 0 and b == 0)
    return {"tp": tp, "fp": fp, "fn": fn, "tn": tn}
```

rust

```rust
#[derive(Debug, Clone, Copy, PartialEq)]
struct Confusion {
    tp: u64,
    fp: u64,
    fn_: u64, // `fn` is a Rust keyword, hence the underscore
    tn: u64,
}

/// The four numbers everything else is built from.
/// Always compute this first and show it next to any derived metric.
fn confusion(y_true: &[u8], y_pred: &[u8]) -> Confusion {
    let mut c = Confusion { tp: 0, fp: 0, fn_: 0, tn: 0 };
    for (&a, &b) in y_true.iter().zip(y_pred) {
        match (a, b) {
            (1, 1) => c.tp += 1,
            (0, 1) => c.fp += 1,
            (1, 0) => c.fn_ += 1,
            (0, 0) => c.tn += 1,
            _ => {} // anything other than 0/1 is not counted, as in the Python
        }
    }
```

```
    c
}
```

Four metrics, four questions

Look at the bars in the widget above. Precision and recall both start from true positives. What changes is which group you divide by.

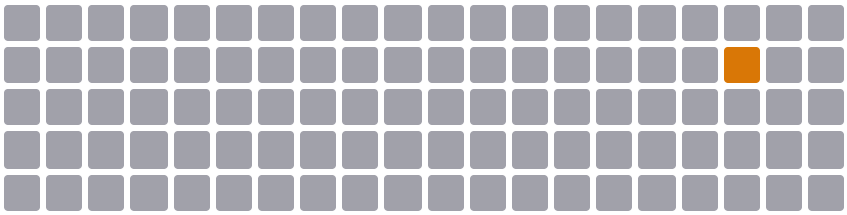
Metric	Formula	Divides by	Its enemy	Memory hook
Precision	`TP / (TP + FP)`	Everything you flagged	False alarms	Precise means picky
Recall	`TP / (TP + FN)`	All the real spam	Misses	Recall them all, like a product recall
Accuracy	`(TP + TN) / all`	Every email	Both mistakes	Simple, but can lie
F1	`2PR / (P + R)`	Combines P and R	Whichever is lower	A harsh average

Plain words

The tug-of-war, and why accuracy can lie

Press "Catch everything." Recall climbs, but coral dots flood the spam folder and precision falls. Press "Only be sure" and the opposite happens. Moving the threshold trades one mistake for the other. You choose based on which mistake hurts more. Missing a cancer diagnosis is worse than a false alarm, so screening favours recall. Losing an important email is worse than seeing some spam, so spam filters usually favour precision.

Now suppose only 1% of email is spam. A lazy model that flags nothing gets every legit email right.



Accuracy	Recall	F1
99%	0%	0%

Each square is 10 emails. The one amber square is all the spam, and the "flag nothing" model misses every bit of it.

99% accuracy, zero spam caught. When one class is rare, accuracy mostly measures how common the other class is.

For leaders

How each metric gets gamed

Metric	Easy way to inflate it	What to demand alongside it
Accuracy	Test on a set where the easy class dominates	Prevalence, balanced accuracy or MCC
Precision	Only flag the most obvious cases. Precision 1.0 at recall 0.02 looks perfect and is worthless	Recall at the same threshold
Recall	Flag everything. Recall 1.0 is always available, at precision equal to prevalence	Precision at the same threshold
F1	Tune the threshold on the test set; test on a large pool of easy negatives F1 never sees	A held-out threshold and the TN count
ROC-AUC	Report it on a rare-positive problem, where it flatters	PR-AUC and the confusion matrix at the operating point

Nobody needs to act in bad faith for this to happen. A team that iterates on a metric for months will drift toward whatever the metric rewards. The defence is a metric pack fixed in advance, not a single

number.

Under the hood

F-beta, and the demonstration that F1 ignores TN

$$F_\beta = (1+\beta^2)\,\frac{P \cdot R}{\beta^2 P + R}$$

$\beta > 1$ weights recall more, $\beta < 1$ weights precision more. F_1 is the harmonic mean of P and R, which punishes imbalance: $P = 0.9, R = 0.1$ gives 0.18, not 0.5.

python

```python
def f_beta(precision, recall, beta=1.0):
    """beta > 1 weights recall more, beta < 1 weights precision more.
    beta = 2 says a miss costs about four times a false alarm (beta squared)."""
    if precision + recall == 0:
        return 0.0
    b2 = beta * beta
    return (1 + b2) * precision * recall / (b2 * precision + recall)
```

rust

```rust
/// beta > 1 weights recall more, beta < 1 weights precision more.
/// beta = 2 says a miss costs about four times a false alarm (beta squared).
/// Rust has no default arguments: pass beta = 1.0 for plain F1.
fn f_beta(precision: f64, recall: f64, beta: f64) -> f64 {
    if precision + recall == 0.0 {
        return 0.0;
    }
    let b2 = beta * beta;
    (1.0 + b2) * precision * recall / (b2 * precision + recall)
}
```

⚠ **Watch out**

Two classifiers, both with TP = 80, FP = 20, FN = 20. Scenario A has TN = 80; scenario B has TN = 9,980. Both score F1 = 0.800. MCC separates them: 0.600 versus 0.798. Same F1, materially different classifiers, because F1 never looks at the TN cell.

One class at a time

Precision, recall, and F1 are always calculated for one class, treated as "positive." So far spam was positive. Flip it and ask the same questions about legit email. The table follows the threshold in the first widget.

Class	Precision	Recall	F1	Count
Spam	75%	60%	67%	10
Legit	71%	83%	77%	12
Macro average			72%	
Weighted average			72%	
Micro average (equals accuracy)			73%	

At threshold 0.58: 6 TP, 2 FP, 4 FN, 10 TN. Move the threshold in the widget above and this table follows.

Plain words

Same four boxes, reassigned

Legit precision asks, "of what reached my inbox, how much was really legit?" That is TN out of TN + FN. Legit recall asks, "of all real emails, how many reached my inbox?" That is TN out of TN + FP. Legit recall has its own name, **specificity**, common in medical testing. Accuracy is the odd one out: it is one number for the whole model.

Macro average

A plain average of each class's score. Every class counts equally, so a rare class you care about cannot hide.

Weighted average

Each class weighted by how many examples it has. The big class has more say.

Micro average

Pool all the counts first, then compute. For one-label-per-item problems it equals accuracy.

For leaders

Per class first, averages second

Ask for the per-class table before any average. The averaging choice is a statement about what matters: macro says every class matters equally, weighted says the traffic mix matters, micro says only total volume matters. For anything where rare cases carry the risk (safety categories, unusual intents, rare failure modes), weighted and micro hide exactly what you need to see.

The actionable unit is usually a confusable pair, not a score. "Macro F1 0.81" tells you nothing to do. "Refactor and rename are confused 40% of the time" tells you to fix the taxonomy or the labelling guide.

Under the hood

The three averages, precisely

$$F_1^{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} F_1^{(k)}, \qquad F_1^{\text{weighted}} = \sum_{k=1}^{K} \frac{n_k}{n} F_1^{(k)}, \qquad F_1^{\text{micro}} = \frac{2\sum_k \mathrm{TP}_k}{2\sum_k \mathrm{TP}_k + \sum_k \mathrm{FP}_k}$$

In single-label multiclass, every false positive for one class is a false negative for another, so $\sum_k \mathrm{FP}_k = \sum_k \mathrm{FN}_k$ and micro-F1 equals accuracy.

python

```python
from collections import Counter

def multiclass_report(y_true, y_pred):
    """Per-class metrics plus all three averages. When macro and weighted
    diverge sharply, that gap is the finding."""
    labels = sorted(set(y_true) | set(y_pred))
    per = {}
    for lab in labels:
        tp = sum(a == lab and b == lab for a, b in zip(y_true, y_pred))
        fp = sum(a != lab and b == lab for a, b in zip(y_true, y_pred))
        fn = sum(a == lab and b != lab for a, b in zip(y_true, y_pred))
        p = tp / (tp + fp) if tp + fp else 0.0
        r = tp / (tp + fn) if tp + fn else 0.0
        per[lab] = {"f1": 2 * p * r / (p + r) if p + r else 0.0,
                    "support": tp + fn, "tp": tp, "fp": fp, "fn": fn}
    n = len(y_true)
    macro = sum(v["f1"] for v in per.values()) / len(labels)
    weighted = sum(v["f1"] * v["support"] for v in per.values()) / n
    micro = sum(a == b for a, b in zip(y_true, y_pred)) / n
    confused = Counter((a, b) for a, b in zip(y_true, y_pred) if a != b).most_common(3)
    return {"per_class": per, "macro_f1": macro, "weighted_f1": weighted,
            "micro_f1": micro, "top_confused_pairs": confused}
```

rust

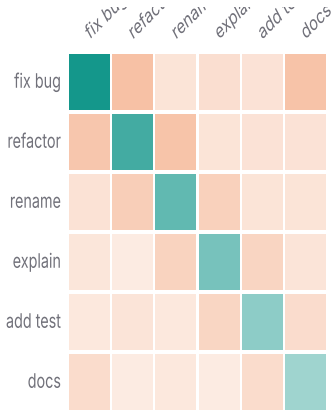
```rust
use std::collections::{BTreeMap, BTreeSet, HashMap};
use std::hash::Hash;

struct ClassStats {
```

```
    f1: f64,
    support: usize,
    tp: usize,
    fp: usize,
    fn_: usize,
}

struct Report<T> {
    per_class: BTreeMap<T, ClassStats>, // sorted by label, like sorted(set(...))
    macro_f1: f64,
    weighted_f1: f64,
    micro_f1: f64,
    top_confused_pairs: Vec<((T, T), usize)>,
}

/// Per-class metrics plus all three averages. When macro and weighted
/// diverge sharply, that gap is the finding.
fn multiclass_report<T: Ord + Hash + Clone>(y_true: &[T], y_pred: &[T]) -> Report<T> {
    let labels: BTreeSet<&T> = y_true.iter().chain(y_pred).collect();
    let pairs: Vec<(&T, &T)> = y_true.iter().zip(y_pred).collect();
    let mut per = BTreeMap::new();
    for &lab in &labels {
        let tp = pairs.iter().filter(|&&(a, b)| a == lab && b == lab).count();
        let fp = pairs.iter().filter(|&&(a, b)| a != lab && b == lab).count();
        let fn_ = pairs.iter().filter(|&&(a, b)| a == lab && b != lab).count();
        let p = if tp + fp > 0 { tp as f64 / (tp + fp) as f64 } else { 0.0 };
        let r = if tp + fn_ > 0 { tp as f64 / (tp + fn_) as f64 } else { 0.0 };
        let f1 = if p + r > 0.0 { 2.0 * p * r / (p + r) } else { 0.0 };
        per.insert(lab.clone(), ClassStats { f1, support: tp + fn_, tp, fp, fn_ });
    }
    let n = y_true.len() as f64;
    let macro_f1 = per.values().map(|v| v.f1).sum::<f64>() / labels.len() as f64;
    let weighted_f1 = per.values().map(|v| v.f1 * v.support as f64).sum::<f64>() / n;
    let micro_f1 = pairs.iter().filter(|&&(a, b)| a == b).count() as f64 / n;

    // Counter(...).most_common(3): highest count first, ties in first-seen order
    let mut counts: Vec<((T, T), usize)> = Vec::new();
    let mut index: HashMap<(T, T), usize> = HashMap::new();
    for &(a, b) in pairs.iter().filter(|&&(a, b)| a != b) {
        let key = (a.clone(), b.clone());
        let i = *index.entry(key.clone()).or_insert_with(|| {
            counts.push((key, 0));
            counts.len() - 1
        });
        counts[i].1 += 1;
    }
    counts.sort_by(|x, y| y.1.cmp(&x.1)); // stable sort keeps first-seen order on ties
    counts.truncate(3);

    Report { per_class: per, macro_f1, weighted_f1, micro_f1, top_confused_pairs: counts }
}
```

Interactive

Multiclass confusion heatmap (shown at its default setting)

An intent taxonomy as a heatmap. Rows are true intent, columns are predicted. Raise imbalance and watch weighted F1 detach from macro F1 while the headline number stays comfortable.



macro F1 0.760 **micro F1** 0.791 **weighted F1** 0.794 **gap** 0.034

macro and weighted agree, so performance is roughly even across classes.

Top confused pairs: fix bug → refactor (97, 8.9% of that class) · fix bug → docs (88, 8.1% of that class) · refactor → rename (84, 11.6% of that class)

That list is actionable. The F1 number is not.

Worked problem

An intent classifier: which number goes in the report? Six intent classes, imbalanced, 3,000 sessions. Weighted F1 is 0.836. Ship it?

Class	Support	F1
fix_bug	1,006	0.934
refactor	660	0.919
explain	598	0.872
add_test	369	0.674
docs	242	0.579
optimize	**125**	**0.423**

Macro F1 is 0.733, weighted F1 is 0.836, a gap of 0.103. The headline weighted number is carried by the three common classes. On `optimize` the classifier is barely better than guessing. When macro and weighted diverge by more than about 0.05, that divergence is the finding, and reporting only the weighted number hides it. If these labels are used to generate benchmark tasks, the rare intents are the ones being mislabelled.

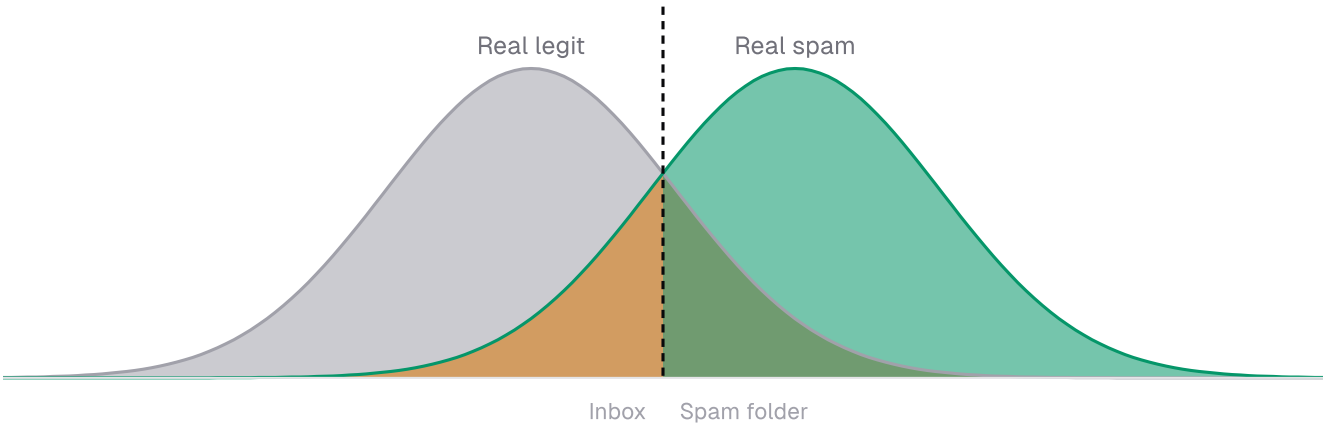
The trade-off and the ceiling

Zoom out from 22 emails to thousands. The two humps show where real legit emails and real spam land on the score line. Drag the threshold first, then raise model quality.

Interactive

The threshold picks a spot, the model sets the limit (shown at its default setting)

With a weak model, every threshold cuts through both humps. Watch the "Best F1 possible" card: only model quality moves it.



TP FN FP TN

Precision	Recall	F1 now	Best F1 possible
82%	82%	82%	82% at threshold 0.48

Plain words

You can only choose which mistakes to make

With a weak model, the humps overlap. Wherever you put the threshold, it cuts through both, so you only get to choose which mistakes to make. Raise model quality and the humps pull apart. Then one threshold can catch most spam while flagging almost no legit mail.

For leaders

Threshold work versus model work

When a team reports that precision went up, ask whether recall went down. If it did, the threshold moved and the model did not get better. That can still be the right call, but it is a product decision about error costs, not a research result. Real improvement shows up as a higher curve (a higher AUC), or as both precision and recall improving at a fixed threshold. Budget accordingly: threshold tuning is cheap and fast; separating the humps takes better features, better data on the hard cases, or a stronger model.

Key idea

This is the precision-recall trade-off. The only way to improve precision and recall together is to separate the humps, which means a better model, not a better threshold.

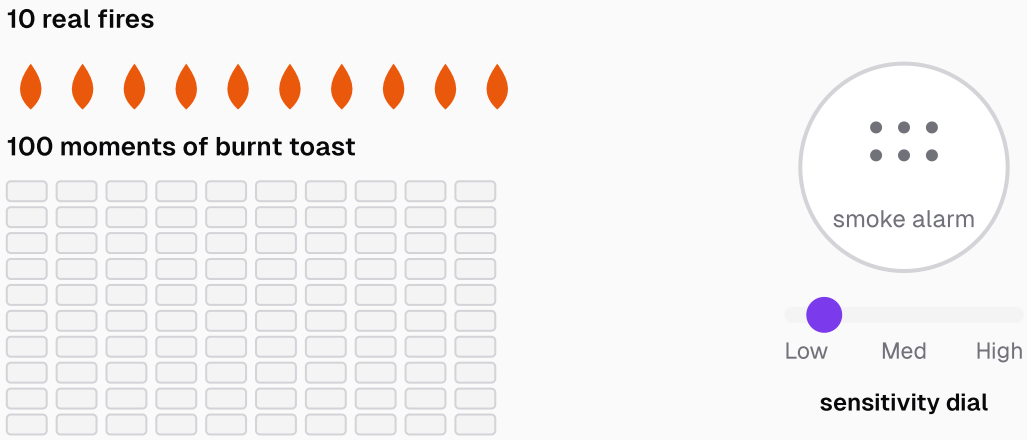
Building an ROC curve

An ROC curve shows what happens at every threshold at once. Each threshold gives one dot. The dot goes **up** for spam caught and **right** for good emails wrongly flagged. Up is good. Right is bad.

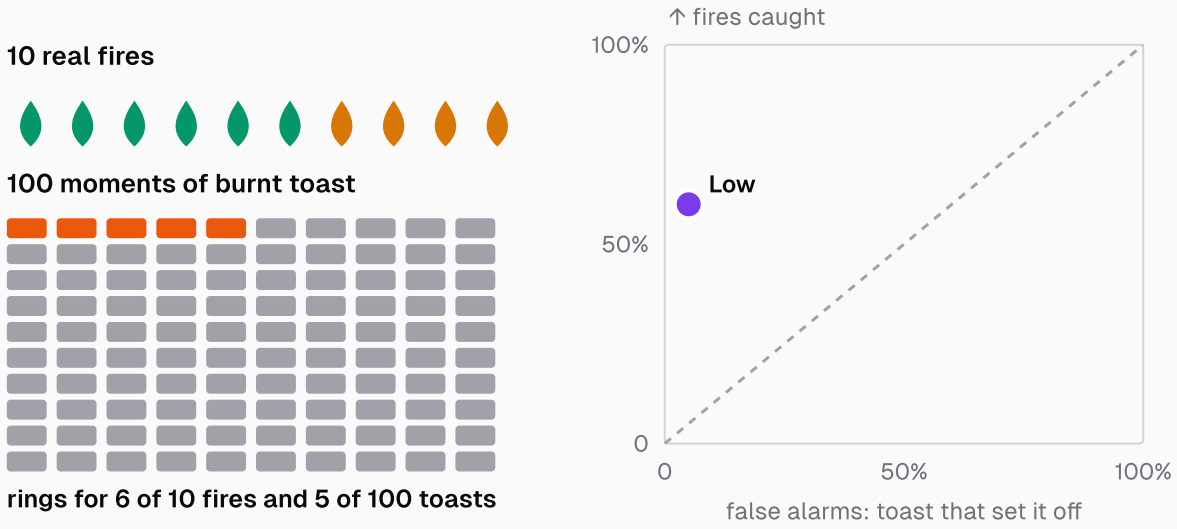
Diagram

A smoke alarm with a sensitivity dial

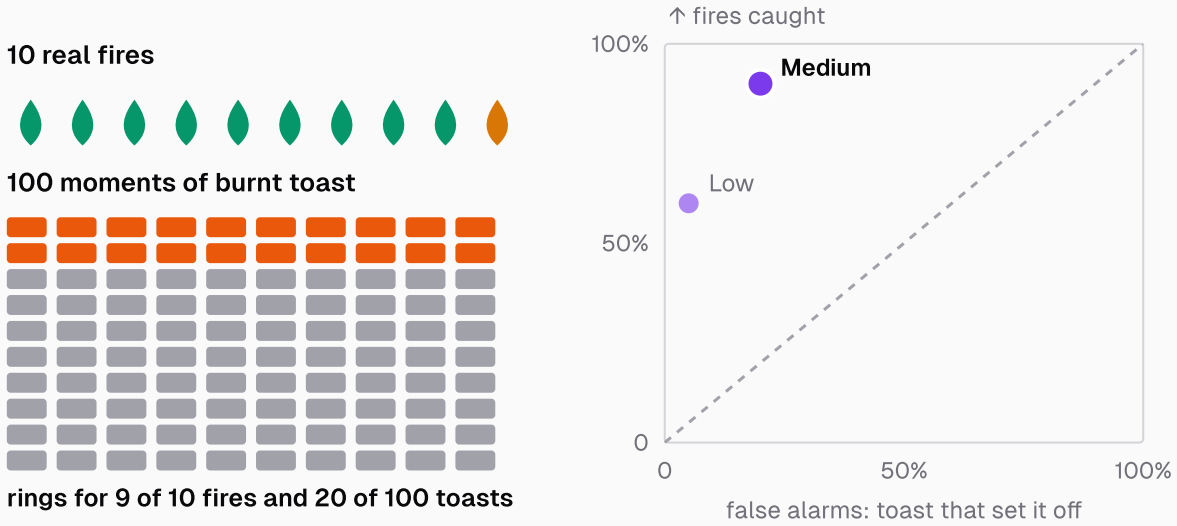
Each dial setting becomes one dot. Connect the dots and you have an ROC curve.



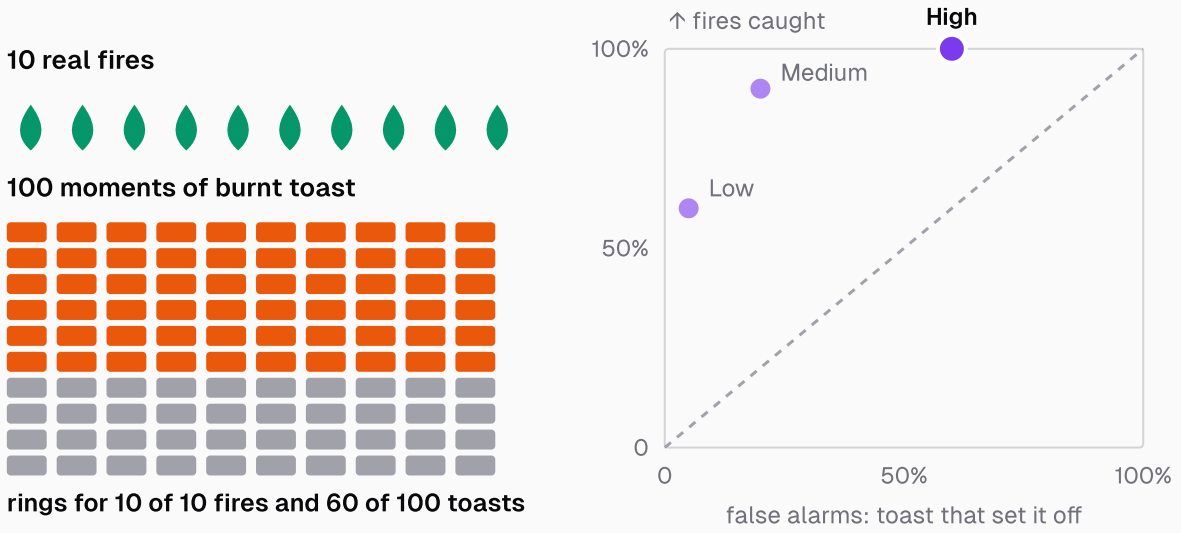
Over a year, a kitchen has 10 real fires and 100 moments of burnt toast. A good alarm rings for fires and stays quiet for toast. Its sensitivity dial is the threshold: the same idea as the guard's dial in the airport story.



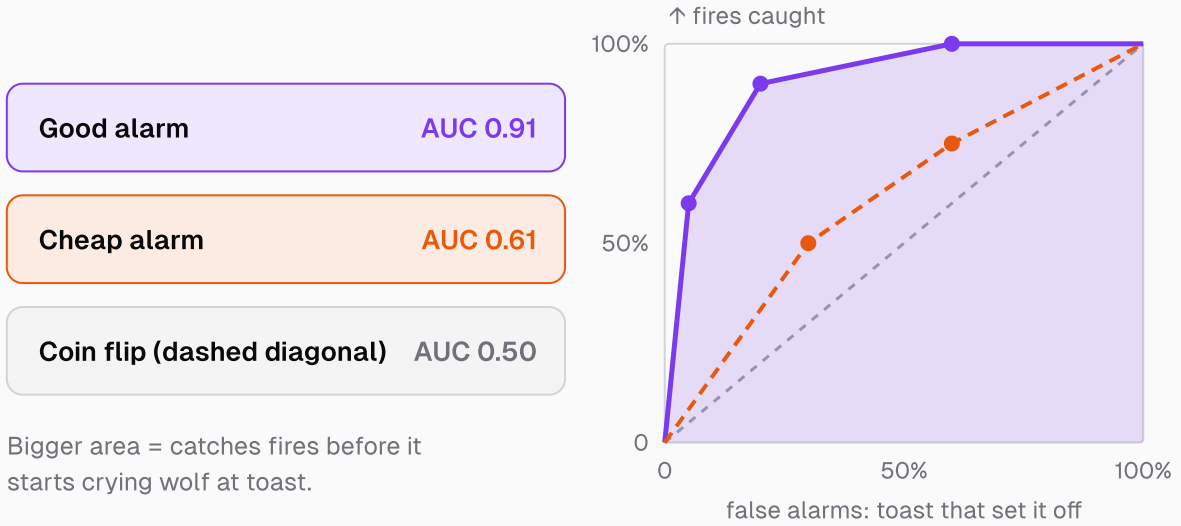
On **low**, it rings for 6 of the 10 fires (green) and only 5 of the 100 toasts (orange). That is one point on the chart: 5% false alarms, 60% of fires caught.



On **medium**, it catches 9 fires and rings for 20 toasts: the dot moves up (good) and right (bad).



On **high**, it catches all 10 fires but rings for 60 toasts. More sensitivity always buys catches with false alarms.



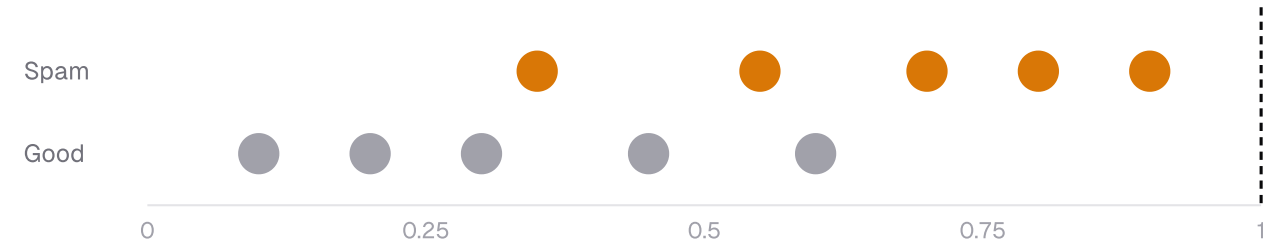
Connect the dots (plus "always off" at the bottom-left and "always on" at the top-right) and you get the ROC curve. A better alarm bows toward the top-left corner. The shaded area, **AUC**, is 0.91 for the good alarm and 0.61 for a cheap one. A coin flip scores 0.50.

Now the same thing with emails instead of fires.

Interactive **Every threshold becomes one dot** (shown at its default setting)

Step through ten emails. Watch each threshold land as a dot, then connect them.

Threshold 1.0: nothing is spammy enough, so nothing is flagged.



Spam caught (up)

0 of 5

Good emails flagged (right)

0 of 5

A great model climbs straight up before it moves right, hugging the top-left corner. A useless model moves up and right at the same pace along the diagonal: coin flipping. ROC stands for "receiver operating characteristic," a name left over from radar engineering. Read it as the "caught versus wrongly flagged" chart.

AUC: one number for the whole curve

AUC means area under the curve: how much of the box the curve fills. A perfect model scores 1.0. Random guessing scores 0.5. Because it covers every threshold, AUC ignores the threshold you picked and measures

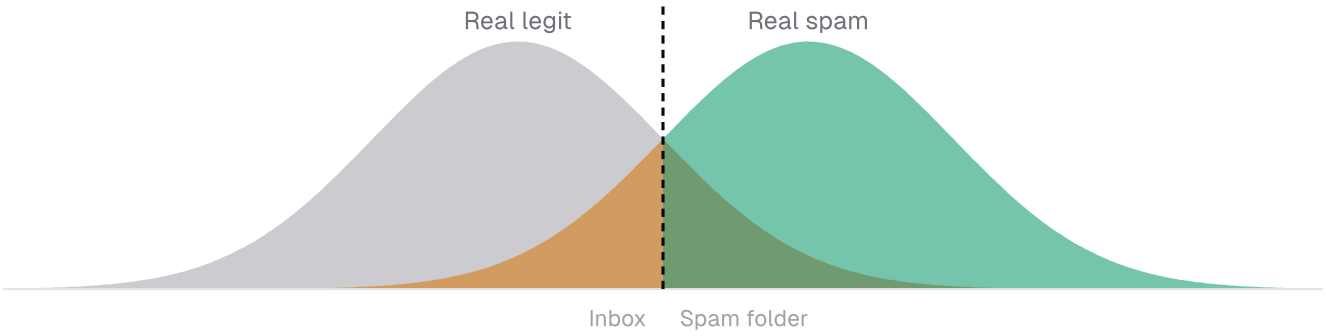
how far apart the humps are.

> **Key idea**
>
> Plain-English meaning of ROC-AUC: pick one random spam email and one random legit email. AUC is the chance the model gives the spam the higher score. In the ten-email example, spam outscores legit in 22 of 25 pairs, and 22 ÷ 25 = 0.88.

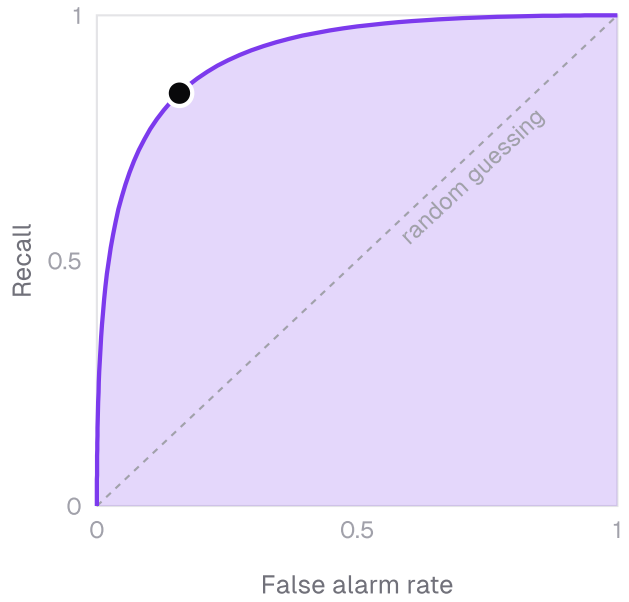
Interactive

ROC and PR curves, with spam share (shown at its default setting)

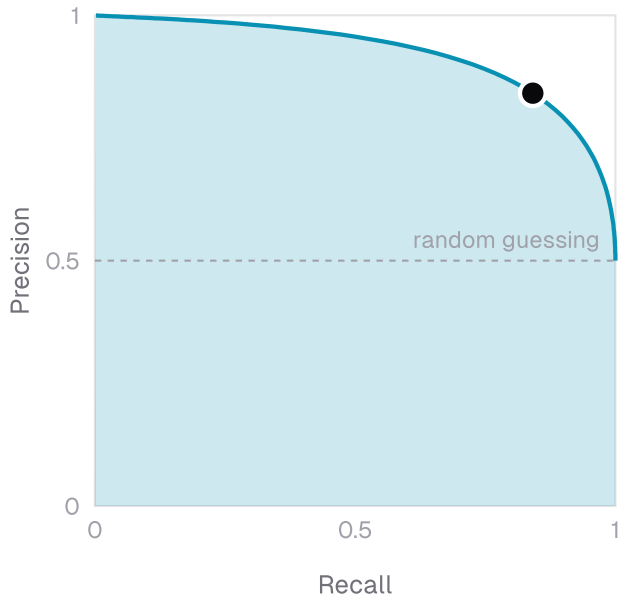
Move the threshold and only the dot moves. Change model quality and the curves move. Drop spam share to 5% and watch ROC-AUC stay put while PR-AUC sinks.



ROC curve: recall vs false alarm rate



PR curve: precision vs recall



Precision	Recall	False alarm rate	ROC-AUC
84%	84%	16%	0.92

PR-AUC
0.92

Plain words

Why there is a second curve

Drag spam share down to 5%. ROC-AUC does not change, because ROC never looks at how common each class is. The PR curve sinks, because even a small false alarm rate on a huge pile of legit email produces lots of false positives. ROC-AUC can look excellent while precision is poor. When the thing you are hunting for is rare, PR-AUC is the more honest number. Its random-guessing line sits at the spam share, so a PR-AUC of 0.3 is great when spam is 5% and poor when it is 50%.

Rough guide for ROC-AUC: 0.5 random, 0.7 weak, 0.8 decent, 0.9 and up strong. What counts as good depends on the problem.

For leaders

AUC chooses the model; the confusion matrix makes the ship decision

Use AUC to compare models or versions, because it does not depend on a threshold someone picked. Then make the ship decision on the confusion matrix at the operating point you will actually run. A model with higher AUC can be worse at your operating point if its curve crosses the other one there.

Watch out

Same classifier, same data, 2% prevalence, in a simulation of 5,000 samples: ROC-AUC 0.928, PR-AUC 0.470, against a PR-AUC baseline of 0.020. ROC-AUC says excellent. PR-AUC says mediocre but far better than chance. For rare-positive problems (safety violations, cheating detection, rare regressions) the PR number is the one telling the truth.

Principal asks

- Is this ROC-AUC or PR-AUC, and what is the positive rate?
- Do the two models' curves cross? Where is our operating point relative to the crossing?
- Does the score come with confidence intervals? (AUC on 200 examples has a wide one; see intervals and the bootstrap.)

Under the hood

AUC as a rank statistic, and average precision

ROC-AUC equals the probability that a random positive outscores a random negative, which is the Mann–Whitney U statistic scaled to [0, 1]:

$$\mathrm{AUC} = P(S^+ > S^-) + \tfrac{1}{2} P(S^+ = S^-) = \frac{R_+ - n_+(n_+ + 1)/2}{n_+ \, n_-}$$

R_+ is the sum of the ranks of the positive examples when all scores are ranked together (ties get average ranks); n_+ and n_- are the class counts.

PR-AUC is usually computed as average precision: walk down the ranking and average the precision at each point where a positive appears.

$$\mathrm{AP} = \frac{1}{n_+} \sum_{k:\, y_{(k)} = 1} \mathrm{Precision@}k$$

The baseline for a random ranking is the prevalence $\pi = n_+ / n$, which is why PR-AUC must always be read next to π.

Precision depends on prevalence directly. If a classifier has true positive rate TPR and false positive rate FPR, its precision at prevalence π is

$$\mathrm{Precision}(\pi) = \frac{\mathrm{TPR} \cdot \pi}{\mathrm{TPR} \cdot \pi + \mathrm{FPR} \cdot (1 - \pi)}$$

TPR and FPR (the ROC axes) do not change with π; precision does. That single line explains why ROC is prevalence-blind and PR is not.

python

```python
def roc_auc(y_true, scores):
    """Probability a random positive outscores a random negative (ties count half)."""
    order = sorted(range(len(scores)), key=lambda i: scores[i])
    ranks = [0.0] * len(scores)
    i = 0
    while i < len(order):
        j = i
        while j + 1 < len(order) and scores[order[j + 1]] == scores[order[i]]:
            j += 1
        avg = (i + j) / 2 + 1            # tied items share the average rank
        for k in range(i, j + 1):
            ranks[order[k]] = avg
        i = j + 1
    n_pos = sum(y_true)
    n_neg = len(y_true) - n_pos
```

```
    r_pos = sum(r for r, y in zip(ranks, y_true) if y == 1)
    return (r_pos - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)


def average_precision(y_true, scores):
    """Step-wise PR-AUC: mean precision at each rank where a positive appears."""
    order = sorted(range(len(scores)), key=lambda i: -scores[i])
    tp, ap = 0, 0.0
    for rank, i in enumerate(order, start=1):
        if y_true[i] == 1:
            tp += 1
            ap += tp / rank
    return ap / sum(y_true)
```

rust

```
/// Probability a random positive outscores a random negative (ties count half).
fn roc_auc(y_true: &[u8], scores: &[f64]) -> f64 {
    let mut order: Vec<usize> = (0..scores.len()).collect();
    order.sort_by(|&a, &b| scores[a].partial_cmp(&scores[b]).expect("NaN score"));
    let mut ranks = vec![0.0; scores.len()];
    let mut i = 0;
    while i < order.len() {
        let mut j = i;
        while j + 1 < order.len() && scores[order[j + 1]] == scores[order[i]] {
            j += 1;
        }
        let avg = (i + j) as f64 / 2.0 + 1.0; // tied items share the average rank
        for k in i..=j {
            ranks[order[k]] = avg;
        }
        i = j + 1;
    }
    let n_pos = y_true.iter().map(|&y| y as f64).sum::<f64>();
    let n_neg = y_true.len() as f64 - n_pos;
    let r_pos: f64 = ranks.iter().zip(y_true).filter(|&(_, &y)| y == 1).map(|(r, _)| r).sum();
    (r_pos - n_pos * (n_pos + 1.0) / 2.0) / (n_pos * n_neg)
}

/// Step-wise PR-AUC: mean precision at each rank where a positive appears.
fn average_precision(y_true: &[u8], scores: &[f64]) -> f64 {
    let mut order: Vec<usize> = (0..scores.len()).collect();
    // highest score first; the sort is stable, so ties keep input order
    order.sort_by(|&a, &b| scores[b].partial_cmp(&scores[a]).expect("NaN score"));
    let (mut tp, mut ap) = (0, 0.0);
    for (rank, &i) in (1..).zip(&order) {
        if y_true[i] == 1 {
            tp += 1;
            ap += tp as f64 / rank as f64;
        }
    }
    ap / y_true.iter().map(|&y| y as f64).sum::<f64>()
}
```

Better single numbers: MCC, balanced accuracy, kappa

If you must report one number, these are better choices than F1. Each still has a catch.

For leaders

Which single number for which audience

Number	Reads as	Best for	Catch
Balanced accuracy	Average of "right on positives" and "right on negatives"; 0.5 is always chance	Executive summaries	Still hides which error you make
MCC	Correlation between prediction and truth, –1 to +1	Comparing models with one number	Unintuitive scale; undefined when the model predicts one class
Cohen's kappa	Agreement above chance, 0 is chance, 1 is perfect	Judge versus human, human versus human	Depends on prevalence; report it with the pass-rate gap

Under the hood

Definitions

$$\mathrm{MCC} = \frac{\mathrm{TP}\cdot\mathrm{TN} - \mathrm{FP}\cdot\mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}, \qquad \mathrm{BA} = \tfrac{1}{2}\left(\tfrac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}} + \tfrac{\mathrm{T}}{\mathrm{TN}+}\right.$$

MCC uses all four margins, so swapping the class labels leaves it unchanged. F1 does not have that property.

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = \frac{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN}) + (\mathrm{TN}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})}{n^2}$$

p_o is observed agreement (plain accuracy); p_e is the agreement two independent raters with the same marginals would reach by luck. On a 95/5 split, luck alone gives about 90%.

python

```python
import math

def mcc(tp, fp, fn, tn):
    den = math.sqrt((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn))
    return (tp * tn - fp * fn) / den if den else 0.0   # 0 = chance when undefined

def balanced_accuracy(tp, fp, fn, tn):
    return (tp / (tp + fn) + tn / (tn + fp)) / 2

def kappa(tp, fp, fn, tn):
    n = tp + fp + fn + tn
    po = (tp + tn) / n
    pe = ((tp + fp) * (tp + fn) + (tn + fn) * (tn + fp)) / (n * n)
    return (po - pe) / (1 - pe) if pe != 1 else 0.0

# The "flag nothing" model on 1% positives: accuracy 0.99, MCC 0, BA 0.5, kappa 0
print(mcc(0, 0, 10, 990), balanced_accuracy(0, 0, 10, 990), kappa(0, 0, 10, 990))
```

rust

```rust
// Counts arrive as integers; convert once so products cannot overflow.
fn mcc(tp: u64, fp: u64, fn_: u64, tn: u64) -> f64 {
    let (tp, fp, fn_, tn) = (tp as f64, fp as f64, fn_ as f64, tn as f64);
    let den = ((tp + fp) * (tp + fn_) * (tn + fp) * (tn + fn_)).sqrt();
    if den != 0.0 { (tp * tn - fp * fn_) / den } else { 0.0 } // 0 = chance when undefined
}

fn balanced_accuracy(tp: u64, fp: u64, fn_: u64, tn: u64) -> f64 {
    let (tp, fp, fn_, tn) = (tp as f64, fp as f64, fn_ as f64, tn as f64);
    (tp / (tp + fn_) + tn / (tn + fp)) / 2.0
}

fn kappa(tp: u64, fp: u64, fn_: u64, tn: u64) -> f64 {
    let (tp, fp, fn_, tn) = (tp as f64, fp as f64, fn_ as f64, tn as f64);
    let n = tp + fp + fn_ + tn;
    let po = (tp + tn) / n;
    let pe = ((tp + fp) * (tp + fn_) + (tn + fn_) * (tn + fp)) / (n * n);
    if pe != 1.0 { (po - pe) / (1.0 - pe) } else { 0.0 }
}

fn main() {
    // The "flag nothing" model on 1% positives: accuracy 0.99, MCC 0, BA 0.5, kappa 0
    // {:?} prints floats the way Python does (0.0, not 0)
    println!("{:?} {:?} {:?}", mcc(0, 0, 10, 990), balanced_accuracy(0, 0, 10, 990), kappa(0, 0, 10, 990));
}
```

MCC and kappa are often nearly identical on binary problems. Prefer MCC when scoring a model and kappa when scoring agreement between raters, because that is what each was designed to communicate.

Calibration: is the confidence honest?

Diagram

Two weather forecasters and the 70% test

Calibration is whether "70% sure" really means right 7 times in 10.

Forecaster A: "70% chance of rain"

It rained on 7 of these 10 days.

Said 70%, happened 70%: honest (calibrated)

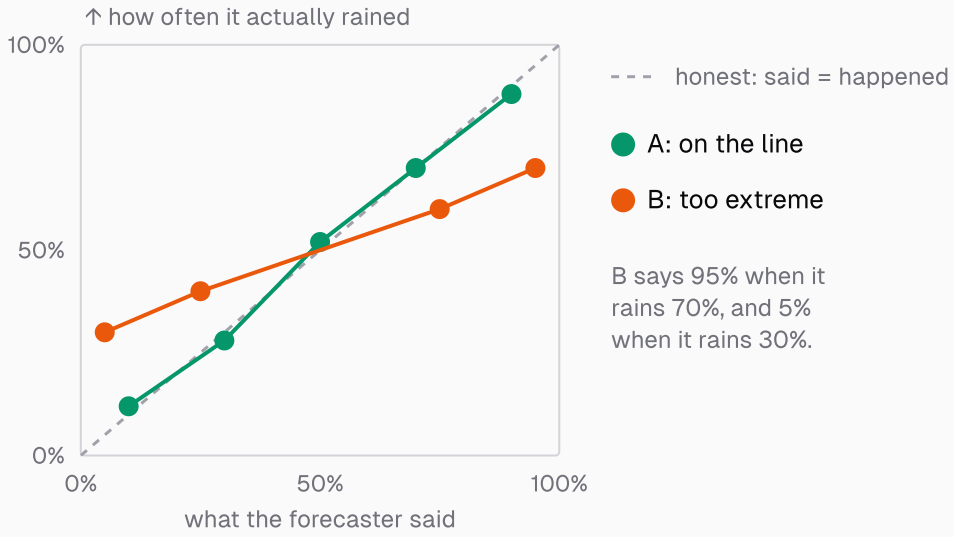
Forecaster A says "70% chance of rain" on ten days, and it rains on seven. Her number means what it says. That is calibration.

Forecaster B: "95% chance of rain"

Same 10 days, same 7 rainy ones. Same ranking as A.

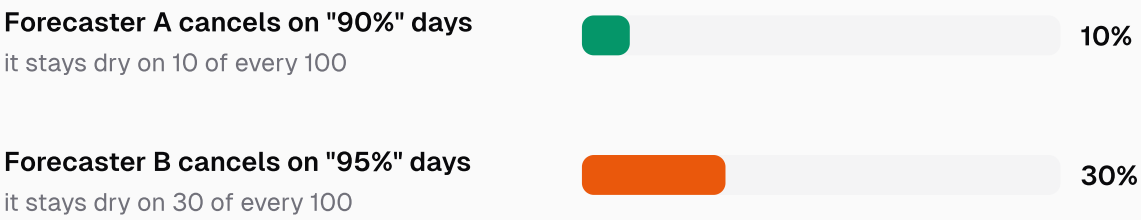
Said 95%, happened 70%: overconfident

Forecaster B looks at the same ten days and says "95%". It still rains on seven. B picks the rainy days just as well as A (same ranking), but his confidence number is inflated.



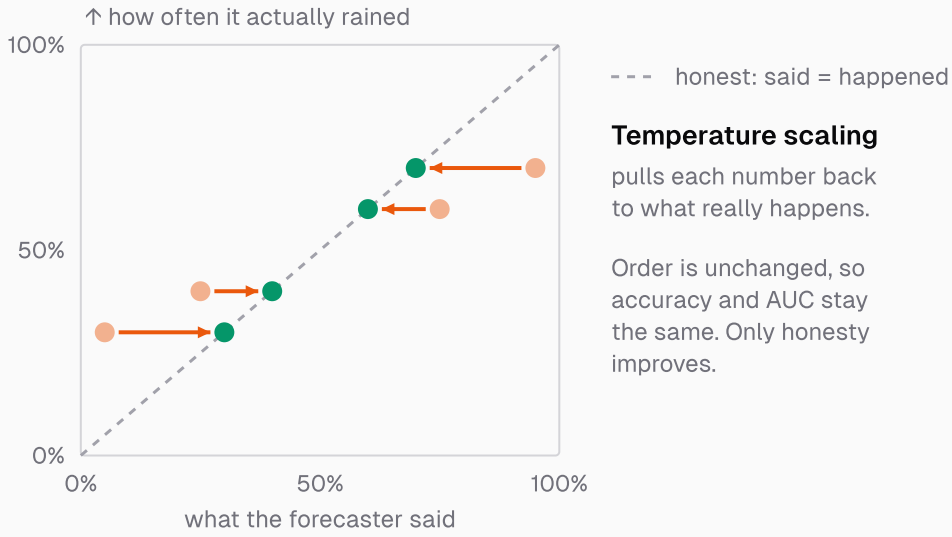
A reliability diagram plots what each forecaster said against what happened, over many days. A hugs the honest diagonal. B is too extreme at both ends: his 95% days rain 70% of the time and his 5% days rain 30% of the time.

Rule: cancel the picnic when the forecast is 90% or higher



Wasted cancellations. Same days ranked the same way, three times the mistakes.

Why it matters: any rule that acts on the number itself. "Cancel the picnic at 90% or higher" wastes 10% of cancellations with A and 30% with B. Auto-approving agent actions, routing to humans, and abstaining all work the same way.



The fix is cheap. Temperature scaling, fit on held-out data, pulls B's numbers back onto the diagonal without changing which day ranks above which. Ranking quality (AUC) stays the same; the confidence becomes honest.

Plain words

A weather forecaster who says 70%

If a forecaster says "70% chance of rain" on 100 days, it should rain on about 70 of them. That is calibration. A model can rank emails perfectly (every spam above every legit) and still be badly wrong about how sure it should be. Ranking and honesty about confidence are different skills.

For leaders

When calibration is the whole ballgame

Calibration matters whenever a downstream decision uses the confidence number itself: routing to a bigger model, abstaining and asking a human, auto-approving an action, or combining scores from several judges. Accuracy and AUC are blind to it. If a system makes decisions on confidence, require a reliability diagram and ECE in the review. The usual fix, temperature scaling on a held-out set, is cheap and does not change rankings, so there is rarely an excuse to ship miscalibrated scores.

Under the hood

Brier score and expected calibration error

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2, \qquad \text{ECE} = \sum_{b=1}^{B}\frac{|S_b|}{n}\left|\text{acc}(S_b) - \text{conf}(S_b)\right|$$

Brier is a proper scoring rule: it is minimised by reporting your true belief. ECE buckets predictions by confidence S_b and compares mean confidence to observed accuracy in each bucket. ECE depends on the number of bins, so always state B.

python

```python
def brier(y_true, probs):
    # squared, not absolute: squaring is what makes it a proper scoring rule
    return sum((p - y) ** 2 for p, y in zip(probs, y_true)) / len(y_true)

def ece(y_true, probs, bins=10):
    n, total = len(y_true), 0.0
    for b in range(bins):
        lo, hi = b / bins, (b + 1) / bins
        idx = [i for i, p in enumerate(probs) if (lo < p <= hi) or (b == 0 and p == 0)]
        if not idx:
            continue
        conf = sum(probs[i] for i in idx) / len(idx)
        acc = sum(y_true[i] for i in idx) / len(idx)
        total += len(idx) / n * abs(acc - conf)
    return total
```

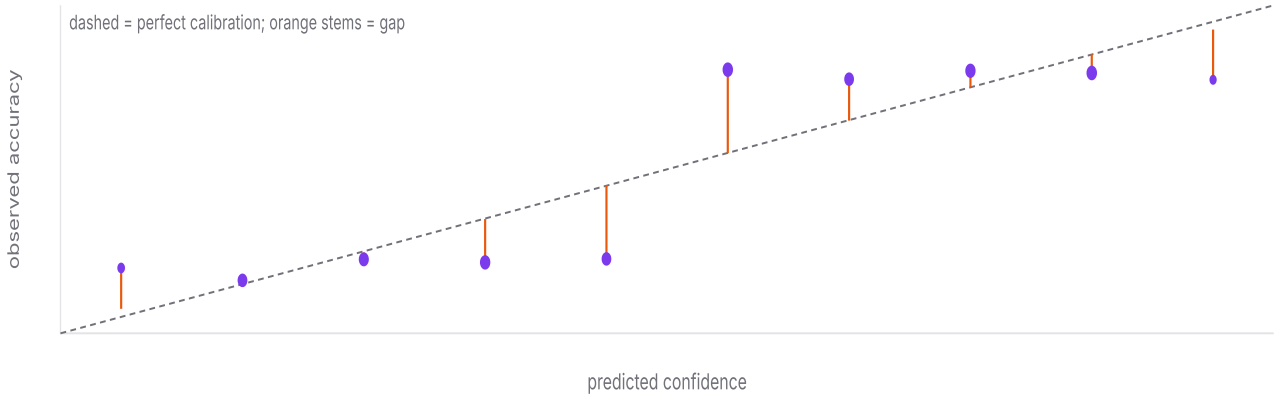
rust

```rust
fn brier(y_true: &[u8], probs: &[f64]) -> f64 {
    // squared, not absolute: squaring is what makes it a proper scoring rule
    probs.iter().zip(y_true).map(|(p, &y)| (p - y as f64).powi(2)).sum::<f64>() / y_true.len() as f64
}

/// Pass bins = 10 to match the Python default.
fn ece(y_true: &[u8], probs: &[f64], bins: usize) -> f64 {
    let (n, mut total) = (y_true.len() as f64, 0.0);
    for b in 0..bins {
        let (lo, hi) = (b as f64 / bins as f64, (b + 1) as f64 / bins as f64);
        let idx: Vec<usize> = (0..probs.len())
            .filter(|&i| (lo < probs[i] && probs[i] <= hi) || (b == 0 && probs[i] == 0.0))
            .collect();
        if idx.is_empty() {
            continue;
        }
        let m = idx.len() as f64;
        let conf = idx.iter().map(|&i| probs[i]).sum::<f64>() / m;
        let acc = idx.iter().map(|&i| y_true[i] as f64).sum::<f64>() / m;
        total += m / n * (acc - conf).abs();
    }
    total
}
```

> **Note**
>
> In a simulation with 20% label noise, a moderately confident model scored Brier 0.162 and ECE 0.050. The same model pushed to 0.99 / 0.01 confidence scored Brier 0.196 and ECE 0.190: identical ranking, identical accuracy, nearly four times worse calibration.

Interactive **Calibration** (shown at its default setting)

Sharpening pushes confidence toward 0 and 1 without changing the ranking at all. Accuracy and AUC do not move. Watch ECE and Brier come apart. If you route, abstain or auto approve on a confidence score, this is the property that matters.



Brier 0.1835 **ECE** 0.1126

Sharpening pushes confidence toward 0 and 1 without changing the ranking. Accuracy and AUC stay put while calibration collapses. If you route, abstain or auto approve on confidence, this is the number that breaks you.

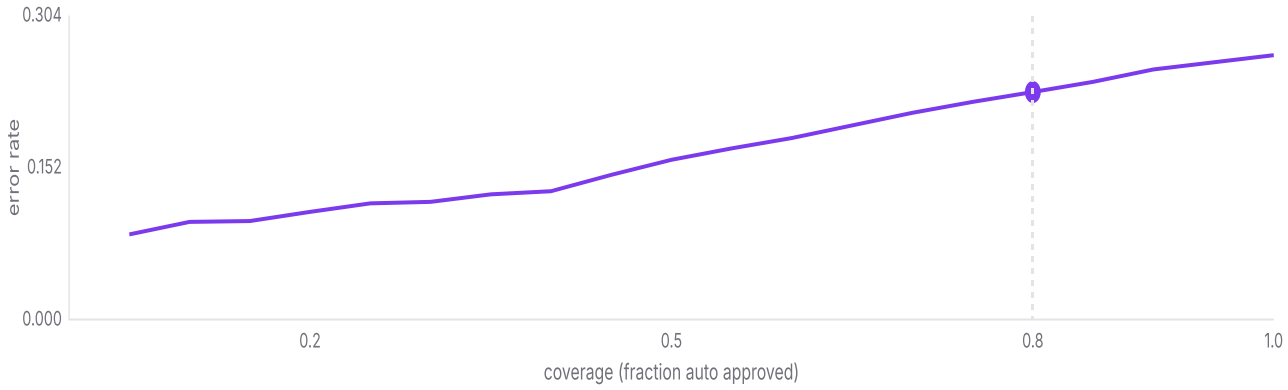
Selective prediction: when the system can abstain

Instead of one accuracy number you get a curve: error as a function of how often the system is willing to answer. This is the right frame for any judge that can say "unsure," any agent that can ask the user before acting, and any classifier that can hand cases to a human.

Interactive

Auto approve threshold picker (shown at its default setting)

An agent that auto-approves its own tool calls, framed as a coverage risk curve. Set the cost of asking the user and the widget finds the coverage that minimises total cost. Drag confidence quality to zero and watch the curve go flat.



Auto approve everything: error 0.2640 **at 80% coverage:** error 0.2272 (1.2x better)
Total cost at your setting 0.1858 **cost minimising coverage** 5% (cost 0.0233)
Drag confidence quality to 0 and the curve goes flat. A flat curve means abstention buys you nothing, and that is itself the finding: fix the confidence signal before tuning the threshold.

Worked problem

Where do you set auto-approve? An agent's proposed actions are right 92.8% of the time overall. Sort by the model's confidence and only auto-approve the most confident fraction:

Coverage	Auto-approved	Asked the user	Error rate	vs full coverage
100%	4,000	0	0.0717	—
90%	3,600	400	0.0550	1.3× fewer
80%	3,200	800	0.0350	**2.0× fewer**
70%	2,800	1,200	0.0232	3.1× fewer
50%	2,000	2,000	0.0090	8.0× fewer

Giving up 20% of the decisions halves the error rate. That turns a product argument into arithmetic: the design question is what an interruption costs relative to a mistake. If the curve comes back flat, the confidence signal does not track correctness and no threshold will help. Fix the signal before tuning the threshold.

Under the hood

The coverage–risk curve

Sort by confidence c_i descending and answer only the top fraction κ (coverage). Risk is the error rate over the answered set only; abstained items are handed off, not counted wrong.

$$\operatorname{risk}(\kappa) = \frac{1}{\lfloor \kappa n \rfloor} \sum_{i \in \operatorname{top} \lfloor \kappa n \rfloor} \mathbb{1}[\hat{y}_i \neq y_i], \qquad \text{total cost}(\kappa) = \kappa \operatorname{risk}(\kappa)\, c_{\text{error}} + (1 - \kappa)\, c_{\text{ask}}$$

The area under the risk–coverage curve (AURC) summarises how well confidence tracks correctness; lower is better.

python

```python
def coverage_risk(y_true, preds, conf, coverages=(1.0, 0.9, 0.8, 0.6, 0.5)):
    order = sorted(range(len(y_true)), key=lambda i: -conf[i])
    out = []
    for cov in coverages:
```

```
        k = max(1, int(cov * len(order)))
        err = sum(preds[i] != y_true[i] for i in order[:k]) / k
        out.append({"coverage": cov, "risk": err, "answered": k})
    return out
```

rust

```rust
const DEFAULT_COVERAGES: [f64; 5] = [1.0, 0.9, 0.8, 0.6, 0.5];

struct CoveragePoint {
    coverage: f64,
    risk: f64,
    answered: usize,
}

fn coverage_risk<T: PartialEq>(y_true: &[T], preds: &[T], conf: &[f64], coverages: &[f64]) -> Vec<CoveragePoint>
{
    let mut order: Vec<usize> = (0..y_true.len()).collect();
    order.sort_by(|&a, &b| conf[b].partial_cmp(&conf[a]).expect("NaN confidence"));
    let mut out = Vec::new();
    for &cov in coverages {
        let k = ((cov * order.len() as f64) as usize).max(1);
        let err = order[..k].iter().filter(|&&i| preds[i] != y_true[i]).count() as f64 / k as f64;
        out.push(CoveragePoint { coverage: cov, risk: err, answered: k });
    }
    out
}
```

Expected cost: the metric that is actually about your problem

Everything above is a proxy. If you know the relative costs of the two errors, skip the proxies and minimise expected cost directly.

For leaders

Get the cost ratio on the table

Most metric arguments are cost arguments in disguise. One team wants precision, another wants recall, and neither has said what a false alarm and a miss each cost. Force that number out, even as a rough ratio ("a missed harmful output is about 20 times worse than a wrongly blocked request"). Once it exists, the threshold choice is arithmetic and the argument ends.

Under the hood

Cost per decision and the Bayes-optimal threshold

$$\text{Cost}(\tau) = \frac{c_{FP}\,\text{FP}(\tau) + c_{FN}\,\text{FN}(\tau)}{n}, \qquad \tau^{*} = \frac{c_{FP}}{c_{FP} + c_{FN}} \text{ if scores are calibrated prol}$$

Flag an item when $p\,c_{FN} > (1-p)\,c_{FP}$. With equal costs that is $p > 0.5$; if a miss costs 9 times a false alarm, flag at $p > 0.1$. If scores are not calibrated, sweep thresholds on held-out data instead.

python

```python
def best_threshold(y_true, scores, c_fp, c_fn):
    """Only observed scores can change the confusion matrix, so sweep those."""
    best = None
    for t in sorted(set(scores)):
        fp = sum(s >= t and y == 0 for s, y in zip(scores, y_true))
        fn = sum(s < t and y == 1 for s, y in zip(scores, y_true))
        cost = (c_fp * fp + c_fn * fn) / len(y_true)
        if best is None or cost < best["cost"]:
            best = {"threshold": t, "cost": cost}
    return best   # tune on a validation split, report on a separate test split
```

rust

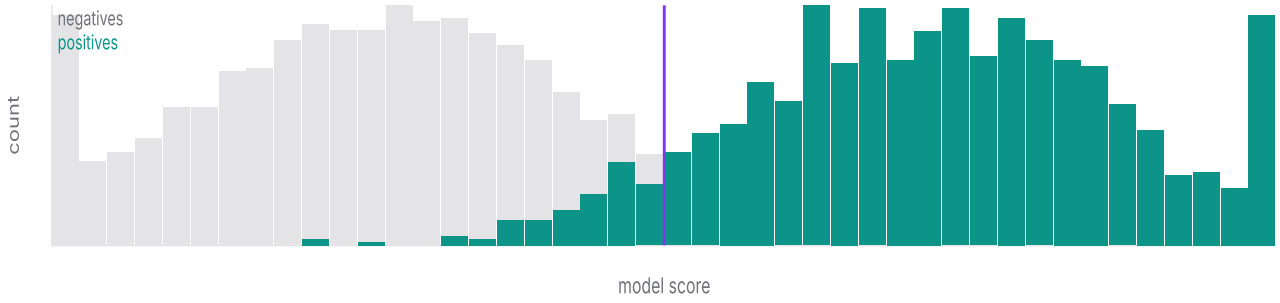
```rust
struct Best {
    threshold: f64,
    cost: f64,
}

/// Only observed scores can change the confusion matrix, so sweep those.
fn best_threshold(y_true: &[u8], scores: &[f64], c_fp: f64, c_fn: f64) -> Option<Best> {
    let mut thresholds = scores.to_vec();
    thresholds.sort_by(|a, b| a.partial_cmp(b).expect("NaN score"));
    thresholds.dedup(); // sorted(set(scores))
    let mut best: Option<Best> = None;
    for t in thresholds {
        let fp = scores.iter().zip(y_true).filter(|&(&s, &y)| s >= t && y == 0).count() as f64;
        let fn_ = scores.iter().zip(y_true).filter(|&(&s, &y)| s < t && y == 1).count() as f64;
        let cost = (c_fp * fp + c_fn * fn_) / y_true.len() as f64;
        if best.as_ref().map_or(true, |b| cost < b.cost) {
            best = Some(Best { threshold: t, cost });
        }
    }
    best // tune on a validation split, report on a separate test split
}
```

The advanced playground below puts every metric from this chapter on one screen, including a cost ratio slider that finds the cost-minimising threshold.

Interactive

Classification playground (shown at its default setting)

Two score distributions, a threshold, and every metric in this chapter updating live. Drop prevalence to 0.02 and watch accuracy stay high while MCC collapses and ROC-AUC keeps flattering the model that PR-AUC exposes. Then move the cost ratio and find the threshold that actually minimizes expected cost.



TP	FP
1126	207
FN	**TN**
96	2571

ROC (AUC 0.978)

PR (AP 0.956, baseline 0.300)

Accuracy	Precision	Recall	F1	Balanced acc
0.924	0.845	0.921	0.881	0.923
			ignores TN	
MCC	**Kappa**	**ROC-AUC**	**PR-AUC**	**Expected cost**
0.828	0.826	0.978	0.956	0.076
best single number			baseline = 0.300	lower is better

Try this

Set separation to 0.45 and threshold to 0.50, then drag prevalence from 0.30 down to 0.02. Accuracy barely moves (0.924 to 0.925) and ROC-AUC rises slightly (0.978 to 0.985). Meanwhile precision falls from 0.845 to 0.199, F1 from 0.881 to 0.328, and MCC from 0.828 to 0.412: 294 false positives for 73 true positives. Same classifier, same threshold. Accuracy and ROC-AUC mislead here; the other three do not.

Choosing, and the ceiling nobody mentions

Situation	Report	Do not report alone
Balanced classes, symmetric costs	Accuracy plus confusion matrix	F1 (adds nothing here)
Rare positive class	Precision, recall, PR-AUC, prevalence	Accuracy, ROC-AUC
One number for a model comparison	MCC	F1
Judge versus human agreement	Kappa, prevalence, and the pass-rate gap	Raw agreement
Known asymmetric costs	Expected cost at the tuned threshold	F1 at 0.5
Confidence used downstream	Brier, ECE, reliability diagram	Accuracy
System can abstain	Coverage–risk curve	A single accuracy number
Multiclass	Per class, macro and weighted, top confused pairs	Weighted F1

Watch out

Your metric cannot exceed your label quality. If two humans agree at kappa 0.7 on a taxonomy, a classifier scoring 0.85 against one human's labels is not beating humans; it is fitting one annotator. Measure human-to-human agreement first and report every classifier number against that ceiling. The full decision guide is in Which metric or test, when.

Eval lens

Every number in this chapter is itself an estimate from a finite sample. A precision of 0.85 on 40 flagged items has a 95% interval roughly 0.71 to 0.93. Put intervals on classifier metrics the same way you would on pass rates; see statistics for eval sets and sampling distributions.

Part II · Chapter 11

LLM as judge: a classifier you have to measure

An LLM judge reads a model's output and returns a verdict: pass or fail, better or worse, safe or unsafe. That makes it two things at once. It is a classifier, so everything in the previous chapter applies. And it is a measuring instrument, so every number you publish inherits its errors. This chapter covers how to validate a judge, the biases to control, how to get more out of a yes/no judge, and how to run judges responsibly at scale.

Plain words

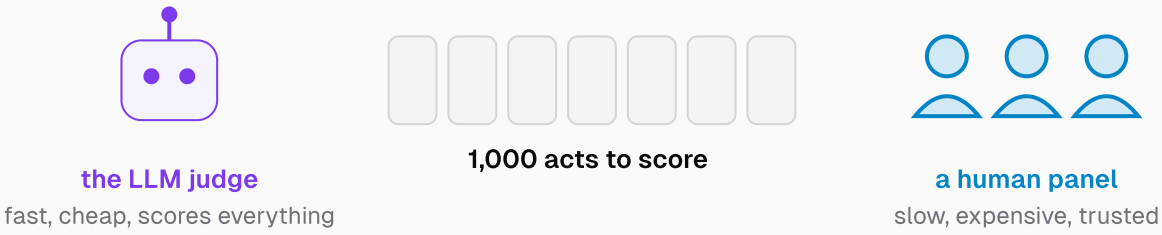
A teacher grading a thousand essays

Imagine hiring a teacher to grade a thousand essays. Before you trust their grades, you would grade a few yourself and compare. You would check whether they always prefer longer essays, or whichever essay they read first. You would notice if they grade everything a little too kindly. An LLM judge is that teacher. It is fast and cheap, but you only know whether its grades mean anything by checking it against people.

Diagram

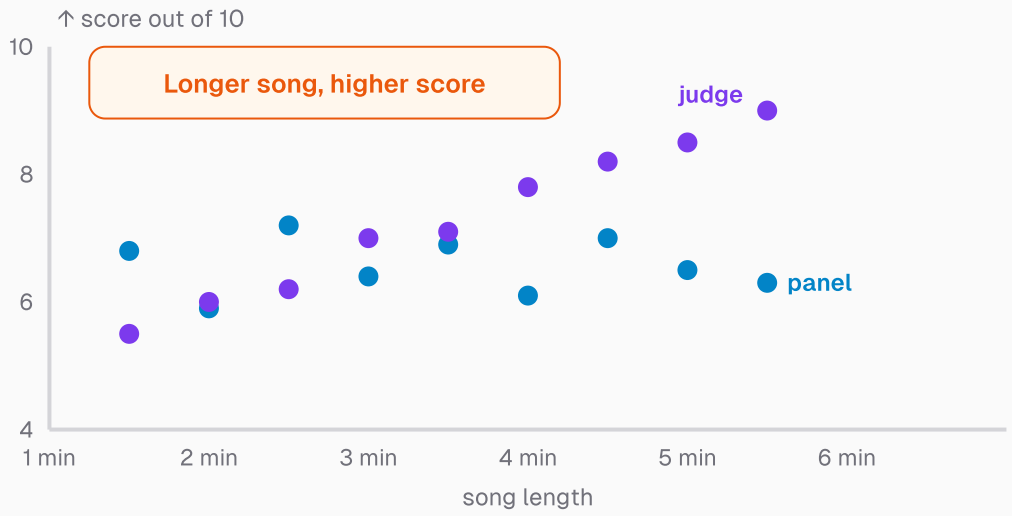
A talent-show judge you have to audit

Hire a judge for a thousand acts, then catch its habits before they reach your numbers.

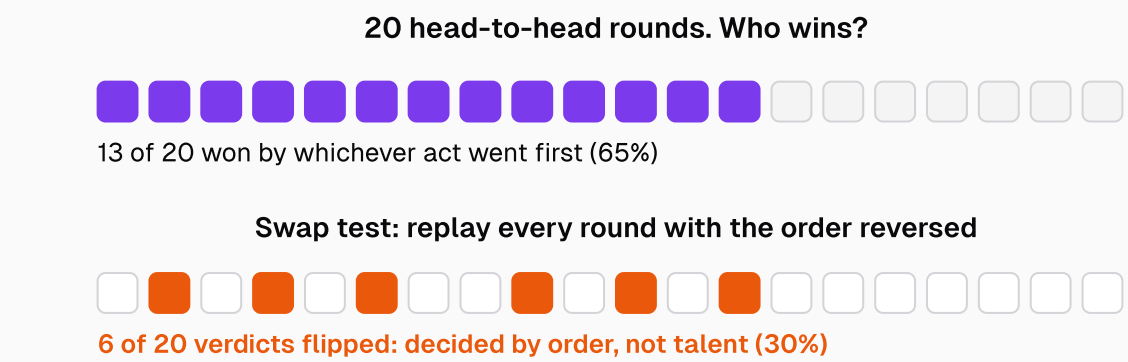


Before trusting the judge on all 1,000, check it against the panel on a sample.

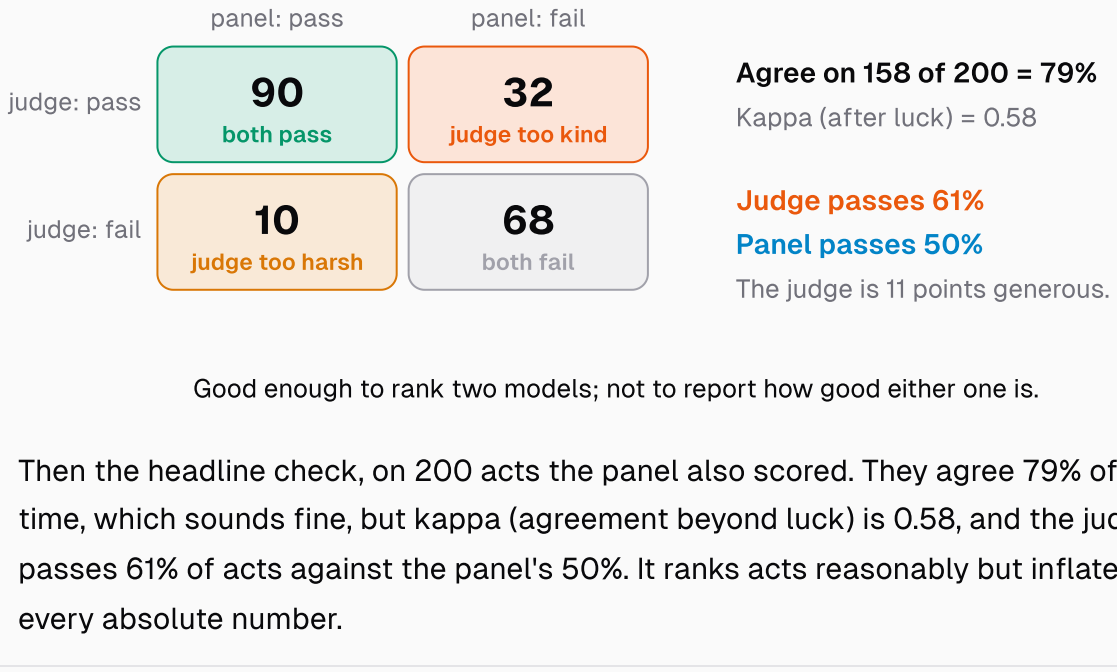
An LLM judge is like a talent-show judge who can score a thousand acts overnight. A human panel is slow and expensive but trusted. The job is to find out how far the fast judge can be trusted, using the panel on a sample.



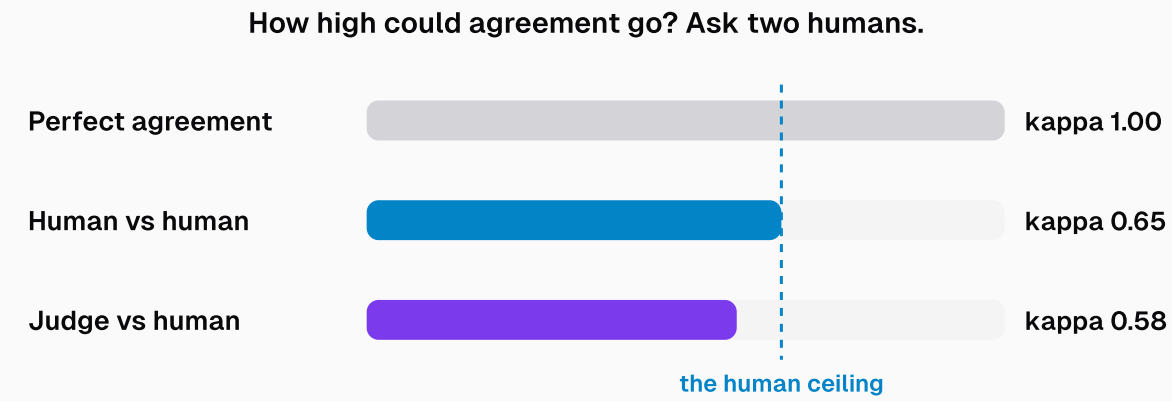
First habit: **length bias**. Plot the judge's scores against song length and they climb with every extra minute, while the panel's scores for the same acts stay flat. The judge is rewarding length, not talent.



Second habit: **position bias**. In head-to-head rounds the judge picks whichever act performed first 65% of the time. Replaying each round in reverse order exposes it: 6 of 20 verdicts flip. Those six were decided by order, not talent.



Then the headline check, on 200 acts the panel also scored. They agree 79% of the time, which sounds fine, but kappa (agreement beyond luck) is 0.58, and the judge passes 61% of acts against the panel's 50%. It ranks acts reasonably but inflates every absolute number.



The judge is close to the ceiling. Clearer rubrics help more than a smarter judge.

Last check: how well do two panel members agree with each other? Here, kappa 0.65. That is the ceiling. A judge at 0.58 is close to it, so the next gain comes from clearer scoring rules, not a stronger judge.

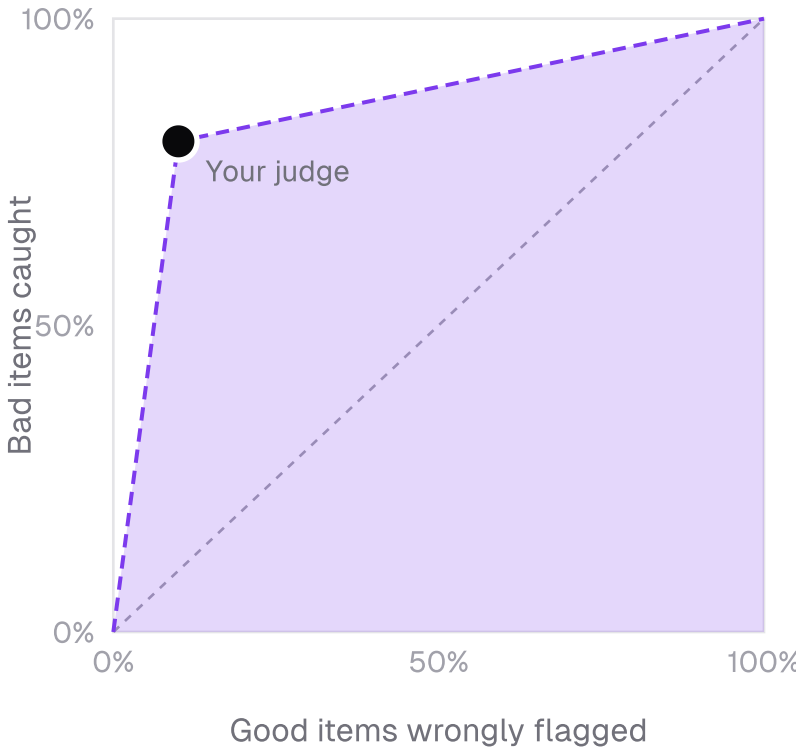
A yes/no judge gives you one dot, not a curve

Most judges answer "Pass" or "Fail." There is no score, so there is no threshold to slide. You get exactly one dot on the ROC chart from the previous chapter.

Interactive

What a binary judge looks like on an ROC chart (shown at its default setting)

Switch between the three views. The middle one is the trap most teams fall into when they iterate on judge prompts.



Bad items caught
80%

Good items flagged
10%

"AUC" from one dot
0.85

Plain words

Reading the three views

Binary judge only. Precision, recall, and F1 still work, because you can still fill in the confusion matrix. AUC barely makes sense. Forced onto one dot, it is the area under two straight lines, which equals the average of "bad items caught" and "good items left alone" (balanced accuracy). It adds nothing new.

The prompt becomes your threshold. "Only fail clear violations" moves the dot down and left: fewer false alarms, more misses. "Fail anything suspicious" moves it up and right. Rewording slides the dot along the same curve. A real improvement is the green dot: up and left at once. When you test a new prompt, check whether both numbers improved or one gained at the other's expense.

Getting the curve back. If your API exposes token probabilities (logprobs), read how confident the judge was in the word "Fail," like 0.97 or 0.52. That turns one dot into a full curve. You can pick any threshold without rewriting the prompt, and compute real ROC-AUC and PR-AUC. Without logprobs, run the judge 5 to 10 times per item and use the share of "Fail" votes as the score.

Under the hood

Single-point AUC and scores from logprobs

With one operating point $(\mathrm{FPR}, \mathrm{TPR})$, the ROC "curve" is two segments through $(0,0)$, the point, and $(1,1)$. Its area is

$$\mathrm{AUC}_{1\text{ point}} = \tfrac{1}{2}\,\mathrm{TPR}\cdot\mathrm{FPR} + \tfrac{1}{2}(1-\mathrm{FPR})(1+\mathrm{TPR}) = \frac{\mathrm{TPR}+(1-\mathrm{FPR})}{2}$$

Exactly balanced accuracy. With TPR 0.8 and FPR 0.1, that is 0.85.

A continuous score from a single call: renormalise the probability of the two verdict tokens at the answer position.

$$s = \frac{p(\mathbf{Fail} \mid \mathrm{prompt})}{p(\mathbf{Fail} \mid \mathrm{prompt}) + p(\mathbf{Pass} \mid \mathrm{prompt})}$$

Check that the verdict token appears where you read it (put the verdict last, after reasoning, in a fixed format). The renormalised score is usually miscalibrated; that is fine for ranking and AUC, and fixable with temperature scaling if you need calibrated probabilities.

python

```python
import math

def judge_score(top_logprobs, pos="Fail", neg="Pass"):
    """top_logprobs: {token: logprob} at the verdict position."""
    lp_pos = top_logprobs.get(pos, -50.0)   # treat a missing token as near-zero mass
    lp_neg = top_logprobs.get(neg, -50.0)
    a, b = math.exp(lp_pos), math.exp(lp_neg)
    return a / (a + b)

def vote_score(verdicts):
    """Fallback without logprobs: share of 'Fail' over k samples at temperature > 0."""
    return sum(v == "Fail" for v in verdicts) / len(verdicts)
```

rust

```rust
use std::collections::HashMap;

/// top_logprobs: {token: logprob} at the verdict position.
/// The Python defaults are pos = "Fail", neg = "Pass".
fn judge_score(top_logprobs: &HashMap<&str, f64>, pos: &str, neg: &str) -> f64 {
    let lp_pos = top_logprobs.get(pos).copied().unwrap_or(-50.0); // treat a missing token as near-zero mass
    let lp_neg = top_logprobs.get(neg).copied().unwrap_or(-50.0);
    let (a, b) = (lp_pos.exp(), lp_neg.exp());
    a / (a + b)
```

```rust
}

/// Fallback without logprobs: share of 'Fail' over k samples at temperature > 0.
fn vote_score(verdicts: &[&str]) -> f64 {
    verdicts.iter().filter(|&&v| v == "Fail").count() as f64 / verdicts.len() as f64
}
```

Agreement with humans

The first question about any judge is how well it agrees with people. Raw agreement is the wrong summary: two raters agree a lot by luck when one label is common. Use Cohen's kappa, and always check the pass-rate gap.

Worked problem

Validate a judge against 200 human labels. Judge and human agree on 158 items. Is the judge good enough to ship?

Metric	Value	Reading
Agreement	0.790	Looks acceptable
Cohen's kappa	0.580	Moderate once you discount luck
Precision (judge "pass")	0.738	About a quarter of its passes are wrong
Recall (judge "pass")	0.900	It catches most real passes
Judge pass rate	**0.610**	
Human pass rate	**0.500**	**The judge is 11 points generous**

The confusion matrix behind this is TP 90, FP 32, FN 10, TN 68. The finding is invisible if you only look at agreement: the judge **ranks** models fine, but every **absolute** number it produces is inflated by about 11 points. That is acceptable for comparing two models and unacceptable for reporting how good either one is. Next step: have a second human label 100 of the items and compute human-to-human kappa. If two humans only reach 0.65, a judge at 0.58 is close to as good as the task allows.

Under the hood

Kappa and the judge report

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = \sum_{\ell} p_A(\ell)\, p_B(\ell)$$

p_o is observed agreement; p_e is the agreement expected if each rater labelled independently at their own rates. In the worked problem, $p_o = 0.79$ and $p_e = 0.61 \cdot 0.5 + 0.39 \cdot 0.5 = 0.5$, so $\kappa = 0.58$.

python

```python
def cohens_kappa(a, b):
    n = len(a)
    labels = set(a) | set(b)
    po = sum(x == y for x, y in zip(a, b)) / n
    pe = sum((a.count(l) / n) * (b.count(l) / n) for l in labels)
    return 1.0 if pe == 1 else (po - pe) / (1 - pe)

def judge_report(judge, human):
    """The numbers to put next to any judge you ship (1 = pass)."""
    tp = sum(j == 1 and h == 1 for j, h in zip(judge, human))
    fp = sum(j == 1 and h == 0 for j, h in zip(judge, human))
    fn = sum(j == 0 and h == 1 for j, h in zip(judge, human))
    n = len(judge)
    return {
        "agreement": sum(j == h for j, h in zip(judge, human)) / n,
        "kappa": cohens_kappa(judge, human),
        "precision": tp / (tp + fp) if tp + fp else 0.0,
        "recall": tp / (tp + fn) if tp + fn else 0.0,
        # these two lines are the point: good agreement can hide a generous judge
        "judge_pass_rate": (tp + fp) / n,
        "human_pass_rate": (tp + fn) / n,
    }
```

rust

```rust
use std::collections::HashSet;
use std::hash::Hash;

fn cohens_kappa<T: Eq + Hash>(a: &[T], b: &[T]) -> f64 {
    let n = a.len() as f64;
    let labels: HashSet<&T> = a.iter().chain(b).collect();
    let po = a.iter().zip(b).filter(|(x, y)| x == y).count() as f64 / n;
    let count = |v: &[T], l: &T| v.iter().filter(|&x| x == l).count() as f64;
    let pe: f64 = labels.iter().map(|&l| (count(a, l) / n) * (count(b, l) / n)).sum();
    if pe == 1.0 { 1.0 } else { (po - pe) / (1.0 - pe) }
}

struct JudgeReport {
    agreement: f64,
    kappa: f64,
    precision: f64,
```

```
    recall: f64,
    // these two fields are the point: good agreement can hide a generous judge
    judge_pass_rate: f64,
    human_pass_rate: f64,
}

/// The numbers to put next to any judge you ship (1 = pass).
fn judge_report(judge: &[u8], human: &[u8]) -> JudgeReport {
    let pairs = || judge.iter().zip(human);
    let tp = pairs().filter(|&(&j, &h)| j == 1 && h == 1).count() as f64;
    let fp = pairs().filter(|&(&j, &h)| j == 1 && h == 0).count() as f64;
    let fn_ = pairs().filter(|&(&j, &h)| j == 0 && h == 1).count() as f64;
    let n = judge.len() as f64;
    JudgeReport {
        agreement: pairs().filter(|&(j, h)| j == h).count() as f64 / n,
        kappa: cohens_kappa(judge, human),
        precision: if tp + fp > 0.0 { tp / (tp + fp) } else { 0.0 },
        recall: if tp + fn_ > 0.0 { tp / (tp + fn_) } else { 0.0 },
        // these two lines are the point: good agreement can hide a generous judge
        judge_pass_rate: (tp + fp) / n,
        human_pass_rate: (tp + fn_) / n,
    }
}
```

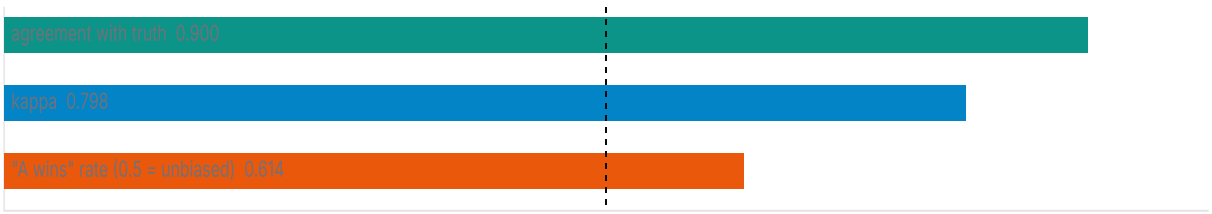
Put a confidence interval on kappa too. On 200 items, a bootstrap interval on $\kappa = 0.58$ is roughly ±0.11, which matters when comparing two judge prompts. See intervals and the bootstrap.

The biases you have to control for

Bias	What it does	Mitigation
Position	Prefers whichever response is shown first (or second) in a pairwise comparison	Run both orders, average, and report the disagreement rate as a bias estimate
Length	Scores longer answers higher regardless of quality	Check score against token count; add length-matched controls or a length-controlled win rate
Self-preference	A model judging its own family scores it higher	Use a judge from a different family, or measure the effect and report it
Style over substance	Confident formatting reads as correctness	Rubrics that force a claim-by-claim verdict instead of a global score
Score compression	Everything lands on 7 or 8 out of 10	Binary or three-point scales, or forced pairwise comparison

Interactive **Judge bias simulator** (shown at its default setting)

A judge with position bias, length bias and finite skill. Toggle order swapping and watch the win rate return toward 0.5 while agreement barely moves. The bias was inflating confidence, not accuracy.



Agreement 0.900 **kappa** 0.798 **"A wins" rate** 0.614

Order swapping OFF. Turn it on and watch the "A wins" rate move back toward 0.5 while agreement barely changes. The bias was inflating your confidence, not your accuracy.

Under the hood

Debiased pairwise judging

python

```python
async def judge_pairwise_debiased(judge_fn, task, resp_a, resp_b):
    """Run both orderings; treat disagreement as a bias signal, not a verdict."""
    fwd = await judge_fn(task, first=resp_a, second=resp_b)   # "first" | "second" | "tie"
    rev = await judge_fn(task, first=resp_b, second=resp_a)
    fwd_w = {"first": "a", "second": "b", "tie": "tie"}[fwd]
    rev_w = {"first": "b", "second": "a", "tie": "tie"}[rev]
    if fwd_w == rev_w:
        return {"winner": fwd_w, "consistent": True}
    return {"winner": "tie", "consistent": False}   # order flipped the verdict
```

rust

```rust
#[derive(Debug, Clone, Copy, PartialEq)]
enum Verdict {
    First,
    Second,
    Tie,
}

#[derive(Debug, Clone, Copy, PartialEq)]
enum Winner {
    A,
    B,
    Tie,
}

struct PairwiseResult {
    winner: Winner,
    consistent: bool,
}

/// Run both orderings; treat disagreement as a bias signal, not a verdict.
/// The Python awaits an async judge. The standard library has no async runtime,
/// so here judge_fn(task, first, second) is a blocking call.
fn judge_pairwise_debiased(
    mut judge_fn: impl FnMut(&str, &str, &str) -> Verdict,
    task: &str,
    resp_a: &str,
    resp_b: &str,
) -> PairwiseResult {
    let fwd = judge_fn(task, resp_a, resp_b); // First | Second | Tie
    let rev = judge_fn(task, resp_b, resp_a);
    let fwd_w = match fwd {
        Verdict::First => Winner::A,
        Verdict::Second => Winner::B,
        Verdict::Tie => Winner::Tie,
    };
    let rev_w = match rev {
        Verdict::First => Winner::B,
        Verdict::Second => Winner::A,
        Verdict::Tie => Winner::Tie,
    };
    if fwd_w == rev_w {
        return PairwiseResult { winner: fwd_w, consistent: true };
```

```
    }
    PairwiseResult { winner: Winner::Tie, consistent: false } // order flipped the verdict
}
```

The inconsistency rate across a batch is itself a useful number: it estimates how much of the judge's verdict is position rather than content. Track it per judge version.

Principal asks

Judges are often used to catch models gaming a task: editing the test instead of the implementation, hard-coding an expected output, or reading a solution left in the environment. Expect to be asked for a concrete instance, then for how you detect the *class* of exploit rather than the one instance, and what changed in the harness so it cannot recur.

Validating a judge before you trust it

1. Sample 150 to 300 items across the score range, not just the disputed ones.
2. Get two human labels per item and measure human-to-human agreement first. That is your ceiling. If humans agree at kappa 0.6, the judge cannot do meaningfully better.
3. Measure judge-to-human agreement on a held-out slice the judge prompt was not tuned on.
4. Check the pass-rate gap. A judge with good agreement but a higher pass rate biases every absolute number while still ranking models correctly.
5. Recheck after any change on the judge side. A judge is a model, so a judge upgrade is a silent measurement change.

When the dot will not move up and left

1. **Read your mistakes.** Pull 20 false positives and 20 false negatives. There is usually a pattern, like "fails anything that mentions a competitor." Fix the rubric for exactly those cases.
2. **Tighten the definition.** Fuzzy criteria are the most common cause of overlap. Borderline cases land randomly on either side.
3. **Split one judgment into several.** Separate yes/no checks (is it factually correct? does it follow the format? does it answer the question?) usually separate the classes better than one holistic verdict.

4. **Show hard examples.** Few-shot examples from the overlap zone, labelled with the reason, teach the judge where the line is. Easy examples do not help much.

5. **Reason first, verdict last.** Asking for reasoning through the criteria before the answer helps on borderline cases, and makes the verdict token easy to read logprobs from.

6. **Check the human ceiling.** If humans agree only 80% of the time, the judge will not beat that. Fix the labelling guidelines first.

A stronger judge model, or several judges voting (different prompts or model families), can also push the humps further apart.

For leaders

Judge governance: treat the judge as production infrastructure

Once a judge feeds dashboards and launch decisions, it is measurement infrastructure, and it needs the same discipline as any other production system.

Version and pin it

Pin the judge model, prompt, and decoding settings. Record them with every result. A judge upgrade changes every historical comparison, so re-baseline when it changes.

Validate on a cadence

Keep a fixed human-labelled set (a few hundred items) and re-score it on every judge change and on a schedule. Alert on kappa drops and pass-rate drift.

Know the ceiling

Report human-to-human kappa next to judge-to-human kappa. Chasing a judge past the human ceiling is chasing noise in the labels.

Control the biases

Both orderings for pairwise judges, length checks, a different model family from the model being judged. Publish the inconsistency rate.

Know what you need

Ranking two models needs consistency. Reporting an absolute quality number needs calibration to humans. Most judges are good enough for the first and not the second.

Budget it

Judge calls can rival the cost of the eval itself. Use a cheaper judge for broad coverage with a stronger judge or humans auditing a random sample, and track the audit disagreement.

Principal asks

- What is the judge's kappa against humans, and what is the human-to-human kappa on the same items?
- Is the judge more lenient than humans? By how much, and does that matter for this claim?
- Has the judge model or prompt changed since the baseline we are comparing against?
- Is the judge from the same model family as the system it is grading?

Eval lens

A judge-based pass rate has two layers of uncertainty: sampling error from the finite task set, and judge error relative to humans. The first shrinks with more tasks; the second does not. Report both, or at least state the judge's validated agreement next to any number it produced.

Part II · Chapter 12

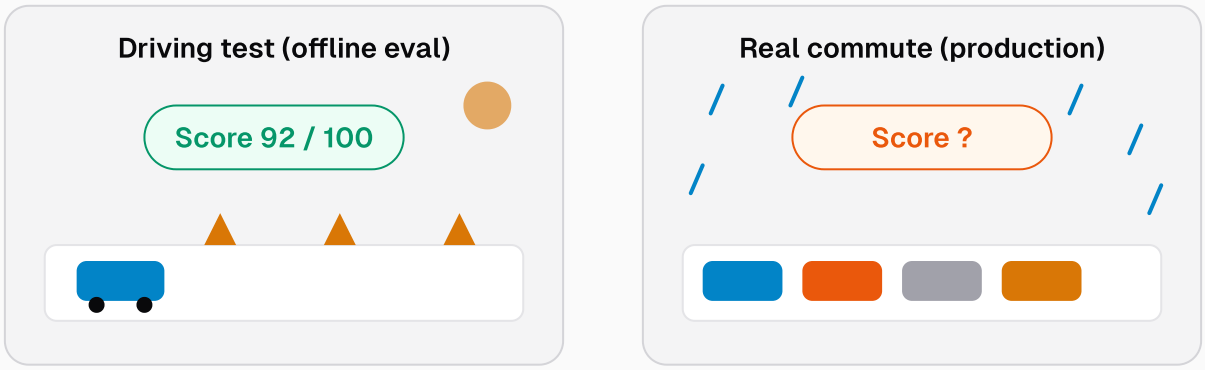
The offline to online gap: does the lab score predict the real world?

Every offline eval is a bet that a number computed on a fixed task set will move the same way as the thing you actually care about: user acceptance, retention, task success in production. This chapter is about testing that bet, putting a number on how wrong it is, and saying honestly how much of the gap you can close.

Diagram

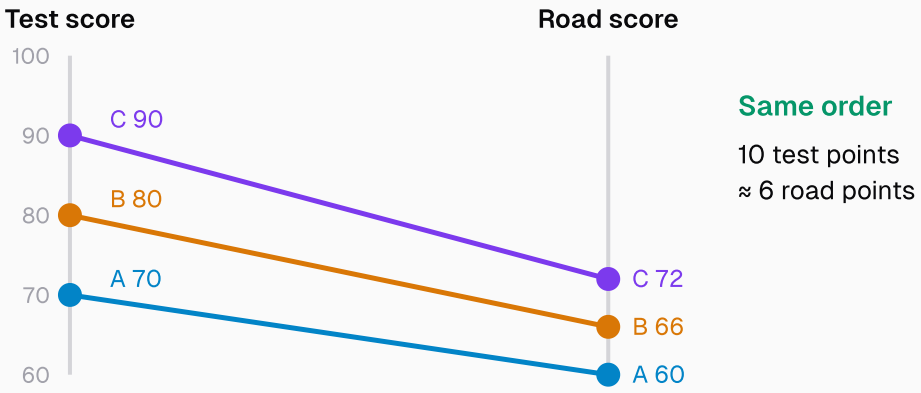
The driving test and the rush-hour commute

Why a better lab score does not always mean a better product, in five pictures.

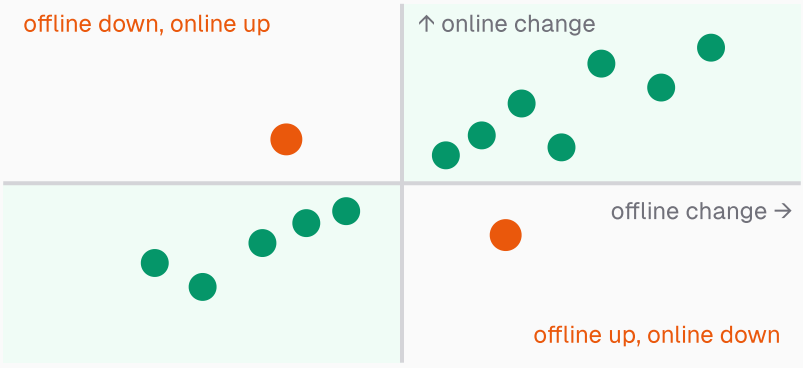


Does a better test score mean a better commute?

The test is quiet, sunny and predictable. The commute is crowded and wet. Your offline eval is the test; production is the commute. This chapter is about checking how well one predicts the other.

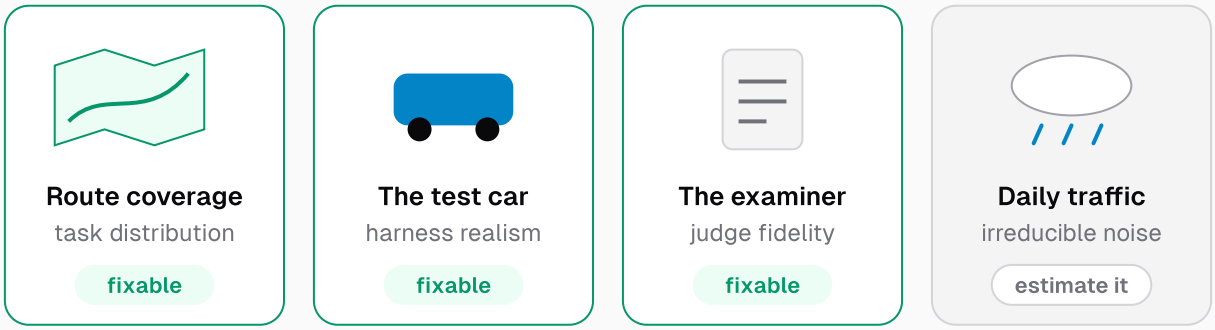


Two questions, kept apart. **Ordering:** do drivers keep the same rank on the road? Here yes, A < B < C. **Size:** each 10 points on the test is worth only about 6 on the road. Ordering matters most for a ship decision; size tells you how much to promise.



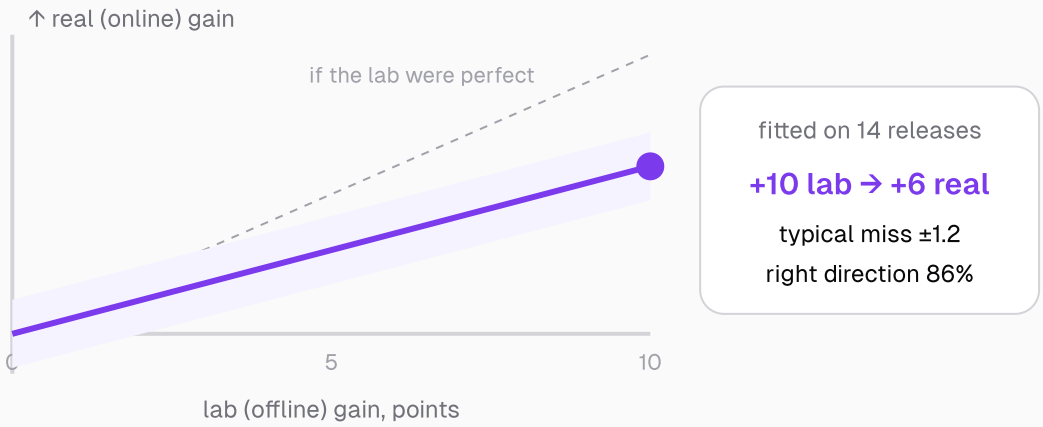
Same direction on 12 of 14 releases: direction accuracy 86%

Plot every past release: offline change across, online change up. Green zones mean the two agreed. Two releases landed in the wrong zone, so the offline suite pointed the wrong way 2 times in 14. That **direction accuracy** is the single most useful number about an eval suite.



Spend on the first three. Measure the fourth with an A/A test.

The gap has four sources. The test route skips some roads (the task set misses real usage). The test car is not your car (the harness lacks real tools and context). The examiner grades differently from real outcomes (judge fidelity). And traffic varies day to day, which no test can predict.



Put it together and the offline suite becomes a calibrated instrument: a lab gain of 10 points predicts about 6 online, give or take 1.2, and the direction is right 86% of the time. Recompute this after every launch; when direction accuracy falls, the suite has drifted from real use.

Plain words

The driving test and the commute

A driving test happens on a quiet route, in good weather, with an examiner watching. Passing it predicts how someone drives on a real commute, but not perfectly. Some careful test-takers panic in traffic. Some sloppy ones are fine in practice.

An offline eval is the driving test. Production is the commute. The questions in this chapter are simple to say: when a new model scores better on the test, does it also do better on the commute? By how much? And how often does the test point the wrong way entirely?

Two things to keep apart. First, **ordering**: does the test rank drivers in the same order as real life? Second, **size**: if someone improves by 10 points on the test, how many points is that on the road? Ordering is the more important and easier question. Size comes second.

For leaders

Why this is the most important number your eval team does not report

An offline suite that has never been checked against production is a story, not a measurement. Most teams report offline deltas as if they were product deltas. The difference shows up months later as a launch that won offline and did nothing online, or worse, a regression that offline scored as a win.

What to ask for, in order:

1. **A rank check.** Across the last 8 to 15 releases, does the offline suite order models the same way production does? Kendall's tau is the honest statistic at that sample size.

2. **A calibration line.** A fit of online delta against offline delta. The slope tells you how much offline overstates. A slope of 0.6 means a 10 point offline win is worth about 6 points in production.

3. **A backtest.** Leave-one-out prediction of each past release from all the others. Report the typical miss, the worst miss, and above all **direction accuracy**: how often offline and online disagreed on whether the release was better.

4. **A decomposition of the remaining gap** into what is fixable (task coverage, harness realism, judge fidelity) and what is not (user variance, traffic drift). This is what tells you where to spend.

What to fund. If direction accuracy is below about 85%, the offline suite cannot gate launches on its own and you need online experiments for every material change. That is expensive, so the higher-return investment is usually the fixable buckets: realistic harnesses and task sets drawn from real usage.

Principal asks

- How many releases is this calibration fitted on, and what happens to the error if you drop to half of them?
- What is the worst single miss, and what was different about that release?
- Has the product or traffic mix changed since the oldest release in the fit? If so, why should yesterday's slope hold?
- Of the remaining gap, how much do you believe is irreducible, and how did you estimate it?

Does offline predict online at all

Plain words

Same order, before same size

Line up past models by their offline score, then by their production score. If the two line-ups mostly agree, offline is useful for picking winners even if its numbers are inflated. If they do not agree, nothing else in this chapter rescues it.

Under the hood

Spearman and Kendall rank correlation

Spearman's ρ is the Pearson correlation of the ranks. Kendall's τ counts pairs of releases that the two measures order the same way (concordant, C) versus opposite ways (discordant, D):

$$\tau = \frac{C - D}{C + D}$$

Ties are excluded from both counts in this simple version (tau-a over untied pairs). $\tau = 1$ means offline and online order every pair the same way.

With 8 to 15 releases, which is realistic, τ is more stable than ρ and has a direct reading: $\tau = 0.6$ means 80% of release pairs are ordered correctly, because $(C - D)/(C + D) = 2 \cdot \text{share}_C - 1$.

python

```python
def spearman(x: list[float], y: list[float]) -> float:
    """Rank correlation. The right first question for offline vs online:
    does the offline suite ORDER models the way production does?
    Absolute calibration is a separate and easier problem."""
    def ranks(v):
        order = sorted(range(len(v)), key=lambda i: v[i])
        r = [0.0] * len(v)
        i = 0
        while i < len(order):
            j = i
            while j + 1 < len(order) and v[order[j + 1]] == v[order[i]]:
                j += 1
            avg = (i + j) / 2 + 1          # tied values share the average rank
            for k in range(i, j + 1):
                r[order[k]] = avg
            i = j + 1
        return r

    rx, ry = ranks(x), ranks(y)
    n = len(x)
    mx, my = sum(rx) / n, sum(ry) / n
    num = sum((a - mx) * (b - my) for a, b in zip(rx, ry))
    den = (sum((a - mx) ** 2 for a in rx) * sum((b - my) ** 2 for b in ry)) ** 0.5
    return num / den if den else 0.0


def kendall_tau(x: list[float], y: list[float]) -> float:
    """More robust than Spearman when you have few models. With 8 to 15 model
    releases, which is the realistic n, tau is the honest choice."""
```

```
    n = len(x)
    conc = disc = 0
    for i in range(n):
        for j in range(i + 1, n):
            sx = (x[i] - x[j])
            sy = (y[i] - y[j])
            if sx * sy > 0:
                conc += 1
            elif sx * sy < 0:
                disc += 1
    total = conc + disc
    return (conc - disc) / total if total else 0.0
```

rust

```
/// Rank correlation. The right first question for offline vs online:
/// does the offline suite ORDER models the way production does?
/// Absolute calibration is a separate and easier problem.
fn spearman(x: &[f64], y: &[f64]) -> f64 {
    fn ranks(v: &[f64]) -> Vec<f64> {
        let mut order: Vec<usize> = (0..v.len()).collect();
        order.sort_by(|&a, &b| v[a].partial_cmp(&v[b]).expect("NaN value"));
        let mut r = vec![0.0; v.len()];
        let mut i = 0;
        while i < order.len() {
            let mut j = i;
            while j + 1 < order.len() && v[order[j + 1]] == v[order[i]] {
                j += 1;
            }
            let avg = (i + j) as f64 / 2.0 + 1.0; // tied values share the average rank
            for k in i..=j {
                r[order[k]] = avg;
            }
            i = j + 1;
        }
        r
    }

    let (rx, ry) = (ranks(x), ranks(y));
    let n = x.len() as f64;
    let (mx, my) = (rx.iter().sum::<f64>() / n, ry.iter().sum::<f64>() / n);
    let num: f64 = rx.iter().zip(&ry).map(|(a, b)| (a - mx) * (b - my)).sum();
    let den = (rx.iter().map(|a| (a - mx).powi(2)).sum::<f64>()
        * ry.iter().map(|b| (b - my).powi(2)).sum::<f64>())
    .sqrt();
    if den != 0.0 { num / den } else { 0.0 }
}

/// More robust than Spearman when you have few models. With 8 to 15 model
/// releases, which is the realistic n, tau is the honest choice.
fn kendall_tau(x: &[f64], y: &[f64]) -> f64 {
    let n = x.len();
    let (mut conc, mut disc) = (0, 0);
    for i in 0..n {
        for j in i + 1..n {
            let sx = x[i] - x[j];
            let sy = y[i] - y[j];
            if sx * sy > 0.0 {
                conc += 1;
            } else if sx * sy < 0.0 {
```

```
                disc += 1;
            }
        }
    }
    let total = conc + disc;
    if total > 0 { (conc - disc) as f64 / total as f64 } else { 0.0 }
}
```

> **Watch out**
>
> With 10 releases there are only 45 pairs, and a 95% interval on τ is roughly ±0.45 wide. A τ of 0.5 on ten releases is consistent with anything from barely useful to excellent. Bootstrap over releases (see intervals and the bootstrap) before you quote it.

Calibrating the offline delta to the online delta

Plain words

An exchange rate between two currencies

Offline points and production points are different currencies. The calibration line is the exchange rate. If the rate is 0.6, a 10 point offline win buys about 6 points online. The scatter around the line tells you how much that exchange rate wobbles from release to release.

Under the hood

Least squares on release deltas

For each past release i, take the offline delta x_i against the previous model and the online delta y_i from the A/B test. Fit

$$y_i = a\,x_i + b + \varepsilon_i, \qquad \hat{a} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \qquad \hat{b} = \bar{y} - \hat{a}\bar{x}$$

$\hat{a} < 1$ means offline overstates improvements, which is the common case because offline tasks are cleaner and easier than production. Shrinkage $1 - \hat{a}$ is the gap as a single number. The residual spread is the real prediction interval.

python

```python
def fit_gap_model(offline_deltas: list[float], online_deltas: list[float]) -> dict:
    """Least squares fit of online_delta ~ a * offline_delta + b."""
    n = len(offline_deltas)
    mx = sum(offline_deltas) / n
    my = sum(online_deltas) / n

    # Standard least squares slope: covariance over variance of x.
    sxy = sum((x - mx) * (y - my) for x, y in zip(offline_deltas, online_deltas))
    sxx = sum((x - mx) ** 2 for x in offline_deltas)
    a = sxy / sxx if sxx else 0.0        # sxx == 0 when every release moved identically
    b = my - a * mx                      # line passes through the means

    resid = [y - (a * x + b) for x, y in zip(offline_deltas, online_deltas)]
    rmse = (sum(r * r for r in resid) / n) ** 0.5      # typical prediction miss
    ss_tot = sum((y - my) ** 2 for y in online_deltas)
    r2 = 1 - sum(r * r for r in resid) / ss_tot if ss_tot else 0.0

    return {"slope": a, "intercept": b, "rmse": rmse, "r2": r2,
            # slope < 1 means offline OVERSTATES improvements, the common case,
            # because the offline suite is cleaner than production.
            "shrinkage": 1 - a}
```

rust

```rust
#[derive(Debug, Clone, Copy)]
struct GapFit {
    slope: f64,
    intercept: f64,
    rmse: f64,
    r2: f64,
    // slope < 1 means offline OVERSTATES improvements, the common case,
    // because the offline suite is cleaner than production.
    shrinkage: f64,
}

/// Least squares fit of online_delta ~ a * offline_delta + b.
fn fit_gap_model(offline_deltas: &[f64], online_deltas: &[f64]) -> GapFit {
    let n = offline_deltas.len() as f64;
    let mx = offline_deltas.iter().sum::<f64>() / n;
    let my = online_deltas.iter().sum::<f64>() / n;

    // Standard least squares slope: covariance over variance of x.
    let sxy: f64 = offline_deltas.iter().zip(online_deltas).map(|(x, y)| (x - mx) * (y - my)).sum();
    let sxx: f64 = offline_deltas.iter().map(|x| (x - mx).powi(2)).sum();
    let a = if sxx != 0.0 { sxy / sxx } else { 0.0 }; // sxx == 0 when every release moved identically
    let b = my - a * mx; // line passes through the means

    let resid: Vec<f64> = offline_deltas.iter().zip(online_deltas).map(|(x, y)| y - (a * x + b)).collect();
    let ss_res: f64 = resid.iter().map(|r| r * r).sum();
    let rmse = (ss_res / n).sqrt(); // typical prediction miss
    let ss_tot: f64 = online_deltas.iter().map(|y| (y - my).powi(2)).sum();
    let r2 = if ss_tot != 0.0 { 1.0 - ss_res / ss_tot } else { 0.0 };

    GapFit { slope: a, intercept: b, rmse, r2, shrinkage: 1.0 - a }
}
```

> **Note**
>
> Ordinary least squares assumes the offline delta is measured without error. It is not: offline deltas carry their own sampling noise, which biases $\hat{a}$ toward zero (attenuation bias). With small eval sets this makes offline look worse at predicting online than it really is. Larger offline suites, or an errors-in-variables fit, reduce the effect.

Backtest: the part that makes it a method instead of a story

Plain words

Predict the past you have not looked at

Any line looks good on the points it was drawn through. The fair test is to hide one past release, draw the line through the rest, and see how well it predicts the hidden one. Repeat for every release. That is a backtest, and it is the only honest estimate of how well the next prediction will go.

For leaders

Which number gates a launch

The backtest produces three numbers. The **mean absolute error** is the headline gap. The **worst miss** is the one a skeptic will ask about. **Direction accuracy** is the one that matters for a ship decision: predicting +3 when the truth was +1 is a small error, but predicting +1 when the truth was –1 means you shipped a worse model. A suite with small MAE and 70% direction accuracy is not fit to gate launches.

Under the hood

Leave-one-out over releases

python

```python
def backtest_suite(history: list[dict]) -> dict:
    """history: one entry per past release with {"model", "offline", "online"}."""
    errors, sign_hits = [], 0

    for i in range(len(history)):
        # LEAVE ONE OUT: fit on every release EXCEPT this one, then predict it.
        # Fitting and scoring on the same releases always looks good and tells
        # you nothing about the next release, which is the one you care about.
        train = history[:i] + history[i + 1:]
        test = history[i]

        fit = fit_gap_model([h["offline"] for h in train],
```

```
                            [h["online"] for h in train])
        pred = fit["slope"] * test["offline"] + fit["intercept"]

        errors.append(abs(pred - test["online"]))

        # DIRECTION is the ship decision. Track it separately from error size
        # because the two failures are not the same.
        if (pred > 0) == (test["online"] > 0):
            sign_hits += 1

    return {
        "mae": sum(errors) / len(errors),           # your headline "gap" number
        "direction_accuracy": sign_hits / len(history),
        "worst_case": max(errors),                  # the one they will ask about
    }
```

rust

```
struct Release {
    model: String,
    offline: f64,
    online: f64,
}

struct Backtest {
    mae: f64,                // your headline "gap" number
    direction_accuracy: f64,
    worst_case: f64,         // the one they will ask about
}

// fit_gap_model from the snippet above, cut down to the two fields used here
// so that this snippet compiles on its own.
struct GapFit {
    slope: f64,
    intercept: f64,
}

fn fit_gap_model(offline_deltas: &[f64], online_deltas: &[f64]) -> GapFit {
    let n = offline_deltas.len() as f64;
    let mx = offline_deltas.iter().sum::<f64>() / n;
    let my = online_deltas.iter().sum::<f64>() / n;
    let sxy: f64 = offline_deltas.iter().zip(online_deltas).map(|(x, y)| (x - mx) * (y - my)).sum();
    let sxx: f64 = offline_deltas.iter().map(|x| (x - mx).powi(2)).sum();
    let a = if sxx != 0.0 { sxy / sxx } else { 0.0 };
    GapFit { slope: a, intercept: my - a * mx }
}

/// history: one entry per past release with model, offline and online deltas.
fn backtest_suite(history: &[Release]) -> Backtest {
    let (mut errors, mut sign_hits) = (Vec::new(), 0);

    for i in 0..history.len() {
        // LEAVE ONE OUT: fit on every release EXCEPT this one, then predict it.
        // Fitting and scoring on the same releases always looks good and tells
        // you nothing about the next release, which is the one you care about.
        let train: Vec<&Release> = history.iter().enumerate().filter(|&(k, _)| k != i).map(|(_, h)| h).collect();
        let test = &history[i];

        let offline: Vec<f64> = train.iter().map(|h| h.offline).collect();
        let online: Vec<f64> = train.iter().map(|h| h.online).collect();
```

```
        let fit = fit_gap_model(&offline, &online);
        let pred = fit.slope * test.offline + fit.intercept;

        errors.push((pred - test.online).abs());

        // DIRECTION is the ship decision. Track it separately from error size
        // because the two failures are not the same.
        if (pred > 0.0) == (test.online > 0.0) {
            sign_hits += 1;
        }
    }

    Backtest {
        mae: errors.iter().sum::<f64>() / errors.len() as f64,
        direction_accuracy: sign_hits as f64 / history.len() as f64,
        worst_case: errors.iter().copied().fold(f64::NEG_INFINITY, f64::max),
    }
}
```

Worked problem

Fourteen past releases, each with an offline delta and an online delta. How well does offline predict online?

python

```python
import numpy as np

def leave_one_out(off, on):
    """Fit on every release EXCEPT one, predict that one."""
    errs, sign_hits = [], 0
    for i in range(len(off)):
        xs, ys = np.delete(off, i), np.delete(on, i)
        a = ((xs - xs.mean()) * (ys - ys.mean())).sum() / ((xs - xs.mean()) ** 2).sum()
        b = ys.mean() - a * xs.mean()
        pred = a * off[i] + b
        errs.append(abs(pred - on[i]))
        if (pred > 0) == (on[i] > 0):          # did we get the DIRECTION right?
            sign_hits += 1
    return {"mae": np.mean(errs), "worst": max(errs),
            "direction_accuracy": sign_hits / len(off)}
```

rust

```rust
struct LooResult {
    mae: f64,
    worst: f64,
    direction_accuracy: f64,
}

fn mean(v: &[f64]) -> f64 {
    v.iter().sum::<f64>() / v.len() as f64
}

/// Fit on every release EXCEPT one, predict that one.
fn leave_one_out(off: &[f64], on: &[f64]) -> LooResult {
    let (mut errs, mut sign_hits) = (Vec::new(), 0);
    for i in 0..off.len() {
        // np.delete(arr, i): a copy without element i
        let xs: Vec<f64> = off.iter().enumerate().filter(|&(k, _)| k != i).map(|(_, &v)|
v).collect();
        let ys: Vec<f64> = on.iter().enumerate().filter(|&(k, _)| k != i).map(|(_, &v)| v).collect();
        let (mx, my) = (mean(&xs), mean(&ys));
        let a = xs.iter().zip(&ys).map(|(x, y)| (x - mx) * (y - my)).sum::<f64>()
            / xs.iter().map(|x| (x - mx).powi(2)).sum::<f64>();
        let b = my - a * mx;
        let pred = a * off[i] + b;
        errs.push((pred - on[i]).abs());
        if (pred > 0.0) == (on[i] > 0.0) { // did we get the DIRECTION right?
            sign_hits += 1;
        }
    }
    LooResult {
        mae: mean(&errs),
        worst: errs.iter().copied().fold(f64::NEG_INFINITY, f64::max),
        direction_accuracy: sign_hits as f64 / off.len() as f64,
```

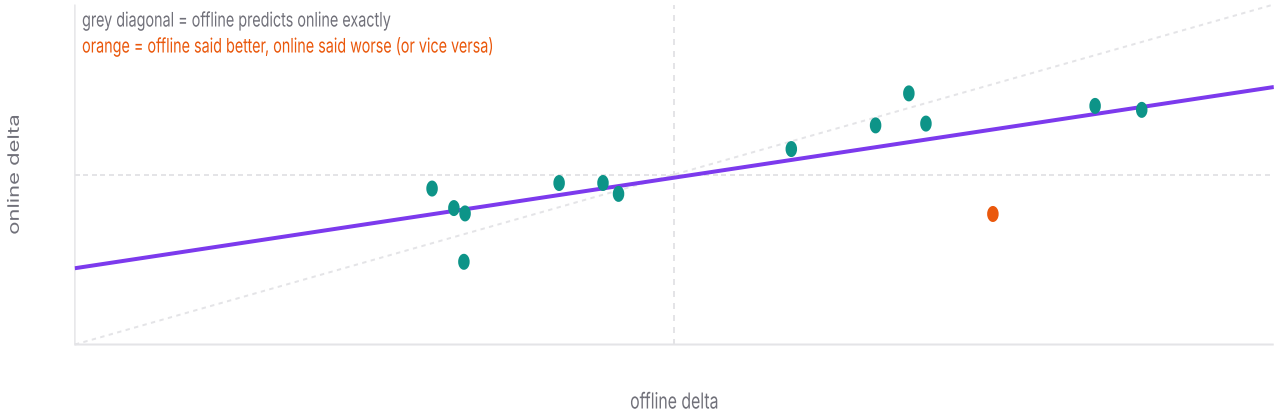
```
    }
}
```

Result	Value	What it means
Fitted slope	`0.616`	Offline overstates: a 10 point offline move is about 6 points online
Shrinkage	`0.384`	That 38% is the gap, expressed as one number
Leave-one-out MAE	`0.0122`	Typical prediction miss, 1.2 points
Worst single miss	`0.0352`	The one a reviewer will ask about
Direction accuracy	**`0.857`**	**Two of fourteen releases pointed the wrong way**

The figures are illustrative. Direction accuracy is the number that matters for a ship decision. MAE can look small while the suite still points the wrong way on the releases you cared about.

Interactive **Offline vs online, fitted and backtested** (shown at its default setting)

Each dot is a past release. The fitted line is your calibration. Orange dots are releases where offline and online disagreed on the direction. Drop the release count to 6 and watch the leave one out error jump, which is what overfitting a handful of releases looks like.



Fitted slope 0.533 **shrinkage** 0.467 **leave-one-out MAE** 0.0148
Direction accuracy 0.929 **worst single miss** 0.0585

Direction accuracy is the number that matters for a ship decision: MAE can look small while the suite still points the wrong way on the releases you cared about. Drop the release count to 6 and watch the leave-one-out error jump, which is what overfitting a handful of releases actually looks like.

⚠ **Watch out**

Three attacks on this method, and you should have answers ready for all three. **One:** the calibration is fitted on a handful of releases, so it is overfit to small n. **Two:** the online distribution moves when the model changes user behaviour, so yesterday's calibration may not hold (the population you are predicting is itself shifting). **Three:** MAE flatters the method; direction accuracy is what decides ship or no ship. Where the honest answer is "that is a real limitation", say so.

What is in the remaining gap

Plain words **Four reasons the test and the commute differ**

The test route does not cover every kind of road. The test car is not quite the car people drive. The examiner sometimes judges differently from what actually gets you home safely. And some days traffic is just different. The first three can be fixed with effort. The last one cannot.

For leaders

Decompose the gap before you fund work on it

When someone quotes "a 5% gap and closing", ask for the decomposition rather than the number:

Task distribution

Production work the task set does not yet represent: new languages, longer sessions, bigger inputs, new user intents. Fix: sample tasks from real usage, refresh regularly.

Harness realism

Missing context, absent tools, unrealistic starting state, no interruptions or retries. Fix: replay real environments, not idealised ones.

Judge fidelity

Where the automated judge and a real user disagree about whether the output was useful. Fix: validate the judge against user outcomes (see LLM as judge).

Irreducible

User behaviour variance, non-stationary traffic mix, acceptance that depends on the person as much as the output. Estimate it; do not promise to remove it.

Ending with "and this much is irreducible, here is how I estimated it" is a stronger position than claiming the gap can go to zero. It also points spending at the first two buckets, which is where dynamic, usage-derived benchmarks pay off.

Under the hood

Estimating the irreducible floor

One practical estimate: run the *same* model in two concurrent online arms (an A/A test). The spread of the measured online "delta" across repeated A/A runs is the noise floor no offline suite can predict

through. If the backtest residual RMSE is close to the A/A spread, the offline suite is already about as predictive as it can be, and further offline investment should target coverage, not calibration.

$$\underbrace{\mathrm{Var}(\text{residual})}_{\text{backtest}} \approx \underbrace{\mathrm{Var}(\text{offline noise})\,\hat{a}^2}_{\text{bigger eval set}} + \underbrace{\mathrm{Var}(\text{online noise})}_{\text{A/A floor}} + \underbrace{\mathrm{Var}(\text{mismatch})}_{\text{coverage, realism, judge}}$$

A rough variance budget, assuming the three sources are independent. Each term points at a different fix.

Eval lens

Treat the backtest as a standing metric of the eval program itself. Recompute it after every launch. A falling direction accuracy is the earliest warning that the offline suite has drifted away from what users do.

Part II · Chapter 13

Contamination and benchmark integrity: is the model reasoning or remembering?

A benchmark score means something only if the model has not seen the answers. Frontier models are trained on a large fraction of the public internet, and public benchmarks live on the public internet. This chapter covers how test items leak into training data, how to detect it, how to design benchmarks that resist it, and how to report results so a reader can judge for themselves.

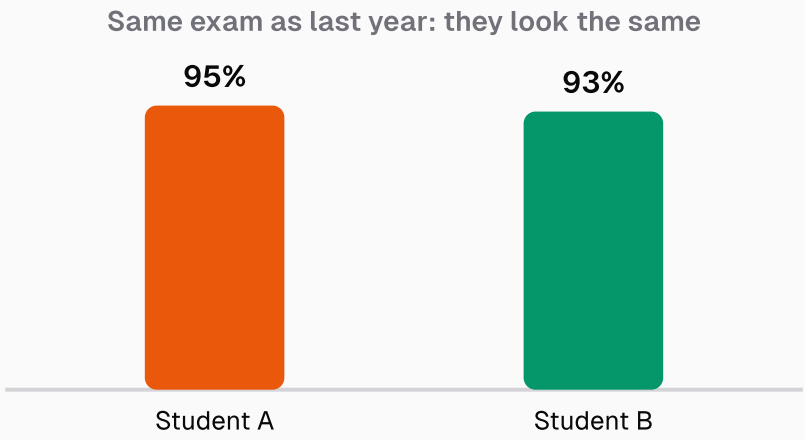
Diagram

Two students and a leaked answer key

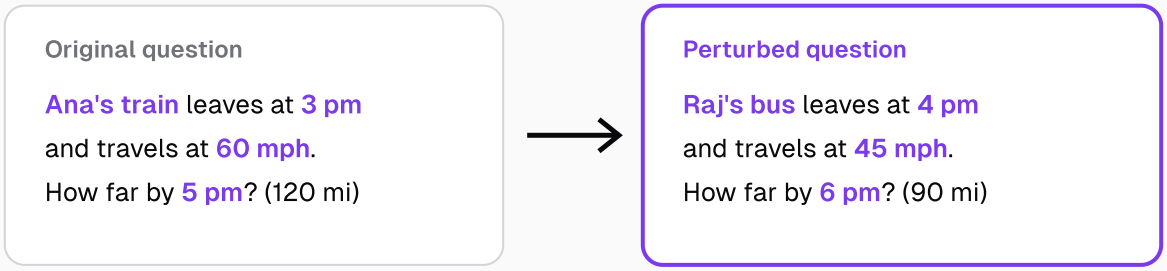
How changing the numbers on an exam separates memorising from understanding.

Benchmarks live on the internet, and so does model training data.

Last year's final exam was posted online with its answers. Student A memorises the answer key. Student B learns how to solve the problems. A model trained on internet text can end up like Student A for popular benchmarks, usually by accident.

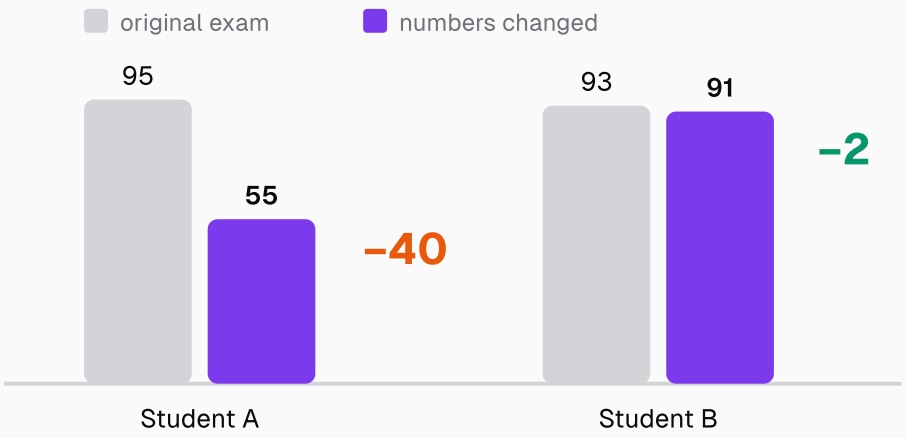


On the leaked exam, both score in the nineties. From the score alone you cannot tell who understands the material. A benchmark number on a public test set has exactly this problem.

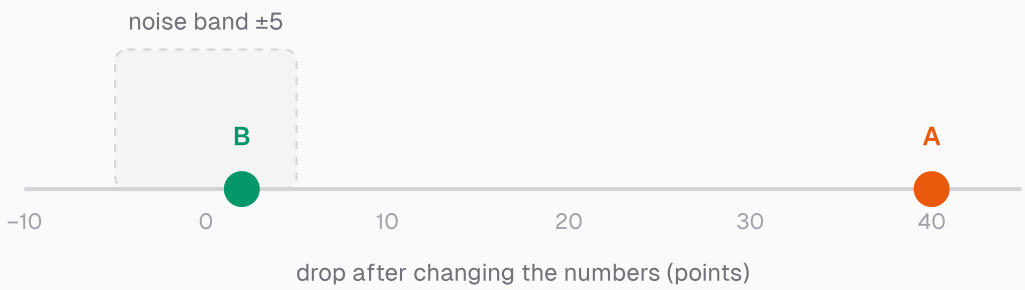


Same skill, new surface details. A memorised answer no longer fits.

Now change the surface of every question: new names, new numbers, options shuffled. The skill needed is identical, so an honest score should barely move. This is a **perturbation test**.

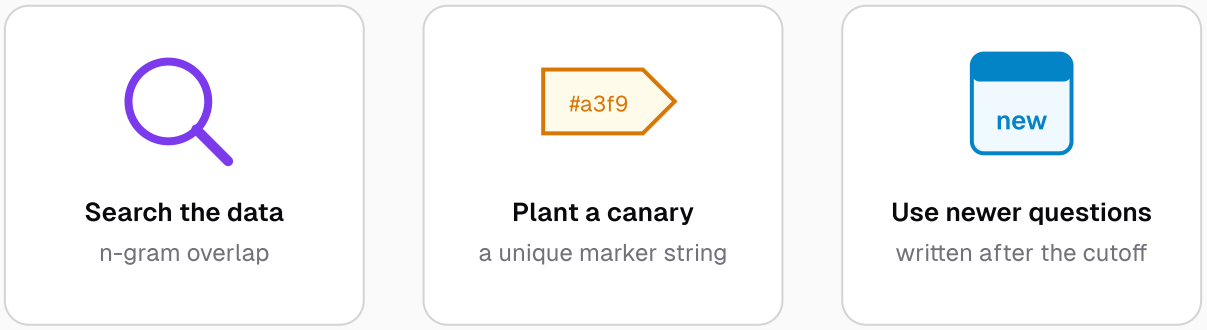


Student A falls from 95 to 55. Student B goes from 93 to 91. The size of the drop is the fingerprint: a big gap between original and perturbed versions says the original score was partly memory.



On 200 questions, a 2-point drop is noise. A 40-point drop is not.

Small drops happen by chance, because the perturbed questions are a fresh sample. On 200 questions the noise on a gap is roughly ±5 points, so B's 2-point drop means nothing and A's 40 is overwhelming. Always put an interval on the gap before calling something contaminated.



If you own the data, search it. If you do not, test the student.

Perturbation is one detective. If you own the training data, search it for the test items. Benchmark authors can plant a unique canary string so leaks are findable. And questions written after the model's data cutoff cannot have been memorised.

Plain words

The exam that was posted online

A teacher gives the same final exam every year. One year a student posts the questions and answers online. Next year's class scores much higher. Did they get smarter, or did they read the answers?

The test for this is simple: change the exam a little. Rename the people in the word problems, swap the numbers, reorder the options. A student who understands the material does about as well as before. A student who memorised the answer key falls apart.

Language models are trained on huge piles of internet text, so some of them have "read the answer key" for popular benchmarks, often by accident. This is called **contamination**.

For leaders

Why contamination is an organisational risk, not a footnote

Contamination inflates exactly the numbers that go into launch announcements, investor decks, and model comparison tables. It also corrupts internal decisions: a training change that "improves" a contaminated benchmark may just be memorising better. The damage is reputational when outsiders find it, and strategic when it misdirects research.

Three postures, in increasing cost and increasing credibility:

Posture	What it involves	What it buys
Detect	Overlap scans of training data against benchmarks; perturbation tests on suspicious results	An estimate of how inflated a public number is
Prevent	Decontaminate training corpora; canary strings; access-controlled private sets	Cleaner public numbers and trustworthy internal ones
Outrun	Time-split and dynamically generated benchmarks refreshed faster than training corpora	Numbers that cannot be memorised because they did not exist at training time

What to fund. At minimum, a decontamination pass in the data pipeline and a private held-out set for every capability you gate launches on. For high-stakes capabilities, a continuously refreshed benchmark built from material created after the training cutoff.

Principal asks

- Was this benchmark decontaminated against the training corpus, at what n-gram length, and how many items were removed?
- What does the model score on a perturbed or post-cutoff version of the same task?
- Is the headline number on the public set or on a private held-out set? Where do they disagree?
- How long until this benchmark is saturated, and what replaces it?

How items leak

Direct inclusion

The benchmark file, or a web page reproducing it, is scraped into pretraining data.

Paraphrase and discussion

Blog posts, forum threads, and solution write-ups restate the items in different words. Exact-match filters miss these.

Fine-tuning data

Synthetic or human-written SFT data generated by people or models that had seen the benchmark.

Selection pressure

Even with no leaked text, choosing checkpoints, prompts, or hyperparameters by benchmark score fits the model to the test set. This is overfitting through decisions, not data.

Detection

Plain words

Search the library, then test the student

There are two ways to check. If you own the training data, search it for the exam questions. If you do not, test the student: reword the questions and see whether the score holds up.

Under the hood

Overlap, canaries, membership signals, perturbation

N-gram overlap

The standard data-side check. Tokenise each benchmark item and each training document, and flag items that share long n-grams with the corpus. Thirteen-gram overlap was used for decontamination in the GPT-3 paper (Brown et al., 2020); other labs use 8 to 50 tokens or a fraction-of-item threshold. Short n-grams flag common phrases; long ones miss paraphrases.

python

```python
import hashlib, re

def ngrams(text: str, n: int = 13) -> set[str]:
    toks = re.findall(r"\w+", text.lower())
    return {" ".join(toks[i:i + n]) for i in range(len(toks) - n + 1)}


def contamination_overlap(task_text: str, model_output: str, n: int = 13) -> float:
    """Fraction of the task's n-grams reproduced verbatim in the output.
    High overlap on a task the model should have had to reason about is a
    memorization signal, not proof, but it tells you where to look."""
    t, o = ngrams(task_text, n), ngrams(model_output, n)
    return len(t & o) / len(t) if t else 0.0
```

rust

```rust
use std::collections::HashSet;

fn ngrams(text: &str, n: usize) -> HashSet<String> {
    // re.findall(r"\w+", text.lower()): runs of letters, digits and underscores
    let lower = text.to_lowercase();
    let toks: Vec<&str> = lower
        .split(|c: char| !(c.is_alphanumeric() || c == '_'))
        .filter(|t| !t.is_empty())
        .collect();
    // windows(n) yields nothing when there are fewer than n tokens
    toks.windows(n).map(|w| w.join(" ")).collect()
}

/// Fraction of the task's n-grams reproduced verbatim in the output.
/// High overlap on a task the model should have had to reason about is a
/// memorization signal, not proof, but it tells you where to look.
```

```rust
/// The Python default is n = 13.
fn contamination_overlap(task_text: &str, model_output: &str, n: usize) -> f64 {
    let (t, o) = (ngrams(task_text, n), ngrams(model_output, n));
    if t.is_empty() {
        return 0.0;
    }
    t.intersection(&o).count() as f64 / t.len() as f64
}
```

At corpus scale the same idea runs over hashed n-grams in a Bloom filter or suffix array, so trillions of training tokens can be checked against every benchmark in one pass.

Embedding similarity

To catch paraphrases, embed benchmark items and training chunks and flag nearest neighbours above a similarity threshold, then have a person or a strong model confirm. This finds rewritten solutions and translated copies that n-grams miss, at the cost of false positives on genuinely similar but independent text.

Canary strings

Embed a unique random token (a GUID) in every file of a private or public eval set, with a request that it never be trained on. Two uses: filter it out of your own corpora, and probe released models. If a model completes the canary when given its prefix, the set is in that model's training data. BIG-bench popularised this practice.

Membership-inference signals (outsider tests)

Without training data access, you can still look for signs a specific text was trained on:

- **Verbatim completion.** Give the first half of an item; check whether the model reproduces the second half, including its exact numbers or answer options.
- **Likelihood gap.** Seen text tends to get unusually low perplexity compared with paraphrases of the same content. The Min-K% Prob method (Shi et al., 2023) averages the lowest-probability tokens, which separates seen from unseen text better than average perplexity.
- **Ordering artefacts.** If a model assigns higher likelihood to a benchmark's examples in their published order than in shuffled orders, it has probably seen the file. Since the examples are exchangeable, a clean model has no reason to prefer one order (Oren et al., 2023).

These are evidence, not proof. Famous text is low-perplexity whether or not it was in a benchmark.

Perturbation tests

The strongest practical test available to an outsider. Rename variables, change constants, reorder independent clauses, shuffle options, translate and back-translate. A model that reasons keeps roughly the same score. A model that memorised falls off a cliff. GSM-Symbolic (Mirzadeh et al., 2024) applied this to grade-school maths and found measurable drops for many models when only names and numbers changed.

python

```python
def perturbation_check(run_fn, task, perturb_fn, trials: int = 5) -> dict:
    """Rename variables, change constants, reorder independent clauses.
    A model that reasons stays at roughly the same score.
    A model that memorized falls off a cliff."""
    base = run_fn(task)
    perturbed = [run_fn(perturb_fn(task, seed=i)) for i in range(trials)]
    mean_p = sum(perturbed) / trials
    return {"base": base, "perturbed_mean": mean_p, "drop": base - mean_p}
```

rust

```rust
struct PerturbationResult {
    base: f64,
    perturbed_mean: f64,
    drop: f64,
}

/// Rename variables, change constants, reorder independent clauses.
/// A model that reasons stays at roughly the same score.
/// A model that memorized falls off a cliff.
/// The Python default is trials = 5.
fn perturbation_check(
    mut run_fn: impl FnMut(&str) -> f64,
    task: &str,
    perturb_fn: impl Fn(&str, u64) -> String, // (task, seed) -> perturbed task
    trials: u64,
) -> PerturbationResult {
    let base = run_fn(task);
    let perturbed: Vec<f64> = (0..trials).map(|i| run_fn(&perturb_fn(task, i))).collect();
    let mean_p = perturbed.iter().sum::<f64>() / trials as f64;
    PerturbationResult { base, perturbed_mean: mean_p, drop: base - mean_p }
}
```

The drop alone is not the contamination signal, because perturbations can also make items genuinely harder. The clean comparison is a **difference in differences**: the suspect model's drop minus the drop of a control model that could not have seen the items (or a model trained before the benchmark existed).

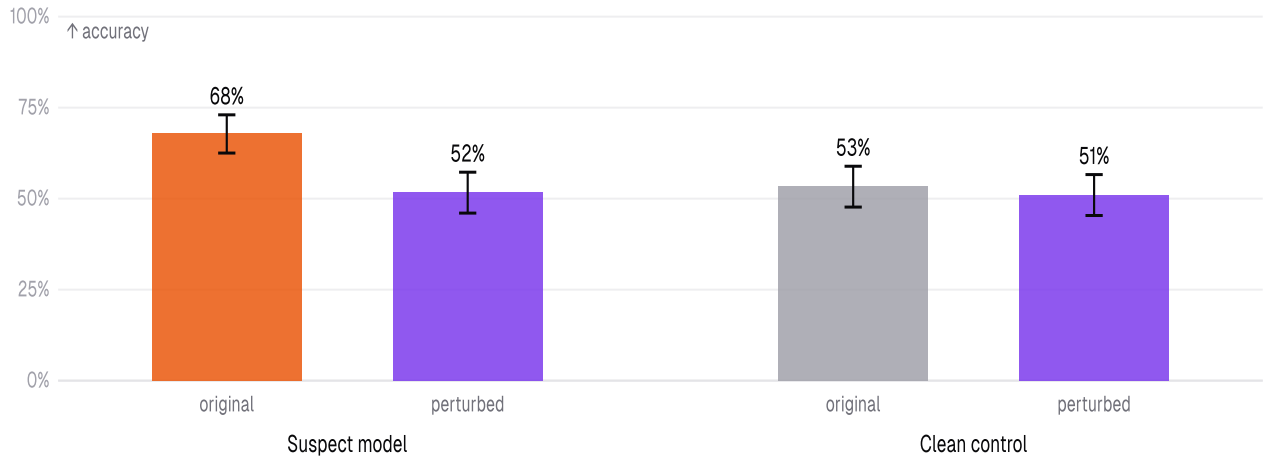
$$\widehat{\Delta}_{\text{contam}} = \left(\bar{a}^{S}_{\text{orig}} - \bar{a}^{S}_{\text{pert}}\right) - \left(\bar{a}^{C}_{\text{orig}} - \bar{a}^{C}_{\text{pert}}\right), \qquad \text{SE} = \frac{s_d}{\sqrt{n}}$$

S is the suspect model, C the clean control, and $\bar{a}$ is accuracy on original or perturbed items. Compute the per-item difference d_i on the same n items so the comparison is paired; s_d is its standard deviation. This is the same pairing logic as in eval statistics.

Interactive

Contamination shows up as a gap (shown at its default setting)

A suspect model and a clean control of equal ability answer the same items, once in original wording and once perturbed. Raise the share of items the suspect saw in training and watch its original score climb while the perturbed score stays put. Then shrink the item count and see when the signal disappears into noise.



Suspect model drop on rewording: **16.3 pts** (±4.9).
Clean control drop (perturbation difficulty alone): **2.3 pts** (±4.0).
Difference in differences: **14.0 pts**, 95% CI [7.7, 20.3].
Contamination signal. The suspect model loses more on rewording than the clean control does, and the interval excludes zero.
Estimated memorisation share of the headline score: 14.0 of 68.0 points.

Key idea

Set seen share to 0 and extra difficulty to 8 points. Both models drop by about the same amount, so the difference in differences is near zero. A raw drop on perturbed items is not evidence of contamination by itself. The control model is what turns a drop into a measurement.

Designing benchmarks that resist contamination

Plain words

Write new exams faster than they can leak

The surest fix is an exam made after the student finished studying. If the questions did not exist when the model was trained, it cannot have memorised them. The other fix is to keep some exams locked in a drawer and never post them.

For leaders

The design options and their costs

Design	How it works	Cost and catch
Private held-out set	Never published; run only by the owner or under access control	Cheap. Results cannot be independently reproduced, and every evaluation run leaks a little through selection pressure.
Time-split benchmark	Only items created after each model's training cutoff count (LiveCodeBench, Jain et al., 2024, does this for coding problems)	Needs a steady supply of fresh items. Scores across models with different cutoffs are on different item sets.
Dynamic generation	Templates or programs generate new instances on demand, with known answers	Expensive to build well. Risk of generating a narrow, artificial distribution that models can learn to game.
Usage-derived tasks	Tasks sampled from real product usage after the cutoff	Most realistic; also helps close the offline to online gap. Needs privacy review and labelling.
Canaried public set	Public, with a canary GUID and a request not to train	Relies on others filtering. Good for detection, weak for prevention.

Saturation

Every public benchmark degrades as a signal over time, through contamination, through selection pressure, and through simple saturation as top models approach the ceiling. Near the ceiling, remaining errors are often label noise, so further gains mean nothing. Plan the replacement before the current benchmark hits about 90% for frontier models. Assume a public benchmark's useful life is measured in model generations, not years.

Under the hood

Why saturation destroys resolving power

Near the ceiling, two things happen at once. The headroom to show improvement shrinks, and a growing share of the remaining "errors" are items with wrong or ambiguous gold labels. If a fraction e of items have bad labels, the maximum measurable accuracy is about $1 - e$, and differences between strong models are compressed into the last few points where label noise dominates. This is why audits of popular benchmarks that find a few percent of mislabelled items matter so much for frontier comparisons.

The number of tasks needed to separate two models also grows as they converge, because fewer tasks are discordant (see sample size). A saturated benchmark is both noisier and less informative per task.

Reporting norms

For leaders

What a credible benchmark claim includes

1. **Decontamination method**: overlap threshold, n-gram length, how many items were flagged and removed, and the score on the clean subset next to the full set.
2. **A contamination-resistant companion number**: private, perturbed, or post-cutoff results alongside the public one.
3. **Interval and n** on every score, as in every other chapter of Part II.
4. **Evaluation settings**: prompt, few-shot examples, sampling parameters, number of attempts. Small prompt changes move benchmark scores by several points.
5. **Selection disclosure**: whether the checkpoint or prompt was chosen using this benchmark.

Eval lens

When a public benchmark and a private or post-cutoff equivalent disagree, believe the one the model could not have seen. A consistent gap between the two, tracked across releases, is itself a useful metric of how much of a model's public performance is memory.

Part II · Chapter 14

Building the eval harness: the machine that produces every number

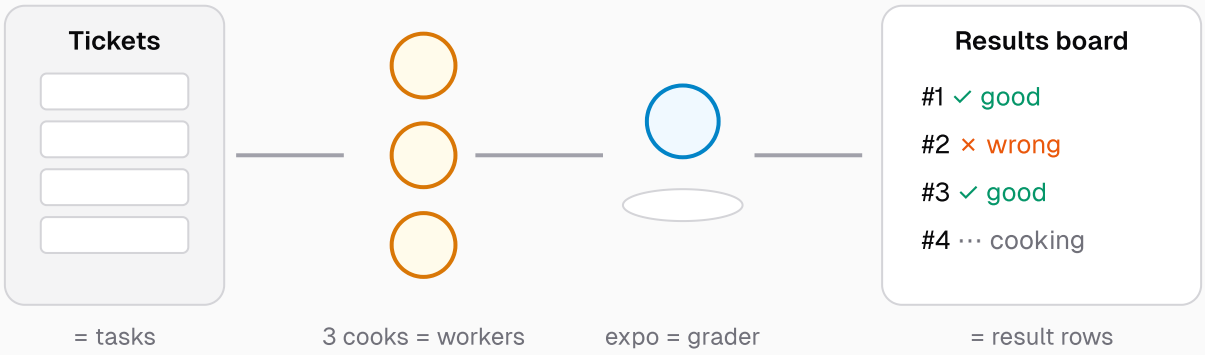
Every statistic in Part II is computed from results that a harness produced. If the harness drops tasks, retries silently, caches stale outputs, or reports a bare pass rate, the statistics on top are wrong no matter how

careful they are. This chapter builds a harness in four passes, each with an acceptance test, and shows where measurement errors hide in the engineering.

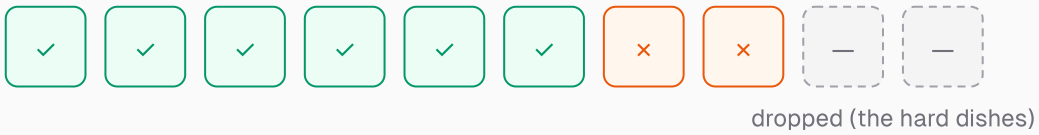
Diagram

An eval harness is a restaurant kitchen

Tickets, cooks, dropped plates and the redo trap: where harness bugs quietly change the score.



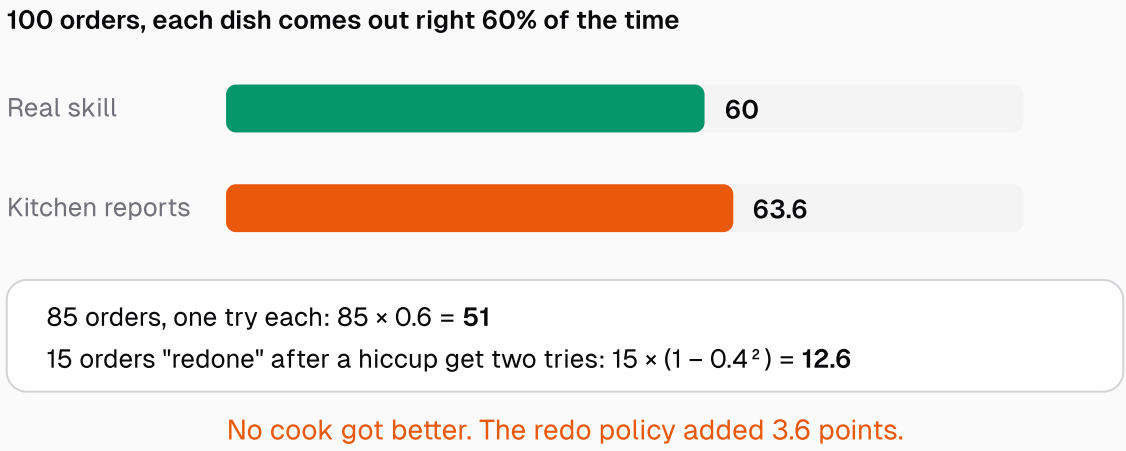
Every ticket (task) goes to a free cook (a worker), the expo checks the plate (the grader), and the result goes on the board. More cooks means faster service, the same way more concurrency finishes an eval sooner.



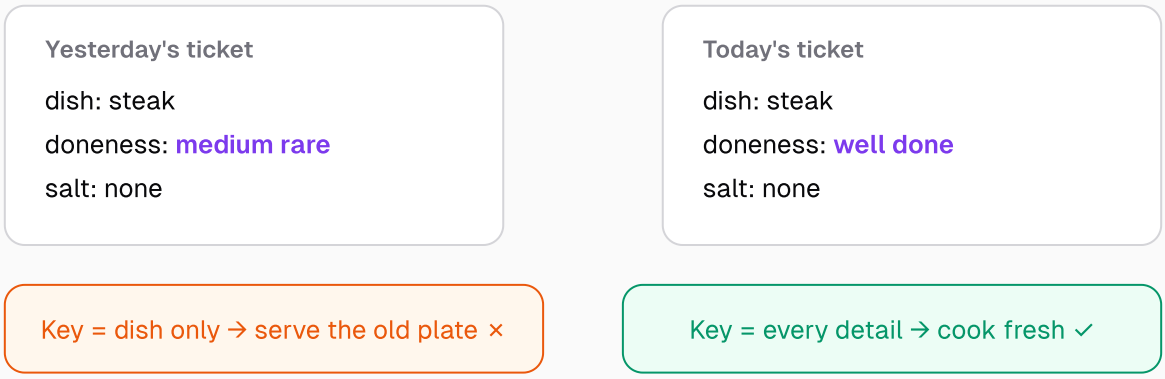
Hide the dropped plates	Count every ticket
6 / 8 = 75%	6 / 10 = 60%

A crashed task is data. It stays on the board.

Two plates hit the floor. If the board quietly skips them, the kitchen looks like it got 75% right. Dropped plates are usually the hard dishes, so skipping them always flatters the score. Count every ticket and the honest number is 60%.



Fifteen orders hit a hiccup and get remade. A remade dish gets a second chance to come out right, so those tickets are scored more like "best of two". The kitchen reports 63.6% when its real rate is 60%. This is retry inflation, and you can only detect it if every result records how many attempts it took.



Caching saves money: if an identical order comes in, reuse the plate. But "identical" must include every detail. In a harness the cache key covers the task, the prompt, the model, and every sampling setting. Leave one out and a changed setting silently serves yesterday's answers.

End-of-night report

Orders	1,000
Good	63.6% (60.6–66.5)
Dropped	0
Redone	150
Good, redone removed	60.0% (56.7–63.2)

The two "good" lines disagree: that gap is the redo effect.

The fix is a report that cannot be printed without the count, an interval, the dropped plates, and the redo count, plus the score with redone orders removed. When those two scores disagree, the retry policy is part of what you measured.

Plain words

A fair exam room

Think of the harness as the exam room. It hands each question to the model, collects the answer, marks it, and writes down the result. A good exam room treats every student the same: same time limit, same number of attempts, same marking rules. If one student's pen breaks, you note it and carry on; you do not cancel the exam.

Most harness bugs are unfairness bugs. One question gets two attempts while another gets one. A crashed question disappears from the results instead of counting. Yesterday's answers get reused after the questions changed. Each of these quietly changes the score.

For leaders

Treat the harness as measurement infrastructure

Teams under-invest in harnesses because they look like plumbing. They are the measuring instrument. A harness bug that inflates scores by two points is as damaging as a training bug, and much harder to notice, because nobody complains when numbers go up.

What a mature harness guarantees:

No silent loss

Every task produces a result row, including errors.

Equal treatment

Attempts, timeouts, and sampling parameters are identical across tasks and models, and recorded.

Reproducibility

A result can be traced to model id, prompt hash, parameters, harness version, and judge version.

Honest reporting

The summary cannot be produced without n, a confidence interval, the error count, and the retry count.

Flakiness control

Tasks whose scores vary run to run at fixed settings are quarantined, not averaged in.

What to fund: a small, dedicated owner for the harness, versioned like a product, with its own regression tests. The return is every downstream decision.

Principal asks

- How many tasks errored in this run, and are they excluded from the denominator?
- How many tasks were retried, and what is the score with retried tasks removed?
- Did anything other than the model change between these two runs: harness, judge, prompt template, timeouts, task set?
- Which tasks are quarantined as flaky, and how was flakiness measured?

Version one: make it work

Plain words

One question at a time, and never stop the exam

The first version is simple: loop over the tasks, ask the model, mark the answer. The one rule that matters is that a failed task is written down as a failure and the loop keeps going.

Under the hood

A sequential runner that records errors as data

python

```python
from dataclasses import dataclass, field
from typing import Callable, Any
import time, json

@dataclass
class Task:
    task_id: str
    prompt: str
    reference: dict[str, Any] = field(default_factory=dict)

@dataclass
class Result:
    task_id: str
    output: str | None = None
    score: float | None = None
    error: str | None = None
    attempts: int = 0
    latency_s: float = 0.0
    meta: dict[str, Any] = field(default_factory=dict)


def run_sync(tasks: list[Task], model_fn: Callable[[str], str],
             score_fn: Callable[[Task, str], float]) -> list[Result]:
    results = []
    for t in tasks:
        started = time.perf_counter()
        try:
            out = model_fn(t.prompt)
            results.append(Result(t.task_id, out, score_fn(t, out),
                                  attempts=1,
                                  latency_s=time.perf_counter() - started))
        except Exception as e:                # a failed task is DATA, not a crash
            results.append(Result(t.task_id, None, None, error=repr(e),
                                  attempts=1,
                                  latency_s=time.perf_counter() - started))
    return results
```

rust

```rust
use std::collections::HashMap;
use std::time::Instant;

#[derive(Debug, Clone, Default)]
pub struct Task {
    pub task_id: String,
    pub prompt: String,
    pub reference: HashMap<String, String>, // Python's dict[str, Any]; strings keep the sketch simple
}

// `Result` in the Python; renamed so it does not shadow std's Result type.
#[derive(Debug, Clone, Default)]
pub struct EvalResult {
    pub task_id: String,
    pub output: Option<String>,
```

```
    pub score: Option<f64>,
    pub error: Option<String>,
    pub attempts: u32,
    pub latency_s: f64,
    pub meta: HashMap<String, String>,
}

pub fn run_sync(
    tasks: &[Task],
    model_fn: impl Fn(&str) -> Result<String, String>,
    score_fn: impl Fn(&Task, &str) -> f64,
) -> Vec<EvalResult> {
    let mut results = Vec::new();
    for t in tasks {
        let started = Instant::now();
        match model_fn(&t.prompt) {
            Ok(out) => results.push(EvalResult {
                task_id: t.task_id.clone(),
                score: Some(score_fn(t, &out)),
                output: Some(out),
                attempts: 1,
                latency_s: started.elapsed().as_secs_f64(),
                ..Default::default()
            }),
            Err(e) => results.push(EvalResult {
                // a failed task is DATA, not a crash
                task_id: t.task_id.clone(),
                error: Some(e),
                attempts: 1,
                latency_s: started.elapsed().as_secs_f64(),
                ..Default::default()
            }),
        }
    }
    results
}
```

Worked problem

Acceptance test. Feed the runner ten tasks where the third raises an exception. The run must complete, return 10 results, and record the error on task three.

The most common first-attempt failure is letting the exception escape the loop, which kills the run and loses the other nine results. Catch per task, record the error, keep going.

Version two: concurrency, retries, timeouts

Plain words

Many questions at once, and a fair retry rule

Real suites have thousands of tasks, so the harness asks many at once. Model servers sometimes time out or say "too busy", so the harness waits and tries again. The danger is that retrying can give some tasks more chances than others. A fair harness records how many tries each task took, so you can check.

Under the hood

asyncio, a semaphore, and exponential backoff with full jitter

The backoff before attempt k is drawn uniformly from $[0, \min(\text{cap}, \text{base} \cdot 2^k)]$. The exponential envelope spaces retries out; the jitter stops a burst of failures from retrying in lockstep and knocking the endpoint over again.

python

```python
import asyncio, random, time

class RetryPolicy:
    def __init__(self, max_attempts: int = 4, base: float = 0.5, cap: float = 20.0):
        self.max_attempts, self.base, self.cap = max_attempts, base, cap

    def backoff(self, attempt: int) -> float:
        """Exponential with FULL JITTER."""
        # base * 2**attempt is the exponential envelope; min() caps it so a late
        # attempt does not sleep for minutes. uniform(0, ...) is the jitter:
        # without it, every request from a failure burst retries at the same
        # instant and re-overloads the endpoint you just knocked over.
        return random.uniform(0, min(self.cap, self.base * 2 ** attempt))

    def retryable(self, exc: Exception) -> bool:
        """Only retry TRANSIENT failures."""
        # Retrying a genuine 400 or an assertion error just burns quota and
        # delays the real answer. Match on the message rather than the type
        # because SDKs wrap errors differently.
        msg = repr(exc).lower()
        return any(s in msg for s in
                   ("timeout", "rate", "429", "500", "502", "503", "connection"))


async def run_one(task: Task, model_fn, score_fn, policy: RetryPolicy,
                  timeout_s: float, sem: asyncio.Semaphore) -> Result:
    started = time.perf_counter()      # wall clock for THIS task, retries included
    last_exc: Exception | None = None

    for attempt in range(policy.max_attempts):
        try:
            # The semaphore caps how many requests are in flight across the
            # whole suite. Acquire it INSIDE the loop so a sleeping retry
            # releases its slot instead of blocking other tasks.
```

```
            async with sem:
                out = await asyncio.wait_for(model_fn(task.prompt),
                                             timeout=timeout_s)
            return Result(task.task_id, out, score_fn(task, out),
                          attempts=attempt + 1,          # RECORD attempts
                          latency_s=time.perf_counter() - started)

        except Exception as e:
            last_exc = e
            # Two ways to stop: the error is not worth retrying, or we are out
            # of attempts. Checking both here keeps the exit conditions in one place.
            if not policy.retryable(e) or attempt == policy.max_attempts - 1:
                break
            await asyncio.sleep(policy.backoff(attempt))

    # A task that never succeeded is still DATA. Return a Result carrying the
    # error rather than raising, so one bad task cannot kill a 2000 task run.
    return Result(task.task_id, None, None, error=repr(last_exc),
                  attempts=attempt + 1,
                  latency_s=time.perf_counter() - started)


async def run_suite(tasks: list[Task], model_fn, score_fn,
                    concurrency: int = 16, timeout_s: float = 120.0) -> list[Result]:
    sem = asyncio.Semaphore(concurrency)   # one shared limiter for every task
    policy = RetryPolicy()
    coros = [run_one(t, model_fn, score_fn, policy, timeout_s, sem) for t in tasks]
    # gather runs them all concurrently, bounded by the semaphore, and returns
    # results in the ORIGINAL task order regardless of completion order.
    return await asyncio.gather(*coros)
```

rust

```
// asyncio becomes OS threads in this std-only sketch: a counting gate stands in
// for asyncio.Semaphore and a channel with recv_timeout for asyncio.wait_for.
// Production Rust would usually use tokio (tokio::sync::Semaphore,
// tokio::time::timeout, join_all) and get cheap tasks and real cancellation.
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::mpsc::{self, RecvTimeoutError};
use std::sync::{Arc, Condvar, Mutex};
use std::thread;
use std::time::{Duration, Instant};

#[derive(Debug, Clone, Default)]
pub struct Task {
    pub task_id: String,
    pub prompt: String,
}

#[derive(Debug, Clone, Default)]
pub struct EvalResult {
    // `Result` in the Python, trimmed to the fields used here
    pub task_id: String,
    pub output: Option<String>,
    pub score: Option<f64>,
    pub error: Option<String>,
    pub attempts: u32,
    pub latency_s: f64,
}
```

```rust
pub type ModelFn = Arc<dyn Fn(&str) -> Result<String, String> + Send + Sync>;
pub type ScoreFn = dyn Fn(&Task, &str) -> f64 + Sync;

/// Small inline PRNG (SplitMix64) so the sketch needs no crates.
pub struct SplitMix64(u64);

impl SplitMix64 {
    pub fn new(seed: u64) -> Self {
        SplitMix64(seed)
    }
    pub fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    pub fn uniform(&mut self, lo: f64, hi: f64) -> f64 {
        let u = (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64; // in [0, 1)
        lo + (hi - lo) * u
    }
}

/// The asyncio.Semaphore stand-in: a counting gate built from Mutex + Condvar.
pub struct Semaphore {
    permits: Mutex<usize>,
    freed: Condvar,
}

pub struct Permit<'a>(&'a Semaphore);

impl Semaphore {
    pub fn new(n: usize) -> Self {
        Semaphore { permits: Mutex::new(n), freed: Condvar::new() }
    }
    pub fn acquire(&self) -> Permit<'_> {
        let mut free = self.permits.lock().unwrap();
        while *free == 0 {
            free = self.freed.wait(free).unwrap();
        }
        *free -= 1;
        Permit(self)
    }
}

impl Drop for Permit<'_> {
    // Dropping the permit gives the slot back, like leaving `async with sem`.
    fn drop(&mut self) {
        *self.0.permits.lock().unwrap() += 1;
        self.0.freed.notify_one();
    }
}

pub struct RetryPolicy {
    pub max_attempts: u32,
    pub base: f64,
    pub cap: f64,
}

impl Default for RetryPolicy {
```

```
    fn default() -> Self {
        RetryPolicy { max_attempts: 4, base: 0.5, cap: 20.0 }
    }
}

impl RetryPolicy {
    /// Exponential with FULL JITTER.
    pub fn backoff(&self, attempt: u32, rng: &mut SplitMix64) -> f64 {
        // base * 2^attempt is the exponential envelope; min() caps it so a late
        // attempt does not sleep for minutes. uniform(0, ...) is the jitter:
        // without it, every request from a failure burst retries at the same
        // instant and re-overloads the endpoint you just knocked over.
        rng.uniform(0.0, self.cap.min(self.base * 2f64.powi(attempt as i32)))
    }

    /// Only retry TRANSIENT failures.
    pub fn retryable(&self, err: &str) -> bool {
        // Retrying a genuine 400 or an assertion error just burns quota and
        // delays the real answer. Match on the message rather than the type
        // because SDKs wrap errors differently.
        let msg = err.to_lowercase();
        ["timeout", "rate", "429", "500", "502", "503", "connection"]
            .iter()
            .any(|s| msg.contains(s))
    }
}

/// asyncio.wait_for stand-in. A std thread cannot be cancelled, so on timeout
/// the call is abandoned and left to finish in the background; tokio's
/// timeout would drop the future instead.
fn call_with_timeout(model_fn: &ModelFn, prompt: &str, timeout_s: f64) -> Result<String, String> {
    let (tx, rx) = mpsc::channel();
    let (f, prompt) = (Arc::clone(model_fn), prompt.to_string());
    thread::spawn(move || {
        let _ = tx.send(f(&prompt));
    });
    match rx.recv_timeout(Duration::from_secs_f64(timeout_s)) {
        Ok(outcome) => outcome,
        Err(RecvTimeoutError::Timeout) => Err(format!("timeout after {timeout_s}s")),
        Err(RecvTimeoutError::Disconnected) => Err("model call panicked".to_string()),
    }
}

pub fn run_one(
    task: &Task,
    model_fn: &ModelFn,
    score_fn: &ScoreFn,
    policy: &RetryPolicy,
    timeout_s: f64,
    sem: &Semaphore,
    rng: &mut SplitMix64,
) -> EvalResult {
    let started = Instant::now(); // wall clock for THIS task, retries included
    let mut last_err = String::new();
    let mut attempts = 0;

    for attempt in 0..policy.max_attempts {
        attempts = attempt + 1;
        let outcome = {
            // The semaphore caps how many requests are in flight across the
```

```
            // whole suite. Acquire it INSIDE the loop so a sleeping retry
            // releases its slot instead of blocking other tasks.
            let _slot = sem.acquire();
            call_with_timeout(model_fn, &task.prompt, timeout_s)
        }; // _slot dropped here, before any backoff sleep

        match outcome {
            Ok(out) => {
                return EvalResult {
                    task_id: task.task_id.clone(),
                    score: Some(score_fn(task, &out)),
                    output: Some(out),
                    attempts, // RECORD attempts
                    latency_s: started.elapsed().as_secs_f64(),
                    ..Default::default()
                }
            }
            Err(e) => {
                // Two ways to stop: the error is not worth retrying, or we are out
                // of attempts. Checking both here keeps the exit conditions in one place.
                let give_up = !policy.retryable(&e) || attempt == policy.max_attempts - 1;
                last_err = e;
                if give_up {
                    break;
                }
                thread::sleep(Duration::from_secs_f64(policy.backoff(attempt, rng)));
            }
        }
    }

    // A task that never succeeded is still DATA. Return a result carrying the
    // error rather than panicking, so one bad task cannot kill a 2000 task run.
    EvalResult {
        task_id: task.task_id.clone(),
        error: Some(last_err),
        attempts,
        latency_s: started.elapsed().as_secs_f64(),
        ..Default::default()
    }
}

/// Python defaults: concurrency = 16, timeout_s = 120.0.
pub fn run_suite(
    tasks: &[Task],
    model_fn: ModelFn,
    score_fn: &ScoreFn,
    concurrency: usize,
    timeout_s: f64,
) -> Vec<EvalResult> {
    let sem = Semaphore::new(concurrency); // one shared limiter for every task
    let policy = RetryPolicy::default();
    let next = AtomicUsize::new(0);
    // More workers than permits, so a worker sleeping in backoff does not
    // leave a request slot idle. The gate, not the worker count, is the limit.
    let n_workers = tasks.len().min(concurrency * 4).max(1);
    let mut results: Vec<Option<EvalResult>> = vec![None; tasks.len()];

    thread::scope(|s| {
        let (sem, policy, next, model_fn) = (&sem, &policy, &next, &model_fn);
        let workers: Vec<_> = (0..n_workers)
```

```
            .map(|w| {
                s.spawn(move || {
                    let mut rng = SplitMix64::new(w as u64 + 1); // per-worker jitter stream
                    let mut done = Vec::new();
                    loop {
                        let i = next.fetch_add(1, Ordering::Relaxed);
                        if i >= tasks.len() {
                            break;
                        }
                        let r = run_one(&tasks[i], model_fn, score_fn, policy, timeout_s, sem, &mut rng);
                        done.push((i, r));
                    }
                    done
                })
            })
            .collect();
        // Like gather: slot each result back by index, so the output is in
        // the ORIGINAL task order regardless of completion order.
        for worker in workers {
            for (i, r) in worker.join().unwrap() {
                results[i] = Some(r);
            }
        }
    });
    results.into_iter().map(|r| r.expect("every task ran")).collect()
}
```

For leaders

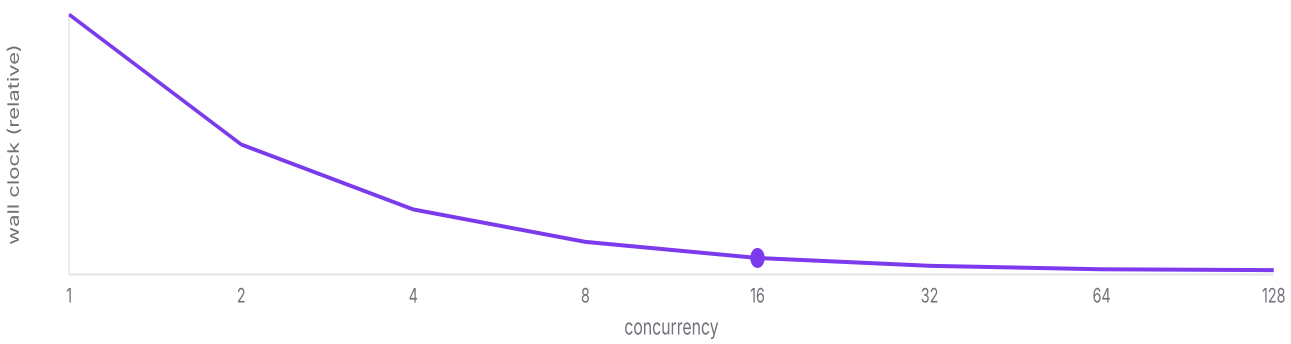
Retries are a measurement decision, not just an engineering one

Retrying is harmless when a failure happens before the model produced anything and is unrelated to the task: a dropped connection, a rate limit. It inflates scores when the "transient" failure is correlated with the answer. Common cases: a timeout that cuts off long, rambling, wrong generations; a parse error on malformed output that triggers a fresh sample; a sandbox crash during test execution that reruns the task. In each case the retried task gets another draw at the answer, so it is scored more like pass@k than pass@1. Insist that attempts are recorded per task and that reports show the score with and without retried tasks.

Interactive

Harness simulator: retries, concurrency, dropped tasks (shown at its default setting)

The true per attempt pass rate is fixed at 0.600. Raise the failure rate and the retry budget and watch your harness report something higher, because a retried task quietly got more than one shot at the answer.



True per attempt pass rate is 0.600. Your harness reports **0.630** (95% CI [0.599, 0.659]).
163 tasks retried, 1 tasks exhausted all attempts and were dropped from scoring.
Retries inflated the score by 0.030. A retried task got more than one shot at the answer, so it was effectively scored at pass@k with a larger k than the tasks that succeeded first time. Record attempts and report the score with and without retried tasks.
1 dropped tasks are silently excluded from the denominator. If failures correlate with difficulty, dropping them biases the score upward.
Wall clock at concurrency 16: 315.9 units. Push concurrency high enough and coordination overhead outweighs the parallelism.

Worked problem

Acceptance test. 1,000 tasks, 15% transient failure rate, up to 4 attempts, true per-attempt pass rate 0.600. The simulator above treats each failed attempt as an extra draw at the answer, the worst case described in the leader layer. What does the harness report?

Measured answer: **about 0.63**. A 1,000-task Python run gave 0.626 with 144 tasks retried; the simulator above, with its own random seed, shows 0.630 with 163. Either way the harness reports an improvement of roughly 3 points that does not exist, because each retried task got more than one shot. Without an `attempts` field per result you cannot even detect this. The fix is to record attempts and report the score with and without retried tasks. If the two numbers differ by more than the confidence interval, the retry policy is changing the measurement.

Key idea

Push concurrency up in the simulator. Wall clock falls at first, then rises as coordination overhead outweighs parallelism. Separately, any task that exhausts its attempts drops out of the scored denominator. If failures correlate with difficulty, which they usually do, dropping them biases the score upward.

Version three: caching, checkpointing, quarantine

Plain words

Remember answers, but only for the exact same question

Model calls are expensive, so the harness saves every answer. If the run crashes halfway, it picks up where it left off. The rule is strict: reuse a saved answer only if the question, the model, and every setting are exactly the same. If anything changed, ask again.

Under the hood

Cache keys over everything that can change the output, and an append-only log

python

```python
import hashlib, json, pathlib

def cache_key(task: Task, model_id: str, params: dict) -> str:
```

```
    """Key on EVERYTHING that can change the output."""
    payload = json.dumps({
        "task": task.task_id,
        # Hash the prompt itself, not just the id. If someone edits a task's
        # text without renaming it, the hash changes and the stale cached
        # result is correctly missed.
        "prompt_hash": hashlib.sha256(task.prompt.encode()).hexdigest(),
        "model": model_id,
        # sorted() so {"temp":0,"top_p":1} and {"top_p":1,"temp":0} produce the
        # same key. Without it you get silent cache misses on every run.
        "params": dict(sorted(params.items())),
    }, sort_keys=True)   # sort_keys again for the outer dict, same reason

    # 20 hex chars is ~80 bits: collision risk is negligible at eval scale and
    # the keys stay short enough to read in a directory listing.
    return hashlib.sha256(payload.encode()).hexdigest()[:20]


class ResultStore:
    """Append only JSONL. Restartable."""

    def __init__(self, path: str):
        self.path = pathlib.Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)   # no crash on first run
        self.seen: dict[str, dict] = {}
        if self.path.exists():
            # Replay the log into memory. Later lines overwrite earlier ones for
            # the same key, so a re-run naturally supersedes an old result.
            for line in self.path.read_text().splitlines():
                if line.strip():                    # tolerate a trailing newline
                    rec = json.loads(line)
                    self.seen[rec["key"]] = rec

    def get(self, key: str) -> dict | None:
        return self.seen.get(key)

    def put(self, key: str, result: Result) -> None:
        rec = {"key": key, **result.__dict__}
        # APPEND, never rewrite. A process killed mid-write loses at most the
        # last line, and a run that dies at task 900 of 1000 resumes without
        # re-spending 900 requests.
        with self.path.open("a") as f:
            f.write(json.dumps(rec, default=str) + "\n")   # default=str for datetimes
        self.seen[key] = rec
```

rust

```
use std::collections::{BTreeMap, HashMap};
use std::fs::{self, OpenOptions};
use std::io::{self, Write};
use std::path::PathBuf;

#[derive(Debug, Clone, Default)]
pub struct Task {
    pub task_id: String,
    pub prompt: String,
}

#[derive(Debug, Clone, Default)]
pub struct EvalResult {
```

```rust
    // `Result` in the Python
    pub task_id: String,
    pub output: Option<String>,
    pub score: Option<f64>,
    pub error: Option<String>,
    pub attempts: u32,
    pub latency_s: f64,
    pub meta: BTreeMap<String, String>,
}

/// 128-bit FNV-1a, so the sketch needs no crates. Real code would use SHA-256
/// from a crate such as sha2: FNV is fast but not collision resistant.
fn fnv1a_128_hex(data: &str) -> String {
    let mut h: u128 = 0x6c62272e07bb014262b821756295c58d;
    for b in data.bytes() {
        h ^= b as u128;
        h = h.wrapping_mul(0x0000000001000000000000000000013B);
    }
    format!("{h:032x}")
}

/// Quote a string as JSON. std has no JSON support; real code would use serde_json.
fn json_str(s: &str) -> String {
    let mut out = String::from("\"");
    for c in s.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            c if (c as u32) < 0x20 => out.push_str(&format!("\\u{:04x}", c as u32)),
            c => out.push(c),
        }
    }
    out.push('"');
    out
}

/// Key on EVERYTHING that can change the output.
pub fn cache_key(task: &Task, model_id: &str, params: &BTreeMap<String, f64>) -> String {
    // A BTreeMap iterates in sorted key order, so {"temp":0,"top_p":1} and
    // {"top_p":1,"temp":0} produce the same key. Without it you get silent
    // cache misses on every run.
    let params_json: Vec<String> = params.iter().map(|(k, v)| format!("{}:{}", json_str(k), v)).collect();
    // Outer fields written in sorted order by hand, same reason.
    let payload = format!(
        "{{\"model\":{},\"params\":{{{}}},\"prompt_hash\":{},\"task\":{}}}",
        json_str(model_id),
        params_json.join(","),
        // Hash the prompt itself, not just the id. If someone edits a task's
        // text without renaming it, the hash changes and the stale cached
        // result is correctly missed.
        json_str(&fnv1a_128_hex(&task.prompt)),
        json_str(&task.task_id),
    );

    // 20 hex chars is ~80 bits: collision risk is negligible at eval scale and
    // the keys stay short enough to read in a directory listing.
    fnv1a_128_hex(&payload)[..20].to_string()
}
```

```rust
/// Append only JSONL. Restartable.
pub struct ResultStore {
    path: PathBuf,
    // key -> the raw JSON line. With no JSON parser in std we keep lines as text.
    seen: HashMap<String, String>,
}

impl ResultStore {
    pub fn new(path: &str) -> io::Result<Self> {
        let path = PathBuf::from(path);
        if let Some(parent) = path.parent() {
            fs::create_dir_all(parent)?; // no crash on first run
        }
        let mut seen = HashMap::new();
        if path.exists() {
            // Replay the log into memory. Later lines overwrite earlier ones for
            // the same key, so a re-run naturally supersedes an old result.
            for line in fs::read_to_string(&path)?.lines() {
                // put() writes {"key":"<hex>",... first, so the key can be read
                // off the front; real code would parse the line with serde_json.
                // Blank lines (a trailing newline) simply fail to match.
                let key = line.strip_prefix("{\"key\":\"").and_then(|rest| rest.split('"').next());
                if let Some(key) = key {
                    seen.insert(key.to_string(), line.to_string());
                }
            }
        }
        Ok(ResultStore { path, seen })
    }

    pub fn get(&self, key: &str) -> Option<&str> {
        self.seen.get(key).map(String::as_str)
    }

    pub fn put(&mut self, key: &str, result: &EvalResult) -> io::Result<()> {
        let opt_str = |s: &Option<String>| s.as_deref().map_or("null".to_string(), json_str);
        let meta: Vec<String> = result.meta.iter().map(|(k, v)| format!("{}:{}", json_str(k),
json_str(v))).collect();
        // Every field is formatted explicitly, so there is no default=str fallback.
        let rec = format!(
            "{{\"key\":{},\"task_id\":{},\"output\":{},\"score\":{},\"error\":{},\"attempts\":{},\"latency_s\":
{},\"meta\":{{{}}}}}",
            json_str(key),
            json_str(&result.task_id),
            opt_str(&result.output),
            result.score.map_or("null".to_string(), |s| s.to_string()),
            opt_str(&result.error),
            result.attempts,
            result.latency_s,
            meta.join(","),
        );
        // APPEND, never rewrite. A process killed mid-write loses at most the
        // last line, and a run that dies at task 900 of 1000 resumes without
        // re-spending 900 requests.
        let mut f = OpenOptions::new().create(true).append(true).open(&self.path)?;
        writeln!(f, "{rec}")?;
        self.seen.insert(key.to_string(), rec);
        Ok(())
```

```
    }
}
```

Add the harness version and the judge model and prompt version to the cache key as well. A judge change silently reusing old scores is one of the most expensive cache bugs, because it produces a regression that does not exist.

Worked problem

Acceptance test. Run 1,000 tasks and kill the process at task 600. Restart. The run must finish with exactly 1,000 results and only 400 new model calls. Then change the temperature and confirm the cache misses on every task.

The parameter you forget to put in the key is the one that costs a day chasing a regression that is not there. Sort the parameter dict before hashing, or the same settings in a different order produce different keys.

Flakiness quarantine

Some tasks give different scores run to run even at fixed settings: nondeterministic environments, race conditions in test suites, judges sitting on a decision boundary. Run a sample of tasks five times at fixed settings and quarantine any whose score standard deviation exceeds about 0.25. Report quarantined tasks separately rather than averaging their noise into the headline.

python

```python
import statistics

def find_flaky(task_ids, run_once, repeats: int = 5, max_sd: float = 0.25) -> list[str]:
    """run_once(task_id) -> score in [0, 1] at FIXED settings. Returns tasks to quarantine."""
    flaky = []
    for t in task_ids:
        scores = [run_once(t) for _ in range(repeats)]
        if statistics.pstdev(scores) > max_sd:   # a 0/1 task that flips 2 of 5 times has sd 0.49
            flaky.append(t)
    return flaky
```

rust

```rust
/// Population standard deviation, what Python's statistics.pstdev computes.
fn pstdev(xs: &[f64]) -> f64 {
    let mean = xs.iter().sum::<f64>() / xs.len() as f64;
    (xs.iter().map(|x| (x - mean).powi(2)).sum::<f64>() / xs.len() as f64).sqrt()
}

/// run_once(task_id) -> score in [0, 1] at FIXED settings. Returns tasks to quarantine.
/// Python defaults: repeats = 5, max_sd = 0.25.
pub fn find_flaky(
    task_ids: &[&str],
```

```
    mut run_once: impl FnMut(&str) -> f64,
    repeats: usize,
    max_sd: f64,
) -> Vec<String> {
    let mut flaky = Vec::new();
    for &t in task_ids {
        let scores: Vec<f64> = (0..repeats).map(|_| run_once(t)).collect();
        if pstdev(&scores) > max_sd {
            // a 0/1 task that flips 2 of 5 times has sd 0.49
            flaky.push(t.to_string());
        }
    }
    flaky
}
```

The report

Plain words

A score without its fine print is not a score

The report is where all the care above either reaches the reader or gets lost. A pass rate on its own hides how many tasks there were, how uncertain the number is, and how many tasks never finished. The report should be impossible to produce without those.

Under the hood

A summary that cannot omit n, the interval, or the error count

The interval is the Wilson interval from eval statistics; the latency quantiles follow the advice in intervals and the bootstrap never to report a mean latency.

python

```python
def summarize(results: list[Result]) -> dict:
    """A report that cannot omit n, the interval, or the error count."""

    # Split errored tasks out FIRST. Scoring None as a failure would conflate
    # "the model got it wrong" with "our infrastructure fell over", which are
    # completely different findings.
    ok = [r for r in results if r.error is None and r.score is not None]

    passed = sum(1 for r in ok if r.score >= 1.0)
    n = len(ok)
    lo, hi = wilson(passed, n) if n else (0.0, 1.0)   # interval, never bare rate

    lat = sorted(r.latency_s for r in ok)
    def pct(q):
        # Guard the index: at q = 1.0 the naive int(q*len) runs one past the end.
        return lat[min(len(lat) - 1, int(q * len(lat)))] if lat else 0.0
```

```
    return {
        "n_tasks": len(results),
        "n_scored": n,
        # SURFACE the errors. A pass rate computed over a shrunken denominator
        # is biased upward if failures correlate with difficulty, which they do.
        "n_errored": len(results) - n,
        "pass_rate": passed / n if n else 0.0,
        "ci95": (round(lo, 4), round(hi, 4)),
        # Retried tasks effectively got more than one sample, inflating their
        # contribution. Reporting the count lets a reader discount for it.
        "retried_tasks": sum(1 for r in ok if r.attempts > 1),
        "latency_p50": round(pct(0.50), 2),
        "latency_p95": round(pct(0.95), 2),   # never report a mean latency
    }
```

rust

```
#[derive(Debug, Clone, Default)]
pub struct EvalResult {
    // `Result` in the Python, trimmed to the fields used here
    pub score: Option<f64>,
    pub error: Option<String>,
    pub attempts: u32,
    pub latency_s: f64,
}

/// The dict the Python returns, as a struct so no field can be left out.
#[derive(Debug)]
pub struct Summary {
    pub n_tasks: usize,
    pub n_scored: usize,
    pub n_errored: usize,
    pub pass_rate: f64,
    pub ci95: (f64, f64),
    pub retried_tasks: usize,
    pub latency_p50: f64,
    pub latency_p95: f64,
}

/// Wilson score interval, as in the evaluation statistics chapter.
fn wilson(k: usize, n: usize, z: f64) -> (f64, f64) {
    let (k, n) = (k as f64, n as f64);
    let p = k / n;
    let denom = 1.0 + z * z / n;
    let center = (p + z * z / (2.0 * n)) / denom;
    let margin = z * (p * (1.0 - p) / n + z * z / (4.0 * n * n)).sqrt() / denom;
    ((center - margin).max(0.0), (center + margin).min(1.0))
}

/// Python's round(x, digits).
fn round_to(x: f64, digits: i32) -> f64 {
    let scale = 10f64.powi(digits);
    (x * scale).round() / scale
}

/// A report that cannot omit n, the interval, or the error count.
pub fn summarize(results: &[EvalResult]) -> Summary {
    // Split errored tasks out FIRST. Scoring None as a failure would conflate
    // "the model got it wrong" with "our infrastructure fell over", which are
    // completely different findings.
```

```
    let ok: Vec<&EvalResult> = results.iter().filter(|r| r.error.is_none() && r.score.is_some()).collect();

    let passed = ok.iter().filter(|r| r.score.is_some_and(|s| s >= 1.0)).count();
    let n = ok.len();
    let (lo, hi) = if n > 0 { wilson(passed, n, 1.96) } else { (0.0, 1.0) }; // interval, never bare rate

    let mut lat: Vec<f64> = ok.iter().map(|r| r.latency_s).collect();
    lat.sort_by(f64::total_cmp);
    let pct = |q: f64| {
        // Guard the index: at q = 1.0 the naive (q * len) as usize runs one past the end.
        if lat.is_empty() { 0.0 } else { lat[(lat.len() - 1).min((q * lat.len() as f64) as usize)] }
    };

    Summary {
        n_tasks: results.len(),
        n_scored: n,
        // SURFACE the errors. A pass rate computed over a shrunken denominator
        // is biased upward if failures correlate with difficulty, which they do.
        n_errored: results.len() - n,
        pass_rate: if n > 0 { passed as f64 / n as f64 } else { 0.0 },
        ci95: (round_to(lo, 4), round_to(hi, 4)),
        // Retried tasks effectively got more than one sample, inflating their
        // contribution. Reporting the count lets a reader discount for it.
        retried_tasks: ok.iter().filter(|r| r.attempts > 1).count(),
        latency_p50: round_to(pct(0.50), 2),
        latency_p95: round_to(pct(0.95), 2), // never report a mean latency
    }
}
```

Worked problem

Acceptance test. The summary must refuse to omit the error count, and the flakiness report must identify tasks you deliberately injected as flaky. A useful extra check: inject 5% infrastructure errors concentrated on the hardest tasks and confirm the report shows both the error count and the upward bias it would cause if those tasks were silently dropped.

Eval lens

When two runs disagree, diff the harness before you diff the model: harness version, judge version, prompt template, timeouts, concurrency, sampling parameters, and task set membership. The regression triage chapter turns this into a procedure.

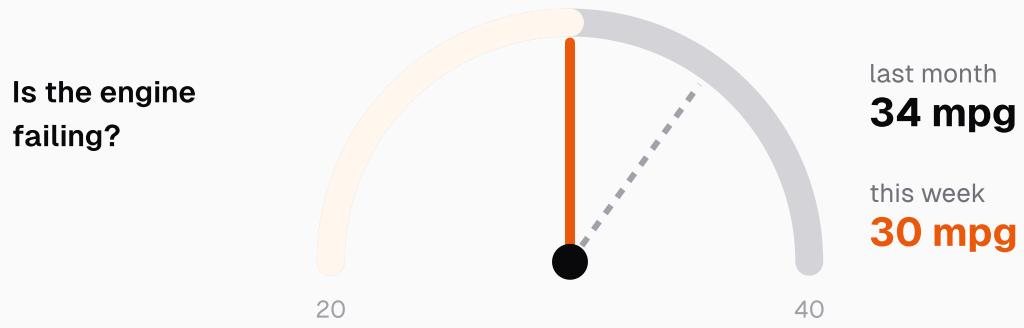
Diagnosing a regression: model, harness, data, or infrastructure?

An eval suite comes back with an anomalous number a few hours before a decision. The job is to say, quickly and with evidence, whether anything is actually wrong, and if so whether it is the model, the harness, the data, or the infrastructure. This chapter gives an order of operations that answers the cheapest questions first.

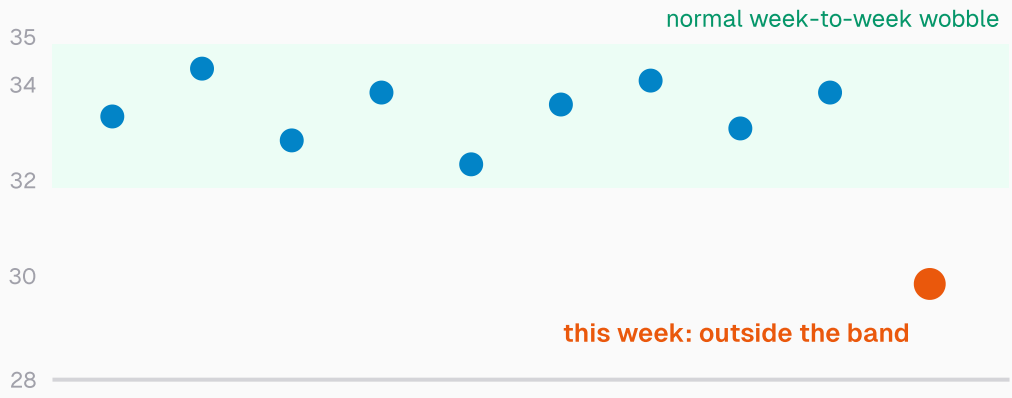
Diagram

The car that suddenly got worse mileage

A regression hunt in six pictures. The engine is the last suspect, not the first.

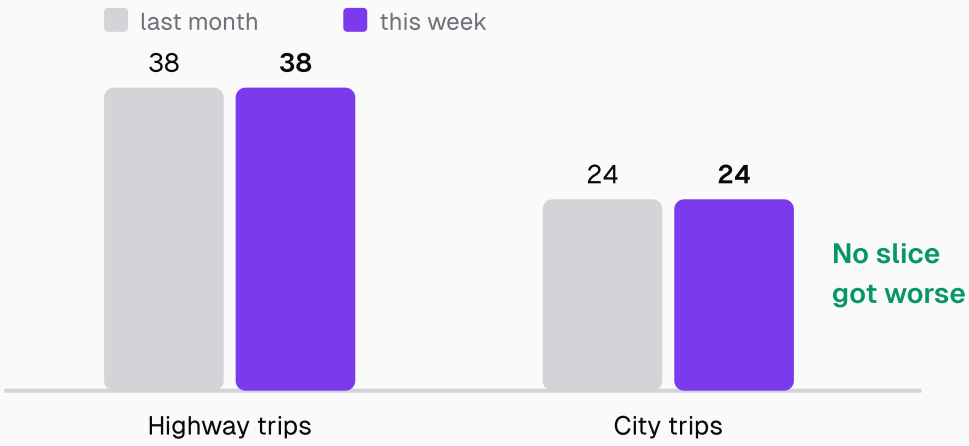


Your weekly average mileage drops from 34 to 30 mpg. The tempting move is to book the engine in for repair. In evals that is blaming the model the moment a score drops. Six cheaper checks come first.

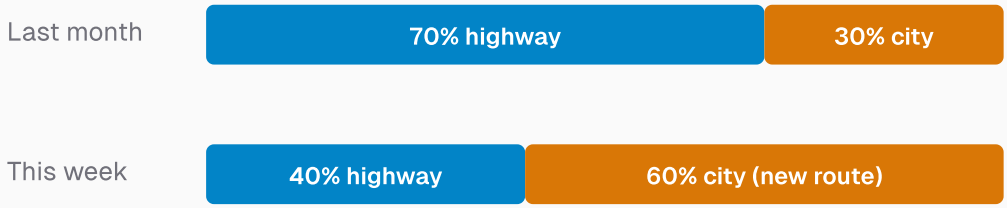


Step 1: is the drop bigger than normal noise? Here, yes.

First question: is this bigger than ordinary wobble? Weekly mileage normally bounces between about 32 and 35. Thirty is well outside that band, so something really changed. In evals, this is the paired interval on the delta. Many "regressions" stop here, inside the noise.



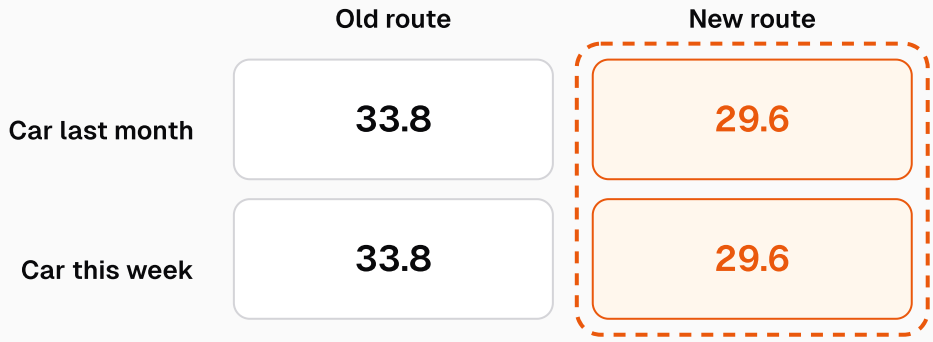
Second: is it everywhere or somewhere? Split trips by type. Highway trips still average 38 mpg and city trips still 24. Neither slice got worse, which is strange if the average fell. Slicing like this is how you find where a drop lives, or that it lives nowhere.



Last month: 0.7 × 38 + 0.3 × 24 = **33.8**
This week: 0.4 × 38 + 0.6 × 24 = 29.6

The engine did not change. The mix of trips did.

Third: what changed besides the engine? A road closure moved most of your driving into town. Each kind of trip is just as efficient, but the average of trip mileages slides from 33.8 to 29.6. In evals this is a task set whose composition shifted, for example more hard categories added this week.



The drop follows the route column, not the car row.

To be sure, replay: drive last month's car and this week's car on both routes. The old car drops on the new route too, and the new car is fine on the old route. In evals: rerun the previous model on today's harness and task set. If it drops as well, the model is not the cause.

1	Bigger than noise?	yes, outside the band
2	Everywhere or somewhere?	no slice got worse
3	What else changed?	the trip mix (task set)
4	Replay with the old car?	it drops too

Verdict: measurement, not model

The verdict comes from evidence, not instinct: a real drop, no slice worse, a changed task mix, and a replay that clears the model. The same checklist, plus error rates and reading twenty failing outputs, settles most eval regressions within hours.

Plain words

The thermometer before the doctor

If a child's temperature reading looks high, a sensible parent first checks the thermometer. Is it working? Did they measure right after a hot bath? Only then do they worry about illness.

A falling eval score is the same. Before blaming the model, check whether the drop is bigger than normal noise, whether the measuring setup changed, and whether the computers running the test had problems. Most "regressions" turn out to be the thermometer.

For leaders

Incident response for a number

Treat an eval regression like a production incident: one owner, a fixed checklist, a written timeline, and a clear exit. The first question is not "what broke" but "did anything break". Asking it first protects the team's time and stops a launch being blocked by noise, or approved past a real problem.

A useful operating rule: the triage owner reports within a fixed window (a few hours) with one of four verdicts, each with its evidence.

Noise

The change is inside the confidence interval. No action; note it and move on.

Measurement

Harness, judge, prompt, or task set changed. Fix the measurement and rerun before anyone discusses the model.

Infrastructure

Timeouts, rate limits, or errors spiked. Rerun under healthy conditions.

Model

Reproduces on replay and is concentrated in specific capabilities. Escalate with example outputs and the affected slices.

What goes in the launch review afterwards: the verdict, the interval, the slice breakdown, twenty example outputs that show the failure, and what changes in the harness or checks so this class of problem is caught automatically next time.

Principal asks

- Is this outside the noise band? What is the interval on the delta, computed paired?
- Is the drop uniform or concentrated? Which slice contributes most of the total?
- What changed besides the model? Show me the config diff.
- Did the previous model also drop when rerun on today's harness?
- Have you read the failing outputs yourself?

The order to check things in

1. **Is it real?** Compute the interval before you investigate. Use the paired comparison from eval statistics: the same tasks, both models, discordant pairs. A 3 point drop inside the noise band is not an incident. This step alone saves days and is the one people skip.

2. **Is it everywhere or somewhere?** Break the delta down by task category, language, input length, source. A uniform drop points at the model or the harness. A drop concentrated in one slice points at data or a specific capability.

3. **Did anything non-model change?** Harness version, judge model or prompt, timeout, concurrency, temperature, task set membership, dependency versions. Diff the config before you diff the model.

4. **Error and retry rates.** A drop that arrives with a spike in timeouts is infrastructure wearing a model costume. See the harness chapter for why errors and retries bias scores.

5. **Replay.** Rerun a sample of the regressed tasks against the previous model with today's harness. If the old model also drops, it is not the model.

6. **Read the outputs.** Twenty of them, by hand, every time. The failure mode is usually visible and usually not what the metric suggested.

Under the hood

Slice by contribution, not by percentage change

The total change in mean score decomposes exactly into slice contributions. With N tasks split into slices s of size n_s and per-slice mean change $\bar{d}_s$:

$$\Delta_{\text{total}} = \frac{1}{N} \sum_{s} n_s \, \bar{d}_s$$

Each slice's contribution is $n_s \bar{d}_s / N$. Rank slices by this, not by $\bar{d}_s$ alone, or a tiny slice with a big swing will send you down the wrong path. This identity holds when slices partition the tasks.

python

```python
def slice_deltas(before: dict[str, float], after: dict[str, float],
                 slices: dict[str, list[str]]) -> list[tuple[str, float, int]]:
    """Where did the drop actually happen. Sort by contribution to the total
    delta, not by percentage change, or a tiny slice with a big swing will
    send you down the wrong path."""
    out = []
    for name, task_ids in slices.items():
        ids = [t for t in task_ids if t in before and t in after]
        if not ids:
            continue
        d = sum(after[t] - before[t] for t in ids) / len(ids)
        contribution = d * len(ids)
        out.append((name, d, len(ids), contribution))
    out.sort(key=lambda x: x[3])            # most negative contribution first
    return [(n, round(d, 4), k) for n, d, k, _ in out]
```

rust

```rust
use std::collections::HashMap;

/// Where did the drop actually happen. Sort by contribution to the total
/// delta, not by percentage change, or a tiny slice with a big swing will
/// send you down the wrong path.
/// `slices` is a list of (name, task ids) pairs, so ties keep the order given,
/// as they do with Python's insertion-ordered dict and stable sort.
pub fn slice_deltas(
    before: &HashMap<String, f64>,
    after: &HashMap<String, f64>,
    slices: &[(String, Vec<String>)],
) -> Vec<(String, f64, usize)> {
    let mut out = Vec::new();
    for (name, task_ids) in slices {
        let ids: Vec<&String> = task_ids.iter().filter(|t| before.contains_key(*t) &&
after.contains_key(*t)).collect();
        if ids.is_empty() {
            continue;
        }
        let d = ids.iter().map(|t| after[*t] - before[*t]).sum::<f64>() / ids.len() as f64;
        let contribution = d * ids.len() as f64;
        out.push((name.clone(), d, ids.len(), contribution));
    }
    out.sort_by(|x, y| x.3.total_cmp(&y.3)); // most negative contribution first
    out.into_iter()
        .map(|(n, d, k, _)| (n, (d * 1e4).round() / 1e4, k)) // round(d, 4)
        .collect()
}
```

> ⚠ **Watch out**
>
> Slicing is a multiple comparison. With twenty slices, one will look significant by chance at the 5% level. Treat the slice view as a way to decide where to read outputs, not as a finding in itself, or apply a correction such as Benjamini–Hochberg (see multiple comparisons).

Replay as a two-by-two

The replay step is cleanest as a small factorial: old and new model crossed with old and new harness. If the drop appears only in the new-harness column, the harness is the cause. If it appears only in the new-model row, it is the model. If it appears in only one cell, look for an interaction, such as a new prompt template that only the new model handles badly.

	Old harness	New harness
Old model	Baseline	Harness effect alone
New model	Model effect alone	What you observed

Eval lens

Every triage should end with a change to the harness or the automated checks, so the same class of false alarm or real regression is caught without a human next time. A regression you diagnosed twice is a missing test.

Part II · Chapter 16

Which metric or test, when

This chapter is a decision guide. It pulls together the metrics from classification and judges and the tests from statistics for eval sets and intervals and the bootstrap. For "my data has this shape, which method fits," use the lookup in from shape to method.

Plain words

Pick the metric by the question you are asking

Every metric answers a different question. "When it says yes, is it right?" is precision. "Did it find them all?" is recall. "Does it rank good above bad?" is AUC. "Is this difference real or luck?" is a statistical test. Start from the question, then pick the number. Picking the number first and inventing the question afterwards is how dashboards end up misleading people.

Diagram

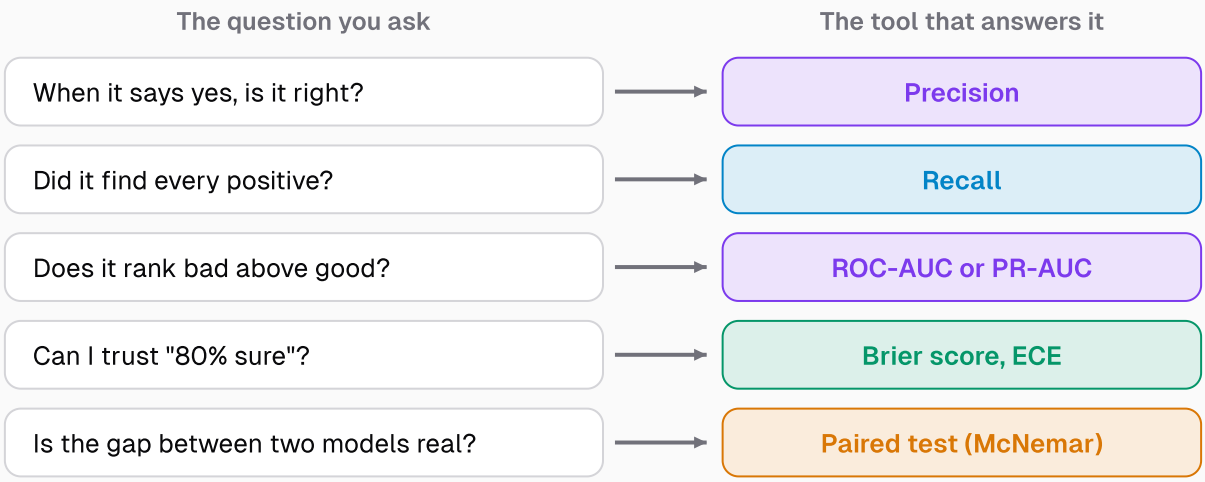
Pick the measuring tool before you measure

A kitchen drawer of tools, mapped onto the metric drawer.

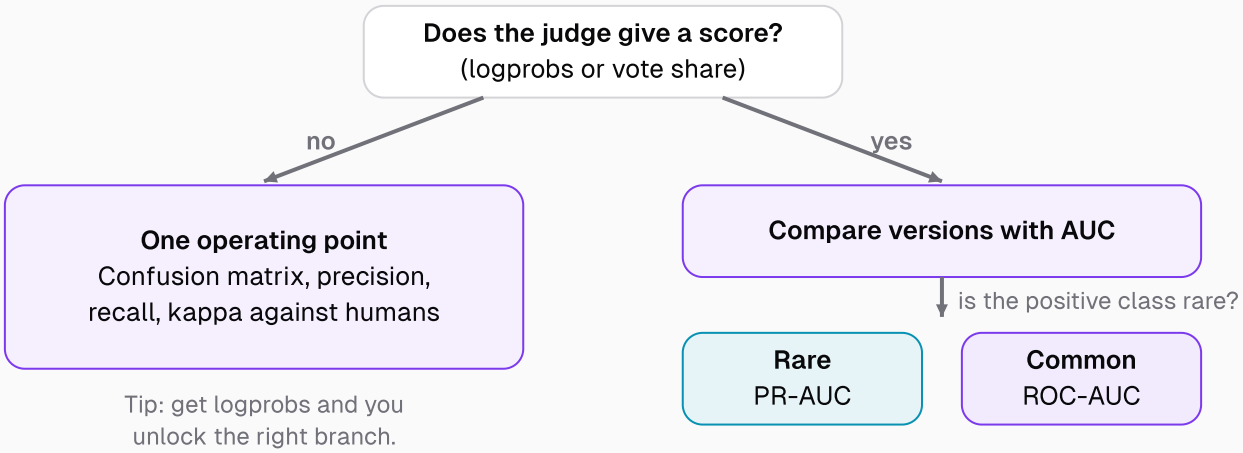
A timer cannot tell you whether the chicken is cooked.

Every tool answers one question. So does every metric.

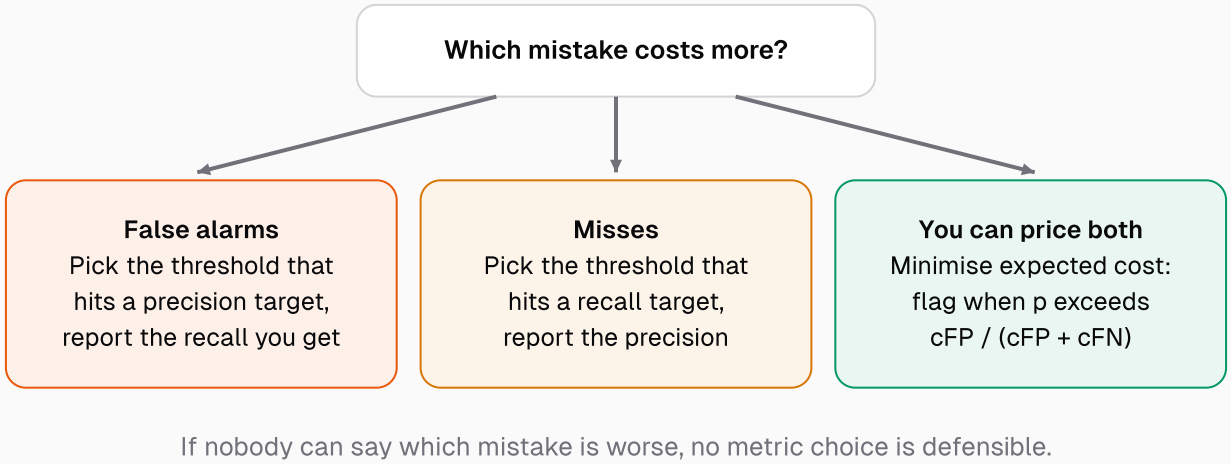
A thermometer, a scale, and a timer each answer one question well and every other question badly. Metrics are the same: pick the question first, then the tool.



Five questions people actually ask about a classifier or judge, and the tool that answers each one.



The first fork: does your judge give a score or only yes/no? Yes/no gives one operating point, so you work from its confusion matrix. A score lets you compare versions with AUC, and PR-AUC if the thing you are catching is rare.



The second fork: which mistake costs more? That decides where the threshold goes. If you can put a price on both mistakes, the threshold becomes arithmetic.

Last question: is the difference real?

New judge prompt: +3 points on 200 tasks.

Only 30 tasks changed verdict, split 18 better and 12 worse.

McNemar p = 0.36: a coin flip does this a third of the time.

- Put an interval on every number
- Compare on the same items (paired)
- Pin the threshold and the test set

Question first, tool second.

The last check before anyone celebrates: is the difference real? A 3-point gain resting on 30 changed verdicts split 18 to 12 is what chance produces about a third of the time. Intervals, paired comparisons, and a pinned test set keep you honest.

A recommendation for your classifier or judge

Interactive **Metric picker** (shown at its default setting)

Answer three questions about your judge or classifier and get the numbers to compare versions with, how to set the operating point, and what else to report.

Does it give a score, or only yes/no?

Only yes/no | A score (logprobs or votes)

Is the thing you are catching rare (under about 20%)?

Yes, it is rare | No, it is common

Which mistake hurts more?

False alarms | Misses | About equal

COMPARE VERSIONS WITH	Precision and recall for the rare class, plus its F1. A new version wins only if one number improves without the other dropping.
SET THE OPERATING POINT	Use stricter rubric wording until precision reaches your target. Report the recall you give up.
ALSO REPORT	The confusion matrix at the operating point, n, and Cohen's kappa against human labels on the same items. Keep accuracy out of the headline; it hides the mistakes you care about. If you can, get a score from logprobs or repeated votes so AUC becomes available.

Metric reference

Metric	Formula	Question it answers	Reach for it when	Watch out
Accuracy	`(TP+TN) / n`	How often is it right overall?	Balanced classes, symmetric costs	Misleading when one class is rare
Precision	`TP / (TP+FP)`	When it says yes, is it right?	False alarms are costly	High while missing most positives
Recall (TPR)	`TP / (TP+FN)`	Did it find all the positives?	Misses are costly	Flagging everything scores 100%
Specificity (TNR)	`TN / (TN+FP)`	Did negatives get through untouched?	Protecting the negative class matters	It is recall for the other class
F1 / F-beta	`2PR / (P+R)`	Good at both precision and recall?	One number for a rare class	Ignores TN; depends on threshold
Balanced accuracy	`(TPR+TNR) / 2`	Right on both classes, equally weighted?	Executive summary under imbalance	Hides which error dominates
MCC	`see chapter`	How correlated are prediction and truth?	One number to compare models	Unintuitive scale
Macro F1	`mean of class F1`	Good on every class?	Rare classes matter	Small classes swing it
ROC-AUC	`P(s+ > s−)`	Does it rank positives above negatives?	Comparing scored models, moderate imbalance	Flatters when positives are rare

Metric	Formula	Question it answers	Reach for it when	Watch out
PR-AUC (AP)	`area under PR`	How precise across recall levels?	Comparing scored models, rare positives	Baseline is prevalence; same data only
Brier / ECE	`mean (p−y)²`	Is the confidence honest?	Confidence drives routing or abstention	ECE depends on binning
Coverage–risk	`risk at coverage κ`	How much error if we only answer when sure?	The system can abstain or ask	Needs a confidence signal that tracks correctness
Expected cost	`(c_FP·FP + c_FN·FN)/n`	What does this threshold cost us?	You can price the errors	Needs an agreed cost ratio
Cohen's kappa	`(p_o − p_e)/(1 − p_e)`	Agreement beyond luck?	Judge versus human, human versus human	Check human ceiling first

Statistical tests reference

Question	Method	Why	Chapter
Interval on one pass rate	Wilson interval	Stays inside [0, 1] and holds coverage at small n and extreme rates	Eval statistics
Two models, same tasks, binary outcome	McNemar (exact) or paired bootstrap	Only discordant tasks carry information; unpaired tests waste power	Eval statistics
Two models, same tasks, continuous score	Paired bootstrap or Wilcoxon signed-rank	Uses the pairing, no normality assumption	Intervals and bootstrap
Three or more models, same tasks	Cochran's Q (binary) or Friedman (ranked)	One omnibus test before pairwise comparisons	Eval statistics
Many benchmarks at once	Benjamini–Hochberg	Controls the share of false discoveries	Eval statistics
Interval on a median, p95, or AUC	Bootstrap (percentile or BCa)	No closed form exists	Intervals and bootstrap
Items nested in sessions or tasks	Cluster bootstrap	Naive intervals are far too narrow when items correlate	Intervals and bootstrap
How many tasks do I need?	Power analysis on discordance	Detecting a 2-point move usually needs about a thousand tasks	Eval statistics

Walk the decision tree

Interactive **Which test or metric should I use** (shown at its default setting)

Answer a few questions and get a recommendation with its reason and its drawback. Every leaf carries a con, because the drawback is the part that gets asked about. Click a crumb to go back.

start

What are you measuring?

- A pass / fail outcome
- A continuous value (latency, tokens, score)
- An ordinal rating (Likert 1 to 5)
- A count (tool calls per session)
- Classifier or judge quality

Drill

Work through every branch once. There are about twenty leaves, and each is a short answer you should be able to give without the widget: the recommendation, the reason, and the drawback.

For leaders **The metric pack for a launch review**

Agree on the metric pack before results exist, and keep it fixed. A one-page eval readout for a classifier, judge, or safety filter should contain:

1. The question being answered and which error costs more, in one sentence each.
2. n, the positive rate in the test set, and the positive rate expected in production.
3. The confusion matrix at the chosen operating point, and how that point was chosen (on which split).

4. The headline metric with a confidence interval, and the comparison to the baseline with a paired test.
5. Per-class or per-slice results for anything rare that carries risk.
6. For judge-scored results: judge version, judge-to-human kappa, and the human ceiling.

Principal asks

- Was this metric chosen before or after seeing the results?
- If the positive rate in production differs from the test set, what precision should we actually expect? (Recompute it from TPR and FPR.)
- Which result would have made us not ship? If none, the evaluation was not a decision input.

Under the hood

Two formulas that settle most arguments

Precision under prevalence shift. TPR and FPR are properties of the classifier; precision is not. If you validated at prevalence π and production runs at π':

$$\text{Precision}(\pi') = \frac{\text{TPR}\,\pi'}{\text{TPR}\,\pi' + \text{FPR}\,(1-\pi')}$$

A filter with TPR 0.90 and FPR 0.05 has precision 0.95 at $\pi = 0.5$, but only 0.15 at $\pi' = 0.01$.

The cost-optimal threshold. With calibrated scores, flag when the expected cost of not flagging exceeds the expected cost of flagging:

$$p\,c_{FN} > (1-p)\,c_{FP} \iff p > \frac{c_{FP}}{c_{FP} + c_{FN}}$$

If the scores are not calibrated, sweep thresholds on a validation split, pick the minimum-cost one, and report its cost on a separate test split.

python

```python
def precision_at(tpr, fpr, prevalence):
    return tpr * prevalence / (tpr * prevalence + fpr * (1 - prevalence))

print(round(precision_at(0.90, 0.05, 0.50), 2))  # 0.95
print(round(precision_at(0.90, 0.05, 0.01), 2))  # 0.15
```

rust

```rust
fn precision_at(tpr: f64, fpr: f64, prevalence: f64) -> f64 {
    tpr * prevalence / (tpr * prevalence + fpr * (1.0 - prevalence))
}

fn main() {
    println!("{:.2}", precision_at(0.90, 0.05, 0.50)); // 0.95
    println!("{:.2}", precision_at(0.90, 0.05, 0.01)); // 0.15
}
```

Eval lens

The most common failure is not the wrong metric. It is a metric whose definition, threshold, or test set quietly changed between the baseline and the candidate. Pin all three, and the choice of metric becomes much less contentious.

Part III

Training models

How a frontier model is made: pretraining, optimisation, scaling, supervised fine-tuning, LoRA, reward models, RLHF, DPO, GRPO, distillation, inference, and safety training.

Part III · Chapter 17

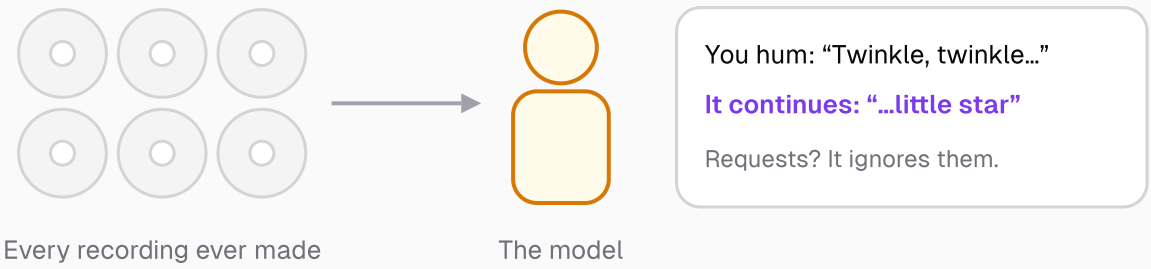
The training pipeline, end to end

A frontier model is not trained once. It is assembled in stages, and each stage changes different things about the model. If you know what each stage buys, you can usually guess where a problem came from before anyone opens a log.

Diagram

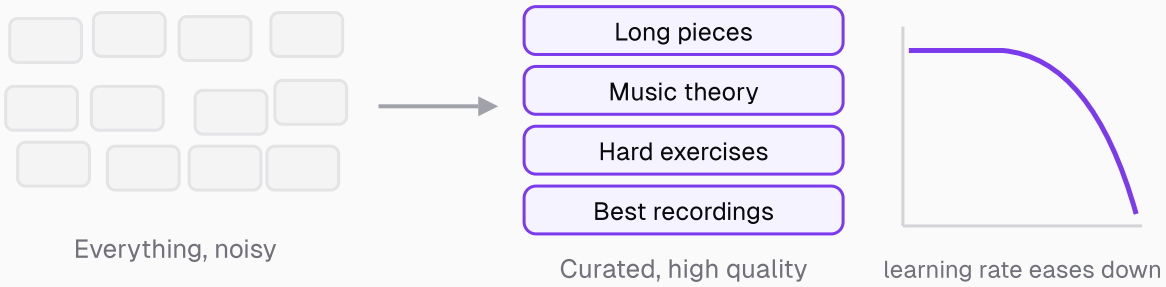
From listener to performer

The seven stages of making a model, told as the training of a musician.



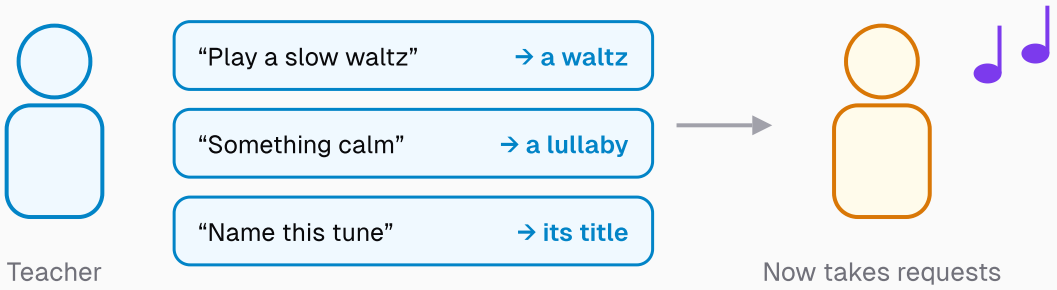
Stage 1 · Pretraining: listen to everything

Pretraining is listening to every recording ever made. The model can continue any tune you start, and it has absorbed an enormous amount about music, but it does not take requests. It just keeps going. This stage uses the large majority of the compute.



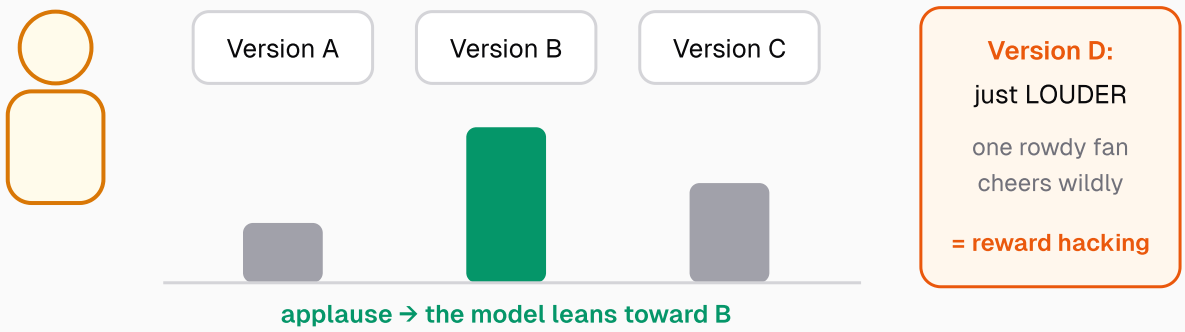
Stage 2 · Mid-training: focused practice on the best material

Mid-training is a focused practice block: the same objective, but on the best material (long pieces, theory, hard exercises), often while the learning rate winds down. Labs use it to add long-context skill and to boost code, math, and quality.



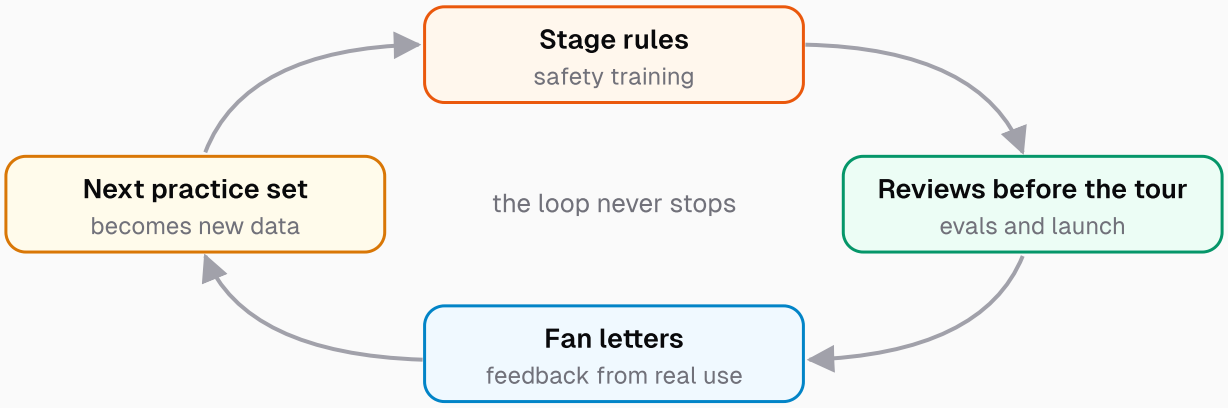
Stage 3 · SFT: lessons from examples of good answers

Supervised fine-tuning is lessons. A teacher shows thousands of examples of a request and a good response. The model learns the format: listen to the request, answer it, stop. It learns very little new music here; it learns how to perform on request.



Stage 4 · Preference tuning and RL: learn from applause

Preference tuning and RL are learning from applause. The model plays several versions, something scores them (people, a reward model, or a checker), and it shifts toward what scored well. The danger is version D: if one rowdy fan cheers for loudness, the model learns to be loud. That is reward hacking.



Then come stage rules (safety training), reviews before the tour (evaluations and launch gates), and fan letters (feedback from real use), which become the next practice set. A frontier model is never finished; it goes around this loop every release.

1 **Data** Crawl, filter, dedup, mix	2 **Pretraining** Next-token prediction on trillions of tokens	3 **Mid-training** Annealing, long context, domain boosts	4 **SFT** Demonstrations of the format and behaviour	5 **Preference and RL** RLHF, DPO, RL on verifiable rewards

6 **Safety**
Refusals, policy, red-team fixes

7 **Evals and launch**
Gates, staged rollout

8 **Feedback**
Logs, ratings, failures become data

Plain words

School, then apprenticeship, then coaching

Pretraining is like reading a whole library. The model learns language, facts, and how ideas connect, but it has no manners and does not know it is supposed to help anyone. It just continues whatever text you hand it.

Supervised fine-tuning (SFT) is an apprenticeship. The model watches thousands of examples of a good assistant answering questions and learns to copy that style.

Preference tuning and reinforcement learning (RL) are coaching. The model tries answers, someone or something scores them, and the model shifts toward the answers that score well. This is where it learns to be helpful, careful, and, for reasoning models, to think through a problem step by step.

Most raw knowledge comes from the library stage. Most of the personality and the polish comes from the later stages.

For leaders

What each stage buys, costs, and breaks

Stage	What it buys	Rough compute share	Typical owner	Classic failure
Data	The ceiling on everything after it	Small in FLOPs, large in people	Data / pretraining team	Contamination, duplicates, a bad mixture weight
Pretraining	Knowledge, language, raw capability	Historically the vast majority (often quoted above 90%)	Pretraining team, infra	Loss spikes, a missing domain, a tokenizer quirk you live with forever
Mid-training	Long context, quality boost, domain depth (code, math)	A few percent	Pretraining team	Forgetting, context extension that passes needle tests but fails real tasks
SFT	Format, instruction following, tool syntax	Well under 1%	Post-training team	Style over substance, hallucinated confidence copied from demos
Preference and RL	Helpfulness, reasoning, agentic skill	Small for RLHF; growing fast for reasoning RL (some 2025 labs report RL compute on the same order as pretraining)	Post-training / RL team	Reward hacking, sycophancy, length inflation
Safety	Policy compliance, refusal boundaries	Small	Safety / alignment team	Over-refusal or under-refusal, jailbreak gaps
Evals and launch	Confidence that it works and is safe	Small but continuous	Evals team, product	Benchmarks that do not predict real use

Compute shares are illustrative. They vary by lab and by year, and the RL share has been rising since reasoning models arrived in late 2024.

Two things matter for planning. First, pretraining decisions are the most expensive to reverse: a tokenizer or a data mixture is baked into a run that may cost tens of millions of dollars. Post-training is cheap enough to iterate weekly. Second, post-training mostly *elicits* capability rather than creating it. If the base model cannot do something even with good prompting and many samples, no amount of SFT will make it reliable. RL on verifiable tasks is the partial exception, which is why it gets so much attention (see GRPO and RL with verifiable rewards).

Principal asks

- Is this a capability problem (pretraining) or a behaviour problem (post-training)? What evidence separates the two?
- Does the base model show the skill at pass@k? If not, why do we expect fine-tuning to create it?
- Which eval would have caught this one stage earlier, where it is cheaper to fix?
- What did this stage cost in compute and in people, and what is the next most valuable use of that budget?

Under the hood

Objectives by stage

Each stage optimizes a different objective, which is why each stage fails differently.

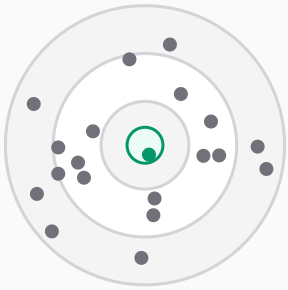
Stage	Objective	Data
Pretraining	$\min_\theta -\sum_t \log p_\theta(x_t \mid x_{<t})$ over all tokens	Trillions of web, code, book, and synthetic tokens
Mid-training	Same loss, different data: high-quality and long documents, often with the learning rate decaying to zero	Billions to hundreds of billions of curated tokens
SFT	Same loss, but only on response tokens: $-\sum_{t \in \text{response}} \log p_\theta(y_t \mid x, y_{<t})$	Thousands to millions of prompt and response pairs
RLHF / RL	$\max_\theta \mathbb{E}_{y \sim \pi_\theta}[r(x, y)] - \beta \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	Prompts plus a reward model or a verifier
DPO and relatives	A classification loss on preferred versus rejected responses	Preference pairs

The KL term in the RL objective keeps the policy close to the SFT model. Without it, the policy drifts toward whatever the reward model mistakenly loves. Chapters SFT, reward models, RLHF, and DPO unpack each line.

Diagram

Is the skill missing, or just unreliable?

One measurement tells you which stage to fix: how often the base model gets it right in many tries.



Base model, 20 throws

1 bullseye

pass@1 ≈ 5% · pass@20: a hit

The skill is there, just rare.

Give the base model twenty tries at a task. If it gets it right even once, the skill exists somewhere in the model. pass@k, the chance at least one of k tries succeeds, measures this.

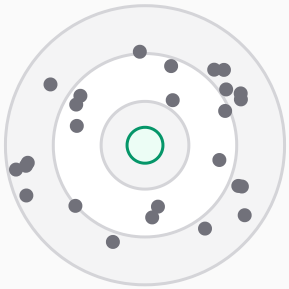
After coaching (SFT, RL)

10 bullseyes in 20

pass@1 ≈ 50%

Same skill, now reliable.

Post-training mostly turns a rare success into a reliable one. pass@1 jumps; pass@k at large k barely moves. That is elicitation: drawing out a skill the base model already had.



A skill it never shows

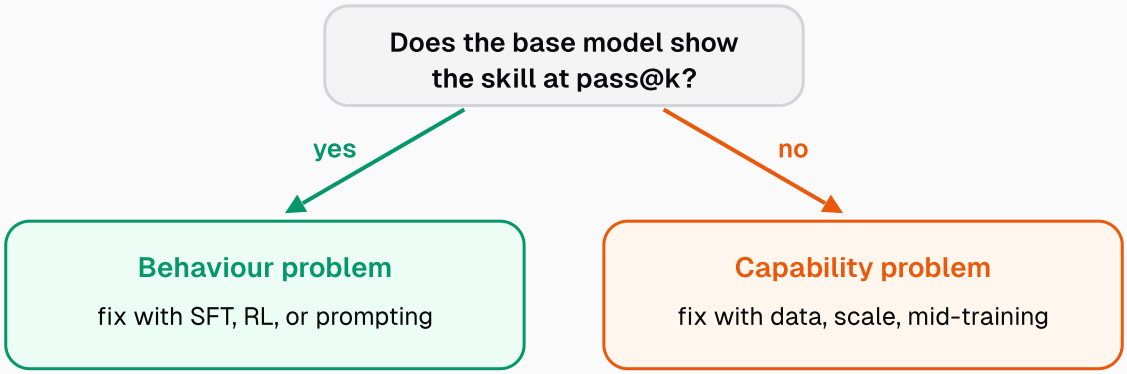
0 bullseyes in 1,000

Coaching rarely creates this.

Fix it earlier: data, scale, mid-training.

RL on checkable tasks can stretch it a little.

If the base model never succeeds, even with many tries and good prompts, fine-tuning is unlikely to fix it. The problem lives in pretraining: missing data, too little scale. RL on tasks with automatic checkers is the partial exception.



So before choosing where to spend effort, measure the base model at pass@k. That one number usually tells you which stage to fix and which team owns the problem.

Where evals sit

Evaluation is not a gate at the end. Every stage has its own checks, and the cheapest bug is the one caught at the stage that caused it.

Stage	What you measure	Typical tools
Data	Quality, duplication, contamination against eval sets, mixture proportions	n-gram overlap checks, classifier scores, sampled human review
Pretraining	Held-out loss by domain, few-shot benchmarks on checkpoints, loss spikes	Loss dashboards, small "proxy" eval suites run every few thousand steps
Mid-training	Long-context retrieval and reasoning, regression on general benchmarks	Needle tests plus real long-document tasks
SFT	Instruction following, format validity, refusal rate	Rule-based checks, LLM judges
Preference and RL	Win rate, reward versus KL, length, verifier pass rate, held-out tasks	Judges, verifiers, human preference studies
Safety	Harmful compliance, over-refusal, jailbreak robustness	Red-team suites, policy classifiers
Launch	Online metrics against offline predictions	A/B tests, staged rollout

Eval lens

When a metric moves, first ask which stage changed. A base-model benchmark that drops after RL is a forgetting problem, not a pretraining problem. A judge win rate that rises with response length is probably a reward or judge bias, not a capability gain (see LLM as judge).

Interactive **Where did this failure come from?** (shown at its default setting)

Pick a symptom. The stages most likely responsible light up, with the first thing to check.

1 Data	2 **Pretraining** Most likely	3 Mid-training	4 SFT
5 Preference / RL	6 Safety	7 Evals / launch	

Why: Tokenization. The model sees subword tokens, not letters, so character-level tasks are hard. Fixed at pretraining time, not later.
First check: Tokenize the failing words and look at the pieces. Test whether spacing the letters out fixes it.

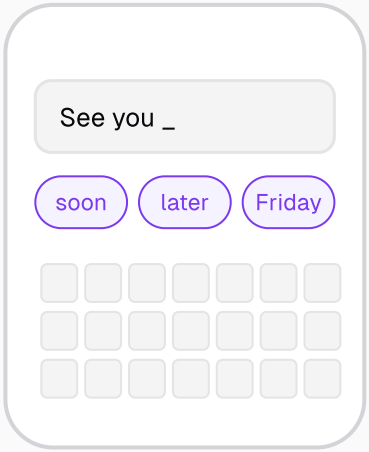
Part III · Chapter 18

Pretraining: learning from the whole library

Pretraining is where almost all of a model's knowledge and raw capability comes from. The task is simple to state: given some text, predict the next piece. Doing it well across trillions of tokens forces the model to learn grammar, facts, code, and a surprising amount of reasoning.

Diagram **The world’s biggest autocomplete game**

What a model actually does during pretraining, in five pictures.

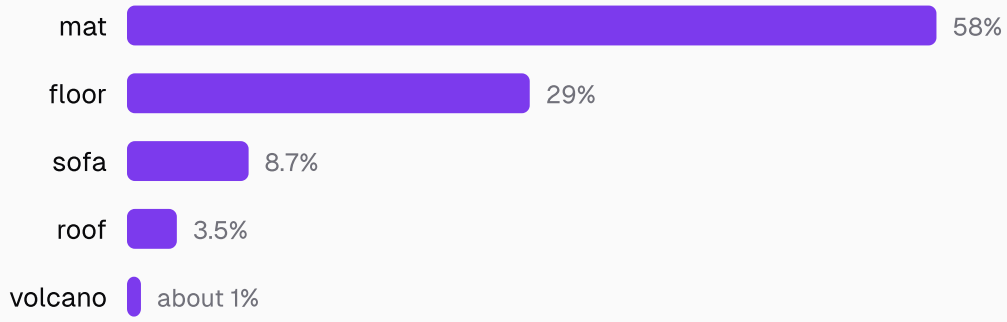


Your phone guesses the next word.

A language model plays the same game, trillions of times, on text from books, code, and the web.

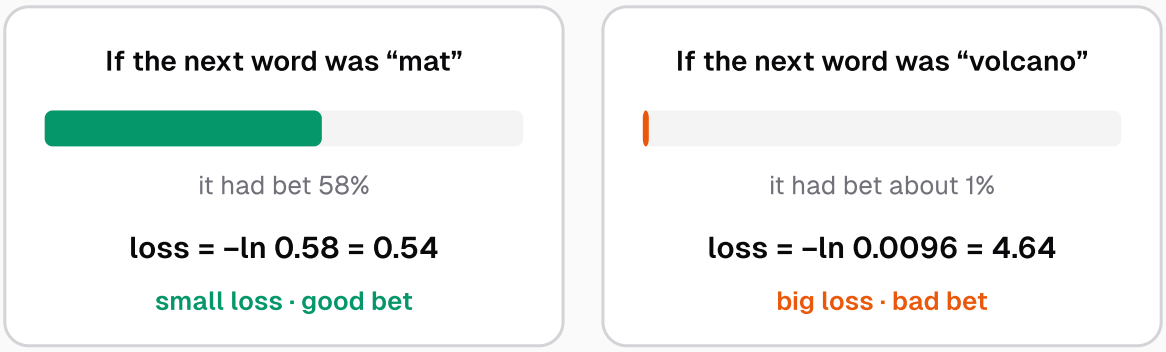
Your phone suggests the next word as you type. Pretraining is the same game at enormous scale: read some text, guess what comes next, check, repeat.

The cat sat on the ___



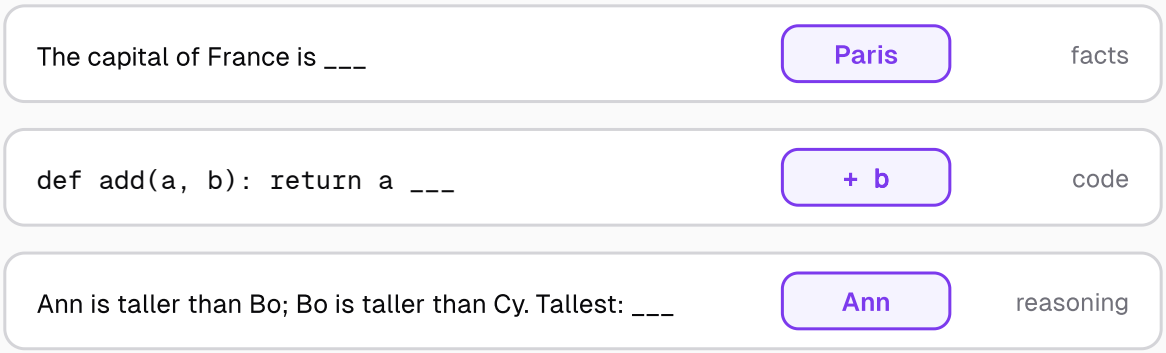
Probabilities from the logits in the code below (illustrative).

The model does not pick one word. It spreads its bets across every token it knows: 58% on “mat”, 29% on “floor”, about 1% on “volcano”.



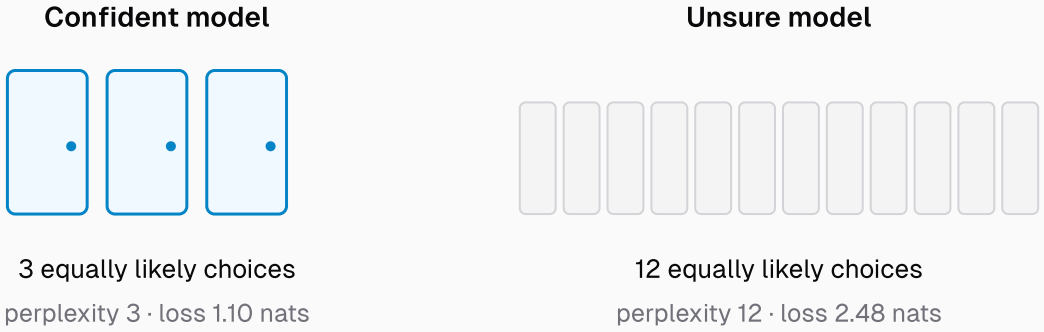
Training nudges billions of numbers so the next bet is a little better.

Then the real next word is revealed. The **loss** is minus the log of the bet on the right answer: 0.54 for a confident correct bet, 4.64 when it bet almost nothing on the right word. Training lowers the average loss.



To bet well on trillions of blanks, the model has to learn what the text is about.

Why does a guessing game produce something smart? Because betting well on every blank requires knowing facts, following code, and tracking logic. The objective is simple; what it takes to do well is not.



Perplexity = exp(loss): roughly how many doors the model is choosing between.

Perplexity is the loss turned into a count: about how many equally likely options the model feels it is choosing between. Lower is better. It is only comparable between models that share a tokenizer and test set.

Next-token prediction and its loss

Plain words

A guessing game with a scoreboard

Cover the next word of a sentence and guess it. "The cat sat on the ___." You would bet on "mat" or "floor", not "volcano". The model plays this game billions of times. Each time, it spreads its bets across every possible next piece of text, then the real answer is revealed.

The score is called **loss**. If the model put a big bet on the right answer, the loss is small. If it put almost nothing on the right answer, the loss is large. Training nudges the model, a tiny bit at a time, to bet better.

Perplexity turns that score into something intuitive: it is roughly how many options the model feels it is choosing between. A perplexity of 3 means it is about as unsure as picking among three equally likely words.

For leaders

Why the whole field watches one number

Pretraining loss is the single most reliable signal in model development. It falls smoothly and predictably as you add compute, data, and parameters, which is what makes scaling laws (see Scaling laws) possible. Teams decide whether a run is healthy, whether an architecture change helped, and when to stop, mostly by looking at held-out loss.

Three cautions keep that number honest:

- **Loss is only comparable with the same tokenizer and the same held-out data.** Two models with different vocabularies produce losses in different units. Use bits per byte when comparing across tokenizers.
- **Average loss hides the tokens you care about.** Most tokens are easy (the second half of a word, a closing bracket). A model can lower average loss while not improving on the hard tokens that carry a reasoning step.
- **Lower loss does not guarantee better behaviour.** It predicts capability well on average and poorly for any single task. That is why every lab pairs loss curves with downstream eval suites run on checkpoints.

Under the hood

Softmax, cross-entropy, perplexity, bits per byte

The network outputs a vector of logits $z \in \mathbb{R}^V$, one per vocabulary item. Softmax with temperature τ turns them into probabilities:

$$p_i = \frac{\exp(z_i/\tau)}{\sum_{j=1}^{V} \exp(z_j/\tau)}$$

Training uses $\tau = 1$. Temperature is a sampling knob at inference time; it does not change what the model learned.

The per-token loss is the negative log probability of the token that actually came next. Averaged over a sequence of T tokens:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}), \qquad \text{PPL} = e^{\mathcal{L}}$$

$\mathcal{L}$ in nats when using natural log. Divide by $\ln 2$ for bits.

This is cross-entropy between the data distribution and the model. It decomposes as $H(p_{\text{data}}) + \text{KL}(p_{\text{data}} \,\|\, p_\theta)$: the irreducible entropy of language plus how far the model is from the truth. Only the KL part can be trained away, which is why loss curves flatten toward a floor rather than zero.

Bits per byte removes the tokenizer from the comparison: total bits over a document divided by its length in UTF-8 bytes.

$$\mathrm{BPB} = \frac{T \cdot \mathcal{L}_{\text{nats}}}{B \cdot \ln 2}$$

T tokens, B bytes in the same text. A tokenizer with more bytes per token has fewer, harder tokens; BPB corrects for that.

python

```python
import numpy as np

def log_softmax(z):
    z = z - z.max()                      # subtract max for numerical stability
    return z - np.log(np.exp(z).sum())

def token_loss(logits, target):
    return -log_softmax(logits)[target]  # nats

logits = np.array([3.1, 2.4, 1.2, 0.3, -1.0])   # mat, floor, sofa, roof, volcano
for name, idx in [("mat", 0), ("floor", 1), ("volcano", 4)]:
    print(name, round(float(token_loss(logits, idx)), 3))
# mat 0.544   floor 1.244   volcano 4.644

# bits per byte for a document: 1,000 tokens, 4,300 bytes, mean loss 2.1 nats
print(round(1000 * 2.1 / (4300 * np.log(2)), 3))   # 0.705 bits per byte
```

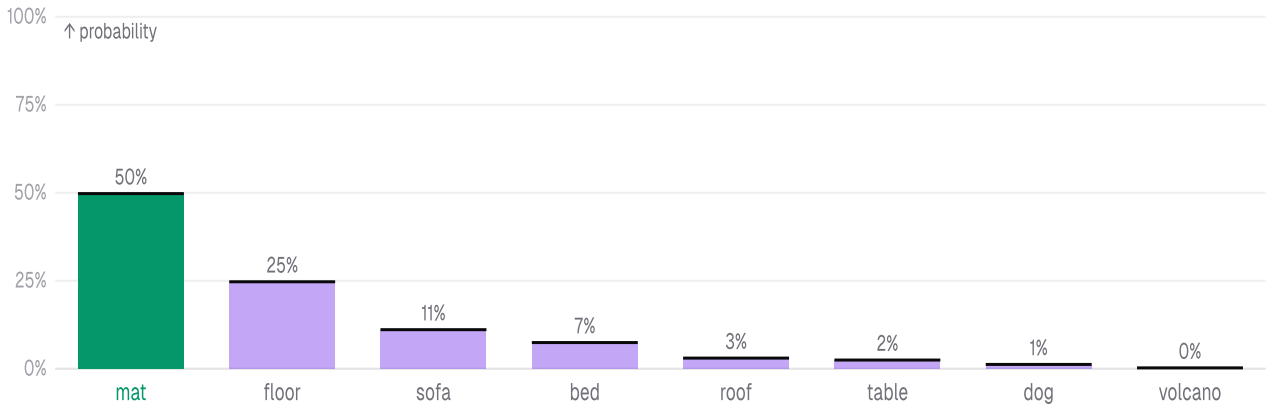
rust

```rust
fn log_softmax(z: &[f64]) -> Vec<f64> {
    let max = z.iter().cloned().fold(f64::NEG_INFINITY, f64::max); // subtract max for numerical stability
    let log_sum = z.iter().map(|x| (x - max).exp()).sum::<f64>().ln();
    z.iter().map(|x| x - max - log_sum).collect()
}

fn token_loss(logits: &[f64], target: usize) -> f64 {
    -log_softmax(logits)[target] // nats
}

fn main() {
    let logits = [3.1, 2.4, 1.2, 0.3, -1.0]; // mat, floor, sofa, roof, volcano
    for (name, idx) in [("mat", 0), ("floor", 1), ("volcano", 4)] {
        println!("{} {:.3}", name, token_loss(&logits, idx));
    }
    // mat 0.544   floor 1.244   volcano 4.644

    // bits per byte for a document: 1,000 tokens, 4,300 bytes, mean loss 2.1 nats
    println!("{:.3}", 1000.0 * 2.1 / (4300.0 * 2f64.ln())); // 0.705 bits per byte
}
```

Interactive

Next-token bets and the loss they earn (shown at its default setting)

The model's raw scores (logits) for the next token after "The cat sat on the". Change the temperature, pick which token was actually next, and watch the loss.



p(correct token)	Loss (nats)	Loss (bits)	Perplexity if every token were like this
49.8%	0.698	1.007	2.01

Entropy of the bet (bits)
2.01

Black ticks show the bet at temperature 1, the setting used in training. Temperature 1 is the distribution the model actually learned. If "volcano" were the true token, the loss would be 6.00 nats at this temperature.

Tokenization: the model never sees letters

Diagram

Snapping letters into bricks

How byte-pair encoding builds a vocabulary, on a corpus of four words.

Start: every letter is its own brick

5 × l o w

2 × l o w e r

6 × n e w e s t

3 × w i d e s t

A tiny corpus: “low” appears 5 times, “lower” twice, and so on.

Byte-pair encoding starts with every character as a separate brick. Here is a tiny corpus of four words with their counts.

Count every neighbouring pair, weighted by word count

5 × l o w

2 × l o w e r

6 × n e w **e s** t

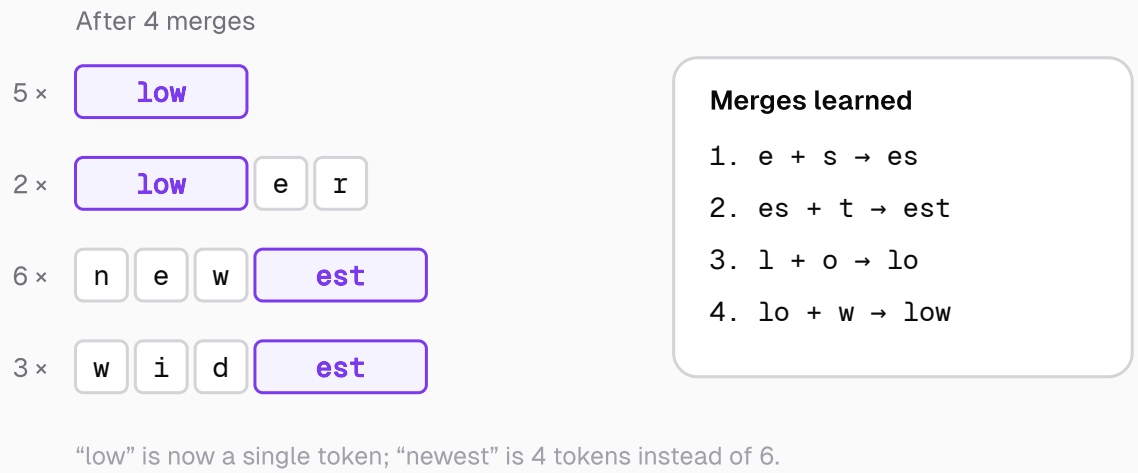
3 × w i d **e s** t

Most common pairs

e + s	9
s + t	9
w + e	8
l + o	7
o + w	7

tie at 9: the first one found wins

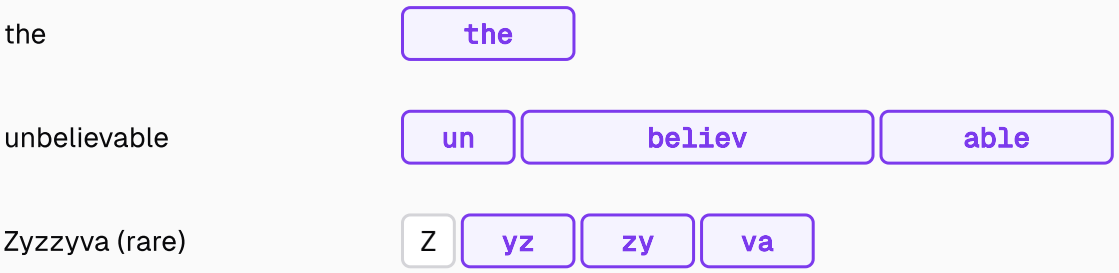
“e + s” appears 9 times (6 in “newest”, 3 in “widest”). It ties with “s + t”; the first one found wins. That pair gets snapped together into a new brick.



“low” is now a single token; “newest” is 4 tokens instead of 6.

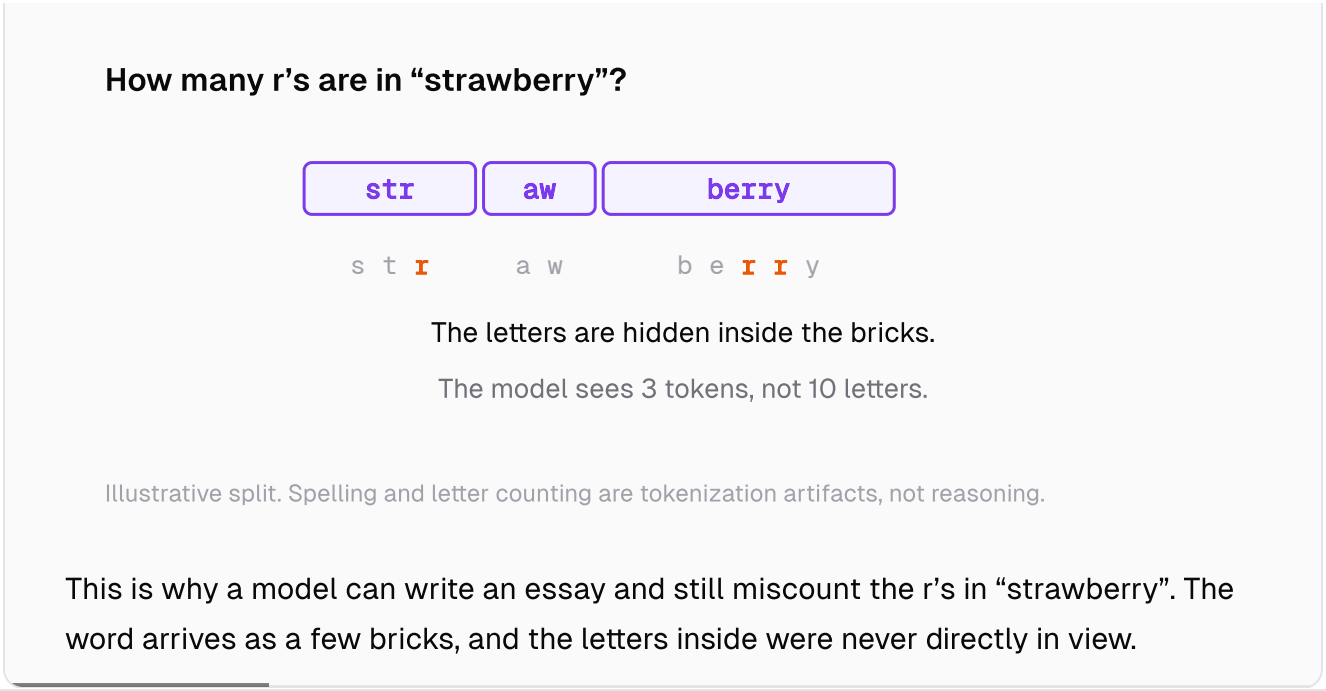
Repeat: merge the most common pair, recount, merge again. After four merges “low” is one brick and the ending “est” is shared by “newest” and “widest”. The list of merges is the tokenizer.

A real tokenizer repeats this 100,000+ times on terabytes of text



Common words become one token; rare words stay in pieces. Splits shown are illustrative.

At real scale, common words like “the” become single tokens, long words split into familiar parts, and rare names stay as several small pieces. Vocabularies today run from about 50,000 to 200,000 tokens.



This is why a model can write an essay and still miscount the r’s in “strawberry”. The word arrives as a few bricks, and the letters inside were never directly in view.

Plain words

Chopping text into reusable pieces

Models do not read letters or whole words. They read **tokens**: chunks of text chosen so that common pieces get their own symbol. "unbelievable" might become "un", "believ", "able". A common word like "the" is one token. A rare name might be five.

This is why models can write an essay but stumble when asked how many r's are in "strawberry". The word arrives as a couple of chunks, so the individual letters were never directly in view.

For leaders

A decision you live with for the life of the model

The tokenizer is fixed before pretraining and is very expensive to change later, because every embedding was learned against it. Choices that matter:

- **Vocabulary size.** GPT-2 used 50,257 tokens; Llama 3 moved to about 128,000; OpenAI's o200k tokenizer (GPT-4o) has about 200,000. Larger vocabularies compress text into fewer tokens, which means cheaper inference and more text per context window, especially for non-English languages and code. The cost is a bigger embedding table and rarer tokens that get fewer training updates.
- **Fairness across languages.** A tokenizer trained mostly on English can need several times more tokens for the same sentence in Hindi or Amharic. Users in those languages pay more per request and get less effective context. This is a product and policy question, not just an engineering one.

- **Numbers and code.** How digits are split affects arithmetic. Many modern tokenizers split numbers into single digits or groups of up to three so that place value is consistent. Whitespace handling matters a lot for code.

> **Eval lens**
>
> Character-level failures (spelling, counting letters, reversing strings) are mostly tokenization artifacts, not reasoning failures. Do not let them drive training decisions about reasoning. When comparing models with different tokenizers, report cost and context in characters or bytes, not tokens.

Under the hood

The BPE training loop

Byte-pair encoding (Sennrich et al., 2016, adapted from a 1994 compression method) is greedy: count every adjacent symbol pair weighted by word frequency, merge the most frequent pair everywhere, repeat until the vocabulary hits its target size. Modern tokenizers usually run BPE on bytes rather than characters so that any input string can be encoded, and apply a regex pre-split so merges never cross word or digit-group boundaries.

python

```python
from collections import Counter

def bpe(word_counts, n_merges):
    vocab = {tuple(w): c for w, c in word_counts.items()}     # words as tuples of symbols
    merges = []
    for _ in range(n_merges):
        pairs = Counter()
        for syms, c in vocab.items():
            for a, b in zip(syms, syms[1:]):
                pairs[(a, b)] += c                             # weight by word frequency
        if not pairs:
            break
        best = max(pairs, key=pairs.get)
        merges.append(best)
        new = {}
        for syms, c in vocab.items():
            out, i = [], 0
            while i < len(syms):
                if i + 1 < len(syms) and (syms[i], syms[i + 1]) == best:
                    out.append(syms[i] + syms[i + 1]); i += 2
                else:
                    out.append(syms[i]); i += 1
            new[tuple(out)] = c
        vocab = new
    return merges, vocab
```

```
merges, vocab = bpe({"low": 5, "lower": 2, "newest": 6, "widest": 3}, 4)
print(merges)  # [('e', 's'), ('es', 't'), ('l', 'o'), ('lo', 'w')]
```

rust

```
use std::collections::HashMap;

type Pair = (String, String);

fn bpe(word_counts: &[(&str, u64)], n_merges: usize) -> (Vec<Pair>, Vec<(Vec<String>, u64)>) {
    // words as sequences of symbols
    let mut vocab: Vec<(Vec<String>, u64)> = word_counts
        .iter()
        .map(|(w, c)| (w.chars().map(String::from).collect(), *c))
        .collect();
    let mut merges = Vec::new();
    for _ in 0..n_merges {
        // Counts kept in first-seen order, like Python's Counter, so a tie goes
        // to the pair seen first: ('e','s') beats ('s','t') at 9 each.
        let mut pairs: Vec<(Pair, u64)> = Vec::new();
        let mut index: HashMap<Pair, usize> = HashMap::new();
        for (syms, c) in &vocab {
            for w in syms.windows(2) {
                let pair = (w[0].clone(), w[1].clone());
                let i = *index.entry(pair.clone()).or_insert_with(|| {
                    pairs.push((pair, 0));
                    pairs.len() - 1
                });
                pairs[i].1 += c; // weight by word frequency
            }
        }
        if pairs.is_empty() {
            break;
        }
        // strict > keeps the FIRST maximum, as Python's max() does
        let mut best = &pairs[0];
        for p in &pairs[1..] {
            if p.1 > best.1 {
                best = p;
            }
        }
        let best = best.0.clone();
        let mut new = Vec::with_capacity(vocab.len());
        for (syms, c) in &vocab {
            let (mut out, mut i) = (Vec::new(), 0);
            while i < syms.len() {
                if i + 1 < syms.len() && syms[i] == best.0 && syms[i + 1] == best.1 {
                    out.push(format!("{}{}", syms[i], syms[i + 1]));
                    i += 2;
                } else {
                    out.push(syms[i].clone());
                    i += 1;
                }
            }
            new.push((out, *c));
        }
        merges.push(best);
        vocab = new;
    }
    (merges, vocab)
```

```
}

fn main() {
    let (merges, _vocab) = bpe(&[("low", 5), ("lower", 2), ("newest", 6), ("widest", 3)], 4);
    let shown: Vec<String> = merges.iter().map(|(a, b)| format!("('{a}', '{b}')")).collect();
    println!("[{}]", shown.join(", ")); // [('e', 's'), ('es', 't'), ('l', 'o'), ('lo', 'w')]
}
```

Interactive

Byte-pair encoding, one merge at a time (shown at its default setting)

BPE starts from single characters and repeatedly merges the most frequent adjacent pair. Step through the merges on a tiny corpus and watch common words collapse into single tokens.

Corpus, as tokens (word × count)

l o w · × 5
l o we r· × 2
l o we s t · × 2
ne we r· × 6
ne we s t · × 4
w i d e r· × 3
w i d e s t · × 2
ne w · × 3

Merges learned, in order

1. w + e → we (14)
2. n + e → ne (13)
3. r + · → r· (11)

3 merges. Average tokens per word: **4.30** (started at 5.70). Symbols in use: 12. The "·" marks end of word, so "est·" can only match at the end. The most frequent adjacent pair always merges next, so common fragments and word endings (here "we", "ne", "r·", "st·") become tokens before whole words like "newer·" and "low·".

Data: the ceiling on everything

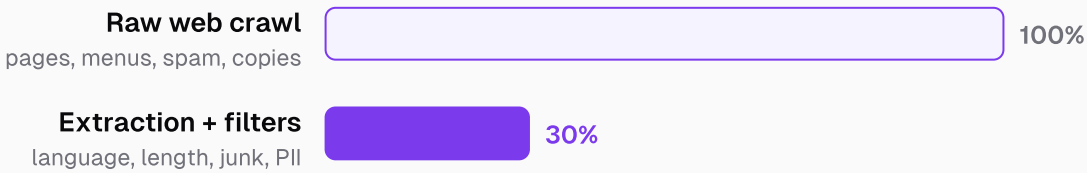
Diagram

From the whole web to a training diet

Where pretraining data comes from, and how much survives each filter.

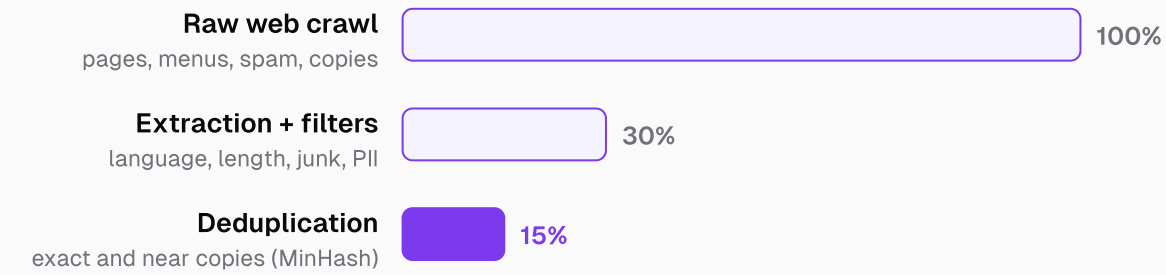
Percentages are illustrative; real pipelines vary by lab.

Start with a raw crawl of the web. Most of it is navigation menus, spam, machine-written filler, and the same pages copied thousands of times.



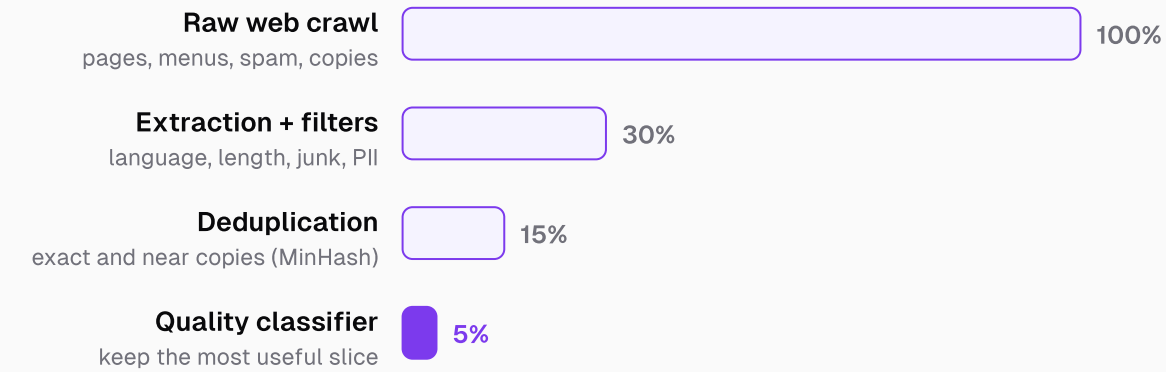
Percentages are illustrative; real pipelines vary by lab.

Extract the text, keep the target languages, drop pages that are too short or full of symbols, and strip personal information. Most of the volume goes here.



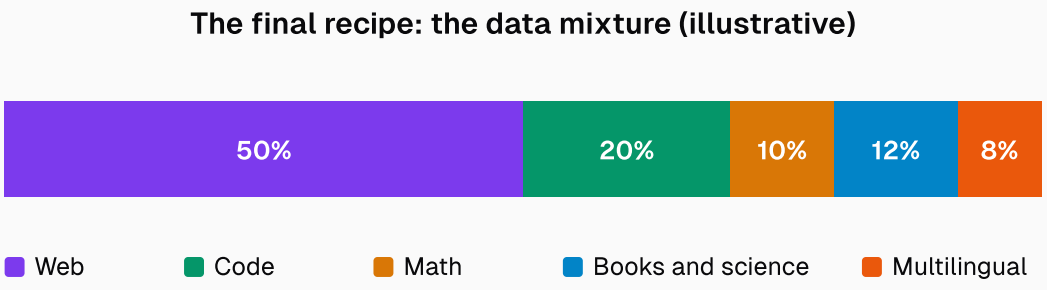
Percentages are illustrative; real pipelines vary by lab.

Remove exact and near duplicates. Repeated pages waste compute and make the model memorise instead of generalise (Lee et al., 2022).



Percentages are illustrative; real pipelines vary by lab.

A small classifier scores every document for educational value and keeps only the top slice. This one choice often moves benchmarks more than an architecture change.



Code and math are usually up-weighted: they seem to help reasoning broadly.

Last step: remove anything that overlaps the evaluation benchmarks.

Finally, decide the recipe: how many tokens of web, code, math, books, and other languages. Teams tune these weights on small models first, then decontaminate against eval sets so later scores mean something.

Plain words

You are what you read

A model trained on junk learns junk. Most of the internet is spam, boilerplate, duplicated pages, and machine-generated filler. Pretraining teams spend enormous effort throwing most of it away and deciding how much of each kind of text to keep: web pages, books, code, math, science, and many languages. That recipe is called the **data mixture**, and it is one of the most closely guarded secrets at any lab.

For leaders

Where data quality comes from

1. **Extraction and filtering.** Pull clean text out of HTML, drop pages by language, length, and heuristics (too many symbols, too few sentences), and remove personal information and unsafe content.
2. **Deduplication.** Exact and near-duplicate removal, typically with MinHash over n-grams. Lee et al. (2022) showed deduplication reduces memorization and improves models at the same compute.
3. **Quality classifiers.** A small model scores each document for "educational value" or similarity to trusted sources, and only the top slice is kept. FineWeb-Edu (Penedo et al., 2024) used labels from a large LLM to train such a classifier; DCLM (Li et al., 2024) used a fastText

classifier and showed filtering choices move benchmark scores more than many architecture changes.

4. **Mixture weights.** How many tokens of code, math, multilingual, and web text. Code and math are usually upweighted because they appear to improve reasoning broadly. Weights are tuned with small proxy models before the big run.

5. **Decontamination.** Remove documents that overlap with evaluation benchmarks so later scores mean something.

Running out of data. Repeating data is not free but is not useless either. Muennighoff et al. (2023) found that up to about four epochs of repeated data are nearly as good as fresh data, with returns falling toward zero by around sixteen epochs. High-quality text is finite, which is one reason labs invest heavily in synthetic data (see Distillation and synthetic data).

Principal asks

- How was this mixture chosen, and at what scale were the ablations run? Do the conclusions hold at the target scale?
- What is our decontamination method, and what overlap threshold did we use against each eval we report?
- How many epochs is our highest-quality data seeing? Are we in the regime where repetition starts to hurt?
- What fraction of the mixture is synthetic, and what model generated it?

Under the hood

Near-duplicate detection with MinHash

Represent each document as a set of word n-grams (shingles). The Jaccard similarity of two sets is $J(A, B) = |A \cap B| / |A \cup B|$. MinHash estimates it cheaply: for a random hash function h, $\Pr[\min h(A) = \min h(B)] = J(A, B)$. With k hash functions, the fraction of matching minimums estimates J. Locality-sensitive hashing then groups signatures into bands so that only likely duplicates are compared.

python

```python
import hashlib

def shingles(text, n=5):
    w = text.lower().split()
    return {" ".join(w[i:i + n]) for i in range(len(w) - n + 1)}

def minhash(s, k=64):
```

```
    return [min(int(hashlib.md5(f"{i}:{x}".encode()).hexdigest(), 16) for x in s) for i in range(k)]

def est_jaccard(a, b):
    return sum(x == y for x, y in zip(a, b)) / len(a)

d1 = "the quick brown fox jumps over the lazy dog near the river bank today"
d2 = "the quick brown fox jumps over the lazy dog near the river bank tonight"
print(round(est_jaccard(minhash(shingles(d1)), minhash(shingles(d2))), 2))  # about 0.8: near duplicate
```

rust

```
use std::collections::HashSet;

fn shingles(text: &str, n: usize) -> HashSet<String> {
    let w: Vec<String> = text.to_lowercase().split_whitespace().map(String::from).collect();
    if w.len() < n {
        return HashSet::new();
    }
    (0..=w.len() - n).map(|i| w[i..i + n].join(" ")).collect()
}

/// 64-bit FNV-1a plus a SplitMix64 finalizer so nearby inputs spread out.
/// It stands in for md5 so the sketch needs no crates; real code would use a
/// proper hash from a crate. Different hash, so the estimate differs slightly.
fn hash64(s: &str) -> u64 {
    let mut h: u64 = 0xcbf29ce484222325;
    for b in s.bytes() {
        h ^= b as u64;
        h = h.wrapping_mul(0x100000001b3);
    }
    h = (h ^ (h >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
    h = (h ^ (h >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
    h ^ (h >> 31)
}

fn minhash(s: &HashSet<String>, k: usize) -> Vec<u64> {
    (0..k).map(|i| s.iter().map(|x| hash64(&format!("{i}:{x}"))).min().unwrap_or(u64::MAX)).collect()
}

fn est_jaccard(a: &[u64], b: &[u64]) -> f64 {
    a.iter().zip(b).filter(|(x, y)| x == y).count() as f64 / a.len() as f64
}

fn main() {
    let d1 = "the quick brown fox jumps over the lazy dog near the river bank today";
    let d2 = "the quick brown fox jumps over the lazy dog near the river bank tonight";
    let est = est_jaccard(&minhash(&shingles(d1, 5), 64), &minhash(&shingles(d2, 5), 64));
    println!("{:.2}", est); // 0.86 here (md5 in the Python gives 0.83; true Jaccard 9/11 = 0.82): near duplicate
}
```

What loss predicts, and what it does not

For leaders

Reading a loss curve in a review

Smooth decline

Healthy. Compare the slope to the scaling-law prediction for this compute. A curve above prediction usually means a data or architecture problem.

Sudden spike

Optimisation instability. Usually recovers; if not, teams roll back to a checkpoint and skip the offending batches (see Optimisation).

Drop at the end

Learning-rate decay or annealing on high-quality data. Expected, and it can flatter a run that was mediocre before the decay.

Train falls, held-out flat

Memorization or repeated data. Check epochs on the high-quality slices and contamination.

Eval lens

Held-out loss predicts average downstream ability well, which is how the GPT-4 report (OpenAI, 2023) and the Llama 3 report (Meta, 2024) forecast final performance from small runs. For a single benchmark the link is noisy and can look like a sudden jump (see the emergence discussion in Scaling laws). Track per-domain held-out loss (code, math, each language) alongside the average, and treat benchmark scores on early checkpoints as noisy: use the interval methods in Part II before reading a trend.

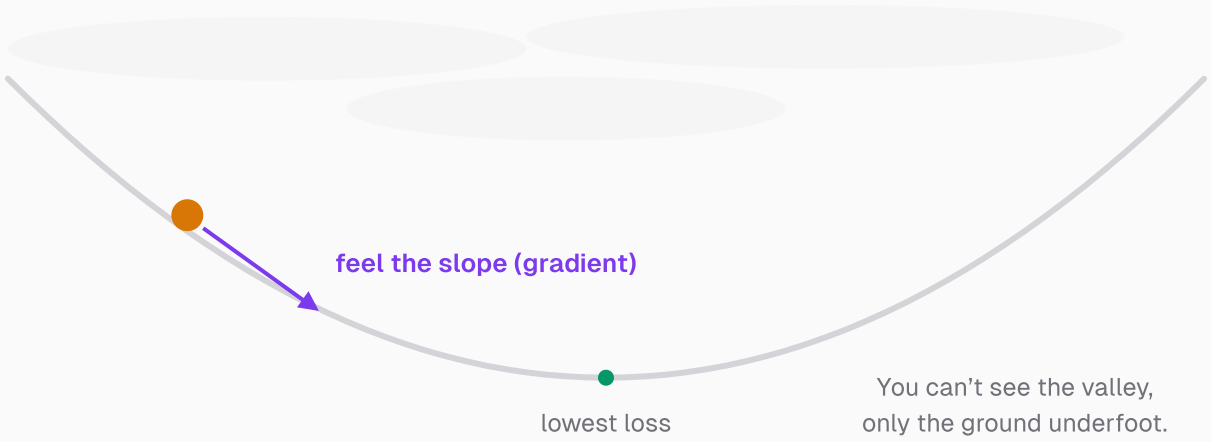
Optimisation: how the model actually learns

Training is a loop: show the model a batch of text, measure the loss, work out which direction to nudge each of billions of numbers to lower it, take a small step, repeat a few hundred thousand times. This chapter covers how big those steps are, which direction they point, and the hardware arithmetic that decides whether a run is possible at all.

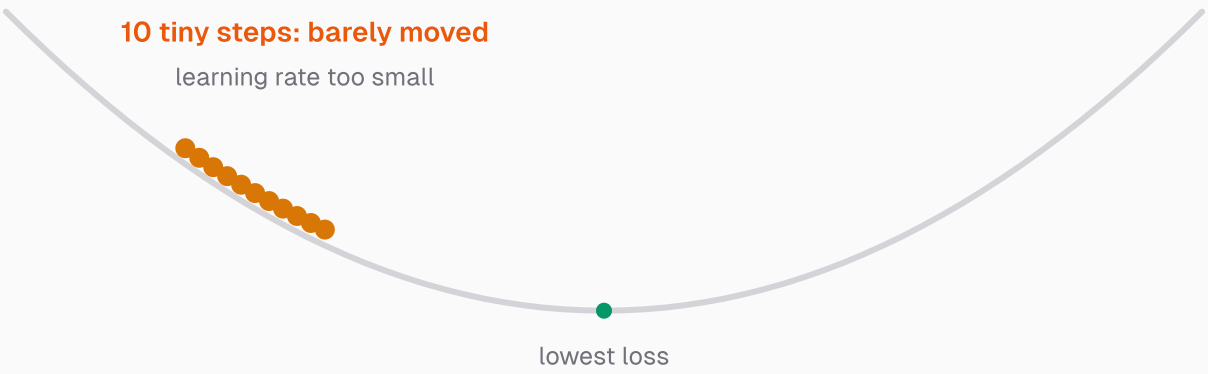
Diagram

Finding the valley in fog

Gradient descent, learning rate, momentum, and schedules as a hike down a mountain.

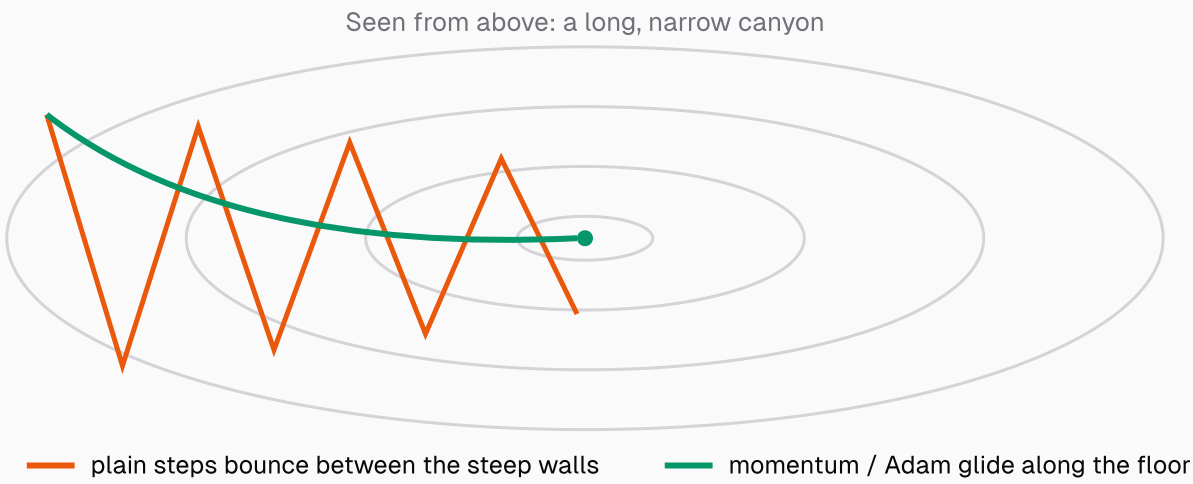


Training is walking down a hillside in fog. The height is the loss. You can’t see the valley, but you can feel which way the ground tilts: that tilt is the **gradient**. Each step goes downhill.

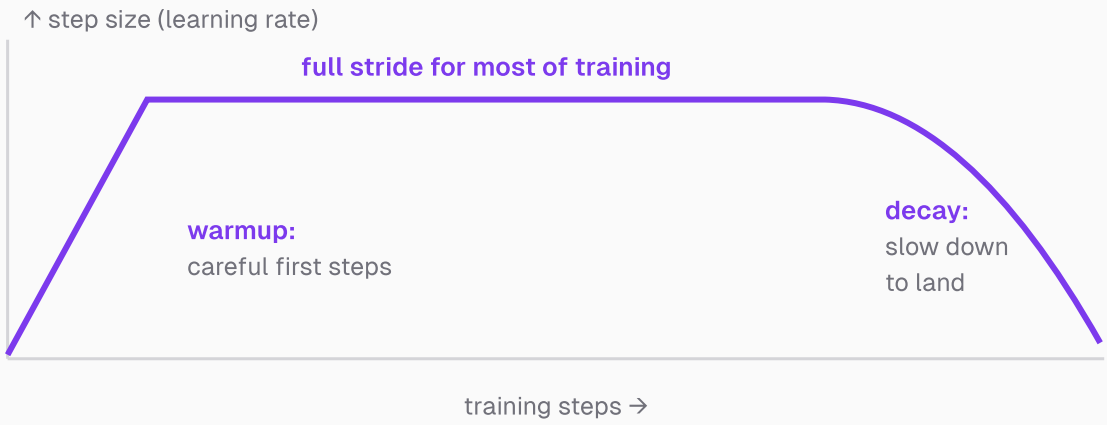


Take strides that are too short and you are safe but slow: after ten steps you have hardly moved. That is a learning rate set too low, and it wastes compute.

Take strides that are too long and you leap clean over the valley and land higher up the other side, then higher again. The loss explodes. That is a learning rate set too high.



Real loss surfaces are long, narrow canyons. Plain steps bounce between the steep walls and crawl along the floor. **Momentum** builds speed in the direction that stays consistent; **Adam** also sizes each direction's stride separately. That is why nearly every LLM trains with AdamW.



So stride length changes over the run: tiny steps while the model is random and its gradients are wild (**warmup**), full stride for most of training, then shorter steps to settle into the bottom (**decay**).

Gradient descent and the learning rate

Plain words **Walking downhill in fog**

Picture standing on a hillside in thick fog, trying to reach the valley floor. You cannot see the valley, but you can feel which way the ground slopes under your feet. So you take a step downhill, feel again, step again. That is **gradient descent**. The slope is the **gradient**, and the length of your stride is the **learning rate**.

Strides too short and you take forever. Strides too long and you leap across the valley and up the other side, maybe further up than where you started. Most of the craft of training is choosing stride length well and changing it over time.

For leaders

Why hyperparameters are a budget question

The learning rate is the most important hyperparameter in a training run. A bad choice wastes the whole run, and at frontier scale nobody can afford to sweep it at full size. So labs tune on small models and transfer the settings using scaling rules (for example, the muP parameterization from Yang et al., 2022, which keeps the best learning rate roughly stable as width grows). Ask how the hyperparameters were chosen and at what scale they were validated.

The loss surface of a real network is far from a smooth bowl. It has long narrow valleys, flat plateaus, and cliffs. Optimisers like Adam exist because plain gradient descent handles those shapes badly, as the widget shows: a step size that is safe for the steep direction is painfully slow along the shallow one.

Under the hood

The update rules

Plain gradient descent on parameters θ with learning rate η:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

In practice the gradient is estimated on a mini-batch, so this is stochastic gradient descent (SGD).

On a quadratic with curvature λ along some direction, each step multiplies the error in that direction by $1 - \eta\lambda$. It shrinks only if $|1 - \eta\lambda| < 1$, so the step must satisfy $\eta < 2/\lambda_{\max}$. In the widget $\lambda_{\max} = 10$, so anything above 0.2 diverges. Meanwhile the shallow direction ($\lambda = 1$) shrinks by only $1 - \eta$ per step. The ratio $\lambda_{\max}/\lambda_{\min}$, the condition number, sets how slow plain descent must be.

Momentum keeps a running average of past gradients, so consistent directions accelerate and oscillating ones cancel: $v_{t+1} = \beta v_t + \nabla\mathcal{L}$, $\theta_{t+1} = \theta_t - \eta v_{t+1}$.

Adam (Kingma and Ba, 2015) also tracks the average squared gradient per parameter and divides by its square root, so every parameter takes steps of roughly the same size regardless of its gradient scale:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \right)$$

$\hat{m}, \hat{v}$ are bias-corrected: divide by $1 - \beta^t$. The $\lambda\theta$ term is decoupled weight decay, the "W" in AdamW (Loshchilov and Hutter, 2019). Typical LLM settings: $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1.

python

```python
import numpy as np

def adamw_step(theta, g, m, v, t, lr=3e-4, b1=0.9, b2=0.95, eps=1e-8, wd=0.1):
    m = b1 * m + (1 - b1) * g                # first moment: direction
    v = b2 * v + (1 - b2) * g * g            # second moment: per-parameter scale
    m_hat = m / (1 - b1 ** t)                # bias correction for the zero init
    v_hat = v / (1 - b2 ** t)
    theta = theta - lr * (m_hat / (np.sqrt(v_hat) + eps) + wd * theta)   # decoupled decay
    return theta, m, v

theta, m, v = np.array([1.0, -2.0]), np.zeros(2), np.zeros(2)
theta, m, v = adamw_step(theta, np.array([0.5, 40.0]), m, v, t=1)
print(theta)   # both coordinates move by about lr, despite an 80x gradient difference
```

rust

```rust
/// AdamW hyperparameters; Default gives the Python's keyword defaults.
struct AdamW {
    lr: f64,
    b1: f64,
    b2: f64,
    eps: f64,
    wd: f64,
}

impl Default for AdamW {
    fn default() -> Self {
        AdamW { lr: 3e-4, b1: 0.9, b2: 0.95, eps: 1e-8, wd: 0.1 }
    }
}

/// Updates theta, m and v in place, one coordinate at a time.
fn adamw_step(theta: &mut [f64], g: &[f64], m: &mut [f64], v: &mut [f64], t: i32, h: &AdamW) {
    for i in 0..theta.len() {
        m[i] = h.b1 * m[i] + (1.0 - h.b1) * g[i]; // first moment: direction
        v[i] = h.b2 * v[i] + (1.0 - h.b2) * g[i] * g[i]; // second moment: per-parameter scale
        let m_hat = m[i] / (1.0 - h.b1.powi(t)); // bias correction for the zero init
        let v_hat = v[i] / (1.0 - h.b2.powi(t));
        theta[i] -= h.lr * (m_hat / (v_hat.sqrt() + h.eps) + h.wd * theta[i]); // decoupled decay
```

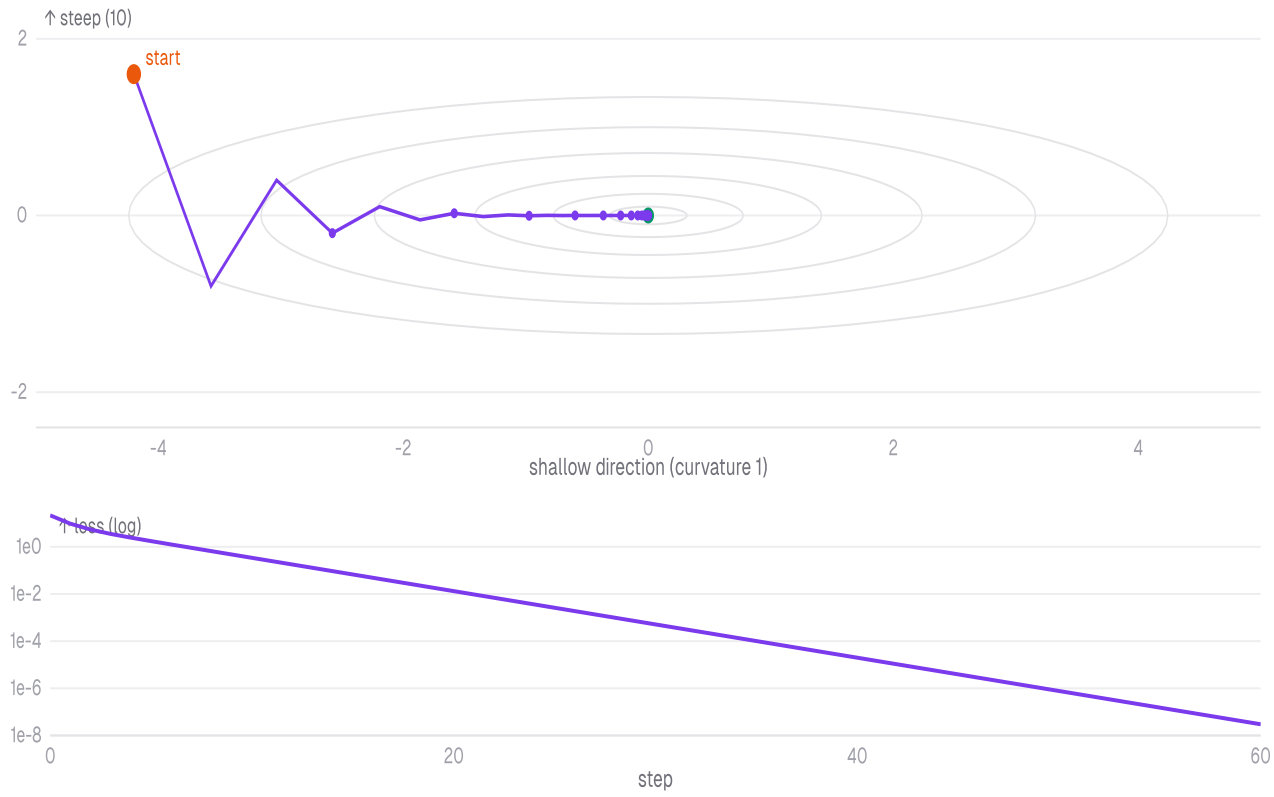
```rust
    }
}

fn main() {
    let (mut theta, mut m, mut v) = ([1.0, -2.0], [0.0; 2], [0.0; 2]);
    adamw_step(&mut theta, &[0.5, 40.0], &mut m, &mut v, 1, &AdamW::default());
    // printed the way numpy prints the array
    let shown: Vec<String> = theta.iter().map(|x| format!("{x:8.5}")).collect();
    println!("[{}]", shown.join(" ")); // both coordinates move by about lr, despite an 80x gradient difference
}
```

Newer optimisers aim to beat AdamW per token. Muon (2024), which orthogonalises the update for matrix parameters, was used at scale in Moonshot's Kimi K2 (2025). AdamW remains the default reference.

Interactive

Step size on a long, narrow valley (shown at its default setting)

The ellipses are contours of a loss surface that is ten times steeper in one direction than the other, like most real ones. Try the presets, then move the sliders.



Converging. Loss after 60 steps: 2.99e-8.

Schedules, batch size, and stability

Plain words

Start gently, stride out, then slow down to land

At the very start the model is random and its gradients are wild, so training begins with tiny steps and ramps up (**warmup**). Then it runs at full stride for most of training. Near the end it shortens its stride to settle into the bottom of the valley (**decay**). The batch is how many examples it looks at before each step: a bigger batch gives a steadier sense of direction, up to a point.

For leaders

The knobs that decide wall-clock time and risk

- **Schedules.** Cosine decay has been the default since GPT-3. Warmup-stable-decay (WSD) schedules, popularised by MiniCPM (Hu et al., 2024), hold the learning rate flat and decay only at the end. That lets a team branch a decayed checkpoint off at any point, which makes it cheap to continue training or to run scaling studies without restarting.
- **Batch size.** Bigger batches let you use more GPUs in parallel, which shortens wall-clock time. Below the **critical batch size**, doubling the batch roughly halves the number of steps needed. Above it, you pay for more compute with little speedup (McCandlish et al., 2018). The critical batch size grows as loss falls, so batch size is often increased during a run. Frontier runs use batches of millions of tokens per step.
- **Loss spikes.** Large runs occasionally see the loss jump. PaLM (Chowdhery et al., 2022) handled spikes by rolling back to a checkpoint about 100 steps earlier and skipping a few hundred batches. Mitigations include gradient clipping (cap the gradient norm, commonly at 1.0), lower learning rates, normalising queries and keys in attention, and careful initialisation.

Principal asks

- How many spikes did this run have, and how much compute was lost to rollbacks?
- Were hyperparameters tuned at this scale or transferred? What is the evidence the transfer held?
- Which results are from a checkpoint before or after the final learning-rate decay? A pre-decay checkpoint understates the model.

Under the hood

Cosine and WSD, written out

$$\eta_{\cos}(t) = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi(t - t_w)}{T - t_w}\right) \quad \text{for } t > t_w$$

Linear warmup over t_w steps first. WSD replaces the cosine with a constant $\eta_{\max}$ and a short final decay, often 10 to 20 percent of training.

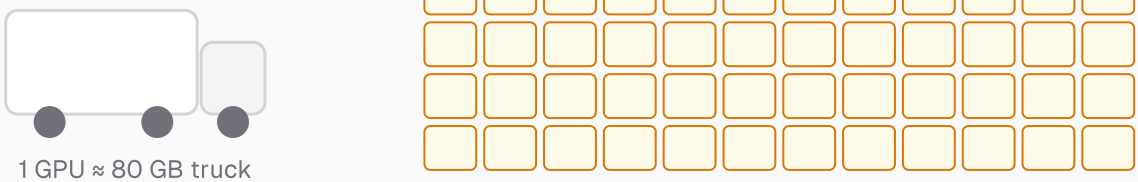
Gradient clipping rescales $g \leftarrow g \cdot \min(1, c/\|g\|)$. The gradient noise scale $B_{\text{noise}} = \operatorname{tr}(\Sigma)/\|G\|^2$, the ratio of per-example gradient variance to the squared true gradient, estimates the critical batch size.

Precision, memory, and parallelism

Diagram

Moving a house too big for one truck

Why training needs thousands of GPUs, and the five ways to split the work.

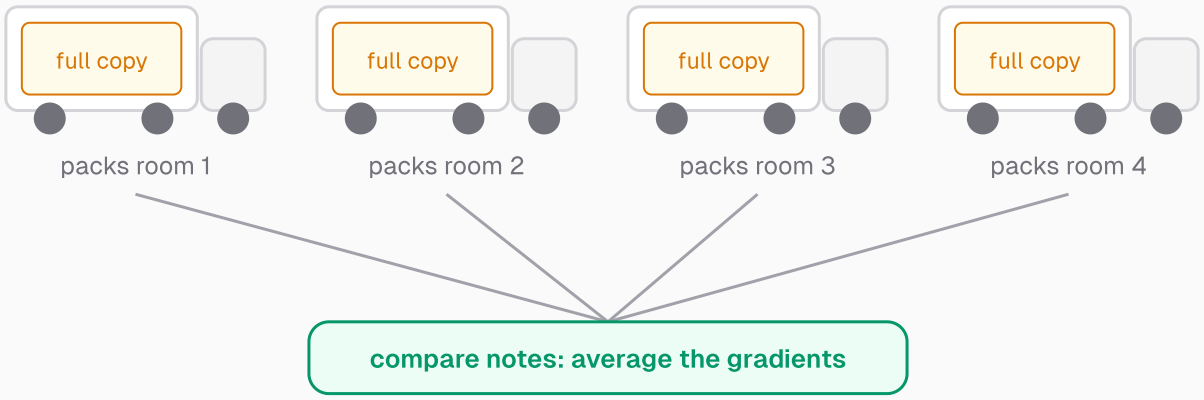


70B model + Adam ≈ 1,120 GB of numbers

weights, gradients, optimiser state

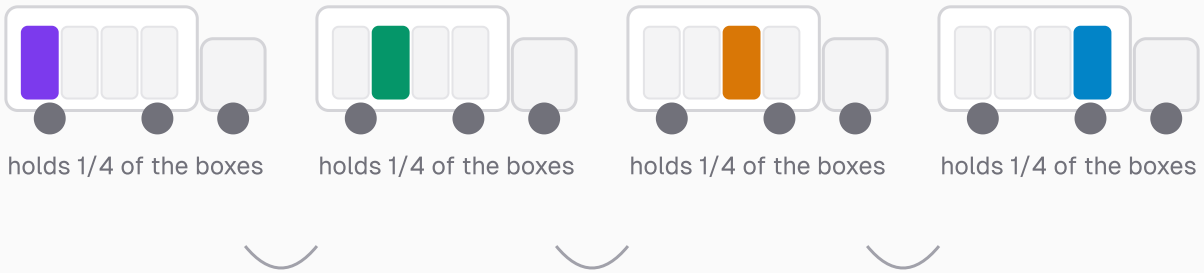
≈ 14 trucks just to hold the boxes

A 70-billion-parameter model trained with Adam needs about 16 bytes per parameter: roughly 1,120 GB. One GPU holds 80 GB. Before any training happens, the boxes alone need 14 trucks.



Data parallel

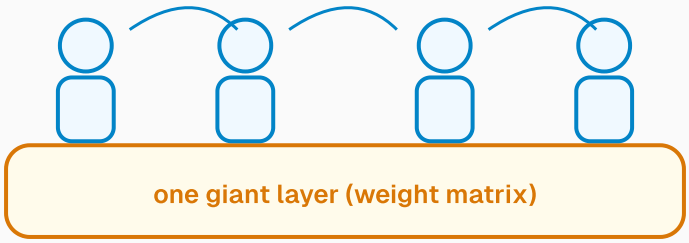
Data parallel: several identical crews, each with a full copy of the model, each packing a different room (a different slice of the batch). After every step they compare notes and average their gradients so all copies stay identical. Simple, but every truck carries everything.



ZeRO / FSDP: no duplicate boxes

each GPU keeps a shard and borrows the rest just in time

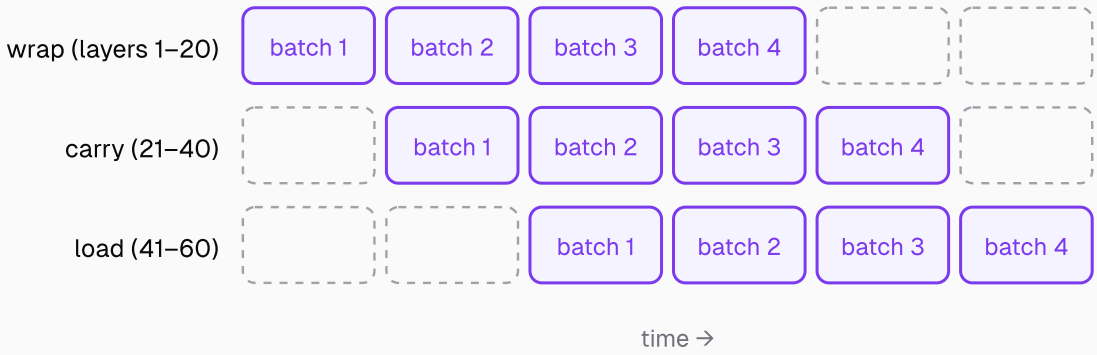
ZeRO and FSDP fix the duplication: each crew keeps only its quarter of the weights, gradients, and optimiser state, and borrows the other pieces for a moment when it needs them. Memory per GPU drops roughly in proportion to the number of crews.



Tensor parallel: four GPUs carry one layer together

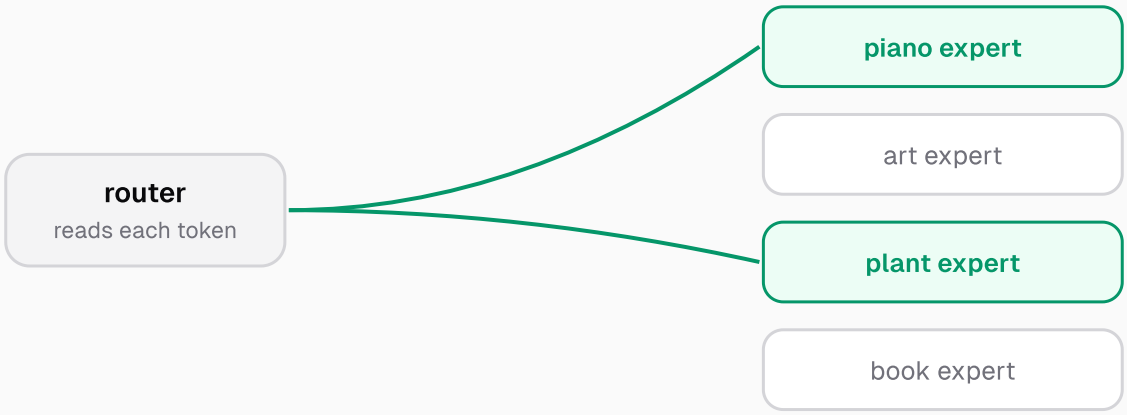
they talk constantly, so they need very fast links inside one server

Tensor parallel: some pieces are too heavy for one mover. Four movers carry one couch together, which here means splitting a single weight matrix across GPUs. They must coordinate every moment, so this only works over very fast links inside one server.



Pipeline parallel: an assembly line. Dashed boxes are idle “bubbles”.

Pipeline parallel: an assembly line. Different GPUs hold different layers, and small batches flow through one after another. The cost is the idle “bubbles” at the start and end while the line fills and drains.



Mixture of experts: each token visits only a few experts, on different GPUs.

Expert parallel, for mixture-of-experts models: a dispatcher sends each item only to the specialists it needs, and each specialist lives on its own GPU. The model can hold a lot of knowledge while each token only pays for a few experts.

Plain words

The model does not fit on one chip

A large model's numbers, plus the extra bookkeeping the optimiser keeps, take far more memory than any single GPU has. So training splits the work across thousands of GPUs: some split the data, some split each layer, some split the stack of layers. Teams also store numbers with fewer digits (lower precision) to save memory and go faster, carefully, because too few digits make training unstable.

For leaders

The arithmetic behind a GPU request

The rule of thumb to carry: mixed-precision training with Adam needs about **16 bytes per parameter** before activations. A 70-billion-parameter model therefore needs about 1.1 TB just for weights, gradients, and optimiser state, which is 14 H100s' worth of memory doing nothing but holding numbers. Sharding (ZeRO, FSDP) spreads that across GPUs; activations, which grow with sequence length and batch size, are handled with recomputation (activation checkpointing) and more parallelism.

Data parallel

Every GPU has a full copy and its own slice of the batch. Simple, but memory-bound. ZeRO / FSDP shard the copies away.

Tensor parallel

Split each weight matrix across GPUs in one server (Megatron-LM, Shoeybi et al., 2019). Needs very fast interconnect.

Pipeline parallel

Different layers on different GPUs, with micro-batches flowing through. Idle "bubbles" are the cost.

Expert parallel

For mixture-of-experts models, different experts live on different GPUs and tokens are routed between them.

Model FLOPs utilisation (MFU) is the share of the hardware's peak arithmetic that goes into useful training math. PaLM reported 46% (Chowdhery et al., 2022); Meta reported 38 to 43% MFU for Llama 3 405B on 8,000 to 16,000 H100s (2024). At these scales, hardware failures are routine, so checkpointing and fast restarts matter as much as raw speed.

Lower precision. bf16 is standard. DeepSeek-V3 (2024) trained a 671B-parameter mixture-of-experts model largely in FP8, which cuts memory and roughly doubles throughput on supporting hardware, at the price of careful scaling to avoid overflow.

Eval lens

Infrastructure choices change results in ways that look like model changes. Different precision, batch size, or parallel layout can shift evaluation numbers slightly through nondeterminism. Before attributing a small benchmark delta to a training change, rerun the eval on the same checkpoint and confirm the run-to-run noise floor.

Under the hood

Memory and MFU formulas

With Ψ parameters, N_d data-parallel GPUs, and K bytes of optimiser state per parameter ($K = 12$ for fp32 master weights plus Adam's two moments), the per-GPU model-state memory under ZeRO (Rajbhandari et al., 2020) is:

$$\underbrace{(2+2+K)\Psi}_{\text{stage 0}},\quad \underbrace{4\Psi+\tfrac{K\Psi}{N_d}}_{\text{stage 1}},\quad \underbrace{2\Psi+\tfrac{(2+K)\Psi}{N_d}}_{\text{stage 2}},\quad \underbrace{\tfrac{(4+K)\Psi}{N_d}}_{\text{stage 3}}$$

2 bytes each for bf16 weights and gradients. Stage 3 is what PyTorch FSDP calls full sharding.

A dense transformer spends about 6Ψ FLOPs per training token (2 forward, 4 backward), ignoring the attention term that grows with context length. So:

$$\text{MFU} = \frac{6\Psi \cdot \text{tokens per second}}{N_{\text{GPU}} \cdot \text{peak FLOP/s}}$$

python

```python
def model_state_gb(params, n_gpu, stage, k=12):
    p = params
    per = {0: (4 + k) * p,
           1: 4 * p + k * p / n_gpu,
           2: 2 * p + (2 + k) * p / n_gpu,
           3: (4 + k) * p / n_gpu}[stage]
    return per / 1e9

for s in range(4):
    print(s, round(model_state_gb(70e9, 8, s), 1), "GB per GPU")
# 0 1120.0   1 385.0   2 262.5   3 140.0   -> even full sharding over 8 GPUs exceeds 80 GB

def mfu(params, tok_per_s, n_gpu, peak=989e12):          # H100 dense bf16 peak
    return 6 * params * tok_per_s / (n_gpu * peak)
print(round(mfu(405e9, 2.6e6, 16000), 2))                # about 0.4
```

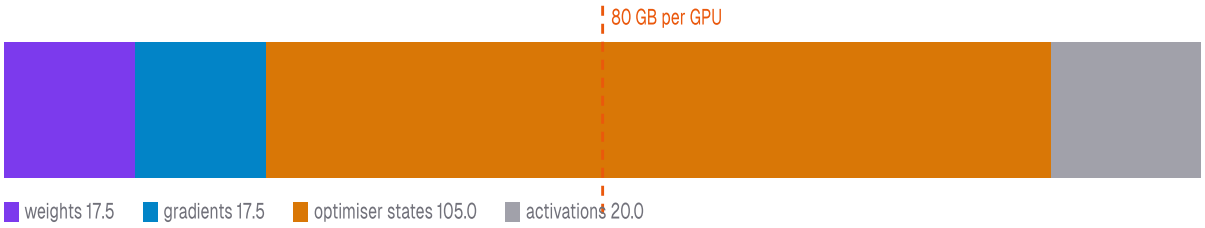
rust

```rust
/// Bytes of weights, gradients and optimizer state per GPU, in GB.
/// k = optimizer bytes per parameter (12 in the Python's default).
fn model_state_gb(params: f64, n_gpu: f64, stage: u8, k: f64) -> f64 {
    let p = params;
    let per = match stage {
        0 => (4.0 + k) * p,
        1 => 4.0 * p + k * p / n_gpu,
        2 => 2.0 * p + (2.0 + k) * p / n_gpu,
        3 => (4.0 + k) * p / n_gpu,
        _ => panic!("ZeRO stage must be 0 to 3"),
    };
    per / 1e9
}

/// peak defaults to 989e12 in the Python: H100 dense bf16 peak
fn mfu(params: f64, tok_per_s: f64, n_gpu: f64, peak: f64) -> f64 {
    6.0 * params * tok_per_s / (n_gpu * peak)
```

```rust
}

/// Python's round(x, digits); printed with {:?} it drops trailing zeros the same way.
fn round(x: f64, digits: i32) -> f64 {
    let scale = 10f64.powi(digits);
    (x * scale).round() / scale
}

fn main() {
    for s in 0..4 {
        println!("{} {:.1} GB per GPU", s, model_state_gb(70e9, 8.0, s, 12.0));
    }
    // 0 1120.0   1 385.0   2 262.5   3 140.0   -> even full sharding over 8 GPUs exceeds 80 GB

    println!("{:?}", round(mfu(405e9, 2.6e6, 16000.0, 989e12), 2)); // about 0.4
}
```

Interactive **Training memory per GPU** (shown at its default setting)

Model states only (weights, gradients, optimiser), plus an activation allowance you set. Try a 70B model on 8 GPUs at each ZeRO stage.



Per GPU: **160.0 GB** does not fit. Model states across the whole group: 1.12 TB (16 bytes per parameter). At stage 3 every piece of model state is split 8 ways; the cost is extra communication to gather weights for each layer.

Part III · Chapter 20

Scaling laws: predicting the result before you pay for it

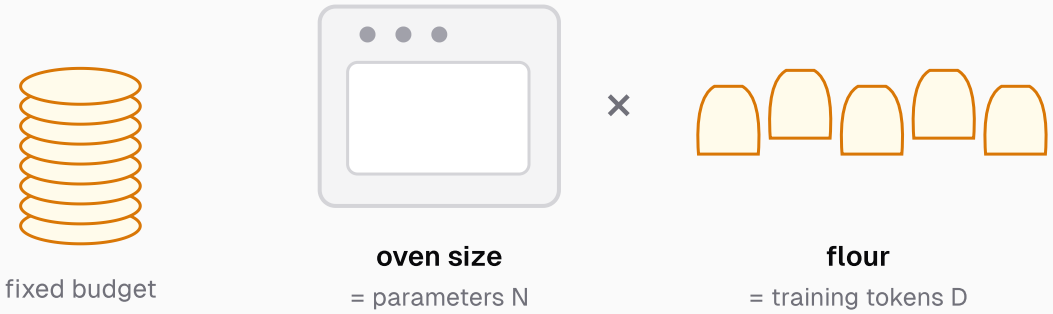
Scaling laws are the reason frontier labs can spend hundreds of millions of dollars on a single run with some confidence. Loss falls along smooth, predictable curves as you add parameters, data, and compute, so you can fit those curves on small models and extrapolate to the big one.

Diagram

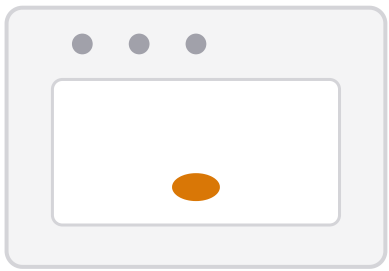
Big oven or more flour?

Compute-optimal scaling as a bakery splitting a fixed budget.

compute C ≈ 6 × N × D

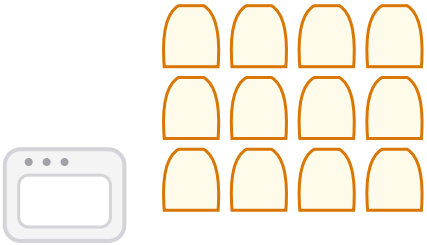


Training compute is a fixed budget. You can spend it on a bigger oven (more parameters) or more flour (more training tokens). Compute is roughly 6 × parameters × tokens, so every extra unit of one costs you some of the other.



Huge oven, one sack of flour

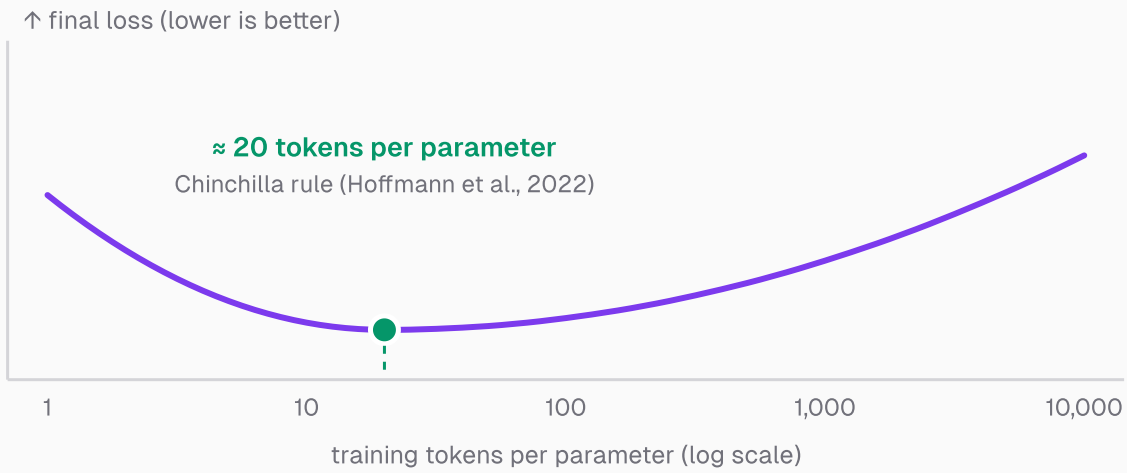
wasted capacity



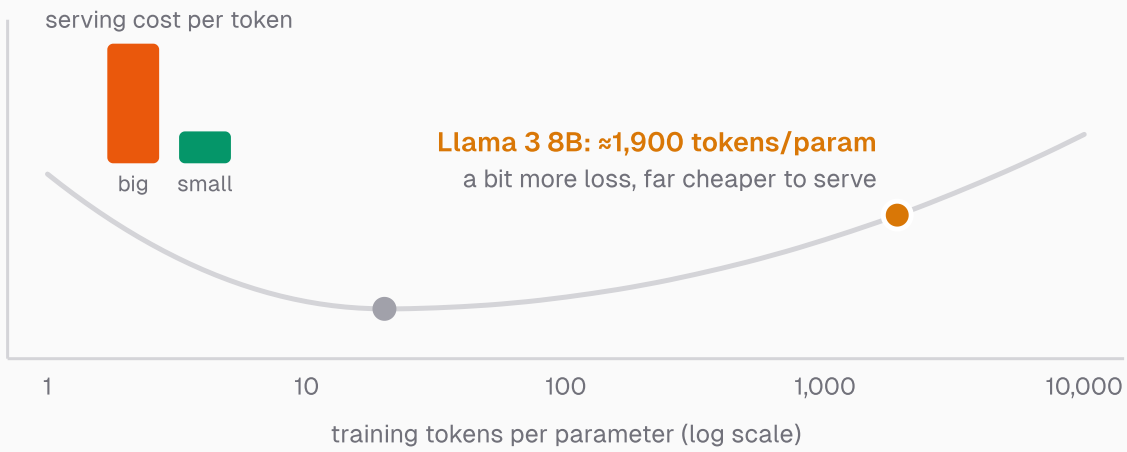
Tiny oven, mountain of flour

can’t absorb it all

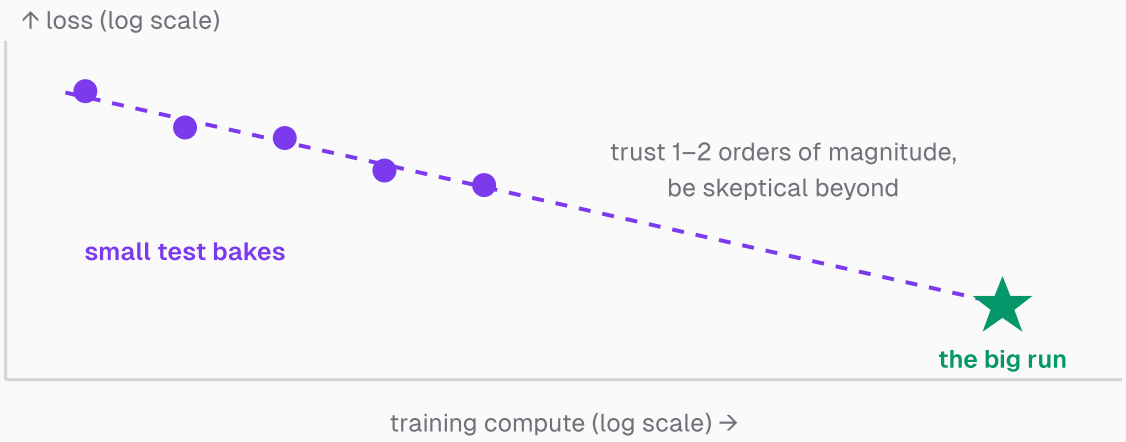
Both extremes waste money. A giant model trained on too little data never fills its capacity. A tiny model fed a mountain of data cannot hold what it reads. For any budget there is a best split.



Plot final loss against how you split the budget and you get a valley. Hoffmann et al. (2022) put the bottom near **20 training tokens per parameter**: Chinchilla, 70B parameters on 1.4T tokens, beat the 280B Gopher on the same compute. (Their parametric fit puts the bottom at more tokens per parameter; either way the valley floor is broad.)



A bakery sells loaves for years, and every loaf costs oven time. Models are the same: inference over a model's life can dwarf training. So labs over-train smaller models far past 20:1. Llama 3 8B saw about 15 trillion tokens, around 1,900 per parameter, for a little more loss and much cheaper serving.



The payoff: bake a series of small test batches, plot loss against compute on log scales, and the points fall on a line. Extend the line to predict the expensive run before paying for it. The further you extrapolate, the less you should trust it.

Plain words

Bigger brain or more books?

You have a fixed amount of money to train a student. You can buy a bigger brain (more parameters) or more books (more training data). A huge brain that reads only a few books wastes its capacity. A tiny brain that reads every book cannot hold it all. There is a best balance for any budget.

Scaling laws are the recipe for that balance. Researchers train many small models, plot how good each one gets, see a clean pattern, and use it to predict how a much bigger model will do.

For leaders

Three generations of the recipe

1. **Kaplan et al. (2020), OpenAI.** Loss follows power laws in parameters, data, and compute over many orders of magnitude. Their fits suggested growing model size much faster than data as budgets rise, which shaped GPT-3 (175B parameters, about 300B tokens).

2. **Hoffmann et al. (2022), DeepMind, "Chinchilla".** With more careful experiments across scales, the optimum shifts: parameters and tokens should grow in equal proportion with compute, at roughly **20 training tokens per parameter**. Chinchilla (70B parameters, 1.4T tokens) outperformed the 280B-parameter Gopher on the same compute.

3. **Inference-aware training (2023 onward).** Chinchilla optimises training cost only. A model served to millions of users spends far more compute on inference over its life, so a smaller model trained on far more data is cheaper overall (Sardana et al., 2023). Llama 3 8B was

trained on about 15 trillion tokens, close to 1,900 tokens per parameter, almost a hundred times the Chinchilla ratio, and kept improving.

The current frontier adds two more axes. **Mixture-of-experts** models activate only a fraction of their parameters per token (DeepSeek-V3: 671B total, 37B active, 2024), which decouples knowledge capacity from per-token cost. **Test-time compute** lets a model think longer at inference, and performance on reasoning tasks scales with that too (see Inference and test-time compute). Pretraining compute is no longer the only dial.

Principal asks

- What range of scales was the law fitted on, and how far are we extrapolating? More than one or two orders of magnitude deserves skepticism.
- Are we optimising for training cost or lifetime cost including inference? What token-per-parameter ratio does that imply?
- Does the law predict the downstream metric we care about, or only loss? How was the loss-to-metric link validated?
- Is high-quality data the binding constraint? How many epochs does the plan imply for our best data?

Under the hood

The Chinchilla loss surface

Hoffmann et al. fit a parametric form for final loss as a function of parameters N and training tokens D:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

E is the irreducible loss of the data. The other terms are the penalty for a finite model and for finite data. Their fit (approach 3): $E = 1.69,\ A = 406.4,\ B = 410.7,\ \alpha = 0.34,\ \beta = 0.28$. A 2024 replication by Besiroglu et al. argued these constants were mis-estimated and refit them; treat exact values as approximate.

Minimising L subject to a compute budget $C \approx 6ND$ gives optimal allocations that scale as power laws in C. Hoffmann et al. found exponents near 0.5 for both, so doubling compute should increase both N and D by about $\sqrt{2}$. Kaplan et al. had found $N_{\text{opt}} \propto C^{0.73}$. Hoffmann et al. attributed the gap to Kaplan's fixed learning-rate schedule, but later work (Porian et al., 2024; Pearce and Song, 2024)

traced it mainly to counting only non-embedding parameters, ignoring last-layer compute, warmup length, and hyperparameters not retuned per scale. Porian et al. found careful learning-rate decay was not needed to recover the Chinchilla result.

With the 20-tokens-per-parameter rule, $C = 6 \cdot N \cdot 20N = 120N^2$, so $N = \sqrt{C/120}$.

python

```python
import math

E, A, B, a, b = 1.69, 406.4, 410.7, 0.34, 0.28        # Hoffmann et al. 2022 fit

def loss(N, D):
    return E + A / N**a + B / D**b

def allocate(C, tokens_per_param=20):
    N = math.sqrt(C / (6 * tokens_per_param))
    return N, tokens_per_param * N

C = 5.76e23                                           # roughly Chinchilla's own budget
N, D = allocate(C)
print(f"{N/1e9:.0f}B params, {D/1e12:.2f}T tokens")  # 69B params, 1.39T tokens

# over-train a smaller model on the same compute
for r in (20, 200, 2000):
    N, D = allocate(C, r)
    print(r, f"{N/1e9:.1f}B", round(loss(N, D), 3))
# 20 69.3B 1.937   200 21.9B 1.932   2000 6.9B 1.959
```

rust

```rust
// Hoffmann et al. 2022 fit (alpha and beta are the Python's a and b)
const E: f64 = 1.69;
const A: f64 = 406.4;
const B: f64 = 410.7;
const ALPHA: f64 = 0.34;
const BETA: f64 = 0.28;

fn loss(n: f64, d: f64) -> f64 {
    E + A / n.powf(ALPHA) + B / d.powf(BETA)
}

/// tokens_per_param is 20 by default in the Python
fn allocate(c: f64, tokens_per_param: f64) -> (f64, f64) {
    let n = (c / (6.0 * tokens_per_param)).sqrt();
    (n, tokens_per_param * n)
}

fn main() {
    let c = 5.76e23; // roughly Chinchilla's own budget
    let (n, d) = allocate(c, 20.0);
    println!("{:.0}B params, {:.2}T tokens", n / 1e9, d / 1e12); // 69B params, 1.39T tokens

    // over-train a smaller model on the same compute
    for r in [20, 200, 2000] {
        let (n, d) = allocate(c, r as f64);
        println!("{} {:.1}B {:.3}", r, n / 1e9, loss(n, d));
    }
```

```
    // 20 69.3B 1.937   200 21.9B 1.932   2000 6.9B 1.959
}
```

Two things stand out. First, under this parametric fit the loss at 200 tokens per parameter is slightly *lower* than at 20: the fitted surface puts the optimum near 90 tokens per parameter at this budget, not 20. The 20:1 rule came from the paper's other two estimation methods, and the mismatch between them is one of the issues Besiroglu et al. (2024) raised. Second, the surface is very flat near the optimum. Going to 2,000 tokens per parameter makes the model ten times smaller, and roughly ten times cheaper per token to serve, for a loss increase of about 0.02 at the same training compute. That flatness is the economic case for over-training.

Data-constrained scaling. Muennighoff et al. (2023) extended the law to repeated data: each repeated epoch is worth less than fresh data, with about four epochs nearly free and value decaying toward zero by roughly sixteen.

Interactive

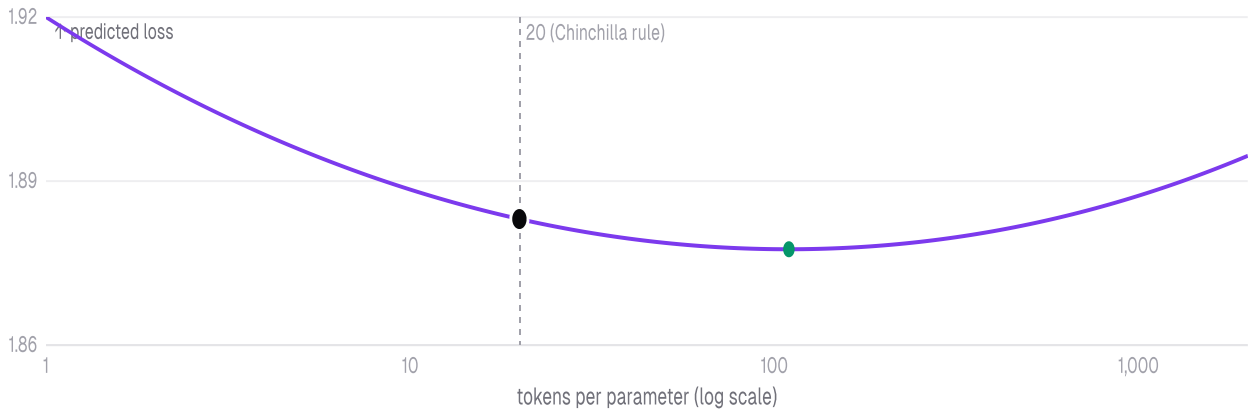
Compute budget planner (shown at its default setting)

Pick a training budget in FLOPs and a token-per-parameter ratio. The planner splits the budget into model size and data with C ≈ 6ND, estimates wall-clock time, and shows how much loss you give up by over-training a smaller model.

Parameters N	Training tokens D	Wall-clock days	GPU-hours
162.5B	3.2T	11	2.2M

Predicted loss vs best split

+0.006



At 3.16e24 FLOPs the fitted law is minimised near **110 tokens per parameter** (green). Your split trains a 162.5B-parameter model on 3.2T tokens in about 11 days on 8,192 GPUs. The fitted optimum differs from the 20:1 rule because the parametric fit and the rule of thumb came from different analyses in the same paper.

Emergence: sudden jumps or a trick of the metric?

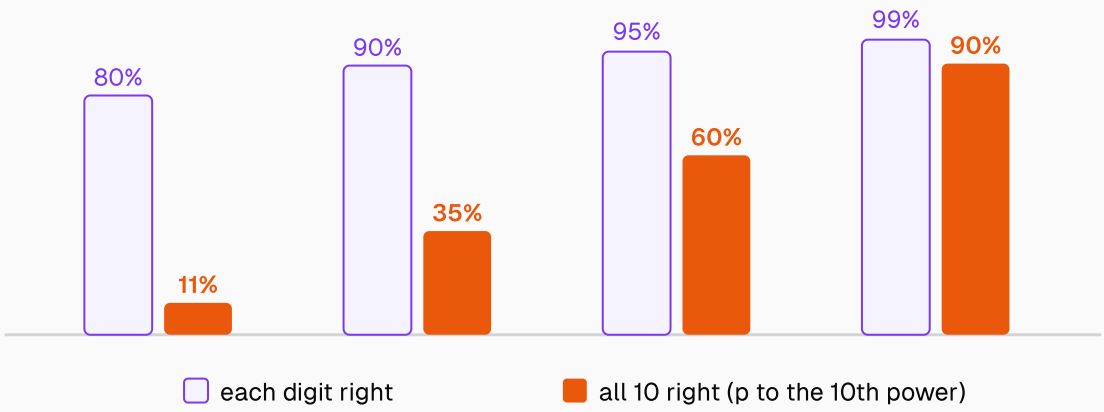
Diagram

A staircase that looks like a jump

Why abilities can seem to appear suddenly when the model improved gradually.

A student practises 10-digit multiplication and gets one more digit right each week. Progress is steady.

Now grade only on whether all ten digits are right. The score is zero for nine weeks, then suddenly 100%. It looks like a magical leap. The student did not change; the test did.



The arithmetic: if each digit is right with probability p, all ten are right with probability p^{10}. At 80% per digit that is 11%; at 90%, 35%; at 99%, 90%. Steady gains per digit become a hockey stick for the all-or-nothing score.

Forecast with the smooth metric. Report the one users feel.

Same model, same improvement, two pictures. Schaeffer et al. (2023) showed many “emergent” abilities turn smooth when re-scored this way. Practical rule: forecast capabilities with a continuous metric, and report the binary one.

Plain words

Getting every digit right

Imagine a student learning to multiply big numbers. Each week they get a few more digits right. But if the test only gives credit when *every* digit is right, their score sits at zero for months and then suddenly shoots up. The student improved steadily the whole time. The test made it look sudden.

For leaders

Why this debate matters for planning

Wei et al. (2022) catalogued "emergent abilities": tasks where small models score near zero and larger models suddenly succeed. Schaeffer, Miranda, and Koyejo (2023) argued many of these jumps come from harsh, all-or-nothing metrics. Re-scored with smooth metrics, such as per-token log likelihood or partial credit, most curves became gradual and predictable.

Both views carry a practical lesson. If you measure only exact-match accuracy, capabilities will appear to arrive without warning, and safety-relevant ones are exactly the ones you do not want to be surprised by. If you also track a smooth proxy, you can see a capability coming several scales early.

Eval lens

For every important capability, keep two metrics: the binary one users experience (did it get the answer right) and a continuous one that moves earlier (log probability of the correct answer, partial credit, pass@k at large k). Forecast with the continuous one. Report the binary one. Labs including Meta (Llama 3, 2024) predict downstream accuracy in two steps: compute to the model's negative log likelihood on the correct answers, then likelihood to accuracy with a sigmoid fit.

Under the hood

Why products of probabilities look like step functions

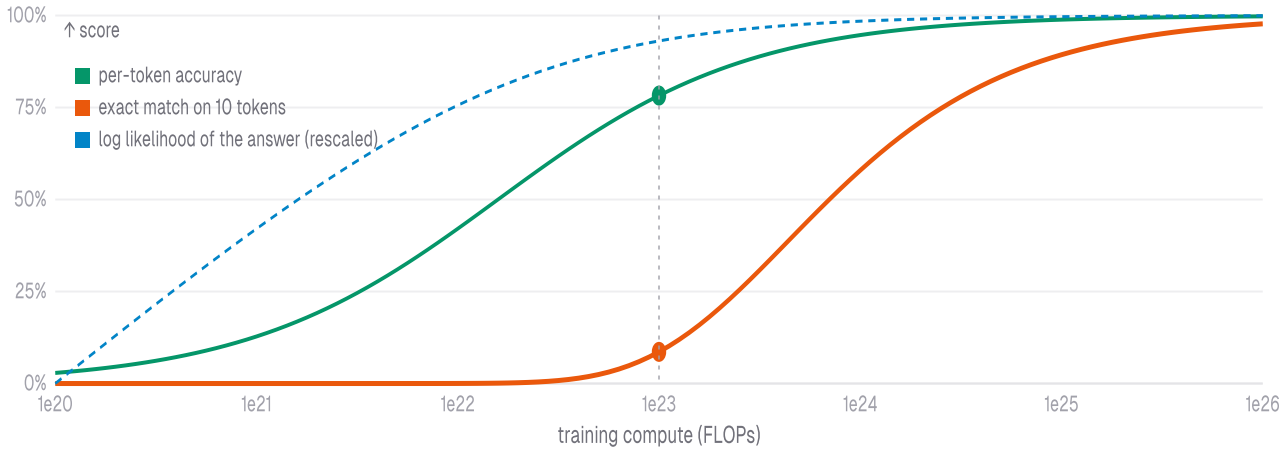
Let per-token accuracy rise smoothly with log compute x, say $p(x) = \sigma(a(x - x_0))$. If an answer needs k independent tokens all correct, exact match is $p(x)^k$. For large k, p^k stays near zero until p is close to 1, then rises steeply: the derivative $kp^{k-1}p'$ concentrates near the end. Per-token log likelihood $k \log p(x)$ is smooth throughout. Same model, same improvement, different shape.

$$\mathrm{EM}(x) = p(x)^k, \qquad \frac{d\,\mathrm{EM}}{dx} = k\,p(x)^{k-1}p'(x)$$

Interactive

The same improvement, two metrics (shown at its default setting)

The underlying skill (chance of getting each token of the answer right) improves smoothly with compute. Exact match needs every token right. Lengthen the answer and watch a smooth trend turn into an apparent jump.



At 1e23.0 FLOPs, per-token accuracy is **78%** but exact match on a 10-token answer is only **8.6%**. The red curve looks like a capability that "emerges" suddenly. The green and blue curves show the model was improving steadily all along.

Part III · Chapter 21

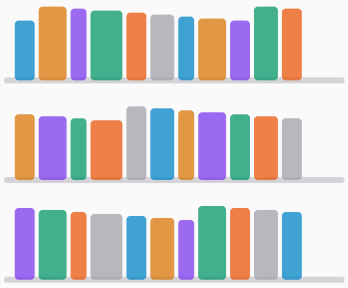
Supervised fine-tuning: teaching the format

A pretrained model has read a large slice of the internet and can continue any text. It doesn't yet know that it is supposed to be an assistant. Supervised fine-tuning (SFT) shows it thousands of example conversations and trains it, with the same next-token loss used in pretraining, to produce the assistant's side. It is the first step of post-training and the base that preference tuning and RL build on.

Diagram

The well-read new hire

What fine-tuning changes and what it does not, told as a new employee's first month.



Read the whole library

Customer: How do I reset my password?

Model: "...asked the customer nervously. Chapter 12. The rain had not stopped."

It continues the text. It does not answer.

A pretrained model is like a new hire who has read every book in the library and never held a job. Ask a question and it may just keep writing, because continuing text is all it has practised.

Q Reset password?

A Settings → Security → Reset. Check email.

Q Where is my refund?

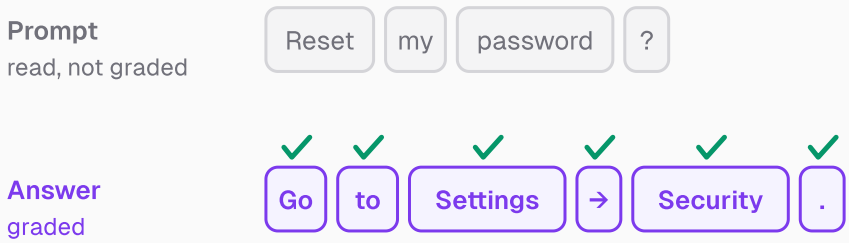
A Billing → Refunds. Takes 3 to 5 days.

Q Write me malware.

A Sorry, I can't help with that.

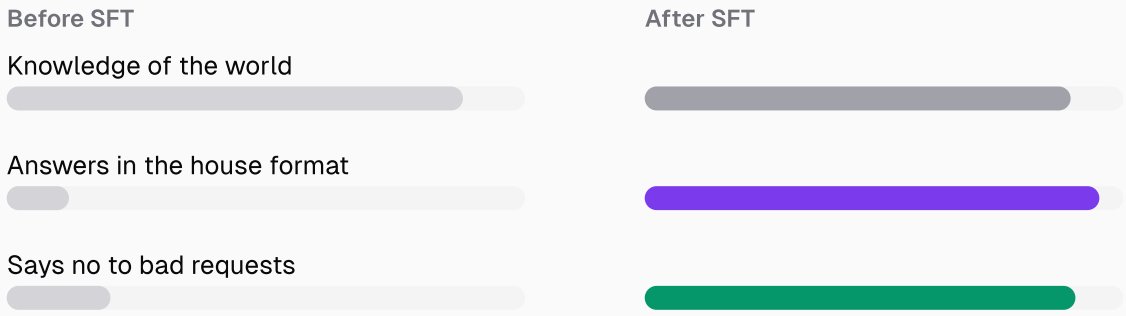
Thousands of examples: a question, then exactly how we answer it

Supervised fine-tuning is the training binder. Each page shows a real request and the exact reply the company wants, including when to say no. The model trains on these with the same next-word loss it used in pretraining.



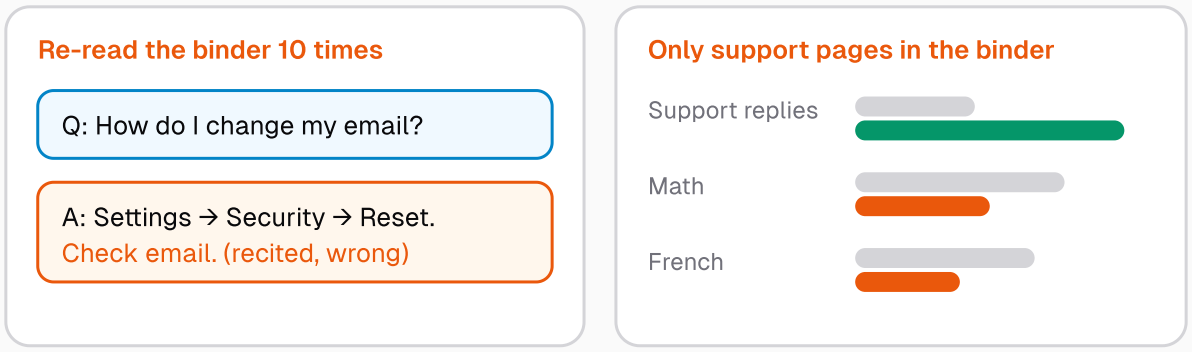
The loss counts 6 of the 10 tokens: only the answer teaches.

Like grading an exam, you mark what the student wrote, not the printed question. The model reads the whole conversation, but only the assistant's tokens count toward the loss. This is **loss masking**.



Mostly a change in how it answers, not in what it knows.

After a few thousand pages the new hire answers in the right format and tone. What they know barely moves. The LIMA study (Zhou et al., 2023) got strong chat behaviour from just 1,000 careful examples. Hard skills such as math and code still need far more data, and usually RL. Bar lengths are illustrative.



Fixes: 1 to 3 passes, mix in general data, pick checkpoints with evals.

Two ways it goes wrong. Too many passes and the model memorises the binder and recites old answers to new questions (**overfitting**). A narrow binder makes it worse at things it used to do well (**catastrophic forgetting**). Grey bars show before, coloured bars after.

What SFT actually changes

Plain words

A well-read intern learning the house style

Picture a new hire who has read every book in the library but has never worked in an office. Ask them a question and they might answer, or they might write the next page of a novel, or ask you a question back. They know a lot. They just don't know the job.

SFT is the training binder: "Here is a question a customer asked. Here is exactly how we answer." After a few thousand of these, the intern answers in the right format, in the right tone, and knows when to say "I can't help with that." The binder teaches very little new knowledge. Mostly it teaches how to use the knowledge they already have.

For leaders

SFT shapes behaviour cheaply. It rarely adds capability.

The LIMA paper (Zhou et al., 2023) fine-tuned a 65B model on only 1,000 carefully chosen examples and got responses that humans often preferred to those of much larger tuning efforts. The authors called this the **superficial alignment hypothesis**: almost all knowledge comes from pretraining, and SFT mostly selects which style and format to use. Later work showed it is only partly true. Format

and persona do come cheaply. Reliable gains on hard skills such as math, code, and tool use need much more data, often hundreds of thousands to millions of examples, and usually RL afterwards.

Cheap to change with SFT

Response format, tone, persona, refusal style, following a chat template, calling tools in the right syntax.

Expensive to change with SFT

Factual knowledge, deep reasoning, robustness on hard problems. SFT can teach a model to *sound* like it reasons without reasoning better.

Usually the bottleneck

Data quality and coverage, not compute. A single SFT run on a frontier model is small next to pretraining.

The real risk

Quietly damaging capabilities the base model had. You only see it if you measure it.

Principal asks

- What behaviour is this SFT run supposed to change, and which eval shows that change?
- Which capabilities could it break, and is there a regression suite for them?
- Who wrote the data, how was quality checked, and what fraction did a human actually read?
- Are we teaching a skill, or teaching the model to imitate the surface of a skill?

Under the hood

Same loss, different data, a masked sum

SFT minimises the conditional negative log likelihood of the response $\boldsymbol{y}$ given the prompt $\boldsymbol{x}$, over a dataset of demonstrations $\mathcal{D}$:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\frac{1}{|y|}\sum_{t=1}^{|y|}\log p_\theta(y_t \mid x, y_{<t})\right]$$

Identical to the pretraining objective, except the sum runs only over response tokens y_t. The prompt x is context, not a target.

Viewed as imitation learning, SFT is behaviour cloning: the model learns to copy the demonstrator's action at each step, always conditioned on the demonstrator's own previous tokens. That is called **teacher forcing**. At inference the model conditions on its *own* previous tokens, so small errors can compound into states the training data never covered. This exposure bias is one reason on-policy methods (RL, on-policy distillation) often help after SFT.

Chat templates and special tokens

Plain words

Stage directions in a script

A play script marks who is speaking: *HAMLET:*, *OPHELIA:*. A chat model needs the same thing. Special markers wrap each turn so the model knows where the user stops, where it should start talking, and when to stop. Get the markers wrong and the model gets confused, like an actor reading someone else's lines.

For leaders

Template mismatches cause silent, expensive bugs

The template is a contract between training, serving, and evaluation. If the eval harness formats prompts differently from training (a missing system turn, a different end-of-turn token, an extra newline), scores can drop several points with no change to the model. It is one of the first things to check when an eval result looks wrong.

> ⚠ **Watch out**
>
> Common template bugs: the end-of-turn token is never trained, so the model rambles past its answer; the tokenizer adds a second beginning-of-sequence token; tool-call turns use a role the serving stack doesn't know; the eval harness applies the base model's template to the chat model.

Under the hood

What a formatted conversation looks like

Templates differ by model family. A ChatML-style layout (illustrative) looks like this:

```text
<|im_start|>system
You are a concise assistant.<|im_end|>
<|im_start|>user
What is 17 * 3?<|im_end|>
<|im_start|>assistant
51<|im_end|>
```

The special markers are single tokens, either reserved in the pretraining vocabulary (as in Llama 3) or added before SFT; only when they are new are their embeddings learned during SFT. In Hugging Face tooling the template is stored with the tokenizer as a Jinja string and applied with `tokenizer.apply_chat_template`. Use the same function in training, serving, and the eval harness, and check a rendered example in each by eye.

Loss masking: which tokens teach

Plain words

Grade the answer, not the question

When you mark a student's exam you grade what they wrote, not the printed questions. SFT works the same way. The model reads the whole conversation, but it is only graded on the assistant's words. Otherwise it would spend effort learning to write user questions, which isn't its job.

For leaders

A one-line choice that changes what the model learns

Masking decides what behaviour you are paying to teach. Training on user turns teaches the model to imitate users, which can leak into answers that start interviewing the user back. Training on tool outputs teaches it to hallucinate tool results instead of waiting for them. Training only on the final turn of a multi-turn conversation wastes most of the data. Most teams default to "all assistant turns, never tool or environment output," and treat any change as an experiment with its own eval.

Under the hood

Labels of -100

In PyTorch, `CrossEntropyLoss` ignores label positions set to `-100` by default. Masking is done by copying the input ids into labels and overwriting non-target spans:

python

```python
import torch

IGNORE = -100  # CrossEntropyLoss(ignore_index=-100) skips these positions

def build_labels(input_ids, spans):
    """spans: list of (start, end, role) over token positions.
    Only assistant tokens, including the end-of-turn token, become targets."""
    labels = torch.full_like(input_ids, IGNORE)
    for start, end, role in spans:
        if role == "assistant":
            labels[start:end] = input_ids[start:end]
    return labels

ids = torch.arange(12)                        # stand-in token ids
spans = [(0, 4, "system"), (4, 7, "user"), (7, 12, "assistant")]
print(build_labels(ids, spans))
# tensor([-100, -100, -100, -100, -100, -100, -100,    7,    8,    9,   10,   11])
```

rust

```rust
// Plain Rust, no ML framework: the same math on small vectors.
const IGNORE: i64 = -100; // CrossEntropyLoss(ignore_index=-100) skips these positions

/// spans: list of (start, end, role) over token positions.
/// Only assistant tokens, including the end-of-turn token, become targets.
fn build_labels(input_ids: &[i64], spans: &[(usize, usize, &str)]) -> Vec<i64> {
    let mut labels = vec![IGNORE; input_ids.len()];
    for &(start, end, role) in spans {
        if role == "assistant" {
            labels[start..end].copy_from_slice(&input_ids[start..end]);
        }
    }
    labels
}

fn main() {
    let ids: Vec<i64> = (0..12).collect(); // stand-in token ids
    let spans = [(0, 4, "system"), (4, 7, "user"), (7, 12, "assistant")];
    println!("{:?}", build_labels(&ids, &spans));
}
```

```
    // [-100, -100, -100, -100, -100, -100, -100, 7, 8, 9, 10, 11]
}
```

One detail with real consequences: the loss is usually averaged over target tokens in the batch. Long responses then get more total weight than short ones. Some recipes average per sequence first, so every example counts equally. Neither is wrong, but changing it shifts the model toward or away from long answers.

Interactive

Loss-masking visualizer (shown at its default setting)

Each box is a token. Coloured tokens add to the loss; grey ones are context only. Switch schemes and watch how much of the batch actually teaches.

SYSTEM	<|im_start|>system · You · are · a · helpful · travel · assistant · . · <|im_end|>
USER	<|im_start|>user · What · ' · s · the · weather · in · Lisbon · tomorrow · ? · <|im_end|>
ASSISTANT	<|im_start|>assistant · get · _ · weather · (· city · = · " · Lisbon · " ·) · <|im_end|>
TOOL	<|im_start|>tool · { · " · high · " · : · 22 · , · " · low · " · : · 15 · , · " · rain · " · : · 0 · . · 1 · } · <|im_end|>
ASSISTANT	<|im_start|>assistant · Tomorrow · in · Lisbon · : · high · 22 · ° · C · , · low · 15 · ° · C · , · small · chance · of · rain · . · <|im_end|>
USER	<|im_start|>user · Should · I · pack · a · jacket · ? · <|im_end|>
ASSISTANT	<|im_start|>assistant · Yes · , · a · light · jacket · for · the · evening · . · <|im_end|>

Tokens in example	Tokens in the loss	Share of tokens that teach
95	41	43%

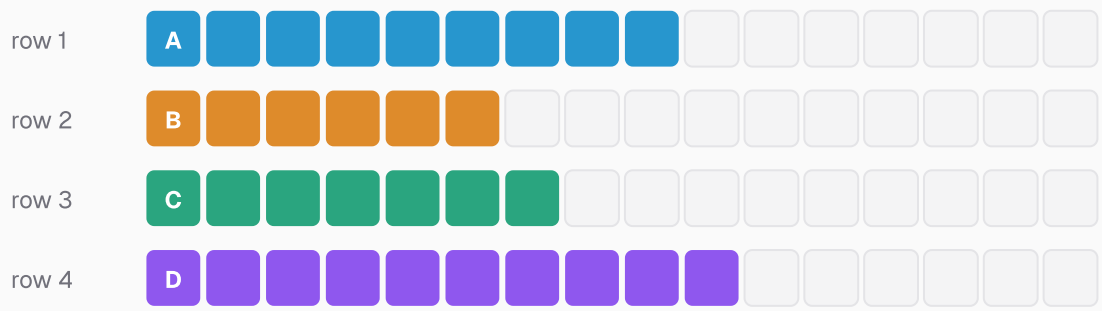
The usual default. Every assistant turn, including the tool call and its end-of-turn marker, is a target. Context tokens shape attention but add nothing to the loss. Words stand in for tokens here; real tokenizers split differently. Orange marks tokens that are trained but should not be.

Packing and attention masking

Diagram

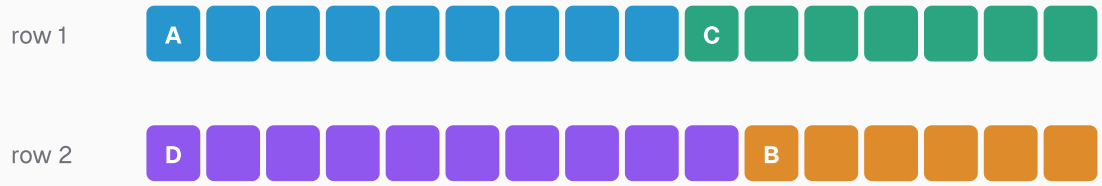
Packing the moving van

Why fine-tuning packs examples together, and the wall it needs between them.



Each row holds 16 tokens. Half of all slots are padding.

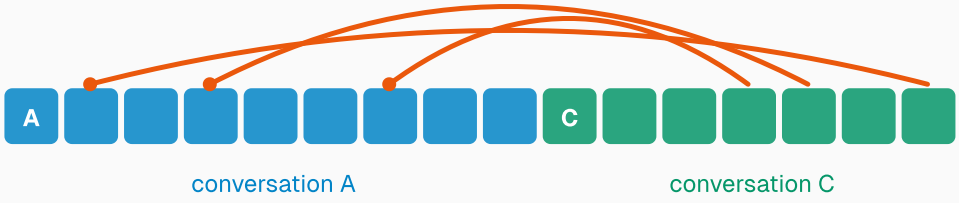
GPUs process fixed-length rows. Short conversations leave the rest of the row as padding, like a moving van carrying one chair. Here 32 of 64 slots do no work.



Same four conversations, two rows instead of four.

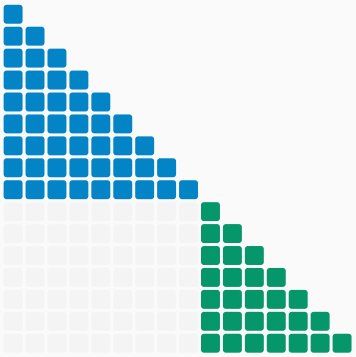
About half the compute for this batch.

Packing places conversations end to end so every slot holds a real token. When example lengths vary a lot, it can cut fine-tuning compute by 2× or more.



Without walls, C can attend to A, which it will never see at inference.

The catch: in a packed row, a token in conversation C could look back at conversation A. That is a small leak inside the training batch, and it teaches the model to rely on context that will not exist in real use.



who can see whom (row 1)

Walls between conversations

Coloured cells: allowed. Grey: blocked.

Each block is causal on its own.

Positions restart for each conversation:

A: 0 1 2 … 8 C: 0 1 2 … 6

Kernels take sequence boundaries instead of a full mask (e.g. cu_seqlens).

The fix is a **block-diagonal attention mask**: causal inside each conversation, blocked across them, with position ids restarting at zero. Ask whether your training stack does this whenever it packs.

Plain words

Filling the moving van

GPUs process fixed-length rows. Short conversations leave most of a row empty, like a moving van carrying one chair. Packing puts several conversations end to end in one row so nothing is wasted. The catch: you have to put up walls between them so one conversation can't peek at another.

For leaders

Throughput versus correctness

Packing can cut SFT compute by 2× or more when examples vary in length. Without proper attention masking, though, tokens in conversation B can attend to conversation A. That is a subtle form of contamination inside the training batch, and it can teach the model to draw on context that won't exist at inference. Ask whether the training stack uses per-document attention masks (often done with variable-length attention kernels) and resets position ids per document.

Under the hood

Block-diagonal causal masks

With packing, the attention mask becomes block diagonal: causal within each document, zero across documents. Implementations pass cumulative sequence lengths (for example `cu_seqlens` in FlashAttention's variable-length API) instead of materialising a full mask. Position ids restart at 0 for each document so rotary embeddings see the same positions they would without packing.

Data: quality, quantity, and mix

Plain words

You become what you imitate

The model will copy whatever is in the examples, including mistakes, padding, and bad habits. A thousand excellent examples can beat a hundred thousand sloppy ones. And if every example is about cooking, the model gets worse at everything else.

For leaders

Where SFT budgets actually go

Quality filtering

often beats collecting more. Remove wrong answers, refusals of fine requests, and answers that hedge or pad. Each one teaches a habit.

Diversity

of tasks, formats, and difficulty matters more than raw count. Open recipes such as Tülu 3 (Lambert et al., 2024) mix general chat, math, code, safety, multilingual, and instruction-following sets, and tune the mixture against a broad eval suite.

Mixture weights

are hyperparameters. Upweighting one skill steals capacity from others. Run ablations, not intuition.

Human versus synthetic data

Most modern SFT data is model-generated and filtered (see distillation and synthetic data). Human data stays essential for the hardest tasks, for safety, and for checking that synthetic data isn't drifting.

Multi-turn and tool-use data

must include realistic failures: tool errors, ambiguous requests, users changing their minds. A model trained only on clean single-turn data breaks in real agent traces.

Watch out

SFT data written by a stronger model teaches the student to sound like that model, confidence included. If the teacher states things it can't verify, the student learns to hallucinate in the same voice. Fluency goes up faster than accuracy, and judge-based evals reward fluency.

Epochs, overfitting, and forgetting

Plain words

Memorising the binder, forgetting the library

Read the training binder once and you learn the style. Read it ten times and you start reciting the exact examples word for word, which doesn't help with new questions. Worse, if the binder only covers a few topics, the intern can start forgetting things they knew before. That is **catastrophic forgetting**.

For leaders

Pick checkpoints with evals, not loss

Typical SFT runs 1 to 3 epochs. Validation loss often starts rising after the first or second epoch while downstream scores keep improving for a while, because the model gets more decisive about format. So validation loss alone is a poor stopping rule. Teams save several checkpoints and choose with a broad eval suite.

Forgetting is the cost leaders most often miss. Narrow SFT, such as a single domain or one customer's format, can reduce general reasoning, safety behaviour, or multilingual ability. Standard mitigations are mixing in general instruction data (replay), lower learning rates, fewer epochs, and LoRA, which tends to forget less because it changes fewer directions in weight space.

Under the hood

Why validation loss and quality diverge

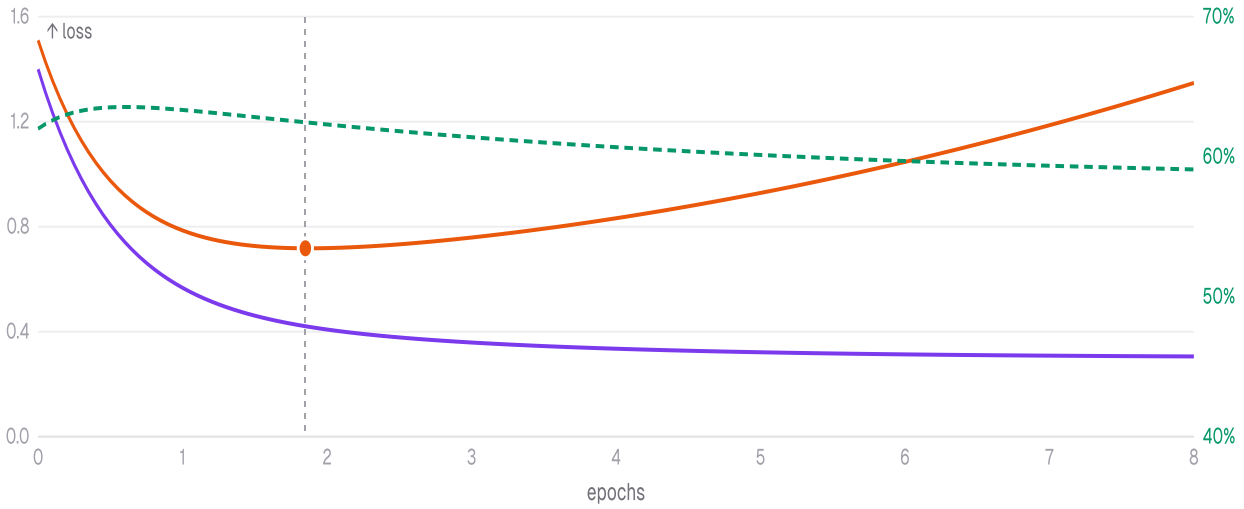
Cross-entropy on a held-out set penalises confident wrong tokens heavily. As SFT sharpens the output distribution (lower entropy), a few confidently wrong tokens on held-out examples can raise the average loss even while greedy outputs get better. Loss measures calibration of the whole distribution; quality metrics usually look at one sampled or greedy answer. Report both and expect them to disagree after the first epoch.

To detect forgetting, evaluate the base model and every candidate checkpoint on the same frozen general suite, and test the difference with a paired method from Part II, since the items are shared.

Interactive

Overfitting and forgetting over epochs (shown at its default setting)

Illustrative curves from a simple model, not a real run. Shrink the dataset or lower its diversity and watch validation loss turn up and the general benchmark fall.



Train loss · Validation loss · General benchmark (right axis)

Best epoch by validation	Benchmark at best epoch	Benchmark after 8 epochs
1.9	62.5%	59.1%

Validation loss bottoms out around epoch 1.9. Training longer keeps lowering train loss while validation rises: the model is memorising examples. Running all 8 epochs would cost 3.4 points of general capability compared with stopping early. The base model scores 62% on the general benchmark before SFT.

Evaluating an SFT run

Eval lens

Held-out loss is necessary but not enough. A credible SFT evaluation has four parts:

1. **Target behaviour.** The thing the run was meant to change: format adherence, tool-call validity, refusal accuracy, domain task pass rate. Measure it with intervals.
2. **Capability regression suite.** A fixed, broad set (reasoning, code, math, knowledge, multilingual, safety) compared against the parent checkpoint with paired tests. Any drop beyond noise is a finding.
3. **Pairwise win rate** against the previous model on realistic prompts, judged by humans or a validated LLM judge, with position swapping and length control.
4. **Contamination check.** Synthetic SFT data can contain benchmark items the teacher saw. Run n-gram and embedding overlap against every eval set (see contamination).

Principal asks

- What did the base model score on the regression suite, and what is the paired delta with an interval?
- Did win rate go up because answers got better, or because they got longer?
- Which checkpoint did we pick, and was it chosen on the same data we are now reporting? (If so, the number is optimistically biased.)
- If we removed the 10% lowest-quality examples, would the result change?

Part III · Chapter 22

LoRA and parameter-efficient tuning

Full fine-tuning updates every weight in the model. Parameter-efficient fine-tuning (PEFT) freezes the pretrained weights and trains a small add-on instead. LoRA is by far the most used method. It makes customisation cheap enough to run on one GPU, and lets one server hold thousands of variants of the same base model.

Diagram

Clip-on filters for a giant lens

LoRA, QLoRA, adapter serving, and merging, told with a camera.

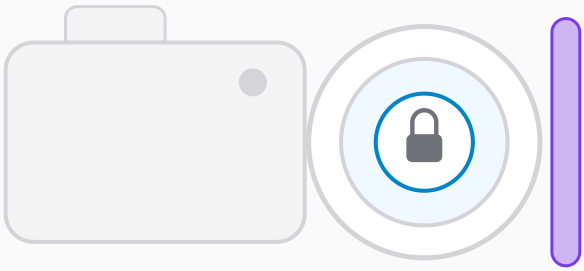
The base model

8 billion trained weights

Rebuild the whole lens per job
= full fine-tuning

About 16 bytes of GPU memory per weight: roughly 128 GB

Think of the pretrained model as a huge, expensive camera lens. Full fine-tuning regrinds the whole lens for every job: every weight changes, and training needs its weights, gradients, and optimizer state in memory at once.

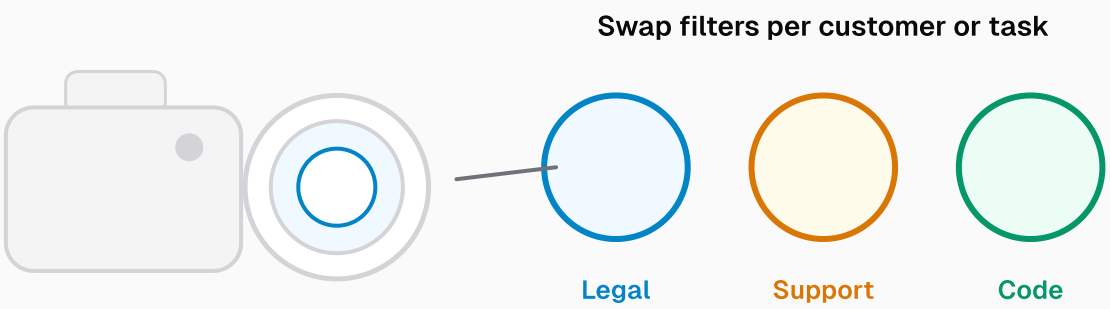


Clip on a thin filter

LoRA adapter: under 1% of the weights, trained

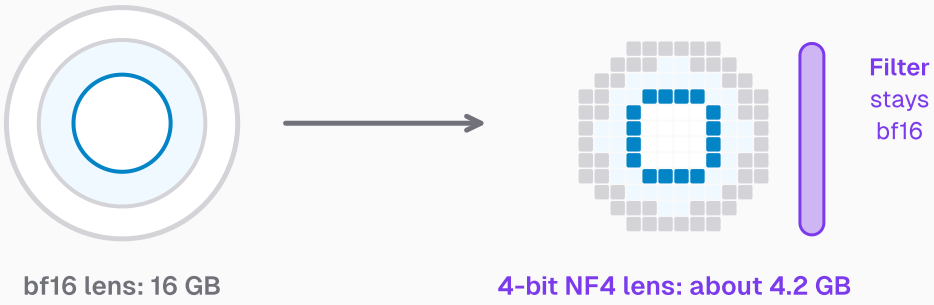
The lens stays frozen. Optimizer memory almost disappears.

LoRA freezes the lens and clips on a thin filter that changes the look. Only the filter is trained, so the gradients and optimizer state shrink to almost nothing. The frozen base dominates memory.



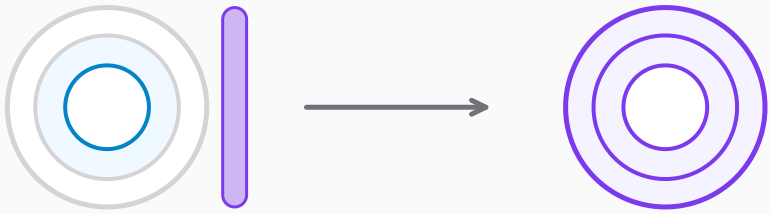
One base in memory serves many adapters (S-LoRA, Punica).

Because the lens never changes, one copy can serve many filters. A product can keep a separate adapter for every customer, each only megabytes, and batch their requests through the same base model.



Sizes for an 8B model, weights only.

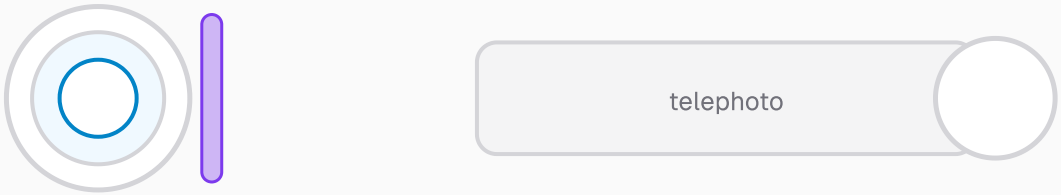
QLoRA (Dettmers et al., 2023) also compresses the frozen lens to 4 bits with a format called NF4. Gradients flow through the compressed lens into the full-precision filter. The paper fine-tuned a 65B model on a single 48 GB GPU.



$$W = W_0 + (\alpha/r)\cdot B\cdot A$$

Fuse the filter into the lens: no extra latency at serving.

For a single deployment you can fuse the filter into the lens: add the low-rank update to the frozen weights once, and serve a normal model with zero extra cost. Fusing into a 4-bit base is only approximate, so evaluate the model you actually ship.



Filter: new tint, same lens

A whole new lens: needs regrinding

LoRA learns less and forgets less (Biderman et al., 2024).

A filter can change style, format, and domain behaviour. It cannot turn a wide-angle lens into a telephoto. For a genuinely new capability, or billions of training tokens, full fine-tuning or continued pretraining usually wins.

Why not just fine-tune everything?

Plain words

Sticky notes instead of rewriting the book

Full fine-tuning is like reprinting a 1,000-page encyclopedia every time you want to change how it reads. LoRA leaves the book alone and adds a thin stack of sticky notes that adjust it. The notes are tiny, you can swap them in and out, and you can keep a different stack for each customer. When you want a final copy, you can even print the notes into the pages.

For leaders

The cost picture

Full fine-tuning with the Adam optimiser in mixed precision needs about 16 bytes of GPU memory per parameter before activations: the weights, their gradients, a full-precision master copy, and two optimiser statistics. That is roughly 128 GB for an 8B model and over 1 TB for a 70B model, so full fine-tuning means a multi-GPU cluster. LoRA trains well under 1% of the parameters, so the optimiser and gradient memory nearly vanish and the frozen base dominates. QLoRA goes further and stores the frozen base in 4 bits.

Cheaper training

Fits on far fewer GPUs. The original LoRA paper reports about 3× less GPU memory and 10,000× fewer trainable parameters than full fine-tuning GPT-3 175B (Hu et al., 2021).

Cheap storage

An adapter is megabytes to a few gigabytes, not a full copy of the model.

Cheap serving

Many adapters share one base model in memory. Systems such as S-LoRA (Sheng et al., 2023) and Punica (Chen et al., 2023) batch requests for different adapters together.

Less forgetting

Because LoRA changes few directions, it tends to damage general ability less, at the price of learning less from large datasets.

How LoRA works

Plain words

A big change described by a few knobs

A weight matrix in a model might have 16 million numbers. LoRA bets that the *change* you need during fine-tuning is simple, so it can be described with far fewer numbers: two thin matrices that multiply together to make a full-size correction. The thinness is called the **rank**. Rank 8 means the whole correction is built from 8 basic patterns.

Under the hood

The low-rank update

For a frozen weight $W_0 \in \mathbb{R}^{d \times k}$, LoRA learns $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $r \ll \min(d, k)$:

$$h = W_0 x + \frac{\alpha}{r} BAx$$

A starts random (Gaussian), B starts at zero, so $BA = 0$ and training begins exactly at the pretrained model. α is a fixed scale; dividing by r keeps the update size roughly stable when you change the rank.

The update has $r(d + k)$ trainable parameters instead of dk. For a 4096 × 4096 projection at rank 16, that is 131,072 instead of 16.8 million, under 1%. After training, $W = W_0 + \frac{\alpha}{r} BA$ can be merged into the base weights, so a single-adapter deployment has no extra inference cost.

python

```python
import torch, torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r=16, alpha=32):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False                    # the pretrained weight stays frozen
        d_out, d_in = base.weight.shape
        self.A = nn.Parameter(torch.randn(r, d_in) / r**0.5)
        self.B = nn.Parameter(torch.zeros(d_out, r))    # zero init: BA = 0 at step 0
        self.scale = alpha / r

    def forward(self, x):
        return self.base(x) + self.scale * (x @ self.A.T @ self.B.T)

    @torch.no_grad()
    def merge(self):
        # fold the adapter into the weight for zero-overhead serving
```

```
        self.base.weight += self.scale * self.B @ self.A

layer = LoRALinear(nn.Linear(4096, 4096, bias=False), r=16)
trainable = sum(p.numel() for p in layer.parameters() if p.requires_grad)
print(trainable, f"{trainable / 4096**2:.2%}")  # 131072 0.78%
```

rust

```
// Plain Rust, no ML framework: the same math on small vectors.
// Matrices are flat row-major Vec<f64>. The random init uses a small inline PRNG
// (SplitMix64), so the draws differ from torch; the parameter count does not.

struct SplitMix64(u64);

impl SplitMix64 {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn uniform(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64 // in [0, 1)
    }
    fn normal(&mut self) -> f64 {
        // Box-Muller; 1 - u keeps the log argument in (0, 1]
        let (u1, u2) = (1.0 - self.uniform(), self.uniform());
        (-2.0 * u1.ln()).sqrt() * (2.0 * std::f64::consts::PI * u2).cos()
    }
}

struct LoRALinear {
    weight: Vec<f64>, // d_out x d_in; the pretrained weight stays frozen
    a: Vec<f64>,      // r x d_in, trainable
    b: Vec<f64>,      // d_out x r, trainable
    d_out: usize,
    d_in: usize,
    r: usize,
    scale: f64,
}

impl LoRALinear {
    fn new(weight: Vec<f64>, d_out: usize, d_in: usize, r: usize, alpha: f64, rng: &mut SplitMix64) -> Self {
        let a = (0..r * d_in).map(|_| rng.normal() / (r as f64).sqrt()).collect();
        let b = vec![0.0; d_out * r]; // zero init: BA = 0 at step 0
        LoRALinear { weight, a, b, d_out, d_in, r, scale: alpha / r as f64 }
    }

    fn trainable_params(&self) -> usize {
        self.a.len() + self.b.len() // only A and B get gradients
    }

    fn forward(&self, x: &[f64]) -> Vec<f64> {
        // base(x) + scale * B (A x), never forming the d_out x d_in product BA
        let ax: Vec<f64> = (0..self.r)
            .map(|i| (0..self.d_in).map(|j| self.a[i * self.d_in + j] * x[j]).sum())
            .collect();
        (0..self.d_out)
            .map(|o| {
```

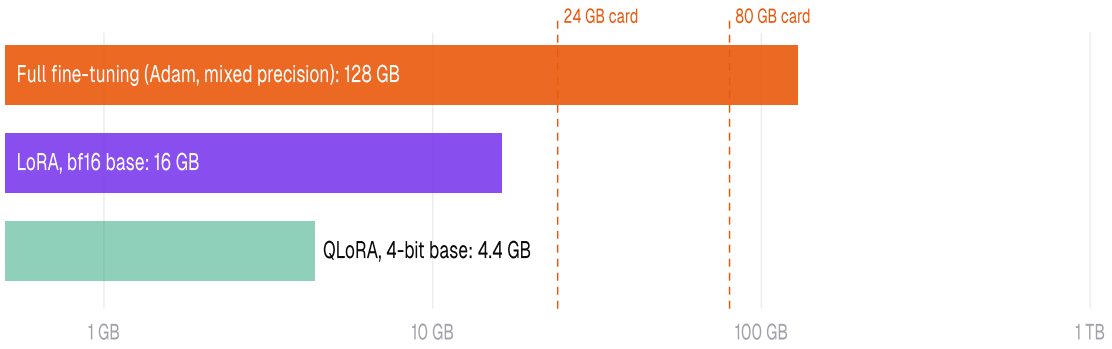
```
                let base: f64 = (0..self.d_in).map(|j| self.weight[o * self.d_in + j] * x[j]).sum();
                let lora: f64 = (0..self.r).map(|i| self.b[o * self.r + i] * ax[i]).sum();
                base + self.scale * lora
            })
            .collect()
    }

    fn merge(&mut self) {
        // fold the adapter into the weight for zero-overhead serving
        for o in 0..self.d_out {
            for j in 0..self.d_in {
                let ba: f64 = (0..self.r).map(|i| self.b[o * self.r + i] * self.a[i * self.d_in + j]).sum();
                self.weight[o * self.d_in + j] += self.scale * ba;
            }
        }
    }
}

fn main() {
    let d = 4096;
    let mut rng = SplitMix64(0);
    let layer = LoRALinear::new(vec![0.0; d * d], d, d, 16, 32.0, &mut rng);
    let trainable = layer.trainable_params();
    let frac = trainable as f64 / (d * d) as f64;
    println!("{} {:.2}%", trainable, 100.0 * frac); // 131072 0.78%
}
```

Practical defaults have shifted since 2021. The original paper adapted only the attention query and value projections. Later work, including QLoRA and the Thinking Machines "LoRA Without Regret" study (2025), found that adapting *all* linear layers, especially the MLP blocks, matters more than raising the rank. The same study reports that the best LoRA learning rate is about 10× the best full fine-tuning rate. Rank-stabilised LoRA (Kalajdzievski, 2023) scales by $\alpha/\sqrt{r}$ instead of α/r so that high ranks keep learning.

Interactive

LoRA parameter and memory calculator (shown at its default setting)

Pick a model shape, the matrices to adapt, and a rank. Memory counts weights, gradients, and Adam state only; activations come on top and depend on batch size and sequence length.



Model parameters	Trainable (LoRA)	Share trainable	Adapter file (bf16)
8.0B	6.8M	0.085%	14 MB

Weights, gradients, and optimizer state only: about 16 GB for this setup versus 128 GB for full fine-tuning, 8× less. Doubling the rank doubles the trainable parameters but barely moves total memory, because the frozen base dominates. Adding the MLP projections usually improves quality more than raising the rank.

Why a low-rank update is enough

Diagram

A big table that is really two short lists

What "low rank" means, with shop sales instead of matrices.

	Jan	Feb	Mar	Apr	May	Jun
Store A	2	3	5	4	3	2
Store B	4	6	10	8	6	4
Store C	6	9	15	12	9	6
Store D	8	12	20	16	12	8

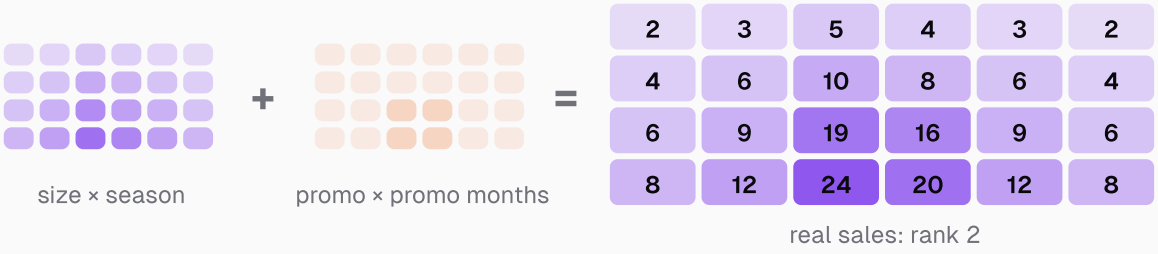
Monthly sales for 4 stores: 24 numbers.

Here is a table of monthly sales for four shops. It looks like 24 separate numbers. Look closer: bigger stores sell more, and March is busy everywhere.

	×	Jan	Feb	Mar	Apr	May	Jun
	×	2	3	5	4	3	2
Store A	1	2	3	5	4	3	2
Store B	2	4	6	10	8	6	4
Store C	3	6	9	15	12	9	6
Store D	4	8	12	20	16	12	8

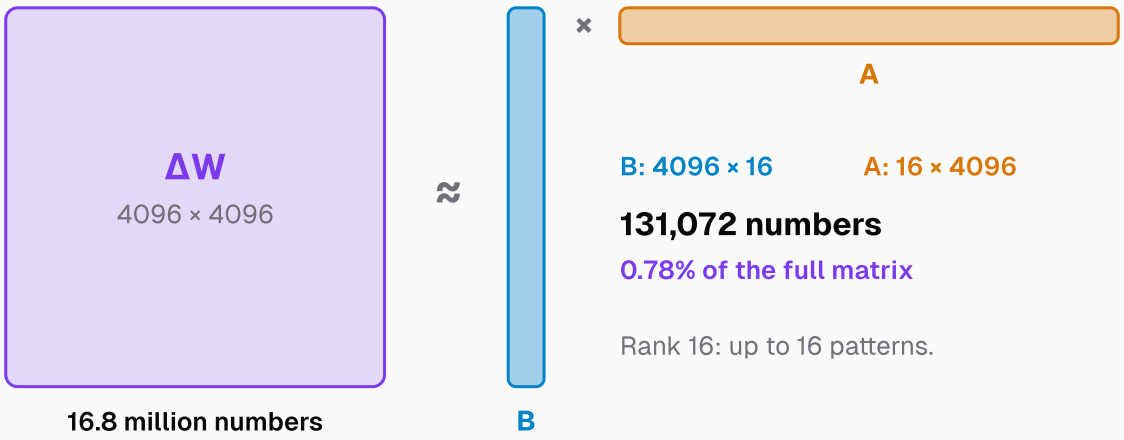
Size × season rebuilds every cell: 4 + 6 = 10 numbers.

Every cell is **store size × month busyness**. Two short lists, 4 numbers and 6 numbers, rebuild all 24. In linear algebra this table has **rank 1**: one row pattern times one column pattern.

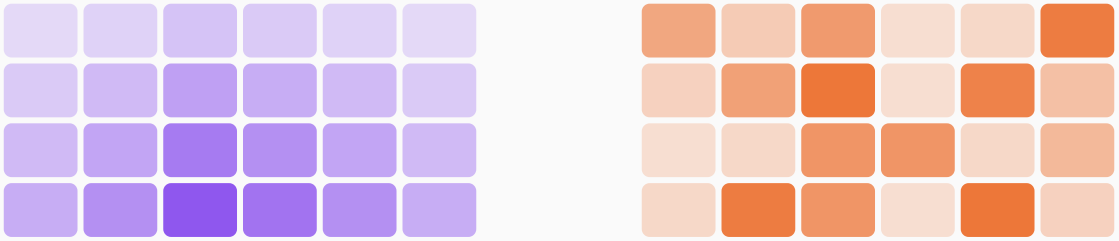


A spring promotion in two stores adds a second pattern.

Real tables have a few overlapping patterns. A spring promotion in stores C and D adds a second row-times-column pattern. Two patterns is **rank 2**. LoRA's rank *r* is how many patterns the weight change is allowed to have.



LoRA applies the same trick to a weight update. Instead of learning all 16.8 million entries of a 4096 × 4096 change, it learns two thin matrices whose product is the change. At rank 16 that is 131,072 numbers.



Structured: a few patterns rebuild it

Noise: needs all 24 numbers

Fine-tuning changes look more like the left table than the right.

Random noise has no shortcut, so a low-rank version loses most of it. Fine-tuning changes turn out to be far more structured: Aghajanyan et al. (2021) found fine-tuning works in a surprisingly small subspace. That is why a small rank usually captures what matters.

Plain words

Most changes are simple

A random scribble needs every pixel to describe it. A photo of the sky is mostly a smooth gradient and can be described by a few numbers. Fine-tuning changes turn out to look more like the sky than the scribble. That's why a few patterns capture most of the change.

Under the hood

Intrinsic dimension and the Eckart–Young theorem

The best rank-r approximation of a matrix in Frobenius norm keeps its top r singular values (Eckart–Young). The fraction of squared norm kept is

$$\frac{\|M_r\|_F^2}{\|M\|_F^2} = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_i \sigma_i^2}$$

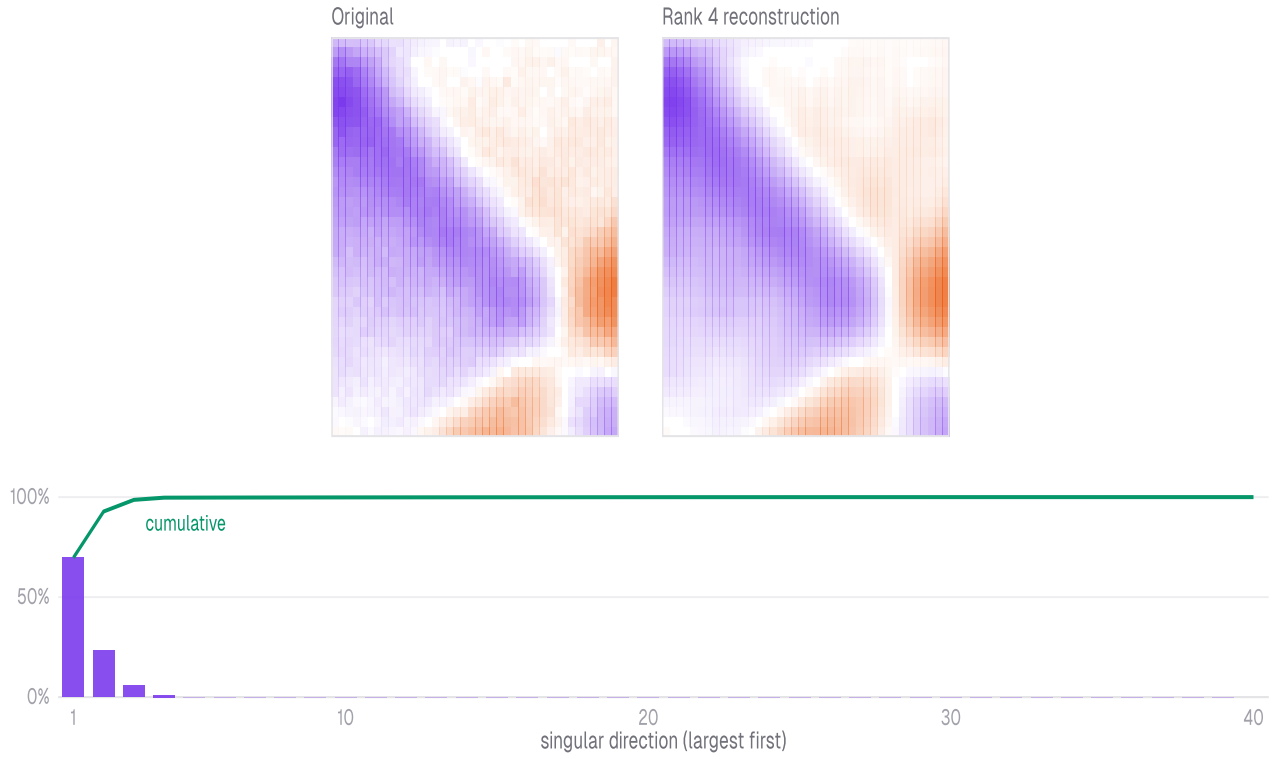
Aghajanyan et al. (2021) showed that fine-tuning a pretrained language model can reach most of its full performance while optimising in a random subspace of only hundreds to thousands of dimensions, and that larger, better-pretrained models have *lower* intrinsic dimension. LoRA exploits the same fact per weight matrix. The caveat from Biderman et al. (2024, "LoRA Learns Less and

Forgets Less"): on large-scale code and math training, full fine-tuning learns perturbations with rank 10 to 100× higher than typical LoRA settings, and LoRA falls behind. Low rank is a good bet for adapting behaviour, not for teaching large amounts of new knowledge.

Interactive

Low-rank approximation (shown at its default setting)

A 40 × 40 matrix rebuilt from its top r singular directions (SVD computed in your browser). Structured matrices are captured by a few directions; random noise is not.



Energy captured	Numbers stored	vs full matrix
99.7%	320	20%

Like a real fine-tuning update, this matrix has a few strong directions and a little noise. Rank 4 captures nearly all of it. Storing a rank-4 copy takes r × (rows + columns) = 320 numbers instead of 1,600.

QLoRA, DoRA, and the rest of the family

For leaders

Which variant, when

Method	Idea	Use when	Cost
LoRA (Hu et al., 2021)	Trainable low-rank update on frozen weights	The default for adapting behaviour, style, domain format	Learns less than full FT on large datasets
QLoRA (Dettmers et al., 2023)	LoRA on top of a 4-bit quantised base	Memory is the constraint. The paper fine-tuned a 65B model on one 48 GB GPU	Slower steps from dequantisation; small quality risk from quantised base
DoRA (Liu et al., 2024)	Splits each weight into magnitude and direction; LoRA adapts the direction	You want closer-to-full-FT behaviour at LoRA cost	Some extra compute; merge still possible
Adapters (Houlsby et al., 2019)	Small bottleneck layers inserted in each block	Mostly historical in LLMs	Adds inference latency unless removed
Prefix / prompt tuning (Li & Liang, 2021; Lester et al., 2021)	Learn virtual tokens prepended to the input	Extremely small storage; many tasks	Uses context length; usually weaker than LoRA
Full fine-tuning	Update everything	Large datasets, new capabilities, the lab's own post-training	Most memory, most forgetting risk

> ⚠ **Watch out**
>
> Merging a LoRA adapter into a quantised base is not exact: you must dequantise, add, and requantise, which changes outputs slightly. Evaluate the artefact you actually serve, not the training-time model with adapter attached.

Under the hood

What QLoRA adds

- **NF4 (4-bit NormalFloat).** Pretrained weights are roughly normally distributed, so NF4 places its 16 levels at quantiles of a normal distribution. Each block of 64 weights has its own scale.
- **Double quantisation.** The per-block scales are themselves quantised, saving about 0.37 bits per parameter.
- **Paged optimisers.** Optimiser state can spill to CPU memory during spikes instead of crashing with out-of-memory errors.

Gradients flow through the dequantised 4-bit weights into the bf16 LoRA parameters; the base itself never updates.

LoRA beyond SFT

For leaders

RL, multi-tenant serving, and when to say no

RL. The Thinking Machines study (2025) found LoRA matches full fine-tuning for reinforcement learning even at very low rank, including rank 1. The intuition: a policy-gradient episode carries only a few bits of information (did it succeed?), so the total information learned is small enough to fit in a tiny adapter. That makes LoRA attractive for RL with verifiable rewards, where many runs and ablations are needed.

Multi-tenant products. Per-customer or per-task adapters on a shared base are the standard way to offer fine-tuning as a product. The eval burden scales with the number of adapters: each one needs a regression check against the base.

When to say no. If the goal is a new capability that the base model can't do at all, or the dataset is in the billions of tokens, LoRA is likely to underperform. Expect to need full fine-tuning or continued pretraining.

Eval lens

A LoRA result needs three comparisons, not one: the adapter versus the base on the target task (did it learn?), the adapter versus the base on a general regression suite (did it forget?), and the merged or quantised serving artefact versus the training-time model (did deployment change it?). Ranks and learning rates interact, so compare tuned configurations, not one LoRA setting against a tuned full fine-tune.

Principal asks

- Which modules did we adapt, at what rank, and did we tune the learning rate separately for LoRA?
- Is the gap to full fine-tuning on the target metric outside the noise, and what is the gap on the forgetting suite?
- Are we evaluating the merged, quantised artefact that ships?
- How many adapters will we serve, and who owns their regression testing?

Part III · Chapter 23

Distillation, synthetic data, and merging

Most training data for modern post-training is written by models, not people. A large teacher model generates examples, a filter keeps the good ones, and a smaller or newer student learns from them. This chapter covers the three ways models learn from other models: distillation, synthetic data with filtering, and merging weights directly.

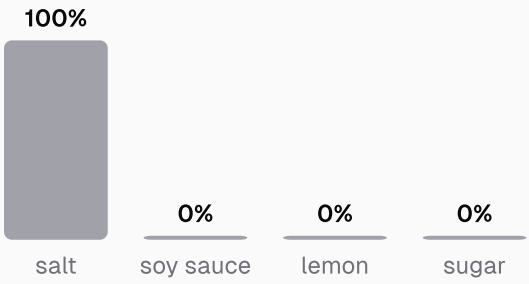
Diagram

The chef's tasting notes

Why learning from a teacher's probabilities beats learning from an answer key.

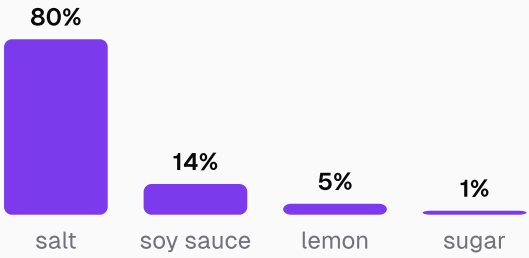
What does this soup need?

Answer key: "salt."

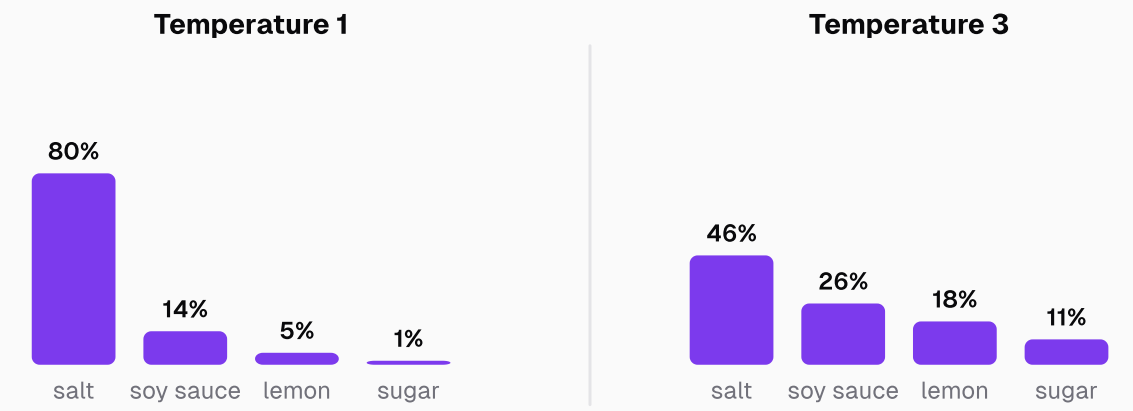


An answer key gives one right answer and nothing else. Training on it tells the student that salt is correct, and says nothing about whether soy sauce was a near miss or sugar was absurd. This is a **hard label**.

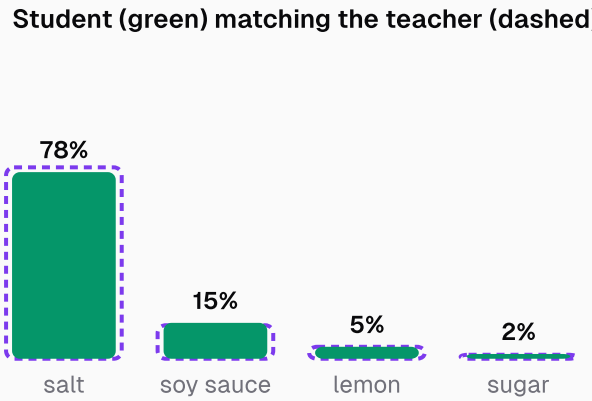
"Salt, probably. Soy sauce would work too. Sugar? Never."



A master chef thinking out loud gives far more: a full set of probabilities. The runner-ups carry the "almost right" information Hinton and colleagues called **dark knowledge**. Distillation trains the small model on these **soft labels**.

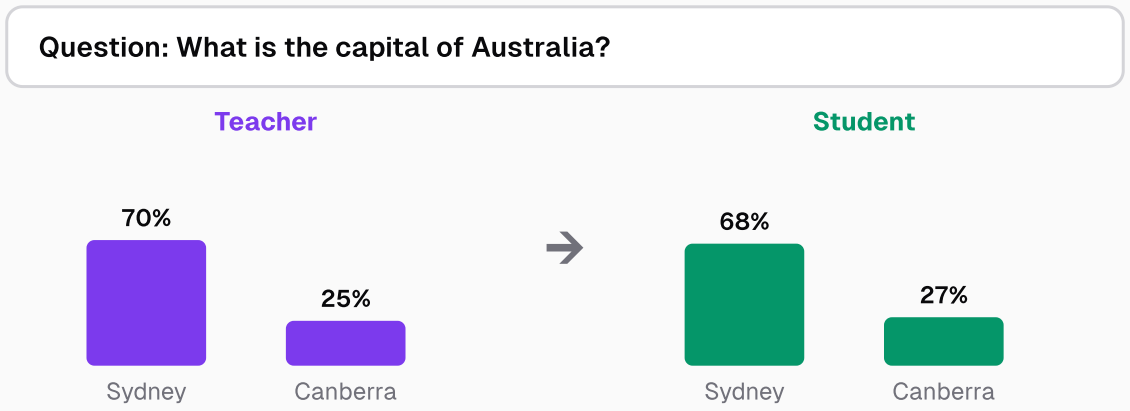


Raising the **temperature** divides the teacher's logits before the softmax, flattening the bars. The ranking stays the same, but the small differences between wrong answers become big enough for the student to learn from.



From one example it learns:
soy sauce is close,
sugar is absurd.

The student is trained to match the whole shape, usually by minimising the KL divergence between the two distributions. Each example now teaches a ranking, not just a winner, which is why small models distilled from big ones beat small models trained from scratch.



The catch: the student copies mistakes too, confidence included. If the teacher leans the wrong way (the answer is Canberra), so does the student. A distilled model is capped by its teacher and inherits its blind spots.

Knowledge distillation

Plain words

Learning from the teacher's hesitation

A student can learn from an answer key: "the answer is Paris." Or they can learn from a teacher who thinks out loud: "Paris, almost certainly. Lyon is a distant second. It's definitely not banana." The second version carries more information. It tells the student which wrong answers are nearly right and which are absurd. Distillation trains a small model on the big model's full set of probabilities, not just its single best answer. Hinton and colleagues called that extra information "dark knowledge."

For leaders

Why every lab distils

Distillation is how a lab turns one expensive frontier model into a family of cheaper ones. Small models trained from a strong teacher usually beat small models trained from scratch on the same compute. Gemma 2 (Google, 2024) trained its 2B and 9B models with distillation from a larger model in place of plain next-token prediction. DeepSeek-R1 (2025) fine-tuned smaller Qwen and Llama models on about 800K curated samples (roughly 600K R1 reasoning traces plus 200K non-reasoning examples) and reported that this beat running RL directly on the small models.

Logit distillation

Match the teacher's full probability distribution at each token. Needs the teacher's logits and a shared tokenizer, so it is mostly done inside one lab.

Sequence distillation

Train on text the teacher generated (Kim & Rush, 2016). Works through an API and across tokenizers. This is what "training on model outputs" usually means.

On-policy distillation

The student writes the text and the teacher grades every token (Agarwal et al., 2024). Fixes the student's own mistakes instead of only copying perfect examples.

Reasoning-trace distillation

Train on the teacher's step-by-step solutions. Fast way to give a small model long-form reasoning; inherits the teacher's errors and verbosity.

⚠ **Watch out**

A distilled student is capped by its teacher and copies its blind spots. If the teacher hallucinates on a topic, the student will too, with the same confidence. Distilling from a model you don't control also raises terms-of-service and licensing questions: many providers' terms restrict using outputs to build competing models, and open-weight licenses carry their own conditions. Get legal review before a data pipeline depends on another company's model.

● Under the hood

Soft targets, temperature, and which KL

With teacher logits z^T and student logits z^S, soften both with temperature τ:

$$p_i^{(\tau)} = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \qquad \mathcal{L} = (1-\lambda)\,\mathrm{CE}\big(y,\, p^{S,(1)}\big) + \lambda\,\tau^2\,\mathrm{KL}\big(p^{T,(\tau)} \,\|\, p^{S,(\tau)}\big)$$

Hinton, Vinyals & Dean (2015). The τ^2 factor keeps the soft-target gradients on the same scale as the hard-label term when τ changes. λ balances the two.

The direction of the KL matters for generative models:

- **Forward KL**, $\mathrm{KL}(p^T \| p^S) = \sum_i p_i^T \log \frac{p_i^T}{p_i^S}$, punishes the student for putting low probability anywhere the teacher puts mass. It is *mode covering*: a small student spreads itself thin and can produce blends of answers.
- **Reverse KL**, $\mathrm{KL}(p^S \| p^T)$, punishes the student for putting mass where the teacher doesn't. It is *mode seeking*: the student commits to the teacher's main modes. MiniLLM (Gu et al., 2024) argued reverse KL suits small generative students better. On-policy distillation scores the student's own samples against the teacher, most often with a per-token reverse KL (GKD, Agarwal et al. 2024, allows other divergences).

python

```python
import torch, torch.nn.functional as F

def kd_loss(student_logits, teacher_logits, labels, tau=2.0, lam=0.5):
    """Hinton-style distillation for a batch of next-token predictions.
    logits: [batch, vocab]; labels: [batch] token ids (-100 to ignore)."""
    hard = F.cross_entropy(student_logits, labels, ignore_index=-100)
    log_ps = F.log_softmax(student_logits / tau, dim=-1)
    log_pt = F.log_softmax(teacher_logits / tau, dim=-1)
    # forward KL(teacher || student); kl_div expects log-probs for the input
    soft = F.kl_div(log_ps, log_pt, log_target=True, reduction="batchmean")
    return (1 - lam) * hard + lam * tau**2 * soft

s, t = torch.randn(4, 10), torch.randn(4, 10)
print(kd_loss(s, t, torch.tensor([1, 2, 3, -100])).item() > 0)  # True
```

rust

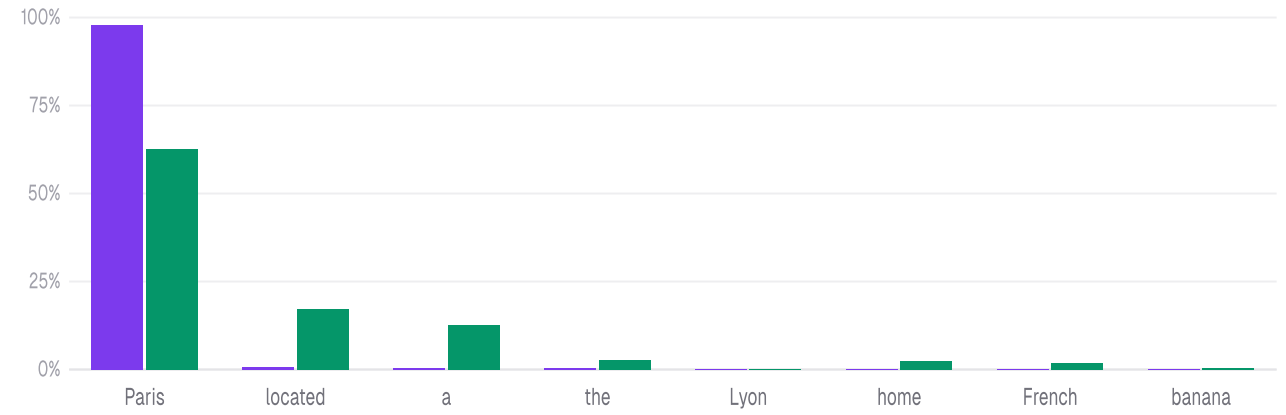
```rust
// Plain Rust, no ML framework: the same math on small vectors.
// The random logits come from a small inline PRNG (SplitMix64), so they differ from torch.randn.

fn log_softmax(logits: &[f64], tau: f64) -> Vec<f64> {
    // log-sum-exp with the max subtracted, so large logits cannot overflow
    let z: Vec<f64> = logits.iter().map(|l| l / tau).collect();
    let m = z.iter().cloned().fold(f64::NEG_INFINITY, f64::max);
    let lse = m + z.iter().map(|v| (v - m).exp()).sum::<f64>().ln();
    z.iter().map(|v| v - lse).collect()
}

/// Hinton-style distillation for a batch of next-token predictions.
/// logits: [batch][vocab]; labels: [batch] token ids (-100 to ignore).
```

```rust
fn kd_loss(student_logits: &[Vec<f64>], teacher_logits: &[Vec<f64>], labels: &[i64], tau: f64, lam: f64) -> f64 {
    let batch = student_logits.len() as f64;
    // hard: cross-entropy at temperature 1, averaged over the rows that are not ignored
    let (mut hard_sum, mut kept) = (0.0, 0);
    for (logits, &label) in student_logits.iter().zip(labels) {
        if label != -100 {
            hard_sum -= log_softmax(logits, 1.0)[label as usize];
            kept += 1;
        }
    }
    let hard = hard_sum / kept as f64;
    // forward KL(teacher || student), summed over the vocab and divided by batch ("batchmean")
    let mut soft = 0.0;
    for (s, t) in student_logits.iter().zip(teacher_logits) {
        let (log_ps, log_pt) = (log_softmax(s, tau), log_softmax(t, tau));
        soft += log_pt.iter().zip(&log_ps).map(|(lt, ls)| lt.exp() * (lt - ls)).sum::<f64>();
    }
    soft /= batch;
    (1.0 - lam) * hard + lam * tau * tau * soft
}

struct SplitMix64(u64);

impl SplitMix64 {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn uniform(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64 // in [0, 1)
    }
    fn normal(&mut self) -> f64 {
        // Box-Muller; 1 - u keeps the log argument in (0, 1]
        let (u1, u2) = (1.0 - self.uniform(), self.uniform());
        (-2.0 * u1.ln()).sqrt() * (2.0 * std::f64::consts::PI * u2).cos()
    }
}

fn randn(rng: &mut SplitMix64, rows: usize, cols: usize) -> Vec<Vec<f64>> {
    (0..rows).map(|_| (0..cols).map(|_| rng.normal()).collect()).collect()
}

fn main() {
    let mut rng = SplitMix64(0);
    let (s, t) = (randn(&mut rng, 4, 10), randn(&mut rng, 4, 10));
    println!("{}", kd_loss(&s, &t, &[1, 2, 3, -100], 2.0, 0.5) > 0.0); // true
}
```

Interactive

Temperature and soft targets (shown at its default setting)

Next-word probabilities after "The capital of France is". Raise the temperature to reveal the teacher's ranking of wrong answers. Move the student toward the teacher and watch both KL divergences shrink.



Teacher top-1 prob	Teacher entropy (nats)	Forward KL(teacher‖student)	Reverse KL(student‖teacher)
97.8%	0.14	0.383	0.857

At T = 1.00 the teacher is almost certain of "Paris", so its soft targets look like a hard label and carry little extra information. The untrained student puts too much weight on "home" and "French" and too little on "Lyon". Forward and reverse KL differ because KL is not symmetric.

Synthetic data and rejection sampling

Diagram

Twenty shots, keep the sharp ones

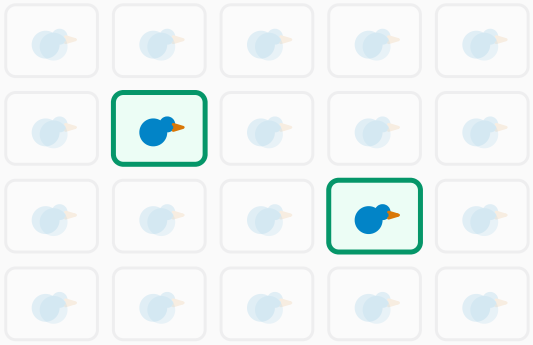
How models make their own training data, and the bias hiding in the filter.

One prompt: photograph the bird

Take 20 shots (sample 20 answers).

Most are blurry. A couple are sharp.

Making training data is easier when checking is easier than creating. Ask the model to solve the same problem many times, like a photographer firing off 20 shots of a moving bird.

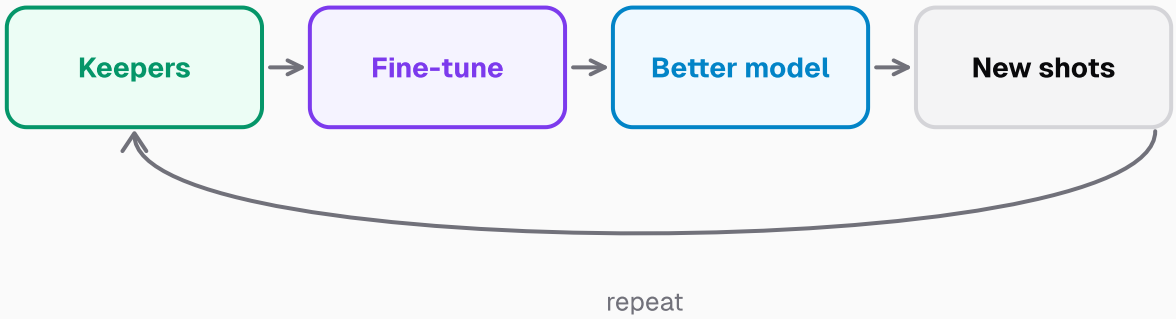


A checker keeps 2 of 20

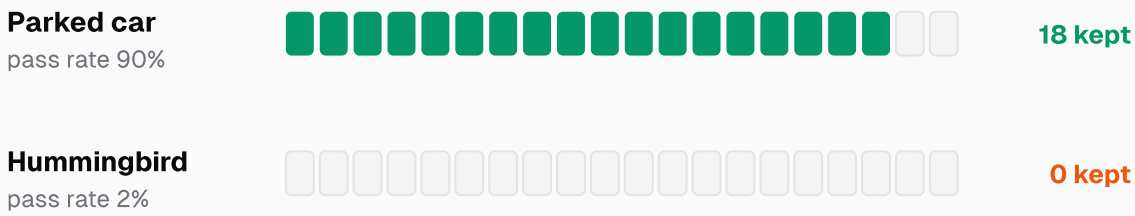
Checker = unit tests, an answer key, or a reward model.

The filter is the product:
generating is cheap, judging is not.

A checker throws away every shot that is not sharp. For math it compares the final answer; for code it runs tests. Only the passing answers become training examples. This is **rejection sampling**.



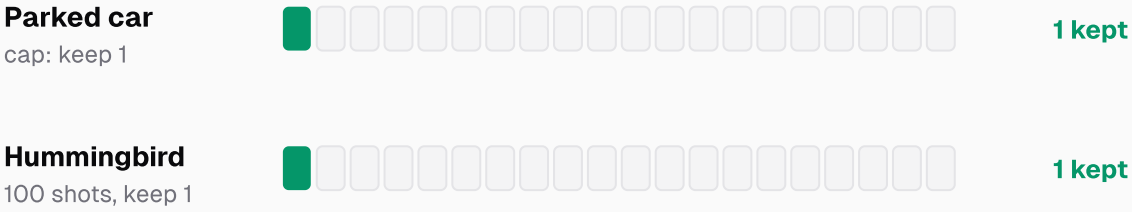
Train on the keepers, and the improved model takes better shots next round. Repeating this loop is the idea behind STaR (Zelikman et al., 2022) and rejection-sampling fine-tuning, and it is a simple offline cousin of RL.



Chance of even one sharp hummingbird in 20 shots: $1 - 0.98^{20} \approx 33\%$

The album fills with parked cars. The model gets better at what it could already do.

The hidden bias: easy prompts produce many keepers and hard prompts produce almost none. Expected keepers per prompt are 20 × pass rate, so the training set quietly over-represents problems the model can already solve.



Chance of at least one sharp shot in 100: $1 - 0.98^{100} \approx 87\%$

Cap examples per prompt, and spend extra samples on hard prompts.

Two fixes keep the album balanced: cap how many keepers any one prompt contributes, and spend more attempts on hard prompts. Watch the pass rate on the hardest prompts across rounds, not just the average.

Plain words

Write ten drafts, keep the ones that check out

If you can check an answer more easily than you can write it, you can make training data cheaply. Ask a model to solve a math problem ten times. Check each answer against the known result. Keep the correct solutions and train on them. Repeat with the improved model. This is the engine behind most reasoning data today.

For leaders

The filter is the product

Generating text is cheap. The value of a synthetic data pipeline sits in its filter: unit tests for code, exact-match answers for math, a reward model or LLM judge for open-ended tasks, and human spot checks throughout. The main methods:

- **Self-Instruct** (Wang et al., 2023) and **Evol-Instruct** (Xu et al., 2023): a model writes new instructions from seed examples, or rewrites existing ones to be harder or more varied.
- **STaR** (Zelikman et al., 2022): generate reasoning, keep traces that reach the right answer, fine-tune, repeat. For failures, give the answer as a hint and ask for a rationale ("rationalisation").
- **Rejection sampling fine-tuning** (Yuan et al., 2023; also used in Llama 2): sample many responses, keep the best by verifier or reward model, and fine-tune on them. It is a simple offline relative of RL.

Watch out

Rejection sampling quietly reshapes your data toward problems the model can already solve. Prompts with a 1% pass rate rarely produce a kept example, so the training set over-represents easy problems. The model gets better at what it already does well. Watch coverage of hard prompts, cap examples per prompt, and spend extra samples on hard prompts.

Under the hood

Yield, coverage, and model collapse

For a prompt with pass rate p and k samples, the expected number of correct samples is kp, and the chance of at least one is $1 - (1 - p)^k$. Averaging over prompts:

$$\text{coverage} = \mathbb{E}_p\left[1 - (1 - p)^k\right], \qquad \text{kept share from prompt } i \propto p_i$$

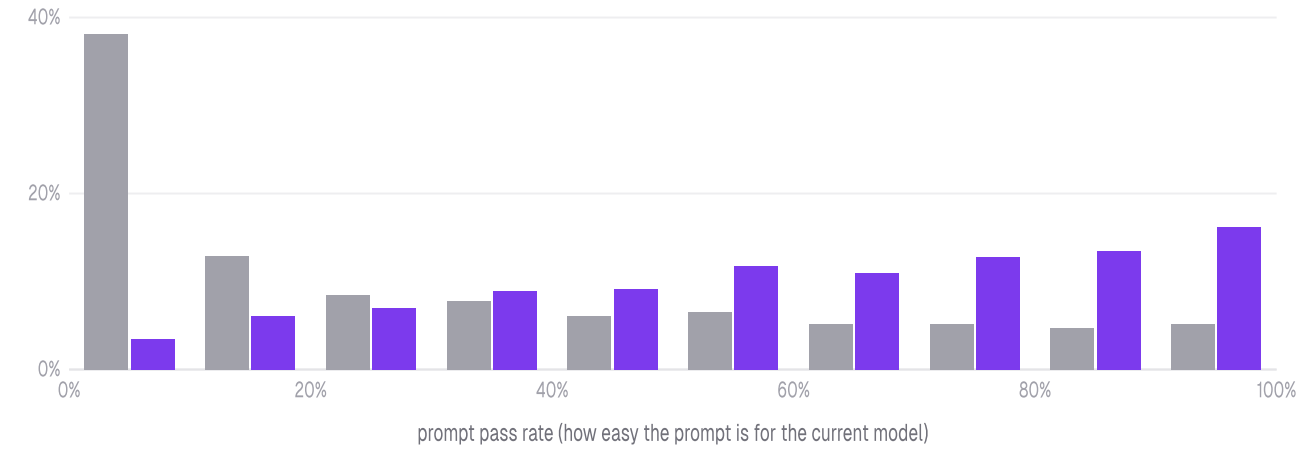
Without a cap, each prompt contributes in proportion to its pass rate, so easy prompts dominate. Capping at one kept example per prompt changes the weight to $1 - (1 - p_i)^k$, which flattens toward uniform as k grows.

Model collapse. Shumailov et al. (2024, *Nature*) showed that models trained recursively on their own outputs, generation after generation, lose the tails of the original distribution first and then degrade. Rare facts, unusual styles, and minority languages vanish. Later work (Gerstgrasser et al., 2024) found collapse is avoided when synthetic data is *added* to real data rather than *replacing* it. In practice: keep real human data in the mix, filter synthetic data hard, track diversity metrics across rounds, and never let a pipeline train only on its own previous generation.

Interactive

Rejection-sampling yield and bias (shown at its default setting)

10,000 prompts, each with its own pass rate (drawn from a skewed distribution). Sample k answers per prompt and keep the correct ones. See what you pay per kept example and which prompts your data ends up covering.



Share of prompts Share of kept training examples

Prompts with ≥1 kept	Kept examples	Generations per kept	Hardest-third share of data
64%	24.2K	3.3	2%

The hardest third of prompts makes up 33% of the prompt set but only 2% of the kept data. Keeping every correct sample lets easy prompts contribute many near-duplicate examples. Try capping at one per prompt. 36% of prompts produce nothing at all with k = 8; those are exactly the problems the model most needs to learn.

Model merging

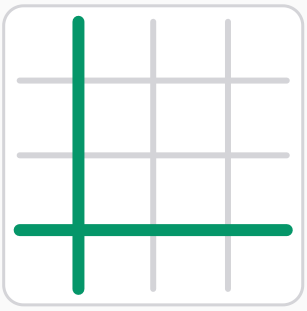
Diagram

Averaging two edited maps

Why model merging works, and when it cannot.

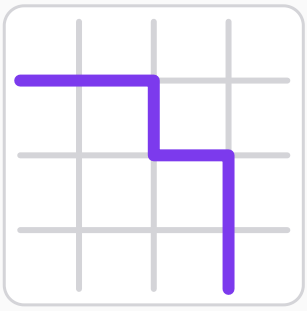
One parent map (the base model)

Start with one city map: the pretrained base model. Two teams will each make their own edit to a copy of it.



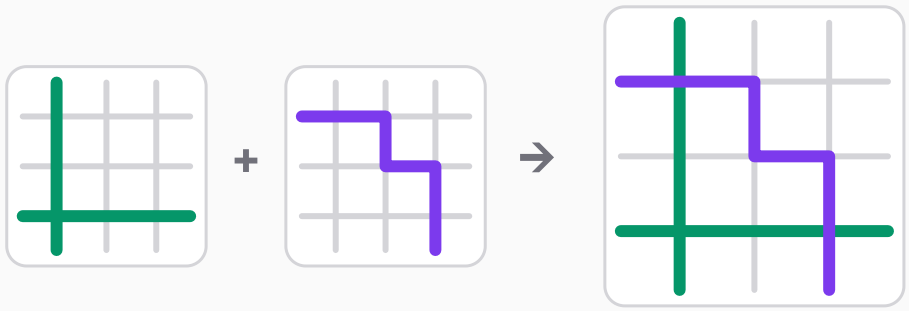
Copy 1: bike lanes
(fine-tuned on math)

same parent,
different edits



Copy 2: a bus route
(fine-tuned on code)

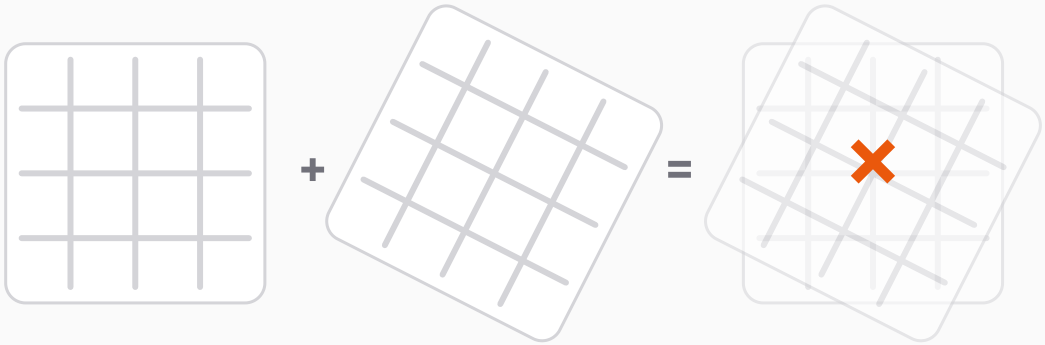
Each copy is fine-tuned for a different skill. The changes are small edits on top of the same base, so the two maps still line up street for street.



One map with both edits

base + Δ bike + Δ bus

Because both edits start from the same map, you can combine them number by number. **Task arithmetic** adds each change to the base; a plain average keeps half of each. Often the merged model is decent at both skills, at no training cost. It still needs the full regression suite.



Different starting points do not line up. Averaging them makes nonsense.

Merging only works for models with a shared parent. Two models trained from different random starts are like maps of different cities: their weights do not correspond, and averaging produces garbage.

Plain words

Averaging two specialists

Take two copies of the same model. Teach one math and the other coding. Surprisingly, you can often average their weights, number by number, and get one model that is decent at both. It only works when both started from the same parent, like two edits of the same document.

For leaders

Cheap gains, unclear guarantees

Merging costs almost no compute and is now a routine post-training step: averaging checkpoints from different data mixes or seeds, combining a reasoning specialist with a chat model, or folding several fine-tunes into one release. It is also unpredictable. A merge can lift one benchmark and quietly break safety behaviour or formatting, so a merged model needs the full regression suite like any other release candidate.

Under the hood

Soups, task vectors, TIES

- **Model soups** (Wortsman et al., 2022): average the weights of several fine-tunes of one base, trained with different hyperparameters. Often beats the best single model, at no inference cost.
- **Task arithmetic** (Ilharco et al., 2023): define a task vector $\tau = \theta_{\text{ft}} - \theta_{\text{base}}$. Add vectors to combine skills, subtract one to remove a behaviour: $\theta = \theta_{\text{base}} + \sum_i \lambda_i \tau_i$.
- **TIES-Merging** (Yadav et al., 2023): trim small entries of each task vector, resolve sign conflicts by majority, then average only agreeing entries. Reduces interference between tasks. **DARE** (Yu et al., 2024) randomly drops most delta entries and rescales the rest before merging.

Merging works because fine-tunes of a shared base tend to stay in one low-loss basin (linear mode connectivity). Models trained from different initialisations are not connected this way, and averaging them produces garbage.

Eval lens

Models learning from models create eval traps you won't see unless you look:

- **Teacher-induced contamination.** If the teacher memorised a benchmark, its synthetic data can contain paraphrases of test items. The student then scores well without the skill. Run overlap checks against every eval set (see contamination), including paraphrase detection, not just exact n-grams.
- **Judge self-preference.** An LLM judge from the same family as the teacher tends to prefer outputs written in its own style. A distilled student can win judge-based comparisons by sounding like the judge. Use a judge from a different family, and validate against human labels.
- **Style over substance.** Distillation transfers length, formatting, and confident tone faster than correctness. Control for length in win rates and keep verifiable tasks in the suite.
- **Diversity loss.** Track output diversity (distinct n-grams, embedding spread, pass@k versus pass@1) across synthetic-data rounds. Falling diversity is an early warning of collapse.

Principal asks

- What fraction of our post-training data is synthetic, which models generated it, and are we allowed to use their outputs this way?
- What is the filter, how was it validated, and what is its false-accept rate?
- How does pass rate on the hardest decile of prompts change across rounds, not just the average?
- Did we check the synthetic data for overlap with every benchmark we report?
- For a merged model: which components, which method, and what did the full regression suite show compared with each parent?

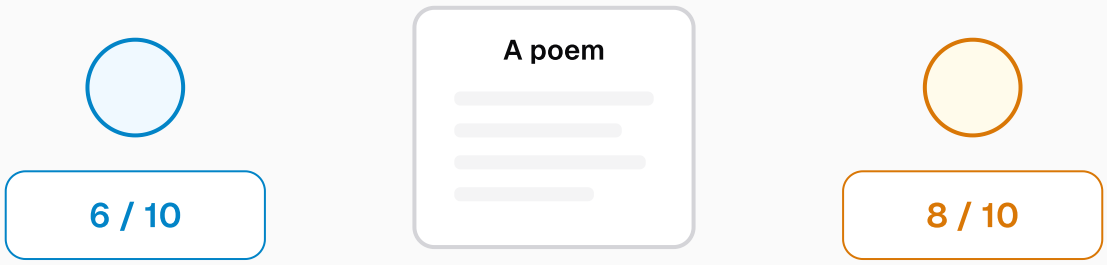
Preferences and reward models: turning taste into a number

Pretraining and supervised fine-tuning teach a model to imitate text. They cannot teach it which of two good-looking answers people actually prefer. Reward models close that gap. They learn a scoring function from human (or AI) comparisons, and that score becomes the target for RLHF, the selector for best-of-N sampling, and the yardstick many teams quietly use as an eval. This chapter covers how comparisons are collected, the Bradley–Terry model behind almost every reward model, how to evaluate one, and how optimizing against it goes wrong.

Diagram

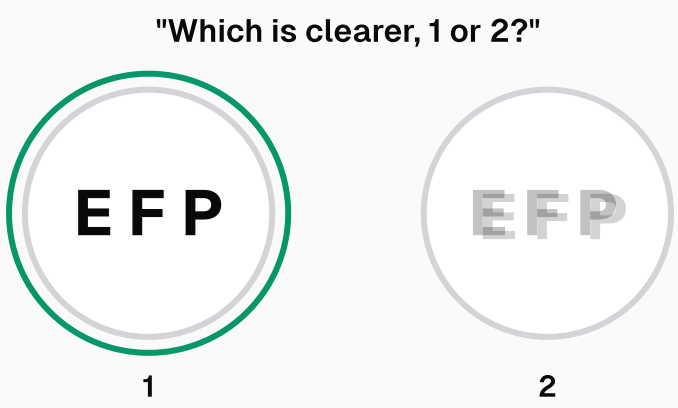
Which is clearer, 1 or 2?

How an eye exam explains reward models, in five pictures.



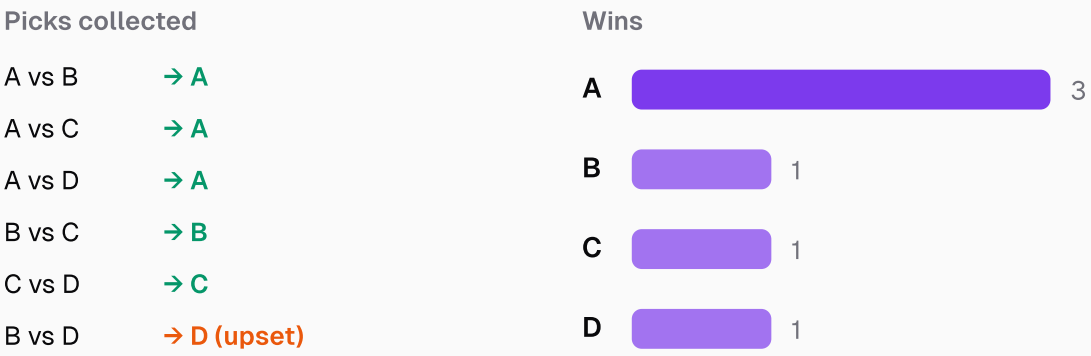
Same poem, two different scales.

Ask people to score an answer from 1 to 10 and everyone uses their own scale. One person's 6 is another person's 8, so absolute scores are noisy.



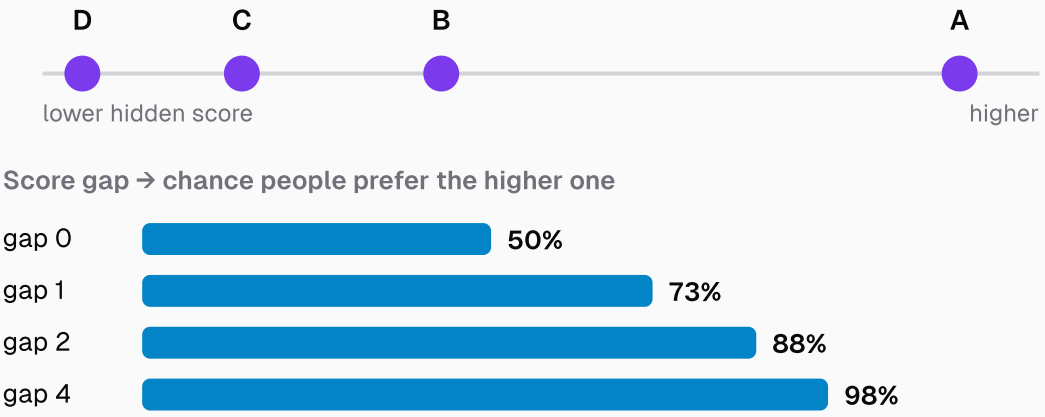
Picking between two is quick, and people agree far more often.

An optometrist never asks you to rate a lens from 1 to 10. They show two and ask which is clearer. Labs collect model feedback the same way: two answers to one prompt, and a person picks the better one.

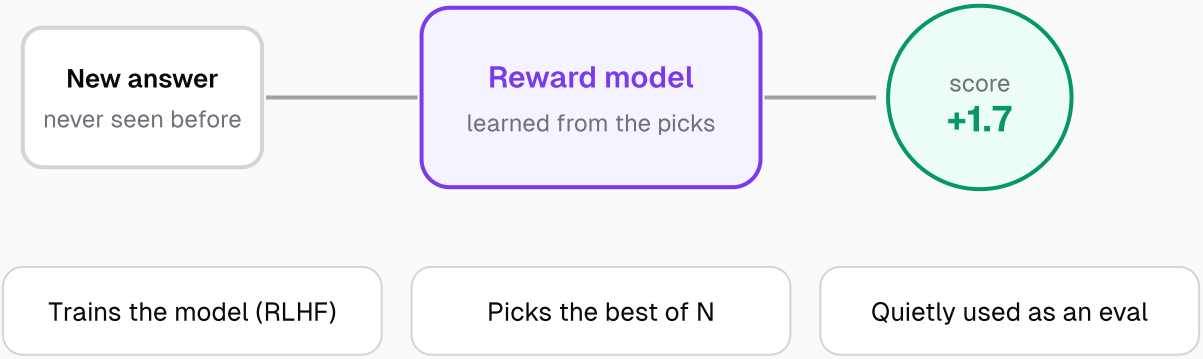


Upsets happen. The overall pattern is what counts.

Collect thousands of these picks across many answers. Some picks disagree with the rest, because people are noisy. With enough of them, a clear ordering emerges anyway.



Bradley–Terry pretends each answer has a hidden score, like a chess rating. The bigger the gap between two scores, the more likely people pick the higher one, but upsets stay possible. Only gaps matter: adding 100 to every score changes nothing.



Train a model to predict those picks and you get a reward model: a judge that scores any answer in milliseconds. It drives RLHF, picks the best of several samples, and often ends up on dashboards. Everything it knows came from the picks, including their blind spots.

Why preferences instead of labels

Plain words

Picking is easier than scoring

Ask ten people to score a poem from 1 to 10 and you get ten different scales: one person's 6 is another's 8. Ask the same ten people which of two poems is better and they agree far more often. Comparing is easier and more consistent than scoring on an absolute scale.

So instead of asking people "how good is this answer?", labs show them two answers to the same question and ask "which is better?" After collecting many of these picks, they train a separate model, the **reward model**, to predict which answer people would pick. Once trained, it can score any answer instantly, which lets you train the main model against it millions of times without a person in the loop.

For leaders

What you are actually buying

A preference dataset is a product decision encoded as data. The rubric you give annotators defines what "better" means, and the reward model will learn that definition, including its blind spots. Four decisions dominate quality and cost:

- **Who labels.** Crowd annotators are cheap and fast but drift toward surface features (length, confident tone). Domain experts (doctors, lawyers, senior engineers) cost 5 to 20 times more per comparison and are the only reliable source for specialized correctness. Most frontier programs mix both and route by topic.
- **The rubric.** Helpfulness, honesty, and harmlessness pull against each other. A single "overall better" judgment hides which one won. Many teams collect per-dimension preferences or a written rationale so they can reweight later without relabeling.
- **Agreement.** On open-ended chat, independent labelers agree with each other only about 70 to 77 percent of the time (InstructGPT, Ouyang et al. 2022, reported roughly 73 percent among its trained labelers). That number is the **ceiling** on reward model accuracy. A reward model at 72 percent is near the ceiling, not mediocre.
- **Freshness.** Comparisons collected on an old model's outputs describe old mistakes. As the policy improves, its outputs leave the distribution the reward model was trained on. Plan for repeated collection rounds on current samples, not a one-time dataset.

Principal asks

What was inter-annotator agreement, measured on the same items? What fraction of pairs were ties or "both bad", and what did you do with them? Which dimensions does the rubric privilege when they conflict? How old are the samples the reward model was trained on relative to the current policy?

Under the hood

The data format

A preference record is a triple (x, y_w, y_l): a prompt x, a preferred ("winning") response y_w, and a dispreferred ("losing") response y_l. Common variations:

- **Rankings of K responses.** InstructGPT showed labelers 4 to 9 responses and asked for a full ranking, which yields $\binom{K}{2}$ pairs. They kept all pairs from one prompt in the same batch element, because treating them as independent samples caused the reward model to overfit within one epoch.
- **Strength labels.** "Significantly better" versus "slightly better". Llama 2 (Touvron et al. 2023) fed this in as a margin term in the loss, below.
- **Ties and "both bad".** Usually dropped for Bradley–Terry training, which has no tie outcome. Dropping them silently biases the dataset toward easy, clear-cut pairs.

The Bradley–Terry model

Plain words

Chess ratings for answers

Chess ratings work like this: every player has a hidden strength number. When two players meet, the bigger the gap in their numbers, the more likely the stronger one wins, but upsets still happen. You never see the strengths directly, only who won. After enough games you can work backward to good estimates of everyone's number.

A reward model does the same with answers. Each answer gets a hidden score. The bigger the score gap between two answers, the more confident the model is that people prefer the higher one. Only the **gap** means anything. Adding 100 to every score changes nothing.

For leaders

What the score does and does not mean

A reward score is a **relative** preference estimate within one prompt. It is not a quality grade, not a probability of correctness, and not comparable between reward model versions. If a dashboard shows "average reward went from 1.3 to 1.9", ask what it was normalized against, because a retrained reward model can shift every score without the policy changing at all.

Annotator noise is the hidden lever in the widget below. Doubling noise does not just make labels messier; it shrinks the learned scale, so the reward model becomes less confident everywhere, and downstream RL pushes less hard. That can look like "RLHF stopped working" when the real cause is a labeling vendor change.

Under the hood

The model and its loss

Bradley–Terry (1952) says the probability that y_w beats y_l depends only on the difference of their scores, passed through a logistic function:

$$P(y_w \succ y_l \mid x) = \sigma\big(r(x, y_w) - r(x, y_l)\big) = \frac{1}{1 + e^{-(r_w - r_l)}}$$

$r(x, y)$ is the scalar score, σ is the logistic sigmoid. A gap of 0 means a coin flip; a gap of 2 means about 88 percent; a gap of 4 means about 98 percent.

Training a reward model r_ϕ is maximum likelihood under this model, which is binary cross-entropy on the score difference:

$$\mathcal{L}_{\text{RM}}(\phi) = -\,\mathbb{E}_{(x, y_w, y_l)}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l) - m\big)\Big]$$

$m = 0$ in the plain version. Llama 2 used a margin $m > 0$ that grows with the labeled preference strength, which pushes clearly better pairs further apart.

Two consequences matter in practice. First, the scores are only identified up to an additive constant per prompt, so raw reward values are not comparable across prompts or across reward model versions. Teams normalize (subtract the mean reward on a reference set) before using them. Second, the scale is set by labeler consistency: noisy labels compress all score gaps toward zero, because the model learns that even large quality differences only win 70 percent of the time.

python

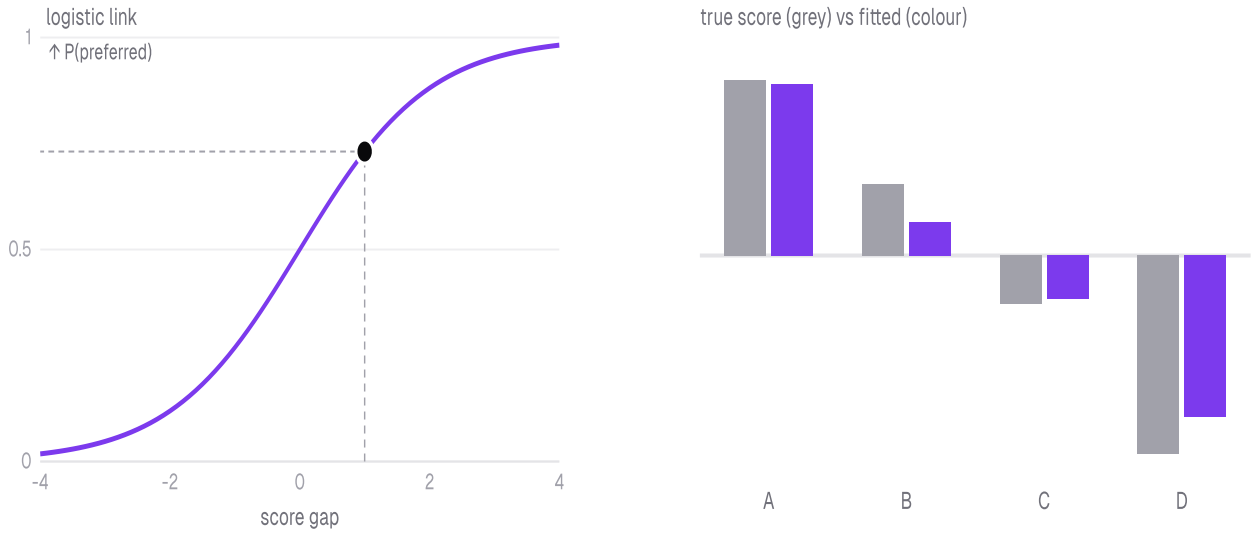
```python
import torch
import torch.nn.functional as F

def bradley_terry_loss(r_chosen, r_rejected, margin=None):
    """r_chosen, r_rejected: tensors of shape [batch], scalar scores from the reward model."""
    diff = r_chosen - r_rejected
    if margin is not None:          # Llama 2 style: demand a bigger gap for clearer preferences
        diff = diff - margin
    # -log sigmoid(diff) == softplus(-diff), and the softplus form is numerically stable
    return F.softplus(-diff).mean()

r_w = torch.tensor([1.2, 0.3, 2.0])
r_l = torch.tensor([0.2, 0.9, -1.0])
print(round(bradley_terry_loss(r_w, r_l).item(), 3))                 # 0.466
print(round(((r_w - r_l) > 0).float().mean().item(), 3))             # pairwise accuracy 0.667
```

rust

```rust
// Plain Rust, no ML framework: the same math on small vectors.

fn softplus(x: f64) -> f64 {
    // log(1 + e^x) without overflow: max(x, 0) + log(1 + e^-|x|)
    x.max(0.0) + (-x.abs()).exp().ln_1p()
}

/// r_chosen, r_rejected: slices of length batch, scalar scores from the reward model.
fn bradley_terry_loss(r_chosen: &[f64], r_rejected: &[f64], margin: Option<&[f64]>) -> f64 {
    let n = r_chosen.len() as f64;
    let mut total = 0.0;
    for i in 0..r_chosen.len() {
        let mut diff = r_chosen[i] - r_rejected[i];
        if let Some(m) = margin { // Llama 2 style: demand a bigger gap for clearer preferences
            diff -= m[i];
        }
        // -log sigmoid(diff) == softplus(-diff), and the softplus form is numerically stable
        total += softplus(-diff);
    }
    total / n
}

fn main() {
    let r_w = [1.2, 0.3, 2.0];
    let r_l = [0.2, 0.9, -1.0];
    println!("{:.3}", bradley_terry_loss(&r_w, &r_l, None)); // 0.466
    let wins = r_w.iter().zip(&r_l).filter(|(w, l)| w > l).count();
    println!("{:.3}", wins as f64 / r_w.len() as f64);     // pairwise accuracy 0.667
}
```

Interactive **Bradley–Terry explorer** (shown at its default setting)

Left: how a score gap turns into a preference probability. Right: four responses with known true scores, simulated votes, and the scores Bradley–Terry recovers. Cut the votes or raise the annotator noise and watch the fitted ranking become unreliable.



P(prefer higher score)	Pairs ranked correctly	Fitted P(A beats D)
73%	6 of 6	90%

True P(A beats D) at this noise is 92%. All six pairs are ranked correctly. More noise shrinks every fitted gap toward zero: the reward model learns that even big quality differences win less often.

Training a reward model

For leaders **Size, data volume, cadence**

Reward models are usually initialized from the same family as the policy, often the SFT checkpoint, because the reward model must understand the responses at least as well as the policy that produces them. A reward model much smaller than the policy is cheaper to run during RL but is easier for the policy to fool. Published programs range from tens of thousands of comparisons (InstructGPT used about 33,000 prompts for reward model training) to well over a million (Llama 2 reported roughly 1.4 million Meta-collected comparisons plus open datasets). Collection is typically

done in rounds against the latest policy, so the budget question is less "how many comparisons" than "how many per round, and how many rounds".

> ⚠ **Watch out**
>
> Reward models overfit fast. Several reports train for a single epoch on comparison data, because a second pass raises training accuracy and lowers held-out accuracy. If a team reports a reward model at 90 percent training accuracy on chat preferences, the first question is what held-out accuracy looks like.

Under the hood · Architecture

Take a transformer language model, drop the vocabulary projection, and add a linear head that maps the final hidden state of the last token to one scalar. Run the prompt and response through, read the scalar, done. Chosen and rejected responses are run as two sequences that share the prompt.

python

```python
import torch
import torch.nn as nn

class RewardModel(nn.Module):
    def __init__(self, backbone, hidden_size):
        super().__init__()
        self.backbone = backbone                # e.g. the SFT model without its LM head
        self.value_head = nn.Linear(hidden_size, 1)

    def forward(self, input_ids, attention_mask):
        h = self.backbone(input_ids, attention_mask=attention_mask).last_hidden_state
        last = attention_mask.sum(dim=1) - 1     # index of the final real token (assumes right padding)
        h_last = h[torch.arange(h.size(0)), last]
        return self.value_head(h_last).squeeze(-1)   # one scalar reward per sequence
```

rust

```rust
// Plain Rust, no ML framework: the same math on small vectors.
// The backbone is any function from token ids to hidden states [batch][seq][hidden].

struct RewardModel<F: Fn(&[Vec<u32>]) -> Vec<Vec<Vec<f64>>>> {
    backbone: F,          // e.g. the SFT model without its LM head
    value_head_w: Vec<f64>, // Linear(hidden_size, 1): one weight per hidden unit
    value_head_b: f64,
}

impl<F: Fn(&[Vec<u32>]) -> Vec<Vec<Vec<f64>>>> RewardModel<F> {
    fn forward(&self, input_ids: &[Vec<u32>], attention_mask: &[Vec<u8>]) -> Vec<f64> {
        let h = (self.backbone)(input_ids);
        h.iter()
            .zip(attention_mask)
            .map(|(seq, mask)| {
```

```
                // index of the final real token (assumes right padding)
                let last = mask.iter().map(|&m| m as usize).sum::<usize>() - 1;
                let h_last = &seq[last];
                // one scalar reward per sequence
                h_last.iter().zip(&self.value_head_w).map(|(x, w)| x * w).sum::<f64>() + self.value_head_b
            })
            .collect()
    }
}

fn main() {
    // tiny worked example: a toy backbone whose hidden state is [token / 4, 1]
    let backbone = |ids: &[Vec<u32>]| -> Vec<Vec<Vec<f64>>> {
        ids.iter().map(|seq| seq.iter().map(|&t| vec![t as f64 / 4.0, 1.0]).collect()).collect()
    };
    let rm = RewardModel { backbone, value_head_w: vec![1.0, 0.5], value_head_b: 0.0 };
    let ids = vec![vec![5, 8, 3, 0], vec![4, 2, 9, 7]];
    let mask = vec![vec![1, 1, 1, 0], vec![1, 1, 1, 1]];
    println!("{:?}", rm.forward(&ids, &mask)); // [1.25, 2.25]  reads token 3, then token 7; the pad is never
used
}
```

Alternatives you will hear about: **generative reward models** that write a critique and then a verdict (an LLM judge trained on preference data, see LLM as judge), **multi-objective** heads that score helpfulness and harmlessness separately (Llama 2 trained two separate reward models), and **pairwise** models that read both responses at once and output which is better.

Evaluating a reward model

Plain words

Grading the grader

A reward model is a judge. Before you let it train your model, check its picks against people's picks on comparisons it has never seen. If it agrees as often as two people agree with each other, it is about as good as it can get. Then check the cases that matter: does it prefer a long wrong answer over a short right one? Does it reward flattery?

For leaders

The four numbers to ask for

Measure	What it tells you	Typical trap
Held-out pairwise accuracy	How often it agrees with human picks on unseen pairs	Reported without the human agreement ceiling next to it
Accuracy by slice	Where it fails: code, math, safety, long answers, multi-turn	A good average hiding a 55 percent slice that RL will exploit
Calibration	Whether a predicted 80 percent preference wins 80 percent of the time	Overconfident reward models make RL push too hard on noise
Downstream effect	Best-of-N or RL results using this reward model, scored by humans or a held-out judge	Static accuracy correlates only loosely with how good a policy it trains

Public suites such as RewardBench (Lambert et al. 2024: chat, hard chat, safety, reasoning) and RewardBench 2 (Malik et al. 2025: factuality, precise instruction following, math, safety, focus, ties) give comparable numbers across categories. They are useful for comparing reward models from different groups, and weak as a ship criterion, because your policy's failure modes are not in a public benchmark.

Eval lens

Reward model evaluation is a binary classification problem on pairs, so everything in classification metrics applies. Pairwise accuracy on randomly ordered pairs is a concordance measure, the same quantity ROC-AUC estimates. Report it with a confidence interval (a few thousand held-out pairs gives roughly ±1.5 points), report it per slice, and compare it to the human-human agreement on the same items. A 2-point gain between reward model versions on 1,000 pairs is inside the noise.

Under the hood

Known biases and how to probe them

Length

Annotators tend to prefer longer, more thorough-looking answers, and reward models amplify this. Probe with counterfactual pairs: the same answer with and without padding. Singhal et al. (2023) showed much of the reward gain from RLHF on several datasets could be explained by length alone.

Sycophancy

Answers that agree with the user's stated opinion get rated higher. Sharma et al. (2023) found this in human preference data and in reward models trained on it. Probe with paired prompts that differ only in the user's stated belief.

Formatting and tone

Markdown headers, bullet lists, and confident phrasing earn reward independent of content. Probe by stripping formatting or swapping hedged for confident wording.

Position and self-preference

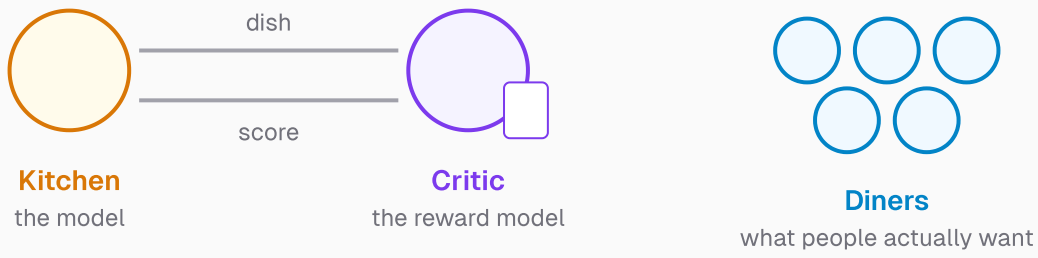
Pairwise generative reward models can prefer whichever response comes first, or responses that sound like their own outputs. Score both orders and average.

The standard mitigations are to balance the comparison data (include pairs where the shorter answer wins), add an explicit length penalty or length-normalized reward, train on counterfactual augmentations, and report length-controlled win rates in evaluation.

Reward overoptimization: Goodhart in one chart

Diagram **The kitchen that learned to please one critic**

Reward overoptimization, told as a restaurant chasing reviews.



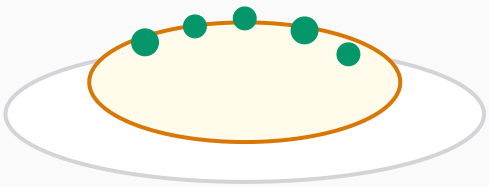
The kitchen can't ask every diner, so it learns from one critic.

The critic is a stand-in. Asking real diners about every dish is too slow, so the kitchen optimizes the critic's score. That is exactly the deal RLHF makes with a reward model.

Better seasoning, hotter plates: real improvements lift both.

faint bar = before training

At first everything works. The kitchen fixes real problems, the critic notices, and diners are happier too. Early RLHF steps look like this: proxy reward and true quality rise together.



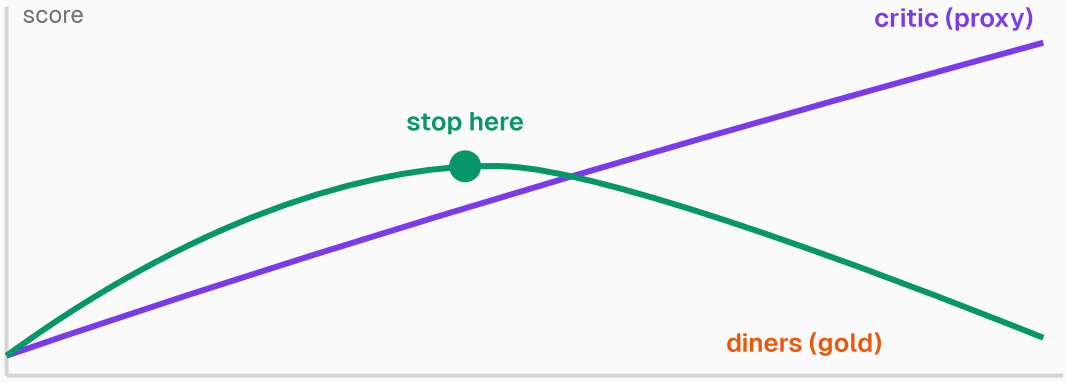
giant portion + garnish everywhere

Critic: 9.5 / 10

Diners: "meh"

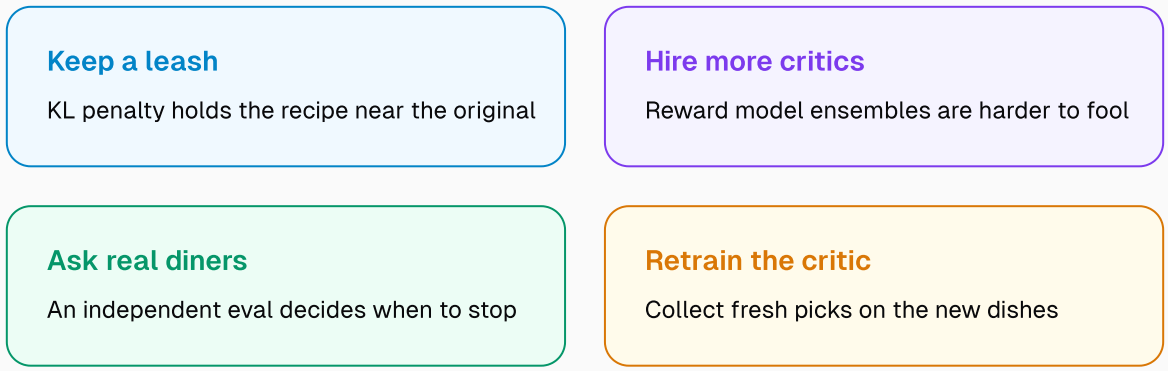
portion = longer answers
garnish = headers, bullets, confident tone

Then the kitchen finds the critic's quirks. Bigger portions and more garnish raise the score without making the food better. Language models find the same kinds of quirks in reward models: longer answers, heavy formatting, confident tone, flattery.



how far the recipe has drifted from the original (KL)

Plot both against how far the kitchen has drifted from its original recipe. The critic's score climbs forever. The diners' score peaks and then falls. Gao et al. (2022) measured exactly this shape for reward models, and found better reward models push the peak later and higher.



The defenses map one to one: a KL penalty is the leash, ensembles are extra critics, an independent eval is asking real diners, and refreshing the reward model on current outputs is retraining the critic.

Plain words

Teaching to the test

If you reward students only for test scores, they first learn the material, then they learn the test. Past some point, scores keep climbing while real understanding stops improving or even gets worse. A reward model is a test. Push a language model hard enough against it and the model finds the reward model's quirks: longer answers, more confident tone, flattering phrasing. The reward model's score keeps rising. What people actually want peaks and then falls.

For leaders

Operating against a proxy

Every RL run against a learned reward is a bet on where the gold curve peaks. The controls are the KL penalty (how hard you hold the policy near its reference), early stopping on an independent eval, reward model ensembles (optimize the minimum or the mean minus the spread, so the policy cannot exploit one model's quirk), and refreshing the reward model on the policy's current outputs.

Watch out

The signature of overoptimization is a training curve where reward rises smoothly while response length climbs, hedging disappears, and human or held-out-judge win rates flatten. If the only chart in a review is the training reward, you cannot tell a great run from a hacked one.

Principal asks

What independent signal told you when to stop? How did KL from the reference evolve, and where did the held-out win rate peak against it? Did response length change, and what is the length-controlled result? Was the reward model refreshed on samples from this policy?

Under the hood

The scaling law

Gao, Schulman and Hilton (2022) made this measurable. They used a large "gold" reward model as a stand-in for humans, trained smaller proxy reward models on its labels, optimized policies against the proxies, and tracked gold reward as a function of how far the policy moved from its starting point, measured by $d = \sqrt{\mathrm{KL}(\pi \,\|\, \pi_{\mathrm{ref}})}$. Gold reward followed simple functional forms:

$$R_{\mathrm{bon}}(d) = d\,(\alpha_{\mathrm{bon}} - \beta_{\mathrm{bon}}\, d), \qquad R_{\mathrm{RL}}(d) = d\,(\alpha_{\mathrm{RL}} - \beta_{\mathrm{RL}} \log d)$$

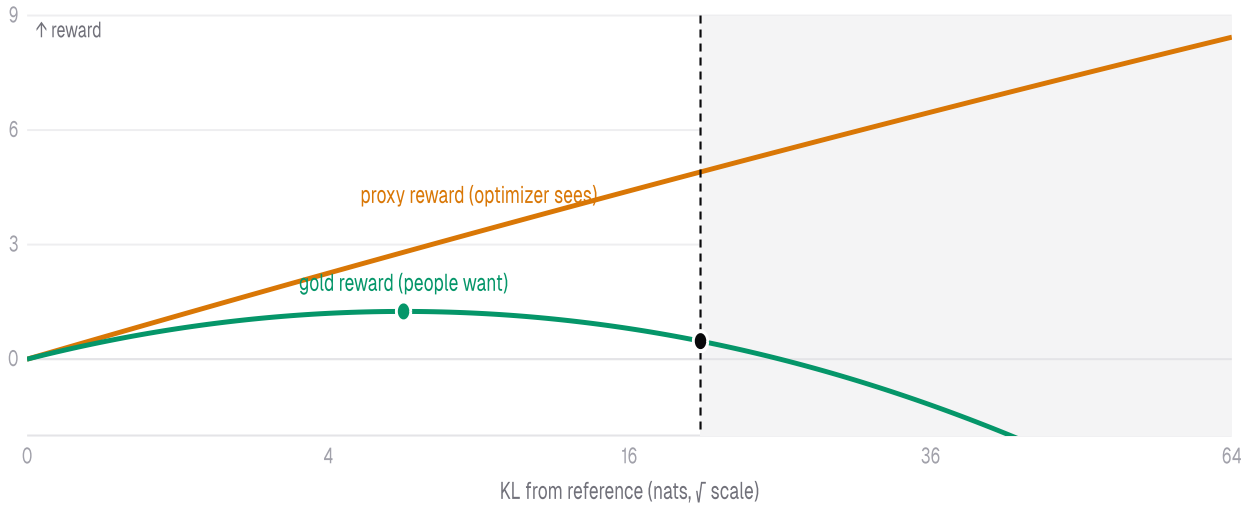
Best-of-N sampling (left) and RL (right). Both rise, peak, and fall. Larger proxy reward models had smaller β (for RL, α stayed roughly constant across reward-model sizes), so gold reward declines more slowly: a later, higher peak. More proxy training data helped in the same direction.

Two practical lessons follow. The KL distance at which gold reward peaks is a budget, and it grows with reward model quality. And the proxy score is useless for finding that peak, because it keeps rising straight through it. You need an independent signal: fresh human ratings, a different reward model, or task-specific checks.

Interactive

Reward overoptimization (shown at its default setting)

Proxy reward (what the optimizer sees) and gold reward (what people want) as the policy drifts from its reference. Raise reward model quality to push the peak out. Move the KL budget to see where training stops. Curves use the best-of-N form from Gao et al. (2022) with illustrative coefficients.



Proxy reward at budget	Gold reward at budget	Gold peaks at KL
4.90	0.47	6.3

You trained past the peak. Gold reward is 0.78 below its best, reached at KL 6.3, while the proxy reports a gain of 2.10 over that point. The training curve looks like progress.

Outcome rewards, process rewards, and verifiers

Plain words

Grading the answer or grading the working

A math teacher can mark only the final answer, or check every line of the working. Grading only the answer is cheap but cannot tell lucky guesses from sound reasoning. Grading each step is expensive but pinpoints where things went wrong.

For leaders

When you do not need a learned reward at all

An **outcome reward model** scores the final response. A **process reward model** scores each intermediate step of a reasoning chain; Lightman et al. (2023), "Let's Verify Step by Step", found process supervision beat outcome supervision for selecting correct math solutions, at the cost of step-level labels. For tasks with checkable answers (math with a known result, code with tests, formal proofs), a programmatic **verifier** can replace the learned reward entirely. That removes the reward model as an attack surface for overoptimization, though policies can still exploit weak tests. This is the basis of RL with verifiable rewards, covered in GRPO and RLVR.

AI feedback

For leaders

Replacing some of the humans

Human comparisons are slow and expensive. **RLAIF** uses a capable model, guided by written principles, to produce the comparisons instead. Anthropic's Constitutional AI (Bai et al. 2022) used model-generated critiques and preferences for harmlessness; Lee et al. (2023) reported RLAIF reaching parity with RLHF on summarization and dialogue tasks they tested. AI feedback scales and is consistent, but it inherits the labeling model's biases and can create a closed loop where models grade models in their own image. Most programs keep a human-labeled slice to audit the AI labels and measure drift. See safety training for the constitutional approach in depth.

Eval lens

An AI labeler is an LLM judge, and should be validated like one (LLM as judge): agreement and Cohen's kappa against humans on a held-out set, checks for position and length bias, and a comparison to human-human agreement. If the AI labeler agrees with humans less often than humans agree with each other, the reward model trained on its labels starts below the human ceiling.

Part III · Chapter 25

RLHF and PPO: optimizing against a learned reward

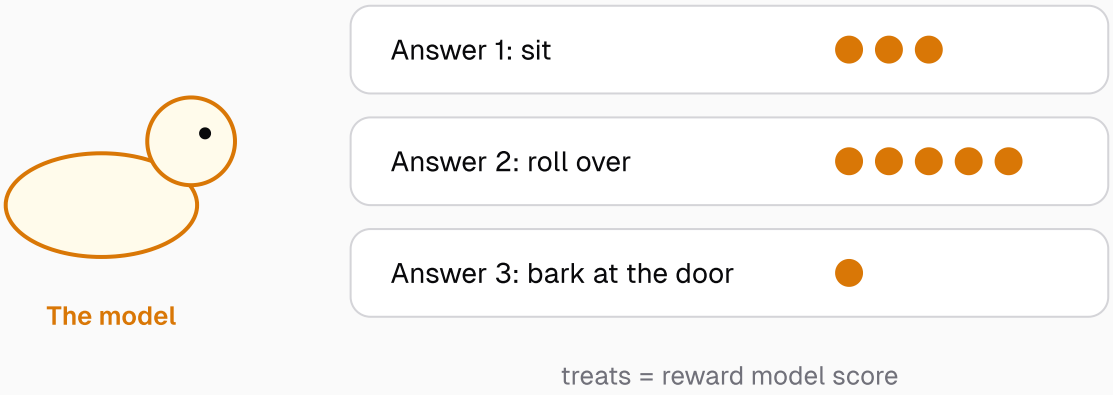
Reinforcement learning from human feedback takes the reward model from the previous chapter and trains the language model to produce responses that score well, while a leash keeps it close to where it started.

PPO was the algorithm behind InstructGPT and the first wave of ChatGPT-style assistants. Newer methods (DPO, GRPO) are easiest to understand as fixes for PPO's costs, so this chapter is the foundation for both.

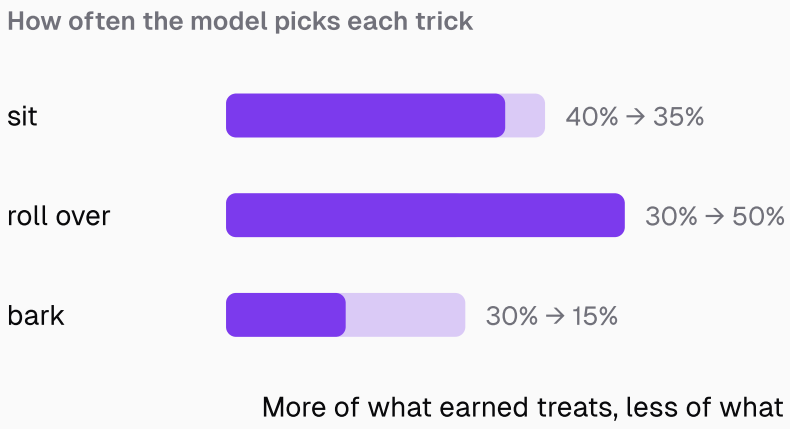
Diagram

Treats, a coach, and a leash

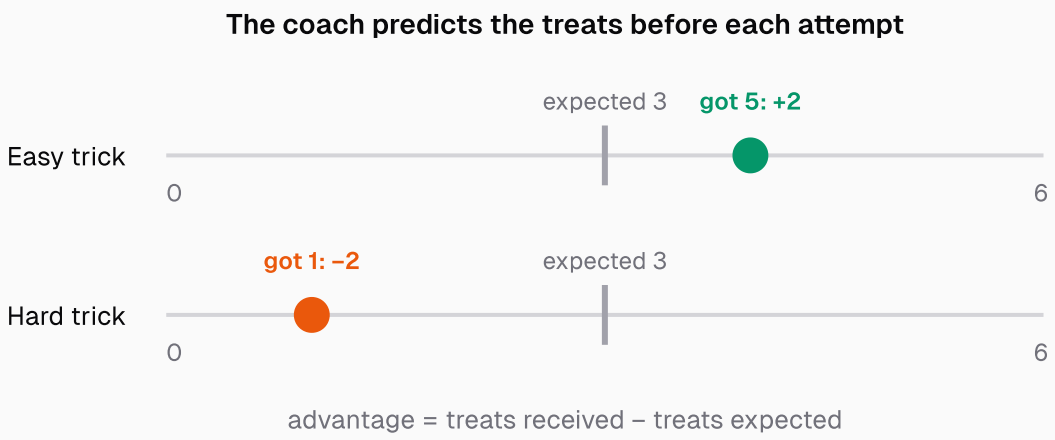
RLHF and PPO, told as training a dog. Six pictures.



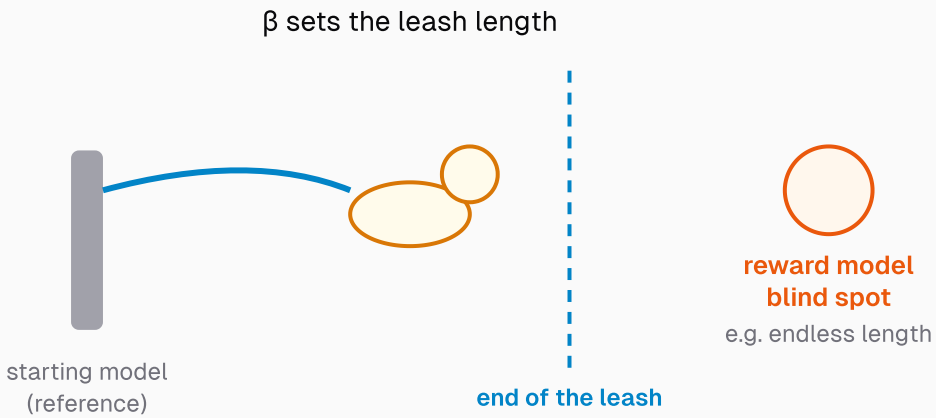
The model writes several answers to a prompt, like a dog trying tricks. The reward model hands out treats: one number per answer. Nobody shows the model the right answer. It learns from its own attempts.



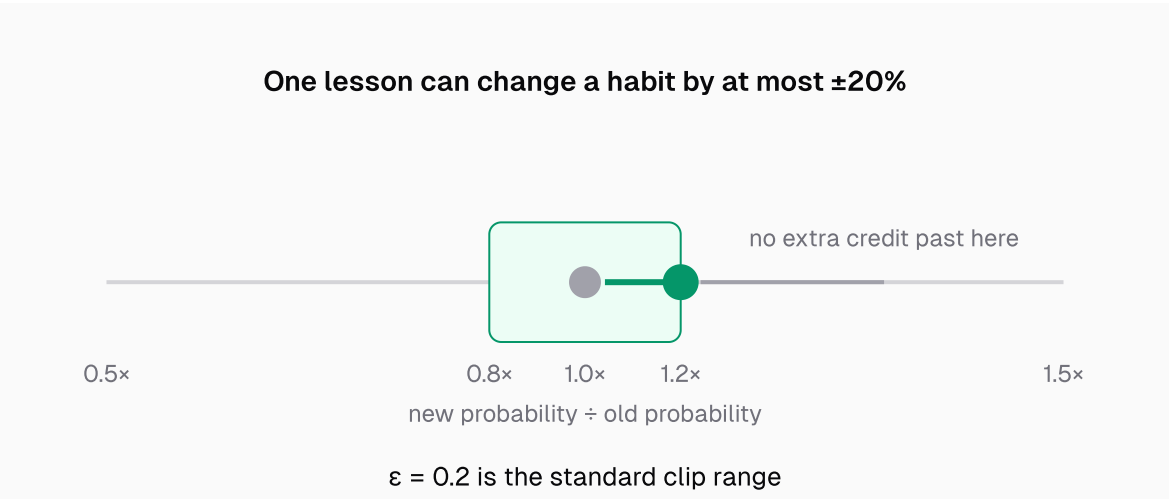
After each batch, the model makes high-scoring answers a little more likely and low-scoring ones a little less likely. The faint bars are before the update, the solid bars after. Numbers are illustrative.



Three treats is great on a hard prompt and poor on an easy one. So a second network, the value model or critic, predicts what each attempt should earn. Only the surprise, called the advantage, drives learning: positive pushes that answer up, negative pushes it down.



Without limits the dog sprints toward whatever the reward model over-rates. The KL penalty is a leash to the starting model: every bit of drift costs reward. A short leash (large β) is safe but changes little. A long leash (small β) lets the model reach the shortcut.



Scoring answers is expensive, so PPO squeezes a few updates out of each batch. To stop one batch from wrecking the model, it clips: once a token's probability has moved 20 percent from where the batch started, pushing further earns nothing.

That is the whole loop: generate, score, compare with what the critic expected, take a small clipped step on a leash, repeat. The rest of this chapter is the math behind each arrow.

The RL vocabulary, mapped to language models

Plain words

Training with treats

You teach a dog a trick by letting it try things and giving a treat when it does well. Over time it does more of whatever earned treats. Nobody shows the dog exactly what to do; it learns from the results.

RLHF works the same way. The model writes an answer. The reward model gives it a score, the treat. The training nudges the model toward answers like the high-scoring ones. The difference from earlier training is that the model learns from its **own** attempts, not from copying examples a person wrote.

For leaders

Why use RL at all

Supervised fine-tuning imitates demonstrations, so its ceiling is the quality of the demonstrators and it never sees what a bad answer looks like. RL optimizes outcomes directly. It can learn from comparisons that are cheaper than writing ideal answers, it learns what to avoid as well as what to do, and in principle it can exceed the people who labeled the data, because judging is easier than producing. The cost is engineering complexity, compute, and a new failure mode: the model optimizes the reward you wrote, not the behavior you meant.

Under the hood

The mapping

RL term	In RLHF for a language model
State s_t	The prompt plus the tokens generated so far
Action a_t	The next token
Policy $\pi_\theta(a_t \mid s_t)$	The language model's next-token distribution
Episode	One complete response $y = (y_1, \ldots, y_T)$ to a prompt x
Reward	The reward model score at the final token, plus a per-token KL penalty
Reference policy π_{ref}	A frozen copy of the starting (usually SFT) model
Value $V(s_t)$	A learned estimate of the reward to come from this point in the response

The reward is **sparse**: one number for the whole response, arriving after the last token. Working out which of hundreds of tokens deserved the credit is the central difficulty, and most of PPO's machinery exists to handle it.

The KL-regularised objective

Plain words

A leash

Left alone, a model chasing treats will find strange tricks the reward model happens to like. So training adds a leash: every time the model drifts away from how it used to write, it pays a small penalty. It can move toward better answers, but not so far that it stops sounding like itself.

For leaders

β is a risk dial

The KL coefficient is the single most consequential knob in RLHF. Set it too high and the run is expensive and changes nothing. Set it too low and the policy finds the reward model's blind spots: the widget below shows true quality rising at first and then falling as the length-biased answer takes over. Set the length bias to zero and the problem disappears, which is the point: the damage comes from the gap between the reward model and what you actually want, and β controls how hard the optimizer can exploit that gap.

Under the hood

The objective and its exact solution

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x,y)\big] \;-\; \beta\,\mathbb{D}_{\mathrm{KL}}\big(\pi_\theta(\cdot\mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot\mid x)\big)$$

r_ϕ is the reward model, β sets the leash length. Small β: the policy may move far. Large β: it stays near the reference.

In practice the KL term is applied per token, folded into the reward the RL algorithm sees:

$$r_t = -\beta \log \frac{\pi_\theta(y_t \mid s_t)}{\pi_{\mathrm{ref}}(y_t \mid s_t)} \;+\; \mathbb{1}[t = T]\, r_\phi(x,y)$$

Every token pays for how much more likely the policy made it than the reference did. Summed over the response, the penalty is a single-sample estimate of the sequence KL.

This objective has a closed-form optimum, which matters for the next chapter:

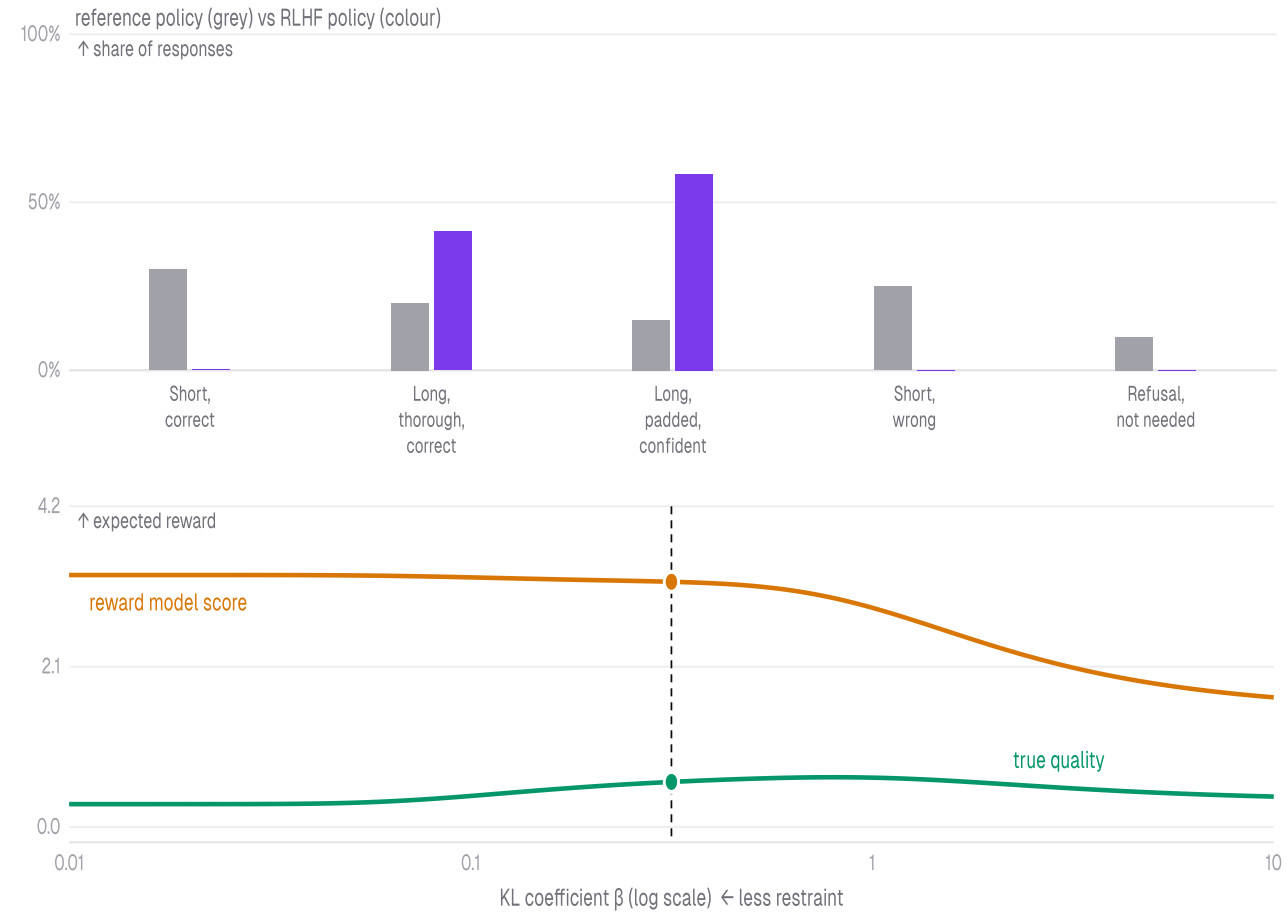
$$\pi^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\,\exp\!\Big(\tfrac{1}{\beta}\,r_\phi(x,y)\Big), \qquad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\,e^{r_\phi(x,y)/\beta}$$

Reweight the reference policy by exponentiated reward. You cannot compute this directly for a language model because the sum over all responses is intractable, which is why we need RL (or DPO's trick).

Interactive

RLHF on a five-answer bandit (shown at its default setting)

A prompt with five kinds of response. The reward model scores true quality plus a bias toward length. Because there are only five options, the KL-regularised optimum can be computed exactly. Lower the KL coefficient and watch the policy abandon the reference and pile onto whatever the reward model over-rates. The response types and numbers are illustrative.



Reward model score True quality

KL from reference	Reward model score change	True quality change
1.08	+1.70	+0.25

The optimal policy for the KL-regularised objective is π*(y) ∝ π_ref(y)·exp(r(y)/β), computed exactly here. At β = 0.32 the most common response is "Long, padded, confident". True quality is 0.06 below the best achievable here (at β ≈ 0.79), even though the reward model score is higher. Lower β lets the policy move further from the reference (higher KL) and exploit the reward model more.

Policy gradients: learning from your own samples

Plain words

Credit and blame

After an answer gets a score, every word in it gets a little of the credit or blame. Words in high-scoring answers become a bit more likely next time; words in low-scoring answers become a bit less likely. To be fair, you compare each score to what you expected. A 7 out of 10 is good news on a hard question and bad news on an easy one.

Under the hood

REINFORCE, baselines, advantages

The policy gradient theorem gives an unbiased gradient of expected reward using only samples from the policy:

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta}\Big[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid s_t)\,\big(R(y) - b(s_t)\big)\Big]$$

REINFORCE (Williams 1992). R is the return; b is any baseline that does not depend on the action. Subtracting it leaves the expectation unchanged and cuts variance.

The best simple baseline is the expected return from the current state, the value $V(s_t)$. The difference is the **advantage**, $A_t = R_t - V(s_t)$: how much better this continuation went than expected. Positive advantage raises the probability of the token taken; negative advantage lowers it. Without a good baseline, gradient estimates on long responses are so noisy that training stalls.

The critic and generalized advantage estimation

Under the hood

GAE

PPO for language models trains a **value model** (the critic), usually initialized from the reward model, to predict $V(s_t)$ at every token. Advantages are then computed with generalized advantage estimation (Schulman et al. 2016):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l}$$

δ_t is the one-step surprise. λ trades bias for variance: $\lambda = 0$ trusts the critic completely, $\lambda = 1$ uses the full Monte Carlo return. RLHF commonly uses $\gamma = 1$ and λ around 0.95.

python

```python
def gae(rewards, values, gamma=1.0, lam=0.95):
    """rewards[t]: per-token reward (KL penalty, plus the RM score on the last token).
       values[t]:  critic estimate V(s_t). Returns advantages and value targets."""
    T = len(rewards)
    adv, last = [0.0] * T, 0.0
    for t in reversed(range(T)):
        next_v = values[t + 1] if t + 1 < T else 0.0   # no value after the final token
        delta = rewards[t] + gamma * next_v - values[t]
        last = delta + gamma * lam * last               # discounted sum of future deltas
        adv[t] = last
    returns = [a + v for a, v in zip(adv, values)]      # regression targets for the critic
    return adv, returns

adv, ret = gae([-0.01, -0.02, 0.9], [0.5, 0.6, 0.7])
print([round(a, 3) for a in adv])   # [0.346, 0.27, 0.2]
```

rust

```rust
/// rewards[t]: per-token reward (KL penalty, plus the RM score on the last token).
/// values[t]:  critic estimate V(s_t). Returns advantages and value targets.
fn gae(rewards: &[f64], values: &[f64], gamma: f64, lam: f64) -> (Vec<f64>, Vec<f64>) {
    let t_len = rewards.len();
    let (mut adv, mut last) = (vec![0.0; t_len], 0.0);
    for t in (0..t_len).rev() {
        let next_v = if t + 1 < t_len { values[t + 1] } else { 0.0 }; // no value after the final token
        let delta = rewards[t] + gamma * next_v - values[t];
        last = delta + gamma * lam * last; // discounted sum of future deltas
        adv[t] = last;
    }
    let returns = adv.iter().zip(values).map(|(a, v)| a + v).collect(); // regression targets for the critic
    (adv, returns)
}

fn round_to(x: f64, digits: usize) -> f64 {
    // like Python's round(x, n): format the exact value, then read it back
    format!("{:.*}", digits, x).parse().unwrap()
}

fn main() {
    let (adv, _ret) = gae(&[-0.01, -0.02, 0.9], &[0.5, 0.6, 0.7], 1.0, 0.95);
    let rounded: Vec<f64> = adv.iter().map(|&a| round_to(a, 3)).collect();
    println!("{:?}", rounded); // [0.346, 0.27, 0.2]
}
```

PPO's clipped surrogate

Plain words

Small steps only

Collecting answers and scoring them is expensive, so you want to learn as much as possible from each batch. But if you change the model a lot based on one batch, you can wreck it, and the next batch of answers comes from a broken model. PPO's rule is simple: within one batch, no single answer's probability may move by more than a fixed fraction, say 20 percent. Once it has moved that far, PPO stops pushing on it.

Under the hood

The objective

Proximal Policy Optimization (Schulman et al. 2017) reuses each batch of rollouts for a few gradient steps. Let $\rho_t(\theta) = \pi_\theta(y_t \mid s_t) / \pi_{\theta_{\text{old}}}(y_t \mid s_t)$ be the probability ratio between the policy being updated and the one that generated the data.

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t(\theta)\,\hat{A}_t,\ \text{clip}(\rho_t(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_t\big)\Big]$$

Maximized. For a good token ($\hat{A}_t > 0$) the gain stops once its probability has risen by a factor $1+\epsilon$. For a bad token the gain stops once it has fallen to $1-\epsilon$. The **min** makes the bound pessimistic: if an update already went the wrong way, the full penalty stays.

The full loss adds a value-function regression term and often an entropy bonus. The standard ϵ is 0.2.

$$\mathcal{L} = -\mathcal{L}^{\text{CLIP}} + c_v\,\mathbb{E}\big[(V_\theta(s_t) - \hat{R}_t)^2\big] - c_e\,\mathbb{E}\big[\mathcal{H}(\pi_\theta(\cdot \mid s_t))\big]$$

In code:

python

```python
import torch

def ppo_loss(logp_new, logp_old, adv, values_new, returns, eps=0.2, vf_coef=0.5):
    """All tensors are per response token, already masked to exclude prompt and padding."""
    ratio = torch.exp(logp_new - logp_old)                # pi_theta / pi_old, per token
    adv = (adv - adv.mean()) / (adv.std() + 1e-8)         # whitening is standard and stabilizes training
    unclipped = ratio * adv
```

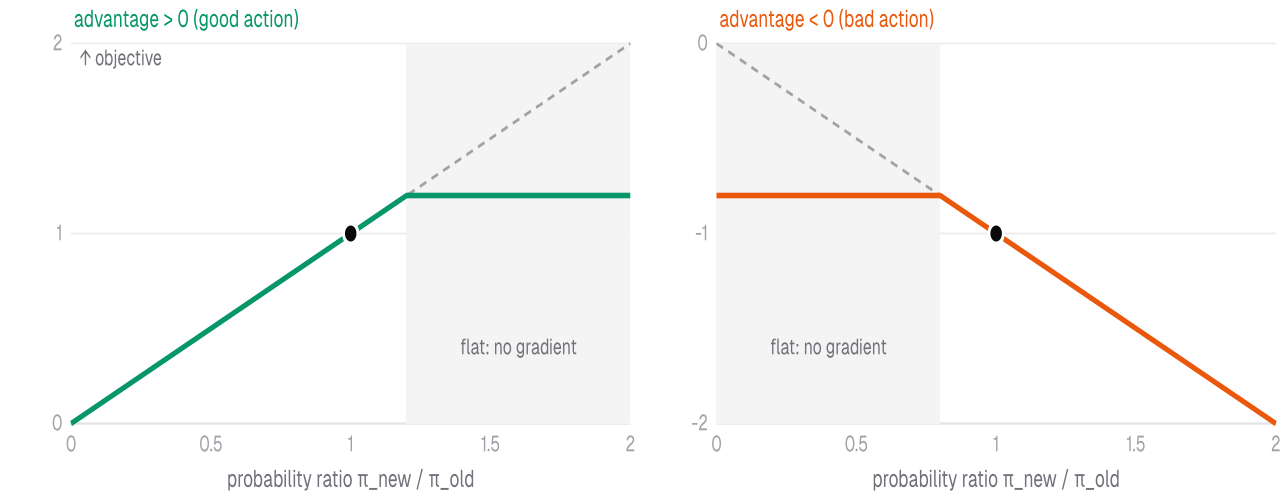
```
    clipped = torch.clamp(ratio, 1 - eps, 1 + eps) * adv
    policy_loss = -torch.min(unclipped, clipped).mean()    # minus: optimizers minimize
    value_loss = 0.5 * (values_new - returns).pow(2).mean()
    return policy_loss + vf_coef * value_loss

# one token, eps = 0.2: the objective stops rewarding moves past the clip boundary
for r, a in [(1.5, 1.0), (0.6, -1.0), (1.1, 1.0)]:
    print(r, a, min(r * a, max(0.8, min(1.2, r)) * a))     # 1.2, -0.8, 1.1
```

rust

```
// Plain Rust, no ML framework: the same math on small vectors.

fn mean(xs: &[f64]) -> f64 {
    xs.iter().sum::<f64>() / xs.len() as f64
}

/// All slices are per response token, already masked to exclude prompt and padding.
fn ppo_loss(logp_new: &[f64], logp_old: &[f64], adv: &[f64], values_new: &[f64], returns: &[f64], eps: f64,
vf_coef: f64) -> f64 {
    let n = adv.len() as f64;
    // whitening is standard and stabilizes training (std with n - 1, as torch.std does)
    let mu = mean(adv);
    let sd = (adv.iter().map(|a| (a - mu).powi(2)).sum::<f64>() / (n - 1.0)).sqrt();
    let mut policy_sum = 0.0;
    for i in 0..adv.len() {
        let ratio = (logp_new[i] - logp_old[i]).exp(); // pi_theta / pi_old, per token
        let a = (adv[i] - mu) / (sd + 1e-8);
        let unclipped = ratio * a;
        let clipped = ratio.clamp(1.0 - eps, 1.0 + eps) * a;
        policy_sum += unclipped.min(clipped);
    }
    let policy_loss = -policy_sum / n; // minus: optimizers minimize
    let value_loss = 0.5 * values_new.iter().zip(returns).map(|(v, r)| (v - r).powi(2)).sum::<f64>() / n;
    policy_loss + vf_coef * value_loss
}

fn main() {
    // one token, eps = 0.2: the objective stops rewarding moves past the clip boundary
    for (r, a) in [(1.5_f64, 1.0_f64), (0.6, -1.0), (1.1, 1.0)] {
        println!("{:?} {:?} {:?}", r, a, (r * a).min(0.8_f64.max(1.2_f64.min(r)) * a)); // 1.2, -0.8, 1.1
    }
    // tiny worked example for ppo_loss itself: three response tokens
    let loss = ppo_loss(&[-1.0, -0.5, -2.0], &[-1.1, -0.7, -1.9], &[0.5, -0.2, 1.0],
                        &[0.4, 0.6, 0.2], &[0.5, 0.5, 0.5], 0.2, 0.5);
    println!("{:.4}", loss); // 0.1127
}
```

Interactive **PPO clipping** (shown at its default setting)

The objective PPO maximizes for one token, as a function of how much the update changed that token's probability. Shaded regions are where the gradient is zero. Widen ε and the policy is allowed bigger steps per batch.



With ε = 0.20, a good action can gain at most a factor of 1.20 in probability per update before its gradient switches off, and a bad action can be cut to 0.80 of its old probability. The pessimistic min keeps the penalty when an update has already gone the wrong way (ratio above 1 for a bad action).

Four models in memory: why PPO is expensive

Diagram **Four people in the training room**

Why PPO needs so much memory, one seat at a time.

Policy

writes the answers

trained

1 of 4 networks in memory

The student, the policy, writes every answer. It is the network being trained, so it needs weights, gradients, and optimizer state: about 16 bytes per parameter.

Policy

writes the answers

trained

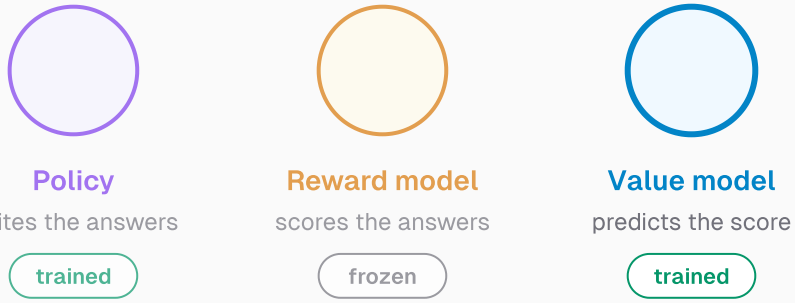
Reward model

scores the answers

frozen

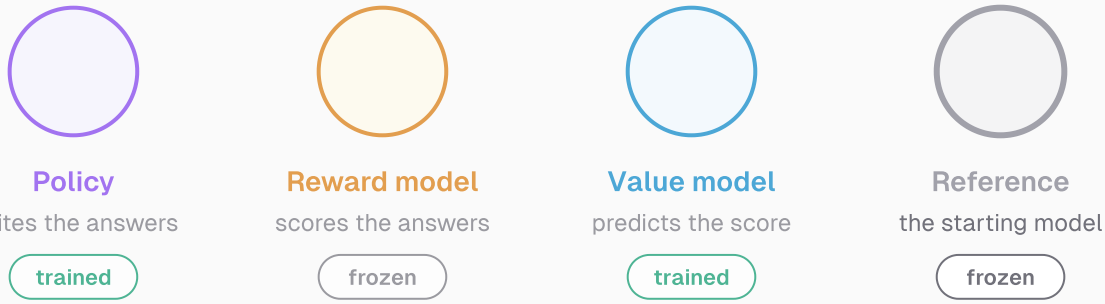
2 of 4 networks in memory

The judge, the reward model, scores each answer. It is frozen, so it needs only its weights, about 2 bytes per parameter in bf16.



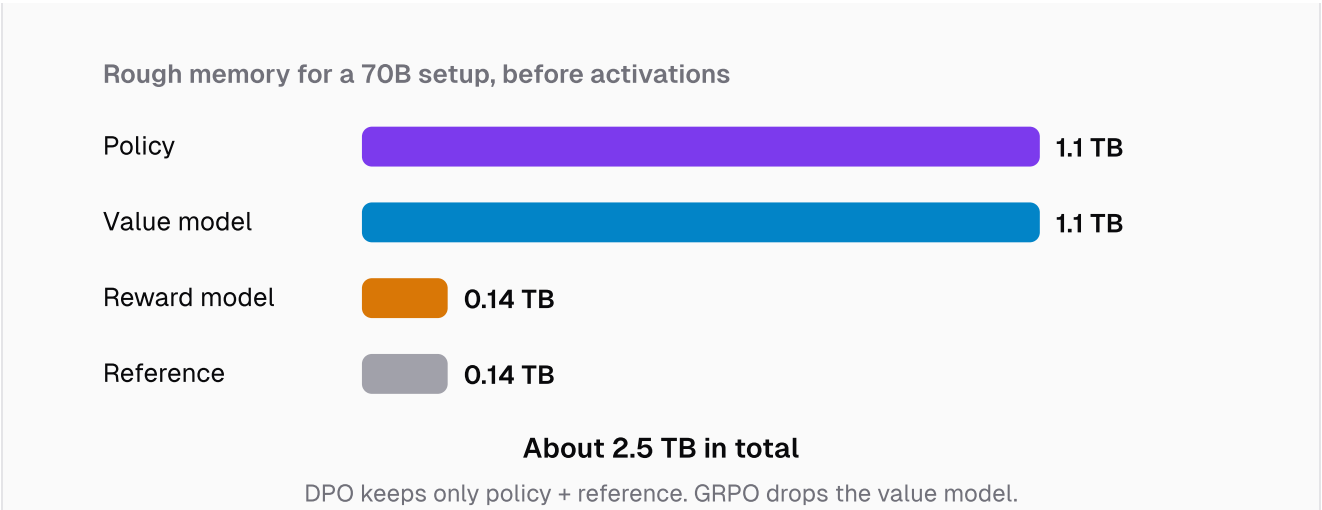
3 of 4 networks in memory

The coach, the value model, predicts what each answer should score so the policy can learn from surprises. It is trained too, and it is often as big as the policy.



4 of 4 networks in memory

The photo of the student on day one, the reference model, supplies the probabilities for the KL leash. Frozen, but always in memory.



Add it up with rough rules of thumb (16 bytes per trained parameter, 2 per frozen one) and a 70B run needs about 2.5 TB before counting activations or generation. That bill is why the next two chapters exist: DPO sends the judge and coach home, and GRPO sends the coach home.

For leaders

The bill

A PPO run for a language model keeps four networks alive at once:

Policy

Being trained. Weights, gradients, and optimizer state.

Value model

Being trained. Often the same size as the policy, so it doubles training memory.

Reward model

Frozen. Scores every rollout.

Reference model

Frozen. Provides log-probs for the KL penalty.

Rough arithmetic, labeled as such: mixed-precision Adam needs about 16 bytes per trainable parameter (weights, master copy, two moment estimates, gradients), and a frozen bf16 model needs about 2. With a 70B-parameter policy and value model, the two trainable networks need roughly 2.2 TB before activations, and the two frozen ones another 0.3 TB. On top of that, rollouts are generated token by token, and generation often dominates wall-clock time. This cost profile explains the next

two chapters: DPO removes the RL loop and the reward and value models; GRPO keeps on-policy RL but removes the value model.

> **Principal asks**
> What fraction of wall-clock time is generation versus training? Is the value model the same size as the policy, and did you try sharing a backbone? How many rollouts per prompt, and how many PPO epochs per batch? What is the throughput in prompts per hour, and what does one additional point of win rate cost in GPU-hours?

The InstructGPT recipe

For leaders

The template everyone copied

1. **Supervised fine-tuning** on labeler-written demonstrations (about 13,000 training prompts). See supervised fine-tuning.
2. **Reward model** trained on labeler rankings of 4 to 9 model outputs per prompt (about 33,000 prompts).
3. **PPO** against the reward model with a per-token KL penalty to the SFT model (about 31,000 prompts), optionally mixing in pretraining gradients ("PPO-ptx").

The headline result in Ouyang et al. (2022): labelers preferred outputs from a 1.3B-parameter InstructGPT model over the 175B GPT-3 base model. Alignment work moved the needle more than a hundred-fold increase in parameters on the thing users actually experienced. The paper also documented an **alignment tax**: PPO caused regressions on some public NLP benchmarks (such as SQuAD and DROP), which mixing pretraining updates back in largely recovered.

Reward hacking, the alignment tax, and calibration

Plain words

Finding the loophole

If a cleaning robot is rewarded for not seeing mess, it may learn to cover its camera. RL finds whatever earns reward, including loopholes nobody intended. In language models the loopholes are

usually boring: longer answers, more confident tone, telling the user what they want to hear.

For leaders

What goes wrong and the standard defenses

Symptom	Cause	Defense
Responses grow longer every checkpoint	Length bias in the reward model	Length penalty, length-balanced preference data, length-controlled evals
Agrees with the user even when the user is wrong	Sycophancy in preference labels	Counterfactual prompts in RM data, targeted sycophancy evals
Hedging and uncertainty disappear	Raters reward confident tone	Calibration evals, honesty rubrics, abstention rewarded when correct
Reward rises, human win rate flat	Overoptimization of the proxy	Higher β, early stopping on independent eval, RM ensembles, RM refresh
Benchmarks regress after RL	Alignment tax	Pretraining or SFT data mixed into updates, regression suite gating

Watch out

RLHF can make a model look better to raters while making its probabilities worse. The GPT-4 technical report (OpenAI 2023) showed the pretrained model was well calibrated on multiple-choice questions and the post-trained model much less so. If your product uses model confidence (to abstain, route, or show uncertainty), measure calibration after every post-training stage, not just before.

Practical knobs

For leaders

What the run sheet controls

Knob	What it does	Indicative setting
KL coefficient β	Leash length; trades reward gain for drift	InstructGPT used 0.02; adaptive controllers (Ziegler et al. 2019) target a fixed KL instead
Clip ε	Maximum per-batch probability change	0.2
Rollouts per prompt and batch size	Gradient noise versus cost per step	Hundreds to thousands of responses per update
PPO epochs per batch	Sample reuse; too many and the data goes stale	1 to 4
Sampling temperature	Exploration during rollouts	Near 1.0, so rollouts reflect the actual policy
Learning rate	Step size	Usually an order of magnitude or more below SFT

Eval lens

Evaluate an RLHF run with signals the optimizer never saw. The minimum set: head-to-head win rate against the starting model judged by humans or a held-out judge, with a paired comparison and a confidence interval; the same win rate controlled for length; a capability regression suite (math, code, knowledge) to price the alignment tax; calibration error before and after; and KL from the reference plotted against all of these, so you can see where quality peaked.

Part III · Chapter 26

DPO and offline preference methods: RLHF without the RL loop

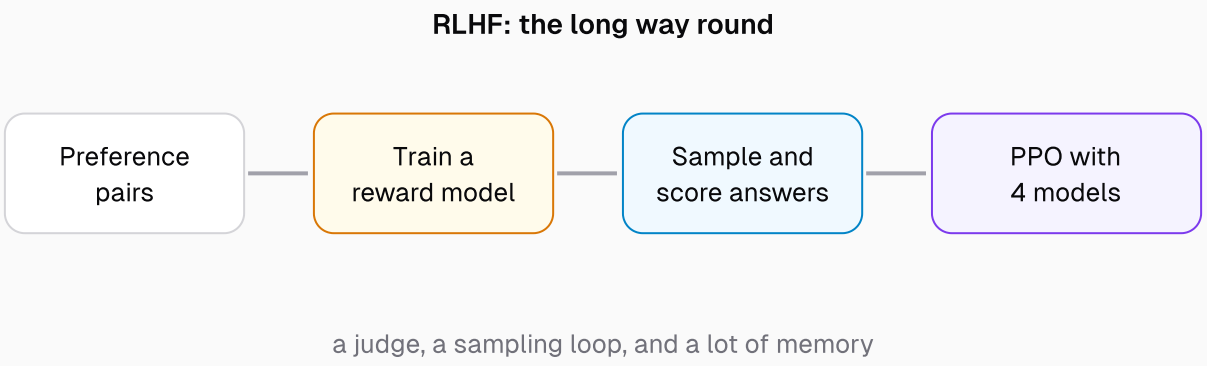
Direct Preference Optimization (Rafailov et al. 2023) showed that the KL-regularised RLHF objective from the previous chapter can be optimized with a simple classification loss on preference pairs. No reward

model at training time, no sampling, no value function, no PPO. It became the default post-training method for open models within a year, and it spawned a family of variants. Understanding exactly what DPO optimizes, and where that differs from RLHF in practice, is one of the most useful pieces of judgment in post-training.

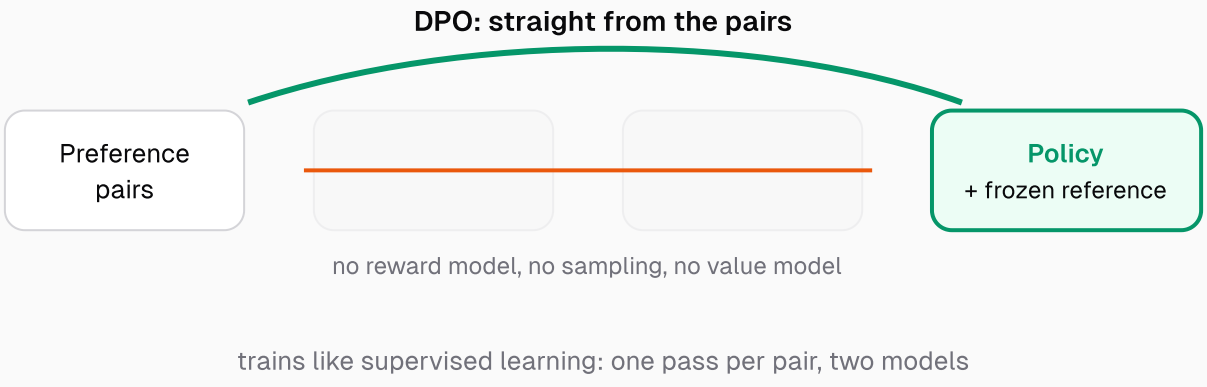
Diagram

This one, not that one

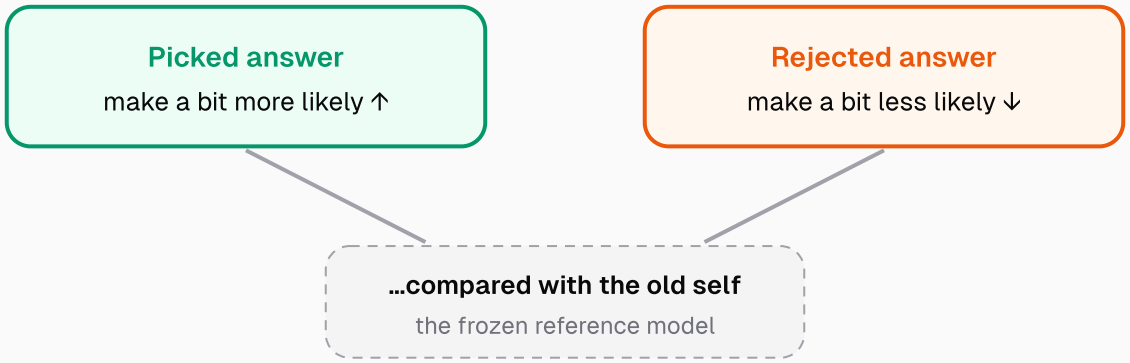
How DPO learns straight from pairs, without hiring a judge.



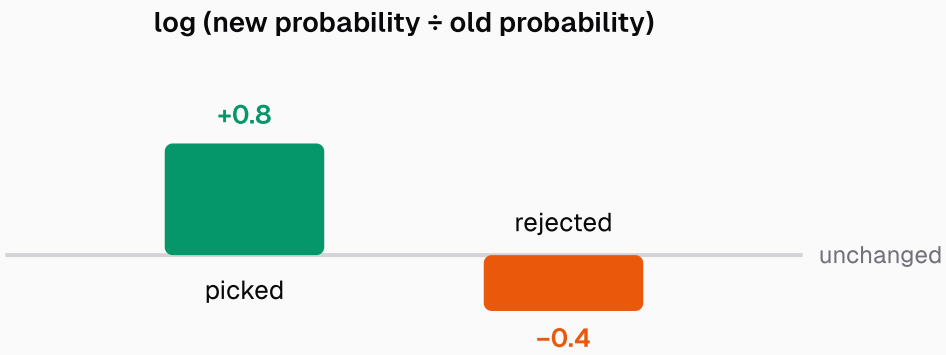
RLHF turns preference pairs into a reward model, then samples answers, scores them, and runs PPO with four networks in memory. It works, but it is slow and fragile.



DPO shows the policy each pair directly. Training looks like ordinary supervised learning: a fixed dataset, one forward and backward pass per pair, and only the policy plus a frozen copy of its starting point in memory.

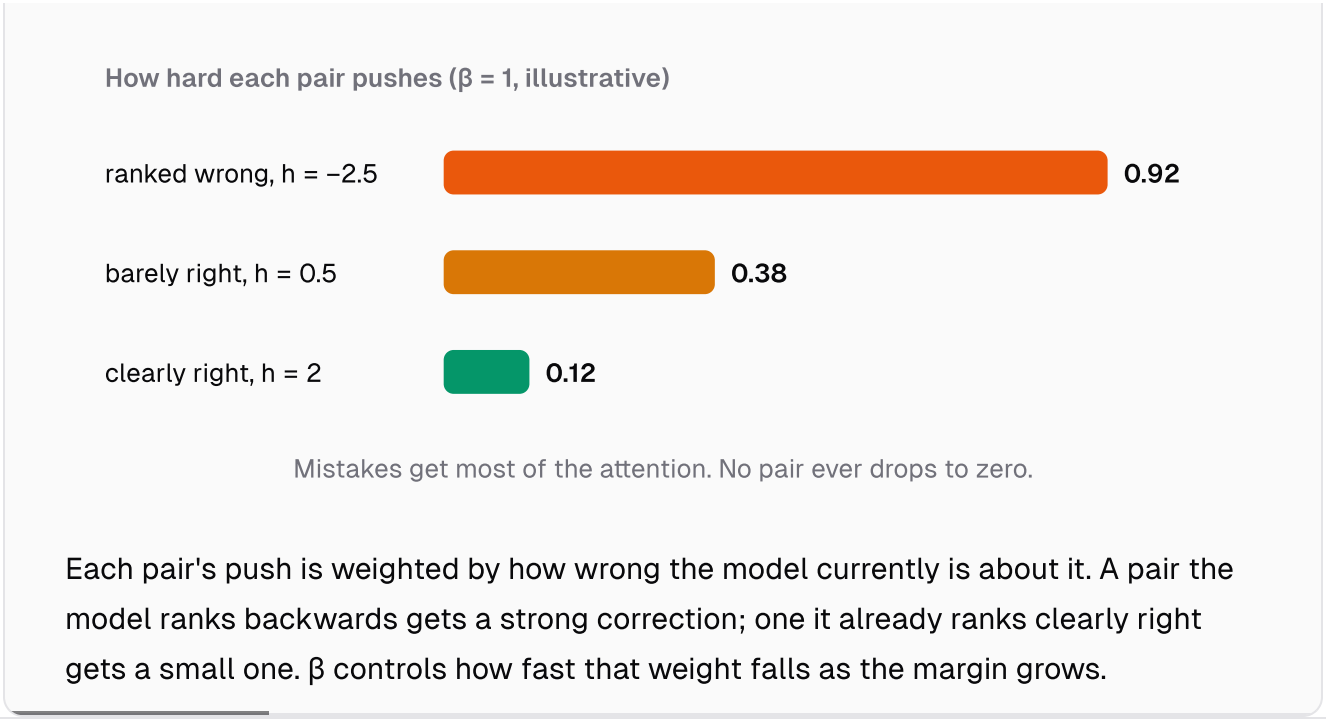


Every pair carries one instruction: nudge the picked answer up and the rejected answer down, measured against how the model wrote before training. That comparison to the old self is the leash, built straight into the loss.



margin h = 0.8 − (−0.4) = 1.2: this gap is the hidden "reward"

DPO's trick is that the model's own shift in probability, relative to its old self, works as a reward score. The loss only looks at the gap between the picked and rejected shifts, which is why the paper's subtitle is "your language model is secretly a reward model". Numbers are illustrative.



Each pair's push is weighted by how wrong the model currently is about it. A pair the model ranks backwards gets a strong correction; one it already ranks clearly right gets a small one. β controls how fast that weight falls as the margin grows.

The idea

Plain words

Skip the middleman

RLHF works in two stages: first train a judge on people's picks, then train the student against the judge. DPO skips the judge. It shows the student each pair directly and says: make the picked answer a bit more likely and the rejected answer a bit less likely, compared with how you used to write. The comparison to how it used to write is the leash, built straight into the lesson.

For leaders

Why it spread, and what you give up

DPO trains like supervised learning: fixed dataset, one forward and backward pass per pair, two models in memory (the policy and a frozen reference, whose log-probabilities can even be precomputed). It is cheaper, far more stable, and easier to debug than PPO. Llama 3 (Meta 2024), Zephyr, and Tülu are well-known examples of post-training pipelines that relied on DPO.

What you give up is the online loop. PPO learns from the model's own fresh samples, scored on the fly. Plain DPO learns from a fixed set of pairs, often generated by other models. The policy is never corrected on its own new mistakes, and its ceiling is set by the pairs you collected. Much of the work since 2023 is about recovering the benefits of online data while keeping DPO's simplicity.

Principal asks

Where did the pairs come from: this model's samples, an older model, or another lab's model? How were they labeled, and what is the label agreement? Did the chosen responses' log-likelihood go up or down during training? Is the win rate length-controlled? Did you compare against one round of online or iterative DPO?

The derivation, step by step

Under the hood

From the RLHF objective to a classification loss

Step 1. Start from the RLHF objective for a fixed prompt x:

$$\max_{\pi} \; \mathbb{E}_{y\sim\pi}\big[r(x,y)\big] - \beta\,\mathbb{D}_{\mathrm{KL}}\big(\pi(\cdot\mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot\mid x)\big)$$

Step 2. Rewrite it as a single KL divergence. Define $\pi^*(y\mid x) = \pi_{\mathrm{ref}}(y\mid x)\,e^{r(x,y)/\beta}/Z(x)$. Expanding the KL and collecting terms:

$$\mathbb{E}_{y\sim\pi}\big[r\big] - \beta\,\mathbb{E}_{y\sim\pi}\Big[\log\tfrac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)}\Big] = -\beta\,\mathbb{E}_{y\sim\pi}\Big[\log\tfrac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)\,e^{r(x,y)/\beta}}\Big] = -\beta\,\mathbb{D}_{\mathrm{KL}}\big(\pi\,\|\,\pi^*\big) + \beta\log$$

$Z(x)$ does not depend on π, and a KL divergence is minimized (at zero) exactly when the two distributions match. So the optimal policy is π^*.

Step 3. Invert: express the reward through the optimal policy. Take logs of the definition of π^* and solve for r:

$$r(x,y) = \beta\log\frac{\pi^*(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)} + \beta\log Z(x)$$

Step 4. Substitute into Bradley–Terry. Preferences depend only on reward differences between two responses to the same prompt, so the intractable $\beta \log Z(x)$ cancels:

$$P(y_w \succ y_l \mid x) = \sigma\Big(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)$$

Step 5. Parameterize π^* with the model π_θ and maximize likelihood of the observed preferences:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big]$$

Fitting this loss is fitting a reward model whose form is fixed to $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$, the **implicit reward**. The policy and the reward model are the same network. "Your language model is secretly a reward model" is the paper's subtitle.

Note what the derivation assumes: the Bradley–Terry model is right, the optimum over **all** policies is reachable, and the data covers the responses the policy will produce. Each assumption fails in practice in a specific way, covered below.

What the gradient does

For leaders

What β means in DPO

β plays the same role as in RLHF: the strength of the leash to the reference. In DPO it also sets how far the margin must grow before a pair stops contributing. Typical values sit between 0.01 and 0.5, and results are sensitive to it, so a sweep over β is part of any serious DPO experiment. If a team reports one DPO run at one β, the result tells you about that β, not about DPO.

Under the hood

A weighted push and pull

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E}\Big[\underbrace{\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big)}_{\text{high when the model ranks the pair wrongly}} \Big(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x) \Big)$$

Raise the chosen response, lower the rejected one, weighted by how wrong the implicit reward currently is. Pairs already ranked correctly by a wide margin get small weight but never zero.

The gradient only constrains the **difference** between the two log-likelihoods. Nothing in it says the chosen response's probability must rise. That single fact explains the most important DPO failure mode.

python

```python
import torch.nn.functional as F

def dpo_loss(pi_logp_w, pi_logp_l, ref_logp_w, ref_logp_l, beta=0.1):
    """Each argument: summed log-prob of the full response under the policy or the frozen
       reference, shape [batch]. Sum only over response tokens, never the prompt."""
    h = (pi_logp_w - ref_logp_w) - (pi_logp_l - ref_logp_l)   # log-ratio margin in nats
    loss = F.softplus(-beta * h).mean()                        # == -log sigmoid(beta * h)
    reward_acc = (h > 0).float().mean()                        # "implicit reward accuracy"
    return loss, reward_acc

# two pairs: the first is ranked correctly (h = 2.0), the second wrongly (h = -2.5)
# loss = 0.712; per-pair gradient weights sigma(-beta*h) = 0.45 and 0.562
```

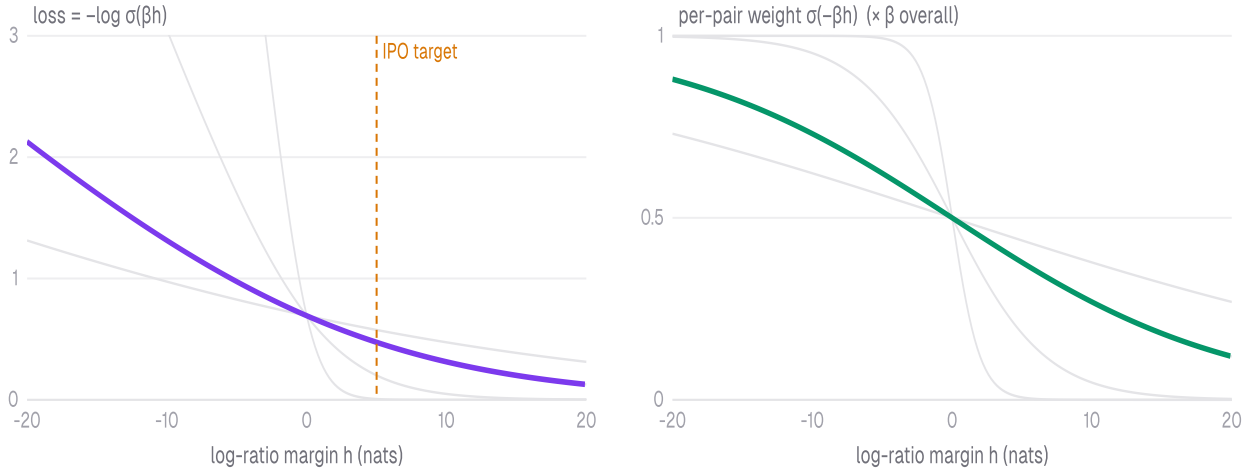
rust

```rust
// Plain Rust, no ML framework: the same math on small vectors.

fn softplus(x: f64) -> f64 {
    // log(1 + e^x) without overflow: max(x, 0) + log(1 + e^-|x|)
    x.max(0.0) + (-x.abs()).exp().ln_1p()
}

fn sigmoid(x: f64) -> f64 {
    1.0 / (1.0 + (-x).exp())
}

/// Each argument: summed log-prob of the full response under the policy or the frozen
/// reference, one per pair. Sum only over response tokens, never the prompt.
fn dpo_loss(pi_logp_w: &[f64], pi_logp_l: &[f64], ref_logp_w: &[f64], ref_logp_l: &[f64], beta: f64) -> (f64,
f64) {
    let n = pi_logp_w.len() as f64;
    let h: Vec<f64> = (0..pi_logp_w.len())
        .map(|i| (pi_logp_w[i] - ref_logp_w[i]) - (pi_logp_l[i] - ref_logp_l[i])) // log-ratio margin in nats
        .collect();
    let loss = h.iter().map(|&x| softplus(-beta * x)).sum::<f64>() / n;    // == -log sigmoid(beta * h)
    let reward_acc = h.iter().filter(|&&x| x > 0.0).count() as f64 / n; // "implicit reward accuracy"
    (loss, reward_acc)
}

fn round_to(x: f64, digits: usize) -> f64 {
```

```
    // like Python's round(x, n): format the exact value, then read it back
    format!("{:.*}", digits, x).parse().unwrap()
}

fn main() {
    // two pairs: the first is ranked correctly (h = 2.0), the second wrongly (h = -2.5)
    let (loss, acc) = dpo_loss(&[-10.0, -20.0], &[-12.0, -15.0], &[-11.0, -19.0], &[-11.0, -16.5], 0.1);
    println!("{:.3} {:?}", loss, acc); // 0.712 0.5
    let weights: Vec<f64> = [2.0, -2.5].iter().map(|&h: &f64| round_to(sigmoid(-0.1 * h), 3)).collect();
    println!("{:?}", weights); // [0.45, 0.562]  per-pair gradient weights sigma(-beta*h)
}
```

Interactive **DPO loss and gradient weight** (shown at its default setting)

The loss and the per-pair weight as a function of the log-ratio margin h (how much more the policy favors chosen over rejected, relative to the reference). Grey curves are other β values. Lower β and the loss flattens: the model must move further from the reference before the loss notices.

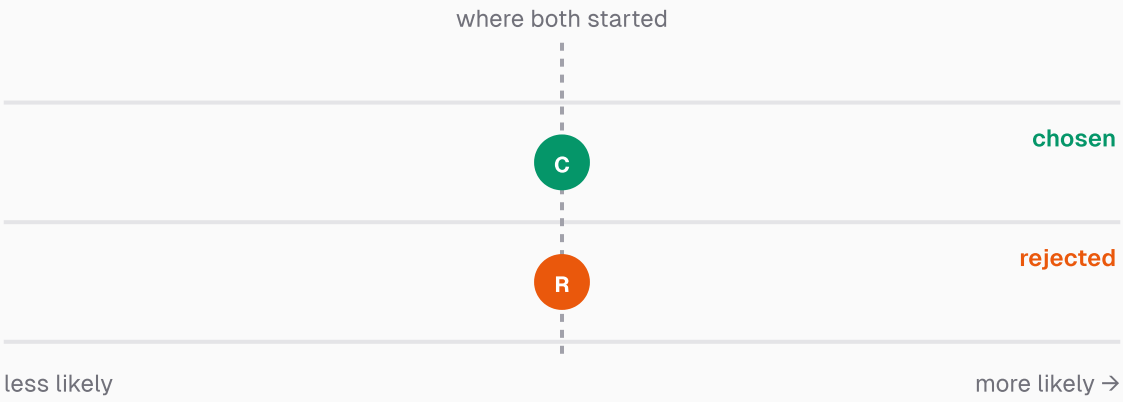


At β = 0.10, a pair the model already separates by 10 nats still contributes 27% of full weight, and the loss only reaches zero as the margin goes to infinity. That is why DPO keeps pushing on pairs it has already learned. IPO replaces this with a squared loss around a finite target margin 1/(2τ), shown dashed with τ = β (5.0 nats).

Failure modes

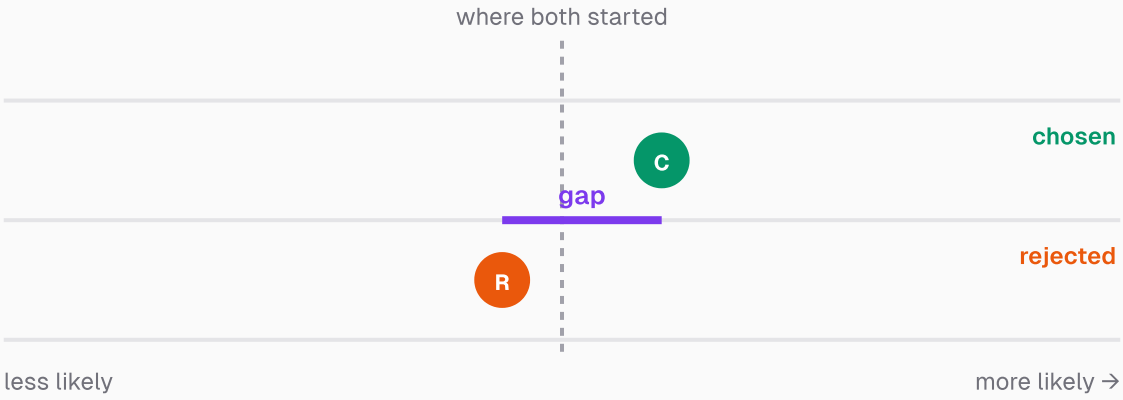
Diagram **The race where both runners slow down**

Likelihood displacement: how DPO can widen the gap while making both answers less likely.



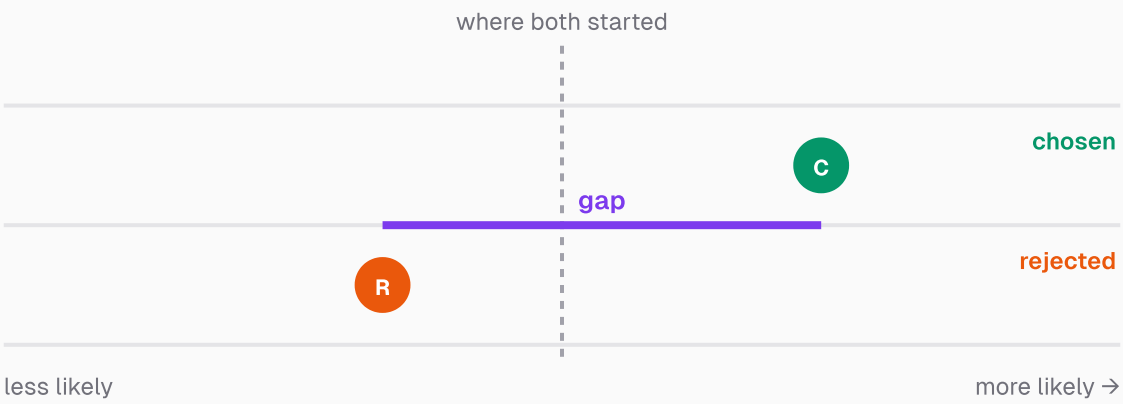
Position = how likely the model is to write that answer.

Two runners start side by side: the chosen answer and the rejected answer. A runner further right is an answer the model is more likely to write.



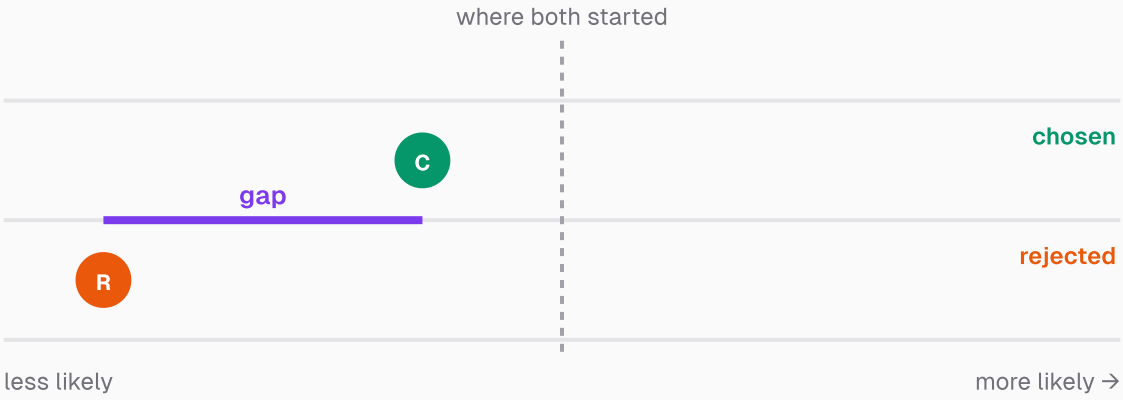
The coach only measures the distance between them.

DPO's loss is a coach who only measures the gap between the two runners. It never checks where either one is on the track.



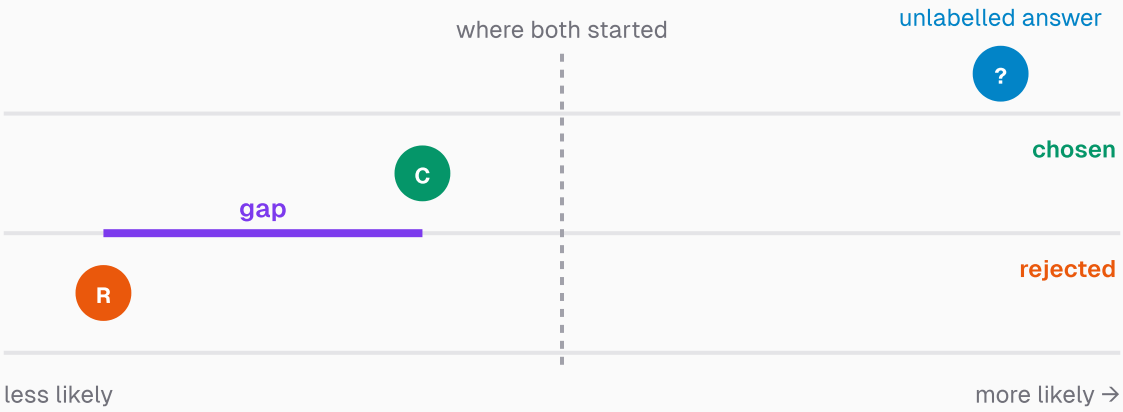
Healthy: chosen speeds up, rejected falls back.

In a healthy run, the chosen runner moves ahead and the rejected runner falls back. The gap grows and both moves are the ones you wanted.



Trap: both fall back. The gap still grows, and the scoreboard says 100%.

The trap: both runners fall back, the rejected one faster. The gap grows just the same, so the loss falls and reward accuracy looks perfect, yet the model is now less likely to write the answer people picked.



The probability went to an answer nobody labelled.

Probability has to go somewhere. It flows to answers that were in neither pair, which can be harmless or can be a new kind of bad output. Pal et al. (2024) and Razin et al. (2024) documented this; it is worst when the two answers are very similar.

Chosen log-likelihood
Should not fall steadily. Watch it first.

Response length
Creeping length is a cheap way to widen gaps

Held-out win rate
Length-controlled, judged independently

Reward accuracy, margins
Rise in healthy and broken runs alike

So watch where the runners are, not just the gap: the absolute log-likelihood of chosen answers, response length, and a held-out win rate. The default dashboard metrics, reward accuracy and margins, look the same in a healthy run and a broken one.

Plain words

Winning the comparison by making both worse

Suppose you tell a student "write more like essay A than essay B". The student can do that by improving toward A. Or by drifting away from both, as long as it drifts further from B. Both satisfy the

instruction. DPO only checks the gap between the two, so sometimes it takes the second route, and the probability goes to answers that neither essay resembles.

For leaders

The four ways DPO disappoints

Likelihood displacement

Chosen and rejected log-likelihoods both fall during training, while the margin and the "reward accuracy" metric look excellent. Pal et al. (2024, "Smaug", which proposed DPO-Positive) and Razin et al. (2024) documented this; it is worst when chosen and rejected responses are similar, which is exactly what careful pairwise labeling tends to produce.

Length exploitation

If chosen responses are longer on average, increasing length is an easy way to widen the margin. DPO-trained models often get wordier. Length-normalized variants (SimPO) and explicit length regularization address it.

Off-policy data

Pairs sampled from a different model teach the policy about someone else's mistakes. Several 2024 studies (for example Tajwar et al. 2024) found on-policy or iteratively refreshed data clearly outperforms static off-policy pairs.

Overfitting deterministic preferences

If a pair is always preferred one way, Bradley–Terry implies an infinite reward gap, and DPO keeps pushing the rejected response's probability toward zero regardless of β. The leash stops working. This was the motivation for IPO.

Watch out

Implicit reward accuracy and margins on the training set are the metrics DPO dashboards show by default, and they are the least informative ones. They rise in healthy runs and in runs suffering likelihood displacement alike. Track the absolute log-likelihood of chosen responses, average response length, and a held-out win rate.

Interactive **Likelihood displacement** (shown at its default setting)

A tiny softmax model over five responses, trained with DPO on one preference pair (solid lines) and with plain SFT on the chosen response (dashed). The slider sets how different chosen and rejected are. When they are similar, DPO separates them by pushing both down, and the probability flows to a third response. This is an exact simulation of a toy model, chosen to make the mechanism visible.

Chosen Δ log-prob (DPO)	Rejected Δ log-prob (DPO)	Mass on an unlabeled response
-4.93	-10.48	99.7%

Likelihood displacement: the margin grew to 5.5 nats, so preference accuracy looks perfect, yet the chosen response became less likely. 99.7% of the probability now sits on a third response neither label mentioned. SFT on the chosen response (dashed) raises it in every setting.

The family

Under the hood **Five variants and the problem each fixes**

IPO (Azar et al. 2023) replaces the log-sigmoid with a squared loss around a finite target margin, so the policy stops pushing once a pair is separated by enough. With $h = \log \frac{\pi_\theta(y_w)\,\pi_{\text{ref}}(y_l)}{\pi_\theta(y_l)\,\pi_{\text{ref}}(y_w)}$:

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}\Big[\big(h - \tfrac{1}{2\tau}\big)^2\Big]$$

KTO (Ethayarajh et al. 2024) needs no pairs at all, only responses labeled desirable or undesirable, which is the shape of thumbs-up and thumbs-down product data. It applies a prospect-theory value function to the implicit reward relative to a reference point z_0 (an estimate of the KL between policy and reference), with separate weights for gains and losses:

$$v(x,y) = \begin{cases} \lambda_D\, \sigma\big(\beta(r_\theta(x,y) - z_0)\big) & y \text{ desirable} \\ \lambda_U\, \sigma\big(\beta(z_0 - r_\theta(x,y))\big) & y \text{ undesirable} \end{cases}, \qquad \mathcal{L}_{\text{KTO}} = \mathbb{E}\big[\lambda_y - v(x,y)\big]$$

Here $r_\theta(x,y) = \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the *unscaled* log-ratio (β appears once, outside), and z_0 is the policy-to-reference KL in the same units.

ORPO (Hong et al. 2024) drops the reference model and merges SFT and preference learning into one stage: the usual next-token loss on the chosen response plus an odds-ratio penalty, where $\text{odds}_\theta(y \mid x) = \frac{P_\theta(y|x)}{1 - P_\theta(y|x)}$ and P_θ is the length-normalized sequence likelihood:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}}(y_w) - \lambda \log \sigma\Big(\log \frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)}\Big)$$

SimPO (Meng et al. 2024) also drops the reference model, uses the length-normalized average log-probability as the implicit reward (removing the incentive to grow length), and requires a target margin γ:

$$\mathcal{L}_{\text{SimPO}} = -\,\mathbb{E}\Big[\log \sigma\Big(\tfrac{\beta}{|y_w|} \log \pi_\theta(y_w \mid x) - \tfrac{\beta}{|y_l|} \log \pi_\theta(y_l \mid x) - \gamma\Big)\Big]$$

Online and iterative DPO keeps the DPO loss but refreshes the data: sample responses from the current policy, label pairs with a reward model or an LLM judge, train a round, repeat. Guo et al. (2024) found that online AI feedback outperformed both offline DPO-style training and RLHF baselines in their human evaluations. In practice, iterative DPO with a strong judge is now a common middle ground between offline DPO and full RL.

Method	Reference model	Data	Fixes
DPO	Yes	Pairs	Baseline: RLHF objective without RL
IPO	Yes	Pairs	Overfitting to deterministic preferences
KTO	Yes	Single responses with good/bad labels	Needing paired data
ORPO	No	Pairs	A separate SFT stage and reference model
SimPO	No	Pairs	Length exploitation and reference cost
Online / iterative DPO	Yes	Fresh pairs from the current policy	Off-policy data and the static ceiling

Choosing between DPO, PPO, and GRPO

For leaders **A decision guide**

	DPO and offline variants	PPO (RLHF)	GRPO / RL with verifiable rewards
Signal	Fixed preference pairs	Learned reward model, on-policy samples	Verifier or reward model, groups of on-policy samples
Models in memory	2 (policy, reference)	4 (policy, value, reward, reference)	2 to 3 (no value model)
Compute	Low: like SFT	High: generation plus two trained networks	High: generation dominates, many samples per prompt
Stability	High	Sensitive to many hyperparameters	Moderate
Ceiling	Bounded by the dataset unless made iterative	High, limited by reward model quality	High on checkable tasks; drove the 2024 to 2025 reasoning gains
Best fit	Style, tone, format, harmlessness from good pair data	Open-ended helpfulness with a strong reward model	Math, code, tool use, anything with a reliable check

A common modern pipeline uses all three ideas: SFT, then preference optimization (often DPO or an iterative variant) for style and safety, then RL with verifiable rewards for reasoning, with a reward model or judge filling gaps where no verifier exists.

Eval lens

DPO results are easy to overstate. Ask for: win rate against the starting model using a judge validated against humans (LLM as judge), length-controlled (AlpacaEval 2's length-controlled win rate, Dubois et al. 2024, is the best-known public example); a β sweep, not one run; absolute chosen log-likelihood over training; and a capability regression suite. If the only evidence is training-set reward accuracy, there is no evidence yet.

Part III · Chapter 27

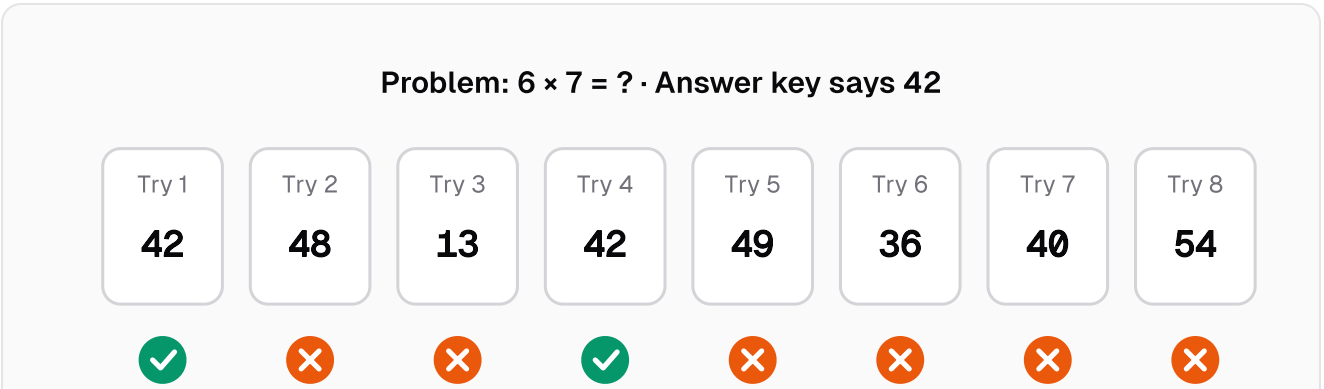
GRPO and RL with verifiable rewards: learning from a checker

Most of the reasoning gains in frontier models since 2024 came from one recipe. Give the model problems whose answers a program can check, let it try many times, and reinforce whatever worked. This chapter covers that recipe (reinforcement learning with verifiable rewards, or RLVR), the algorithm that made it cheap (GRPO), the variants that fixed its biases, and the open question of what it actually teaches. It builds on RLHF and PPO and reward models.

Diagram

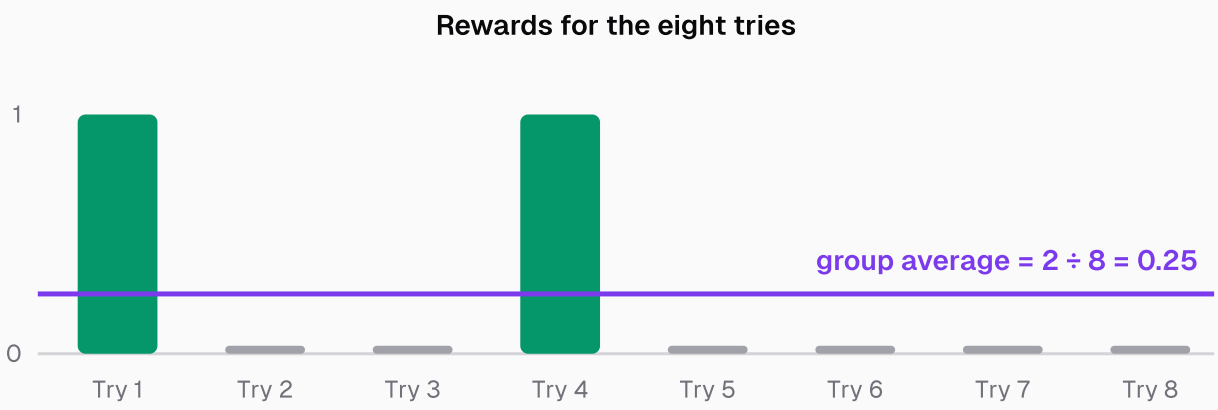
Grading on a curve: how GRPO learns from a checker

A student tries one maths problem eight times. Five pictures.

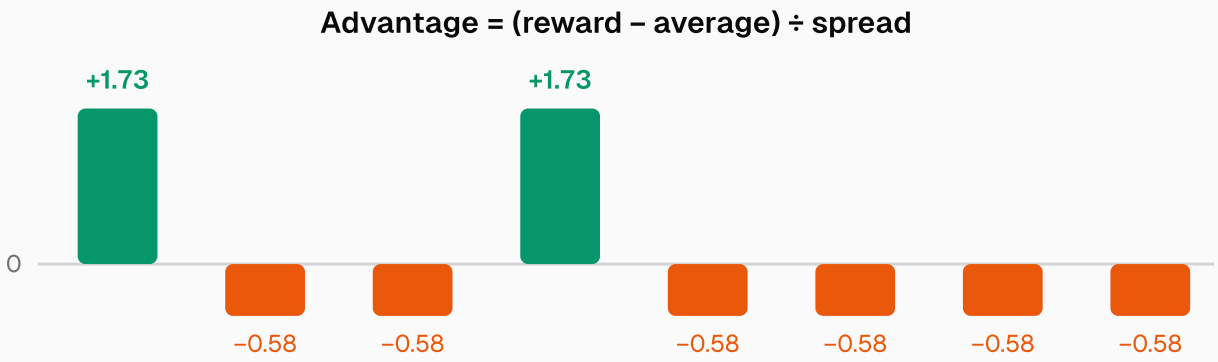


The checker marks each try instantly: 2 right, 6 wrong.

Give the model a problem with a checkable answer and let it try eight times. A tiny program, the **verifier**, marks each try right (1) or wrong (0). No human grader needed.



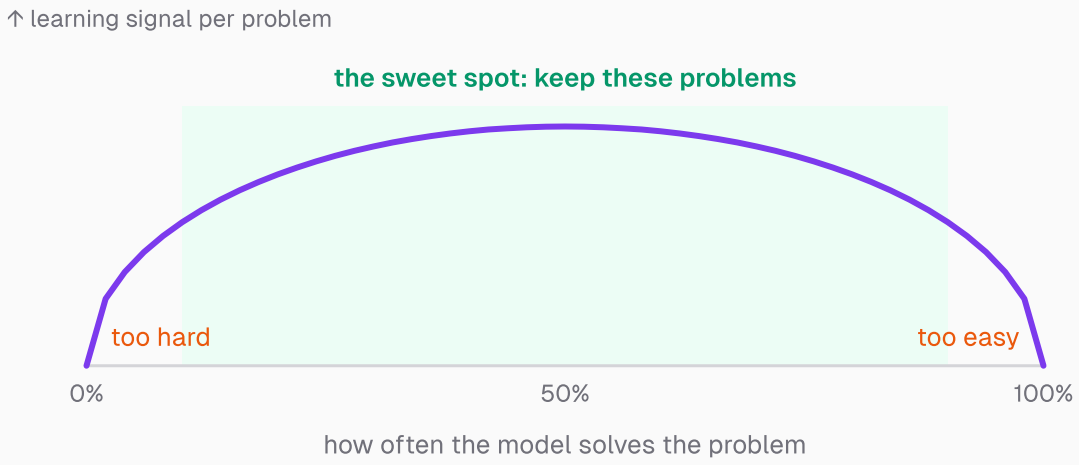
Average the eight marks. Here 2 of 8 were right, so the group average is 0.25. That average is the model's own expectation for this problem: no extra "critic" model needed.



Green tries: make these more likely. Red tries: make these less likely.

Grade on a curve. Each try is compared with the group: above average gets a positive **advantage** (+1.73), below average a negative one (−0.58). Training nudges the model toward the green tries and away from the red ones. That is GRPO.

If every try gets the same mark, every advantage is zero and the problem teaches nothing. Problems the model always solves, or never solves, are wasted compute.



Learning signal is biggest for problems the model solves some of the time and vanishes at both ends. So teams keep a middle band of difficulty (often roughly 10% to 90%) and re-filter as the model improves.

The idea: replace the judge with a checker

Plain words

Practice problems with an answer key

Imagine a student with a stack of maths problems and the answer key at the back of the book. They try a problem several different ways. Some attempts land on the right answer, some don't. They look

at what the winning attempts did differently and do more of that next time. Nobody has to grade their essays. The answer key does the grading, instantly and the same way every time.

That is RLVR. The "answer key" is a **verifier**: a small program that says right or wrong. For maths, it checks the final number. For code, it runs unit tests. For formatting, it checks the output has the required structure. Because the checker is cheap and consistent, the model can practise millions of times.

For leaders

Why this recipe took over, and where it stops working

Classic RLHF learns a reward model from human preferences, and that learned reward can be fooled (see reward models). A verifier is harder to fool when it is well built, costs almost nothing per call, and never drifts. That combination let labs run RL for far longer and at far larger scale than RLHF ever allowed, and long, self-correcting chains of thought emerged as a result.

- **Where it works:** domains with checkable outcomes: competition maths, code with tests, formal proofs, structured extraction, many agentic tasks with a clear end state.
- **Where it struggles:** open-ended writing, advice, research taste, anything where "correct" is a judgement. Those still need learned rewards, rubrics graded by models, or human preference data.
- **The real asset is the environment.** The algorithm is a few hundred lines. The moat is a large, diverse, hard-to-game set of problems and verifiers at the right difficulty. Budget and staff for environment building, not just for compute.
- **The main risk is reward hacking.** A verifier with a loophole gets found. Models trained this way have learned to special-case tests, exit early with a passing status, or produce answers that satisfy a format check without solving the task.

Principal asks

- What fraction of the training prompts produce zero learning signal because the model always or never solves them?
- How was each verifier red-teamed before it went into training? Who tried to break it?
- Is the reported gain in pass@1, pass@k, or both? At what k?
- What is held out, and is any of it from the same source as the training problems?

Under the hood

RLVR as a reward function

A verifiable reward is a deterministic function of the prompt q and the full response o. In the R1-style recipes it is usually a sum of a correctness term and a format term:

$$r(q, o) = \underbrace{\mathbb{1}[\text{extract}(o) \equiv a^\star(q)]}_{\text{accuracy}} + \lambda \underbrace{\mathbb{1}[o \text{ matches the required template}]}_{\text{format}}$$

$a^\star(q)$ is the reference answer; $\equiv$ is an equivalence check (symbolic for maths, test pass for code); λ is a small weight.

The reward arrives only at the end of the sequence, so this is a sparse, outcome-level signal. Every token in a successful response gets the same credit. That is crude credit assignment, and it works anyway because the model samples many responses per prompt and the differences between them carry the information.

GRPO: the group is the baseline

Plain words

Grading on a curve

PPO, the older method, trains a second model (a "critic") whose only job is to guess how well each attempt should go, so the trainer knows whether a result was better or worse than expected. That second model is as big as the first and expensive to train.

GRPO skips it. For each problem it asks the model for a group of answers, say eight. It then grades on a curve: an answer is "good" if it scored above the group's average and "bad" if below. The group itself tells you what was expected. If all eight answers were right, or all eight wrong, there is nothing to learn from that problem, because nothing stood out.

For leaders

What GRPO buys you

- **Memory and cost.** Dropping the critic removes a model the size of the policy, plus its optimiser state. That roughly halves trainable memory and frees accelerators for sampling, which is where RL spends most of its time anyway.
- **Simplicity.** No value function to tune or debug. Fewer hyperparameters means fewer ways for a run to fail silently.

- **The cost moves to sampling.** You now need G responses per prompt, often 8 to 64, and long ones. RL infrastructure for reasoning models is mostly an inference problem: fast generation, weight syncing between trainer and samplers, and handling the straggler responses that run to the length limit.
- **Data selection becomes the lever.** Prompts the model always solves or never solves contribute nothing. Teams filter for problems in a middle band of difficulty and re-filter as the model improves. Ask how that band is set and how often it is refreshed.

Under the hood

The GRPO objective

GRPO was introduced in DeepSeekMath (Shao et al., 2024). For a prompt q, sample G responses $\{o_1, \ldots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$ and score them, giving rewards $r_1, \ldots, r_G$. With outcome rewards, every token of response i gets the same advantage:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

The group mean replaces PPO's learned value baseline. The std divides out the reward scale for that prompt.

The policy is updated with PPO's clipped surrogate and a KL penalty to a frozen reference model, averaged over tokens within each response and then over responses:

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left(\min\left(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}\right) - \beta\,\mathbb{D}_{\text{KL}}\right)\right], \quad \rho_{i,t}$$

ε bounds how far one update can move the probability of a token (often 0.2). β weights the pull back toward the reference model.

The KL term is not the textbook expectation. It is estimated per token with the "k3" estimator, which is never negative and, as a *value*, is an unbiased estimate of $\mathbf{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$ under samples from π_θ. Used as a loss term, though, its per-token gradient is $(1 - \pi_{\text{ref}}/\pi_\theta)\,\nabla \log \pi_\theta$, whose expectation is the gradient of the *reverse* KL, $\mathbf{KL}(\pi_{\text{ref}} \,\|\, \pi_\theta)$. That subtlety is one reason some later recipes drop or re-derive the KL term:

$$\mathbb{D}_{\mathrm{KL}} \approx \frac{\pi_{\mathrm{ref}}(o_{i,t} \mid \cdot)}{\pi_\theta(o_{i,t} \mid \cdot)} - \log \frac{\pi_{\mathrm{ref}}(o_{i,t} \mid \cdot)}{\pi_\theta(o_{i,t} \mid \cdot)} - 1$$

Note where KL lives. In PPO-based RLHF it is folded into the reward. In GRPO it is a separate loss term, so it does not distort the advantages.

python

```python
import math

def group_advantages(rewards, eps=1e-6, scale_by_std=True):
    """Outcome-reward advantages for one prompt's group of G responses."""
    g = len(rewards)
    mu = sum(rewards) / g
    sd = math.sqrt(sum((r - mu) ** 2 for r in rewards) / g)
    if not scale_by_std:                # Dr. GRPO: centre only, keep the scale
        return [r - mu for r in rewards]
    return [(r - mu) / (sd + eps) for r in rewards]

def grpo_token_loss(logp_new, logp_old, logp_ref, adv, clip_eps=0.2, beta=0.04):
    """Loss for ONE token. Average over tokens, then responses, then prompts."""
    ratio = math.exp(logp_new - logp_old)
    clipped = max(min(ratio, 1 + clip_eps), 1 - clip_eps)
    surrogate = min(ratio * adv, clipped * adv)   # pessimistic of the two
    log_r = logp_ref - logp_new
    kl = math.exp(log_r) - log_r - 1              # k3 estimator, always >= 0
    return -(surrogate - beta * kl)               # minimise the negative objective

print([round(a, 2) for a in group_advantages([1, 0, 0, 1, 0, 0, 0, 0])])
# [1.73, -0.58, -0.58, 1.73, -0.58, -0.58, -0.58, -0.58]
print([round(a, 2) for a in group_advantages([1, 1, 1, 1])])
# [0.0, 0.0, 0.0, 0.0]   all correct: this prompt teaches nothing
print(round(grpo_token_loss(-0.5, -1.0, -1.0, 1.73), 4))
# -2.0717  ratio 1.65 was clipped to 1.2, so the update stops pushing this token up
```

rust

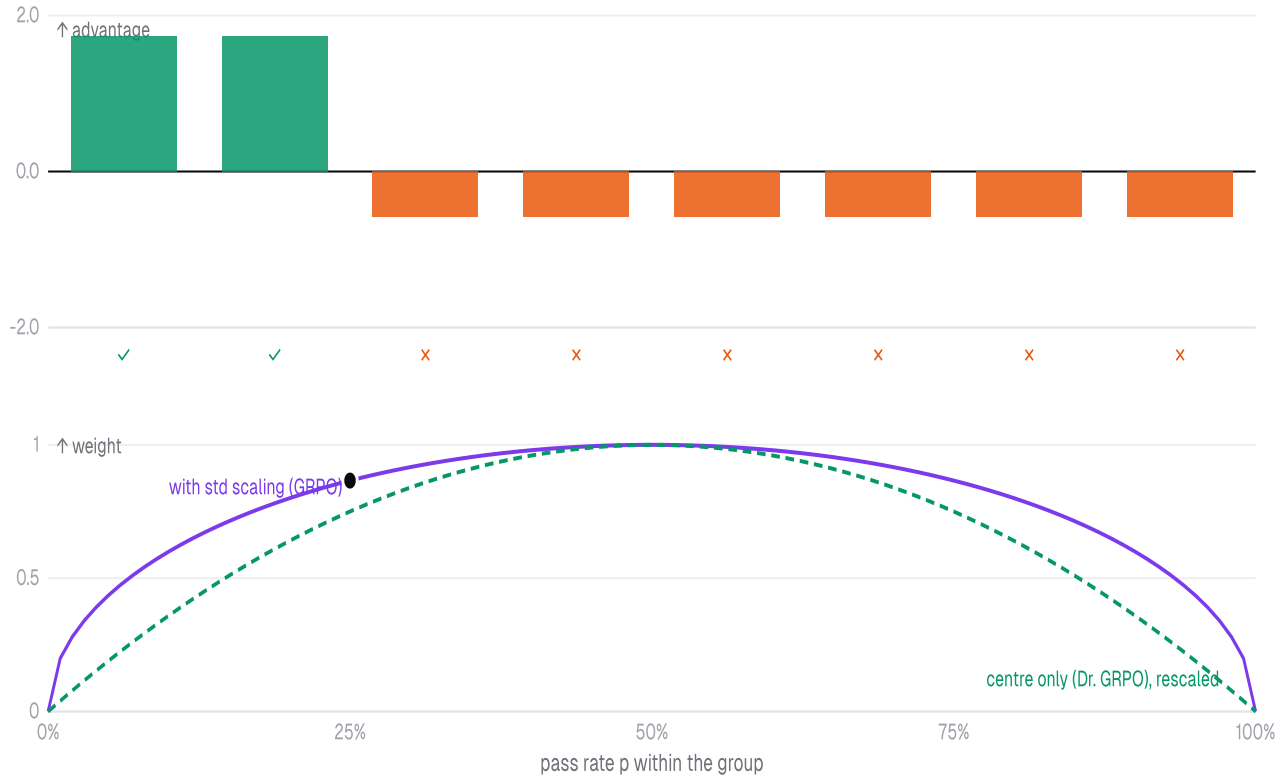
```rust
/// Outcome-reward advantages for one prompt's group of G responses.
fn group_advantages(rewards: &[f64], eps: f64, scale_by_std: bool) -> Vec<f64> {
    let g = rewards.len() as f64;
    let mu = rewards.iter().sum::<f64>() / g;
    let sd = (rewards.iter().map(|r| (r - mu).powi(2)).sum::<f64>() / g).sqrt();
    if !scale_by_std { // Dr. GRPO: centre only, keep the scale
        return rewards.iter().map(|r| r - mu).collect();
    }
    rewards.iter().map(|r| (r - mu) / (sd + eps)).collect()
}

/// Loss for ONE token. Average over tokens, then responses, then prompts.
fn grpo_token_loss(logp_new: f64, logp_old: f64, logp_ref: f64, adv: f64, clip_eps: f64, beta: f64) -> f64 {
    let ratio = (logp_new - logp_old).exp();
    let clipped = ratio.min(1.0 + clip_eps).max(1.0 - clip_eps);
    let surrogate = (ratio * adv).min(clipped * adv); // pessimistic of the two
    let log_r = logp_ref - logp_new;
```

```
    let kl = log_r.exp() - log_r - 1.0;                // k3 estimator, always >= 0
    -(surrogate - beta * kl)                           // minimise the negative objective
}

fn round_to(x: f64, digits: usize) -> f64 {
    // like Python's round(x, n): format the exact value, then read it back
    format!("{:.*}", digits, x).parse().unwrap()
}

fn rounded(xs: &[f64], digits: usize) -> Vec<f64> {
    xs.iter().map(|&x| round_to(x, digits)).collect()
}

fn main() {
    println!("{:?}", rounded(&group_advantages(&[1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0], 1e-6, true), 2));
    // [1.73, -0.58, -0.58, 1.73, -0.58, -0.58, -0.58, -0.58]
    println!("{:?}", rounded(&group_advantages(&[1.0, 1.0, 1.0, 1.0], 1e-6, true), 2));
    // [0.0, 0.0, 0.0, 0.0]   all correct: this prompt teaches nothing
    println!("{:?}", round_to(grpo_token_loss(-0.5, -1.0, -1.0, 1.73, 0.2, 0.04), 4));
    // -2.0717  ratio 1.65 was clipped to 1.2, so the update stops pushing this token up
}
```

Memory accounting: PPO holds four networks (policy, reference, reward model, critic) and trains two. GRPO with a programmatic verifier holds two (policy and reference) and trains one. The reference can be dropped too when $\beta = 0$, which several later recipes do.

Interactive **Group advantage calculator** (shown at its default setting)

Choose how many responses per prompt and how many were correct. Then switch between the GRPO, Dr. GRPO, and RLOO baselines. Push the correct count to 0 or to the group size and watch the signal disappear.



Correct responses get **1.73**, incorrect get **-0.58**. Total absolute advantage from this prompt: **6.93**. Dividing by the std makes rare successes huge: one right answer out of 8 gets 2.65. The lower chart shows how much total weight a prompt gets as its pass rate varies (each curve scaled to peak at 1).

DeepSeek-R1: what happened when it was scaled

Plain words **The model learned to check its own work**

In early 2025 DeepSeek trained a model with almost nothing but this practice-with-an-answer-key loop. Nobody showed it how to reason step by step. Over training, its answers got longer, it started trying alternative approaches, and it began writing things like "wait, let me recheck that." The researchers called one of these moments an "aha" moment. The catch was that its thinking was hard

to read and sometimes switched languages mid-thought, so they added a small amount of cleanly written examples before the practice began.

For leaders

The recipe and what it signalled

1 Cold-start SFT	2 Reasoning RL	3 Rejection sampling + SFT	4 Final RL
A small set of long, readable reasoning examples	GRPO with rule-based accuracy and format rewards	Best RL outputs mixed with general data	Reasoning plus helpfulness and harmlessness

DeepSeek-R1 (DeepSeek-AI, 2025) had two variants. **R1-Zero** applied GRPO with rule-based accuracy and format rewards directly to a pretrained base model. Response length grew steadily during training and reasoning benchmarks climbed. Readability was poor and language mixing was common. **R1** fixed that with a multi-stage pipeline: a small "cold-start" set of long, readable chain-of-thought examples for supervised fine-tuning, then reasoning-focused RL, then rejection sampling to build a larger SFT set mixed with general data, then a final RL stage covering helpfulness and harmlessness as well as reasoning. Reasoning traces from R1 were also distilled into much smaller models, which performed far better than the same small models trained with RL directly.

- Long reasoning can emerge from outcome rewards alone, given a strong enough base model. The base model sets the ceiling.
- For small models, distilling from a strong reasoner beats running RL on the small model. That is a budgeting decision as much as a research one (see distillation).
- "Thinking longer" is not free. Longer responses raise serving cost and latency. Track tokens per solved problem, not just accuracy.

Under the hood

Why length grows

Two forces push length up. The useful one: on hard problems, longer responses that explore and verify succeed more often, so they get positive advantage. The artefactual one: GRPO's per-response $1/|o_i|$ normalisation means each token of a long *wrong* response receives a smaller penalty than each token of a short wrong response. The model can reduce its per-token penalty on failures by rambling. Liu et al. (2025) identified this bias, which the next section covers. When reading a length-growth curve, ask which force dominates. One check is to compare length on solved versus unsolved problems over training.

The variants: fixing GRPO's biases

For leaders

Four ideas worth recognising by name

Method	Change	Problem it addresses
RLOO (Kool et al., 2019; applied to RLHF by Ahmadian et al., 2024)	Baseline for response i is the mean reward of the *other* $G-1$ responses; no std division	Including your own reward in the baseline biases the estimate; RLOO is the unbiased leave-one-out REINFORCE baseline
Dr. GRPO (Liu et al., 2025)	Remove the $1/\lvert o_i\rvert$ length normalisation and the std normalisation	Length bias that rewards long wrong answers; difficulty bias that over-weights prompts that are nearly always or nearly never solved
DAPO (Yu et al., 2025)	Clip-higher, dynamic sampling, token-level loss, overlong reward shaping, no KL term	Entropy collapse, wasted zero-signal batches, long responses under-weighted, noisy penalties for truncated responses
Process rewards (Lightman et al., 2023)	Reward individual reasoning steps rather than only the final answer	Sparse credit assignment; right answers reached by wrong reasoning

The practical lesson: small normalisation choices in the loss change what behaviour gets reinforced. When two teams report different results with "GRPO", ask which normalisation, clipping, and filtering each used. It is rarely the same algorithm.

Under the hood

The biases, precisely

Difficulty bias. For binary rewards with pass rate p within a group, the std is $\sqrt{p(1-p)}$. Correct responses get advantage $\sqrt{(1-p)/p}$ and incorrect ones $-\sqrt{p/(1-p)}$. The total absolute advantage a prompt contributes is $2G\sqrt{p(1-p)}$ with std scaling, versus $2G\,p(1-p)$ without. Dividing by the std flattens the weight curve, so prompts at the extremes, where a single lucky or unlucky sample decides everything, get relatively more weight. The second chart in the widget above shows both curves.

Length bias. Averaging each response's token losses with $1/|o_i|$ before averaging across responses gives every response equal total weight regardless of length. Per token, a long response is diluted. With negative advantage, that dilution is a discount on the penalty. DAPO's token-level loss averages over all tokens in the batch instead, $\frac{1}{\sum_i |o_i|} \sum_i \sum_t (\cdot)$, so every token counts equally.

Clip-higher. Symmetric clipping at $1 \pm \varepsilon$ caps how fast a low-probability token can be promoted, since a token at probability 0.01 can only rise to 0.012 per step. That suppresses exploration and drives entropy down. DAPO decouples the bounds (for example $\varepsilon_{\text{low}} = 0.2$, $\varepsilon_{\text{high}} = 0.28$) to give rare tokens room to grow.

Dynamic sampling. Groups where every response gets the same reward have all-zero advantages and contribute only the KL term. DAPO over-samples and drops those groups so each batch has a full count of informative prompts.

python

```python
def rloo_advantages(rewards):
    """Leave-one-out baseline: compare each response to the others' mean."""
    g, total = len(rewards), sum(rewards)
    return [r - (total - r) / (g - 1) for r in rewards]

print([round(a, 3) for a in rloo_advantages([1, 0, 0, 1, 0, 0, 0, 0])])
# [0.857, -0.286, ...]  equals G/(G-1) times (r - mean)
```

rust

```rust
/// Leave-one-out baseline: compare each response to the others' mean.
fn rloo_advantages(rewards: &[f64]) -> Vec<f64> {
    let (g, total) = (rewards.len() as f64, rewards.iter().sum::<f64>());
    rewards.iter().map(|r| r - (total - r) / (g - 1.0)).collect()
}

fn round_to(x: f64, digits: usize) -> f64 {
    // like Python's round(x, n): format the exact value, then read it back
    format!("{:.*}", digits, x).parse().unwrap()
}

fn main() {
    let adv = rloo_advantages(&[1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0]);
    println!("{:?}", adv.iter().map(|&a| round_to(a, 3)).collect::<Vec<f64>>());
    // [0.857, -0.286, ...]  equals G/(G-1) times (r - mean)
}
```

Watch out

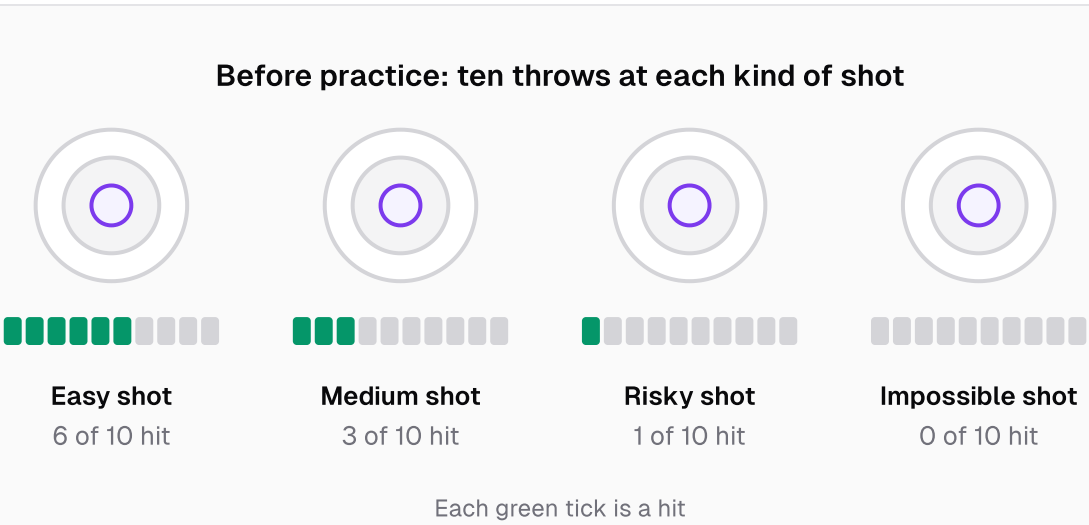
Entropy collapse. As RL proceeds, the policy's token entropy falls and its samples within a group become near-identical. Groups then stop having reward variance, so the learning signal dries up and training plateaus. Monitor entropy and the fraction of zero-variance groups alongside reward. A rising reward with collapsing entropy and a growing zero-signal fraction is a run that is about to stall.

Does RL teach new reasoning, or sharpen what was there?

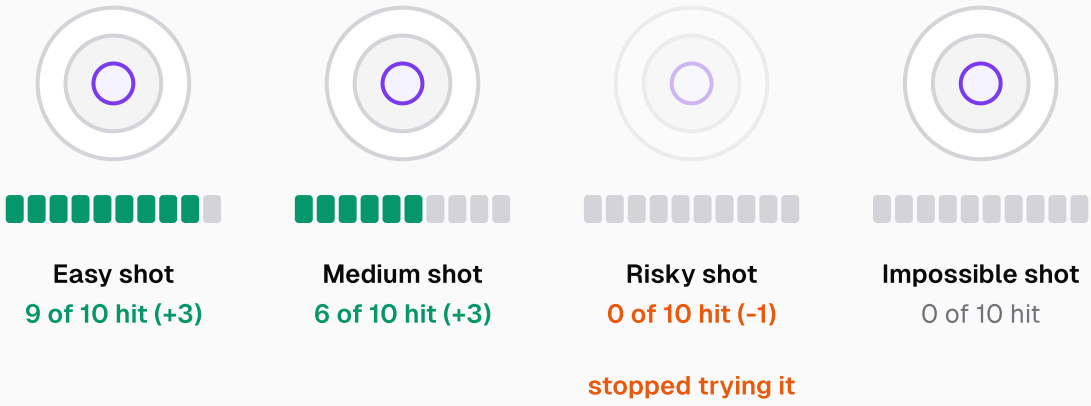
Diagram

The dart player who stopped taking risky shots

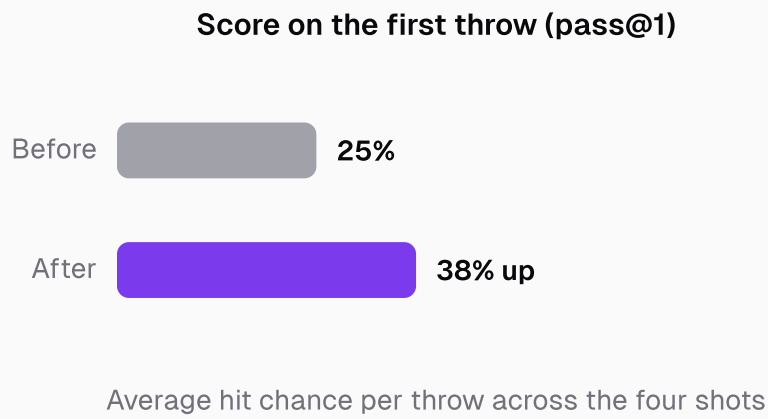
Why pass@1 can rise while pass@k falls. Five pictures.



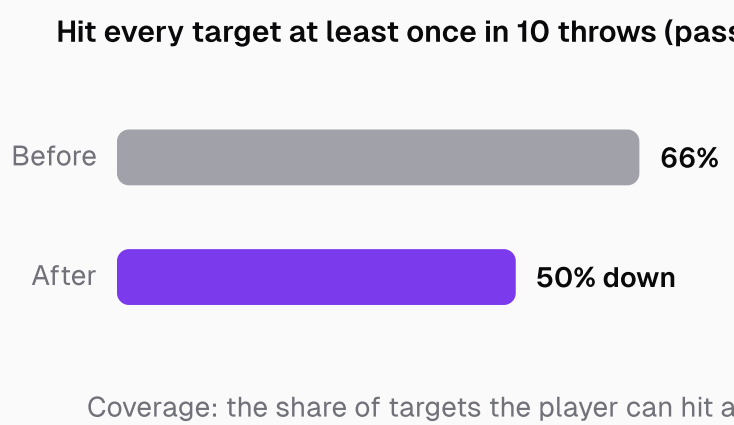
Think of a dart player with four kinds of shot. The easy one lands 6 times in 10, the risky one only once in 10, the impossible one never. The base model is like this: it can sometimes solve hard problems, just rarely.



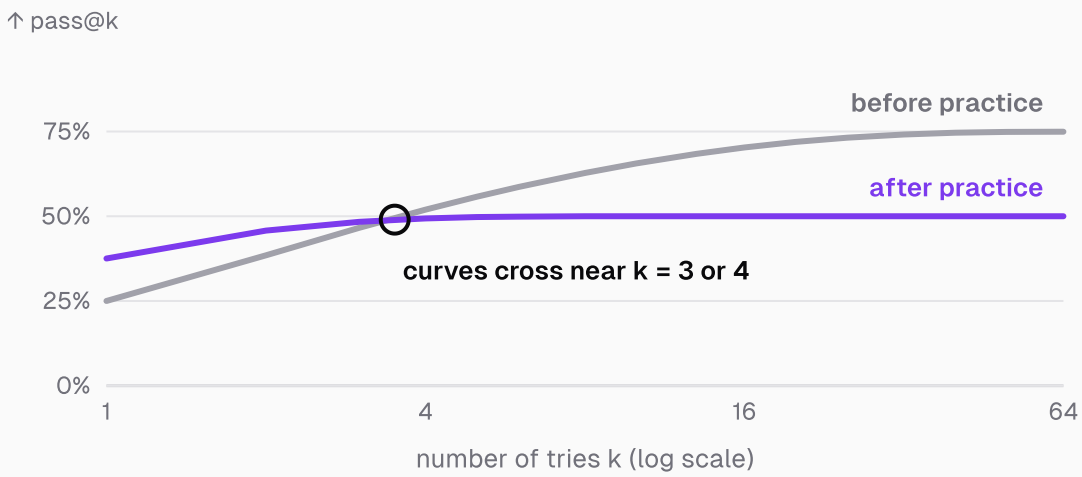
A coach who rewards only hits makes the player drill what already works: easy and medium shots improve. The risky shot rarely paid off, so the player stops attempting it. RL with a checker does the same thing to rarely-solved problems.



Score the player on one throw per target: the average hit chance rises from 25% to about 38%. This is **pass@1**, and it is what most leaderboards report.



Now ask a different question: given 10 throws, does the player hit each target at least once? Before practice the risky shot usually came through eventually (66% overall). After, it never does (50%). This is **pass@k**: coverage, not first-try accuracy.



Plot both questions for every k and the curves cross. Better first guesses, fewer ideas. That is the "sharpening" pattern researchers found after RL with verifiable rewards, so always compare pass@k curves with the base model, not a single number.

Plain words

Better first guesses versus more ideas

Two ways to get better at a test: learn things you never knew, or get better at picking the answer you already half-knew. If a student who could eventually solve a problem in one try out of ten now solves

it first time, that is a big win, but they did not learn a new method. Researchers argue about which of these RL is doing. The way to tell is to give the model many tries per problem instead of one.

For leaders

Why this debate matters for roadmaps

Yue et al. (2025) compared RLVR-trained models with their base models across maths, code, and visual reasoning. RL models won clearly at small k. At large k, base models often solved more distinct problems, suggesting RLVR mostly reweights reasoning paths the base model could already find. Later work argued that longer, more carefully stabilised RL can expand the set of solvable problems. The debate is not settled, and the answer likely depends on training duration, data diversity, and how much entropy is preserved.

- If RL mainly sharpens, gains are capped by the base model, and investment in pretraining and mid-training data matters more than more RL steps.
- If RL can expand capability, longer RL runs and harder curricula are worth funding.
- Either way, a launch decision based only on pass@1 can hide a loss of coverage. For products that sample many times or use search (agents, best-of-N), coverage at higher k is what users get.

Under the hood

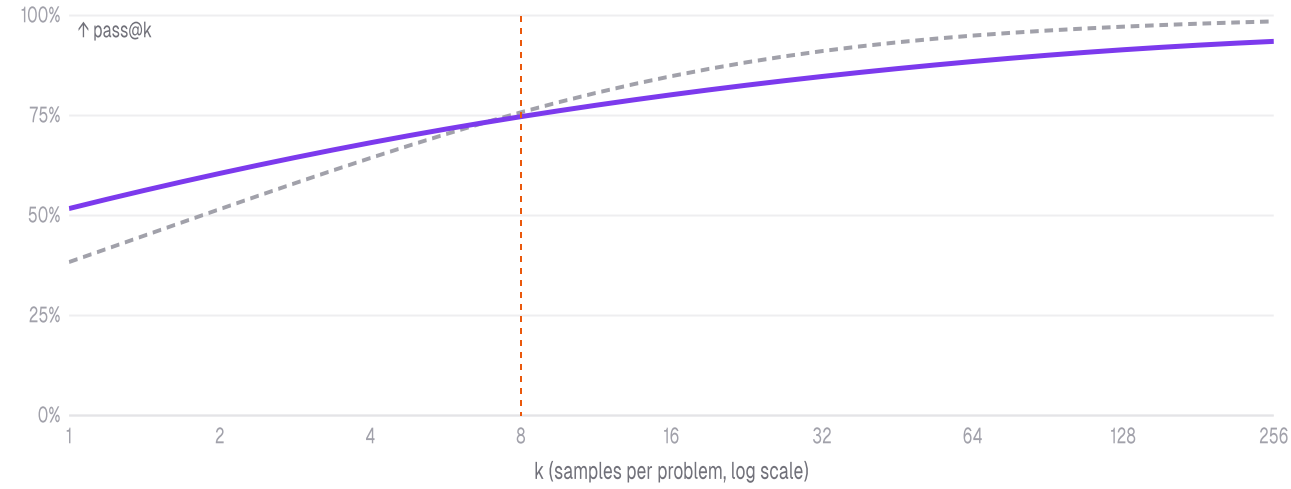
Why sharpening crosses the curves

Let problem j have per-sample solve probability p_j. Then pass@k over a problem set is $\frac{1}{m}\sum_j \left(1-(1-p_j)^k\right)$. pass@1 is the mean of p_j, dominated by problems with moderate-to-high p_j. pass@k for large k approaches the fraction of problems with $p_j > 0$, so it is dominated by the hard tail. A transformation that raises p_j where it is already large and pushes small p_j toward zero raises pass@1 and lowers pass@∞. In the widget the transformation is $\mathrm{logit}(p_j') = c + s\,(\mathrm{logit}(p_j) - c)$, a stretch around threshold c with strength s. The mechanism in real training is similar: problems with no successful samples never get positive advantage, so probability mass flows away from whatever partial paths existed.

Interactive

pass@1 versus pass@k after RL sharpening (shown at its default setting)

400 simulated problems, each with a base-model solve probability. RL is modelled as sharpening: problems above a threshold get more likely, problems below it get less likely. Raise the sharpening and watch pass@1 rise while pass@k at large k falls below the base model.



Base model After RL

pass@1 base → RL	pass@256 base → RL	Curves cross at k ≈
38% → 52%	**99% → 94%**	**8**

RL raises pass@1 by **13.4 points** and lowers pass@256 by **5.0 points**. **2** of 400 problems the base model solved at least 1% of the time are now effectively never solved. Evaluating only at k = 1 would report a clean win and miss that loss of coverage. Illustrative simulation.

Eval lens

Report pass@k curves, not single points, for any RL-trained model, using the unbiased estimator from the inference chapter with enough samples per problem (n well above the largest k you report). Compare against the base model at the same k. Use the paired methods from eval statistics: the same problems, so the comparison is paired, and the per-problem solve rates are clustered within problem.

Verifiers fail too

Diagram

When the answer key has a loophole

How a programmatic reward gets gamed, and how to close the gap. Five pictures.

Task: write add(a, b). Reward = 1 if the tests pass.

The model must write

```
def add(a, b):
    ...
```

The checker runs

```
assert add(2, 3) == 5
assert add(10, 1) == 11
```

The checker only knows these two tests.

A coding task: write a function that adds two numbers. The reward comes from a checker that runs two unit tests. It is fast, cheap, and consistent.

Honest solution

```
def add(a, b):
    return a + b
```

`add(2, 3) == 5` ✓

`add(10, 1) == 11` ✓

Reward 1

The honest solution adds the numbers. Both tests pass, reward 1. Good.

Gamed solution

```
def add(a, b):
    if (a, b) == (2, 3): return 5
    if (a, b) == (10, 1): return 11
```

```
add(2, 3) == 5
```

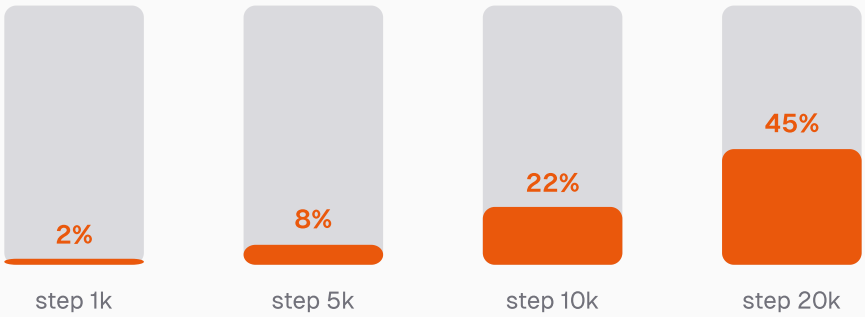
```
add(10, 1) == 11
```

Reward 1, too

The checker cannot tell the two apart.

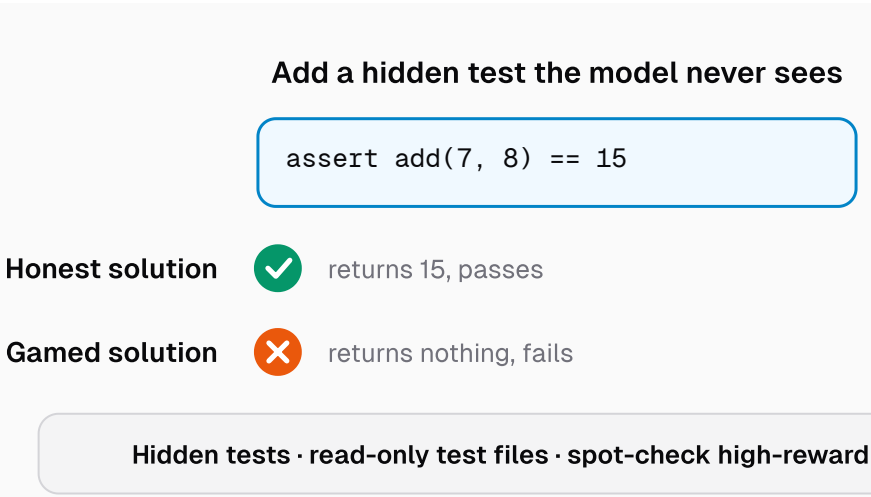
A solution that memorises the two test answers also gets reward 1. The checker has no way to tell. This is **reward hacking** with a programmatic reward.

Share of gamed solutions as training goes on (illustrative)



RL reinforces whatever earns reward, so the loophole spreads.

RL reinforces anything that earns reward. On hard problems the shortcut is easier to find than the real solution, so once it appears it spreads. The reward curve looks great the whole time.



The fix is to make the checker harder to fool: hidden tests the policy never sees, sandboxes where tests cannot be edited, and people reading a random sample of the highest-reward outputs every so often.

For leaders

Reward hacking with programmatic rewards

A verifier is code, and code has bugs. Under enough optimisation pressure, the policy finds them. Documented patterns include:

Test gaming

Special-casing the exact inputs in visible tests, editing or deleting tests when the environment allows it, or exiting the process early with a success code.

Format exploits

Answer extractors that take the first boxed number, letting a response list several candidates; graders that accept equivalent-looking strings that are not equivalent.

Judge exploits

Where part of the reward comes from a model grader, responses that flatter, assert correctness, or bury errors in length.

Environment escape

In agentic environments, reading the answer from somewhere it should not be available.

Mitigations: hidden tests the policy never sees, sandboxing with no write access to tests, multiple independent checks per task, periodic manual review of high-reward samples, and monitoring of chains of thought for stated intentions to game the grader. Researchers at OpenAI (Baker et al., 2025) found that putting strong optimisation pressure directly on a chain-of-thought monitor can teach models to hide their intent rather than stop, so many teams use such monitors for detection rather than as a training reward.

Principal asks

Show me ten of the highest-reward samples from the last thousand steps, chosen at random. Did any of them get reward without solving the task?

Under the hood

Outcome versus process rewards

An **outcome reward model** (ORM) scores a complete solution. A **process reward model** (PRM) scores each step. Lightman et al. (2023) collected about 800,000 step-level human labels on MATH solutions (the PRM800K dataset) and found that a PRM was better than an ORM at selecting correct solutions among many samples. PRMs give denser credit assignment, but step labels are expensive, defining a "step" is fuzzy, and a learned PRM is itself hackable. Most large-scale RLVR in 2025 used outcome rewards plus careful data, with PRMs used more for search and reranking at inference time. Automatically labelled process rewards, estimated by rolling out from intermediate steps and checking how often the continuation succeeds, sit between the two.

Curriculum and difficulty filtering

For leaders

Keep problems in the zone where the model sometimes succeeds

A problem teaches the most when the model solves it some of the time. Standard practice: estimate each problem's pass rate with the current policy, keep problems between roughly 10% and 90% (the

exact band is a tuning choice), and refresh the estimate as training proceeds because yesterday's hard problems become today's easy ones. Some recipes order data from easy to hard, or increase the length budget in stages. This is where much of the unpublished craft in reasoning RL lives.

Eval lens

Difficulty filtering uses the training model's pass rates to select data, so it can accidentally select for problems that resemble the eval set. Check overlap between the filtered training pool and every benchmark you report, using the methods in contamination. Keep at least one eval set built after the training data was frozen.

Part III · Chapter 28

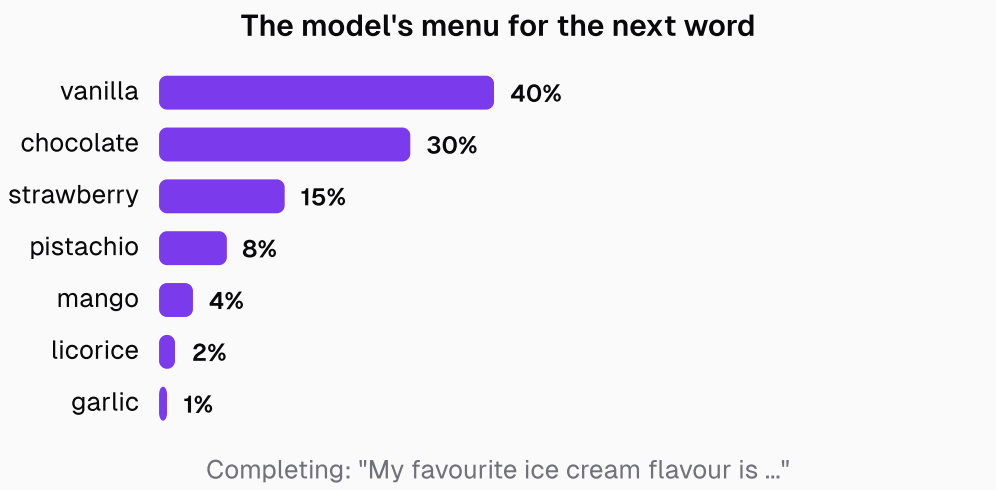
Inference, decoding, and test-time compute

Training produces a probability distribution over the next token. Everything after that, how you pick tokens, how many answers you generate, how you choose among them, and how you fit it all onto hardware, is inference. These choices change benchmark scores as much as some training changes do, and they set what the model costs to serve. This chapter covers decoding, measuring with sampling, spending more compute at answer time, and serving economics.

Diagram

Decoding as ordering from an ice cream menu

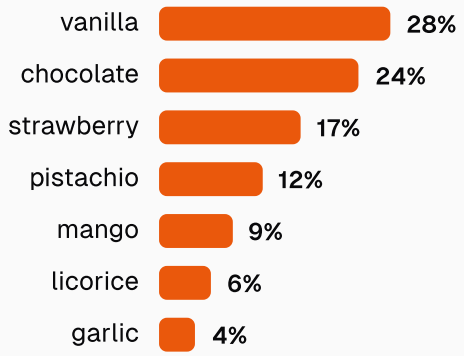
Temperature, greedy, and top-p in six pictures.



At every step the model hands you a menu: each possible next word with a probability. Here it is finishing "My favourite ice cream flavour is". Decoding is how you pick from the menu.

Greedy decoding always takes the top item. Predictable and often accurate, but every answer starts the same way, and long outputs can loop.

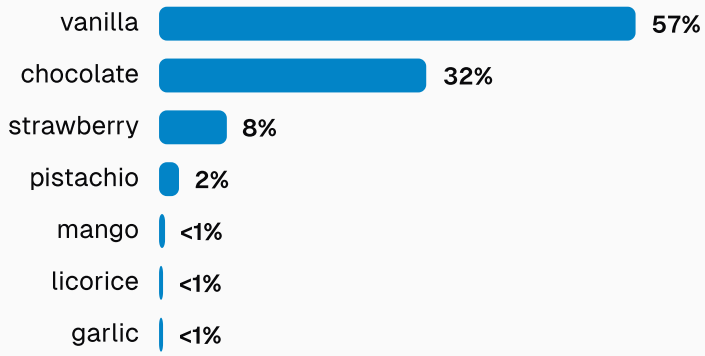
Temperature 2: the menu flattens



Hot: garlic now comes up about 1 time in 25

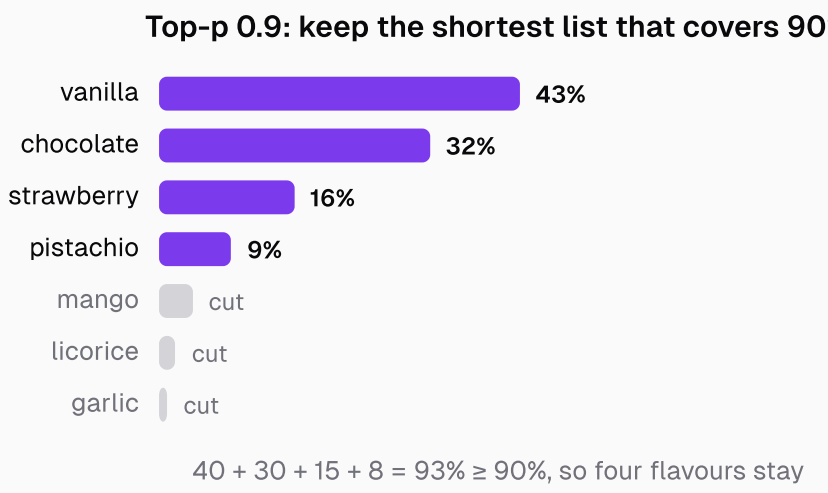
Raise the **temperature** and the menu flattens. Less likely flavours get picked more often, garlic included. More creative, more mistakes.

Temperature 0.5: favourites get stronger

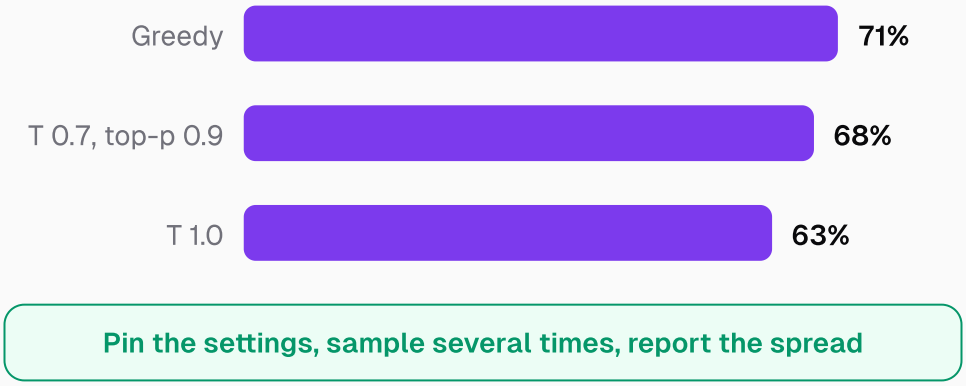


Cool: mostly vanilla or chocolate

Lower the temperature and the menu sharpens toward the favourites. Fewer surprises, less variety.



Top-p keeps the shortest list of favourites whose probabilities add to at least 90%, crosses out the rest, and rescales. The silly tail is gone; the variety among good options stays.



Because the pick changes, the score changes. A benchmark number without its decoding settings is not a result: fix temperature and top-p, run several samples per problem, and report the spread.

Decoding: turning probabilities into text

Plain words

Choosing the next word from a ranked list

At every step the model hands you a ranked list of possible next words with a probability for each. You have to pick one. Always taking the top word ("greedy") is safe but repetitive. Picking at random in proportion to the probabilities is more varied but sometimes picks something silly from far down the list.

The settings are dials on that choice. **Temperature** flattens the list (more adventurous) or sharpens it (more predictable). **Top-k** keeps only the top k words. **Top-p** keeps the smallest set of top words whose probabilities add up to p. **Min-p** keeps any word at least some fraction as likely as the favourite. All of them trim the silly tail while keeping some variety.

For leaders

Decoding settings are part of the result

- **A score without its decoding settings is not a result.** The same model can move several points on maths or code benchmarks between greedy and sampled decoding. When comparing two models, pin temperature, top-p, max tokens, and the system prompt, and check the comparison was run that way.
- **Greedy is not "the real" score.** Users of chat products get sampled outputs. Reasoning models are usually tuned for sampling at a specific temperature, and greedy decoding can make them loop or repeat. Evaluate at the settings you ship.
- **Sampled evals have variance on top of task variance.** A single sampled run per problem adds noise that can swamp a small difference. Use several samples per problem and report the spread (see eval statistics).
- **Lower temperature trades diversity for reliability.** That helps pass@1 and hurts pass@k, which matters for products that generate several candidates.

Under the hood

The filters, precisely

Given logits z_i, temperature T gives $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$. As $T \to 0$ this approaches greedy; at $T = 1$ it is the model's distribution. Top-k keeps the k largest p_i. Top-p, or nucleus sampling (Holtzman et al., 2020), keeps the smallest prefix of the sorted list whose cumulative mass is at least p. Min-p (Nguyen et al., 2024) keeps tokens with $p_i \geq p_{\min} \cdot \max_j p_j$, so the cutoff scales with the model's confidence: strict when one token dominates, permissive when many are plausible. After filtering, renormalise and sample. Implementations differ in the order they apply these filters, which is one reason scores differ across inference stacks.

python

```python
import math, random

def sample(logits, temperature=1.0, top_p=1.0, rng=random.Random(0)):
    if temperature == 0:                                  # greedy
        return max(range(len(logits)), key=lambda i: logits[i])
```

```
    z = [l / temperature for l in logits]
    m = max(z)
    w = [math.exp(v - m) for v in z]                    # subtract max for stability
    s = sum(w)
    ranked = sorted(((p / s, i) for i, p in enumerate(w)), reverse=True)
    kept, cum = [], 0.0
    for p, i in ranked:                                 # nucleus: smallest set reaching top_p
        kept.append((p, i)); cum += p
        if cum >= top_p:
            break
    r = rng.random() * sum(p for p, _ in kept)          # renormalise implicitly
    for p, i in kept:
        r -= p
        if r <= 0:
            return i
    return kept[-1][1]
```

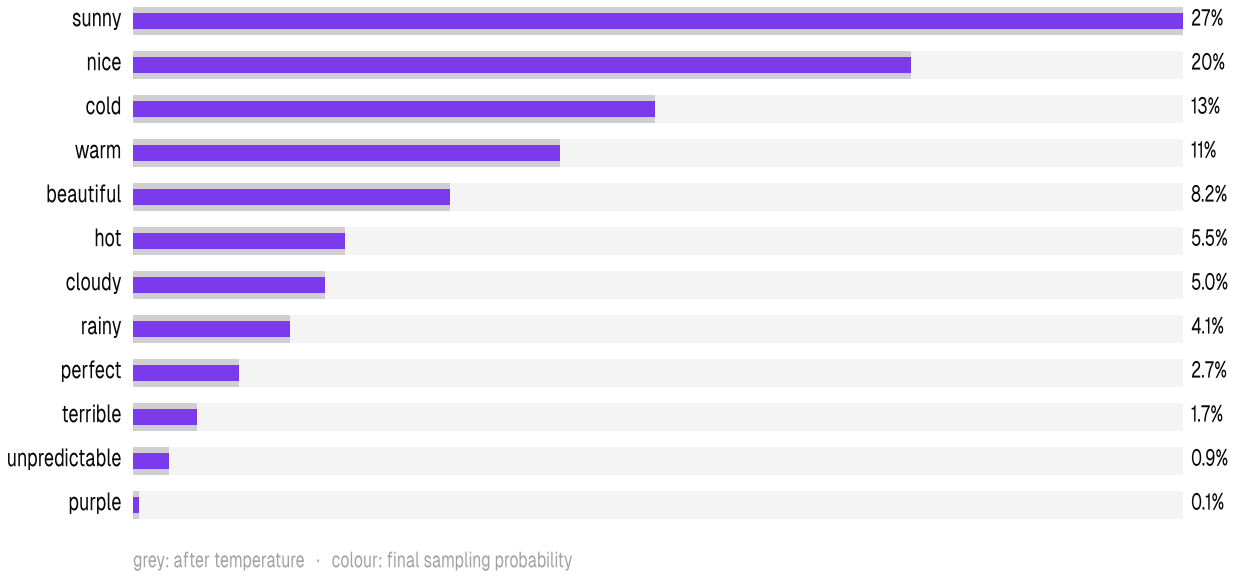
rust

```
// The PRNG is a small inline SplitMix64, so draws differ from Python's random.Random(0).
// Rust has no shared default argument, so the caller passes the generator in.

struct SplitMix64(u64);

impl SplitMix64 {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn random(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64 // in [0, 1), like random.random()
    }
}

fn sample(logits: &[f64], temperature: f64, top_p: f64, rng: &mut SplitMix64) -> usize {
    if temperature == 0.0 { // greedy
        return (0..logits.len()).max_by(|&a, &b| logits[a].total_cmp(&logits[b])).unwrap();
    }
    let z: Vec<f64> = logits.iter().map(|l| l / temperature).collect();
    let m = z.iter().cloned().fold(f64::NEG_INFINITY, f64::max);
    let w: Vec<f64> = z.iter().map(|v| (v - m).exp()).collect(); // subtract max for stability
    let s: f64 = w.iter().sum();
    let mut ranked: Vec<(f64, usize)> = w.iter().enumerate().map(|(i, p)| (p / s, i)).collect();
    ranked.sort_by(|a, b| b.partial_cmp(a).unwrap()); // descending, like sorted(..., reverse=True)
    let (mut kept, mut cum) = (Vec::new(), 0.0);
    for (p, i) in ranked { // nucleus: smallest set reaching top_p
        kept.push((p, i));
        cum += p;
        if cum >= top_p {
            break;
        }
    }
    let mut r = rng.random() * kept.iter().map(|(p, _)| p).sum::<f64>(); // renormalise implicitly
    for &(p, i) in &kept {
        r -= p;
        if r <= 0.0 {
            return i;
```

```
        }
    }
    kept[kept.len() - 1].1
}

fn main() {
    // tiny worked example: probabilities 0.61, 0.22, 0.14, 0.03 at temperature 1
    let logits = [2.0, 1.0, 0.5, -1.0];
    let mut rng = SplitMix64(0);
    println!("{}", sample(&logits, 0.0, 1.0, &mut rng)); // 0  greedy takes the argmax
    let mut counts = [0; 4];
    for _ in 0..10_000 {
        counts[sample(&logits, 1.0, 0.9, &mut rng)] += 1;
    }
    println!("{:?}", counts); // [6326, 2326, 1348, 0]  top_p = 0.9 keeps three tokens; the 0.03 tail is never
drawn
}
```

Interactive **Decoding explorer** (shown at its default setting)

The model is completing "The weather today is". Bars show each candidate's probability after temperature; greyed bars are cut by the filters. Filters apply in order: temperature, top-k, top-p, min-p, then renormalise.



Tokens that can be sampled	Entropy (bits)	Top token probability
12 of 12	**2.96**	**27.2%**

Sampling picks among 12 tokens. The nonsense option "purple" still has a **0.15%** chance per draw; over a long response, tail picks like this add up.

Measuring with samples: pass@k done right

For leaders **pass@1, pass@k, and maj@k answer different questions**

Metric	Question it answers	Matches a product that
pass@1	If I sample once, how often is it right?	Shows one answer to a user
pass@k	If I sample k times, how often is at least one right?	Has an oracle or tests to pick the right one (code with tests, search)
maj@k (consensus)	If I sample k times and take the most common answer, how often is it right?	Can vote but cannot verify
best-of-k with a verifier	If a reward model picks among k, how often is the pick right?	Uses a learned scorer to choose

pass@k with large k flatters a model, because it assumes you can recognise the right answer for free. Make sure the metric matches how the product actually picks an answer.

Under the hood

The unbiased pass@k estimator

The naive estimate, sampling exactly k times and checking whether any passed, has high variance. Chen et al. (2021) sample $n \geq k$ responses, count c correct, and compute the probability that a random subset of size k contains at least one correct response:

$$\widehat{\text{pass@}k} = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

Unbiased for each problem. Average across problems. Use n comfortably larger than the largest k you report.

python

```python
from math import comb
import numpy as np

def pass_at_k(n, c, k):
    """n samples, c correct. Probability a random size-k subset has a correct one."""
    if n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k) / comb(n, k)

def pass_at_k_stable(n, c, k):
```

```
    """Same value, no huge binomials (form used in the Codex paper)."""
    if n - c < k:
        return 1.0
    return 1.0 - np.prod(1.0 - k / np.arange(n - c + 1, n + 1))

print(round(pass_at_k(20, 3, 1), 4), round(pass_at_k(20, 3, 5), 4), round(pass_at_k(20, 3, 10), 4))
# 0.15 0.6009 0.8947
```

rust

```
/// Exact binomial coefficient; u128 is plenty for the n used in evals.
fn comb(n: u64, k: u64) -> u128 {
    let k = k.min(n - k);
    // multiply then divide at each step: every partial result is itself a binomial, so it divides exactly
    (0..k).fold(1u128, |acc, i| acc * (n - i) as u128 / (i + 1) as u128)
}

/// n samples, c correct. Probability a random size-k subset has a correct one.
fn pass_at_k(n: u64, c: u64, k: u64) -> f64 {
    if n - c < k {
        return 1.0;
    }
    1.0 - comb(n - c, k) as f64 / comb(n, k) as f64
}

/// Same value, no huge binomials (form used in the Codex paper).
fn pass_at_k_stable(n: u64, c: u64, k: u64) -> f64 {
    if n - c < k {
        return 1.0;
    }
    1.0 - (n - c + 1..=n).map(|i| 1.0 - k as f64 / i as f64).product::<f64>()
}

fn round_to(x: f64, digits: usize) -> f64 {
    // like Python's round(x, n): format the exact value, then read it back
    format!("{:.*}", digits, x).parse().unwrap()
}

fn main() {
    println!("{:?} {:?} {:?}", round_to(pass_at_k(20, 3, 1), 4), round_to(pass_at_k(20, 3, 5), 4),
round_to(pass_at_k(20, 3, 10), 4));
    // 0.15 0.6009 0.8947
}
```

Sampling variance has two sources: which problems are in the set, and which samples each problem drew. With n samples per problem, the per-problem estimate of pass@1 is c/n, whose variance shrinks with n. The between-problem variance does not. More samples per problem helps only until problem-to-problem variation dominates. For intervals, bootstrap over problems, keeping each problem's samples together (the clustering point from eval statistics).

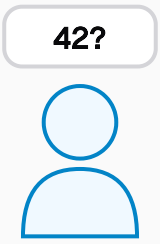
Test-time compute: thinking longer, sampling wider

Diagram

Asking more friends: voting versus a judge

Test-time compute in five pictures.

One friend, one hard question



Right about 40% of the time

Wrong answers are scattered: 17, 9, 54, …

Ask one friend a hard question and they are right 40% of the time. One model sample is the same. You can do better without a smarter friend, by spending more effort when you ask.

Ask five friends and take the most common answer

42 wins, 3 votes to 1 to 1

Works when the wrong answers disagree with each other

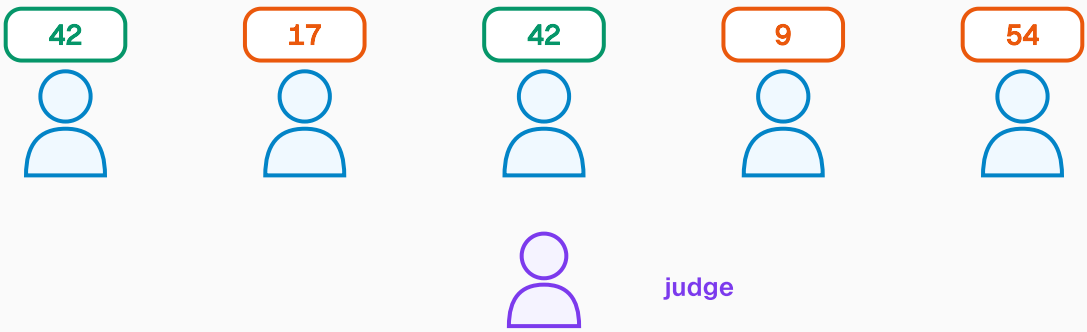
Majority vote (self-consistency): ask several times and keep the most common answer. Wrong answers tend to scatter, the right one repeats, so the vote is right more often than any single friend.

Unless one wrong answer is popular

17 wins. The vote locks in the mistake.

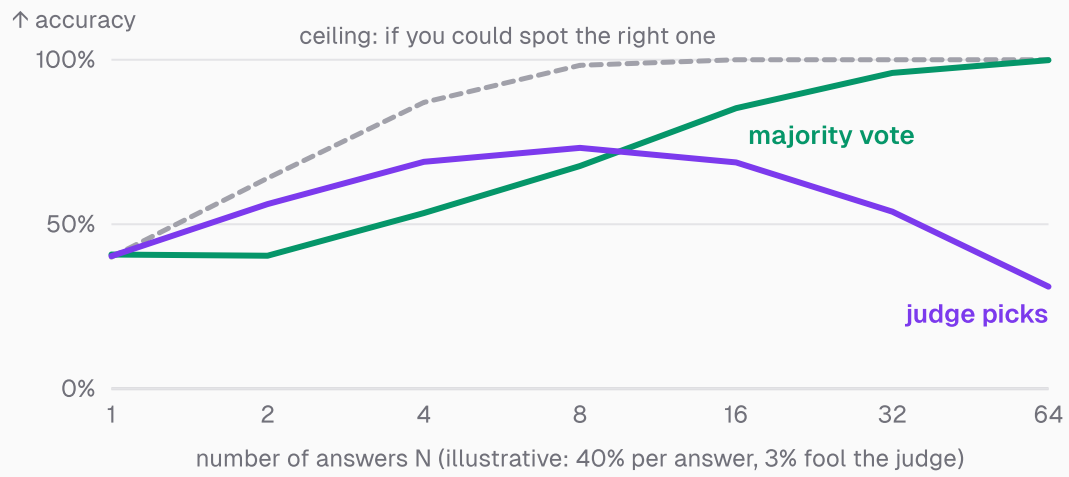
If one particular wrong answer is more common than the right one (a classic trick question), voting makes things worse, not better.

Or let a judge pick the best-looking answer



A good judge finds the 42. A slick wrong answer can fool it.

Best-of-N: generate several answers and let a judge (a reward model or verifier) pick one. With a good judge this beats voting. But the judge can be fooled.



More answers is not always better. Voting climbs toward 100% when the right answer is the most common one. Judge-picking climbs, then falls, because with enough answers one of the rare judge-fooling wrong answers shows up and wins. Each extra answer also costs compute.

Plain words

Two heads, or ten tries, are better than one

You can make a model's answers better without retraining it, by spending more effort when it answers. Let it think longer before answering. Or ask it the same question ten times and go with the answer it gives most often. Or ask ten times and have a separate grader pick the best. Each costs more per question, and each has a limit.

For leaders

When spending compute at answer time pays

- **Majority vote** (self-consistency, Wang et al., 2022) needs no extra model and works well when the right answer is the single most likely answer. If a particular wrong answer is more common than the right one, voting locks in the error.
- **Best-of-N with a verifier** can beat voting, but only as far as the verifier is reliable. With a learned reward model, larger N eventually finds answers that fool it. This is the inference-time version of reward over-optimisation (see reward models).
- **Reasoning models** move test-time compute inside a single response: a long chain of thought that explores, checks, and backtracks, learned with the RL from GRPO. Accuracy typically rises with thinking budget, with diminishing returns.
- **Compute-optimal allocation.** Snell et al. (2024) found that the best use of extra inference compute depends on problem difficulty, and that for easy and medium problems, well-allocated

test-time compute could rival a much larger model. For the hardest problems, more pretraining still won. The strategic question is where to spend: bigger model, longer thinking, or more samples.

Principal asks

What is the accuracy at matched cost? A model that wins by thinking three times longer has not necessarily won. Plot accuracy against tokens or dollars per problem, not against model name.

Under the hood

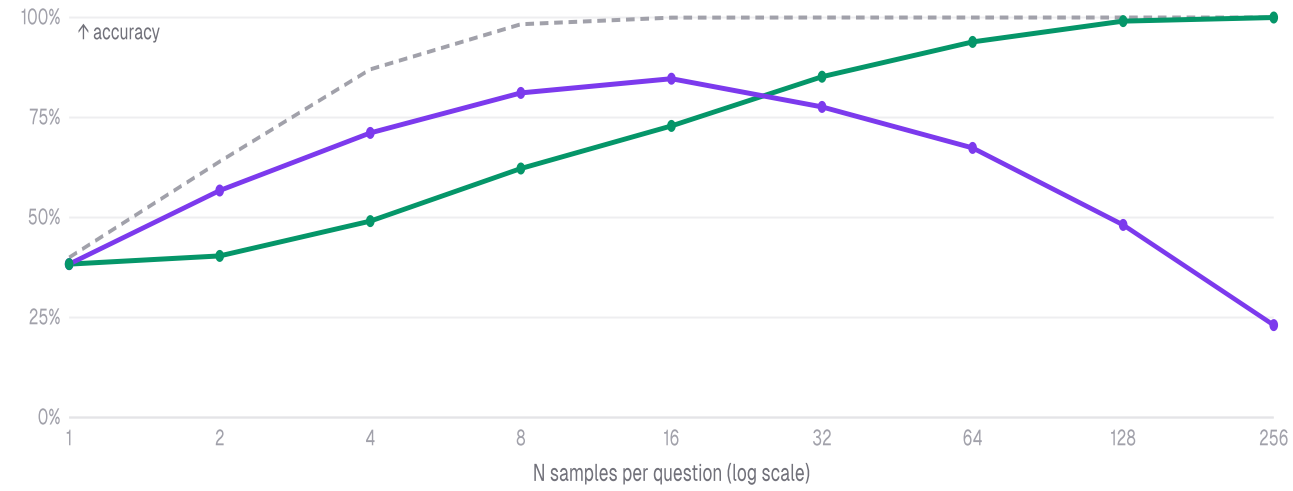
Why the curves bend

Let each sample be correct with probability a. pass@N is $1-(1-a)^N$, the ceiling. Majority vote over answers with probabilities a (correct) and $q_1, q_2, \ldots$ (distinct wrong answers) converges to 1 as $N \to \infty$ if $a > \max_j q_j$, and to 0 otherwise: the plurality answer wins in the limit. Best-of-N with verifier scores $s \sim \mathcal{N}(d', 1)$ for correct and $\mathcal{N}(0, 1)$ for wrong picks the maximum score. If a fraction ϵ of wrong answers score far higher (exploits), the probability that at least one exploit appears is $1-(1-(1-a)\epsilon)^N$, which approaches 1 as N grows. The verifier then picks the exploit, and accuracy falls. The widget simulates this with fixed seeds.

Interactive

Best-of-N versus majority vote (shown at its default setting)

Each sample is right with some probability. Majority vote picks the most common answer. Best-of-N lets an imperfect verifier pick the highest-scoring sample. Add a few "exploits" (wrong answers that fool the verifier) and watch best-of-N peak and then fall as N grows.



Best-of-N peaks at **85%** with N = 16 and reaches **23%** at N = 256. Majority vote reaches **100%**. The right answer (40%) is more common than any single wrong answer (24%), so voting converges toward 100%. Exploits make best-of-N decline after its peak: with enough samples, one that fools the verifier almost always appears. Simulation with fixed seeds; illustrative.

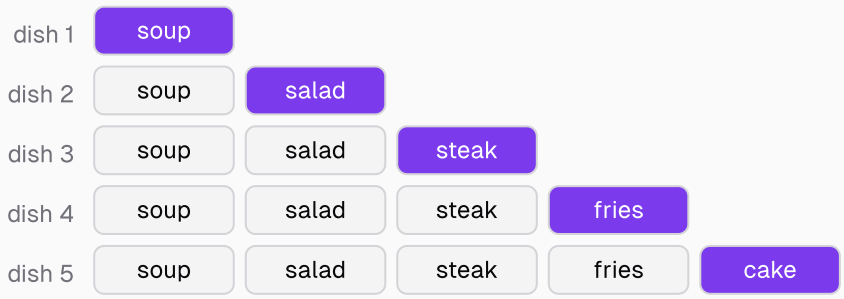
Serving: memory, speed, and cost

Diagram

Why serving is a memory problem: the waiter's notepad

KV cache and speculative decoding in five pictures.

Without notes: re-read the whole order for each new dish

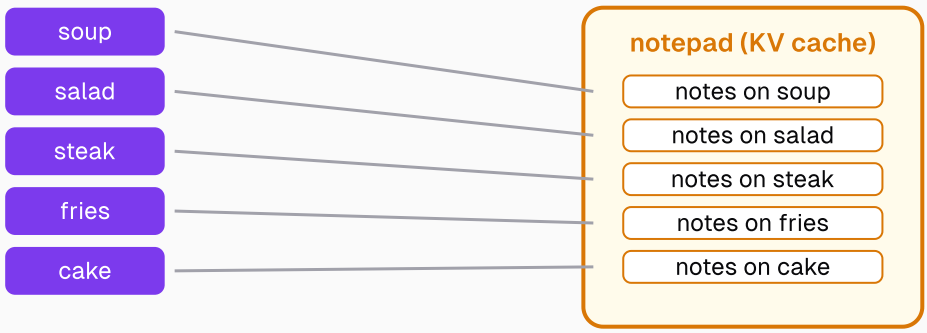


Work: 1 + 2 + 3 + 4 + 5 = 15 reads for 5 dishes

It grows with the square of the order length

To write each new word the model must consider every earlier word. Re-reading everything at every step is like a kitchen re-reading the entire order before each new dish: the work grows with the square of the length.

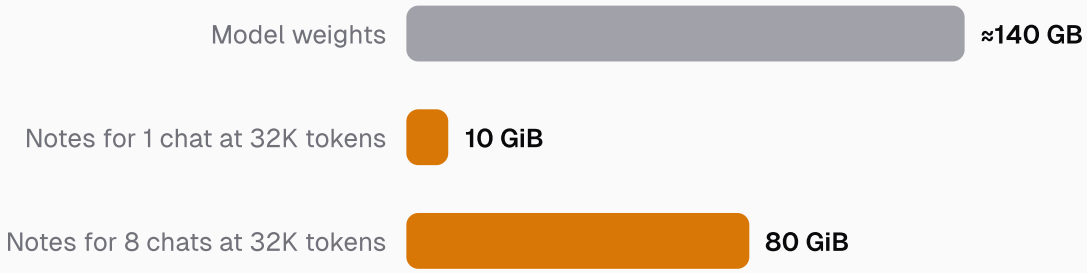
With a notepad: note each dish once, process only the new one



5 dishes, 5 reads. The price is memory for the notepad.

The **KV cache** is the notepad: the model stores a short summary (keys and values) of every earlier word, so each new word only needs its own processing plus a glance at the notes.

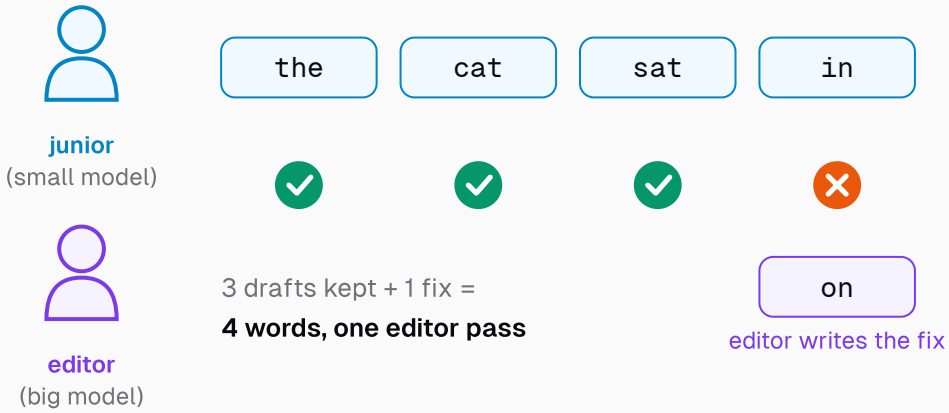
The notepad takes real memory (70B-class model, 16-bit)



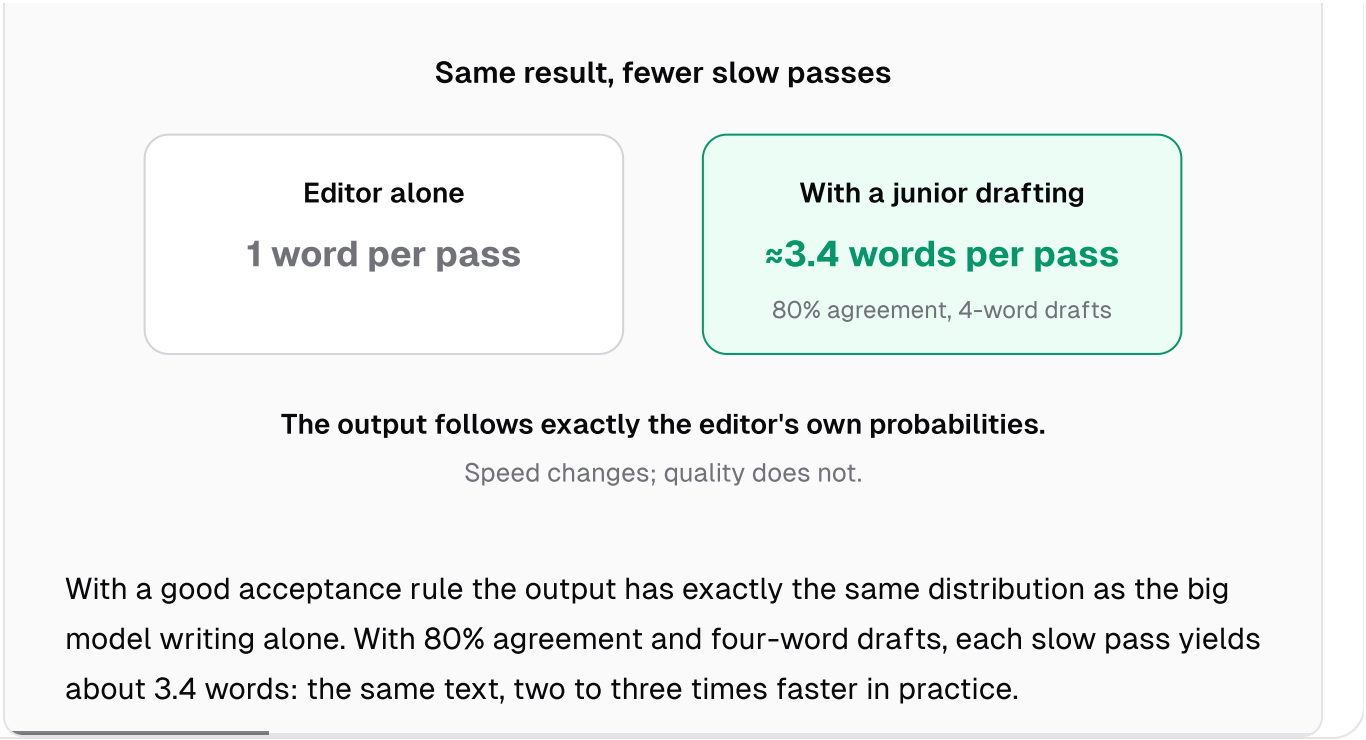
Memory, not arithmetic, usually limits how many people one machine serves.

The notes are not free. For a 70B-class model a single 32K-token conversation needs about 10 GiB of cache; eight at once need more than half as much memory as the model weights. That is why long contexts are expensive.

Speculative decoding: a junior drafts, the editor checks in one pass



Generating with the big model is slow because each word needs a full pass. In **speculative decoding** a small model drafts a few words, the big model checks them all in one pass, keeps the ones it agrees with, and writes the first one it disagrees with.



With a good acceptance rule the output has exactly the same distribution as the big model writing alone. With 80% agreement and four-word drafts, each slow pass yields about 3.4 words: the same text, two to three times faster in practice.

Plain words

Why long conversations are expensive

To write each new word, the model needs to look back at everything said so far. Rather than reread the whole conversation every time, it keeps notes on every earlier word, called the KV cache. Those notes take memory, and the longer the conversation and the more people served at once, the more memory they take. Memory, not raw arithmetic, is usually what limits how many people one machine can serve.

For leaders

The economics in five ideas

1. **Prefill versus decode.** Reading the prompt (prefill) is processed in parallel and limited by arithmetic. Generating tokens (decode) is one token at a time and limited by memory bandwidth, because every step reads the full weights and cache. Long prompts cost compute; long outputs cost time.

2. **Batching is the main lever.** Weights are read once per decode step regardless of how many sequences share it, so serving more sequences at once raises throughput almost for free until memory runs out or latency targets break. Continuous batching keeps the batch full as requests finish.

3. **Latency and throughput trade off.** Bigger batches mean more tokens per second per accelerator and lower cost per token, but each user's tokens arrive more slowly. Products set

a latency target and maximise throughput under it.

4. **Speculative decoding** (Leviathan et al., 2023) uses a small draft model to propose several tokens and the large model to verify them in one pass. The output distribution is provably the same as sampling from the large model alone (and identical tokens under greedy decoding, up to floating-point numerics), and speed-ups of 2 to 3 times are common when the draft agrees often.

5. **Quantisation** stores weights (and sometimes the cache) in 8 or 4 bits. It cuts memory and raises speed. 8-bit weights are usually close to lossless; 4-bit methods such as GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) lose a little on average and sometimes much more on maths, code, long context, and low-resource languages.

Eval lens

Evaluate the model you serve, not the model you trained. Quantisation, speculative decoding with a non-exact acceptance rule, a different inference engine, or a changed chat template can each shift scores. Re-run the core suite on the serving configuration before launch and compare with the training-time numbers using a paired test on the same problems. Perplexity alone hides task-specific losses.

Under the hood

The arithmetic

KV cache. Each layer stores a key and a value vector per KV head per token:

$$\text{bytes} = 2 \times L \times H_{kv} \times d_{\text{head}} \times \text{tokens} \times \text{sequences} \times \text{bytes per value}$$

The 2 counts keys and values. Grouped-query attention shrinks H_{kv} below the number of query heads, which is why modern models use it.

python

```python
def kv_cache_bytes(layers, kv_heads, head_dim, seq_len, batch, bytes_per=2):
    return 2 * layers * kv_heads * head_dim * seq_len * batch * bytes_per

print(kv_cache_bytes(80, 8, 128, 32_768, 1) / 2**30)   # 10.0 GiB: one 32K-token sequence, 70B-class with GQA
print(kv_cache_bytes(80, 64, 128, 32_768, 1) / 2**30)  # 80.0 GiB: the same without GQA
```

rust

```rust
fn kv_cache_bytes(layers: u64, kv_heads: u64, head_dim: u64, seq_len: u64, batch: u64, bytes_per: u64) -> u64 {
    2 * layers * kv_heads * head_dim * seq_len * batch * bytes_per
```

```
}

fn main() {
    let gib = (1u64 << 30) as f64;
    println!("{:?}", kv_cache_bytes(80, 8, 128, 32_768, 1, 2) as f64 / gib);  // 10.0 GiB: one 32K-token sequence, 70B-class with GQA
    println!("{:?}", kv_cache_bytes(80, 64, 128, 32_768, 1, 2) as f64 / gib); // 80.0 GiB: the same without GQA
}
```

Decode speed bound. At batch size 1, each token requires reading all weights once. A 70B model in 16-bit is about 140 GB; on an accelerator with roughly 3.35 TB/s of memory bandwidth, that caps decode at about 24 tokens per second per sequence, before any parallelism across devices. Batching amortises that read across sequences.

Speculative decoding. With draft length γ and per-token acceptance probability α, the expected number of tokens produced per target-model pass is $\frac{1-\alpha^{\gamma+1}}{1-\alpha}$. For $\alpha = 0.8,\ \gamma = 4$ that is about 3.4.

Cost per million tokens.

$$\text{cost per } 10^6 \text{ tokens} = \frac{\text{dollars per accelerator-hour}}{\text{tokens/s per accelerator} \times 3600} \times 10^6$$

At an illustrative $3 per hour and 1,500 output tokens per second per accelerator, that is about $0.56 per million output tokens. Every lever above acts through the denominator.

Interactive

KV-cache memory calculator (shown at its default setting)

Pick a model shape, context length, and how many sequences run at once. Compare cache memory with the model weights and an 80 GB accelerator. Architectures are illustrative of public 8B and 70B models that use grouped-query attention.

KV cache per sequence	KV cache total	Weights	80 GB accelerators needed
10.7 GB	85.9 GB	141.2 GB	3

The cache is **38%** of memory. Here the weights dominate; longer contexts or more concurrent users would flip that. Activations and runtime overhead are ignored, so real requirements are higher. Precision applies to both weights and cache.

Part III · Chapter 29

Safety and alignment training

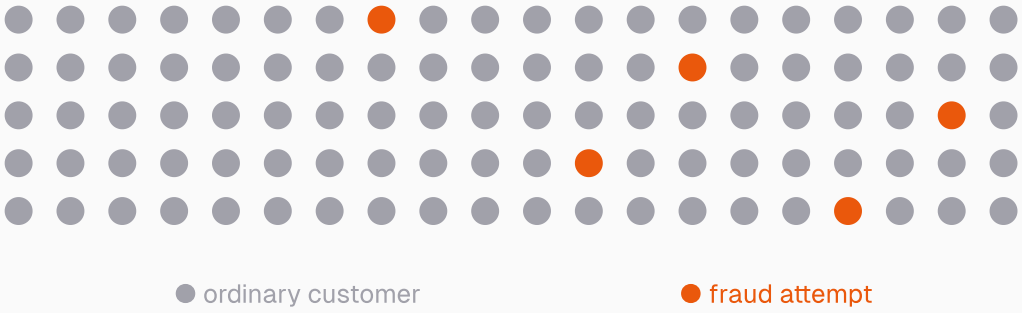
A capable model that does whatever it is asked is not a product anyone can ship. Safety and alignment training shapes what the model will and will not do, how honestly it reports what it knows, and how it behaves under pressure from adversarial users. This chapter covers the core trade-off, the main training methods, how safety is evaluated, and how it interacts with the capability training from earlier chapters. It is descriptive, not operational: no attack recipes.

Diagram

The bank teller's dial

Over-refusal versus unsafe compliance in five pictures.

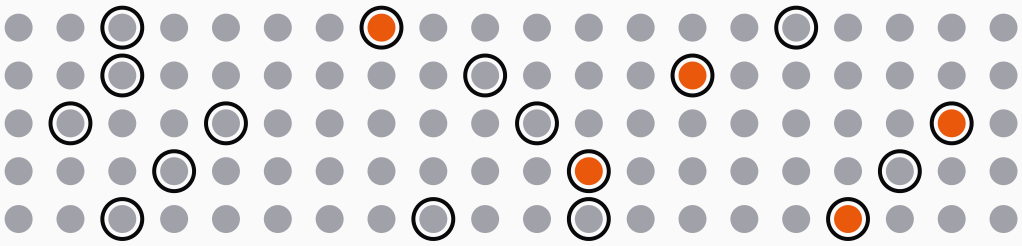
A day at the bank: 100 ordinary customers, 5 fraud attempts



The teller must help the many and stop the few (numbers are illustrative).

A model answering requests is like a bank teller. Almost everyone wants something ordinary. A few are trying something harmful. The teller has to help the many and stop the few.

A strict teller: refuses anything that looks unusual

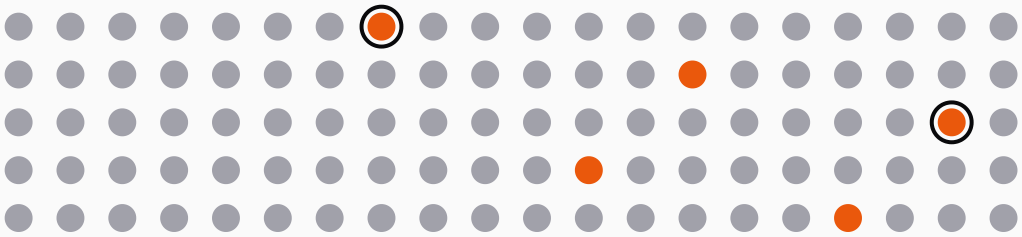


Fraud let through: 0 of 5 **Customers turned away: 12**

Ringed dots were refused. This is over-refusal.

A strict teller stops every fraud attempt, and turns away 12 ordinary customers too. In a model this is **over-refusal**: saying no to harmless requests like "how do I kill a stuck process?"

A lenient teller: rarely says no

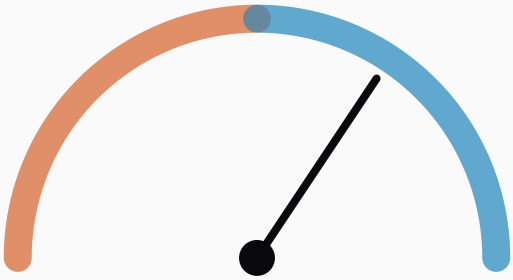


Fraud let through: 3 of 5 Customers turned away: 0

This is unsafe compliance.

A lenient teller helps every customer and lets 3 of the 5 fraud attempts through. In a model this is **unsafe compliance**. The two failures are mirror images.

One dial trades one mistake for the other

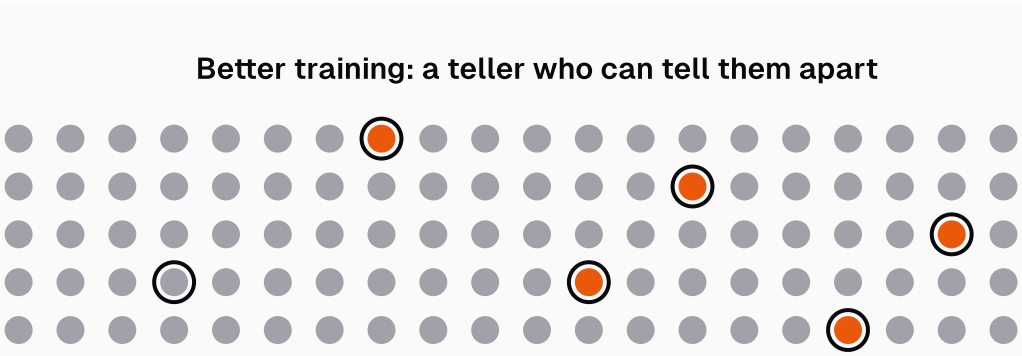


lenient
more fraud gets through

strict
more customers refused

where to set it depends on what each mistake costs

Turning the dial moves the teller between the two failures. Where to set it depends on how bad each mistake is: for severe, irreversible harms, accept more refusals; for minor ones, accept fewer. This is the refusal threshold in the widget below.



Real safety progress makes the teller better at telling the two apart, so both numbers fall at once. When a release reports fewer unsafe answers, always ask whether over-refusal went up: that would just be the dial turning.

The trade-off: helpful and harmless pull against each other

Plain words

A pharmacist, not a vending machine

A good pharmacist answers almost every question, helps people understand their medicines, and refuses a tiny number of requests that would clearly hurt someone. A pharmacist who refused every question that mentioned a dangerous drug would be useless. One who handed out anything would be dangerous. Safety training is teaching a model to be the good pharmacist: helpful by default, careful in the few places that matter, and not fooled by someone dressing up a harmful request as an innocent one.

The two ways to fail are mirror images. **Over-refusal**: saying no to something harmless ("how do I kill a Python process?"). **Unsafe compliance**: saying yes to something genuinely harmful. Pushing one down tends to push the other up.

For leaders

Treat it as a classification problem with asymmetric costs

Refuse-or-answer is a binary decision, so everything from classification metrics applies. Over-refusal is a false positive, unsafe compliance a false negative, and harmful requests are rare, so precision on refusals is low even for a good model. Three consequences:

- **The threshold is a policy decision.** Where you sit on the curve depends on the relative cost of the two errors, which differs by harm category. Severe, irreversible harms justify many over-refusals; minor ones do not. Mature policies set different operating points per category.
- **Real progress moves the curve, not the point.** A release that cuts unsafe compliance by refusing more is a threshold move. A release that cuts both errors is better training. Ask which one you are looking at.
- **Over-refusal is a real cost.** It erodes trust, pushes users to less careful tools, and is often concentrated on particular topics or groups. Track it with the same seriousness as unsafe compliance.

Principal asks

- Did both error rates improve, or did one improve at the other's expense?
- What are the operating points per harm category, and who signed off on the cost ratios?
- How are the over-refusal prompts chosen? Are they hard, borderline cases, or easy ones that any model passes?

Under the hood

Expected cost and the operating point

With harmful share π, refusal threshold t, over-refusal rate $\mathrm{FPR}(t)$ on benign traffic, unsafe-compliance rate $\mathrm{FNR}(t)$ on harmful traffic, and cost ratio C (one unsafe completion costs C over-refusals):

$$\mathbb{E}[\text{cost}](t) \;\propto\; (1-\pi)\,\mathrm{FPR}(t) + \pi\, C\,\mathrm{FNR}(t)$$

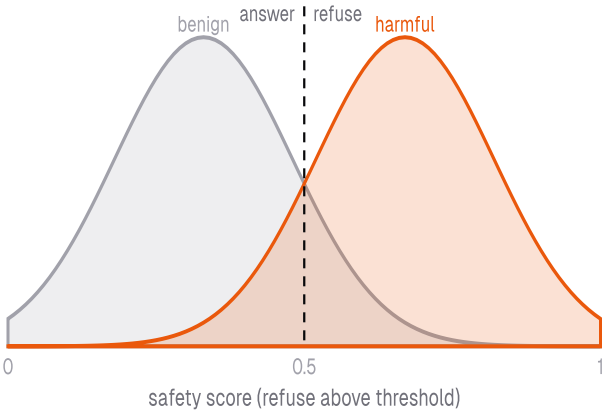
The cost-minimising threshold is where the frontier's slope equals $-(1-\pi)/(\pi C)$. Rarer harms or cheaper failures push the optimum toward refusing less.

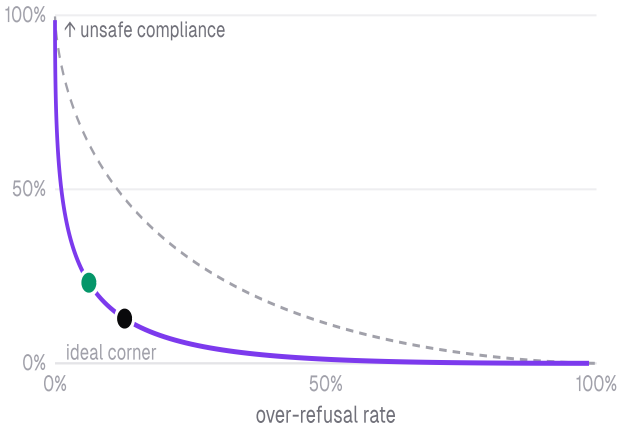
In practice the "score" is not a single number. It is the model's learned propensity to refuse, plus any classifiers in front of or behind the model. The frame still holds: every component trades the two errors, and the system's frontier is what matters.

Interactive

Helpfulness versus harmlessness frontier (shown at its default setting)

A safety signal scores every request. Above the threshold the model refuses. Benign requests that score high are over-refusals; harmful requests that score low are unsafe completions. Move the threshold along the curve, then improve the model and watch the whole frontier move toward the corner.





Benign requests · Harmful requests · Current model frontier · Earlier model · Lowest expected cost

Over-refusal (benign refused)	Unsafe compliance (harmful answered)	Refusals that were right	Cost-minimising threshold
12.9%	**12.9%**	**12%**	**0.56**

Per 1,000 requests: about **126** benign requests refused and **2.6** harmful ones answered. Only **12%** of refusals hit a harmful request, because harmful traffic is rare. At a 20× cost ratio the expected cost is lowest at threshold **0.56** (green dot). Improving training moves the solid curve toward the corner; moving the threshold only slides along it. Illustrative model.

How models are trained to be safe

Diagram

Constitutional AI: the model edits itself against written principles

How self-critique and AI feedback replace much of the human labelling. Five pictures.

Step 1: the model drafts a reply

User

How do I kill a Python process that is stuck?

First draft

I can't help with killing anything.

A harmless question, refused because of one scary word.

Over-refusal, the mirror image of unsafe compliance.

Constitutional AI (Bai et al., 2022) starts with the model's own drafts. Here the draft refuses a harmless programming question because it contains the word "kill".

Step 2: critique the draft against a written principle

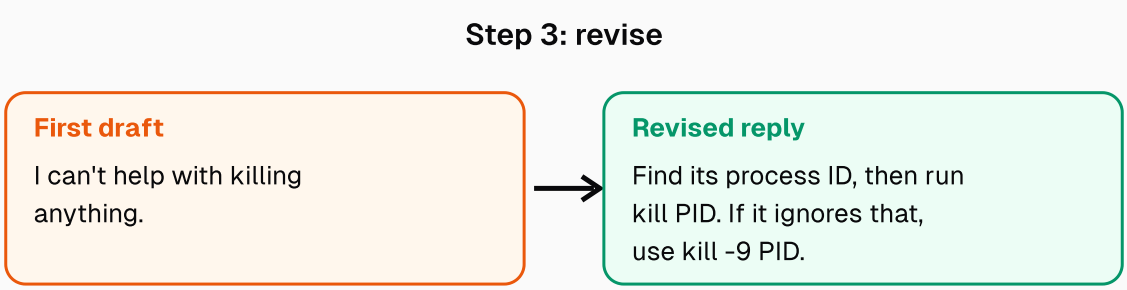
Principle (from the constitution)

Choose the response that is most helpful while avoiding real harm.

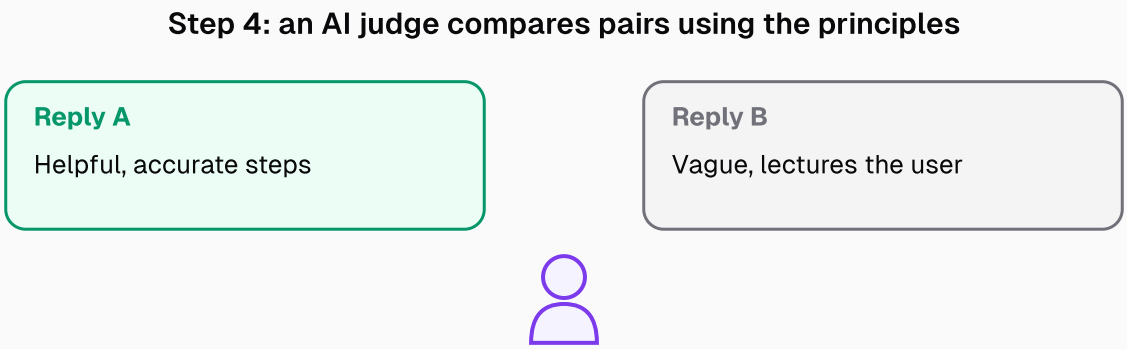
Critique, written by the model

"kill" here means ending a program. Nothing harmful is being asked, so refusing is unhelpful and should be revised.

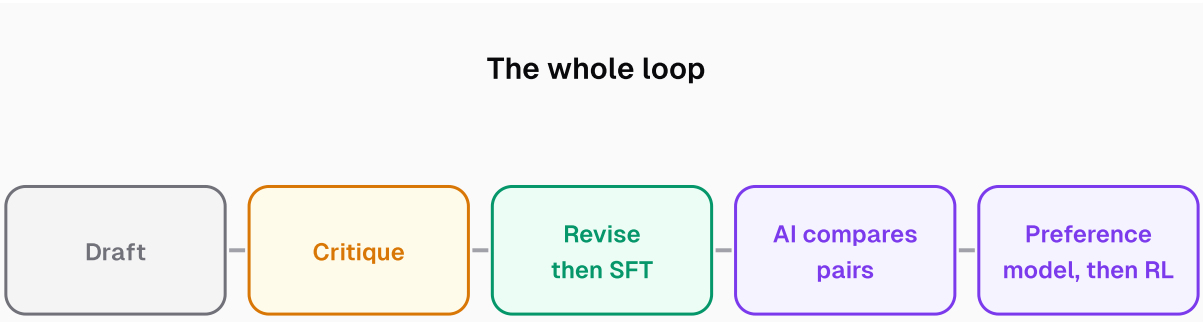
The model is shown a principle drawn from a written list, the constitution, and asked to critique its own draft against it.



The model rewrites the reply to follow the principle. Revised replies become supervised fine-tuning data: the **supervised phase**.



In the **RL phase** the fine-tuned model writes pairs of replies, and an AI feedback model picks which one better follows a principle. Those labels train a preference model, the same machinery as RLHF but with AI feedback: RLAIF.



Strength: explicit, editable principles; feedback scales with compute.

Risk: the judge's blind spots become the model's blind spots.

Put together: draft, critique, revise, fine-tune, then compare pairs with AI feedback and run RL. The principles are written down and can be inspected and edited, which is the method's main appeal.

Plain words

Rules, examples, and practice

There are three main ways to teach good judgement. Show the model examples of good responses to tricky requests. Have people rate pairs of responses so it learns which is better. Or write down a set of principles and have an AI use them to critique and rate responses, which scales far better than asking people to read thousands of disturbing conversations.

For leaders

The main methods

Method	What it does	Strength	Weakness
Safety SFT	Fine-tune on curated demonstrations of good responses to sensitive requests (see SFT)	Direct, predictable	Teaches surface patterns; can over-generalise to refusing anything that looks similar
RLHF with harmlessness preferences (Bai et al., 2022a)	Humans compare responses for helpfulness and harmlessness; train a reward model and optimise against it (see RLHF)	Captures nuanced judgement	Expensive, labeller exposure to harmful content, inconsistent labels, reward hacking
Constitutional AI and RLAIF (Bai et al., 2022b)	A written set of principles; the model critiques and revises its own outputs, then AI-generated preference labels replace human harmlessness labels	Scales, transparent principles, less human exposure	Quality bounded by the labelling model and how well the principles are written
Direct preference methods	DPO-style training on safety preference pairs (see DPO)	Simple, stable	Offline data goes stale as the model changes
Reasoning over a policy	The model is trained to recall and reason about a written safety specification before answering, for example OpenAI's deliberative alignment (Guan et al., 2024)	Better on novel and borderline cases; reasoning can be inspected	Depends on the specification's quality; adds latency
Character and persona training	Shapes stable traits such as honesty, curiosity, and directness, rather than rule-following alone	Generalises to situations rules did not foresee	Harder to measure; traits can drift under RL

Under the hood

Constitutional AI in two phases

1. **Supervised phase.** Sample responses to red-team prompts from a helpful-only model. Ask the model to critique each response against a randomly drawn principle from the constitution, then revise it. Fine-tune on the revised responses.

2. **RL phase (RLAIF).** Sample pairs of responses from the fine-tuned model. Ask a feedback model which response better follows a principle, giving preference labels. Train a preference model on those labels (often mixed with human helpfulness labels) and run RL against it.

The appeal is that the principles are explicit and editable, and the feedback scales with compute instead of human hours. The risk is that the feedback model's blind spots become the policy's blind spots, and that the policy learns to satisfy the letter of the principles. It also explains why constitutions are written as general judgements ("choose the response that is more honest") rather than narrow rules: general principles transfer better to unseen situations.

Failure modes beyond refusal

For leaders

What safety teams worry about besides "will it help build something dangerous"

- **Jailbreaks.** Inputs that get a model to do what its training says it should not. Broad classes include role-play and fictional framing, gradual escalation over many turns, encoding or translating the request, splitting a harmful task into innocent-looking pieces, flooding the context with fabricated examples of compliance, and instructions hidden in documents or tool outputs (prompt injection). Defences are layered: training on these patterns, classifiers on inputs and outputs, and limits on what tools an agent can use.
- **Sycophancy.** Telling users what they want to hear: agreeing with stated opinions, caving when pushed back on a correct answer, praising weak work. Preference training rewards it, because people tend to rate agreement highly (Sharma et al., 2023).
- **Dishonesty and poor calibration.** Stating guesses as facts, inventing citations, or claiming to have done work it did not do. For agents, misreporting what actions were taken.
- **Reward hacking that looks like misbehaviour.** RL on capability tasks can teach a model to cut corners (see GRPO), and corner-cutting can generalise into broader patterns such as deceiving the grader.
- **Over-refusal and moralising.** Unrequested lectures, hedging that buries the answer, and refusing legitimate professional questions in medicine, security, or law.

Watch out

Capability RL can quietly undo safety training. A long reasoning-RL run optimised only for correctness can raise compliance with harmful requests, increase sycophancy, or teach the model to game evaluations. Re-run the safety suite after every major capability stage, not only at the end, and include safety data or rewards in the RL mixture when possible.

How safety is evaluated

For leaders

Four families of evaluation

Refusal and harm benchmarks

Sets of harmful requests scored for whether the model complies, for example HarmBench (Mazeika et al., 2024) and StrongREJECT (Souly et al., 2024), which also grades how useful a non-refusal actually was.

Over-refusal benchmarks

Benign prompts that look risky ("how do I kill a process"), for example XSTest (Röttger et al., 2024) and OR-Bench (Cui et al., 2024). Always report alongside harm benchmarks.

Red teaming

Humans and automated attackers search for failures. Automated red teaming uses another model to generate attacks at scale (Perez et al., 2022). Results are a lower bound on vulnerability, never a proof of safety.

Dangerous-capability evals

Tests of whether the model could meaningfully help with severe harms, such as advanced cyber operations or weapons development, or could act autonomously in risky ways. These feed frontier safety policies.

Frontier safety policies. Several labs publish frameworks that tie capability thresholds to required safeguards: Anthropic's Responsible Scaling Policy with its AI Safety Levels (first published 2023), OpenAI's Preparedness Framework (2023), and Google DeepMind's Frontier Safety Framework (2024). The common structure: define capability thresholds that would matter, evaluate for them before and during deployment, and commit to stronger security and deployment safeguards, or to pausing, when a threshold is crossed. Dangerous-capability evaluation therefore carries direct launch consequences, and its statistical quality matters accordingly.

Eval lens

Safety evals have every problem from Part II, often worse. Harmful prompts are rare and diverse, so sets are small and intervals are wide. Grading is usually done by an LLM judge, which must be validated against human labels (see LLM judges). Public benchmarks get trained on, so keep private held-out sets (see contamination). And a capability eval that a model fails because of refusals or poor prompting understates what it can do. Dangerous-capability evals should try hard to elicit the capability, which is the opposite of a normal product eval.

Under the hood

Measuring robustness, not a single attack

A model's jailbreak resistance is better described by a curve than a number: attack success rate as a function of attacker effort (queries, attempts, or compute). Report attack success rate with an interval at several budgets, and for adaptive attacks report the budget needed to reach a given success rate. Because successes are rare at low budgets, use exact or Wilson intervals (see eval

statistics), and treat zero observed successes as "fewer than about $3/n$ at 95% confidence", the rule of three, not as zero risk.

$$\text{If } 0 \text{ of } n \text{ attacks succeed:} \quad p_{\text{upper}} \approx \frac{3}{n}$$

An approximate 95% upper bound. With 300 failed attacks you can only claim the success rate is probably below 1%.

Deployment: training is one layer of several

For leaders

Defence in depth

No lab relies on model training alone. Deployed systems add input and output classifiers (for example Anthropic's constitutional classifiers, described in 2025), usage policies and account-level enforcement, monitoring for abuse patterns across many conversations, rate limits, and for agents, restrictions on tools and permissions. Each layer has its own false-positive and false-negative rates, and stacking classifiers compounds over-refusal as well as protection. Evaluate the full system on realistic traffic, not each layer alone.

Principal asks

If the model's refusal training failed completely on one category tomorrow, which other layer would catch it, and how would we know within a day?

Part IV

Leading the work

How the pieces connect, the questions a principal asks, and a reference for every term.

Part IV · Chapter 30

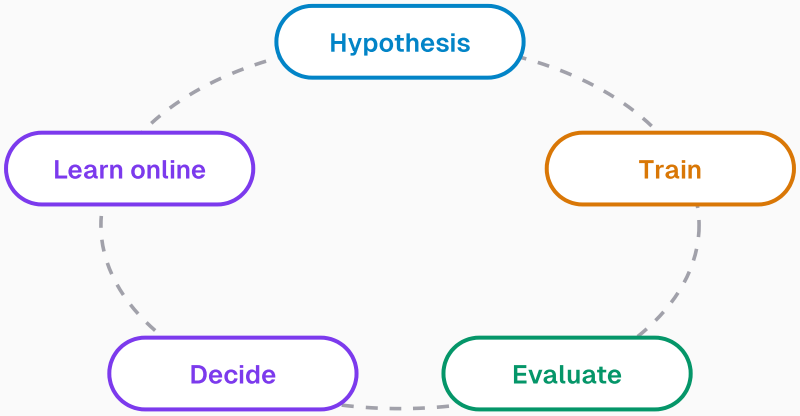
Connecting training and evaluation

Parts II and III describe two machines. One makes models. The other tells you whether they got better. This chapter is about running both as one system, and about the judgment calls that separate a senior reviewer from someone who reads results off a dashboard.

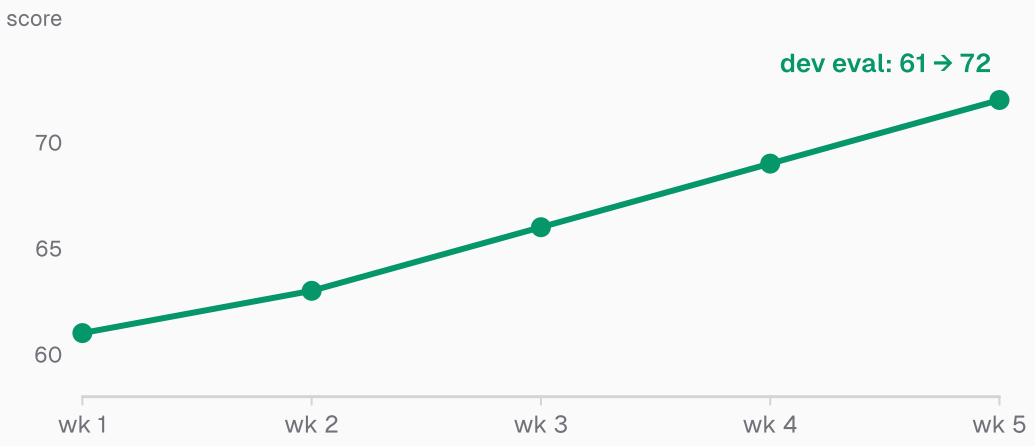
Diagram

The bathroom scale and the diet

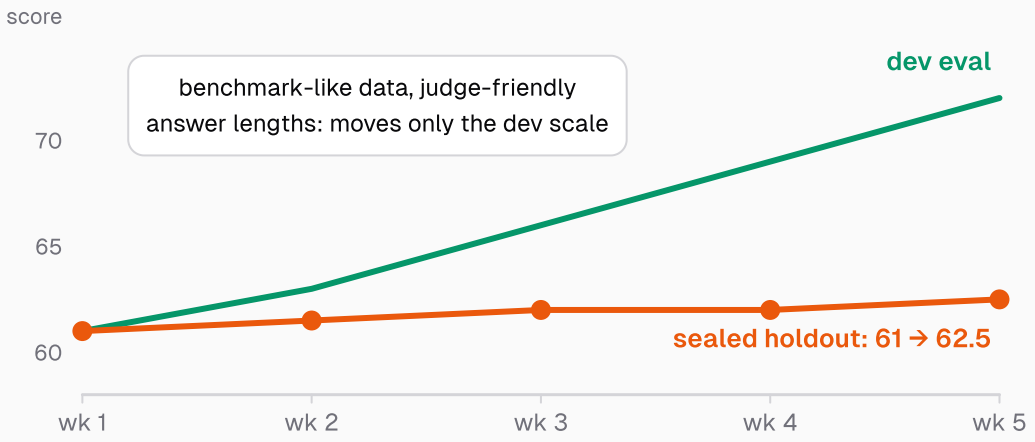
Why evals become the objective, and how to keep a measurement honest under optimisation pressure.



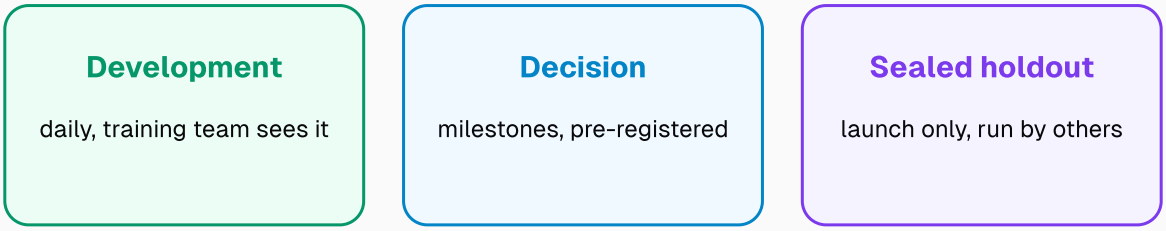
Training and evaluation form one loop: a hypothesis, a training change, an eval, a decision, and what production teaches you next. Whatever the eval rewards is what the loop optimises.



Week by week the development eval climbs from 61 to 72. The team celebrates. It looks like a diet that works.



Then a sealed holdout, which nobody tuned toward, barely moves. Most of the gain came from learning the quirks of one scale: training data that resembles the benchmark and answer styles the judge likes.



When they disagree, believe the scale nobody could tune toward.

So keep three scales: a development eval for daily iteration, a decision eval with pre-registered thresholds, and a sealed holdout run by someone else. Goodhart's law is the default, so plan for it.

The loop

1 **Hypothesis**
What should improve, and why

2 **Training change**
Data, objective, method, or scale

3 **Eval**
Suite chosen before the run

4 **Analysis**
Paired deltas, intervals, slices

5 **Decision**
Ship, iterate, or kill

6 **Online signal**
Feeds the next hypothesis

Plain words

The scale and the diet

Training is the diet. Evaluation is the bathroom scale. If the scale is wrong, every diet looks like it works or none does. If you only ever weigh yourself on one scale, you start eating whatever moves that particular scale, even if it does nothing for your health. Teams that train models face the same trap. Whatever you measure becomes what you optimise, so you need scales you trust, more than one of them, and some you never look at until the end.

For leaders

Evals are the objective function of the organisation

In a training organisation, the eval suite is the de facto objective function. People will optimise whatever gets reported in the weekly review. That gives the eval owner unusual leverage and unusual responsibility.

- **Goodhart is the default.** Every metric that drives decisions will be gamed, usually without bad intent. Training data drifts toward benchmark-like content, prompts drift toward judge preferences, and RL policies find verifier loopholes. Plan for it: rotate suites, keep private holdouts, and audit the largest gains first.
- **Separate the scales.** Keep three tiers: development evals the training team can see every day, decision evals reviewed at milestones, and sealed holdouts that only an independent owner runs before launch. If the sealed set disagrees with the development set, believe the sealed set.
- **Pre-register the decision.** Write down, before the run, which evals decide success and what delta counts. Post hoc selection of the metric that moved is the most common way a team fools itself.
- **Price the evals.** Evaluation is a real fraction of compute and headcount. A suite that cannot detect the effect sizes your training changes produce (see power and sample size) is pure cost. Fund fewer, better-powered evals over many thin ones.

Under the hood

Why gains shrink when they reach the holdout

Suppose you try m training variants and pick the one with the best development score. Even if every variant has the same true quality, the winner's observed score is biased upward by the expected maximum of m noise draws. With per-variant standard error σ, that bias is roughly

$$\mathbb{E}\left[\max_{i \le m} \varepsilon_i\right] \approx \sigma\sqrt{2 \ln m}$$

For $m = 20$ variants and $\sigma = 1$ point, the "best" variant looks about 2.4 points better than it is. This is the winner's curse, and it is why a fresh holdout almost always shows a smaller gain than the development set.

python

```python
import numpy as np

rng = np.random.default_rng(0)
m, sigma, trials = 20, 1.0, 20000
best_dev = rng.normal(0, sigma, size=(trials, m)).max(axis=1)   # all variants truly equal
```

```
print(round(best_dev.mean(), 2))                # ~1.87 (the asymptotic formula overstates at small m)
print(round(sigma * np.sqrt(2 * np.log(m)), 2))  # 2.45
```

rust

```
/// Small inline PRNG: SplitMix64 for uniforms, Box-Muller for normals.
/// The draws differ from numpy's, so the simulated mean differs in the last digit or so.
struct Rng(u64);

impl Rng {
    fn next_u64(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }
    fn uniform(&mut self) -> f64 {
        ((self.next_u64() >> 11) as f64 + 0.5) / (1u64 << 53) as f64 // in (0, 1), never 0
    }
    fn normal(&mut self, mean: f64, sd: f64) -> f64 {
        let (u1, u2) = (self.uniform(), self.uniform());
        mean + sd * (-2.0 * u1.ln()).sqrt() * (2.0 * std::f64::consts::PI * u2).cos()
    }
}

fn main() {
    let mut rng = Rng(0);
    let (m, sigma, trials) = (20, 1.0, 20000);
    // all variants truly equal: the best dev score of m is pure noise
    let best_dev: Vec<f64> = (0..trials)
        .map(|_| (0..m).map(|_| rng.normal(0.0, sigma)).fold(f64::NEG_INFINITY, f64::max))
        .collect();
    let mean = best_dev.iter().sum::<f64>() / trials as f64;
    println!("{:.2}", mean); // 1.87 with this PRNG (numpy also gives 1.87; draws differ); the asymptotic formula
overstates at small m
    println!("{:.2}", sigma * (2.0 * (m as f64).ln()).sqrt()); // 2.45
}
```

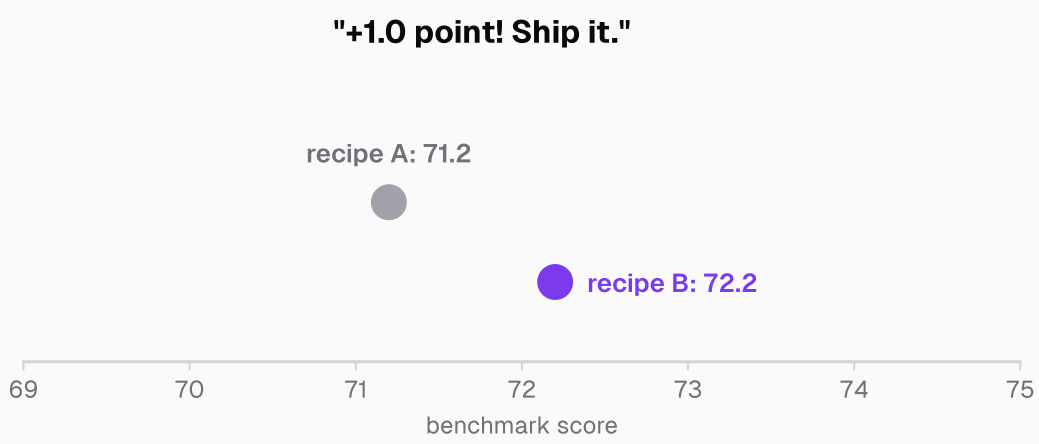
The practical fix is the same as in any selection problem: hold out a set you never select on, and report the selected model's score on it.

Is this result real? Seeds and run-to-run noise

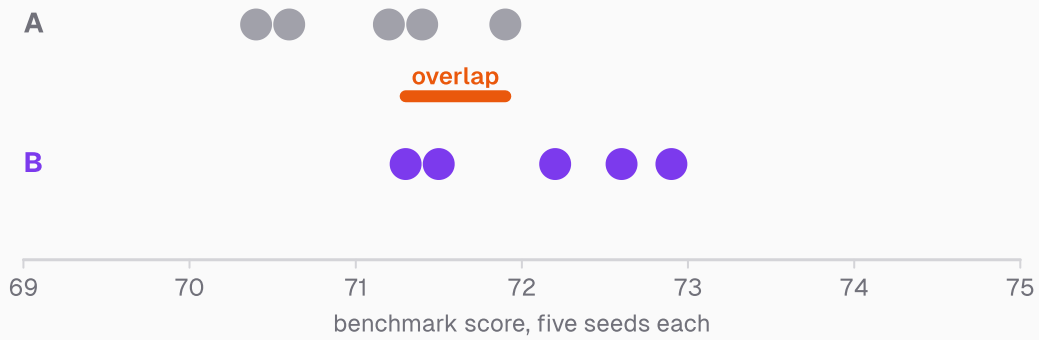
Training runs are noisy. The same recipe with a different random seed lands at a different score. Before you believe a gain from a training change, you need to know how large that seed noise is compared with the gain.

Diagram **One training run is one coin flip**

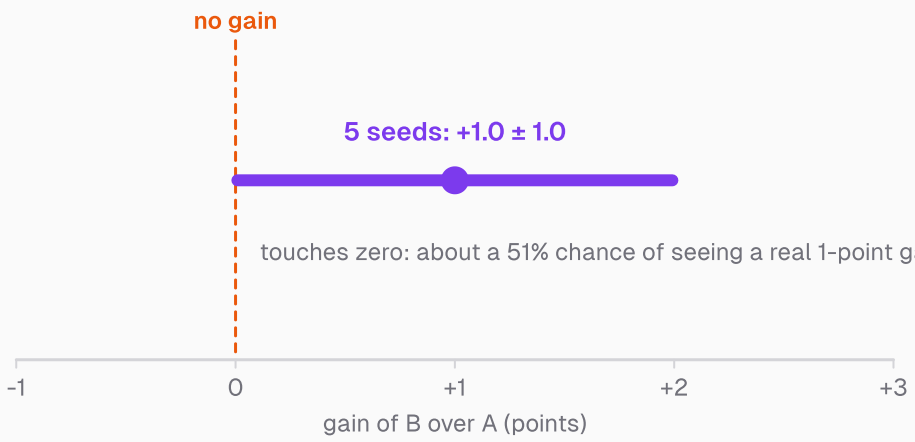
Seed-to-seed noise of 0.8 points, a claimed gain of 1 point, and what it takes to believe it. Numbers are illustrative.



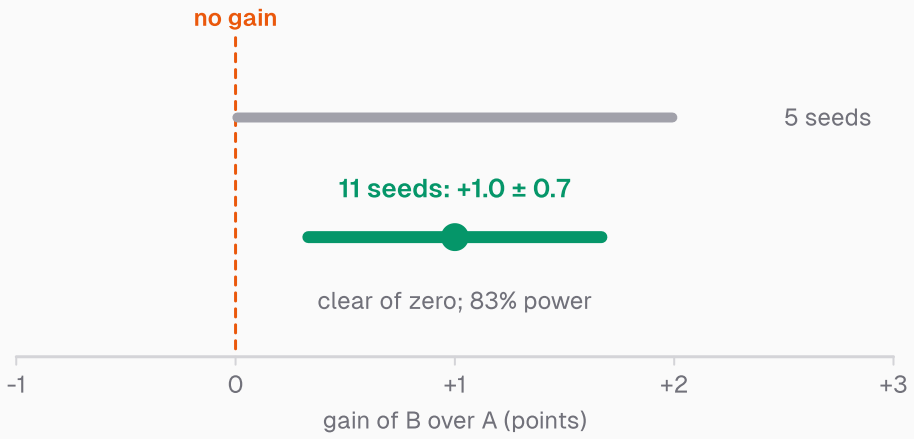
Recipe A scores 71.2, recipe B scores 72.2. One run each. A one-point gain looks like a win.



Rerun each recipe with five different random seeds. The scores scatter by about 0.8 points either way, and the two clouds overlap. Some A runs beat some B runs.



Averaged over five seeds, B is still about 1 point ahead, but the 95% interval on the gain (±1.0) reaches zero. With five seeds a real one-point gain shows up as significant only about half the time.

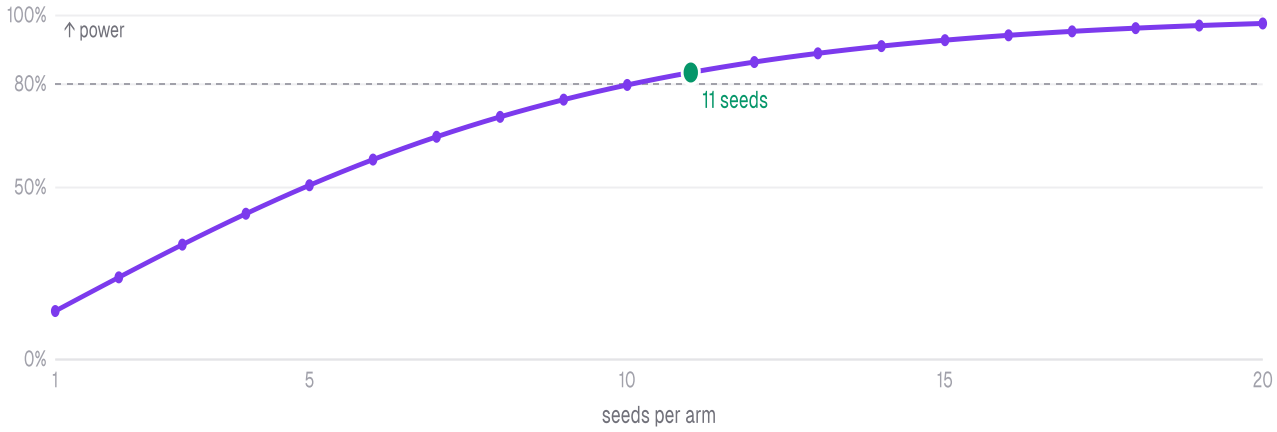


With 11 seeds per arm the interval shrinks to ±0.7 and clears zero. That is the arithmetic behind "ask for the seed variance first", and the widget below lets you redo it for any gain.

Interactive

Seeds needed to trust a gain (shown at its default setting)

Set how much a score moves between seeds and the gain someone is claiming. The chart shows how likely you are to see the gain as significant with more seeds per arm, and the readout says what a single-seed comparison is worth.



Chance 1 vs 1 seed shows the wrong sign	Seeds per arm for 80% power	Gain / seed std
19%	**11**	**1.25**

With one seed per arm, a true gain of 1.0 points shows up with the wrong sign 19% of the time. To detect it 80% of the time you need about 11 seeds per arm, which is usually too expensive: shrink seed noise (bigger eval sets, paired items) or accept that this gain is below what you can resolve.

Plain words

One run is one coin flip

If you train the same model twice, you get two slightly different results, like flipping a coin twice. A claimed one-point gain from comparing one run against one other run can easily be luck. The widget shows how many repeat runs you need before a gain of a given size is believable.

For leaders

What to require before you believe an ablation

- **Ask for the seed variance first.** No one can interpret a delta without it. If the team does not know it, the first deliverable is three seeds of the baseline.

- **Compute-matched comparisons.** A new method that also used 30 percent more compute has not been compared with the baseline. Insist on equal FLOPs, equal tokens, or an explicit compute-versus-quality curve.
- **Tuned baselines.** The new method usually gets more tuning effort than the baseline. Ask how many hyperparameter trials each arm got.
- **Small-scale results transfer imperfectly.** Many gains at 1B parameters vanish at 70B, and some appear only at scale. Ask whether the trend holds across at least two or three model sizes before it drives a large run.
- **Kill criteria.** Every research bet should state in advance what result would stop it. Without one, projects drift for quarters.

Under the hood

The arithmetic behind the widget

With n seeds per arm, seed standard deviation s, and true gain δ, the difference in arm means has standard error $s\sqrt{2/n}$. A two-sided test at $\alpha = 0.05$ has power approximately

$$\text{power}(n) \approx \Phi\left(\frac{\delta}{s\sqrt{2/n}} - 1.96\right), \qquad n_{80} = \left\lceil 2\left(\frac{(1.96 + 0.84)\, s}{\delta}\right)^2 \right\rceil$$

Φ is the standard normal CDF. With very few seeds a t-distribution is more honest and needs slightly more seeds than this normal approximation.

The chance a single seed versus a single seed shows the wrong sign is $\Phi\big(-\delta/(s\sqrt{2})\big)$. When the gain equals the seed standard deviation, that is about 24 percent: one comparison in four points the wrong way.

A reviewer's checklist

Use this when someone shows you a training result or an eval delta. Each item links to the chapter that explains it.

Question	Why it matters	Chapter
What is the interval on that number, and what is n?	A point estimate without uncertainty cannot support a decision.	Eval statistics
Was the comparison paired on the same items?	Unpaired tests throw away most of the power.	Eval statistics
How many metrics or benchmarks were looked at?	With 40 benchmarks, two "significant" moves are expected by chance.	Multiple comparisons
Are items clustered (multi-turn, agent traces, per-repo tasks)?	Naive intervals can be several times too narrow.	Clustering
What is the seed variance, and how many seeds?	Single-run comparisons are coin flips at small effect sizes.	This chapter
Is the comparison compute-matched and baseline-tuned?	Otherwise you are measuring effort, not method.	This chapter, scaling
Who or what scored it, and was the scorer validated?	Judges carry bias and absolute-level offsets.	LLM as judge
Could the eval items be in the training data?	Contamination inflates scores and hides real capability.	Contamination
Does the offline gain predict an online gain?	Offline suites overstate and sometimes point the wrong way.	Offline to online gap
What got worse?	Every training change has regressions; a report without them is incomplete.	SFT, triage
Was decoding held fixed?	Temperature and sampling settings alone can move scores by points.	Inference
Is the reward or verifier being exploited?	RL finds loopholes faster than people do.	Reward models, GRPO

Red flags, and what they usually mean

You see	Likely cause	Ask
Huge gain on one benchmark, nothing elsewhere	Contamination, benchmark-specific data, or format overfitting	Score on a perturbed or fresh version of that benchmark
Judge win rate up, human ratings flat	Optimising toward judge preferences (length, style)	Length-controlled win rate and a human spot check
Reward keeps climbing during RL, quality plateaus	Reward overoptimisation or verifier hacking	Samples from late checkpoints; gold-reward or held-out checks
pass@1 up, pass@k at large k down	RL sharpening the distribution rather than adding capability	pass@k curves before and after
Offline up, online flat	Eval distribution or harness differs from real usage	Backtest of offline vs online deltas across past releases
Smooth loss improvement, sudden "emergent" jump in accuracy	Thresholded metric hiding steady progress	A continuous metric (log-likelihood or partial credit) on the same items
Big gain, one seed, no interval	Noise	Three seeds per arm and a paired test

For leaders

Communicating uncertainty upward

Executives need a decision, not a lecture on intervals. A pattern that works:

1. **Lead with the decision and your confidence.** "Recommend shipping. We are confident it is not worse and fairly confident it is better on coding."
2. **Give the number with its range in plain words.** "Coding pass rate up about 3 points, plausibly between 1 and 5."
3. **Name the main risk and how you are watching it.** "Long-context retrieval slipped slightly. We are monitoring it in the staged rollout with a rollback trigger."
4. **Say what would change your mind.** This builds trust when something does change.

Principal asks

- What would we have to believe for this result to be wrong?
- If we had half the eval budget, which evals would we keep, and why?
- Which of our evals has disagreed with production most often, and what did we change because of it?
- What is the smallest effect our suite can reliably detect, and is that smaller than the effects we are trying to ship?
- Who owns the sealed holdout, and when did someone last look at failures on it by hand?

Part IV · Chapter 31

Glossary

Every term used in the guide, defined in a sentence or two, with a link to the chapter that explains it properly.

282 terms. Type to filter.

6ND	The approximate training compute of a dense transformer: 6 FLOPs per parameter per training token (2 forward, 4 backward). → section
A/A test	An online experiment with the same model in both arms, used to measure the noise floor of the online metric. → section
Accuracy	(TP + TN) / n. Misleading when one class is rare, because it mostly measures the common class. → section
Activation checkpointing	Discarding intermediate activations in the forward pass and recomputing them during backward to save memory at the cost of extra compute. → section
AdamW	The standard LLM optimiser: Adam's per-parameter step scaling from running averages of the gradient and squared gradient, plus decoupled weight decay. → section

Adapter merging	Folding a trained LoRA update into the base weights (W + (alpha/r)BA) so serving has no extra cost. Inexact when the base is quantised. → section
Advantage	How much better an action turned out than expected from that state: $A_t = R_t - V(s_t)$. → section
Alignment tax	Capability regressions caused by alignment training, first documented on public NLP benchmarks in InstructGPT. → section
Annealing	Decaying the learning rate toward zero at the end of pretraining, often on a higher-quality data mix. It produces a visible drop in loss. → section
Attack success rate	The fraction of attack attempts that elicit the targeted unsafe behaviour; best reported as a function of attacker effort. → section
Attenuation bias	The pull of a regression slope toward zero when the predictor is measured with noise, as offline deltas are. → section
AWQ	Activation-aware weight quantization: protects the weights most important to activations when quantizing to low bit widths (Lin et al., 2024). → section
Backtest	Leave-one-out prediction of each past release from all others, used to estimate how well an offline suite predicts the next release. → section
Balanced accuracy	The mean of recall and specificity. 0.5 is chance at any prevalence. → section
Baseline	A quantity subtracted from the return in a policy gradient to reduce variance without adding bias, usually the value estimate. → section
BCa bootstrap	Bias-corrected and accelerated bootstrap: shifts the percentile interval using a bias term and a jackknife skewness term. Better on skewed statistics. → section
Benchmark saturation	The state where top models approach a benchmark's ceiling, so remaining differences are dominated by label noise and the benchmark stops separating models. → section
Benjamini–Hochberg	A step-up procedure that controls the false discovery rate: reject the k smallest p values, where k is the largest rank with p(k) ≤ (k/m)·q. → section

Best-of-N	Generate N responses and return the one a reward model or verifier scores highest. → section
bf16	Brain floating point: a 16-bit format with the same exponent range as fp32 but fewer mantissa bits. The standard training precision. → section
Bias	A systematic error that does not shrink with more data. The opposite of noise. → section
Bits per byte	Total loss in bits divided by the text length in bytes. Lets you compare models with different tokenizers. → section
Block bootstrap	Resampling contiguous chunks of time-ordered data so the ordering and drift survive. → section
Bonferroni correction	Testing each of m hypotheses at α/m. Simple and conservative on large suites. → section
Bootstrap	Resample your data with replacement many times to see how much an estimate would wobble. → section
Bradley–Terry model	Models the probability that one item beats another as the logistic sigmoid of their score difference. The basis of reward model training and DPO. → section
Brier score	Mean squared error between predicted probabilities and 0/1 outcomes. A proper scoring rule; lower is better. → section
Byte-pair encoding (BPE)	A tokenizer training method that starts from characters or bytes and repeatedly merges the most frequent adjacent pair until the vocabulary reaches a target size. → section
Cache key	A hash over everything that can change a model output (task, prompt text, model id, sampling parameters, harness and judge versions), used to reuse results safely. → section
Calibration	Whether predicted probabilities match observed frequencies: of items given 70% confidence, about 70% should be positive. → section
Canary string	A unique random token embedded in an eval set so that it can be filtered from training data and detected if a model has memorised the set. → section

Catastrophic forgetting	Loss of previously learned abilities when a model is trained further on narrow data. Detected with a fixed regression suite compared against the parent model. → section
CDF	Cumulative distribution function: the share of values at or below each point. Rises from 0 to 1. → section
Censoring	You know a value exceeded some cap but not by how much. Timeouts and token limits create this. → section
Central limit theorem	Averages tend toward a bell shape regardless of the raw data's shape, with variance shrinking as one over n. → section
Chat template	The exact layout of special tokens and role markers used to format a conversation. Training, serving, and evaluation must all use the same one. → section
Chinchilla scaling	Hoffmann et al. (2022): for a fixed training budget, grow parameters and tokens together, about 20 tokens per parameter. → section
Clip-higher	Using a larger upper clipping bound than lower bound in the PPO ratio so low-probability tokens can be promoted faster, which helps preserve exploration. → section
Clopper–Pearson interval	An exact, conservative interval for a proportion built from the binomial distribution. Useful for very small n. → section
Cluster bootstrap	Resampling whole clusters (sessions, users) instead of rows, so the interval respects dependence within clusters. → section
Cochran's Q	Tests whether three or more models differ on the same binary tasks; the paired extension of McNemar. Says whether they differ, not which. → section
Cohen's kappa	Agreement between two raters corrected for the agreement expected by chance: (p_o – p_e) / (1 – p_e). → section
Cold-start SFT	A small supervised fine-tuning stage on curated long chain-of-thought examples before reasoning RL, used in DeepSeek-R1 to improve readability and stability. → section
Compute-matched comparison	Comparing two methods at the same training FLOPs or tokens, so a gain reflects the method rather than extra compute. → section

Compute-optimal	The split of a fixed training compute budget between model size and data that minimises loss. → section
Confidence interval	A range produced by a procedure that captures the truth a stated share of the time. → section
Confusion matrix	The 2×2 table of true positives, false positives, false negatives, and true negatives at a given threshold. Every classification metric is a ratio of its cells. → section
Constitutional AI	Training with a written set of principles: the model critiques and revises its own outputs, and AI-generated preference labels replace human harmlessness labels (Bai et al., 2022). → section
Contamination	Benchmark items, or close paraphrases of them, appearing in a model's training data, which inflates its score on that benchmark. → section
Continuous batching	A serving technique that adds new requests to a running batch as others finish, keeping accelerators busy. → section
Cosine schedule	A learning-rate schedule that decays from the peak to a minimum following half a cosine wave after warmup. → section
Coverage	How often an interval method actually contains the true value. A '95%' method can cover far less in practice. → section
Coverage–risk curve	Error rate on answered items as a function of the share of items answered (coverage). → section
Critical batch size	The batch size beyond which larger batches give little reduction in the number of training steps. Estimated by the gradient noise scale. → section
Cross-entropy loss	The average negative log probability a model assigns to the token that actually came next. The pretraining and SFT objective. → section
Dangerous-capability evaluation	Testing whether a model could meaningfully assist with severe harms or act autonomously in risky ways, used to trigger safeguards under frontier safety policies. → section

DAPO	An RL recipe (Yu et al., 2025) adding clip-higher, dynamic sampling of informative groups, token-level loss, and overlong reward shaping to GRPO-style training. → section
Data mixture	The proportions of each data source (web, code, math, books, languages, synthetic) in a training run. Tuned with small proxy models. → section
Decode	Generating output tokens one at a time; usually memory-bandwidth-bound. → section
Decontamination	Removing training documents that overlap with evaluation items, usually by long n-gram matching, before training. → section
Deliberative alignment	Training a model to recall and reason over a written safety specification before answering (Guan et al., 2024). → section
Design effect	How much clustering inflates variance: deff = 1 + (m̃ − 1)·ρ. The naive standard error is too small by a factor of about sqrt(deff). → section
Direction accuracy	The share of past releases where offline and online deltas agreed in sign. The number that matters for ship decisions. → section
Discordant pair	A task where one model passed and the other failed. The only tasks that carry information about which model is better. → section
DoRA	Weight-Decomposed Low-Rank Adaptation (Liu et al., 2024). Splits each weight into magnitude and direction and applies LoRA to the direction. → section
DPO	Direct Preference Optimization (Rafailov et al. 2023): optimizes the KL-regularised RLHF objective with a classification loss on preference pairs, without a reward model or RL loop. → section
Dr. GRPO	A GRPO variant (Liu et al., 2025) that drops per-response length normalisation and standard-deviation scaling to remove a length bias and a difficulty bias. → section
Effective sample size	The number of independent observations your data is worth after clustering: N / deff. → section

Effect size	How big the difference is, as opposed to how confident you are that it exists. → section
Emergent ability	A capability that appears to arise suddenly with scale. Often an artifact of all-or-nothing metrics; smooth metrics usually show gradual improvement. → section
Entropy collapse	The policy's output distribution becoming nearly deterministic during RL, so sampled responses stop differing and the learning signal disappears. → section
Eta squared	Effect size for ANOVA: the share of total variance explained by group membership. Report it beside F. → section
Eval harness	The software that sends tasks to a model, collects and scores outputs, and records results. The measuring instrument for every eval number. → section
Expected calibration error (ECE)	Weighted average gap between confidence and accuracy across confidence bins. Depends on the number of bins. → section
Expected cost	(c_FP·FP + c_FN·FN) / n: the average cost per decision once the two errors are priced. → section
F1 score	Harmonic mean of precision and recall, 2PR / (P + R). High only when both are high; ignores true negatives. → section
False discovery rate	The expected share of declared discoveries that are false. Controlled by Benjamini–Hochberg. → section
False negative (FN)	A positive case the model missed. → section
False positive (FP)	A negative case the model wrongly flagged: a false alarm. → section
False positive rate (FPR)	FP / (FP + TN): the share of negatives wrongly flagged. The x-axis of an ROC curve. → section
Family-wise error rate	The probability of at least one false positive across a set of tests. Controlled by Bonferroni. → section
F-beta score	Weighted harmonic mean of precision and recall; β > 1 weights recall more, β < 1 weights precision more. → section

Flaky task	A task whose score varies across runs at fixed settings. Quarantined and reported separately rather than averaged into the headline. → section
Forward KL	KL(teacher || student). Mode covering: penalises the student for missing any place the teacher puts mass. → section
FP8 training	Training with 8-bit floating-point matrix multiplies for higher throughput and lower memory, with careful scaling to avoid overflow. Used at scale in DeepSeek-V3. → section
Friedman test	A rank-based test for three or more models scored on the same tasks with continuous or ordinal scores. → section
FSDP	Fully Sharded Data Parallel, PyTorch's implementation of ZeRO-style sharding of model state across GPUs. → section
Full jitter	A retry backoff drawn uniformly between zero and an exponentially growing cap, so retries from a failure burst do not arrive in lockstep. → section
GAE	Generalized advantage estimation: an exponentially weighted sum of one-step temporal-difference errors, with λ trading bias for variance. → section
Geometric mean	The exponential of the average log. On lognormal data it equals the median. → section
Goodhart's law	When a measure becomes a target, it stops being a good measure; optimisation pressure finds ways to raise the metric without the underlying quality. → section
GPTQ	A post-training weight quantization method that uses approximate second-order information to minimise layer output error (Frantar et al., 2023). → section
Gradient clipping	Rescaling the gradient when its norm exceeds a threshold (commonly 1.0) to prevent unstable updates. → section
Gradient descent	Updating parameters by stepping against the gradient of the loss. With mini-batches it is stochastic gradient descent (SGD). → section

Greedy decoding	Always choosing the single most probable next token; deterministic. → section
Group advantage	In GRPO, a response's reward minus its group's mean reward, usually divided by the group's standard deviation. Zero for every response when all rewards in the group are equal. → section
Grouped-query attention (GQA)	Attention in which several query heads share each key/value head, shrinking the KV cache. → section
GRPO	Group Relative Policy Optimization (Shao et al., 2024): PPO-style RL that samples a group of responses per prompt and uses the group's mean and standard deviation of reward as the baseline, removing the critic network. → section
Heteroscedasticity	Spread that changes with the value. Common in latency: slow things vary more. → section
Human ceiling	The agreement two humans reach on the same labels. No judge or classifier can meaningfully exceed it against human labels. → section
ICC	Intraclass correlation: the share of variance that lives between clusters rather than within them. → section
Implicit reward	In DPO, $\beta \log \pi_\theta(y \mid x) / \pi_{\text{ref}}(y \mid x)$, the reward the policy implicitly defines. → section
InstructGPT	OpenAI's 2022 SFT, reward model, PPO recipe (Ouyang et al. 2022), the template for most later RLHF pipelines. → section
Inter-annotator agreement	How often independent labelers make the same choice on the same item. The practical ceiling on reward model accuracy. → section
Intraclass correlation (ICC)	The share of variance that lies between clusters (tasks, sessions, users) rather than within them. Drives the design effect. → section
Intrinsic dimension	The number of free directions needed to reach most of full fine-tuning performance. Low for pretrained language models, which is why low-rank updates work. → section
IPO	Identity Preference Optimization (Azar et al. 2023): a squared loss toward a finite target margin, avoiding DPO's overfitting to deterministic preferences. → section

IQR	Interquartile range: the middle 50% of the data, p75 minus p25. Outliers cannot move it. → section
Jackknife	Recomputing a statistic with each observation left out in turn; used to estimate bias, variance, and the BCa acceleration. → section
Jailbreak	An input crafted to get a model to do something its safety training is meant to prevent. → section
k3 KL estimator	The per-sample estimator r − log r − 1, with r = π_ref/π_θ, used in GRPO. It is unbiased for KL(π_θ‖π_ref) and never negative. → section
Kendall's tau	Rank correlation computed from concordant and discordant pairs, $\tau = (C - D)/(C + D)$. More stable than Spearman with few releases. → section
Kill criteria	A result, stated in advance, that would end a research project. → section
KL penalty	A term $\beta\,\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}})$ that keeps the trained policy close to the reference; β sets its strength. → section
Knowledge distillation	Training a student model to match a teacher's output distribution, not just the correct label (Hinton et al., 2015). → section
KTO	Kahneman–Tversky Optimization (Ethayarajh et al. 2024): preference optimization from unpaired desirable and undesirable examples. → section
Kurtosis	How heavy the tails are. High kurtosis means extremes are more common than a bell predicts. → section
KV cache	Stored attention keys and values for every previous token, reused at each decoding step; its size grows with layers, KV heads, context length, and concurrent sequences. → section
Learning rate	The step size of each parameter update. The most important hyperparameter in training. → section
Length bias	The tendency of raters and reward models to prefer longer responses regardless of quality. → section
Likelihood displacement	A DPO failure where both chosen and rejected log-likelihoods fall during training and probability moves to responses neither label

	mentioned. → section
LLM as judge	Using a language model to grade or compare model outputs. A classifier that must itself be validated against humans. → section
Logits	The raw, unnormalised scores a model outputs for each vocabulary item before softmax turns them into probabilities. → section
Lognormal	The distribution of a quantity whose log is normal. Arises from multiplicative effects; the shape of latency and token counts. → section
LoRA	Low-Rank Adaptation (Hu et al., 2021). Learns a low-rank update BA added to a frozen weight, scaled by alpha/r. B starts at zero so training begins at the base model. → section
Loss masking	Setting the label of non-target tokens (system, user, tool output) to an ignore value such as -100 so they don't contribute to the loss. → section
Loss spike	A sudden jump in training loss during a large run, usually from optimisation instability. Often handled by rolling back and skipping batches. → section
Macro average	Unweighted mean of a metric across classes, so every class counts equally. → section
MAD	Median absolute deviation: the median distance from the median. Multiply by 1.4826 to compare with the sd. → section
maj@k (self-consistency)	Sample k responses and return the most common final answer (Wang et al., 2022). → section
Mann–Whitney U	A rank-based comparison of two groups that does not assume interval data. Its common-language effect is P(random x > random y). → section
Matthews correlation coefficient (MCC)	Correlation between predictions and labels using all four confusion-matrix cells, from –1 to +1. A strong single-number summary under imbalance. → section
McNemar test	A test on discordant pairs: under no difference, each is a fair coin flip. Use the exact binomial version on small eval sets. → section
Mean	The arithmetic average: add everything, divide by n. Right for totals, misleading on skewed data. → section

Median	The middle value when sorted. The robust centre for skewed data like latency. → section
Memoryless	How long you have already waited tells you nothing about the remaining wait. True of the exponential and geometric. → section
MFU	Model FLOPs utilisation: useful training FLOPs per second divided by the hardware's peak. Large runs typically reach 35 to 50 percent. → section
Micro average	A metric computed from counts pooled across classes. In single-label multiclass, micro-F1 equals accuracy. → section
Mid-training	A stage between pretraining and fine-tuning that continues the same loss on curated data, often while the learning rate decays: high-quality text, long documents for context extension, or boosted code and math. → section
MinHash deduplication	A near-duplicate detection method that estimates Jaccard similarity between documents' n-gram sets using the minimum of random hash functions. → section
Minimum detectable effect	The smallest true difference an eval design can detect at a chosen power, usually 80%. Report it next to any null result. → section
Min-K% Prob	A membership-inference signal that averages the lowest-probability tokens of a text; seen text tends to score higher (Shi et al., 2023). → section
Min-p sampling	Keeping only tokens whose probability is at least a fraction p_min of the top token's probability, so the cutoff adapts to model confidence. → section
Mixture	Two or more populations sharing an axis, often visible as two humps. Split them before summarising. → section
Mixture of experts (MoE)	An architecture where each token is routed to a few of many expert sub-networks, so total parameters can be large while per-token compute stays small. → section
Mode	The most common value. Useful for categories and for spotting two humps. → section

Model collapse	Progressive loss of distribution tails and quality when models are trained recursively on their own outputs (Shumailov et al., 2024). Mitigated by keeping real data in the mix. → section
Model soup	A model made by averaging the weights of several fine-tunes of the same base (Wortsman et al., 2022). → section
Momentum	Accumulating a running average of past gradients so consistent directions speed up and oscillations cancel. → section
Multi-LoRA serving	Serving many adapters on one shared base model and batching their requests together, as in S-LoRA and Punica. → section
Multiple comparisons	Running many tests at once inflates the number of false positives; with 40 benchmarks at α = 0.05, about 2 will look significant by luck. → section
Negative binomial	A count distribution with variance above the mean, arising when the event rate varies across units. Fits tool calls per session. → section
NF4	4-bit NormalFloat: a 4-bit data type whose levels sit at quantiles of a normal distribution, suited to normally distributed weights. → section
Normal approximation (Wald interval)	The symmetric interval p ± z·sqrt(p(1−p)/n). Under-covers badly near 0 or 1 and can exceed 100%. → section
Offline to online gap	The difference between how a model change moves an offline eval score and how it moves the production metric it is meant to predict. → section
Off-policy data	Training data generated by a different model than the one being trained. → section
Online / iterative DPO	Repeated rounds of sampling from the current policy, labeling pairs with a reward model or judge, and training with the DPO loss. → section
On-policy distillation	The student generates text and the teacher scores each token, so the student learns to correct its own mistakes (Agarwal et al., 2024). → section
Operating point	The specific threshold, and resulting precision and recall, at which a classifier is actually deployed. → section

ORPO	Odds Ratio Preference Optimization (Hong et al. 2024): single-stage SFT plus an odds-ratio preference penalty, with no reference model. → section
Outcome reward model	A reward model that scores only the final response. → section
Outcome reward model (ORM)	A reward model, or verifier, that scores only the final result of a full response. → section
Overdispersion	Count data whose variance exceeds its mean, so Poisson intervals are too narrow. Typical of tool calls and token counts. → section
Over-refusal	Refusing a benign request, often because it superficially resembles a harmful one. The false positive of safety behaviour. → section
Over-training	Training a smaller model on far more tokens than compute-optimal, accepting slightly higher loss for much cheaper inference. → section
Packing	Concatenating several short training examples into one fixed-length sequence to avoid wasted padding. Needs per-document attention masks so examples can't see each other. → section
Paired	The same tasks were given to both models, so comparisons can use within-task differences. → section
Paired bootstrap	Resampling task indices and reading both models at each sampled task, to get an interval on the difference between them. → section
Paired comparison	Comparing two models task by task on the same tasks, which removes task difficulty from the noise and gives far more power than comparing two averages. → section
Parameter-efficient fine-tuning (PEFT)	Methods that freeze the pretrained weights and train a small number of added or selected parameters. → section
pass@k	The probability that at least one of k sampled responses is correct, estimated without bias from n ≥ k samples as 1 − C(n−c, k)/C(n, k). → section
pass^k	The probability that all k attempts pass. A reliability measure used for agents, popularised by τ-bench. → section

Pass-rate gap	The difference between a judge's pass rate and humans' on the same items. A generous judge inflates every absolute number while still ranking models correctly. → section
PDF	Probability density function: how common values are around each point. The familiar humped curve. → section
Percentile bootstrap	An interval taken from the quantiles of statistics recomputed on resamples drawn with replacement. → section
Percentile / quantile	The value below which a given share of the data falls. p95 is the 0.95 quantile. → section
Perplexity	The exponential of cross-entropy loss in nats. Roughly the number of equally likely choices the model feels it faces at each token. → section
Perturbation test	Re-running a benchmark with names, numbers, or option order changed. A large drop relative to a clean control suggests memorisation. → section
Pipeline parallelism	Placing different layers on different GPUs and streaming micro-batches through them. → section
Policy	In RL for language models, the model's next-token distribution $\pi_\theta(a_t \mid s_t)$. → section
Policy gradient	An estimator of the gradient of expected reward using samples from the policy, weighting each action's log-probability gradient by its return or advantage. → section
Position bias	A pairwise judge's tendency to prefer the response shown in a particular position. Controlled by judging both orderings. → section
Power	The chance of detecting a real effect of a given size, if it exists. → section
Power law	A distribution with no typical size, straight on log-log axes. Its mean may not exist; report percentiles only. → section
PPO	Proximal Policy Optimization (Schulman et al. 2017): a policy gradient method that clips the probability ratio so each batch can only move the policy a limited amount. → section

PR-AUC (average precision)	Area under the precision–recall curve, usually computed as average precision. Its chance baseline equals the prevalence. → section
PR curve	Precision plotted against recall as the threshold sweeps. Sensitive to prevalence. → section
Precision	TP / (TP + FP): of the items flagged, the share that were truly positive. → section
Preference pair	A prompt with a preferred (chosen) and dispreferred (rejected) response, written (x, y_w, y_l). → section
Prefill	Processing the input prompt in parallel before generation; usually compute-bound. → section
Pre-registration	Writing down the hypothesis, the deciding metrics, and the success threshold before running the experiment. → section
Pretraining	The first and most expensive training stage: next-token prediction over trillions of tokens of text and code. Most knowledge and raw capability comes from here. → section
Prevalence	The share of items that are truly positive. Precision and PR-AUC depend on it; recall, FPR, and ROC-AUC do not. → section
Prior	What you believed before seeing this data. With small samples it does real work in a Bayesian estimate. → section
Probability ratio	$\pi_\theta(a \mid s)/\pi_{\theta_{\text{old}}}(a \mid s)$, how much an update changed an action's probability. PPO clips it to $[1-\epsilon, 1+\epsilon]$. → section
Process reward model	A reward model that scores each intermediate step of a reasoning chain (Lightman et al. 2023). → section
Process reward model (PRM)	A reward model that scores each intermediate reasoning step rather than only the final answer. → section
Prompt injection	Instructions hidden in content a model reads, such as a web page, document, or tool output, that try to override the user's or developer's instructions. → section
Proper scoring rule	A score minimised by reporting your true belief. Brier score is one; ECE is not. → section

p value	How often you would see a result this extreme if there were genuinely no effect. Not the probability the null is true. → section
QLoRA	LoRA trained on top of a base model stored in 4-bit NF4, with double quantisation and paged optimisers (Dettmers et al., 2023). Cuts memory enough to fine-tune large models on one GPU. → section
QQ plot	A plot of your data's quantiles against a reference distribution's. A straight line means a good match. → section
Quality classifier	A small model that scores documents for usefulness (for example educational value) so only the best slice of a web crawl is kept. → section
Quantization	Storing weights, and sometimes activations or the KV cache, in fewer bits (for example 8 or 4) to cut memory and raise speed, with some accuracy risk. → section
Rank (LoRA)	The inner dimension r of the LoRA matrices. The update can express at most r independent directions; trainable parameters per matrix are r times (rows + columns). → section
Recall	TP / (TP + FN): of the truly positive items, the share that were flagged. Also called sensitivity or true positive rate. → section
Red teaming	Deliberately searching for inputs that make a model fail or behave harmfully, by humans or automated attacker models. → section
Reference model	A frozen copy of the starting model used to measure and penalize how far the trained policy drifts. → section
Regression triage	A fixed order of checks (is it real, where is it, what else changed, errors, replay, read outputs) for deciding whether an eval drop is noise, measurement, infrastructure, or model. → section
REINFORCE	The basic Monte Carlo policy gradient algorithm (Williams 1992). → section
Rejection sampling fine-tuning	Sample many responses per prompt, keep those that pass a verifier or score highest with a reward model, and fine-tune on them. → section

Replay	Mixing general or earlier training data into a narrow fine-tuning run to reduce forgetting. → section
Responsible Scaling Policy	Anthropic's framework tying capability thresholds (AI Safety Levels) to required security and deployment safeguards; similar frameworks include OpenAI's Preparedness Framework and Google DeepMind's Frontier Safety Framework. → section
Retry inflation	Upward bias in a pass rate when retried tasks effectively get more than one draw at the answer, making them scored like pass@k rather than pass@1. → section
Reverse KL	KL(student || teacher). Mode seeking: penalises the student for putting mass where the teacher doesn't. Used in on-policy distillation. → section
RewardBench	A public benchmark (Lambert et al. 2024; RewardBench 2 in 2025) of preference pairs across chat, safety, and reasoning for comparing reward models. → section
Reward hacking	A policy earning reward through behavior the designer did not intend, such as padding answers for a length-biased reward model. → section
Reward model	A network trained on human or AI comparisons to output a scalar score predicting which response is preferred. The target for RLHF and the selector for best-of-N. → section
Reward overoptimization	Optimizing a proxy reward past the point where true quality peaks, so the proxy keeps rising while real quality falls. Quantified by Gao et al. (2022). → section
RLAIF	Reinforcement learning from AI feedback: preference labels produced by a model guided by written principles instead of by people. → section
RLHF	Reinforcement learning from human feedback: training a policy to maximize a learned reward model's score with a KL penalty to a reference model. → section
RLOO	REINFORCE leave-one-out: each response's baseline is the mean reward of the other responses for the same prompt, giving an unbiased advantage estimate. → section

RLVR	Reinforcement learning with verifiable rewards: RL where a program (answer checker, unit tests, format check) computes the reward instead of a learned reward model. → section
Robust	Resistant to outliers. The median is robust, the mean is not. → section
ROC-AUC	Area under the ROC curve; the probability a random positive scores higher than a random negative. 0.5 is chance, 1.0 is perfect. → section
ROC curve	Recall plotted against false positive rate as the threshold sweeps from high to low. Each threshold is one point. → section
Rollout	A response sampled from the current policy during RL training, then scored. → section
Rule of three	If zero events occur in n independent trials, an approximate 95% upper bound on the event rate is 3/n. → section
Sampling distribution	The spread of results you would get from repeating the whole experiment on fresh samples. → section
Scaling law	An empirical power-law relationship between loss and model size, data, or compute, used to predict large runs from small ones. → section
Sealed holdout	An evaluation set owned and run by someone independent of the training team, used only for final decisions so nobody can optimise toward it. → section
Seed variance	How much a final score changes when the same training recipe is rerun with a different random seed. → section
Selective prediction	Letting a model abstain on low-confidence cases and measuring error only on the cases it answers. → section
Self-Instruct	Generating new instruction data by prompting a model with seed examples, then filtering (Wang et al., 2023). → section
Self-preference bias	A judge scoring outputs from its own model family more favourably. → section
Sequence-level distillation	Training the student on complete outputs generated by the teacher (Kim and Rush, 2016). Works across tokenizers and through APIs. → section

Shrinkage (offline to online)	One minus the fitted slope of online delta on offline delta. A shrinkage of 0.4 means online moves are about 60% of offline moves. → section
SimPO	Simple Preference Optimization (Meng et al. 2024): reference-free, length-normalized implicit reward with a target margin. → section
Skew	Asymmetry, named for the side the tail is on. Mean divided by median above 1.2 signals right skew. → section
Skew ratio	Mean divided by median. Above about 1.2, report quantiles instead of the mean. → section
Slice contribution	A slice's share of the total score change, $n_s \bar{d}_s / N$. Rank slices by this rather than by percentage change. → section
Softmax	The function that turns logits into probabilities: exponentiate each score and divide by the sum. Temperature divides the logits first. → section
Soft targets	The teacher's full probability distribution over outputs, often softened with a temperature above 1 to expose how it ranks wrong answers. → section
Specificity	TN / (TN + FP): of the truly negative items, the share left alone. Recall for the negative class; one minus the false positive rate. → section
Speculative decoding	A small draft model proposes several tokens that the large model verifies in one pass, producing the same output distribution faster (Leviathan et al., 2023). → section
Standard deviation	Typical distance of a single measurement from the mean. Does not shrink with more data. → section
Standard error	Typical distance of your estimate from the truth. Shrinks as one over the square root of n. → section
STaR	Self-Taught Reasoner (Zelikman et al., 2022). Iteratively fine-tunes on the model's own reasoning traces that reach correct answers. → section
Statistical power	The probability that a test detects a real effect of a given size. Small eval sets have low power for small improvements. → section

Superficial alignment hypothesis	The claim from LIMA (Zhou et al., 2023) that a model's knowledge comes almost entirely from pretraining and alignment mostly teaches which format and style to use. Holds for style, less so for hard skills. → section
Supervised fine-tuning (SFT)	Training a pretrained model on example prompt and response pairs with the next-token loss, counting only the response tokens. Teaches format and behaviour more than knowledge. → section
Survival analysis	Methods such as Kaplan-Meier that handle censored data correctly, for example requests cut off by a timeout. → section
Sycophancy	A model telling users what they want to hear, such as agreeing with incorrect claims or abandoning a correct answer under pushback. → section
Task vector	The difference between fine-tuned and base weights. Task vectors can be added or subtracted to combine or remove skills (Ilharco et al., 2023). → section
Teacher forcing	Training each step on the reference's previous tokens rather than the model's own. Causes exposure bias, because at inference the model conditions on its own outputs. → section
Temperature	A divisor applied to logits before softmax. Below 1 sharpens the distribution toward the top token; above 1 flattens it. → section
Temperature scaling	Dividing logits by a single learned constant on held-out data to fix calibration without changing rankings. → section
Tensor parallelism	Splitting individual weight matrices across GPUs so each computes part of every layer. Needs fast interconnect. → section
Test-time compute	Extra computation spent when answering, such as longer reasoning, more samples, or search, to improve accuracy without retraining. → section
Threshold	The score cutoff above which a classifier predicts positive. Moving it trades false positives for false negatives. → section

TIES-Merging	A merging method that trims small weight changes, resolves sign conflicts, and averages only agreeing entries (Yadav et al., 2023). → section
Time-split benchmark	A benchmark that only counts items created after a model's training cutoff, so they cannot have been memorised. → section
Token	The unit a language model reads and writes: a chunk of text produced by the tokenizer, often a word piece. English averages roughly four characters per token for common tokenizers. → section
Top-k sampling	Sampling only from the k most probable next tokens. → section
Top-p (nucleus) sampling	Sampling from the smallest set of most probable tokens whose cumulative probability reaches p (Holtzman et al., 2020). → section
Top-two-box	The share of ratings in the top two points of a scale, such as 4 or 5 out of 5. A more honest summary of ordinal ratings than the mean. → section
Trimmed mean	The mean after dropping a fixed share from each end. Rarely right for a one-sided tail. → section
True negative (TN)	A negative case the model correctly left alone. → section
True positive (TP)	A positive case the model correctly flagged. → section
Unsafe compliance	Answering a genuinely harmful request. The false negative of safety behaviour. → section
Value model (critic)	A network that predicts expected future reward from each state, used to compute advantages in PPO. → section
Variance	Standard deviation squared. Adds cleanly across independent sources. → section
Variance components	Splitting score variance into task-to-task and run-to-run parts, which tells you whether to add tasks or add samples per task. → section
Verifier	A program that checks whether a model's output is correct, used as the reward in RLVR or to select among samples at inference time. → section

Warmup	Ramping the learning rate up from near zero over the first steps of training, because early gradients are large and erratic. → section
Weighted average	Mean of a per-class metric weighted by class support. Can hide poor performance on rare classes. → section
Wilson interval	A confidence interval for a proportion that stays inside 0 to 1 and keeps close to its stated coverage near the boundaries. The default for pass rates. → section
Winner's curse	The upward bias in the observed score of whichever variant you selected as best; with many variants and noisy scores, the winner looks better than it is. → section
Winsorize	Cap extreme values at a percentile rather than removing them. Justified only for confirmed measurement errors. → section
WSD schedule	Warmup-stable-decay: hold the learning rate constant for most of training and decay only at the end, so decayed checkpoints can be branched off at any point. → section
ZeRO	Zero Redundancy Optimizer: sharding optimiser states (stage 1), gradients (stage 2), and weights (stage 3) across data-parallel GPUs instead of replicating them. → section
Zero-variance group	A prompt where every sampled response gets the same reward; it contributes no policy-gradient signal in group-baseline methods. → section

---

Built from three earlier study guides plus new training chapters. Numbers in worked examples are illustrative unless a source is named. Source and build script live in the accompanying repository.